Choupo

The Open Source Chemical Process Simulator

Tutorials Guide

Choupo-dev

Authors: Vítor Geraldes.

Typeset in LaTeX.

Acknowledgment. Choupo was conceived and architected by Vítor Geraldes. Substantial parts of the initial implementation, documentation, and tutorial corpus were produced with assistance from Anthropic Claude Code using Claude Opus/Fable models. This guide is published under human curation, review, and editorial responsibility by Vítor Geraldes.

License

```
Attribution-ShareAlike 4.0 International

=======================================================================

Creative Commons Corporation ("Creative Commons") is not a law firm and
does not provide legal services or legal advice. Distribution of
Creative Commons public licenses does not create a lawyer-client or
other relationship. Creative Commons makes its licenses and related
information available on an "as-is" basis. Creative Commons gives no
warranties regarding its licenses, any material licensed under their
terms and conditions, or any related information. Creative Commons
disclaims all liability for damages resulting from their use to the
fullest extent possible.

Using Creative Commons Public Licenses

Creative Commons public licenses provide a standard set of terms and
conditions that creators and other rights holders may use to share
original works of authorship and other material subject to copyright
and certain other rights specified in the public license below. The
following considerations are for informational purposes only, are not
exhaustive, and do not form part of our licenses.

     Considerations for licensors: Our public licenses are
     intended for use by those authorized to give the public
     permission to use material in ways otherwise restricted by
     copyright and certain other rights. Our licenses are
     irrevocable. Licensors should read and understand the terms
     and conditions of the license they choose before applying it.
     Licensors should also secure all rights necessary before
     applying our licenses so that the public can reuse the
     material as expected. Licensors should clearly mark any
     material not subject to the license. This includes other CC-
     licensed material, or material used under an exception or
     limitation to copyright. More considerations for licensors:
    wiki.creativecommons.org/Considerations_for_licensors

     Considerations for the public: By using one of our public
     licenses, a licensor grants the public permission to use the
     licensed material under specified terms and conditions. If
     the licensor's permission is not necessary for any reason--for
     example, because of any applicable exception or limitation to
     copyright--then that use is not regulated by the license. Our
     licenses grant only permissions under copyright and certain
     other rights that a licensor has authority to grant. Use of
     the licensed material may still be restricted for other
     reasons, including because others have copyright or other
     rights in the material. A licensor may make special requests,
     such as asking that all changes be marked or described.
     Although not required by our licenses, you are encouraged to
     respect those requests where reasonable. More considerations
     for the public:
    wiki.creativecommons.org/Considerations_for_licensees

=======================================================================

Creative Commons Attribution-ShareAlike 4.0 International Public
License

By exercising the Licensed Rights (defined below), You accept and agree
to be bound by the terms and conditions of this Creative Commons
Attribution-ShareAlike 4.0 International Public License ("Public
License"). To the extent this Public License may be interpreted as a
contract, You are granted the Licensed Rights in consideration of Your
acceptance of these terms and conditions, and the Licensor grants You
such rights in consideration of benefits the Licensor receives from
making the Licensed Material available under these terms and
conditions.


Section 1 -- Definitions.

  a. Adapted Material means material subject to Copyright and Similar
     Rights that is derived from or based upon the Licensed Material
     and in which the Licensed Material is translated, altered,
     arranged, transformed, or otherwise modified in a manner requiring
     permission under the Copyright and Similar Rights held by the
     Licensor. For purposes of this Public License, where the Licensed
     Material is a musical work, performance, or sound recording,
     Adapted Material is always produced where the Licensed Material is
     synched in timed relation with a moving image.

  b. Adapter's License means the license You apply to Your Copyright
     and Similar Rights in Your contributions to Adapted Material in
     accordance with the terms and conditions of this Public License.

  c. BY-SA Compatible License means a license listed at
     creativecommons.org/compatiblelicenses, approved by Creative
     Commons as essentially the equivalent of this Public License.

  d. Copyright and Similar Rights means copyright and/or similar rights
     closely related to copyright including, without limitation,
     performance, broadcast, sound recording, and Sui Generis Database
     Rights, without regard to how the rights are labeled or
     categorized. For purposes of this Public License, the rights
     specified in Section 2(b)(1)-(2) are not Copyright and Similar
```

Rights.

e. Effective Technological Measures means those measures that, in the absence of proper authority, may not be circumvented under laws fulfilling obligations under Article 11 of the WIPO Copyright Treaty adopted on December 20, 1996, and/or similar international agreements.

f. Exceptions and Limitations means fair use, fair dealing, and/or any other exception or limitation to Copyright and Similar Rights that applies to Your use of the Licensed Material.

g. License Elements means the license attributes listed in the name of a Creative Commons Public License. The License Elements of this Public License are Attribution and ShareAlike.

h. Licensed Material means the artistic or literary work, database, or other material to which the Licensor applied this Public License.

i. Licensed Rights means the rights granted to You subject to the terms and conditions of this Public License, which are limited to all Copyright and Similar Rights that apply to Your use of the Licensed Material and that the Licensor has authority to license.

j. Licensor means the individual(s) or entity(ies) granting rights under this Public License.

k. Share means to provide material to the public by any means or process that requires permission under the Licensed Rights, such as reproduction, public display, public performance, distribution, dissemination, communication, or importation, and to make material available to the public including in ways that members of the public may access the material from a place and at a time individually chosen by them.

l. Sui Generis Database Rights means rights other than copyright resulting from Directive 96/9/EC of the European Parliament and of the Council of 11 March 1996 on the legal protection of databases, as amended and/or succeeded, as well as other essentially equivalent rights anywhere in the world.

m. You means the individual or entity exercising the Licensed Rights under this Public License. Your has a corresponding meaning.

Section 2 -- Scope.

a. License grant.

   1. Subject to the terms and conditions of this Public License, the Licensor hereby grants You a worldwide, royalty-free, non-sublicensable, non-exclusive, irrevocable license to exercise the Licensed Rights in the Licensed Material to:

      a. reproduce and Share the Licensed Material, in whole or in part; and

      b. produce, reproduce, and Share Adapted Material.

   2. Exceptions and Limitations. For the avoidance of doubt, where Exceptions and Limitations apply to Your use, this Public License does not apply, and You do not need to comply with its terms and conditions.

   3. Term. The term of this Public License is specified in Section 6(a).

   4. Media and formats; technical modifications allowed. The Licensor authorizes You to exercise the Licensed Rights in all media and formats whether now known or hereafter created, and to make technical modifications necessary to do so. The Licensor waives and/or agrees not to assert any right or authority to forbid You from making technical modifications necessary to exercise the Licensed Rights, including technical modifications necessary to circumvent Effective Technological Measures. For purposes of this Public License, simply making modifications authorized by this Section 2(a)(4) never produces Adapted Material.

   5. Downstream recipients.

      a. Offer from the Licensor -- Licensed Material. Every recipient of the Licensed Material automatically receives an offer from the Licensor to exercise the Licensed Rights under the terms and conditions of this Public License.

      b. Additional offer from the Licensor -- Adapted Material. Every recipient of Adapted Material from You automatically receives an offer from the Licensor to exercise the Licensed Rights in the Adapted Material under the conditions of the Adapter's License You apply.

      c. No downstream restrictions. You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, the Licensed Material if doing so restricts exercise of the Licensed Rights by any recipient of the Licensed Material.

```
      6. No endorsement. Nothing in this Public License constitutes or
         may be construed as permission to assert or imply that You
         are, or that Your use of the Licensed Material is, connected
         with, or sponsored, endorsed, or granted official status by,
         the Licensor or others designated to receive attribution as
         provided in Section 3(a)(1)(A)(i).

  b. Other rights.

      1. Moral rights, such as the right of integrity, are not
         licensed under this Public License, nor are publicity,
         privacy, and/or other similar personality rights; however, to
         the extent possible, the Licensor waives and/or agrees not to
         assert any such rights held by the Licensor to the limited
         extent necessary to allow You to exercise the Licensed
         Rights, but not otherwise.

      2. Patent and trademark rights are not licensed under this
         Public License.

      3. To the extent possible, the Licensor waives any right to
         collect royalties from You for the exercise of the Licensed
         Rights, whether directly or through a collecting society
         under any voluntary or waivable statutory or compulsory
         licensing scheme. In all other cases the Licensor expressly
         reserves any right to collect such royalties.


Section 3 -- License Conditions.

Your exercise of the Licensed Rights is expressly made subject to the
following conditions.

  a. Attribution.

      1. If You Share the Licensed Material (including in modified
         form), You must:

           a. retain the following if it is supplied by the Licensor
              with the Licensed Material:

                i. identification of the creator(s) of the Licensed
                   Material and any others designated to receive
                   attribution, in any reasonable manner requested by
                   the Licensor (including by pseudonym if
                   designated);

               ii. a copyright notice;

              iii. a notice that refers to this Public License;

               iv. a notice that refers to the disclaimer of
                   warranties;

                v. a URI or hyperlink to the Licensed Material to the
                   extent reasonably practicable;

           b. indicate if You modified the Licensed Material and
              retain an indication of any previous modifications; and

           c. indicate the Licensed Material is licensed under this
              Public License, and include the text of, or the URI or
              hyperlink to, this Public License.

      2. You may satisfy the conditions in Section 3(a)(1) in any
         reasonable manner based on the medium, means, and context in
         which You Share the Licensed Material. For example, it may be
         reasonable to satisfy the conditions by providing a URI or
         hyperlink to a resource that includes the required
         information.

      3. If requested by the Licensor, You must remove any of the
         information required by Section 3(a)(1)(A) to the extent
         reasonably practicable.

  b. ShareAlike.

     In addition to the conditions in Section 3(a), if You Share
     Adapted Material You produce, the following conditions also apply.

      1. The Adapter's License You apply must be a Creative Commons
         license with the same License Elements, this version or
         later, or a BY-SA Compatible License.

      2. You must include the text of, or the URI or hyperlink to, the
         Adapter's License You apply. You may satisfy this condition
         in any reasonable manner based on the medium, means, and
         context in which You Share Adapted Material.

      3. You may not offer or impose any additional or different terms
         or conditions on, or apply any Effective Technological
         Measures to, Adapted Material that restrict exercise of the
         rights granted under the Adapter's License You apply.


Section 4 -- Sui Generis Database Rights.

Where the Licensed Rights include Sui Generis Database Rights that
apply to Your use of the Licensed Material:
```

a. for the avoidance of doubt, Section 2(a)(1) grants You the right to extract, reuse, reproduce, and Share all or a substantial portion of the contents of the database;

b. if You include all or a substantial portion of the database contents in a database in which You have Sui Generis Database Rights, then the database in which You have Sui Generis Database Rights (but not its individual contents) is Adapted Material, including for purposes of Section 3(b); and

c. You must comply with the conditions in Section 3(a) if You Share all or a substantial portion of the contents of the database.

For the avoidance of doubt, this Section 4 supplements and does not replace Your obligations under this Public License where the Licensed Rights include other Copyright and Similar Rights.

Section 5 -- Disclaimer of Warranties and Limitation of Liability.

a. UNLESS OTHERWISE SEPARATELY UNDERTAKEN BY THE LICENSOR, TO THE EXTENT POSSIBLE, THE LICENSOR OFFERS THE LICENSED MATERIAL AS-IS AND AS-AVAILABLE, AND MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND CONCERNING THE LICENSED MATERIAL, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHER. THIS INCLUDES, WITHOUT LIMITATION, WARRANTIES OF TITLE, MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, ABSENCE OF LATENT OR OTHER DEFECTS, ACCURACY, OR THE PRESENCE OR ABSENCE OF ERRORS, WHETHER OR NOT KNOWN OR DISCOVERABLE. WHERE DISCLAIMERS OF WARRANTIES ARE NOT ALLOWED IN FULL OR IN PART, THIS DISCLAIMER MAY NOT APPLY TO YOU.

b. TO THE EXTENT POSSIBLE, IN NO EVENT WILL THE LICENSOR BE LIABLE TO YOU ON ANY LEGAL THEORY (INCLUDING, WITHOUT LIMITATION, NEGLIGENCE) OR OTHERWISE FOR ANY DIRECT, SPECIAL, INDIRECT, INCIDENTAL, CONSEQUENTIAL, PUNITIVE, EXEMPLARY, OR OTHER LOSSES, COSTS, EXPENSES, OR DAMAGES ARISING OUT OF THIS PUBLIC LICENSE OR USE OF THE LICENSED MATERIAL, EVEN IF THE LICENSOR HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH LOSSES, COSTS, EXPENSES, OR DAMAGES. WHERE A LIMITATION OF LIABILITY IS NOT ALLOWED IN FULL OR IN PART, THIS LIMITATION MAY NOT APPLY TO YOU.

c. The disclaimer of warranties and limitation of liability provided above shall be interpreted in a manner that, to the extent possible, most closely approximates an absolute disclaimer and waiver of all liability.

Section 6 -- Term and Termination.

a. This Public License applies for the term of the Copyright and Similar Rights licensed here. However, if You fail to comply with this Public License, then Your rights under this Public License terminate automatically.

b. Where Your right to use the Licensed Material has terminated under Section 6(a), it reinstates:

   1. automatically as of the date the violation is cured, provided it is cured within 30 days of Your discovery of the violation; or

   2. upon express reinstatement by the Licensor.

   For the avoidance of doubt, this Section 6(b) does not affect any right the Licensor may have to seek remedies for Your violations of this Public License.

c. For the avoidance of doubt, the Licensor may also offer the Licensed Material under separate terms or conditions or stop distributing the Licensed Material at any time; however, doing so will not terminate this Public License.

d. Sections 1, 5, 6, 7, and 8 survive termination of this Public License.

Section 7 -- Other Terms and Conditions.

a. The Licensor shall not be bound by any additional or different terms or conditions communicated by You unless expressly agreed.

b. Any arrangements, understandings, or agreements regarding the Licensed Material not stated herein are separate from and independent of the terms and conditions of this Public License.

Section 8 -- Interpretation.

a. For the avoidance of doubt, this Public License does not, and shall not be interpreted to, reduce, limit, restrict, or impose conditions on any use of the Licensed Material that could lawfully be made without permission under this Public License.

b. To the extent possible, if any provision of this Public License is deemed unenforceable, it shall be automatically reformed to the minimum extent necessary to make it enforceable. If the provision cannot be reformed, it shall be severed from this Public License without affecting the enforceability of the remaining terms and conditions.

Trademarks

Contents

How to use this guide

Every Choupo tutorial is a complete, runnable case:

```
source etc/bashrc
runCase tutorials/<category>/<group>/<name>
```

This guide gives each tutorial the CONTEXT the case file cannot: the theory block at the head of each group states the physics and the equations you need (condensed – the Theory Guide carries the full derivations), and each entry tells you **what the case does** and **what to expect** before you run it. The per-tutorial text is assembled from the cases' own headers – the case file is always the source of truth, and reading it IS part of the exercise.

Three reading rules:

1. **Run first, read second, re-run third.** The log at verbosity 3 shows every Newton iteration, every K-value, every announcement – the glass box is the point.
2. **Expect the announcements.** Choupo never silently helps: initial guesses, bounds that bind, model fallbacks, unit overrides – all are printed. If a run is quiet about something, it did not happen.
3. **Golden numbers are commitments.** Cases carrying an `expected` file pin their KPIs; the validation AADs quoted in the entries are locked by the regression suite and re-verified on every change.

Learning paths

The process-simulation path. Start with `flash01_benzene_toluene` (the Rachford–Rice loop, visible), then `flash` non-ideal cases (activity models bend the K-values), `distillation` (stages couple; Wang–Henke vs simultaneous), one `flowsheets` recycle case (tears and convergence), and close with a `gibbs` flame (equilibrium without mechanism). Then the batch side: an `ignition` case for stiffness – run the `_rk4` twin and watch the explicit method die – and a `ctrl` loop for feedback.

The electrolytes path. `pitzer01_nacl` ($\gamma_\pm$, φ, a_w of one salt), then `pitzer_gamma_hamer_wu` (the catalogue vs NBS data, zero refit), `pitzer_nacl_sp77_hot` (temperature, to 200°C), speciation (`scaling` cases: SI, pH, Davies-vs-Pitzer forking an engineering answer), the eNRTL series (`enrtl_mixed_nacl_ethanol_esteso` mixed-solvent; `enrtl_multisalt_nacl_kcl` two salts, two formulations, an experimental verdict), and the process payoff: evaporators and crystallisers where all of it sets duties and yields.

The combustion path. `gibbs06_h2_flame_radicals` (the radical pool at equilibrium; the c_p extrapolation trap), `ignition01_h2o2` (GRI kinetics, the stiffness lesson via the `_rk4` twin), `ignition02_h2air_slack` (quantitative: τ_{ign} vs shock-tube data), `nox01_zeldovich_postflame` (thermal NO; kinetics meets Gibbs at 98.9 %).

The props-bench path. `estimate_acetone` (CREATE: groups → constants, error shown), a `scan` case (SEE), `vle01_consistency` (TEST the data), a `fit` case (Levenberg–Marquardt with identifiability), then the promote-and-use loop into a steady case. The Props Guide is this path's companion.

Category I: Steady-state flowsheets (choupoSolve)

Solved as one nonlinear system $F(\mathbf{x}) = \mathbf{0}$: units in sequence, recycles as tears, everything announced.

absorption

The theory you need. Gas absorption transfers a solute from gas to liquid across stages; Henry's law ($p_i = H_i x_i$) anchors the equilibrium at dilution. Expect the L/G ratio to control the operating line's slope, and a minimum solvent rate at the pinch with the equilibrium line. *Full derivation: Theory Guide, the absorption section.*

`absorber01_NH3_water`

`tutorials/steady/absorption/absorber01_NH3_water`

What it does: NH3 absorption from air into water (Kremser, 6 stages)

The story: The textbook gas-absorption case: scrubbing ammonia out of an air stream with water, in a counter-current packed/plate column, solved by the Kremser group method (the built-in ‘absorber‘).

Ammonia is very soluble in water (Henry’s constant H ≈ 0.99 bar at 298 K → K ≈ 1 at atmospheric pressure), so a modest water rate and a handful of stages recover most of it — unlike CO2/water (H ≈ 1640 bar), where water is a poor solvent.

Feed gas: 100 kmol/h, 10 % NH3 / 90 % N2 (N2 = inert carrier) Solvent: 150 kmol/h pure water (L/V = 1.5) Column: 6 theoretical stages, 298.15 K, 1 atm

The model is NON-ISOTHERMAL: ammonia dissolution is strongly exothermic (ΔH ≈ -34 kJ/mol), so the heat of absorption heats the column. The energy balance is solved coupled with the mass balance.

Expected: liquid heats up ≈ +17 K (top 309 K, internal bulge ~318 K, bottom 315 K — the classic absorber "temperature bulge") recovery_NH3 ≈ 0.675 (only ~68 %, NOT the 97 % an isothermal Kremser would predict — because T rises, H rises, K rises, A falls: the exotherm self-limits the absorption. Real NH3 / amine absorbers are cooled for this reason.) N2 stays in the gas (K_N2 » 1 → no absorption) NH3 mass balance closes (in = clean-gas NH3 + rich-liquid NH3)

Design question ("how many stages for 90 %?") is a DesignSpec on ‘$stages‘, NOT a mode of this unit — the column has the stages it has; the recovery is what comes out.

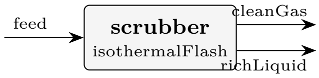
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`cleanGas`	`F`	`0.0259019084984`
`stream`	`gasFeed`	`F`	`0.0277777777778`
`stream`	`richLiquid`	`F`	`0.043542535946`
`stream`	`solvent`	`F`	`0.0416666666667`
`kpi`	`scrubber`	`A_N2`	`0.0027498297201`
`kpi`	`scrubber`	`A_NH3`	`1.53522727273`
`kpi`	`scrubber`	`F_cleanGas`	`0.0259019084984`
`kpi`	`scrubber`	`F_richLiquid`	`0.043542535946`
`kpi`	`scrubber`	`K_N2`	`545.488322072`
`kpi`	`scrubber`	`K_NH3`	`0.977054034049`
`kpi`	`scrubber`	`L_in`	`0.0416666666667`
`kpi`	`scrubber`	`L_over_V`	`1.5`
`kpi`	`scrubber`	`T_bottom`	`315.417418114`
`kpi`	`scrubber`	`T_top`	`308.88795701`

. . . and 10 more reference values in the case’s `expected`.

`absorption01_CO2_water`

`tutorials/steady/absorption/absorption01_CO2_water`



What it does: Single-stage CO2 absorption in water (Henry's law)

The story: single-stage CO2 absorption in water, modelled via the v0.28 Henry's-law infrastructure.

Feed: a 5100 kmol/h mixed stream of CO2 / N2 / water already at 298.15 K and 5 bar, with mole fractions matching what you'd obtain by mixing 100 kmol/h of a 10/90 CO2/N2 sour-gas with 5000 kmol/h of liquid water (L/G ~ 50:1 mole). An IsothermalFlash then partitions the species between vapour and liquid.

Why a pre-mixed feed and not Mixer + Flash? The Mixer carries a liquid-enthalpy balance that needs Cp_liq for every component, and CO2 is a permanent gas at 298 K with no liquid-Cp data (Tc = 304 K). Pre-mixing externally keeps the tutorial focused on the Henry's-law K-value selector — the point of the case.

Physics: H_CO2(298 K) = 1.64e8 Pa = 1640 bar (Henry's law) K_CO2 = H / P = 1.64e8 / 5e5 = 328 K_N2 » 1 (no Henry entry, no Antoine close to 298 K either — falls back to Raoult and Psat extrapolation) K_water « 1 (most water stays in liquid; Psat(298) ~ 3 kPa)

Observed result: cleanGas F = 66.0 kmol/h y_CO2 = 0.123 y_N2 = 0.871 y_water = 0.0063 richLiquid F = 5034 kmol/h x_CO2 = 3.75e-4 x_N2 = 6.5e-3 x_water = 0.993 V/F = 0.013 K_CO2 (y/x) = 328 ? exactly H_CO2(298)/P = 1.64e8 / 5e5 K_water = 0.0063 ? Psat_water(298)/P = 3.17 kPa / 500 kPa mass balance CO2: 10 in = 8.11 vapour + 1.89 liquid ~ 19 % of feed CO2 absorbed (a single-stage scrubber is far from perfect; an industrial column with 5-10 theoretical stages would recover > 95 %).

Toggling the GUI Units menu to Mass-basis shows the absorption as CO2 mass transfer between the two streams in kg/h, useful for scale-up arithmetic.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`cleanGas`	`F`	`0.0164210205273`
`stream`	`feed`	`F`	`1.41666666667`
`stream`	`richLiquid`	`F`	`1.40024564614`
`kpi`	`scrubber`	`F_alpha`	`1.40024564614`
`kpi`	`scrubber`	`F_beta`	`0.0164210205273`
`kpi`	`scrubber`	`F_in`	`1.41666666667`
`kpi`	`scrubber`	`K_CO2`	`327.999999984`
`kpi`	`scrubber`	`K_N2`	`110.543208463`
`kpi`	`scrubber`	`K_water`	`0.00633349747981`
`kpi`	`scrubber`	`P`	`500000`
`kpi`	`scrubber`	`Q`	`0`
`kpi`	`scrubber`	`Q_kW`	`0`
`kpi`	`scrubber`	`T`	`298.15`
`kpi`	`scrubber`	`V_over_F`	`0.0115913086075`

... and 4 more reference values in the case's `expected`.

`acetone05_luyben_absorber`

`tutorials/steady/absorption/acetone05_luyben_absorber`

What it does: Luyben's acetone absorber: 9 stages, water washing acetone out of the hydrogen offgas – against his own acetone and bottoms

The story: The unit acetone04 was measured FOR. Luyben's absorber washes acetone out of the reactor's hydrogen-rich offgas with water: 9 stages at 1.5 atm, 20 kmol/h of water on top, 39.80 kmol/h of gas at the bottom.

W. L. Luyben, Ind. Eng. Chem. Res. 50 (2011) 1206-1218, DOI 10.1021/ie901923a, Figure 1.

WHAT IS INDEPENDENT HERE, stated before the numbers because the last case in this family had to say the opposite. The gas inlet is Luyben's own ("Gas (to absorber)": 39.80 kmol/h, acetone 0.1147, IPA 0.0033) EXCEPT for its hydrogen and water, which were derived from the gas LEAVING using H2's inertness and a balance. So:

NOT independent – the outlet H2 and water (the inlet was built from them) INDEPENDENT – everything about the ACETONE, and the whole bottoms stream. Nothing in 0/ was built from those.

Luyben's answer, for the record, before this case is run:

bottoms 20.05 kmol/h acetone 0.1020 → 2.045 kmol/h absorbed offgas 39.76 kmol/h acetone 0.0634 → 2.521 kmol/h lost recovery 2.045 / 4.565 = 44.8 %

His own table closes on itself to 0.02 % in both acetone and water, which is two checks on a hand transcription and the reason those numbers are trusted as targets.

WHAT acetone04 PREDICTS FOR THIS UNIT. Measured there, on the same thermodynamics this case runs: gamma_acetone = 10.27 at 1 mol % in water and 318 K, giving K = 6.84. A model that holds acetone too weakly in the water will absorb LESS of it than Luyben does. That is the prediction; the run below is the test.

The thermodynamics is PREDICTIVE UNIFAC with H2 authorised outside the activity model – the same declared package as acetone03, for the same reasons, recorded in constant/thermoPhysPropDict.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.00574366573439
stream	gasIn	F	0.0110555555556
stream	offgas	F	0.0108674453767
stream	washWater	F	0.00555555555556
kpi	absorber	A_H2	0.00044153375233
kpi	absorber	A_acetone	0.0857422914441
kpi	absorber	A_isopropanol	4.51803004485
kpi	absorber	A_water	8.0125706439
kpi	absorber	F_cleanGas	0.0108674453767
kpi	absorber	F_richLiquid	0.00574366573439
kpi	absorber	K_H2	1138.10679288
kpi	absorber	K_acetone	5.86073166871
kpi	absorber	K_isopropanol	0.11122382052
kpi	absorber	K_water	0.0627155235376

... and 16 more reference values in the case's `expected`*.*

extract01_ethanol_water_benzene

tutorials/steady/absorption/extract01_ethanol_water_benzene

What it does: Counter-current LL extraction: recover ethanol from water with benzene

The story: Counter-current LL extraction column

Recover ETHANOL (the solute) from a dilute aqueous feed (WATER = the carrier) using BENZENE as the extracting SOLVENT. This is the classic liquid-liquid extraction problem: a multistage counter-current liquid-liquid cascade.

Topology (the ‘extractor‘ unit op): the feed (water + ethanol) enters at stage 1; fresh solvent (benzene) enters at stage N; the two liquid phases pass counter-currently — the raffinate (water-rich, ethanol-depleted) flows 1 → N, the extract (benzene-rich, ethanol-enriched) flows N → 1; EXTRACT leaves stage 1, RAFFINATE leaves stage N.

Each theoretical stage is ONE liquid-liquid equilibrium, solved by the SHARED Gibbs-energy-minimisation LL flash (IsothermalFlash, phaseSet LL) — robust against the K = 1 saddle that traps Newton-on-residual methods in LLE (CLAUDE.md sec.9). The cascade is torn by successive substitution (mirroring the distillation bubble-point loop).

Thermo: predictive UNIFAC for both liquid phases (water / ethanol / benzene groups inline), the same model used by the ternary LLE scan tutorial — no fitted pairs needed; self-DESCRIBING (inline manifest), UNIFAC records resolve from data/standards/ at runtime.

KPIs to watch: recovery_ethanol (fraction of ethanol leaving in the extract), distribution_ethanol (z_E / z_R), F_extract / F_raffinate, massClosure (must be ~0).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	extract	F	0.0359293973028
stream	feed	F	0.0277777777778
stream	raffinate	F	0.0251772962492
stream	solvent	F	0.0333333333333
kpi	extractor01	F_extract	0.0359293973028
kpi	extractor01	F_feed	0.0277777777778
kpi	extractor01	F_raffinate	0.0251772962492
kpi	extractor01	F_solvent	0.0333333333333
kpi	extractor01	P	101325
kpi	extractor01	T	298.15
kpi	extractor01	converged	1
kpi	extractor01	distribution_benzene	295.27509389
kpi	extractor01	distribution_ethanol	0.555106921422
kpi	extractor01	distribution_water	0.00706545462823

. . . and 13 more reference values in the case’s `expected`*.*

`extraction01_waterButanol`

`tutorials/steady/absorption/extraction01_waterButanol`

What it does: Water/n-butanol LL flash at 298 K (experimental)

The story: Water + 1-butanol at 25 °C. This system has a well-known liquid-liquid miscibility gap below 125.5 °C (upper critical solution temperature). A 70/30 water/nButanol feed (inside the gap) → splits into a water-rich (aqueous) phase ~98 mol% water and a butanol-rich (organic) phase ~59 mol% nButanol / 41 mol% water.

Demonstrates: ‘phases (...)‘ syntax with TWO liquid phases ‘phaseSet LL‘ in the IsothermalFlash Michelsen TPD stability test to find a robust initial guess

Water + 1-butanol at 25 °C and 1 atm. The pair has a well-known liquid-liquid miscibility gap below its upper critical solution temperature (125.5 °C), so a feed whose composition lies inside the gap spontaneously splits into two liquid phases. This case feeds a 70/30 water/nButanol mixture (inside the gap) to an isothermal decanter and lets the flash find the two-liquid split.

A single feed splits into two liquids, both at 298.15 K, vf = 0:

The split is found by Michelsen TPD stability testing (to seed a robust initial guess) followed by direct Gibbs-energy minimisation — the LL path the engine uses precisely because a Newton/successive-substitution flash on a symmetric γ-model would collapse to the trivial $K = 1$ saddle.

- phases (...) syntax with TWO liquid phases; - phaseSet LL in the IsothermalFlash; - the Michelsen TPD stability test as the initial-guess generator.

- The LL flash reuses the VL solver's printout, so the two liquids are shown under Vapor flow V / Liquid flow L headings (and x/y/K labels). Read them as phase-α (organic) and phase-β (aqueous), not as a vapour and a liquid. - The decanter is isothermal and adiabatic in reality (an enthalpy hand-check gives H_in ≈ H_out), so the physically correct duty is ≈ 0. The KPI Q and any utility/cost the run reports for this LL split are a known artifact of the shared flash duty path — do not cost this unit from them.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	aqueous	F	0.0140665296362
stream	feed	F	0.0277777777778
stream	organic	F	0.0137112481416
kpi	decanter01	F_alpha	0.0140665296362
kpi	decanter01	F_beta	0.0137112481416
kpi	decanter01	F_in	0.0277777777778
kpi	decanter01	P	101325
kpi	decanter01	T	298.15
kpi	decanter01	betaFraction	0.493604933098
kpi	decanter01	phaseSet	1
stream	aqueous	T	298.15
stream	feed	T	298.15
stream	organic	T	298.15

`stripper01_NH3_water`

`tutorials/steady/absorption/stripper01_NH3_water`

What it does: NH3 stripping from water with air (Kremser, 6 stages, warm)

The story: The inverse of absorber01: instead of scrubbing NH3 OUT of a gas, here we strip NH3 OUT of water with a lean gas (air ~ N2). Counter-current Kremser; the rich liquid enters the top, the stripping gas the bottom.

Run warm (330 K): NH3 is less soluble hot (K rises), so the stripping factor S = K V/L exceeds 1 and the strip is efficient. But desorption is endothermic, so the column cools – which lowers K and self-limits the stripping (the mirror image of the absorber's exotherm).

Liquid: 100 kmol/h, 5 % NH3 in water, 330 K Gas: 100 kmol/h N2 (V/L = 1), 330 K Column: 6 theoretical stages

Expected: S_NH3 > 1 → most of the NH3 stripped into the gas; stripped liquid nearly NH3-free; rich gas carries the NH3; mass closes 100 %.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`liquidFeed`	`F`	`0.0277777777778`
`stream`	`richGas`	`F`	`0.029140357434`
`stream`	`strippedLiquid`	`F`	`0.0264151981216`
`stream`	`strippingGas`	`F`	`0.0277777777778`
`kpi`	`stripper`	`F_richGas`	`0.029140357434`
`kpi`	`stripper`	`F_strippedLiquid`	`0.0264151981216`
`kpi`	`stripper`	`K_NH3`	`3.62862883208`
`kpi`	`stripper`	`K_water`	`0.170562530402`
`kpi`	`stripper`	`L_in`	`0.0277777777778`
`kpi`	`stripper`	`S_NH3`	`3.62862883208`
`kpi`	`stripper`	`S_water`	`0.170562530402`
`kpi`	`stripper`	`T_bottom`	`313.524974667`
`kpi`	`stripper`	`T_top`	`315.973930965`
`kpi`	`stripper`	`V_in`	`0.0277777777778`

. . . and 10 more reference values in the case's `expected`*.*

crystallisation

The theory you need. Crystallisation removes solute when the solution crosses saturation: $m > m_{\mathrm{sat}}(T)$, with $K_{\mathrm{sp}}(T)$ from ΔG_{diss} and its T-dependence from ΔH_{diss} (van 't Hoff). The heat of crystallisation is ION-DERIVED – $\Delta H_{\mathrm{cryst}} = -(\Delta H^{\infty}_{\mathrm{soln}} + \bar{L}_2(m_{\mathrm{sat}}))$ – never a stored constant (a solid's formation enthalpy is $\sum \nu_i h^{\mathrm{aq}}_{f,i} - \Delta H_{\mathrm{soln}}$, derived, single-sourced). Antisolvent cases add the mixed-solvent electrolyte model (eNRTL): ethanol drops the dielectric constant, $\gamma_{\pm}$ rises, salt crashes out. Expect supersaturation as the driving force and yields set by the solubility curve, not the kinetics (these cases are equilibrium-staged). *Full derivation: Theory Guide, the crystallisation + electrolyte-enthalpy chapters.*

crystalliser01_sugar

tutorials/steady/crystallisation/crystalliser01_sugar

What it does: Cooling crystalliser: sugar solution 80 → 20 C, yield by solubility

The story: / flowsheetDict (LEAF node)

A hot sugar solution (z_sucrose 0.15, ~3.35 kg/kg water, below saturation at 80 C where c_sat ~ 3.8) is cooled to 20 C (c_sat ~ 2.0 kg/kg). The excess crystallises. Magma = crystals (s[]) + saturated mother liquor.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	magma	F	0.0277777777778
kpi	cryst	Q_removed	271677.114808
kpi	cryst	T_op	293.15
kpi	cryst	c_sat	2.026
kpi	cryst	crystal_flow	0.564469958333
kpi	cryst	crystal_mol	0.00164906487154
kpi	cryst	liquorFlow	0.0261287129062
kpi	cryst	solute_dissolved	0.861767541667
kpi	cryst	solute_in	1.4262375
kpi	cryst	yield	0.395775569169
stream	feed	T	353.15
stream	magma	T	293.15
kpi	cryst	Q_kW	-271.677114808

`crystalliser02_msmpr`

`tutorials/steady/crystallisation/crystalliser02_msmpr`

What it does: MSMPR sugar crystalliser: population balance by method of moments

The story: / flowsheetDict (LEAF node)

Continuous MSMPR crystalliser. A hot near-saturated sugar solution is fed to a cooled, well-mixed vessel of working volume V; the magma (crystals + mother liquor) is withdrawn at the same volumetric rate, so the residence time is tau = V / Q. ‘model MSMPR‘ selects the population-balance path (method of moments); the kinetics are referenced from constant/crystallisation.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	magma	F	0.0277777777778
kpi	cryst	CV	1
kpi	cryst	L_dominant	0.000223952804481
kpi	cryst	L_meanMass	0.000298603739308
kpi	cryst	L_meanNumber	7.46509348269e-05
kpi	cryst	Q_removed	269525.057151
kpi	cryst	T_op	293.15
kpi	cryst	c_sat	2.026
kpi	cryst	crystal_flow	0.396190602513
kpi	cryst	crystal_mol	0.00115744690287
kpi	cryst	growthRate	1.95272330048e-08
kpi	cryst	liquorFlow	0.0266203308749
kpi	cryst	magmaDensity	302.920529908

. . . and 13 more reference values in the case’s `expected`.

crystalliser03_pbe_size_independent

tutorials/steady/crystallisation/crystalliser03_pbe_size_independent

What it does: Crystalliser FVM-PBE: sanity vs MSMPR (size-independent growth)

The story: / flowsheetDict (LEAF node)

Same physical setup as crystalliser02_msmpr (hot sugar syrup, 5 m^3 vessel at 20 °C, k_g 1e-7, k_b 1e9) – only the solver method changes: 'model FVM' discretises the population balance n(L) on Nbins evenly-spaced size bins, marching the steady PBE forward in L with first-order upwind. With sizeFactor 0 (the default), G(L) is constant and the FVM result must match the analytical MSMPR exponential.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	magma	F	0.0277777777778
kpi	cryst	CV	0.999540291345
kpi	cryst	L_dominant	0.000224609375
kpi	cryst	L_meanMass	0.000298633437171
kpi	cryst	L_meanNumber	7.48298114726e-05
kpi	cryst	Q_removed	269576.1605
kpi	cryst	T_op	293.15
kpi	cryst	bins	256
kpi	cryst	c_sat	2.026
kpi	cryst	crystal_flow	0.400186610054
kpi	cryst	crystal_mol	0.00116912099742
kpi	cryst	growthRate_L0	1.90635340048e-08
kpi	cryst	liquorFlow	0.0266086567804

. . . and 14 more reference values in the case's `expected`.

`crystalliser04_size_dependent_growth`

`tutorials/steady/crystallisation/crystalliser04_size_dependent_growth`

What it does: Crystalliser FVM-PBE with size-dependent growth (sizeFactor > 0)

The story: / flowsheetDict (LEAF node)

Same hot-sugar feed and 5 m^3 vessel at 20 °C as crystalliser03, only ‘sizeFactor‘ is non-zero — the size-dependent growth law that breaks the method-of-moments closure but is handled natively by the FVM.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`magma`	`F`	`0.0277777777778`
`kpi`	`cryst`	`CV`	`1.03562508094`
`kpi`	`cryst`	`L_dominant`	`0.000232421875`
`kpi`	`cryst`	`L_meanMass`	`0.000329374475484`
`kpi`	`cryst`	`L_meanNumber`	`7.45169890057e-05`
`kpi`	`cryst`	`Q_removed`	`269660.031771`
`kpi`	`cryst`	`T_op`	`293.15`
`kpi`	`cryst`	`bins`	`256`
`kpi`	`cryst`	`c_sat`	`2.026`
`kpi`	`cryst`	`crystal_flow`	`0.406744893212`
`kpi`	`cryst`	`crystal_mol`	`0.00118828062534`
`kpi`	`cryst`	`growthRate_L0`	`1.83025070562e-08`
`kpi`	`cryst`	`liquorFlow`	`0.0265894971524`
. . . and 14 more reference values in the case’s `expected`*.*			

crystalliser05_nacl_pitzer

tutorials/steady/crystallisation/crystalliser05_nacl_pitzer

What it does: NaCl crystalliser: saturation from the PITZER ion activity (Ksp = (gamma_pm∗m_sat)^2). Feed is a SUPERSATURATED brine (post-evaporation, ~7.6 mol/kg); the unit drops it to saturation (6.144 mol/kg) and crystallises the excess.

The story: NaCl crystalliser whose SATURATION comes from the Pitzer ion activity (Ksp = (gamma_pm ∗ m_sat)^2), via the 'electrolyte {}' block – NaCl carries no solubility{} curve, so the electrolyte model is what makes this possible. The feed is a SUPERSATURATED brine (~7.6 mol/kg, as it would leave an evaporator); the unit drops it to saturation (6.144 mol/kg) and crystallises the excess. Components live case-local in constant/components/; other curated records resolve from data/standards/ at runtime.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	magma	F	0.0277777777778
kpi	cryst	Ksp_activity	38.4846213082
kpi	cryst	Q_removed	1251.26049958
kpi	cryst	T_op	298.15
kpi	cryst	c_sat	0.35905536
kpi	cryst	crystal_flow	0.036683987968
kpi	cryst	crystal_mol	0.000627720533333
kpi	cryst	gamma_pm_sat	1.00970010562
kpi	cryst	liquorFlow	0.0271500572444
kpi	cryst	m_sat	6.144
kpi	cryst	solute_dissolved	0.158116012032
kpi	cryst	solute_in	0.1948
kpi	cryst	yield	0.18831616
. . . and 3 more reference values in the case's `expected`*.*			

`crystalliser06_nacl_antisolvent`

`tutorials/steady/crystallisation/crystalliser06_nacl_antisolvent`

What it does: Antisolvent (drowning-out) crystallisation of NaCl: a SATURATED aqueous brine is mixed with ethanol; the eNRTL mixed-solvent term lowers the dielectric, raises gamma_pm, and DROPS the saturation molality m_sat – so NaCl precipitates with NO cooling and NO evaporation. Saturation from the SAME ion-activity product Ksp anchored in pure water, solved with the mixed-solvent gamma_pm.

The story: ANTISOLVENT (drowning-out) crystallisation

[cryst] crystals (solid NaCl product) ethanol motherLiquor (saturated supernatant: water + ethanol + residual NaCl)

The crystalliser receives BOTH feeds directly – a saturated aqueous NaCl brine AND the ethanol antisolvent – combines them internally, and discharges its TWO natural products: the crystals and the saturated mother liquor. No separate mixer, no separate filter: an equilibrium crystalliser already KNOWS how much solid forms and what the saturated liquor is.

The physics: ethanol's low dielectric (24.3 vs water's 78.5) lowers the mixture permittivity, which RAISES the mean ionic activity gamma_pm (eNRTL mixed-solvent + Born), so the saturation molality m_sat COLLAPSES (6.14 → ~1.3 mol/kg) and NaCl precipitates with NO cooling and NO boil-off. The saturation is the SAME ion-activity product Ksp = (gamma_pm ∗ m_sat)^2 anchored in pure water, solved with the mixed-solvent gamma_pm at the feed's salt-free ethanol fraction – the physics the aqueous Pitzer model cannot reach (the reason eNRTL exists).

Basis: 100 kmol/h saturated brine (x_water 0.900, x_NaCl 0.100; ~6.14 mol/kg) + 60.0 kmol/h ethanol → salt-free x_ethanol ~ 0.40. Components live case-local in constant/components/; the eNRTL ion/tau records resolve from data/standards/ at runtime.

(Want imperfect separation – a wet cake with clinging liquor, fines loss, washing? Feed the single-output ‘magma‘ form to a downstream ‘filter‘.)

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`brine`	`F`	`0.0277777777778`
`stream`	`crystals`	`F`	`0.00202035108762`
`stream`	`ethanol`	`F`	`0.0166666666667`
`stream`	`motherLiquor`	`F`	`0.0424240933568`
`kpi`	`cryst`	`Ksp_activity`	`34.8439846653`
`kpi`	`cryst`	`Q_removed`	`6559.87377671`
`kpi`	`cryst`	`T_op`	`298.15`
`kpi`	`cryst`	`antisolvent_xfrac`	`0.399920015997`
`kpi`	`cryst`	`c_sat`	`0.0517444790334`
`kpi`	`cryst`	`cakeLiquidFlow`	`0.000329661969914`
`kpi`	`cryst`	`cakeMoisture`	`0.1`
`kpi`	`cryst`	`cakeWetness_pct`	`9.09090909091`
`kpi`	`cryst`	`crystal_flow`	`0.0988038720389`
`kpi`	`cryst`	`crystal_mol`	`0.00169068911771`

. . . and 13 more reference values in the case's `expected`.

crystalliser07_nacl_antisolvent_cooled

tutorials/steady/crystallisation/crystalliser07_nacl_antisolvent_cooled

What it does: Antisolvent + COOLING crystallisation of NaCl: the same drowning-out brine + ethanol as crystalliser06, but the crystalliser is COOLED to 5 C. The eNRTL is now temperature-dependent (tau(T)=tau_25∗298.15/T, A_DH(T) via eps_w(T)/rho_w(T), Born at T) and Ksp(T) shifts by van't Hoff with NaCl's enthalpy of solution – so cooling adds to the antisolvent's effect (yield 61% → 70%). Exercises the T-dependent electrolyte path.

The story: brine [cryst @ 5 C] crystals (wet NaCl cake) ethanol motherLiquor (saturated supernatant)

Identical drowning-out feed to crystalliser06 (saturated brine + ethanol, salt-free x_ethanol ~ 0.40), but the crystalliser is COOLED to 5 C. Two levers now stack: ANTISOLVENT – ethanol lowers the dielectric → mixed-solvent gamma_pm(T). COOLING – the electrolyte model is temperature-dependent: tau(T) = tau_25 ∗ 298.15/T (the Delta_g/RT form, like the molecular NRTL), the Debye-Huckel A_DH(T) follows eps_w(T)/rho_w(T), the Born term is at T, and the saturation product Ksp(T) shifts by van't Hoff with NaCl's enthalpy of solution (dissolutionEnthalpy in NaCl.dat). Result: m_sat ~ 0.885 → 0.681 mol/kg, yield 61% → 70% vs the isothermal 06. (For NaCl alone cooling is weak – solubility is nearly T-flat; the antisolvent carries most of the effect. A salt with a large dHsol, e.g. KNO3, would gain far more from cooling.)

Components live case-local in constant/components/; the eNRTL ion/tau records resolve from data/standards/ at runtime.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`brine`	`F`	`0.0277777777778`
`stream`	`crystals`	`F`	`0.00231987381407`
`stream`	`ethanol`	`F`	`0.0166666666667`
`stream`	`motherLiquor`	`F`	`0.0421245706304`
`kpi`	`cryst`	`Ksp_activity`	`31.135192385`
`kpi`	`cryst`	`Q_removed`	`82721.44242`
`kpi`	`cryst`	`T_op`	`278.15`
`kpi`	`cryst`	`antisolvent_xfrac`	`0.399920015997`
`kpi`	`cryst`	`c_sat`	`0.0398065230351`
`kpi`	`cryst`	`cakeLiquidFlow`	`0.000380305320939`
`kpi`	`cryst`	`cakeMoisture`	`0.1`
`kpi`	`cryst`	`cakeWetness_pct`	`9.09090909091`
`kpi`	`cryst`	`crystal_flow`	`0.113348382739`
`kpi`	`cryst`	`crystal_mol`	`0.00193956849313`

. . . and 13 more reference values in the case's `expected`.

crystalliser08_nacl_antisolvent_msmpr

tutorials/steady/crystallisation/crystalliser08_nacl_antisolvent_msmpr

What it does: Continuous MSMPR antisolvent crystallisation of NaCl: a saturated brine + ethanol fed to a well-mixed vessel (residence time tau = V/Q). The eNRTL antisolvent supersaturation S = c/c_sat(eNRTL) drives nucleation + growth; the method of moments closes the population balance and returns the CRYSTAL-SIZE DISTRIBUTION n(L) (mean size, CV) – what the equilibrium model cannot give.

The story: CONTINUOUS MSMPR + ANTISOLVENT

brine [cryst MSMPR] magma (crystals WITH a size distribution n(L) ethanol + saturated mother liquor)

Same drowning-out feed as crystalliser06 (saturated brine + ethanol), but the vessel is a CONTINUOUS, well-mixed MSMPR crystalliser of working volume V; the magma leaves at the same volumetric rate, so the residence time is tau = V/Q.

Where the equilibrium model stops at the YIELD, the population balance goes on to the CRYSTAL-SIZE DISTRIBUTION: the antisolvent supersaturation S = c / c_sat(eNRTL, T, x_ethanol) drives growth G = k_g (S-1)^g and nucleation B0 = k_b (S-1)^b; the method of moments gives n(L) = n0 exp(-L/Gtau), the mean sizes (L43 = 4 Gtau), the CV, and the magma PSD – plotted in the GUI.

The supersaturation here comes from the eNRTL ion-activity Ksp (the antisolvent lowers m_sat), NOT a solubility curve – the MSMPR now reads the electrolyte saturation, the same helper the equilibrium model uses. Components + kinetics live under constant/; the eNRTL ion/tau pair resolves from data/standards/ at runtime (self-DESCRIBING inline manifest).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	brine	F	0.0277777777778
stream	ethanol	F	0.0166666666667
stream	magma	F	0.0444444444444
kpi	cryst	CV	1
kpi	cryst	L_dominant	2.02534396647e-05
kpi	cryst	L_meanMass	2.70045862196e-05
kpi	cryst	L_meanNumber	6.75114655489e-06
kpi	cryst	Q_kW	-6.1566687571
kpi	cryst	Q_removed	6156.6687571
kpi	cryst	T_op	298.15
kpi	cryst	c_sat	0.0517444790334
kpi	cryst	crystal_flow	0.092730856228
kpi	cryst	crystal_mol	0.00158677029822
kpi	cryst	growthRate	1.92664299147e-09

. . . and 15 more reference values in the case's `expected`.

crystalliser09_KHT_KCl_series

tutorials/steady/crystallisation/crystalliser09_KHT_KCl_series

What it does: Two crystallisers in series separating KCl from potassium bitartrate (KHT, cream of tartar) by manipulating T and adding ethanol. CRYST1 COOLS the hot feed to 5 C → KHT (sparingly soluble, steep solubility curve) crystallises while KCl stays dissolved. CRYST2 doses ETHANOL into the KCl-rich mother liquor → KCl drops out by drowning-out (eNRTL mixed-solvent). Two salts, two saturation routes (curve vs ion-activity), one package – a stress test of the property system.

The story: SELECTIVE crystallisation of two salts

feed (hot: KCl + KHT) [cryst1: COOL to 5 C] KHT_cake (cream of tartar)

ethanol [cryst2: ANTISOLVENT] KCl_cake

finalLiquor (EtOH+water+salt)

[recovery: DISTILLATION] recoveredEthanol (recycle-grade)

stillage (water + residual salt)

Two salts with very different solubilities are separated by two DIFFERENT levers, each crystalliser targeting one salt (‘solute ...‘):

CRYST1 cools the hot feed. Potassium bitartrate (KHT) is sparingly soluble with a STEEP solubility curve, so cooling 50 → 5 C precipitates most of it; KCl, far more soluble and only mildly T-sensitive, stays dissolved. KHT’s saturation is its measured solubility CURVE c_sat(T) – no ion parameters.

CRYST2 doses ethanol into the KCl-rich mother liquor. KCl drops out by drowning-out: the eNRTL mixed-solvent term raises gamma_pm as the dielectric falls, collapsing m_sat. KCl’s saturation is the ion-activity Ksp.

The SAME package carries both routes; ‘useElec‘ is decided per solute, so KHT uses its curve while KCl uses the eNRTL. Self-DESCRIBING (inline manifest); curated records resolve from data/standards/ at runtime.

Basis (per kg water): 55.5 water, 3.0 KCl (below the 5 C saturation ~3.9), and 0.078 KHT (saturated at 50 C).

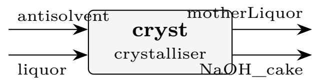
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	KCl_cake	F	0.00138859586739
stream	KHT_cake	F	5.38404641059e-05
stream	feed	F	0.0277777777792
stream	finalLiquor	F	0.0541131192255
stream	freshEthanol	F	0.00416666666667
stream	liquor1	F	0.0277239373151
stream	recoveredEthanol	F	0.0236111111111
stream	stillage	F	0.0305020081144
kpi	cryst1	Q_kW	-93.1162690166
kpi	cryst1	Q_removed	93116.2690166
kpi	cryst1	T_op	278.15
kpi	cryst1	c_sat	0.0034013435
kpi	cryst1	cakeLiquidFlow	2.54671494088e-05
kpi	cryst1	cakeMoisture	0.1

...and 57 more reference values in the case’s `expected`.

crystalliser10_naoh_antisolvent

tutorials/steady/crystallisation/crystalliser10_naoh_antisolvent



What it does: NaOH antisolvent crystallisation (water + ethanol, eNRTL drowning-out) – the acceptance-case core

The story: H2O + NaOH + ethanol, every effect that IS calibrated active, every one that is NOT loudly announced: ACTIVE: drowning-out saturation (eNRTL mixed-solvent; Na-OH taus anchored to the Parker-slotted Pitzer surface), BPE-grade water activity, differential-Ksp lineage, wet-cake split. ANNOUNCED OMISSIONS (honest): heat of dilution / crystallisation heat use the dHcryst constant (the eNRTL kernel is NOT calorimetrically fitted – per-kernel flag); water-ethanol heat of mixing gated until the H_E refit; the solid is ANHYDROUS NaOH until the NaOH.H2O hydrate forms entry lands.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	NaOH_cake	F	0.00183771747926
stream	antisolvent	F	0.00833333333333
stream	liquor	F	0.0138888888889
stream	motherLiquor	F	0.020384504743
kpi	cryst	Ksp_activity	7365.8380048
kpi	cryst	Q_kW	72.7276039114
kpi	cryst	Q_removed	-72727.6039114
kpi	cryst	T_op	298.15
kpi	cryst	antisolvent_xfrac	0.461538461538
kpi	cryst	c_sat	0.181200024556
kpi	cryst	cakeLiquidFlow	0.000203756483719
kpi	cryst	cakeMoisture	0.1
kpi	cryst	cakeWetness_pct	9.09090909091
kpi	cryst	crystal_flow	0.0653535379386

. . . and 13 more reference values in the case's `expected`*.*

`crystalliser11_kcl_antisolvent_ethanol`

`tutorials/steady/crystallisation/crystalliser11_kcl_antisolvent_ethanol`

What it does: Antisolvent (drowning-out) crystallisation of KCl: a SATURATED aqueous KCl brine (4.76 mol/kg) is mixed with ethanol; the eNRTL mixed-solvent term lowers the dielectric, raises gamma_pm, and DROPS the saturation molality – so KCl precipitates with NO cooling and NO evaporation. Saturation is the SAME ion-activity product Ksp anchored in pure water, solved with the mixed-solvent gamma_pm at the feed's salt-free ethanol fraction. KCl-water-ethanol experimental reference: Lopes, Farelo & Ferra, J. Solution Chem. 28 (1999) 117.

The story: ANTISOLVENT (drowning-out) of KCl

[cryst] crystals (solid KCl product) ethanol motherLiquor (saturated supernatant: water + ethanol + residual KCl)

The KCl counterpart of crystalliser06 (NaCl). KCl is the second salt the catalogue's KCl.dat names as "recovered by ANTISOLVENT (drowning-out with ethanol)"; this case realises it.

The physics: ethanol's low dielectric (24.3 vs water's 78.5) lowers the mixture permittivity, which RAISES the mean ionic activity gamma_pm (eNRTL mixed-solvent + Born), so the KCl saturation molality m_sat COLLAPSES from its aqueous 4.76 mol/kg and KCl precipitates with NO cooling and NO boil-off. The saturation is the SAME ion-activity product Ksp = (gamma_pm ∗ m_sat)^2 anchored in pure water, solved with the mixed-solvent gamma_pm at the feed's salt-free ethanol fraction.

Basis: 100 kmol/h saturated aqueous brine (x_water 0.9210, x_KCl 0.0790; 4.76 mol KCl / kg water) + 60.0 kmol/h ethanol → salt-free x_ethanol ~ 0.39. Components live case-local in constant/components/; the eNRTL ion/tau records resolve from data/standards/ at runtime.

Experimental KCl-water-ethanol reference (the drowning-out anchor): Lopes, Farelo & Ferra, J. Solution Chem. 28 (1999) 117.

The KCl counterpart of crystalliser06 (NaCl): a saturated aqueous KCl brine is mixed with ethanol, whose low dielectric collapses the KCl solubility, so KCl crystallises with no cooling and no evaporation. KCl is the salt the catalogue's KCl.dat names as ∗"recovered by ANTISOLVENT (drowning-out with ethanol)"∗ — this case realises it.

Ethanol's relative permittivity (24.3) is far below water's (78.5). Dosing it lowers the mixed-solvent dielectric, which raises the mean ionic activity coefficient $\gamma\pm$ (eNRTL mixed-solvent + Born term), so the saturation molality collapses. The saturation is the same ion-activity product Ksp = $(\gamma\pm \cdot m\_sat)^2$ anchored in pure water, re-solved with the mixed-solvent $\gamma\pm$ at the feed's salt-free ethanol fraction — the physics the aqueous Pitzer model cannot reach (the reason eNRTL exists).

100 kmol/h saturated brine (x_water 0.921, x_KCl 0.079 = 4.76 mol/kg) + 60 kmol/h ethanol → salt-free x_ethanol ≈ 0.39:

The KCl–water–ethanol activity behaviour modelled here is measured in A. Lopes, F. Farelo & M. I. A. Ferra, "Activity Coefficients of Potassium Chloride in Water–Ethanol Mixtures", ∗J. Solution Chem.∗ 28 (1999) 117 — mean activity coefficients of KCl by emf in 5–20 % w/w ethanol, 25–45 °C. That $\gamma\pm$ rise with ethanol content is exactly the driver of the drowning-out captured above.

- The eNRTL τ for K–Cl is the illustrative 1:1-alkali-chloride proxy (= Chen & Evans water–NaCl), so the numbers are qualitative. The Farelo data is the path to a quantitative refit of the KCl–water–ethanol interaction (the paper reports Pitzer $\beta^0/\beta^1/C?$ per solvent composition). - The mass / yield is the reliable output. The crystalliser's heat term carries the shared electrolyte-enthalpy caveat (a dissolved salt's formation datum lives in its ions, so the stream enthalpy falls back to the sensible datum and is announced) — identical to crystalliser06.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	brine	F	0.0277777777778
stream	crystals	F	0.0021050886953
stream	ethanol	F	0.0166666666667
stream	motherLiquor	F	0.0423393557491
kpi	cryst	Ksp_activity	7.69319557877
kpi	cryst	Q_kW	-28.9281906957
kpi	cryst	Q_removed	28928.1906957
kpi	cryst	T_op	298.15
kpi	cryst	antisolvent_xfrac	0.394477317554
kpi	cryst	c_sat	0.0311001943033
kpi	cryst	cakeLiquidFlow	0.000423217143227
kpi	cryst	cakeMoisture	0.1
kpi	cryst	cakeWetness_pct	9.09090909091
kpi	cryst	crystal_flow	0.125385206079

. . . and 13 more reference values in the case's `expected`.

crystalliser12_nacl_pitzer_antisolvent

tutorials/steady/crystallisation/crystalliser12_nacl_pitzer_antisolvent

What it does: Pitzer + antisolvent: the calorimetric L_phi fit is AQUEOUS, so a mixed solvent drops it – and the duty basis says so

The story: crystalliser06_nacl_antisolvent – ANTISOLVENT (drowning-out) crystallisation

[cryst] crystals (solid NaCl product) ethanol motherLiquor (saturated supernatant: water + ethanol + residual NaCl)

The crystalliser receives BOTH feeds directly – a saturated aqueous NaCl brine AND the ethanol antisolvent – combines them internally, and discharges its TWO natural products: the crystals and the saturated mother liquor. No separate mixer, no separate filter: an equilibrium crystalliser already KNOWS how much solid forms and what the saturated liquor is.

The physics: ethanol's low dielectric (24.3 vs water's 78.5) lowers the mixture permittivity, which RAISES the mean ionic activity gamma_pm (eNRTL mixed-solvent + Born), so the saturation molality m_sat COLLAPSES (6.14 → ~1.3 mol/kg) and NaCl precipitates with NO cooling and NO boil-off. The saturation is the SAME ion-activity product Ksp = (gamma_pm ∗ m_sat)ˆ2 anchored in pure water, solved with the mixed-solvent gamma_pm at the feed's salt-free ethanol fraction – the physics the aqueous Pitzer model cannot reach (the reason eNRTL exists).

Basis: 100 kmol/h saturated brine (x_water 0.900, x_NaCl 0.100; ~6.14 mol/kg) + 60.0 kmol/h ethanol → salt-free x_ethanol ~ 0.40. Components live case-local in constant/components/; the eNRTL ion/tau records resolve from data/standards/ at runtime.

(Want imperfect separation – a wet cake with clinging liquor, fines loss, washing? Feed the single-output 'magma' form to a downstream 'filter'.)

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	brine	F	0.0277777777778
stream	crystals	F	0
stream	ethanol	F	0.0166666666667
stream	motherLiquor	F	0.0444444444444
kpi	cryst	Ksp_activity	38.4846213082
kpi	cryst	Q_kW	-0
kpi	cryst	Q_removed	0
kpi	cryst	T_op	298.15
kpi	cryst	antisolvent_xfrac	0.399920015997
kpi	cryst	c_sat	0.35905536
kpi	cryst	cakeLiquidFlow	0
kpi	cryst	cakeMoisture	0.1
kpi	cryst	cakeWetness_pct	0
kpi	cryst	crystal_flow	0

... and 13 more reference values in the case's `expected`.

`overlay02_nacl_calorimetric`

`tutorials/steady/crystallisation/overlay02_nacl_calorimetric`

What it does: NaCl crystalliser: saturation from the PITZER ion activity (Ksp = (gamma_pm*m_sat)^2). Feed is a SUPERSATURATED brine (post-evaporation, ~7.6 mol/kg); the unit drops it to saturation (6.144 mol/kg) and crystallises the excess.

The story: crystalliser05_nacl_pitzer (LEAF node)

NaCl crystalliser whose SATURATION comes from the Pitzer ion activity (Ksp = (gamma_pm ∗ m_sat)^2), via the 'electrolyte {}' block – NaCl carries no solubility{} curve, so the electrolyte model is what makes this possible. The feed is a SUPERSATURATED brine (~7.6 mol/kg, as it would leave an evaporator); the unit drops it to saturation (6.144 mol/kg) and crystallises the excess. Components live case-local in constant/components/; other curated records resolve from data/standards/ at runtime.

A NaCl crystalliser (cloned from crystalliser05) whose case-local constant/components/NaCl.dat declares overlayOf NaCl; and recalibrates ONLY solidPhases.halite.calorimetric.dissolutionEnthalpy (3880 → 5880 J/mol).

The ThermoPackageBuilder deep-merges this delta over the standard record, so the crystalliser duty reflects it: Q_removed = 3690.997 W here vs 2435.556 in crystalliser05 (ratio 1.515 = 5880/3880). Every other KPI (Ksp_activity, m_sat, c_sat, crystal_mol, gamma) is byte-identical to crystalliser05 – logK25, dH, the dissolution reaction and the crystal block all still come from the standard.

This is the second half of the roadmap Phase A acceptance test: the SINGLE unified reader applies the overlay at ALL three component-loading sites (SpeciationSolver, Database, ThermoPackageBuilder), not just the SI path.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`magma`	`F`	`0.0277777777778`
`kpi`	`cryst`	`Ksp_activity`	`38.4846213082`
`kpi`	`cryst`	`Q_kW`	`-2.50670156624`
`kpi`	`cryst`	`Q_removed`	`2506.70156624`
`kpi`	`cryst`	`T_op`	`298.15`
`kpi`	`cryst`	`c_sat`	`0.35905536`
`kpi`	`cryst`	`crystal_flow`	`0.036683987968`
`kpi`	`cryst`	`crystal_mol`	`0.000627720533333`
`kpi`	`cryst`	`gamma_pm_sat`	`1.00970010562`
`kpi`	`cryst`	`liquorFlow`	`0.0271500572444`
`kpi`	`cryst`	`m_sat`	`6.144`
`kpi`	`cryst`	`solute_dissolved`	`0.158116012032`
`kpi`	`cryst`	`solute_in`	`0.1948`

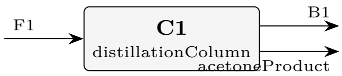
... and 3 more reference values in the case's `expected`.

distillation

The theory you need. A column is N flash stages coupled by counter-current flows. Each stage obeys MESH equations: Material balance, Equilibrium ($y_i = K_i x_i$), Summation ($\sum y = \sum x = 1$), and Heat balance. Choupo ships two solution strategies, selectable per column: Wang–Henke (bubble-point, solving the tridiagonal material balances stage by stage; robust for near-ideal systems, slow near azeotropes) and `simultaneous` (one Newton on ALL the MESH residuals; ∼5–6 iterations even through azeotropes). Expect: reflux and boilup set the split; a minimum reflux below which no stage count reaches the spec; and azeotropes as fixed points no column can cross (x_D pinned at the azeotropic composition). *Full derivation: Theory Guide, the distillation chapter (McCabe–Thiele + MESH).*

acetone06_luyben_column_C1

tutorials/steady/distillation/acetone06_luyben_column_C1



What it does: Luyben's acetone column C1 on predictive UNIFAC: the test of a prediction written before the column was built

The story: THE COLUMN THAT TESTS A PREDICTION WRITTEN BEFORE IT EXISTED.

W. L. Luyben, Ind. Eng. Chem. Res. 50 (2011) 1206-1218, DOI 10.1021/ie901923a. Column C1 of his Figure 1: 66 stages, feed on stage 54, reflux ratio 2.78, distillate 32.25 kmol/h, 1 atm. It takes the combined flash liquid plus absorber bottoms (his F1) and is asked to make the ACETONE PRODUCT at 99.9 mol %.

THE PREDICTION, quoted from the sibling props case rather than reinvented. 'props/compare/acetone04_acetone_water_vle' measured the acetone/water VLE this column rests on BEFORE any column was built, and found that predictive UNIFAC puts a minimum-boiling azeotrope at x_acetone ~ 0.978 where Luyben's UNIQUAC has only a pinch. It is a tiny feature – 0.044 K deep, |y - x| never above 7.7e-4 – and acetone04 recorded, in advance and in writing:

"A column C1 built on this thermodynamics should be expected to asymptote near 97.9 % and fail his spec – and if it had been built before this was measured, the column would have been blamed for a defect that lives in the activity model."

So THIS case has a pass/fail criterion that is not its own golden: either the distillate stalls near the predicted ceiling, or the prediction was wrong. Both outcomes are results. Neither is a reason to touch the thermodynamics.

THE FEED IS LUYBEN'S OWN F1, not this project's upstream answer. acetone03 computes a flash liquid and acetone05 an absorber bottoms, and their sum is what physically feeds C1 – but wiring them in would test the column and two upstream units at once, and acetone05 already reports that it recovers a sixth of what the paper does. Feeding the PUBLISHED stream isolates the column: whatever it gets wrong here is the column and its thermodynamics. The composition in 0/F1 is his mole fractions times his 72.85 kmol/h.

WHAT IS NOT LUYBEN'S, and why each choice is the honest one:

UNIQUAC vs UNIFAC. He runs UNIQUAC; Choupo ships no fitted pair for any of these binaries and inventing one would convert unsourced into falsely sourced. Predictive UNIFAC is what is available, and acetone04 measured what it costs on the binary that decides this column.

The rigorous MESH ('model simultaneous'). Wang-Henke is the corpus default and is documented as unstable through an azeotrope, which is precisely the feature under test here. Choosing the method that CAN resolve the feature is not tuning; choosing the one that cannot would have hidden it.

Isopropanol's liquid heat capacity is Rowlinson-Bondi off the same Joback fragmentation as the rest of its record. Its measured accuracy on hydrogen-bonding molecules is 30-70 %, stated in the record itself. It is 5 mol % of the feed and it moves the DUTIES, not the split.

THIS COLUMN IS MODELLED DOWNSTREAM OF LUYBEN'S VENT, so it carries no hydrogen. His C1 has a partial condenser with a vent; Choupo's column has a total condenser and no vapour distillate. Both facts are stated before the reason, because the reason has to be argued rather than asserted:

- his vent is 0.0465 kmol/h at H2 0.2798, i.e. 0.0130 kmol/h of hydrogen, against the 0.0146 kmol/h that F1 carries. The vent removes essentially ALL of it. A column model with no vent and a total condenser therefore represents the column AFTER the vent, which is a statement about WHICH UNIT is being modelled – not an approximation to the physics of the one that is.

- what that costs is quantified rather than hidden: the same vent carries 0.0335 kmol/h of ACETONE (his own figure), 0.10 % of the product. Not modelling the vent therefore FLATTERS this column's acetone recovery by that much, and the comparison below must be read knowing it.

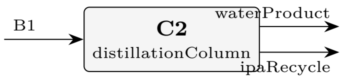
IT WAS RUN THE OTHER WAY FIRST, and the run is the reason this paragraph exists rather than a preference. With H2 in the feed the MESH converged (16 Newton iterations, ||F|| 2.4e-11) to essentially the same distillate – x_acetone 0.9722 vs the shipped run's goldened 0.9731 – and then the energy report REFUSED: hydrogen has no liquid heat capacity, because at 330 K it has no liquid. The refusal is right, and it is the honest form of the modelling error: a total condenser was silently asking a permanent gas to leave as a liquid. See this case's CLAUDE.md.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	B1	F	0.0112717083333
stream	F1	F	0.0202300416667
stream	acetoneProduct	F	0.00895833333333
kpi	C1	B	0.0112717083333
kpi	C1	D	0.00895833333333
kpi	C1	Q_condenser_kW	-997.746586798
kpi	C1	Q_reboiler_kW	1030.49217736
kpi	C1	R	2.78
kpi	C1	T_bottom	353.606283382
kpi	C1	T_top	329.375698025
kpi	C1	feedStage	54
kpi	C1	nFeeds	1
kpi	C1	nSideDraws	0
kpi	C1	nStages	66
... and 17 more reference values in the case's `expected`.			

`acetone07_luyben_column_C2`

`tutorials/steady/distillation/acetone07_luyben_column_C2`



What it does: Luyben's water column C2: the IPA recycle meets the azeotrope acetone01 measured

The story: THE SECOND COLUMN, AND THE SECOND PREDICTION WRITTEN BEFORE IT.

W. L. Luyben, Ind. Eng. Chem. Res. 50 (2011) 1206-1218, DOI 10.1021/ie901923a. Column C2 of his Figure 1: 19 stages, feed on stage 16, reflux ratio 0.849, 1 atm. It takes C1's bottoms and makes the plant's WATER PRODUCT at the bottom (34.67 kmol/h, 99.9 mol % water) while sending the unconverted isopropanol back overhead as the RECYCLE (5.88 kmol/h at x_IPA 0.6500).

THE PREDICTION, quoted from acetone01 rather than reinvented, and committed to the repository BEFORE this case was written:

"C2's overhead should cap at roughly x_IPA 0.59 and cannot reach his 0.650. The gap is 0.06 in mole fraction, about 9 % relative, and it is a HARD ceiling rather than a shortfall in stages or reflux."

The reason is one measured number: acetone01 puts the UNIFAC IPA/water azeotrope at x_IPA 0.5901 against Luyben's UNIQUAC value of 0.6732. A column cannot distil past a minimum-boiling azeotrope from below.

AND THE DIRECTION OF THIS ERROR IS THE OPPOSITE OF C1's, which is the part worth carrying away. C1's azeotrope is a UNIFAC INVENTION – Luyben's model has none in acetone/water – so it costs purity the paper does not lose. Here the azeotrope is REAL in both models and UNIFAC merely puts it in the wrong place. One failure is a spurious feature; the other is a misplaced real one. They are not the same defect and should not be reported as one.

THE FEED IS LUYBEN'S PUBLISHED B1, not acetone06's answer – the same choice and the same reason as in C1: it isolates the column. His B1 is 40.56 kmol/h at 370 K, A 0.0001 / IPA 0.0951 / W 0.9048. The CLOSED flowsheet (acetone08) is where the two columns finally see each other's real streams, and the difference between this case and that one is exactly the propagation the plant exists to show.

THE DISTILLATE RATE IS HIS, 5.88 kmol/h, and that matters here in a way it did not in C1. Fixing the rate means the column must send that much material overhead whatever its composition can be – so if the ceiling binds, it shows up as a WETTER recycle at the same flow, not as a smaller one. That is the honest way to pose it: the plant's IPA inventory has to go somewhere, and the question is what quality it comes back at.

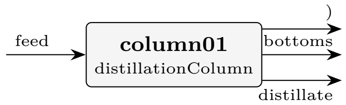
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`B1`	`F`	`0.0112666944444`
`stream`	`ipaRecycle`	`F`	`0.00163333333333`
`stream`	`waterProduct`	`F`	`0.00963336111111`
`kpi`	`C2`	`B`	`0.00963336111111`
`kpi`	`C2`	`D`	`0.00163333333333`
`kpi`	`C2`	`Q_condenser_kW`	`-120.936005804`
`kpi`	`C2`	`Q_reboiler_kW`	`114.900183788`
`kpi`	`C2`	`R`	`0.849`
`kpi`	`C2`	`T_bottom`	`366.174785539`
`kpi`	`C2`	`T_top`	`355.876498071`
`kpi`	`C2`	`feedStage`	`16`
`kpi`	`C2`	`nFeeds`	`1`
`kpi`	`C2`	`nSideDraws`	`0`
`kpi`	`C2`	`nStages`	`19`

. . . and 17 more reference values in the case's `expected`.

`column01_benzene_toluene`

`tutorials/steady/distillation/column01_benzene_toluene`



What it does: Benzene/toluene distillation column (Wang-Henke)

The story: Benzene/toluene distillation — classical textbook problem with Raoult, no azeotrope, easy convergence. Used to validate the column code.

The same equimolar benzene/toluene feed (100 kmol/h, now at 370 K) in a rigorous column: 15 stages (reboiler counted), feed on stage 8, R = 2, distillate 50 kmol/h, Raoult VLE, solved by the Wang–Henke bubble-point method. The golden: x_D = 0.9812 benzene, x_B = 0.9812 toluene, T_top 354.21 K, T_bottom 382.89 K, condenser -1281 kW, reboiler 1279 kW.

1. The specification is the column, not the split. Compare with shortcut01: there you declared the recoveries and got N; here you declare N, the feed stage, R and D — and the purity is the ∗answer∗. Both cases separate the same feed; only the question differs. 2. Watch it converge. The Log tab prints Outer iter max T_top T_bottom for all 108 bubble-point iterations, the final max = 9.3e-8. Wang–Henke is slow and steady; column02_simultaneous solves the identical column by a Newton on all the MESH equations in a handful of iterations — run both and count. 3. The profile is printed stage by stage. T and x for every tray, the feed tray marked, the reboiler last: 354.21 K and 95.3 % benzene on tray 1, 382.89 K and 98.1 % toluene in the reboiler. That table is the McCabe–Thiele diagram in numbers. 4. Two duties, two utilities, priced from the catalogue. The reboiler is served by low-pressure steam (0.585 kg/s, 55.3 ?/h at the catalogue price) and the condenser by cooling water (30.6 kg/s, 1.84 ?/h) — the same picker every steady report uses, and the ?/h is a catalogue number, not a market one. 5. Read the caveats. Both Antoine fits are asked slightly outside their declared windows on the way to the answer (benzene at 365 K, toluene at 380.7 K), announced and still returned.

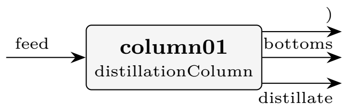
Raise refluxRatio to 3: purities climb, both duties climb with them — the trade-off every column design turns on. Then try feedStage 4: the same reflux buys less separation when the feed lands in the wrong place.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	column01	T_top	354.21089412
kpi	column01	T_bottom	382.893530951
kpi	column01	x_D_LK	0.981245045999
kpi	column01	x_B_HK	0.981245183716
utility	column01	heating.steamLP.duty_kW	1279.30480005
utility	column01	heating.steamLP.T	382.893530951
utility	column01	heating.steamLP.kg_s	0.585226349518
utility	column01	heating.steamLP.eur_h	55.265967362
utility	column01	cooling.coolingWater.duty_kW	-1281.04348002
utility	column01	cooling.coolingWater.T	354.21089412
utility	column01	cooling.coolingWater.kg_s	30.6469732063
utility	column01	cooling.coolingWater.eur_h	1.84470261124
stream	bottoms	F	0.0138888888889
stream	bottoms	T	382.893530951
. . . and 15 more reference values in the case's `expected`*.*			

column02_simultaneous

tutorials/steady/distillation/column02_simultaneous



What it does: Rigorous distillation by simultaneous MESH Newton (benzene/toluene; validates against column01)

The story: The SAME benzene/toluene column as column01, but solved by the rigorous SIMULTANEOUS-correction (MESH Newton) method instead of the sequential bubble-point (Wang-Henke). Selected by ‘model simultaneous;‘.

Pedagogical point: on an ideal system both methods give the SAME answer — this case validates the MESH solver against column01. But the simultaneous Newton converges in ~6 iterations (quadratic) vs ~100 for the bubble-point, and — the reason it exists — it stays stable on azeotropic systems where the sequential method does not.

Expected (identical to column01): T_top ≈ 354.2 K, T_bottom ≈ 382.9 K, x_D(benzene) ≈ 0.981, x_B(toluene) ≈ 0.981.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.0138888888889`
`stream`	`distillate`	`F`	`0.0138888888889`
`stream`	`feed`	`F`	`0.0277777777778`
`kpi`	`column01`	`B`	`0.0138888888889`
`kpi`	`column01`	`D`	`0.0138888888889`
`kpi`	`column01`	`R`	`2`
`kpi`	`column01`	`T_bottom`	`382.893527916`
`kpi`	`column01`	`T_top`	`354.210890137`
`kpi`	`column01`	`feedStage`	`8`
`kpi`	`column01`	`nStages`	`15`
`kpi`	`column01`	`x_B_HK`	`0.981245125322`
`kpi`	`column01`	`x_D_LK`	`0.981245125322`
`utility`	`column01`	`heating.steamLP.duty_kW`	`1299.61040477`
`utility`	`column01`	`heating.steamLP.T`	`382.893527916`
. . . and 13 more reference values in the case’s `expected`.			

column03_azeotrope_mesh

tutorials/steady/distillation/column03_azeotrope_mesh

What it does: Azeotropic distillation (ethanol/water, NRTL) by the rigorous simultaneous MESH method

The story: Ethanol/water (NRTL) distillation by the rigorous SIMULTANEOUS (MESH Newton) method. This is the payoff of the v0.48 solver: ethanol/water forms an azeotrope at x_eth ~ 0.894 (1 atm), and the sequential bubble-point (Wang-Henke) is unstable here – on this very case it does NOT converge (200 outer iters). The MESH Newton converges in ~5 iterations to a PHYSICAL distillate (x_eth ~ 0.80, below the azeotrope).

Feed: 100 kmol/h, 30/70 ethanol/water, 1 atm. The column feed enthalpy is set by feedQuality = 1.0 (saturated liquid), NOT by the stream T: the stated T = 363 K is ~7 K above the 30/70 bubble point (~355.5 K at 1 atm), so the stream itself is slightly superheated — the MESH model uses feedQuality, so this does not affect the column solution. Column: 16 stages, feed at 8, R = 2.5, D = 25 kmol/h, method simultaneous.

Expected: converged; x_D(ethanol) ~ 0.80 (< 0.894 azeotrope – physical); T_top ~ 351.4 K (the azeotropic temperature); mass closes 100%.

NOTE: asking for a distillate ABOVE the azeotrope (x_eth > 0.894) is physically impossible – no method converges, and that is correct. The MESH method does not invent a non-physical "past the azeotrope" answer (which the bubble-point used to do).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0208333333333
stream	distillate	F	0.00694444444444
stream	feed	F	0.0277777777778
kpi	column	B	0.0208333333333
kpi	column	D	0.00694444444444
kpi	column	R	2.5
kpi	column	T_bottom	357.713408957
kpi	column	T_top	351.388958309
kpi	column	feedStage	8
kpi	column	nStages	16
kpi	column	x_B_HK	0.868205515093
kpi	column	x_D_LK	0.804616545279
utility	column	heating.steamLP.duty_kW	935.650111764
utility	column	heating.steamLP.T	357.713408957

... and 13 more reference values in the case's `expected`.

`column04_multifeed_sidedraw`

`tutorials/steady/distillation/column04_multifeed_sidedraw`

What it does: Simultaneous MESH column: two feeds + a liquid side draw (rigorous MESH)

The story: Multiple feeds as FLOWSHEET STREAMS (‘inputs (feed feed2)‘) + a liquid side draw. operation.feeds maps each input stream to a stage (information follows the streams → the plant-boundary mass + energy balance closes).

The rigorous simultaneous MESH column extended to multiple feeds and side draws — the most-used features of a commercial simulator column, on the glass-box Newton solver.

A benzene / toluene / o-xylene split (light / intermediate / heavy):

- primary feed — 100 kmol/h, benzene/toluene/o-xylene 0.40/0.40/0.20, enters high (stage 5); - second feed — 60 kmol/h, heavier (0.05/0.45/0.50), enters low (stage 11); - liquid side draw — 15 kmol/h pulled from stage 8, between the two feeds.

Mass closes (100 + 60 = 42 + 15 + 103 = 160 kmol/h). Q_condenser = -1436 kW, Q_reboiler = +1527 kW. The side draw does exactly what it should — bleed the intermediate component where it is most enriched on the trays.

The simultaneous MESH is a bubble-point-reduced Newton (CMO flows). The extension generalises the single fixed feed into a per-stage flow staircase: L[j], V[j] are rebuilt by a CMO march that adds each feed's liquid/vapour parts and subtracts each draw; every stage's material balance carries an F_j z_{F,j} source and (L_j+U_j), (V_j+W_j) outflows. The general balance reduces exactly to the old single-feed equations when there is one feed and no draw — so every existing distillation tutorial is unchanged (verified).

outputs (distillate bottoms <draw1> <draw2> ...) — side-draw streams follow distillate and bottoms, in stage order.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.0286111111111`
`stream`	`distillate`	`F`	`0.0116666666667`
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`intermediateCut`	`F`	`0.00416666666667`
`kpi`	`radfracLite`	`B`	`0.0286111111111`
`kpi`	`radfracLite`	`D`	`0.0116666666667`
`kpi`	`radfracLite`	`Q_condenser_kW`	`-1435.80274013`
`kpi`	`radfracLite`	`Q_reboiler_kW`	`1516.14164522`
`kpi`	`radfracLite`	`R`	`3`
`kpi`	`radfracLite`	`T_bottom`	`395.926996928`
`kpi`	`radfracLite`	`T_top`	`354.978489481`
`kpi`	`radfracLite`	`feedStage`	`5`
`kpi`	`radfracLite`	`nFeeds`	`2`
`kpi`	`radfracLite`	`nSideDraws`	`1`

. . . and 17 more reference values in the case's `expected`.

column05_reactive_methylacetate

tutorials/steady/distillation/column05_reactive_methylacetate

What it does: Reactive distillation: methyl acetate synthesis (validation vs P?pken et al. 2001, run S-1)

The story: Both feeds are FLOWSHEET STREAMS ('inputs (feed acidFeed)') – so the plant-boundary mass + energy balance sees them. 'operation.feeds' only maps each stream to a stage (no inline composition – the credo: information follows the streams).

The simultaneous MESH column with an equilibrium reaction on the catalytic stages (rigorous column rung 4), compared to a published experiment.

Mole-conserving, so the CMO internal-flow profile is untouched. Each catalytic stage carries one extra unknown — the molar extent ?_j — closed by the activity-based equilibrium residual Σ_i ν_i ln(γ_i x_i) = ln K_a(T), with γ_i from the liquid activity model and K_a(T) by van't Hoff: K_a(T) = K_a(298)·exp(-Δh°_r/R·(1/T - 1/298.15)). Converged by homotopy (solve non-reactive, then switch the reaction on).

> T. P?pken, S. Steinigeweg & J. Gmehling, ∗"Synthesis and Hydrolysis of > Methyl Acetate by Reactive Distillation\{}ldots"∗, Ind. Eng. Chem. Res. 40 > (2001) 1566. Run S-1: N = 25, reactive stages 11–19, HOAc fed above / > MeOH below the zone, reflux 2.1, P $\approx$ 1.017 bar. Measured endpoints, cited, in > constant/experimental/methyl_acetate_RD_profile.csv.

The equilibrium constant and thermodynamics are from the companion paper P?pken, G?tze & Gmehling, Ind. Eng. Chem. Res. 39 (2000) 2601: K_a(298) $\approx$ 38.7, Δh°_r $\approx$ -5.67 kJ/mol $\rightarrow$ K_a $\approx$ 29 in the reactive zone (~65 °C).

This is not a tight match, and the reason is physically important: with the correct equilibrium K_a ($\approx$29) the model gives the equilibrium ceiling (96 %), and it correctly brackets the measurement from above. The measured 87.6 % is ∗below∗ the equilibrium because the real column is kinetically limited — the paper's recommended model is an adsorption-based finite ∗rate∗ (per gram of catalyst), not equilibrium. An earlier run with an (incorrect, too low) K_a = 5.2 gave 90.7 %, a closer ∗number∗ but for the wrong reason; using a wrong constant to fake agreement would be dishonest.

So the path to a bit-tight match is the kinetic rate model, not the thermo.

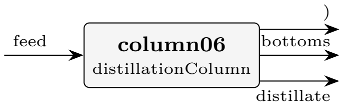
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	9.13888888889e-06
stream	distillate	F	8.36111111111e-06
stream	feed	F	8.94444444444e-06
kpi	rdColumn	B	9.13888888889e-06
kpi	rdColumn	D	8.36111111111e-06
kpi	rdColumn	Q_condenser_kW	-0.790521316742
kpi	rdColumn	Q_reboiler_kW	0.626813788646
kpi	rdColumn	R	2.1
kpi	rdColumn	T_bottom	363.229492539
kpi	rdColumn	T_top	330.062558898
kpi	rdColumn	conversion	0.960508806863
kpi	rdColumn	feedStage	20
kpi	rdColumn	nFeeds	2
kpi	rdColumn	nSideDraws	0

. . . and 17 more reference values in the case's `expected`.

column06_fullMESH

`tutorials/steady/distillation/column06_fullMESH`



What it does: Full-MESH (Naphtali-Sandholm) distillation – energy-consistent V_j/L_j, seeded from the CMO solution (benzene/toluene; compare column02)

The story: The SAME benzene/toluene column as column02_simultaneous, but solved by the FULL MESH (Naphtali-Sandholm), selected by ‘model fullMESH;‘. The full MESH promotes the internal flows V_j, L_j to UNKNOWNS and adds a per-stage total-mass + ENERGY balance, so the flows follow the REAL latent heats instead of the constant-molal-overflow (CMO) assumption behind the ‘simultaneous‘ (bubble-point-reduced) method.

Robustness trick: the full MESH is SEEDED from the converged CMO profile — the cheap model bootstraps the rigorous one; ~4 Newton iterations from that seed.

Expected — close to, but NOT equal to, column02 (the gap IS the energy-balance correction the CMO drops): T_top ~ 354.3 K, T_bottom ~ 382.8 K, x_D(benzene) ~ 0.979 (CMO: 0.981). Benzene/toluene latent heats are similar (30.8 vs 33.2 kJ/mol) so the gap is small here; on a wide-boiling column the full MESH earns its cost.

The same benzene/toluene column as column02_simultaneous, but solved by the full MESH (model fullMESH) instead of the bubble-point/CMO reduction.

The simultaneous method is a bubble-point-reduced MESH: it assumes constant molal overflow (CMO — equimolal latent heats), so the internal vapour/liquid flows V_j, L_j are fixed by a mass march and only the compositions x_j and stage temperatures T_j are solved.

*The full MESH (Naphtali–Sandholm) promotes V_j and L_j to unknowns and adds, per stage, the total-mass balance and the energy balance — so the flows follow the *real* latent heats instead of assuming them equal. The extra per-stage energy equation is the enthalpy residual the CMO never carried:*

Total condenser: V_0 = 0, L_0 = R·D; reboiler: L_{N-1} = B; their energy balances give Q_cond / Q_reb as results.

The full-MESH is seeded from the converged CMO profile — the cheap model launches the rigorous one. Here it converges in ~4 Newton iterations from that seed (it would be far harder from a cold start).

Benzene (Δh_vap ≈ 30.8 kJ/mol) and toluene (≈ 33.2 kJ/mol) have similar latent heats, so CMO is a good approximation and the two answers nearly coincide — the ~0.002 / ~0.1 K gap IS the energy-balance correction the full-MESH makes and the CMO drops. On a wide-boiling column (a depropaniser) the gap is much larger; that is where the full-MESH earns its cost.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.0138888888889`
`stream`	`distillate`	`F`	`0.0138888888889`
`stream`	`feed`	`F`	`0.0277777777778`
`kpi`	`column06`	`B`	`0.0138888888889`
`kpi`	`column06`	`D`	`0.0138888888889`
`kpi`	`column06`	`Q_condenser_kW`	`-1281.19539111`
`kpi`	`column06`	`Q_reboiler_kW`	`1299.72437334`
`kpi`	`column06`	`R`	`2`
`kpi`	`column06`	`T_bottom`	`382.774927808`
`kpi`	`column06`	`T_top`	`354.339103085`
`kpi`	`column06`	`feedStage`	`8`
`kpi`	`column06`	`nFeeds`	`1`
`kpi`	`column06`	`nSideDraws`	`0`
`kpi`	`column06`	`nStages`	`15`

. . . and 13 more reference values in the case’s `expected`.

`column07_naphtaliSandholm`

`tutorials/steady/distillation/column07_naphtaliSandholm`

What it does: Full-MESH (Naphtali-Sandholm) validation vs AIChE J. 17 (1971) 148, Problem 2: nC4/iC5/nC5

Reproduces the ideal-hydrocarbon distillation of Naphtali & Sandholm, "Multicomponent Separation Calculations by Linearization", AIChE Journal 17(1), 148–153 (1971) — DOI [10.1002/aic.690170130](https://doi.org/10.1002/aic. — the paper that introduced the very full-MESH method Choupo's model fullMESH implements.

n-butane / isopentane / n-pentane, 20 plates, liquid feed (T = 30 °F = 272.04 K, 500 mol of each) on plate 10, reflux ratio 1000, 80 % of the feed recovered in the bottoms (B = 1200). A liquid sidestream on plate 1 (the condenser) takes 30 % of the reflux.

Equivalent spec used here. The distillate (≈1) and the condenser sidestream (≈299) leave the ∗total∗ condenser with the same composition, so they are combined into ONE top product of 300 with reflux $R = L_1/300 = 996.68/300 = 3.3223$*. This gives identical internal flows (*$L_1 = 996.68$*,* $V_2 = 1296.68$*), hence the identical column — without needing a condenser side draw.*

Converges in 3 full-MESH Newton iterations (seeded from the CMO profile).

Products Choupo —— top product (300) 99.99 % n-C_4 *bottoms n-*C_4 */ i-*C_5 */ n-*C_5 *200 / 500 / 500 B 1200*

The profile shape — the flat n-butane-rich top, the sharp rise across the feed zone, the pentane-plateau, the reboiler climb — matches stage-by-stage. The ~1–2 K residual is thermo, not method: Choupo uses fitted Antoine vapour pressures, the paper used the K-value tables in Brown's ∗Unit Operations∗ (1950).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.416666666667`
`kpi`	`column07`	`B`	`0.333333333333`
`kpi`	`column07`	`D`	`0.0833333333333`
`kpi`	`column07`	`Q_condenser_kW`	`-8082.6120755`
`kpi`	`column07`	`Q_reboiler_kW`	`9374.37429576`
`kpi`	`column07`	`R`	`3.3223`
`kpi`	`column07`	`T_bottom`	`296.58946842`
`kpi`	`column07`	`T_top`	`272.663334648`
`kpi`	`column07`	`feedStage`	`10`
`kpi`	`column07`	`nFeeds`	`1`
`kpi`	`column07`	`nSideDraws`	`0`
`kpi`	`column07`	`nStages`	`20`
`kpi`	`column07`	`x_B_HK`	`0.416665691626`
`kpi`	`column07`	`x_D_LK`	`0.999874145869`

. . . and 10 more reference values in the case's `expected`*.*

column08_radfrac_multidraw

`tutorials/steady/distillation/column08_radfrac_multidraw`

What it does: rigorous column: multi-feed + multi-side-draw distillation (C3/C4/C5/C6), full-MESH

A C3/C4/C5/C6 depropanizer-style column at 16 bar, split four ways by the rigorous full-MESH, exercising the two most-used rigorous column features at once: TWO feeds (a light cut high, a heavy cut low) and TWO liquid side draws.

Two saturated-liquid feeds (100 kmol/h each, 290 K / 320 K): a light cut on stage 10, a heavy cut on stage 20. 30 stages, R = 4. Mass closes exactly (200 = 45 + 55 + 55 + 45 kmol/h) and the energy balance closes: the feed-to-product enthalpy gap (1137.7 kW) equals the net duty $Q_{\{}text{reb}}+Q_{\{}text{cond}} = 1942.3 - 804.6$ kW. Converges in a few full-MESH iterations from the CMO seed.

This case was first written at 1 atm, and the energy balance would not close — because at 1 atm and 290 K the light hydrocarbons (propane b.p. 231 K, n-butane 272 K) are above their boiling points: the feed streams flash to VAPOUR (vf = 1.0), yet the column was told quality 1.0 (saturated liquid). The latent heat of the unrecognised vapour feed was exactly the missing ~1 MW. At 16 bar the feeds are genuinely liquid (vf = 0, consistent with quality 1.0) and the balance closes — which is also why a real depropanizer runs at elevated pressure: so the propane distillate condenses against cooling water (here 321.6 K). A feed's thermal state must be specified consistently with its actual phase.

Both feeds are real flowsheet streams (inputs (feed1 feed2), mapped to stages by operation.feeds), so the plant-boundary mass + energy balance sees them (∗information follows the streams∗). Side draws are outputs in stage order: outputs (distillate bottoms drawC4 drawC5).

Self-consistently validated (mass + energy close, four cuts in volatility order) on the full-MESH solver that IS paper-validated against Naphtali & Sandholm (1971) in column07_naphtaliSandholm. No external multi-feed+multi-draw single-column benchmark with published profiles is readily available in the open literature.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0125
stream	distillate	F	0.0125
stream	drawC4	F	0.0152777777778
stream	drawC5	F	0.0152777777778
stream	feed1	F	0.0277777777778
stream	feed2	F	0.0277777777778
kpi	radfrac	B	0.0125
kpi	radfrac	D	0.0125
kpi	radfrac	Q_condenser_kW	-804.58622364
kpi	radfrac	Q_reboiler_kW	1698.71150765
kpi	radfrac	R	4
kpi	radfrac	T_bottom	454.498489397
kpi	radfrac	T_top	321.647346922
kpi	radfrac	feedStage	10

. . . and 19 more reference values in the case's `expected`*.*

column09_tray_hydraulics

tutorials/steady/distillation/column09_tray_hydraulics

What it does: Sieve-tray hydraulics: the column that separates, sized so its trays do not flood

The story: column01, asked the question a converged MESH solution never answers: CAN THE TRAYS PASS THE TRAFFIC?

A converged column tells you what separates. It says nothing about whether the vapour rising through the holes will carry the liquid up with it (entrainment flooding), whether the liquid will back up the downcomer until it reaches the tray above (downcomer flooding), or whether it will simply rain through the holes instead of bubbling (weeping). A column that floods separates nothing, and the stage equations never notice.

Add an ‘hydraulics {}‘ block and the column runs a RATING pass on its own converged profile. Two modes, the same equations in opposite directions:

give ‘diameter‘ → RATING. How close to flood is each tray? omit ‘diameter‘ → DESIGN. What diameter keeps every tray at or below ‘floodFraction‘ of its flooding velocity? The widest tray sets the column. That is this case.

Nothing here is guessed. The orifice coefficient C_o is a chart in the literature (Liebson, Kelley & Bullington 1957, against hole-area ratio and plate-thickness/hole-diameter); it is a DECLARED number you own, printed back at you. The weep-point constant K2 is a chart too – leave it out and the column tells you the weep check was not performed rather than inventing it.

Read the printed table. The flow parameter F_LV climbs below the feed (more liquid, less vapour per unit mass), which lowers Fair’s capacity parameter and therefore the flooding velocity – yet the stripping trays are not the tight ones here, because the vapour volumetric flow falls faster than the capacity. The column is sized by whichever tray loses that argument.

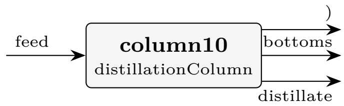
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.0138888888889`
`stream`	`distillate`	`F`	`0.0138888888889`
`stream`	`feed`	`F`	`0.0277777777778`
`kpi`	`column09`	`B`	`0.0138888888889`
`kpi`	`column09`	`D`	`0.0138888888889`
`kpi`	`column09`	`L_rect`	`0.0277777777778`
`kpi`	`column09`	`L_strip`	`0.0555555555556`
`kpi`	`column09`	`Q_condenser_kW`	`-1281.04348002`
`kpi`	`column09`	`Q_reboiler_kW`	`1279.30480005`
`kpi`	`column09`	`R`	`2`
`kpi`	`column09`	`T_bottom`	`382.893530951`
`kpi`	`column09`	`T_top`	`354.21089412`
`kpi`	`column09`	`V_rect`	`0.0416666666667`
`kpi`	`column09`	`V_strip`	`0.0416666666667`

. . . and 22 more reference values in the case’s `expected`.

column10_flooding

tutorials/steady/distillation/column10_flooding



What it does: The same column, built too narrow: every tray floods – and it still converges

The story: the SAME column as column09, built 1.10 m wide instead of the 1.316 m that column09 designed for it.

Nothing about the separation changes. The distillate is still 98.12 % benzene, the bottoms still 98.12 % toluene, the MESH equations converge exactly as before, and every number in the stream table is identical. The column is a perfectly good column – on paper.

It floods. The worst tray runs at 114.5 % of its entrainment-flooding velocity: the vapour is fast enough to carry the liquid up to the tray above instead of letting it fall. In a real column the trays would fill, the pressure drop would run away, and the separation printed above would not happen at all.

This is the whole reason hydraulics exists as a separate pass. A converged solution is not a feasible column, and the stage equations have no way of knowing the difference. Widen it (column09), space the trays further apart, or cut the reflux – but do not believe the stream table until a tray can pass the traffic it carries.

Note the downcomer, too: the backup reaches 268.5 mm against a 275 mm limit. Narrow the column a little further and it would flood from below as well, for an entirely different reason.

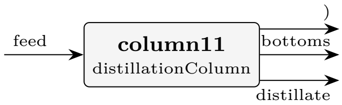
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0138888888889
stream	distillate	F	0.0138888888889
stream	feed	F	0.0277777777778
kpi	column10	B	0.0138888888889
kpi	column10	D	0.0138888888889
kpi	column10	L_rect	0.0277777777778
kpi	column10	L_strip	0.0555555555556
kpi	column10	Q_condenser_kW	-1281.04348002
kpi	column10	Q_reboiler_kW	1279.30480005
kpi	column10	R	2
kpi	column10	T_bottom	382.893530951
kpi	column10	T_top	354.21089412
kpi	column10	V_rect	0.0416666666667
kpi	column10	V_strip	0.0416666666667

... and 22 more reference values in the case's `expected`.

column11_murphree

tutorials/steady/distillation/column11_murphree



What it does: Murphree tray efficiency: 14 REAL trays plus the reboiler (nStages 15, reboiler counted), not ideal stages – and the purity you actually get

The story: the same 15-stage benzene/toluene column as column09, with ONE line added: murphreeEfficiency 0.70.

Every column before this one is built from EQUILIBRIUM stages: the vapour leaving a tray is in equilibrium with the liquid leaving it. No real tray does that. The vapour spends a finite time in the froth and comes off somewhere between what entered it and what equilibrium would give. Murphree measured the shortfall on the vapour side:

y_j = E_MV . y∗_j + (1 - E_MV) . y_{j+1}, y∗_j = K_j x_j

With E_MV = 1 the tray is an ideal stage and nothing changes – every existing case still runs byte for byte. With E_MV < 1, 'nStages' stops counting ideal stages and starts counting REAL TRAYS. Which matters, because the hydraulics pass below sizes those trays: until now we were dimensioning, to the millimetre, a tray we then assumed was perfect.

E_MV is a DECLARED number. It comes from a correlation (O'Connell), a chart, or plant data, and it is yours to own – exactly like the orifice coefficient C_o and the weep constant K2 in the hydraulics block. Choupo will not invent it.

Compare against column09 (same column, E_MV = 1):

distillate benzene 98.12 % (ideal) → 95.05 % (E_MV = 0.70) bottoms toluene 98.12 % (ideal) → 95.05 %

Fourteen real trays do less than fourteen ideal ones. That is the whole content of a tray efficiency, and it is why a column designed on ideal stages and built on real trays comes up short. Sweep E_MV and watch the limit close:

E_MV 0.50 0.70 0.90 0.99 0.999 1.000 x_D 0.9100 0.9505 0.9738 0.9806 0.9812 0.98125 <- the ideal stage

Nothing is switched on at E_MV = 1; the effective K collapses to K and the case reproduces column09 to the last bit.

NOTE: the efficiency is folded into an effective K, with the vapour from the tray below lagged one outer pass – the linear solve keeps its tridiagonal shape. At convergence the lag vanishes and the Murphree relation holds exactly. The MESH 'simultaneous' method REFUSES the keyword rather than quietly returning ideal stages: there the relation would have to enter the residual and Jacobian.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0138888888889
stream	distillate	F	0.0138888888889
stream	feed	F	0.0277777777778
kpi	column11	B	0.0138888888889
kpi	column11	D	0.0138888888889
kpi	column11	L_rect	0.0277777777778
kpi	column11	L_strip	0.0555555555556
kpi	column11	Q_condenser_kW	-1284.29753602
kpi	column11	Q_reboiler_kW	1281.01358292
kpi	column11	R	2
kpi	column11	T_bottom	381.492133108
kpi	column11	T_top	355.136328304
kpi	column11	V_rect	0.0416666666667
kpi	column11	V_strip	0.0416666666667
... and 23 more reference values in the case's `expected`*.*			

column12_stage_is_a_flash

tutorials/steady/distillation/column12_stage_is_a_flash

What it does: An equilibrium stage IS a flash: a tray's own two products, recombined and re-flashed, must come back as the same tray

The story: AN EQUILIBRIUM STAGE IS A FLASH – the identity witness.

A column's MESH equations and a flash drum's Rachford-Rice are two DIFFERENT pieces of code that claim to compute the same thing: the equilibrium split of a mixture at (T, P). Nothing in the corpus made them say it to each other. This case does.

feed -→ [tower] -→ distillate -→ bottoms -→ stageLiquid (the liquid LEAVING stage 5) -→ stageVapour (the vapour LEAVING stage 5)

stageLiquid + stageVapour -→ [recombine] -→ stageMix stageMix -→ [restage] -→ checkLiquid, checkVapour

THE CLAIM. Stage 5 is an equilibrium stage: by definition the vapour leaving it is in equilibrium with the liquid leaving it. So if the two are put back together and the mixture is allowed to separate again, at the same pressure and with no heat added or removed, NOTHING must happen: the same temperature, the same two compositions, and a vapour fraction equal to the ratio of the two draws.

T(checkLiquid) == T(stageLiquid) == T(stageVapour) z(checkLiquid) == z(stageLiquid) z(checkVapour) == z(stageVapour) F(checkVapour) / (F(checkLiquid) + F(checkVapour)) == W / (U + W)

Nothing above is a number anyone typed. Every one of them is produced by the run, on both sides, and the gate ‘check_stage_identity‘ compares them. That is what makes this a witness rather than a demonstration.

WHY IT IS WORTH A CASE. The column asks the thermodynamic package for K-values per stage; the flash asks it for an equilibrium split. Those are different entry points and, for a package that resolves CHEMISTRY, they are different code. The effective K-value a reacting stage uses (‘ThermoPackage::stageK‘, added 2026-08-04 for the sour-water column) is exactly the sort of definition that can be subtly wrong – wrong normalisation, wrong basis, the reaction counted twice – and still produce a column that converges to a plausible-looking profile. This identity is what catches it. The molecular case here is the CONTROL: it must hold on a system where nobody doubts the answer, before the same construction is trusted on one where they should.

HOW THE RECOMBINATION WORKS, and the one thing to understand about it. ‘recombine‘ is a plain mixer, and a Choupo mixer MIXES – it does not do the vapour/liquid split ("mixers mix, flashes separate"). Handed a liquid and a vapour it conserves total enthalpy exactly and reports the result as a single dominant phase at whatever temperature carries that enthalpy. That temperature is FICTITIOUS – a nominal vapour read out at 182.9 K, far below its saturation – and the case does not use it for anything. What the mixer is doing here is carrying an ENTHALPY across one arrow, faithfully; ‘restage‘ is an ADIABATIC flash, so enthalpy is precisely the quantity it reads back, and the fictitious temperature cancels out of the answer. The identity is therefore an ENERGY statement as well as an equilibrium one: it would fail if the column and the flash disagreed about the enthalpy of either phase.

WHY THE DRAWS ARE ON ONE STAGE AND NOT TWO. Both draws come off stage 5. A liquid draw from stage 5 and a vapour draw from stage 6 would also recombine into something – and it would not be an equilibrium mixture, so the flash would move it. The point of the case is that when the two phases genuinely leave the SAME stage, the flash cannot move them at all.

What it demonstrates: that Choupo's two independent equilibrium routines — the column's MESH equations and the flash drum's Rachford-Rice — give the same answer, by making them say it to each other inside one run.

Stage 5 is an equilibrium stage, so the vapour leaving it is in equilibrium with the liquid leaving it. Put them back together, let them separate again at the same pressure with no heat added, and nothing may happen. The run must produce

Not one of those numbers is typed into the case. Both sides are computed, by different code, and bin/curate/check_stage_identity.py compares them.

Three things at once, and each of them has been wrong in a simulator somewhere:

1. The K-value definition. The column asks the package for K-values per stage; the flash asks it for a split. A wrong normalisation or a wrong basis in one of them shifts the profile without ever making the column fail to converge. 2. The enthalpy of each phase. The recombination carries an ∗enthalpy∗ across one arrow and the adiabatic flash reads it back. If the column and the flash priced the vapour differently, the recovered temperature would be wrong even with perfect K-values. 3. That a two-phase inlet is priced as two phases. Building this case is what found adiabaticFlash pricing every inlet as a sub-cooled liquid regardless of its vapour fraction — silently, since no case in the corpus had ever fed it one.

The identity holds to about 1e-7 relative, not to machine precision, and the reason is visible in the log: the adiabatic flash stops its outer Newton at an energy residual of ~2.5e-4 J/mol out of 5.4e+4 J/mol. That is the limit of the ∗check∗, not of the physics. The gate is set at 1e-6 relative, which is a factor of ten of headroom over the observed agreement.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.00833333333333`
`stream`	`checkLiquid`	`F`	`0.00277777761205`
`stream`	`checkVapour`	`F`	`0.00277777794351`
`stream`	`distillate`	`F`	`0.0138888888889`
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`stageLiquid`	`F`	`0.00277777777778`
`stream`	`stageMix`	`F`	`0.00555555555556`
`stream`	`stageVapour`	`F`	`0.00277777777778`
`kpi`	`recombine`	`F_out`	`0.00555555555556`
`kpi`	`recombine`	`P_out`	`101325`
`kpi`	`recombine`	`T_out`	`182.884624593`
`kpi`	`recombine`	`n_in`	`2`
`kpi`	`recombine`	`vf`	`1`
`kpi`	`restage`	`F_in`	`0.00555555555556`
. . . and 37 more reference values in the case's `expected`.			

column13_sour_water_stage_identity

tutorials/steady/distillation/column13_sour_water_stage_identity

What it does: The stage identity under CHEMISTRY: a sour-water tray's own two products, recombined and re-flashed, must come back as the same tray

The story: THE STAGE IDENTITY, UNDER CHEMISTRY.

This is column12 (‘column12_stage_is_a_flash‘) with one thing changed: the thermodynamic package resolves an aqueous EQUILIBRIUM. Read that case first – the construction, the mixer's fictitious temperature and the tolerance are all explained there and not repeated here.

feed -→ [tower] -→ distillate, bottoms -→ stageLiquid, stageVapour (stage 2) stageLiquid + stageVapour -→ [recombine] -→ stageMix stageMix -→ [restage] -→ checkLiquid, checkVapour

WHY IT IS A DIFFERENT TEST FROM column12, and not a copy.

In a molecular world a stage's K-values and a flash's split come from the same handful of lines: ‘ThermoPackage::stageK‘ forwards straight to ‘Kvec‘, and the identity, while worth having, is checking arithmetic that was never in doubt.

Here it is not the same code. A package that resolves chemistry has no K-value to hand out: the volatility of NH3 is set by how much of it is NH4+, which depends on the pH, which depends on how much CO2 has left. ‘stageK‘ therefore solves the stage's own equilibrium at its own (T, P, z) and returns an effective APPARENT K – a definition, made by us, which nothing outside this case checks. A wrong normalisation, a basis confusion between apparent components and species, or the speciation counted twice would all produce a column that converges to a plausible profile. This case is what refuses it.

AND THE DEFINITION WAS WRONG ONCE, WHICH IS THE ARGUMENT FOR THE CASE. The first version read K = y/x off the flash. That is right whenever the stage is two-phase, and it hands back a column of ZEROS the moment the trial state is subsaturated – which a column solver's trial states usually are, on the way to the answer. Every bubble-point residual then reads -1, the Jacobian is singular in T, and the MESH stops without moving: "Newton iters: 0". A K-value is an INCIPIENT quantity, defined over a liquid whether or not a vapour happens to be there, so the subsaturated branch now uses the equilibrium partial pressures over the fully speciated liquid, K_i = (p_i/P)/x_i. The two branches agree where they meet, because at a two-phase state the partials sum to P.

WHAT MUST HOLD, all of it produced by the run and none of it authored:

T(checkLiquid) == T(stageLiquid) == T(stageVapour) z(checkLiquid) == z(stageLiquid) (apparent component basis) z(checkVapour) == z(stageVapour) F(checkVapour)/(F(checkLiquid)+F(checkVapour)) == W/(U+W)

and, in addition to column12, the thing only a reacting stage can be asked: the two liquids must carry the SAME SPECIATION. The column's stage liquid and the re-flashed liquid are the same material at the same state, so their ion inventories must agree species by species – not merely their apparent compositions. That is the check that would catch an effective K which happens to reproduce the apparent split while resting on a different chemistry underneath.

THE COST, said out loud. Every stage K-value evaluation is a full reactive flash, and the MESH Jacobian is numerical, so one Newton pass costs of the order of (stages x unknowns) reactive flashes. The column is FOUR stages for that reason and no other: this is an identity witness, not a design study. A student who wants a real sour-water stripper wants the S2 case, which does not exist yet.

WHAT THE COLUMN ACTUALLY DOES, since it is worth reading the profile. It converges in 7 Newton iterations to |F| ~ 8e-10, and the answer is the physics: NH3 climbs from 0.0054 in the reboiler liquid to 0.30 in the overhead vapour while CO2 strips out ahead of it, and T falls from 371.5 K at the bottom to 354.6 K at the top. No gammaPhi column can produce that profile, because in a molecular world ammonia's volatility does not depend on how much carbon dioxide has left the liquid.

What it demonstrates: a distillation column solved under an aqueous equilibrium, and the proof that the effective K-value it uses per stage is the same equilibrium a flash computes — checked through two independent code paths, on the apparent components ∗and∗ on the ions.

Read column12_stage_is_a_flash first. This is the same construction; only the thermodynamic package differs.

Four stages, sour water (NH_3 + CO_2 in water), 1 atm, R = 2, D = 3 kmol/h. It converges in 7 Newton iterations to ?F? ≈ 8e-10, and the profile is the physics:

No gammaPhi column can produce that profile, because in a molecular world ammonia's volatility does not depend on how much carbon dioxide has left the liquid. Here it does: the carbon dioxide a liquid still carries is what holds its ammonia as NH_4?, so stripping the CO_2 sets the ammonia free and the free ammonia then strips. (The usual shorthand for that is "stripping CO_2 raises the pH" — which is true of a fixed solution and is ∗not∗ how it reads tray by tray here. See the per-tray table below: it is the carbonate loading that orders cleanly, not the pH.)

Both halves of stage 2 are drawn as real streams, recombined, and re-flashed at the same (T, P). Nothing may happen — and nothing does, to about 1e-9 relative:

The ion inventories agreeing is the claim that a comparison of apparent compositions alone would miss: an effective K could reproduce the apparent split while resting on a different chemistry underneath.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0259166666667
stream	checkLiquid	F	0.00041666665957
stream	checkVapour	F	0.000138888895986
stream	distillate	F	0.000833333333333
stream	feed	F	0.0273055555556
stream	stageLiquid	F	0.000416666666667
stream	stageMix	F	0.000555555555556
stream	stageVapour	F	0.000138888888889
kpi	recombine	F_out	0.000555555555556
kpi	recombine	P_out	101325
kpi	recombine	T_out	483.325341357
kpi	recombine	n_in	2
kpi	recombine	vf	0
kpi	restage	F_in	0.000555555555556

. . . and 37 more reference values in the case's `expected`.

column14_klemola_c4_splitter

tutorials/steady/distillation/column14_klemola_c4_splitter

What it does: Klemola-Ilme industrial C4 splitter: 74 real trays at the measured E_MV = 1.191, products anchored on the plant's own key-component analysis

The story: an INDUSTRIAL i-butane/n-butane fractionator, 74 real trays, 2.9 m diameter, run on its own measured specification.

PRIMARY, and the only source used for any number below:

Klemola, K. T.; Ilme, J. K. Distillation Efficiencies of an Industrial-Scale i-Butane/n-Butane Fractionator. Ind. Eng. Chem. Res. 1996, 35 (12), 4579-4586. IE960390R.

The transcription record – Tables 1 and 2 number by number, the efficiency results, and the four things the paper does NOT publish – is docs/design/klemola-c4-splitter-reference-case.md. No number in this case is absent from that record; where the record says the paper is silent, this header DECLARES an assumption and says it is one.

WHY THIS CASE EXISTS. The corpus's other distillation tutorials are pinned on self-recorded goldens; the closest thing to an external anchor (acetone06/07) compares Choupo against another SIMULATION. This is the first column anchored on a measured plant. What it puts under test is the COLUMN MODEL – the MESH solve, the declared tray efficiency, the material balance – not an activity model: the system is nearly ideal, so a disagreement here cannot be blamed on thermodynamics and then forgotten.

1. WHAT IS TRANSCRIBED, PER NUMBER (record Sections 2 and 3, Tables 1 / 2)

number of trays 74 Table 1 → nStages 75 (see 2) feed tray 37 Table 2 → feedStage 37 (see 2) column top pressure 658.6 kPa Table 2 → P (see A2) feed pressure 892.67 kPa Table 2 → used in A1 only feed flow rate 26234 kg/h Table 2 → 0/feed (see 3) distillate flow rate 8115 kg/h Table 2 → distillateRate (see 4) reflux flow rate 92838 kg/h Table 2 → refluxRatio (see 5) bottoms flow rate 18119 kg/h Table 2 → NOT an input (see 6) Murphree tray efficiency 119.1 % text p.4580 → murphreeEfficiency reflux temperature 18.5 C Table 2 → NOT USED (see 7) boiler duty 10.240 MW Table 2 → NOT an input (see 8)

Table 1's tray geometry (2.900 m diameter, 0.600 m spacing, 4.9 m2 active area, 772 valves, 18.82 % free hole area, ...) is transcribed in the record for the reader and is used by NOTHING here: no hydraulics block is declared, because the anchor is the separation, not the rating. Ballast V-1 valve trays are not the sieve tray Choupo's hydraulics pass models, and rating them with a sieve correlation would be a number with no source behind it.

2. STAGE COUNT AND FEED STAGE – and one AMBIGUITY, declared

Choupo's Wang-Henke cascade numbers stage 1 = TOP TRAY and stage N = REBOILER; the total condenser sits outside the stage list (the distillate is y_1, the vapour leaving the top tray, condensed). So 74 real trays plus the reboiler gives

nStages 75 (= 74 trays + 1 reboiler) feedStage 37 (Table 2, taken as counted from the TOP)

ASSUMPTION A0 – the paper prints "feed tray 37" and does not state the direction of its tray numbering; the record does not either, and the primary text was checked. Counted from the bottom instead, the feed would sit on tray 38 of 74 as Choupo numbers them: 37 vs 38 rectifying trays out of 74. Stage 1 = top is taken because it is Choupo's own convention and because the difference is one tray in the middle of a 74-tray column. Anyone who establishes the paper's direction should change this line and say so.

3. THE FEED – wt % to mole flows, arithmetic shown

Table 2 gives the feed as 26234 kg/h at these wt % (the paper's own units and its own rounding to 0.1 wt %). THE COLUMN SUMS TO 100.1, NOT 100 – that is the rounding, and it has to be resolved before anything can be converted. It is resolved by NORMALISING the wt % to unity and applying them to the measured 26234 kg/h, so that the total mass flow – which the paper's own overall balance closes exactly (18119 + 8115 = 26234) – is preserved. The alternative (applying the raw wt % literally) would invent 26 kg/h of feed.

MW from each component's own data/standards record. kmol/h = kg/h / MW.

component wt% wt% norm kg/h MW kmol/h x

propane 1.5 1.49850 393.1169 44.0960 8.915024 0.0196925 isoButane 29.4 29.37063 7705.0909 58.1222 132.567090 0.2928294 nButane 67.7 67.63237 17742.6753 58.1230 305.260832 0.6742952 1Butene 0.2 0.19980 52.4156 56.1063 0.934219 0.0020636 isoButene 0.2 0.19980 52.4156 56.1063 0.934219 0.0020636 trans2Butene 0.1 0.09990 26.2078 56.1063 0.467110 0.0010318 neopentane 0.1 0.09990 26.2078 72.1488 0.363246 0.0008024 isopentane 0.8 0.79920 209.6623 72.1488 2.905971 0.0064190 nPentane 0.1 0.09990 26.2078 72.1488 0.363246 0.0008024

TOTAL 100.1 100.00000 26234.0000 452.710957 1.0000000

mean feed MW = 26234 / 452.710957 = 57.94867 kg/kmol

Those kmol/h are what 0/feed carries.

ASSUMPTION A1 – THE FEED THERMAL STATE. Table 2 gives the feed PRESSURE (892.67 kPa) and no feed temperature; record Section 5.2 names this gap and names "saturated liquid at the feed pressure" as the natural assumption. That is what is assumed here, and it has a consequence the case must carry rather than hide: a liquid saturated at 892.67 kPa is NOT saturated on a tray at 658.6 kPa – it flashes on the way in. Both steps were computed with THIS CASE'S OWN thermo (the dict in constant/, no other model), by two throwaway runs whose numbers are reproduced here so a reader can repeat them:

bubbleT of the feed composition at 892.67 kPa → T = 334.4218 K adiabaticFlash of that stream to 658.6 kPa → V/F = 0.0929333445 T = 323.5875148 K

feedQuality q = 1 - V/F = 0.9070666555

and 0/feed carries the COLUMN-INLET state (323.5875148 K, 658600 Pa, vaporFraction 0.0929333445), not the line state. The paper's 892.67 kPa therefore appears in this case only as the premise of A1, stated above. NOTE the arity cost, declared: q lives in two places (this key and the stream's vaporFraction) because the Wang-Henke path takes q as a number and does not read it off the inlet. They must be kept equal; the derivation above is how you check them.

Sensitivity, MEASURED rather than asserted – the competing assumption was run. Declaring instead "saturated liquid AT THE COLUMN PRESSURE" gives T = 323.2706 K (bubble point at 658.6 kPa) and q = 1.0 exactly, and the whole column moves by:

x_D(i-butane) 0.92571 → 0.92576 (+5e-5) x_B(i-butane) 0.00352 → 0.00350 (-2e-5)

i.e. below the resolution of anything this case compares against. A1 is therefore chosen on FIDELITY TO THE RECORD, not on the answer, and the simpler assumption is available to anyone who prefers one derived number to three.

4. DISTILLATE RATE – a mass spec the engine takes in moles

Table 2 gives 8115 kg/h; 'distillateRate' is a MOLAR flow. The conversion needs a mean MW, and the only one available is the one from the paper's own MEASURED top composition (top wt % of Table 2, which sums to 100.0 exactly):

component wt% kg/h MW kmol/h y

propane 5.3 430.0950 44.0960 9.753606 0.0686760 isoButane 93.5 7587.5250 58.1222 130.544353 0.9191740 nButane 0.2 16.2300 58.1230 0.279235 0.0019661 1Butene 0.4 32.4600 56.1063 0.578545 0.0040736 trans2Butene 0.6 48.6900 56.1063 0.867817 0.0061104 i-butene / neopentane / i-pentane / n-pentane: 0 wt % as printed

TOTAL 100.0 8115.0000 142.023556 1.0000000

mean distillate MW = 8115 / 142.023556 = 57.13841 kg/kmol → distillateRate 142.023556 kmol/h

SAID PLAINLY: this input therefore borrows the measured distillate COMPOSITION, which is one of the quantities the case then predicts. A fully self-consistent conversion would iterate the mean MW against the predicted composition. It is not iterated here, and the size of what is left on the table is MEASURED rather than waved at: the run's own converged/distillate comes out at mean MW 57.2186, i.e. +0.14 % on the 57.13841 used above, so the distillate leaves at 8126.4 kg/h against the Table-2 8115 kg/h. One iteration would close that; it is left open because the residue is smaller than the 0.1 wt % resolution of the composition it came from. If a future edit moves the distillate composition materially, this number must be recomputed with it.

5. REFLUX RATIO – why the MASS ratio IS the MOLAR ratio here

Table 2 gives reflux 92838 kg/h and distillate 8115 kg/h, both MASS. The condenser is TOTAL, so the reflux and the distillate are drawn from one condensed stream and have IDENTICAL composition, hence identical mean MW. L/D in moles = (L_mass/MW)/(D_mass/MW) = L_mass/D_mass exactly – no composition-weighted conversion is needed, and none is done:

R = 92838 / 8115 = 11.440296 (11.4402957...)

A NOTE ON A NUMBER IN CIRCULATION: 11.4441 has been quoted for this ratio. It is not what the division gives. 8115 x 11.4441 = 92859.9 kg/h, 22 kg/h above the Table-2 reflux. This case uses 11.440296.

6. BOTTOMS – an OUTPUT, and it does not land on 18119 kg/h

B = F - D is the engine's mole balance: 452.710957 - 142.023556 = 310.687401 kmol/h. Priced at the PREDICTED bottoms composition that comes out at 18107.6 kg/h, 0.06 % under the Table-2 18119 kg/h. It cannot land on it exactly: F's molar total is fixed by the feed mass and the feed's measured composition, D's by the distillate mass and the distillate's measured composition, and the paper's three compositions do not close per component (record Section 5.4 – the wt % columns sum to 100.1, 100.0 and 100.3). The mass residue is the printing, not the column. 18119 kg/h is therefore transcribed above and used by nothing.

7. REFLUX TEMPERATURE 18.5 C – transcribed, NOT USED, and why

18.5 C is far below the bubble point of the overhead at 658.6 kPa (about 41 C, and the run prints its own top-stage temperature). The plant's condenser therefore SUBCOOLS its reflux. Choupo's Wang-Henke path returns saturated reflux and has no subcooling keyword; declaring one would be inventing a model. The number is transcribed so nobody has to wonder whether it was overlooked, and the consequence is stated: the real column's internal liquid traffic in the top few trays is higher than the constant-molar-overflow value used here, by the amount the subcooled reflux condenses on contact. That is a KNOWN, UNMODELLED difference between this case and the plant, not a tuning knob.

8. BOILER DUTY 10.240 MW – transcribed, and NOT claimed as an anchor

The run reports a reboiler duty of about 8.47 MW against the Table-2 10.240 MW. The case does NOT anchor on that, and the reason is in its own output rather than in an argument.

The Wang-Henke path solves M and E under CONSTANT MOLAR OVERFLOW: the internal traffic is fixed by R, D and q, and NO per-stage energy balance is solved. The duties are then reported by closing the column's overall enthalpy change, and the flowsheet's independent energy audit raises the first-law advisory on this run – 98.42 % closure, a residual of 2.96 kW, 0.017 % of the boundary enthalpy. That is small, and it is NOT evidence that the duty is right: the ledger and the duty are computed from the same three stream states, so agreement between them says the arithmetic is consistent, not that a column with no energy equation found the right vapour traffic. The companion arm column15 makes the point unmissable – the same three streams to within 0.8 kW of enthalpy, and its audit closes at -256.84 % with a 670.7 kW residual. Two runs of one column, one ledger claim each, three orders of magnitude apart. The corpus's other Wang-Henke columns sit further out still (column01/09/11's own energyBalance_byUnit.csv), so this is a property of the solver path and of the reporting around it, and it predates this case by a long way.

Also unmodelled and worth naming beside the duty: the plant's subcooled reflux (7) and the condenser's own subcooling duty are not in this figure.

A duty comparison resting on any of that would be a number pretending to be evidence. 10.240 MW is transcribed; 8.47 MW is reported; neither is an anchor row.

9. THERMO – mirroring the paper, not substituting for it

The paper states its own model: liquid activity by UNIFAC, vapour pressure by Antoine, vapour fugacity by the original Soave-Redlich-Kwong. All three are public in this tree, so constant/thermoPhysPropDict declares exactly that and no comparison here hides a model swap. All nine components carry a UNIFAC group decomposition in data/standards/components/, and the only main-group pair the mixture needs (CH2 <→ C=C, for the three butenes) is in data/standards/parameters/UNIFAC/interactions.dat. There is no approximations{} block and none is needed: every component is priced by the model that is declared.

TWO CAVEATS THE RUN ANNOUNCES FOR ITSELF, repeated here so they are read before the numbers and not after: seven of the nine component records carry ‘reviewStatus interim‘ (imported from CoolProp 7.2.0, not yet checked against primaries). The thermodynamics under this anchor is not itself anchored. propane's Antoine fit is declared valid to 320.7 K and the stripping section runs to about 329 K, so its Psat is EXTRAPOLATED there. Propane is 1.5 wt % of the feed and leaves overhead; the extrapolation is announced by the engine on every run and is not corrected here.

10. METHOD – Wang-Henke, and it is not a preference

‘model simultaneous‘ REFUSES a murphreeEfficiency other than 1: there the Murphree relation would have to enter the residual and the Jacobian, and it does not. The efficiency is implemented on the Wang-Henke path only, folded into an effective K with the vapour from the tray below lagged one outer pass, so this case is Wang-Henke because that is where the measured input is honoured. It costs iterations: the corpus's other columns converge in tens of outer passes, this one in several hundred, because 74 trays of a close-boiling pair is a long cascade. maxOuterIter is raised accordingly.

11. E_MV = 1.191 IS A MEASURED INPUT, AND IT IS ABOVE 1

The paper back-calculates a Murphree TRAY efficiency of 119.1 % (74 real trays behaving as 88 ideal ones; overall column efficiency 118.9 %). Above 100 % is not an error: E_MV compares the vapour leaving a tray with equilibrium against that tray's OUTLET liquid, and on a 2.9 m cross-flow tray the liquid enters rich and leaves lean, so the vapour has met liquid richer than the outlet. Nothing anywhere exceeds equilibrium. The engine ADMITS the value and announces it, and states what it does not check (the plug-flow ceiling, which needs the point efficiency and mV/L – neither is declared).

It is an INPUT. Predicting it needs the rate-based programme Choupo deliberately does not have; the paper spends its own length on that difficulty and its Table 4 shows ten published methods disagreeing across a factor of five. This case reads the measurement; it does not reproduce it.

The companion case column15_klemola_ideal_trays is this column with murphreeEfficiency 1.0 and nothing else changed. The gap between the two is the pedagogical payload: how much of this separation the efficiency carries.

12. HAND ESTIMATE – what the specs imply, before the engine is trusted

Relative volatility of the keys from the same Antoine constants the case uses, at 325 K (mid-column):

Psat(i-C4) = 7.2334 bar, Psat(n-C4) = 5.1636 bar → alpha = 1.4008

FENSKE, on the measured key split (moles from the tables above):

S = (d_iC4/b_iC4)(b_nC4/d_nC4) = (130.544/0.932)(304.898/0.279) = 1.5287e5 N_min = ln S / ln alpha = 11.937 / 0.33698 = 35.4 ideal stages

UNDERWOOD, on the four real components (q = 1 for this estimate):

theta = 1.2483 R_min = 7.626 R / R_min = 11.440296 / 7.626 = 1.500

That last line is the strongest check available on the transcription: the reflux and distillate rates, taken from two separate lines of Table 2 and converted independently, land on exactly 1.5 x R_min – the textbook design point. Two mistranscribed numbers do not do that.

GILLILAND then gives X = 0.3066, Y = 0.3761, N = 57.4 ideal stages – fewer than the 74 trays present, and fewer than the 88 ideal trays the paper's own rigorous fit needs. The disagreement is expected and is not evidence against either: Gilliland is a correlation fitted across many systems and is known to be optimistic at high N/N_min on close-boiling pairs, and 88 is a fitted rigorous simulation, not a measurement. What the estimate establishes is the ORDER: this separation needs tens of stages, not five and not five hundred, and the declared reflux is comfortably above minimum. If the engine returned a near-perfect split on ten trays, or failed to separate at all on 74, the hand estimate would have caught it.

13. THE ANCHOR – and what a PASS on it does NOT mean

Compared: i-butane and n-butane, in BOTH products, against the Table-2 measurements. Only the keys. Record Section 5.4 forbids a trace-component anchor before this case was built – 0.1 wt % resolution on a 0.2 wt % number is +/-25 % before anything physical happens, and the traces do not close at all (propane +23 %, i-butene -100 %). The keys close to about 1 %, which is what a key-component anchor needs. The band on any anchor row belongs to the table's own rounding, not to the solver's tolerance.

This is an ENDPOINT anchor. The paper publishes no measured tray temperature profile and no measured tray composition profile – its figures are EFFICIENCY profiles from the back-calculation – so nothing here tests what the column model predicts INSIDE. The internals stay unanchored; the natural second case is a different primary (Vogelpohl's total-reflux profiles), never blended into this one.

What a PASS would mean: that the MESH solve plus a declared efficiency reproduces an industrial column's products from its own specification, on public thermodynamics. What it would NOT mean: that the internals are right (nothing here measures them), nor that Choupo could have PREDICTED the efficiency, nor that the trace components are modelled well.

14. THE ANCHOR TABLE – observed, and it does NOT all agree

Predicted product compositions, converted back to wt % from this case's own converged/ streams, beside the Table-2 measurements. The "band" column is the measurement's OWN rounding: the paper prints to 0.1 wt %, so a printed value v carries +/-0.05 wt %.

DISTILLATE (predicted 8126.4 kg/h vs the 8115 kg/h spec – see 4)

component predicted measured band verdict

i-butane 94.033 93.5 +/-0.05 +0.53 wt % HIGH n-butane 0.0015 0.2 +/-0.05 LOW by ~130x

BOTTOMS (predicted 18107.6 kg/h vs the Table-2 18119 kg/h – see 6)

component predicted measured band verdict

i-butane 0.351 0.3 +/-0.05 INSIDE the band n-butane 97.984 98.1 +/-0.05 -0.12 wt %, just out

THREE OF THE FOUR KEY ROWS LAND WHERE A KEY-COMPONENT ANCHOR WANTS THEM, around the 1 % level at which record section 5.4 says the keys close at all. THE FOURTH DOES NOT, and it is not a rounding argument: the model rejects n-butane overhead about two orders of magnitude harder than the plant did. 0.2 wt % is at the printing resolution (+/-25 % before any physics), but 130x is not a resolution effect.

It is recorded, not explained and not tuned. What can be said without guessing: the disagreement is NOT the efficiency, because the control arm column15 at E_MV = 1 gives 0.0144 wt % – still an order of magnitude below the measurement. Under this model 74 IDEAL trays already over-separate this key, which is the opposite direction from the paper's own finding that 74 real trays behave as 88 ideal ones. Candidates a future slice could test, each of which this case declares as an assumption or a caveat above: the single column pressure (A2), the tray-numbering direction (A0), the unmodelled subcooled reflux (7), the absent per-stage energy balance (8), and seven interim component records (thermo dict). Whichever it is, an anchor that disagreed and said so is worth more than one that agreed after something was adjusted to make it.

THE OTHER FIVE COMPONENTS ARE NOT ANCHOR ROWS and are listed only so nobody reads their absence as concealment: propane 4.84 vs 5.3 wt % overhead and 0.00 vs 0.3 in the bottoms; trans-2-butene 0.00 vs 0.6 overhead; i-butene 0.58 vs 0 overhead. Record section 5.4 already showed these do not close in the SOURCE – propane +23 %, i-butene -100 %, trans-2-butene +155 % between the feed and the two products as printed – so a number computed here has nothing to be compared against. Anchoring on them would be measuring the printing.

15. THE TWO ARMS, side by side

column14 (E_MV = 1.191) against column15 (E_MV = 1.0), everything else identical:

quantity E_MV 1.191 E_MV 1.0 measured

distillate i-butane, wt % 94.033 93.975 93.5 distillate n-butane, wt % 0.0015 0.0144 0.2 bottoms i-butane, wt % 0.351 0.378 0.3 bottoms n-butane, wt % 97.984 97.977 98.1 top temperature, K 314.250 314.116 – reboiler temperature, K 329.437 329.435 – outer iterations 349 426 –

WHAT THE EFFICIENCY BUYS, read off that table: essentially nothing on the major components – i-butane overhead moves 0.06 wt %, n-butane in the bottoms 0.007 – and a real factor on the MINOR key in each product: the n-butane carried overhead drops about 10x, and the i-butane lost to the bottoms falls 7 %.

That shape is worth more than a single number. At 74 trays and R = 11.4 this column is far past the point where separating power limits the major components; extra power goes entirely into the impurity that is left, which is exactly where a plant's money is. A student who expects a 19 % efficiency bonus to show up as a 19 % anything will not find it, and finding out why is the lesson.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0863020558333
stream	distillate	F	0.0394509877778
stream	feed	F	0.125753043611
kpi	splitter	B	0.0863020558333
kpi	splitter	D	0.0394509877778
kpi	splitter	L_rect	0.45133097767
kpi	splitter	L_strip	0.565397370357
kpi	splitter	Q_condenser_kW	-8654.33073632
kpi	splitter	Q_reboiler_kW	8467.13778877
kpi	splitter	R	11.440296
kpi	splitter	T_bottom	329.436820919
kpi	splitter	T_top	314.249966433
kpi	splitter	V_rect	0.490781965448
kpi	splitter	V_strip	0.479095314524
. . . and 16 more reference values in the case's `expected`.			

column15_klemola_ideal_trays

tutorials/steady/distillation/column15_klemola_ideal_trays

What it does: The Klemola-Ilme C4 splitter with IDEAL trays: the same column at E_MV = 1, the control arm that says what the measured efficiency is worth

The story: the CONTROL ARM of the Klemola-Ilme anchor.

This is column14_klemola_c4_splitter with ONE line changed:

murphreeEfficiency 1.191; → murphreeEfficiency 1.0;

Everything else – the 74 trays, the feed on tray 37, the feed state, the pressure, the distillate rate, the reflux ratio, the thermo dict, the three 0/ files, the convergence controls – is byte-identical to that case, and if the two ever drift apart this case stops being a control. Read column14's flowsheetDict for the provenance of every number; it is the primary document and none of it is repeated here.

Klemola, K. T.; Ilme, J. K. Distillation Efficiencies of an Industrial-Scale i-Butane/n-Butane Fractionator. Ind. Eng. Chem. Res. 1996, 35 (12), 4579-4586.

WHY THIS IS A SEPARATE CASE AND NOT A SECOND ARM INSIDE column14

A Choupo case is ONE pass: one converged/ state, one KPI set, one golden. Two efficiencies could have been swept in a single case through system/outerDict, and that was rejected: a sweep publishes the DRIVER's KPI table, so the two arms' products would live in a sweep artefact instead of in two real converged/ directories, and the anchor – which is an ENDPOINT anchor, a comparison of product compositions – would have nowhere natural to sit. Two cases give two converged states, two goldens, and two independently sealable units of evidence, at the cost of the duplication named above.

WHAT THE GAP BETWEEN THE ARMS MEANS – and what it does NOT

E_MV = 1 makes every one of the 74 trays an EQUILIBRIUM stage: the vapour leaving it is in equilibrium with the liquid leaving it. No real tray does that, and this plant's trays do BETTER than that – the paper back-calculates 119.1 %, because on a 2.9 m cross-flow tray the liquid is not well mixed, so the vapour has been in contact with liquid richer than the outlet.

The difference between this case's products and column14's is therefore the answer to a question a student can otherwise only be told: how much of an industrial separation is carried by the tray efficiency, as against the tray COUNT and the reflux? Run both and subtract. The paper's own version of the same arithmetic is that 74 real trays behave as 88 ideal ones.

WHAT IT IS NOT. This case is not an anchor and must never be compared with the Table-2 measurements as though it were one: it deliberately declares an efficiency the plant does not have. Its role is to be the OTHER END of a one-parameter difference. If it happened to sit closer to the measured products than column14 does, that would be a finding about the model, not a reason to prefer E_MV = 1 – 119.1 % is the measurement.

Note also that E_MV = 1 is the engine's DEFAULT, and the key is written out in full below anyway. An absent key would mean the same thing to the solver and something quite different to a reader: it would look like an oversight in a case whose entire subject is the efficiency.

WHAT THE TWO ARMS ACTUALLY GAVE

quantity E_MV 1.191 E_MV 1.0 measured

distillate i-butane, wt % 94.033 93.975 93.5 distillate n-butane, wt % 0.0015 0.0144 0.2 bottoms i-butane, wt % 0.351 0.378 0.3 bottoms n-butane, wt % 97.984 97.977 98.1 top temperature, K 314.250 314.116 – reboiler temperature, K 329.437 329.435 – outer iterations 349 426 –

The efficiency moves the MAJOR components by almost nothing (i-butane overhead by 0.06 wt %) and the MINOR key in each product by a real factor (n-butane carried overhead drops about 10x; i-butane lost to the bottoms falls 7 %). At 74 trays and R = 11.4 the column is far past the point where separating power limits the majors, so extra power goes entirely into the impurity that is left. column14's flowsheetDict section 15 argues that at length; this is the same table so that either case can be read alone.

A FINDING THIS ARM IS WHAT ESTABLISHED, and it belongs here because this is the case that establishes it: at E_MV = 1 the model ALREADY rejects n-butane overhead an order of magnitude harder than the plant measured (0.0144 wt % against 0.2). So the anchor's one clear disagreement is not caused by the declared efficiency – turning the efficiency off does not recover the measurement, it only halves the distance in the log. Under this model 74 ideal trays over-separate this key, which runs opposite to the paper's own conclusion that 74 real trays behave as 88 ideal ones. Recorded; not explained here and not tuned away.

ONE MORE OBSERVATION, and it is about the ENGINE rather than the column. These two runs produce product streams whose tabulated enthalpies agree to 0.8 kW, and their flowsheet energy audits disagree wildly: column14 closes at 98.42 % (residual 2.96 kW) and this arm at -256.84 % (residual 670.7 kW). A difference that large between two nearly identical states does not come from the physics of the two arms; it is in how the energy report prices the products, and this pair is a compact reproducer for anyone who goes looking. Neither case claims a duty anchor – see column14's section 8.

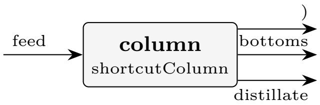
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.0863020558333`
`stream`	`distillate`	`F`	`0.0394509877778`
`stream`	`feed`	`F`	`0.125753043611`
`kpi`	`splitter`	`B`	`0.0863020558333`
`kpi`	`splitter`	`D`	`0.0394509877778`
`kpi`	`splitter`	`L_rect`	`0.45133097767`
`kpi`	`splitter`	`L_strip`	`0.565397370357`
`kpi`	`splitter`	`Q_condenser_kW`	`-8659.56425253`
`kpi`	`splitter`	`Q_reboiler_kW`	`8471.59434507`
`kpi`	`splitter`	`R`	`11.440296`
`kpi`	`splitter`	`T_bottom`	`329.435279168`
`kpi`	`splitter`	`T_top`	`314.116313789`
`kpi`	`splitter`	`V_rect`	`0.490781965448`
`kpi`	`splitter`	`V_strip`	`0.479095314524`

. . . and 15 more reference values in the case's `expected`.

`shortcut01_benzene_toluene`

`tutorials/steady/distillation/shortcut01_benzene_toluene`



What it does: Shortcut (FUG) design of a benzene/toluene column

The story: Preliminary column design by the Fenske-Underwood-Gilliland (FUG) shortcut method (the built-in ‘shortcutColumn‘). The same benzene/ toluene separation as the rigorous ‘column01‘, but designed in closed form: you specify the key recoveries and the reflux, and FUG returns the minimum stages, the minimum reflux, the actual stage count and the feed location.

Feed: 100 kmol/h, 50/50 benzene/toluene, saturated liquid, 1 atm. Keys: LK = benzene, HK = toluene ($\alpha \approx 2.5$). Spec: 99 % of the benzene to the distillate, 1 % of the toluene. Reflux: R = 1.3 · R_min.

Expected ($\alpha \approx 2.5$): N_min ≈ 10.1 (Fenske, total reflux) R_min ≈ 1.3 (Underwood) N ≈ 22 (Gilliland, at R = 1.3 R_min) distillate ~99 % benzene, bottoms ~99 % toluene; mass closes 100 %.

Compare with the rigorous column01 (Wang-Henke): FUG is a fast design estimate; the rigorous column is the verification. FUG does NOT handle azeotropes — it assumes constant relative volatility.

An equimolar benzene/toluene feed (100 kmol/h at 365 K, 1 atm) is to be split 99 % / 1 %: 99 % of the benzene overhead, 1 % of the toluene with it. Fenske–Underwood–Gilliland answers in one pass, no stages solved. The golden: N_min = 10.07 (Fenske), R_min = 1.294 (Underwood, $\theta = 1.427$), operating R = 1.682 (declared as 1.3·R_min), N = 21.6 theoretical stages with the feed at 10.8 from the top; α_LK/HK = 2.491 at the feed bubble point; D = B = 50 kmol/h; x_D = x_B = 0.99.

1. Three declared numbers, four printed results. recoveryLK 0.99; recoveryHK 0.01; refluxFactor 1.3; — and the engine will not invent the reflux factor for you: a column whose reflux the engine chose is one nobody can defend at a viva, so the key is REQUIRED. 2. The only thermodynamics is one bubble point. α is evaluated once, at the feed's bubble temperature, and held constant — the method's founding assumption, and the run says so in its result block. 3. A shortcut is recorded as a different question. Look at the top of the result: problemDivergence carries a declaredApproximation for this unit — ∗requested: distillation separation; solved: Fenske-Underwood-Gilliland shortcut (constant relative volatility at the feed bubble point; no azeotrope representable)∗. It is not an error and it is not hidden: a stream table cannot show that the column was a shortcut, so the record travels with the answer. 4. Now build the rigorous one. column01_benzene_toluene puts 15 real stages with the feed on 8 at R = 2 — read that README and compare N = 21.6 at 1.3·R_min with what 15 stages at a higher reflux actually deliver.

Set refluxFactor 1.05: N climbs steeply toward infinity as R → R_min — Gilliland's curve, in two runs. Then recoveryLK 0.999: N_min jumps, because Fenske is logarithmic in the split.

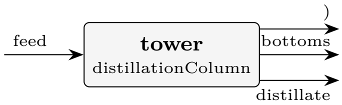
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.0138888888889`
`stream`	`distillate`	`F`	`0.0138888888889`
`stream`	`feed`	`F`	`0.0277777777778`
`kpi`	`column`	`B`	`0.0138888888889`
`kpi`	`column`	`D`	`0.0138888888889`
`kpi`	`column`	`N_min`	`10.067785029`
`kpi`	`column`	`N_theoretical`	`21.5880731317`
`kpi`	`column`	`R`	`1.68248609289`
`kpi`	`column`	`R_min`	`1.29422007145`
`kpi`	`column`	`alpha_LK_HK`	`2.49137883567`
`kpi`	`column`	`feed_stage`	`10.7940365659`
`kpi`	`column`	`theta`	`1.42716041594`
`kpi`	`column`	`x_B_HK`	`0.99`
`kpi`	`column`	`x_D_LK`	`0.99`

. . . and 3 more reference values in the case's `expected`.

`stripper01_sour_water`

`tutorials/steady/distillation/stripper01_sour_water`



What it does: S2: a sour-water stripper over the chemistry – eight reactive trays, the CO2-strips-first mechanism measured across the whole column.

The story: S2 of the sour-water programme: a REAL multi-stage column over the chemistry, not a two-stage identity rig.

column13 proved the machinery: one reactive flash per stage per residual, the apparent components the state, the ions internal to each tray. This case is what the machinery exists FOR – a sour-water stripper: feed near the top, most of the trays below it stripping, the reboiler making the vapour that carries NH3 and CO2 overhead.

THE MECHANISM IS THE LESSON (docs/design/sour-water-stripper-scope.md ?4.3): stripping CO2 RAISES the liquid pH down the column, and the rising pH converts NH4+ back to free NH3, which then strips. The separation and the speciation are the same problem. Eight trays instead of column13's four give the mechanism room to show, and it shows STRICTLY, measured on this converged profile: the carbonate loading falls tray by tray from 0.524 mol/kg (top) to 0.024 (reboiler) – over an order of magnitude – while the free-ammonia fraction rises 0.802 → 0.947 with no exception, and the pH climbs 8.46 → 8.80 below the top tray. (The top tray itself sits ~7 K colder than tray 2 and its pH reads higher for the temperature of its own Kw – comparing pH across that jump compares two water dissociations, which is why the pH claim starts at tray 2.) check_tray_chemistry's ordering claim is tested across this whole span rather than across four points.

WHAT BUILDING IT FOUND, because the first honest run is an instrument (both fixes are the engine's, recorded in the scope doc ?6d): a MESH initialisation ramps T across the column, and an 8-stage ramp visits temperatures a 4-stage ramp never did – including ones above the feed's two-phase band, where the reactive Newton has no interior V/F and stalled. stageK now prices such a trial INCIPIENT over the hypothetical speciated liquid (announced once), the same construction its subsaturated branch always used – continuous K, and the aid shapes the path, never the answer (a converged stage sits at saturation, where the constructions agree); the per-tray chemistry report used to FLASH each converged tray liquid – a liquid pinned by the MESH to its own bubble point, i.e. the flash's degenerate corner – and at reflux 0.5 five trays of eight printed NaN for chemistry the package resolves easily. The report now asks the right question (speciate the LIQUID), and NaN is reserved for a tray whose speciation itself fails.

WHAT THIS CASE DECLARES, said plainly (the column12/13 rule): 'model simultaneous;' – the rigorous MESH, whose per-stage equilibrium is 'ThermoPackage::stageK': for this reactive package, one reactive flash per stage per residual evaluation. Nested, not a species-basis MESH – the Jacobian sees apparent components only. the per-tray chemistry (pH, ionic strength, every molality) rides in reports/unitOperations/tower/profile.csv – engine-owned, the same surface column13 writes.

DATA SCOPE: sealed (bin/choupo-import); records mirrored under constant/ and sha256-verified by constant/propertyManifest.

*Eight reactive trays over a sour water (2.5 mol% NH_3, 1 mol% CO_2), feed on tray 2, reflux 0.5, distillate 5 of 100 kmol/h. Converges in 9 MESH Newton iterations; the overhead carries 33.4 mol% NH_3 at 355 K, the bottoms leave at 99.1 % water. This is S2 of the sour-water programme (docs/design/sour-water-stripper-scope.md ?6d) — the case the effective stage K (ThermoPackage::stageK) was built *for*, after column13 proved the machinery on an identity rig.*

A molecular column cannot express this separation. The volatility of NH_3 is set by a chemistry the column itself moves:

1. CO_2 strips first — it is the more volatile solute, and the carbonate loading of the liquid falls tray by tray, across an order of magnitude (0.52 → 0.024 mol CO_2-species per mol NH_3-species). 2. The falling carbonate frees the ammonia. The CO_2 a liquid still carries is what holds its ammonia protonated as NH_4?; as it leaves, the pH climbs (8.46 → 8.80 below the top tray) and the free-ammonia fraction rises 0.802 →

0.947 — strictly, with no exception, on every tray. 3. The freed ammonia then strips. The whole cascade is visible in reports/unitOperations/tower/profile.csv: per-tray pH, ionic strength and every species molality, engine-owned.

*The surge — the mechanism's own signature. Free ammonia is not merely stripped; it is *produced* by the deprotonation the falling carbonate allows. Just below the feed the production outruns the stripping: m_NH3aq rises from 1.316 (tray 2, the feed tray) to 1.424 (tray 3) and only then falls strictly to the reboiler (0.464). A first instinct would be to exempt this from the "everything is stripped downward" check; instead it is pinned as its own claim (check_tray_chemistry T5), because hiding a mechanism's signature to make a monotonicity check pass would be the check defeating its subject. Sabotage-verified: move the feed to tray 5 and T5 alone fires.*

The top tray sits ~7 K colder than tray 2, so its pH reads higher for the temperature of its own K_w — comparing pH across that jump compares two water dissociations, which is why the pH claim starts at tray 2.

** The MESH initial ramp vs the two-phase band. An 8-stage linear T ramp visits temperatures a 4-stage ramp never did — including Tf + 15 K, *above* the feed's two-phase band, where the reactive Newton has no interior V/F and stalled before the first iteration printed. stageK now catches the typed ReactiveVLE::NonConvergence and prices such a trial *incipient* over the hypothetical speciated liquid (announced once) — the same K_i = (p_i^eq/P)/x_i its subsaturated branch always used, so the K surface is continuous across the band. A converged stage sits at saturation, where the constructions agree: the aid shapes the path, never the answer. * The per-tray chemistry report flashed a liquid pinned to its own bubble point. The MESH's residual ($\Sigma y = 1$) places every converged tray liquid exactly at its bubble point — the two-phase flash's degenerate corner — and at reflux 0.5 five trays of eight printed NaN for chemistry the package resolves easily. The report now asks the right question (speciateReactiveAsLiquid); NaN is reserved for a tray whose speciation itself fails.*

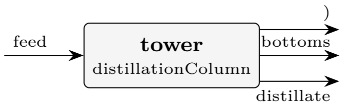
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0263888888889
stream	distillate	F	0.00138888888889
stream	feed	F	0.0277777777778
kpi	tower	B	0.0263888888889
kpi	tower	D	0.00138888888889
kpi	tower	Q_condenser_kW	-51.4325432244
kpi	tower	Q_reboiler_kW	71.9165526448
kpi	tower	R	0.5
kpi	tower	T_bottom	370.531146763
kpi	tower	T_top	355.393273713
kpi	tower	feedStage	2
kpi	tower	nFeeds	1
kpi	tower	nSideDraws	0
kpi	tower	nStages	8

. . . and 13 more reference values in the case's `expected`.

stripper02_sour_water_h2s

tutorials/steady/distillation/stripper02_sour_water_h2s



What it does: The three-family sour water: CO2 strips overhead while H2S concentrates downward and the pH FALLS – the selectivity lesson the two-family stripper cannot teach.

The story: the THREE-family sour water: NH3 + CO2 + H2S, and the SELECTIVITY lesson the two-family stripper cannot teach.

stripper01 shows the mechanism: CO2 strips, pH rises, ammonia frees. Add the third volatile weak electrolyte and the story INVERTS in the most instructive way, measured on this converged profile:

CO2 still strips overhead, hard: its loading falls from 0.195 to 1e-5 (per mol of ammonia species) between tray 2 and the reboiler; H2S does NOT follow – its loading RISES down the column, 0.18 to 0.51: the H2S/CO2 selectivity of a sour-water stripper at these conditions favours CO2 removal, and the sulfide concentrates in the liquid heading for the bottoms; and the pH FALLS down the column (8.49 → 7.61), the OPPOSITE of stripper01's climb: the departing ammonia and the accumulating sulfide acidify faster than the departing CO2 alkalinises.

A student who has read stripper01 should predict a rising pH here and be wrong – that is the point. The chemistry is the same machinery (three networks at once: ammonia, carbonate, sulfide – the H2S bridge is 'aqueousSpeciation sulfide', HS- + H+, declared on the component), and the per-tray profile carries all three families' molalities.

Same column as stripper01 otherwise: 8 reactive trays, feed on tray 2, reflux 0.5, D = 5 of 100 kmol/h, 'model simultaneous' (one reactive flash per stage per residual evaluation; the Jacobian sees apparent components only).

DATA SCOPE: sealed (bin/choupo-import); records mirrored under constant/ and sha256-verified by constant/propertyManifest.

The three-family sour water: NH_3 (2.5) + CO_2 (1.0) + H_2S (0.5 kmol/h) in 96 kmol/h water, over the same 8 reactive trays as stripper01. Three speciation networks run at once on every tray — ammonia, carbonate, sulfide — and the per-tray profile carries all of them.

stripper01 teaches: CO_2 strips → pH rises → NH_4? deprotonates → ammonia strips. Add H_2S and a student should predict the same rising pH. The converged profile says otherwise:

CO_2 is the more volatile acid gas at these conditions, so the stripper is selective: the carbonate leaves overhead while the sulfide stays with the liquid — and with the ammonia also leaving, the residual H_2S acidifies the bottoms faster than the departing CO_2 can alkalinise it. The pH direction is not a law of strippers; it is an outcome of ∗which∗ acid gas wins, and this pair of cases shows both outcomes with the same machinery.

∗ reports/unitOperations/tower/profile.csv — per-tray pH, ionic strength and every species molality of all three families (m_H2Saq, m_HS beside the carbonate and ammonia columns). ∗ The distillate: an acid-gas overhead (NH_3 + CO_2 + a little H_2S) — the real design question of a sour-water unit (where each contaminant ends up) is answered per tray, not just at the ends. ∗ A tray whose H_2S has stripped to numerical zero reports the sulfide molalities as 0 exactly — a solved tray with an empty family is a zero, never a NaN (NaN stays reserved for a tray whose speciation itself failed).

model simultaneous; — one reactive flash per stage per residual evaluation; the Jacobian sees apparent components only. Sealed (bin/choupo-import).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0263888888889
stream	distillate	F	0.00138888888889
stream	feed	F	0.0277777777778
kpi	tower	B	0.0263888888889
kpi	tower	D	0.00138888888889
kpi	tower	Q_condenser_kW	-52.339216667
kpi	tower	Q_reboiler_kW	76.4475297294
kpi	tower	R	0.5
kpi	tower	T_bottom	372.031545214
kpi	tower	T_top	356.430868411
kpi	tower	feedStage	2
kpi	tower	nFeeds	1
kpi	tower	nSideDraws	0
kpi	tower	nStages	8
...and 23 more reference values in the case's `expected`*.*			

drying

evapDryer01_nacl

tutorials/steady/drying/evapDryer01_nacl

What it does: Evaporative dryer: free surface water on NaCl crystals evaporated into dry nitrogen, adiabatic, the binding limit announced

The story: FREE WATER, NOT BOUND WATER.

'evaporativeDryer' is the honest counterpart of 'solidDryer': where that one models a HYGROSCOPIC material whose moisture is GAB-sorbed and dries toward an equilibrium isotherm, this one models a NON-sorbing crystal whose moisture is free surface water. There is no isotherm, so there is no equilibrium moisture to stop at – the drying stops at whichever of three real limits binds FIRST, and the unit ANNOUNCES which:

1. the free water runs out; 2. the exhaust reaches saturation (the constant-rate air-capacity limit); 3. the gas cannot supply the latent + sensible heat.

Limits 2 and 3 are coupled – evaporation cools the gas, which lowers its carrying capacity – and the unit bisects on the removed water until the two agree, rather than applying them in sequence and hoping.

Wet salt: 20 kmol/h NaCl crystals (1169 kg/h) carrying 15 kmol/h of free water. Dry nitrogen at 400 K. The gas flow is deliberately generous, so the binding limit here is the FIRST one: the water runs out. Halve the nitrogen and the announcement changes – that is the case to run next.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	drySolid	F	0.00555555555556
stream	hotGas	F	0.166805555556
stream	humidExhaust	F	0.170972222222
stream	wetSolid	F	0.00972222222222
kpi	dryer	T_out	360.675490975
kpi	dryer	X_final	0
kpi	dryer	X_initial	0.231198665298
kpi	dryer	air_in_kmol_h	600.5
kpi	dryer	drySolid_flow	0.324666666667
kpi	dryer	exhaust_humidity	0.0392946211155
kpi	dryer	moisture_pct_wb	0
kpi	dryer	water_removed	0.0750625
stream	drySolid	T	360.675490975
stream	hotGas	T	400

...and 2 more reference values in the case's `expected`.

evapDryer02_energy_limited

`tutorials/steady/drying/evapDryer02_energy_limited`

What it does: The same dryer starved of gas: 40 kmol/h instead of 600, so the ENERGY floor binds and 16.6 wt% of the water stays on the crystal

The story: FREE WATER, NOT BOUND WATER.

‘evaporativeDryer‘ is the honest counterpart of ‘solidDryer‘: where that one models a HYGROSCOPIC material whose moisture is GAB-sorbed and dries toward an equilibrium isotherm, this one models a NON-sorbing crystal whose moisture is free surface water. There is no isotherm, so there is no equilibrium moisture to stop at – the drying stops at whichever of three real limits binds FIRST, and the unit ANNOUNCES which:

1. the free water runs out; 2. the exhaust reaches saturation (the constant-rate air-capacity limit); 3. the gas cannot supply the latent + sensible heat.

Limits 2 and 3 are coupled – evaporation cools the gas, which lowers its carrying capacity – and the unit bisects on the removed water until the two agree, rather than applying them in sequence and hoping.

THIS CASE IS THE STARVED ONE. Identical to evapDryer01_nacl except for the gas: 40 kmol/h instead of 600. The announcement changes to [energy-limited], a WARNING says the gas cannot supply the drying heat, T_out is floored at the wet solid’s own temperature, and 16.6 wt% of the water stays on the crystal – a dryer that does not dry, reported as such instead of returning a dry solid it cannot produce.

Read the exhaust humidity beside it: 0.95, exactly the cap. Both limits are within rounding of each other here, which is what the coupling means – the gas cools as it evaporates, and running out of heat and running out of room are the same wall approached from two sides.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`drySolid`	`F`	`0.00914216942312`
`stream`	`hotGas`	`F`	`0.01125`
`stream`	`humidExhaust`	`F`	`0.0118300527991`
`stream`	`wetSolid`	`F`	`0.00972222222222`
`kpi`	`dryer`	`T_out`	`310.725305009`
`kpi`	`dryer`	`X_final`	`0.199012881389`
`kpi`	`dryer`	`X_initial`	`0.231198665298`
`kpi`	`dryer`	`air_in_kmol_h`	`40.5`
`kpi`	`dryer`	`drySolid_flow`	`0.389279515491`
`kpi`	`dryer`	`exhaust_humidity`	`0.949998302456`
`kpi`	`dryer`	`moisture_pct_wb`	`16.5980603276`
`kpi`	`dryer`	`water_removed`	`0.0104496511758`
`stream`	`drySolid`	`T`	`310.725305009`
`stream`	`hotGas`	`T`	`400`

. . . and 2 more reference values in the case’s `expected`.

solidDryer01_sugar

tutorials/steady/drying/solidDryer01_sugar

What it does: Solid dryer: wet sugar powder dried to sorption equilibrium

The story: / flowsheetDict (LEAF node)

Wet sugar dried by a REAL hot-air STREAM (no airTemperature/RH parameters, no phantom duty): the hot air BRINGS the heat (its sensible cooling) and CARRIES AWAY the moisture (a humid exhaust). Adiabatic → the outlet T is a RESULT; the plant energy balance closes with real streams.

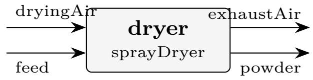
Wet solid: 10 kmol/h sucrose (3423 kg/h) + 38 kmol/h bound water (X_in=0.20). Hot air: 800 kmol/h at 420 K, DRY (2 mol% water → RH ~0.4%) – hot dry air, like a real sugar dryer. Its low RH sets a low GAB equilibrium moisture, so the sugar dries to X_eq ~0.001 (~0.1 wt%); the exhaust leaves carrying the evaporated water (838 > 800 kmol/h, the pickup visible in the molar balance).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	drySolid	F	0.00283583138708
stream	hotAir	F	0.222222222229
stream	humidExhaust	F	0.23271974852
stream	wetSolid	F	0.0133333576778
kpi	solidDryer	T_out	347.5126791
kpi	solidDryer	X_equilibrium	0.00109945367934
kpi	solidDryer	X_final	0.00109945367934
kpi	solidDryer	X_initial	0.199991235822
kpi	solidDryer	air_in_kmol_h	800.000000025
kpi	solidDryer	drySolid_flow	0.951878730237
kpi	solidDryer	moisture_pct_wb	0.109824620851
kpi	solidDryer	water_activity	0.00431158404412
kpi	solidDryer	water_removed	0.189112936127
stream	drySolid	T	347.5126791
... and 3 more reference values in the case's `expected`*.*			

sprayDryer01_sugar

tutorials/steady/drying/sprayDryer01_sugar



What it does: Spray dryer: 40 wt% sugar solution → powder (rotary atomiser, Ranz-Marshall)

The story: flowsheet (one unit, two feeds, two products).

feed: 100 kg/h of a 40 wt% sucrose solution at 25 °C. 40 kg/h sucrose / 342.30 = 0.11686 kmol/h 60 kg/h water / 18.015 = 3.33056 kmol/h total 3.4474 kmol/h ; z_sucrose = 0.03390

dryingAir: 50 kmol/h of hot air (real air (N2/O2/Ar)) at 200 °C, 1 bar. Dry air (no inlet humidity) for a clean wet-bulb demo.

The dryer atomises the feed with a 0.1 m disc spinning at 20 000 rpm.

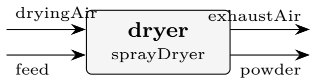
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	dryingAir	F	0.0138888888889
stream	exhaustAir	F	0.0148058483113
stream	feed	F	0.000957611111111
stream	powder	F	4.06516887181e-05
kpi	dryer	Nu	2.24583682227
kpi	dryer	Pr	0.746977616919
kpi	dryer	Re	0.203916784363
kpi	dryer	Sc	0.6026290984
kpi	dryer	Sh	2.22885536787
kpi	dryer	T_out	372.025243986
kpi	dryer	T_wetbulb	318.053221267
kpi	dryer	d_droplet	6.04023981348e-05
kpi	dryer	d_droplet_micron	60.4023981348
kpi	dryer	d_particle	3.80926671659e-05

. . . and 36 more reference values in the case's `expected`.

sprayDryer02_residence_sweep

tutorials/steady/drying/sprayDryer02_residence_sweep



What it does: Spray dryer: chamber-height sweep → the drying curve (residence = L/v; constant + falling rate → equilibrium)

The story: sprayDryer01's sugar dryer with the drying-chamber HEIGHT swept by system/outerDict (0.01 → 0.3 m, 40 pts): residence time = L / v__particle traces the DRYING CURVE – constant-rate then falling-rate, down to the equilibrium moisture.

The base flowsheet is sprayDryer01__sugar's, unchanged: one unit, two feeds, two products.

feed: 100 kg/h of a 40 wt% sucrose solution at 25 °C. 40 kg/h sucrose / 342.30 = 0.11686 kmol/h 60 kg/h water / 18.015 = 3.33056 kmol/h total 3.4474 kmol/h ; z__sucrose = 0.03390

dryingAir: 50 kmol/h of hot air (real air (N2/O2/Ar)) at 200 °C, 1 bar. Dry air (no inlet humidity) for a clean wet-bulb demo.

The dryer atomises the feed with a 0.1 m disc spinning at 20 000 rpm.

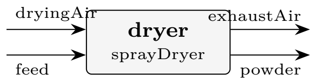
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	dryingAir	F	0.0138888888889
stream	exhaustAir	F	0.0148058483112
stream	feed	F	0.000957611111111
stream	powder	F	4.06516887566e-05
kpi	dryer	Nu	2.23180639411
kpi	dryer	Pr	0.746977616919
kpi	dryer	Re	0.181305061943
kpi	dryer	Sc	0.6026290984
kpi	dryer	Sh	2.21579410729
kpi	dryer	T_out	372.025243986
kpi	dryer	T_wetbulb	318.053221267
kpi	dryer	X_critical	0.15
kpi	dryer	X_equilibrium	0.0132756494842
kpi	dryer	X_final	0.0132756495467

. . . and 36 more reference values in the case's `expected`*.*

sprayDryer03_pressure_nozzle

tutorials/steady/drying/sprayDryer03_pressure_nozzle



What it does: Spray dryer: 40 wt% sugar solution → powder (PRESSURE-SWIRL nozzle at 40 bar, d32=9575∗dP^-1/3; Ranz-Marshall)

The story: spray dryer: pressure-swirl-nozzle atomisation (d32 from nozzle dP).

feed: 100 kg/h of a 40 wt% sucrose solution at 25 °C. 40 kg/h sucrose / 342.30 = 0.11686 kmol/h 60 kg/h water / 18.015 = 3.33056 kmol/h total 3.4474 kmol/h ; z_sucrose = 0.03390

dryingAir: 50 kmol/h of hot air (real air (N2/O2/Ar)) at 200 °C, 1 bar. Dry air (no inlet humidity) for a clean wet-bulb demo.

The dryer atomises the feed with a PRESSURE-SWIRL nozzle at 40 bar (d32 = 9575∗dP^(-1/3) um, the class-average design curve) instead of a wheel.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	dryingAir	F	0.0138888888889
stream	exhaustAir	F	0.0148058483113
stream	feed	F	0.000957611111111
stream	powder	F	4.06516887181e-05
kpi	dryer	Nu	2.24532614778
kpi	dryer	Pr	0.746977616919
kpi	dryer	Re	0.203070475508
kpi	dryer	Sc	0.6026290984
kpi	dryer	Sh	2.2283799688
kpi	dryer	T_out	372.025243986
kpi	dryer	T_wetbulb	318.053221267
kpi	dryer	X_critical	0.15
kpi	dryer	X_equilibrium	0.0132756494842
kpi	dryer	X_final	0.0132756494842

. . . and 36 more reference values in the case's `expected`*.*

sprayDryer04_profiles

tutorials/steady/drying/sprayDryer04_profiles

What it does: Spray dryer DISTRIBUTED: axial profiles of moisture, size and temperature along the chamber

The story: spray dryer: axial profiles of moisture, droplet size and T along the chamber.

feed: 100 kg/h of a 40 wt% sucrose solution at 25 °C. 40 kg/h sucrose / 342.30 = 0.11686 kmol/h 60 kg/h water / 18.015 = 3.33056 kmol/h total 3.4474 kmol/h ; z_sucrose = 0.03390

dryingAir: 50 kmol/h of hot air (real air (N2/O2/Ar)) at 200 °C, 1 bar. Dry air (no inlet humidity) for a clean wet-bulb demo.

The dryer atomises the feed with a 0.1 m disc spinning at 20 000 rpm.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`dryingAir`	`F`	`0.0138888888889`
`stream`	`exhaustAir`	`F`	`0.0148058483113`
`stream`	`feed`	`F`	`0.000957611111111`
`stream`	`powder`	`F`	`4.06516887181e-05`
`kpi`	`dryer`	`Nu`	`2.24583682227`
`kpi`	`dryer`	`Pr`	`0.746977616919`
`kpi`	`dryer`	`Re`	`0.203916784363`
`kpi`	`dryer`	`Sc`	`0.6026290984`
`kpi`	`dryer`	`Sh`	`2.22885536787`
`kpi`	`dryer`	`T_out`	`372.025243986`
`kpi`	`dryer`	`T_wetbulb`	`318.053221267`
`kpi`	`dryer`	`X_critical`	`0.15`
`kpi`	`dryer`	`X_equilibrium`	`0.0132756494842`
`kpi`	`dryer`	`X_final`	`0.0132756494842`

. . . and 36 more reference values in the case's `expected`.

sprayDryer05_whey

tutorials/steady/drying/sprayDryer05_whey

What it does: Spray dryer: whey (prob 5.2) – 500 kg/h, X0=5, rotary co-current, sorption-isotherm residual moisture

The story: spray dryer: whey feed (500 kg/h, X0=5), rotary co-current, sorption-isotherm residual.

feed: 100 kg/h of a 40 wt% sucrose solution at 25 °C. 40 kg/h sucrose / 342.30 = 0.11686 kmol/h 60 kg/h water / 18.015 = 3.33056 kmol/h total 3.4474 kmol/h ; z_sucrose = 0.03390

dryingAir: ~500 kmol/h of hot AIR (N2/O2/Ar) at 180 C – sized to carry the whey water. Whey solids modelled as sucrose (lactose proxy, a disaccharide MW~342).

The dryer atomises the feed with a 0.1 m disc spinning at 20 000 rpm.

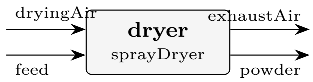
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	dryingAir	F	0.138888888889
stream	exhaustAir	F	0.14529654835
stream	feed	F	0.00649261111111
stream	powder	F	8.49516499934e-05
kpi	dryer	Nu	2.33731434528
kpi	dryer	Pr	0.745230561117
kpi	dryer	Re	0.384509154435
kpi	dryer	Sc	0.602916461126
kpi	dryer	Sh	2.31430911286
kpi	dryer	T_out	382.7774359
kpi	dryer	T_wetbulb	315.824008236
kpi	dryer	X_critical	0.5
kpi	dryer	X_equilibrium	0.0134981947613
kpi	dryer	X_final	0.0134981947613

. . . and 36 more reference values in the case's `expected`.

sprayDryer06_rea

`tutorials/steady/drying/sprayDryer06_rea`



What it does: Spray dryer REA (model chen): Reaction Engineering Approach kinetics – smooth rate, no critical-moisture break

The story: spray dryer: Reaction Engineering Approach (REA) drying kinetics (Chen model).

feed: 100 kg/h of a 40 wt% sucrose solution at 25 °C. 40 kg/h sucrose / 342.30 = 0.11686 kmol/h 60 kg/h water / 18.015 = 3.33056 kmol/h total 3.4474 kmol/h ; z_sucrose = 0.03390

dryingAir: 50 kmol/h of hot air (real air (N2/O2/Ar)) at 200 °C, 1 bar. Dry air (no inlet humidity) for a clean wet-bulb demo.

The dryer atomises the feed with a 0.1 m disc spinning at 20 000 rpm.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	dryingAir	F	0.0138888888889
stream	exhaustAir	F	0.0148058483113
stream	feed	F	0.000957611111111
stream	powder	F	4.06516887181e-05
kpi	dryer	Nu	2.24583682227
kpi	dryer	Pr	0.746977616919
kpi	dryer	Re	0.203916784363
kpi	dryer	Sc	0.6026290984
kpi	dryer	Sh	2.22885536787
kpi	dryer	T_out	372.025243986
kpi	dryer	T_wetbulb	318.053221267
kpi	dryer	X_critical	0.15
kpi	dryer	X_equilibrium	0.0132756494842
kpi	dryer	X_final	0.0132756494842

. . . and 36 more reference values in the case's `expected`.

sprayDryer07_design

tutorials/steady/drying/sprayDryer07_design

What it does: Spray dryer DESIGN: give targets (powder size, residence), the sizing pass inverts to the geometry

The story: spray dryer: DESIGN mode – targets in, the sizing pass inverts to geometry.

feed: 100 kg/h of a 40 wt% sucrose solution at 25 °C. 40 kg/h sucrose / 342.30 = 0.11686 kmol/h 60 kg/h water / 18.015 = 3.33056 kmol/h total 3.4474 kmol/h ; z_sucrose = 0.03390

dryingAir: 50 kmol/h of hot air (real air (N2/O2/Ar)) at 200 °C, 1 bar. Dry air (no inlet humidity) for a clean wet-bulb demo.

The dryer atomises the feed with a 0.1 m disc spinning at 20 000 rpm.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`dryingAir`	`F`	`0.0138888888889`
`stream`	`exhaustAir`	`F`	`0.0148058483113`
`stream`	`feed`	`F`	`0.000957611111111`
`stream`	`powder`	`F`	`4.06516887181e-05`
`kpi`	`dryer`	`Nu`	`2.24583682227`
`kpi`	`dryer`	`Pr`	`0.746977616919`
`kpi`	`dryer`	`Re`	`0.203916784363`
`kpi`	`dryer`	`Sc`	`0.6026290984`
`kpi`	`dryer`	`Sh`	`2.22885536787`
`kpi`	`dryer`	`T_out`	`372.025243986`
`kpi`	`dryer`	`T_wetbulb`	`318.053221267`
`kpi`	`dryer`	`X_critical`	`0.15`
`kpi`	`dryer`	`X_equilibrium`	`0.0132756494842`
`kpi`	`dryer`	`X_final`	`0.0132756494842`
. . . and 36 more reference values in the case's `expected`*.*			

economics

The theory you need. Sizing (stirred tanks, shell-and-tube areas) feeds costing (Guthrie modules); prices are CASE data (`constant/economics`, dated and cited) – the engine refuses to invent a price. Expect honest refusal when a price is missing, never a silent default.

`economics01_esterification_dcf`

`tutorials/steady/economics/economics01_esterification_dcf`

What it does: Esterification plant: CAPEX (Guthrie) → OPEX (COM_d) → discounted-cash-flow appraisal (NPV/IRR/payback)

The story: A FIXED (designed) esterification plant, sized for a full economic appraisal rather than an optimisation:

feed -→ reactor (CSTR) -→ heater -→ separator (flash) |→ liqProd (ethyl acetate rich) |→ vapProd

The feed is scaled to plant throughput so the reactor volume and the heat-exchanger area land INSIDE the Turton correlation validity ranges (V in [0.3, 520] m^3, A in [10, 1000] m^2) – the resulting CAPEX is a physically meaningful base for the DCF, not an extrapolated artefact.

Prices (product/raw-material/labour) live in constant/economics, each dated and primary-cited. The postDict chain is sizing → costing → economics; the economics pass prints the COM_d / NPV / IRR / payback appraisal and publishes the headline scalars into the economics KPIs.

A fixed (designed) ethyl-acetate plant taken all the way from equipment sizing to a discounted-cash-flow economic appraisal:

1. sizing — StirredTank for the CSTR, ShellTubeHX for the heater, sized from each unit's KPIs. 2. costing — Guthrie/Turton purchased + bare-module + total-module cost, CEPCI-updated to the target year, in EUR. 3. economics — the new EconomicsPass: it builds the CAPEX ladder (FCI, working capital, TCI), the Turton cost-of-manufacture COM_d, revenue, a straight-line-depreciation cash-flow timeline, then NPV, IRR (bisection bracket + Newton polish), and discounted + simple payback.

Every cost factor and DCF assumption is dict-owned and printed — read the appraisal block and you can re-derive every number by hand.

*Depreciation enters the DCF exactly once (it is *not* in COM_d) — the classic textbook double-count is avoided. Working capital is an outflow at startup and is recovered untaxed at end-of-life.*

Market/time-specific data — product and feed prices, labour rate, FX — lives only in constant/economics, each value dated and primary-cited (an index/bulletin/agency named in a provenance{} block). The EconomicsPass refuses to run if a required price is absent, with a remedy message; set economics.refuseOnMissingPrice 0; in the postDict to switch to defaulted-but-loud. No price ever defaults to a literal in code.

The economics pass also publishes its headline scalars into result.kpis["economics"] (IRR, NPV, paybackYears, FCI, TCI, COM_d), which is what lets a sweep or optimisation read economics as an ordinary response — see economics02_irr_sweep.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.138888888889`
`stream`	`hotStream`	`F`	`0.138888888889`
`stream`	`liqProd`	`F`	`0.126899267211`
`stream`	`reactorOut`	`F`	`0.138888888889`
`stream`	`vapProd`	`F`	`0.011989621678`
`kpi`	`economics`	`COM_d`	`221241889.353`
`kpi`	`economics`	`FCI`	`407855.018165`
`kpi`	`economics`	`NPV`	`-964539867.664`
`kpi`	`economics`	`TCI`	`469033.27089`
`kpi`	`economics`	`WC`	`61178.2527248`
`kpi`	`economics`	`revenue`	`22620564.309`
`kpi`	`heater`	`F`	`0.138888888889`
`kpi`	`heater`	`P`	`101325`
`kpi`	`heater`	`Q`	`300000`

. . . and 43 more reference values in the case's `expected`*.*

`economics02_irr_sweep`

`tutorials/steady/economics/economics02_irr_sweep`

What it does: Sweep reactor volume; read economics.IRR/NPV/payback per point (SweepDriver runs the post chain)

The story: A FIXED (designed) esterification plant, sized for a full economic appraisal rather than an optimisation:

feed -→ reactor (CSTR) -→ heater -→ separator (flash) |→ liqProd (ethyl acetate rich) |→ vapProd

The feed is scaled to plant throughput so the reactor volume and the heat-exchanger area land INSIDE the Turton correlation validity ranges (V in [0.3, 520] m^3, A in [10, 1000] m^2) – the resulting CAPEX is a physically meaningful base for the DCF, not an extrapolated artefact.

Prices (product/raw-material/labour) live in constant/economics, each dated and primary-cited. The postDict chain is sizing → costing → economics; the economics pass prints the COM_d / NPV / IRR / payback appraisal and publishes the headline scalars into the economics KPIs.

The same esterification plant as economics01, but driven by a sensitivity sweep over the reactor volume that reads economic responses at every point:

The SweepDriver now runs the full post-processing chain (sizing → costing → economics) on each converged point, so the headline economic scalars the EconomicsPass publishes into result.kpis["economics"] resolve as ordinary unit.kpi responses. A sweep can finally ∗see cost∗:

A bigger reactor raises conversion (more product, more revenue) but raises CAPEX and feed throughput — the IRR falls monotonically over this range, a trade-off invisible from a single CAPEX number.

> The range floors at 4 m^3 on purpose: below ~3 m^3 the lower conversion > drops the mixture bubble point and the fixed-duty heater would cross the > saturation dome (a heater is sensible-only) — the deliberate validity edge.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.138888888889`
`stream`	`hotStream`	`F`	`0.138888888889`
`stream`	`liqProd`	`F`	`0.125841050463`
`stream`	`reactorOut`	`F`	`0.138888888889`
`stream`	`vapProd`	`F`	`0.013047838426`
`kpi`	`economics`	`COM_d`	`221264557.696`
`kpi`	`economics`	`FCI`	`460663.338135`
`kpi`	`economics`	`NPV`	`-952396923.312`
`kpi`	`economics`	`TCI`	`529762.838855`
`kpi`	`economics`	`WC`	`69099.5007202`
`kpi`	`economics`	`revenue`	`25155181.8032`
`kpi`	`heater`	`F`	`0.138888888889`
`kpi`	`heater`	`P`	`101325`
`kpi`	`heater`	`Q`	`300000`

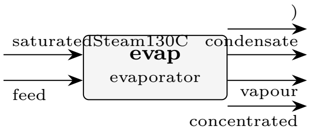
. . . and 43 more reference values in the case's `expected`*.*

evaporation

The theory you need. An evaporator boils solvent off a solution. Two pieces of physics beyond a flash: boiling-point elevation, $T_{\mathrm{boil}} = T_{\mathrm{sat}}(P) + \mathrm{BPE}$ with the BPE from the electrolyte activity of water ($\ln a_w = -\varphi\,\nu\,m\,M_w/1000$); and the heat of dilution/concentration (the relative apparent molar enthalpy $L_\varphi(m,T)$) entering the duty. Expect: the BPE to GROW as the brine concentrates, steam economy below 1 for a single effect, and – for NaCl – the full Silvester–Pitzer T-dependence of $\gamma_\pm$ shaping the answer between 25 and 200°C. *Full derivation: Theory Guide, the electrolyte chapters.*

evaporator01_brine

tutorials/steady/evaporation/evaporator01_brine



What it does: Single-effect evaporator: 35 g/L brine at 1 bar; the fixed steam duty/UA evaporates ~82 % of the feed water (V/F=0.82), enriching NaCl ~5.5x (x 0.011 → 0.059) – duty-driven, not a target (BPE included)

The story: single-effect brine concentrator (NaCl-water), PHYSICAL-EQUIPMENT model (v0.29+).

We say WHAT THE HARDWARE IS (heat-exchange area, overall HTC, operating pressure on the process side, heating-steam temperature) and the simulator tells us WHAT IT DOES (V/F, duty Q, T_boil with BPE, steam demand, economy). There is no "vaporFraction" or "duty" spec to choose — the equipment shape itself IS the spec.

Feed: seawater-equivalent 35 g/L NaCl in water, 100 kmol/h (~ 1.8 kg/s of seawater, the volume of a small medium-capacity desalination unit), 25 °C. At MW_NaCl = 58.44 g/mol and rho_water ~ 1 g/cm^3:

35 g/L / 58.44 g/mol = 0.599 mol NaCl per litre water = 0.599 mol/kg water (dilute approx) x_NaCl_feed ≈ 0.599 / (0.599 + 55.5) = 0.01067

Heating side: saturated steam at 130 °C (= 403.15 K, P ≈ 2.7 bar).

Equipment: heat-exchange area = 20 m^2 overall U = 2000 W/(m^2·K) operating pressure = 1 bar (process side)

Expected order-of-magnitude (computed by the solver, not specified): T_sat(water, 1 bar) ≈ 373 K (~ 100 °C) ΔT_drive (steam - process) ≈ 30 K Q = U · A · ΔT ≈ 2000 · 20 · 30 ≈ 1200 kW V evaporated ≈ Q / ΔHvap ≈ 110 kmol/h ≈ 1980 kg/h water V/F (mol-basis) ≈ 0.82 (the CONVERGED run; the rough hand-estimate above is order-of-magnitude only) Steam consumption ≈ Q / ΔHvap_at_steam ≈ 2000 kg/h Economy ≈ V_evap / F_steam ≈ 0.99 x_NaCl_outlet ≈ 0.059 (~5.5x the feed, matching the run header)

Industrial relevance: this is the building block of multi-effect evaporators in seawater desalination (MED, MSF), brine crystallisers, and the sugar industry. Real plants run under vacuum (T_boil ~ 60 °C at 0.2 bar) to minimise scaling and use BPE-loaded multi-effect cascades to recover the latent heat — see evaporator02_triple_effect_sugar.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	concentrated	F	0.00498887089392
stream	condensate	F	0.0277777777778
stream	feed	F	0.0277777777778
stream	saturatedSteam130C	F	0.0277777777778
stream	vapour	F	0.0227889068839
kpi	evap	A	20
kpi	evap	A_m2	20
kpi	evap	BPE	1.79842779271
kpi	evap	F_conc	0.00498887089392
kpi	evap	F_conc_kmol_h	17.9599352181
kpi	evap	F_conc_mass	0.101856029987
kpi	evap	F_cond	0.500416666667
kpi	evap	F_cond_kg_h	1801.5
kpi	evap	F_in	0.0277777777778

... and 24 more reference values in the case's `expected`*.*

evaporator02_triple_effect_sugar

tutorials/steady/evaporation/evaporator02_triple_effect_sugar

What it does: Triple-effect cocurrent evaporator: 22500 kg/h sugar solution 5%→25% (textbook Example 10.4)

The story: SOURCE. The exercise number below is the numbering of the course notes this case was built from; the TEXTBOOK those notes drew on is not recorded anywhere in this case. It used to be labelled with a course acronym, and that acronym named the WRONG course (a CFD one), so it was removed on 2026-08-12 rather than replaced with a guess. The gap is left visible: a missing citation a reader can see beats a plausible one nobody can check. (‘sucrose.dat‘ does carry a primary for the Cp correlation – Coulson & Richardson 6th ed. Example 14.4.)

textbook Example 10.4: triple-effect cocurrent evaporator concentrating a 22 500 kg/h sucrose solution from 5 wt% sucrose toward ~ 25 wt%, using 200 kPa saturated steam to drive Effect 1 and a 15 kPa vacuum on Effect 3.

Cascade (cocurrent: liquid 1→2→3, generated vapour drives the next effect’s heating side):

F=22 500 kg/h L1 L2 L3 x=0.05 →[1] ——→ [2] ——→ [3] —-→ ^ ^ ^ | | | Steam V1 V2 V3 → condenser 200 kPa (Effect 1’s (Effect 2’s T_sat~393K vapour drives vapour drives Effect 2) Effect 3) P1 = 80 kPa P2 = 45 kPa P3 = 15 kPa

PHYSICAL-EQUIPMENT model (v0.29+). Each effect is its own physical evaporator with an ‘area‘ and a ‘U‘; the simulator computes V/F, Q, T_boil and steam demand from the energy balance. Effect 1 consumes utility steam; the vapour V1 that Effect 1 produces is declared as the heating-side input to Effect 2, and similarly V2 → Effect 3. The flowsheet runs SEQUENTIALLY left-to-right: no outer iteration is needed because the inputs of each effect are produced by the preceding one (or by the utility stream).

Equipment we picked (the "spec"): A_1 = A_2 = A_3 = 100 m^2 U_1 = 2319 W/(m^2·K) (Effect 1, clean steam) U_2 = 2194 W/(m^2·K) (Effect 2) U_3 = 1296 W/(m^2·K) (Effect 3, lowest pressure → lowest k)

The textbook target L_3 ~ 4500 kg/h at x = 0.25 corresponds to evaporating ~ 18 000 kg/h total water; that target is NOT enforced here — the equipment dimensions ARE the spec, the outlet concentration is the RESULT. A later tutorial wraps this case in a DesignSpec block that manipulates A_i (or, more commonly, U_i at fixed A_i = A) to drive L_3 to the design target.

1. Cocurrent multi-effect literally just chains single-effect evaporators. Choupo’s flowsheet engine handles the cascading naturally — the V_i stream of Effect i appears as the heating-side input of Effect i+1, and the V_i mass flow is computed by Effect i’s energy balance.

2. Economy: in single-effect (evaporator01_brine) the economy was ~ 0.83 kg evap / kg steam. Running this cascade gives total V_evap / utility steam ~ 2.0 (= 17 972 / 8 917) — more than a doubling, simply by chaining the heat through three effects. Industrial design targets ~ 2.5 with optimised area allocation and pressure cascade.

3. BPE matters more in Effect 3 (most concentrated solute, lowest U); the student SEES T_boil deviate further from T_sat(water, P_3) in the per-effect run header.

Honest simplification for v0.29:

The Evaporator unit op reads only T from its heating-steam input (and ignores the F). This means the F of V_1 that effect 2 actually CONSUMES on its heating chest is computed from effect 2’s own energy balance and is NOT constrained to equal the F of V_1 that effect 1 PRODUCED. In a real plant those two flows must close (all of V_1 condenses in effect 2’s chest, with no other path). Here the v0.29 model treats the three energy balances independently — there is a residual mismatch in the V_i mass that the cascade quietly absorbs.

Closing the inter-effect mass balance turns the cascade into a coupled system (one constraint per link) that wants an outer DesignSpec wrap: manipulate (T__boil__i) or (A__i) to enforce "V__i produced == V__i consumed at effect i+1's chest". That is the v0.30 multi-effect tutorial.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	L1	F	0.243400997575
stream	L2	F	0.150822370746
stream	L3	F	0.0533879418767
stream	V1	F	0.0870984081237
stream	V2	F	0.0925786268285
stream	V3	F	0.0974344288697
stream	cond1	F	0.1375
stream	cond2	F	0.0870984081237
stream	cond3	F	0.0925786268285
stream	feed	F	0.330499405699
stream	utilitySteam_200kPa	F	0.1375
kpi	effect1	A	100
kpi	effect1	A_m2	100
kpi	effect1	BPE	0.107198891506
... and 92 more reference values in the case's `expected`*.*			

evaporator03_co_current

tutorials/steady/evaporation/evaporator03_co_current

What it does: Co-current forward-feed triple-effect evaporator, user-defined components (textbook problem 10.38, case a)

The story: SOURCE. The exercise number below is the numbering of the course notes this case was built from; the TEXTBOOK those notes drew on is not recorded anywhere in this case. It used to be labelled with a course acronym, and that acronym named the WRONG course (a CFD one), so it was removed on 2026-08-12 rather than replaced with a guess. The gap is left visible: a missing citation a reader can see beats a plausible one nobody can check. (‘sucrose.dat‘ does carry a primary for the Cp correlation – Coulson & Richardson 6th ed. Example 14.4.)

triple-effect evaporator, forward-feed, user-defined components, textbook Example 10.38 (a).

Problem (textbook givens): Feed 1.25 kg/s, 10 wt% solids, 294 K Live steam 170 kPa (saturated vapour) P__3 such that effect 3 boils at 325 K => P__3 = 13.5 kPa U__1 3100 W/(m^2·K) U__2 2500 W/(m^2·K) U__3 1700 W/(m^2·K) Spec productConc = 40 wt% solids Equal areas A__1 = A__2 = A__3 = A

Question (in system/outerDict): find A, P__1, P__2 and the live- steam consumption.

Wiring (liquid + vapour BOTH flow 1 → 2 → 3):

feed (294 K, 10 %) productConc (40 %) | ˆ v | +———+ L1to2 +———+ L2to3 +———+ | effect1 |———→| effect2 |———→| effect3 | | P__1 | | P__2 | | P__3 | +—-+—-+ +—-+—-+ +—-+—-+ ˆ ˆ ˆ liveSteam V1 (from 1) V2 (from 2) (170 kPa) chest of 2 chest of 3

V__3 → condenser (outside the dict) cond1, cond2, cond3 → condensate terminals

Notes for the student: Thermo properties (MW, Cp, Antoine, ...) of ‘agua‘ and ‘solidos‘ live in constant/-components/<name>.dat — these are USER-DEFINED components, NOT from the standard catalogue. Open them to see the textbook’s Cp = 4.18 kJ/(K·kg) assumption baked in, and to confirm BPE is set to zero (no K__b on ‘agua‘). Forward-feed with COLD feed (294 K) penalises effect 1: most of its duty goes into sensible heating, not evaporation. Case (b) of the textbook pre-heats the feed to 355 K — swap T__feed below to compare economies.

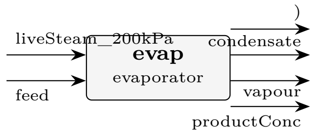
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	L1to2	F	0.0469536109199
stream	L2to3	F	0.0295647943065
stream	V1	F	0.0161887935578
stream	V2	F	0.0173888166134
stream	V3	F	0.018462595509
stream	cond1	F	0.0262363636123
stream	cond2	F	0.0161887935578
stream	cond3	F	0.0173888166134
stream	feed	F	0.0631424044778
stream	liveSteam	F	0.0262363636123
stream	productConc	F	0.0111021987975
kpi	effect1	A	16.5383245281
kpi	effect1	A_m2	16.5383245281
kpi	effect1	BPE	0

. . . and 92 more reference values in the case’s `expected`*.*

evaporator04_orange_juice_simple

tutorials/steady/evaporation/evaporator04_orange_juice_simple



What it does: Single-effect evaporator concentrating orange juice 10% to 50% solids (textbook Example 10.2 a-c, without BPE)

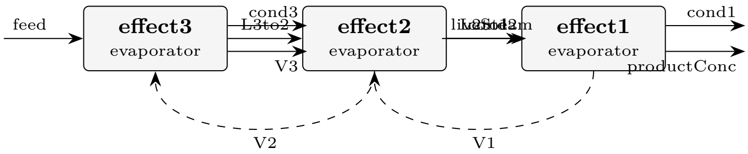
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	condensate	F	0.360686102898
stream	feed	F	0.350791135988
stream	liveSteam_200kPa	F	0.360686102898
stream	productConc	F	0.0424059629635
stream	vapour	F	0.308385173024
kpi	evap	A	75.5133330137
kpi	evap	A_m2	75.5133330137
kpi	evap	BPE	0
kpi	evap	F_conc	0.0424059629635
kpi	evap	F_conc_kmol_h	152.661466669
kpi	evap	F_conc_mass	1.38888555242
kpi	evap	F_cond	6.49776014371
kpi	evap	F_cond_kg_h	23391.9365174
kpi	evap	F_in	0.350791135988

... and 24 more reference values in the case's `expected`.

evaporator05_counter_current

tutorials/steady/evaporation/evaporator05_counter_current



What it does: Counter-current backward-feed triple-effect evaporator (textbook problem 10.38 variant)

The story: SOURCE. The exercise number below is the numbering of the course notes this case was built from; the TEXTBOOK those notes drew on is not recorded anywhere in this case. It used to be labelled with a course acronym, and that acronym named the WRONG course (a CFD one), so it was removed on 2026-08-12 rather than replaced with a guess. The gap is left visible: a missing citation a reader can see beats a plausible one nobody can check. (‘sucrose.dat‘ does carry a primary for the Cp correlation – Coulson & Richardson 6th ed. Example 14.4.)

COUNTER-CURRENT (BACKWARD-FEED) triple-effect evaporator, Mode-2, credo-pure.

Problem (textbook givens, textbook problem 10.38 numerics, counter-current variant): Feed 1.25 kg/s, 10 wt% solids, 294 K Live steam 170 kPa (saturated vapour) entering EFFECT 1 (hot end) P_3 (cold end) such that T_3 = 325 K → 13.5 kPa U_1 3100 W/(m^2·K) U_2 2500 W/(m^2·K) U_3 1700 W/(m^2·K) Spec productConc = 40 wt% solids (exits from effect 1) Equal areas A_1 = A_2 = A_3 = A (single $A by reference)

Wiring (LIQUID flows 3 → 2 → 1 ; VAPOUR flows 1 → 2 → 3 ; OPPOSITE directions => counter-current):

feed (294 K, 10 %) productConc (40 %, exits effect 1) | ^ v | +———+ L3to2 +———+ L2to1 +———+ | effect3 |<———-| effect2 |<———-| effect1 | | COLD | | MID | | HOT | +—-+—-+ +—-+—-+ +—-+—-+ ^ ^ ^ | | | V2 (from 2) V1 (from 1) liveSteam chest of 3 chest of 2 (170 kPa)

v V3 → condenser cond1, cond2, cond3 → condensate terminals

Both V_1 and V_2 flow OPPOSITE to the liquid: effect 1 produces V_1 which feeds effect 2’s chest; effect 2 produces V_2 which feeds effect 3’s chest. In the directed graph this creates a CYCLE:

effect3 –L3to2-→ effect2 –L2to1-→ effect1 ^ | | V2 V1 | | v +– effect2 <– effect1 (V1 chest) +

So we DECLARE V_1 and V_2 as TEAR STREAMS. The flowsheet executive runs effect3 → effect2 → effect1 once per pass, with the initial V_1 / V_2 guesses from the streams block; at end of pass the newly-produced V_1, V_2 are compared with the old. Wegstein (or plain direct substitution) iterates until they match.

In Mode 2 each effect computes T_boil and P_op locally from the chest T and the hardware (A, U) — so the only thing that the Wegstein loop actually iterates is V_i.F (the produced flows). T and P_op stabilise IMMEDIATELY (single pass). Convergence in ~3-5 Wegstein passes.

Design unknowns (in outerDict): $A common heat-exchange area $F_steam live-steam flow

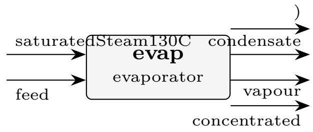
Constraints: productConc.F_mass = 0.3125 kg/s (40 wt%) effect3.P = 13.5 kPa (textbook terminal vacuum)

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	L2to1	F	0.0319496046345
stream	L3to2	F	0.0496904316429
stream	V1	F	0.020847035226
stream	V2	F	0.0177408270084
stream	V3	F	0.0134519728349
stream	cond1	F	0.0227907734916
stream	cond2	F	0.0208470355984
stream	cond3	F	0.0177408244446
stream	feed	F	0.0631424044778
stream	liveSteam	F	0.0227907734916
stream	productConc	F	0.0111025694085
kpi	effect1	A	17.1719452001
kpi	effect1	A_m2	17.1719452001
kpi	effect1	BPE	0
. . . and 92 more reference values in the case's `expected`.			

evaporator06_nacl_pitzer

tutorials/steady/evaporation/evaporator06_nacl_pitzer



What it does: Single-effect NaCl evaporator with the boiling-point elevation from the PITZER electrolyte model (a_w), not the ideal K_b*m. Concentrated brine → BPE several degrees, the real value.

The story: / flowsheetDict Single-effect NaCl evaporator whose boiling-point elevation comes from the PITZER water activity (a_w), not the ideal K_b*m: concentrated brine is a REAL electrolyte, and the BPE follows from the osmotic coefficient. The thermo arrives via the aqueousNaCl_pitzer property package (see constant/thermoPhysPropDict) – swap to evaporator07's eNRTL to compare.

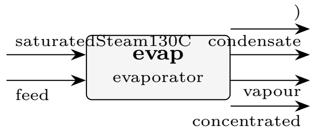
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	concentrated	F	0.0203403273098
stream	condensate	F	0.0125
stream	feed	F	0.0277777777778
stream	saturatedSteam130C	F	0.0125
stream	vapour	F	0.00743745046799
kpi	evap	A	20
kpi	evap	A_m2	20
kpi	evap	BPE	5.06216808829
kpi	evap	F_conc	0.0203403273098
kpi	evap	F_conc_kmol_h	73.2251783152
kpi	evap	F_conc_mass	0.422576829819
kpi	evap	F_cond	0.2251875
kpi	evap	F_cond_kg_h	810.675
kpi	evap	F_in	0.0277777777778

. . . and 26 more reference values in the case's `expected`*.*

evaporator07_nacl_enrtl

tutorials/steady/evaporation/evaporator07_nacl_enrtl



What it does: Single-effect NaCl evaporator with the boiling-point elevation from the eNRTL water activity (electrolyte-NRTL) – the same slot as model pitzer, one word changed. eNRTL is the model for salt in a non-ideal molecular solvent.

The story: / flowsheetDict Single-effect NaCl evaporator whose boiling-point elevation comes from the eNRTL water activity (a_w), not the ideal K_b*m – the SAME flowsheet as evaporator06, with only the electrolyte model swapped (pitzer → eNRTL): run both and compare the BPE the two G^E surfaces predict.

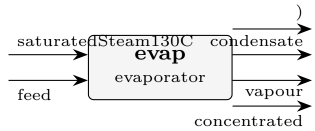
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	concentrated	F	0.0200845615144
stream	condensate	F	0.0125
stream	feed	F	0.0277777777778
stream	saturatedSteam130C	F	0.0125
stream	vapour	F	0.00769321626341
kpi	evap	A	20
kpi	evap	A_m2	20
kpi	evap	BPE	3.86220786107
kpi	evap	F_conc	0.0200845615144
kpi	evap	F_conc_kmol_h	72.3044214517
kpi	evap	F_conc_mass	0.417969209015
kpi	evap	F_cond	0.2251875
kpi	evap	F_cond_kg_h	810.675
kpi	evap	F_in	0.0277777777778

. . . and 25 more reference values in the case's `expected`.

evaporator08_naoh_dilution_heat

`tutorials/steady/evaporation/evaporator08_naoh_dilution_heat`



What it does: Single-effect NaOH evaporator: the heat of dilution IN the duty (fitted L_phi, Parker-validated)

The story: the heat of dilution, IN the duty

Single-effect caustic (NaOH) evaporator – the McCabe enthalpy-concentration classic. NaOH is the FIRST calorimetrically fitted salt (Silvester-Pitzer dbeta/dT vs Parker NSRDS-NBS 2, rel-AAD 4.97%), so this evaporator's duty CLIMBS THE MEASURED L_phi CURVE: concentrating caustic absorbs extra heat beyond sensible+latent, and the run prints it as its own named line:

heat of dilution = X kW (L_phi: m a → b mol/kg, calorimetric fit)

Contrast evaporator06 (whose CASE-LOCAL pair overlay omits the T-slots, so ITS NaCl runs calorimetricFit false – the standards Na-Cl record is fitted since the Parker refit): same machinery, but there the dilution heat is OMITTED and the run says so loudly. The feed (m ~ 2.0) concentrates to m ~ 4-5 mol/kg – inside the Parker fit window (<= 6), so no extrapolation flag; push the steam rate up and watch the [electrolyte] EXTRAPOLATED advisory fire.

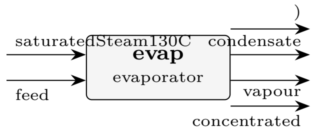
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	concentrated	F	0.0118365226457
stream	condensate	F	0.0208333333333
stream	feed	F	0.0277777777778
stream	saturatedSteam130C	F	0.0208333333333
stream	vapour	F	0.0159412551321
kpi	evap	A	20
kpi	evap	A_m2	20
kpi	evap	BPE	6.45099255003
kpi	evap	F_conc	0.0118365226457
kpi	evap	F_conc_kmol_h	42.6114815246
kpi	evap	F_conc_mass	0.234606344352
kpi	evap	F_cond	0.3753125
kpi	evap	F_cond_kg_h	1351.125
kpi	evap	F_in	0.0277777777778

. . . and 26 more reference values in the case's `expected`*.*

evaporator09_nacl_sucrose_brine

tutorials/steady/evaporation/evaporator09_nacl_sucrose_brine



What it does: Evaporating a brine that also carries sugar: the electrolyte BPE is computed on the salt molality, and the rest is announced, not absorbed

The story: / flowsheetDict Single-effect NaCl evaporator whose boiling-point elevation comes from the PITZER water activity (a_w), not the ideal K_b*m: concentrated brine is a REAL electrolyte, and the BPE follows from the osmotic coefficient. The thermo arrives via the aqueousNaCl_pitzer property package (see constant/thermoPhysPropDict) – swap to evaporator07's eNRTL to compare.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	concentrated	F	0.0211538965752
stream	condensate	F	0.0125
stream	feed	F	0.0281111111111
stream	saturatedSteam130C	F	0.0125
stream	vapour	F	0.00695721453587
kpi	evap	A	20
kpi	evap	A_m2	20
kpi	evap	BPE	4.91270305537
kpi	evap	F_conc	0.0211538965752
kpi	evap	F_conc_kmol_h	76.1540276709
kpi	evap	F_conc_mass	0.545327280136
kpi	evap	F_cond	0.2251875
kpi	evap	F_cond_kg_h	810.675
kpi	evap	F_in	0.0281111111111

. . . and 26 more reference values in the case's `expected`.

flash

The theory you need. An isothermal flash splits a feed of composition z_i into vapour y_i and liquid x_i at fixed (T, P). Phase equilibrium is $y_i = K_i x_i$ with $K_i = \gamma_i P_i^{\text{sat}}/P$ (γ–φ formulation), and the vapour fraction β solves the Rachford–Rice equation

$$\sum_i \frac{z_i\,(K_i - 1)}{1 + \beta\,(K_i - 1)} = 0,$$

monotone in β – Newton converges in a handful of iterations, all of them visible at verbosity 3. Expect: a bubble point when the equation's root hits $\beta = 0$, a dew point at $\beta = 1$, and non-ideal systems ($\gamma_i \neq 1$: NRTL/Wilson/UNIQUAC/UNIFAC) bending K_i with composition. Liquid–liquid and three-phase cases replace successive-substitution with direct Gibbs-energy minimisation (the symmetric γ-model trap: Newton collapses onto the trivial $K = 1$ root). *Full derivation: Theory Guide, the VLE and flash chapters.*

`adiabaticFlash01_benzene_toluene`

`tutorials/steady/flash/adiabaticFlash01_benzene_toluene`

What it does: Adiabatic flash, benzene/toluene 40/60, 380K/3bar → 1atm

The story: Adiabatic flash: sub-cooled 40/60 benzene/toluene liquid at 380 K, 3 bar expanded to 1 atm. ΔT and V/F come from the energy balance.

The outlet pressure is a plain editable value – tweak it in the GUI's Properties box (transient tinkering) and re-run, or edit it here.

A benzene/toluene liquid (40/60 mol, 100 kmol/h) at 380 K and 3 bar is let down to 1 atm across a drum with no heat added. Nothing tells the drum what temperature to settle at. The energy balance does: the outlet comes out at 368.66 K, an 11.34 K drop, with 5.19 % of the feed flashed to vapour (5.19 kmol/h vapour, 94.81 kmol/h liquid). Every number here is the case's own golden (expected), which bin/runTests re-checks.

1. Q = 0 is a definition, not a setting. The dict declares only the outlet pressure (system/flowsheetDict, operation { P 1.01325 bar; }). There is no T to give; an adiabatic drum has none to receive. 2. Two Newtons, one inside the other. The outer one walks T until the outlet enthalpy equals the inlet's; at every trial T the inner one solves the Rachford-Rice flash. Watch it in the Log tab — each outer step prints its H-residual, and the last one prints H_residual at the round-off floor (-2.2e-3 W on a 100 kmol/h stream). 3. Where the 11 K went. Vaporising 5 % of the feed costs latent heat, and the only place it can come from is the sensible heat of the liquid. Open Streams: the vapour is benzene-rich, the liquid barely moved in composition, and both leave at the same 368.66 K. 4. Compare with flash01_benzene_toluene — the same feed held at a ∗fixed∗ 370 K. There the temperature is an input and the drum needs a duty to hold it. The two cases are the same split asked two ways.

5. Read the caveat the engine prints — it is about THIS case. The feed is at 380 K, and benzene's Antoine fit is declared for 287.7–354.07 K. The run says so on the console ([psat] ? OUTSIDE its declared Trange) and again in the caveat block at the end: the vapour pressure was ∗extrapolated∗, still returned, and the answer stands with that stated. A number you were warned about is a number you can defend; one you were not is the dangerous kind.

Edit operation { P ? } to 0.5 bar in the Case tab and run again: a deeper let-down flashes more, and the drum ends colder. Then look at the stream table printed ∗before∗ the solve — it shows the 0/ seed values, which look like an answer and are not one; the converged table further down is the answer.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`flashAdiab`	`T_out`	`368.661803634`
`kpi`	`flashAdiab`	`V_over_F`	`0.0518811133842`
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`liquid`	`F`	`0.0263366357393`
`stream`	`vapor`	`F`	`0.00144114203845`
`stream`	`feed`	`T`	`380`
`stream`	`liquid`	`T`	`368.661803634`
`stream`	`vapor`	`T`	`368.661803634`
`kpi`	`flashAdiab`	`F_in`	`0.0277777777778`
`kpi`	`flashAdiab`	`F_liquid`	`0.0263366357393`
`kpi`	`flashAdiab`	`F_vapor`	`0.00144114203845`
`kpi`	`flashAdiab`	`H_residual`	`-0.00215364377073`
`kpi`	`flashAdiab`	`P`	`101325`
`kpi`	`flashAdiab`	`T_drop`	`11.3381963657`
. . . and 1 more reference values in the case's `expected`.			

analysis01_water_analysis_inlet

tutorials/steady/flash/analysis01_water_analysis_inlet

What it does: A laboratory water analysis as a first-class inlet: mg/L with alkalinity as CaCO3, a measured density, and a +0.78 % ion imbalance reconciled onto chloride within a declared 10 % limit – measurement, reconciled inventory and equilibrium recorded as three separate layers

The story: A LABORATORY ANALYSIS AS AN INLET.

The inlet of this flash is not a composition somebody worked out: it is a water analysis as a laboratory sends it – mg/L, alkalinity reported as CaCO3, a measured density, per-line uncertainties, and an ion balance that is off by 0.78 %.

THREE LAYERS, SEPARATELY RECORDED, and the separation is the whole point:

1 0/feed the MEASUREMENT. Never rewritten by a run (ratified 2026-08-09, O1) – a reconciliation that edited it would destroy the thing it was reconciling, and the next run would reconcile the reconciliation. 2 converged/feed calculated { analysisReconciliation { ? } conservedInventory { ? } } the RECONCILED conserved inventory, with the imbalance before, the imbalance after, and every correction that was applied. This is the layer the engine did not have: an answer never says what it was corrected from. 3 the flash the EQUILIBRIUM solved from layer 2.

WHAT IS MEASURED HERE (the numbers the gate re-derives independently):

ion balance BEFORE +0.7813 % (cations minus anions, over the sum) ion balance AFTER 0.0000 % chloride adjusted +5.1208 % of the measured 45.0 mg/L limit declared 10 % – exceeded, it REFUSES with the numbers

FIVE REFUSALS GUARD IT, each by name, and check_aqueous_analysis.py fires every one through the real reader: no DENSITY ROUTE – mg/L is per volume, a stream is a flow; an ITERATIVE density is a declared A1 gap, refused rather than done silently; a correction OVER THE LIMIT – past the declared bound it stops being a reconciliation and becomes the analysis rewritten to fit; pH NAMED AS THE BALANCING VARIABLE – pH may participate only under an explicit declared rule, and that grammar is A2's; ‘method weightedLeastSquares;‘ – THE NAMED NEXT SLICE. Refused by name rather than silently approximated by the one method A1 has; TWO MATERIAL FORMS in one file – the seventh canonical form is exclusive with the other six, same as they are with each other.

And with no ‘reconciliation {}‘ block at all: an analysis already closing within the tolerance passes through with an announcement stating what was measured; one that does not REFUSES, naming the two legal remedies. What never happens is an adjustment nobody asked for.

Contract: docs/design/aqueous-analysis-inlet-scope.md (slice A1). Compare: flash18_water_analysis_basis – the same water as an INVENTORY, where charge closure is a contract and a violation is refused.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0307824830015
stream	liquid	F	0.0307824830015
stream	vapor	F	0
kpi	flash01	F_alpha	0.0307824830015
kpi	flash01	F_beta	0
kpi	flash01	F_in	0.0307824830015
kpi	flash01	P	101325
kpi	flash01	T	313.15
kpi	flash01	V_over_F	0
kpi	flash01	pH	7.90774504802
kpi	flash01	p_eq_sum_atm	0.0747733964818
kpi	flash01	phaseSet	0
stream	feed	T	313.15
stream	liquid	T	313.15
... and 1 more reference values in the case's `expected`*.*			

analysis02_weighted_least_squares

tutorials/steady/flash/analysis02_weighted_least_squares

What it does: The same laboratory water as analysis01, reconciled by constrained weighted least squares over four measured quantities under electroneutrality plus an elemental-conservation redundancy: the correction spreads across all four at under 0.29 sigma each instead of shoving 5.12 % into chloride

The story: THE SAME WATER, RECONCILED PROPERLY.

analysis01 closes a +0.78 % ion balance by moving ONE ion: chloride absorbs the whole residue and shifts 5.12 %. That is a legitimate convention and it is what a water chemist does by hand – but it is a CHOICE dressed as an answer. Nothing about the analysis says chloride was the wrong number; what the sheet actually says is that FOUR quantities were measured, each with its own uncertainty, and that they cannot all be right at once.

A2 asks the question the uncertainties make askable:

what is the composition CLOSEST to what was measured – measuring closeness in each quantity's OWN declared uncertainty – that satisfies the conservation laws exactly?

That is constrained weighted least squares, and it is a one-way PRE-STEP:

laboratory analysis → reconciliation → ONE conserved admissible composition → aqueous equilibrium / speciation

The arrow never reverses. The reconciler lives in its own translation unit (src/streams/AnalysisReconciler.{H,cpp}) which includes NOTHING from thermo/, so it cannot name a speciation solver, an activity model or a thermodynamic package – the equilibrium is structurally unable to reach back into the measurements and make the chemistry easier to satisfy.

THE SHEET. A calcium bicarbonate groundwater, the same water analysis01 carries, plus one line a real laboratory almost always also sends:

Ca 84.0 mg/L ICP-OES 2 % Cl 45.0 mg/L IC 5 % alkalinity 143.0 mg/L as CaCO3 titration 4 % totalHardness 208.0 mg/L as CaCO3 EDTA titration 3 % <– NEW

The hardness line is the interesting one. It measures the SAME CALCIUM the ICP measured, by a completely different method, and the two disagree by 0.84 % – which is unremarkable laboratory life and exactly why nobody can say which is right. It is declared ‘element Ca;‘: it carries NO material of its own (that would count the calcium twice), it is a REDUNDANT determination, and its only effect is the law that ties it to the Ca row.

Note what is NOT declared on it. ‘alkalinity ... as CaCO3‘ needs ‘perFormulaUnit 2‘ because nothing in the formula CaCO3 says a titration counts two bicarbonate per unit – that is a CONVENTION. ‘totalHardness { element Ca; as CaCO3; }‘ needs no such key and is refused if it carries one: how many Ca there are in a CaCO3 is ARITHMETIC, and the formula already says it.

TWO LAWS ARE ENFORCED, and both are declared rather than assumed:

electroneutrality Sum z_i n_i = 0 elementalConservation the measured Ca total = the Ca the species rows carry

...plus non-negativity, which is not in the list because it is not optional.

WHAT THE FIT DOES. Two residues have to disappear – +0.0650 meq/L of charge and +0.0177 mmol/L of calcium disagreement – and the fit spends them where the measurements are weakest:

Ca -0.2735 sigma (-0.547 %) Cl +0.1562 sigma (+0.781 %) alkalinity +0.2813 sigma (+1.125 %) totalHardness +0.1000 sigma (+0.300 %) Sum ((x-m)/sigma)^2 = 0.1883

FOUR quantities move, none by a third of its own uncertainty, against analysis01's single 5.12 % shove. The declared limit is 3 sigma and the worst correction is 0.28 – and the run REFUSES with the numbers if any quantity would go past it.

READ 'converged/feed' FOR THE POINT OF THE SLICE. Every corrected quantity states what was reported, what it became, its sigma and weight, its correction IN SIGMA, which law caused it (exactly – the per-law parts sum to the correction, they are not shares invented afterwards), and how much of that law's own residue it removed. And the block opens with a legend saying that NOTHING in it is a chemistry result: the ions and the solved pH are in 'speciation {}', keyed by species rather than by the author's sheet labels, and carrying no 'reportedValue' because nothing in them was reported.

Provenance: a REPRESENTATIVE analysis composed for this witness, not a transcription of a published one. The disagreements are deliberate and are what the case exists to exercise.

Contract: docs/design/aqueous-analysis-inlet-scope.md (slice A2). Compare: analysis01_water_analysis_inlet – the same water, one ion moved; flash18_water_analysis_basis – the same water as an INVENTORY, where charge closure is a contract and a violation is refused.

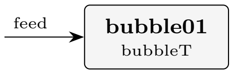
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0307825083878
stream	liquid	F	0.0307825083878
stream	vapor	F	0
kpi	flash01	F_alpha	0.0307825083878
kpi	flash01	F_beta	0
kpi	flash01	F_in	0.0307825083878
kpi	flash01	P	101325
kpi	flash01	T	313.15
kpi	flash01	V_over_F	0
kpi	flash01	pH	7.90867882055
kpi	flash01	p_eq_sum_atm	0.074797651023
kpi	flash01	phaseSet	0
stream	feed	T	313.15
stream	liquid	T	313.15

... and 1 more reference values in the case's `expected`.

bubbleT01_ethanol_water

`tutorials/steady/flash/bubbleT01_ethanol_water`



What it does: Bubble-T of 30/70 ethanol/water at 1 atm (NRTL)

The story: Bubble-T of a 30/70 ethanol/water liquid at 1 atm (NRTL gamma, non-ideal). A saturation calc, not a flash: the unit solves sum(K_i x_i) = 1 for T at the given P (no product streams). Expected: T_bubble ~ 354.4 K with y_ethanol ~ 0.58 – ethanol enriched in the incipient vapour well beyond its 0.30 in the liquid.

30/70 ethanol/water (100 kmol/h) at 1 atm: at what temperature does the first bubble form, and what is in it? The answer from the golden: T_bubble = 354.42 K (81.27 °C), and the first vapour is 58.2 mol% ethanol from a liquid that is 30 % — the enrichment every distillation column lives on.

1. Raoult would get this wrong, so the case declares NRTL. Open constant/thermoPhysPropDict: the liquid activity model is named, and the run announces it at the top of the log (liquid activity.NRTL; vapour idealGas). Nothing is chosen for you. 2. One equation, one unknown. Σ y_i = Σ K_i x_i = 1, solved for T by Newton. The Log tab prints the whole table: five iterations from a 366.6 K guess, driven to 4.4e-12. Count them; read ΔT shrink. 3. K carries the non-ideality. The result table gives K_ethanol = 1.940 and K_water = 0.597 at 354.4 K. With Raoult alone (K = Psat/P) ethanol's K would be far smaller here — the difference is the activity coefficient, and it is why the vapour is ethanol-rich. 4. This is one point on a curve. dewT01_ethanol_water asks the mirror question (Σ x = Σ y/K = 1) for a vapour of the same composition, and the two together bracket the two-phase region at 1 atm.

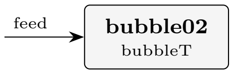
Change the feed to 90/10 ethanol/water in 0/feed and run again: the bubble point moves toward ethanol's boiling point, and the vapour is barely richer than the liquid — you are near the azeotrope, which is what column03_azeotrope_mesh is about.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
kpi	bubble01	P	101325
kpi	bubble01	T_bubble	354.424819099
kpi	bubble01	converged	1
kpi	bubble01	y_ethanol	0.581915403358
kpi	bubble01	y_water	0.418084596646
stream	feed	T	350

`bubbleT02_uniquac_ethanol_water`

`tutorials/steady/flash/bubbleT02_uniquac_ethanol_water`



What it does: Bubble-T of 30/70 ethanol/water at 1 atm (UNIQUAC) – validates the UNIQUAC model against the NRTL/Wilson twins

The story: Bubble-T of a 30/70 ethanol/water liquid at 1 atm (UNIQUAC).

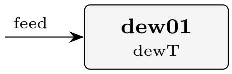
Same conditions as bubbleT01 (NRTL): a strongly non-ideal binary whose bubble temperature and vapour composition are set by the activity coefficients. The UNIQUAC result should land close to the NRTL/Wilson twins (T_bubble ~ 353.5 K, the vapour enriched in ethanol towards the azeotrope, y_ethanol ~ 0.58).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
kpi	bubble02	P	101325
kpi	bubble02	T_bubble	353.469206954
kpi	bubble02	converged	1
kpi	bubble02	y_ethanol	0.582711434226
kpi	bubble02	y_water	0.417288565785
stream	feed	T	350

dewT01_ethanol_water

`tutorials/steady/flash/dewT01_ethanol_water`



What it does: Dew-T of 30/70 ethanol/water vapour at 1 atm (NRTL)

The story: Dew-T of a 30/70 ethanol/water vapour at 1 atm (NRTL gamma, non-ideal). The mirror of bubbleT01: the unit solves sum(y_i / K_i) = 1 for T at the given P (a saturation calc, no product streams); the incipient liquid is water-enriched. Dew-T sits ABOVE the bubble-T of the same overall composition – the two bound the two-phase window.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`feed`	`T`	`370`

`flash01_benzene_toluene`

`tutorials/steady/flash/flash01_benzene_toluene`

What it does: Isothermal flash benzene/toluene at 370 K, 1 bar (Raoult)

The story: THE first tutorial: isothermal flash of a 40/60 benzene/toluene feed (100 kmol/h) at 370 K, 1 bar, ideal Raoult K-values (K_i = Psat_i/P). 370 K sits inside the two-phase window (bubble ~368 K, dew ~374 K), so the feed splits into a benzene-enriched vapour and a toluene-enriched liquid – the Rachford-Rice solve is printed iteration by iteration at verbosity 3.

The same 40/60 benzene/toluene feed (100 kmol/h) as adiabaticFlash01, this time held at 370 K and 1 bar. Raoult's law, ideal vapour, the Rachford-Rice equation solved by Newton in one unknown. The golden: V/F = 0.3040 (30.40 kmol/h vapour, 69.60 kmol/h liquid), K_benzene = 1.651, K_toluene = 0.674.

1. type → model → operation. The unit is an isothermalFlash, the machine is named (model rachfordRice;), and the numbers live in operation { T 370.0 K; P 1.0 bar; }. That order is the grammar of every unit in Choupo. 2. K is a ratio of pressures here. With Raoult's law K_i = Psat_i(T) / P. The Props tab lets you read both vapour pressures at 370 K and check the two K's by hand — a two-line calculation a student should do once. 3. Rachford-Rice has one root in (0, 1) when the feed is between its bubble and dew points. The Log tab prints each Newton iterate of V/F; count them. 4. This drum adds no heat — and that is worth understanding. The golden reports Q_kW = 0. The feed is declared at 370 K and 1 bar, the drum holds 370 K and 1 bar, and enthalpy is a state function of (T, P, z): the feed stream ∗already is∗ the two-phase mixture the drum separates. Move T away from the feed's 370 K and a duty appears — that is the isothermal flash earning its name. (This README first said "holding T costs a duty"; the case's own golden said otherwise.)

Move T between the feed's bubble point and dew point and watch V/F sweep from 0 to 1; flash06_sweep_T does exactly that automatically with an outerDict, which is the next case to open.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`flash01`	`V_over_F`	`0.30398357314`
`kpi`	`flash01`	`T`	`370`
`stream`	`vapor`	`F`	`0.00844398814279`
`stream`	`liquid`	`F`	`0.019333789635`
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`feed`	`T`	`370`
`stream`	`liquid`	`T`	`370`
`stream`	`vapor`	`T`	`370`
`kpi`	`flash01`	`F_alpha`	`0.019333789635`
`kpi`	`flash01`	`F_beta`	`0.00844398814279`
`kpi`	`flash01`	`F_in`	`0.0277777777778`
`kpi`	`flash01`	`K_benzene`	`1.65136809583`
`kpi`	`flash01`	`K_toluene`	`0.673501829388`
`kpi`	`flash01`	`P`	`100000`

. . . and 3 more reference values in the case's `expected`.

`flash02_ethanol_water`

`tutorials/steady/flash/flash02_ethanol_water`

What it does: Isothermal flash 50/50 ethanol/water at 355 K (NRTL)

The story: isothermal flash, ethanol/water, NRTL

NOTE (golden re-recorded): the operation pressure formerly read a BARE 'P 1.01325;' (no unit) → parsed as 1.01325 Pa, not bar. At ~1 Pa the K-values blow up to O(1e5), so the flash silently reported a fully superheated vapour (V/F = 1, phaseSet = 0) and "Converged: yes" with no warning – a wrong all-vapour answer for a 50/50 ethanol/water at 355 K, 1 atm, which MUST be two-phase. Adding the 'bar' unit restores the physically-correct partial vaporisation; the 'expected' golden was re-recorded accordingly (was the wrong all-vapour result).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`liquid`	`F`	`0.00562323865754`
`stream`	`vapor`	`F`	`0.0221545391202`
`kpi`	`flash02`	`F_alpha`	`0.00562323865754`
`kpi`	`flash02`	`F_beta`	`0.0221545391202`
`kpi`	`flash02`	`F_in`	`0.0277777777778`
`kpi`	`flash02`	`P`	`101325`
`kpi`	`flash02`	`T`	`355`
`kpi`	`flash02`	`V_over_F`	`0.797563408329`
`kpi`	`flash02`	`phaseSet`	`0`
`stream`	`feed`	`T`	`355`
`stream`	`liquid`	`T`	`355`
`stream`	`vapor`	`T`	`355`
`kpi`	`flash02`	`K_ethanol`	`2.22741301451`
... and 3 more reference values in the case's `expected`.			

flash03_wilson_ethanol_water

tutorials/steady/flash/flash03_wilson_ethanol_water

What it does: Isothermal flash 50/50 ethanol/water at 355 K (Wilson activity model)

The story: same conditions as flash02, but Wilson activity model

NOTE (golden re-recorded): the operation pressure formerly read a BARE ‘P 1.01325;‘ (no unit) → parsed as 1.01325 Pa, not bar. At ~1 Pa the K-values blow up to O(1e5), so the flash silently reported a fully superheated vapour (V/F = 1, phaseSet = 0) and "Converged: yes" with no warning – a wrong all-vapour answer for a 50/50 ethanol/water at 355 K, 1 atm, which MUST be two-phase. Adding the ‘bar‘ unit restores the physically-correct partial vaporisation; the ‘expected‘ golden was re-recorded accordingly (was the wrong all-vapour result).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquid	F	0.0063767199082
stream	vapor	F	0.0214010578696
kpi	flash03	F_alpha	0.0063767199082
kpi	flash03	F_beta	0.0214010578696
kpi	flash03	F_in	0.0277777777778
kpi	flash03	P	101325
kpi	flash03	T	355
kpi	flash03	V_over_F	0.770438083305
kpi	flash03	phaseSet	0
stream	feed	T	355
stream	liquid	T	355
stream	vapor	T	355
kpi	flash03	K_ethanol	2.07217514574

. . . and 3 more reference values in the case’s `expected`.

`flash04_molarFlows_input`

`tutorials/steady/flash/flash04_molarFlows_input`

What it does: Isothermal flash, feed declared via molarFlows (benzene 40 + toluene 60 kmol/h)

The story: same benzene/toluene flash as flash01, but the feed is declared via per-species molar flows.

Instead of writing F = 100 kmol/h + composition { benzene 0.40; toluene 0.60; }, this case writes the absolute molar flow of each species:

molarFlows { benzene 40 kmol/h; toluene 60 kmol/h; }

The parser sums to F_total = 100 kmol/h and derives x_benzene = 0.4, x_toluene = 0.6. Physically equivalent to flash01 – and the result table is identical to printer precision (V/F = 0.304, L = 69.6 kmol/h, V = 30.4 kmol/h).

Demonstrates the (d) form of stream declaration; see userGuide "Composition basis" for the five accepted forms.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`liquid`	`F`	`0.019333789635`
`stream`	`vapor`	`F`	`0.00844398814279`
`kpi`	`flash04`	`F_alpha`	`0.019333789635`
`kpi`	`flash04`	`F_beta`	`0.00844398814279`
`kpi`	`flash04`	`F_in`	`0.0277777777778`
`kpi`	`flash04`	`P`	`100000`
`kpi`	`flash04`	`T`	`370`
`kpi`	`flash04`	`V_over_F`	`0.30398357314`
`kpi`	`flash04`	`phaseSet`	`0`
`stream`	`feed`	`T`	`370`
`stream`	`liquid`	`T`	`370`
`stream`	`vapor`	`T`	`370`
`kpi`	`flash04`	`K_benzene`	`1.65136809583`
. . . and 3 more reference values in the case's `expected`.			

flash05_ternary_massFlows

tutorials/steady/flash/flash05_ternary_massFlows

What it does: Ternary flash benzene/toluene/n-hexane, feed in mass flows (5000 kg/h)

The story: ternary benzene/toluene/n-hexane flash, feed declared via per-species MASS flows.

This is the (e) form of stream input from the v0.27 grammar (see userGuide "Composition basis"): instead of writing F total plus a composition, the student writes the absolute mass flow of each species and the parser sums + converts to molar:

m_i [kg/s] → F_mol,i [kmol/s] = m_i / MW_i F_total [kmol/s] = Sigma F_mol,i x_i = F_mol,i / F_total

Feed composition by mass (5000 kg/h total): benzene 1500 kg/h (30 wt%) MW = 78.11 kg/kmol toluene 2500 kg/h (50 wt%) MW = 92.14 kg/kmol n-hexane 1000 kg/h (20 wt%) MW = 86.18 kg/kmol

Conversion to molar: F_benzene = 1500 / 78.11 = 19.203 kmol/h F_toluene = 2500 / 92.14 = 27.131 kmol/h F_nHexane = 1000 / 86.18 = 11.604 kmol/h F_total = 57.938 kmol/h x_benzene = 0.3315 ; x_toluene = 0.4682 ; x_nHexane = 0.2003

At T = 365 K, P = 1 bar the n-hexane (Tb = 342 K) is well above its boiling point and goes preferentially to the vapour; toluene (Tb = 384 K) is sub-cooled and stays preferentially in the liquid; benzene (Tb = 353 K) partitions. Result:

molar basis F = 57.94 kmol/h V/F = 0.494 L = 29.34 kmol/h V = 28.60 kmol/h

mass basis F = 5000.0 kg/h V/F_mass = 0.487 L = 2567.3 kg/h V = 2432.7 kg/h

Mass fractions tell the partition story directly:

feed: w_benzene 0.300 w_toluene 0.500 w_nHexane 0.200 liquid: w_benzene 0.244 w_toluene 0.624 w_nHexane 0.133 vapor: w_benzene 0.360 w_toluene 0.369 w_nHexane 0.271

Mass balance closes per species — e.g. n-hexane: feed: 5000 ∗ 0.200 = 1000 kg/h liquid: 2567 ∗ 0.133 = 341 kg/h vapor: 2433 ∗ 0.271 = 659 kg/h → 341 + 659 = 1000.

Toggle the units menu in the GUI to Mass-basis to see the F and composition flip back to kg/h + w_i, matching the input statement symmetrically.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0160941453232
stream	liquid	F	0.00814913402453
stream	vapor	F	0.00794501129864
kpi	flash05	F_alpha	0.00814913402453
kpi	flash05	F_beta	0.00794501129864
kpi	flash05	F_in	0.0160941453232
kpi	flash05	P	100000
kpi	flash05	T	365
kpi	flash05	V_over_F	0.493658478851
kpi	flash05	phaseSet	0
stream	feed	T	365
stream	liquid	T	365
stream	vapor	T	365
kpi	flash05	K_benzene	1.43515088002

... and 4 more reference values in the case's `expected`*.*

flash06_sweep_T

tutorials/steady/flash/flash06_sweep_T

What it does: Flash benzene/toluene at 1 bar: T swept 360→385 K across the bubble→dew window (V/F S-curve + duty)

The story: the SAME 40/60 benzene/toluene isothermal flash as flash01 (100 kmol/h, 1 bar, Raoult); the T value below is only the BASE point – system/outerDict sweeps it 360 → 385 K across the bubble→dew window (~368..374 K) and reads the V/F S-curve + duty as responses.

The flash01 benzene/toluene drum, with its temperature swept from 360 K to 385 K in 1 K steps by a system/outerDict — the first time the simulator is run ∗many times∗ rather than once. The result is the V/F S-curve and, beside it, the duty each temperature costs.

The sweep table is written to sweep_flashT.csv in the case folder. Read down it:

1. The knob is a dict path. parameter { target units[0].operation.T; range (360 385); nPoints 26; } — ∗any∗ scalar a unit's operation block carries can be swept the same way. The dict is the schema; there is no separate list of "sweepable" things. 2. Bubble and dew fall out of the curve. V/F leaves 0 between 367 and 368 K and reaches 1 between 374 and 375 K. You did not ask for the bubble and dew points; you read them off. 3. Q = 0 at 370 K is not a coincidence. The feed is declared at 370 K: at the feed's own temperature the drum adds nothing (the point flash01 makes on its own). Below it the drum ∗removes∗ heat to condense; above it the drum ∗supplies∗ heat to boil. 4. Between two points the curve is not a line. V/F goes 0.30 → 0.60 → 0.96 over 370–374 K — steepest near the dew point. A hand calculation at two temperatures would miss the shape entirely.

Sweep pressure instead: change target to units[0].operation.P with a range across 0.5–2 bar and watch the same feed move from all-vapour to all-liquid. flash07_gridsweep_TP sweeps both at once and draws the envelope.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquid	F	0.0110077140444
stream	vapor	F	0.0167700637334
kpi	flash01	F_alpha	0.0110077140444
kpi	flash01	F_beta	0.0167700637334
kpi	flash01	F_in	0.0277777777778
kpi	flash01	K_benzene	1.74456473693
kpi	flash01	K_toluene	0.716223951412
kpi	flash01	P	100000
kpi	flash01	Q	284075.447071
kpi	flash01	Q_kW	284.075447071
kpi	flash01	T	372
kpi	flash01	V_over_F	0.603722294402
kpi	flash01	phaseSet	0
. . . and 7 more reference values in the case's `expected`.			

`flash07_gridsweep_TP`

`tutorials/steady/flash/flash07_gridsweep_TP`

What it does: 2-D grid sweep: flash V/F over T x P (the flash envelope)

The story: one isothermal flash, benzene/toluene 40/60.

The base point is 370 K, 1 bar. The gridSweep in system/outerDict overrides BOTH operation.T and operation.P across a grid, so V/F traces the flash envelope (V/F=0 bubble line, V/F=1 dew line) over the T x P plane.

What to expect (golden-locked):

kind	unit/stream	key	expected
`csv`	`gridsweep_TP.csv`	`8:flashDrum.V_over_F`	0.235178
`csv`	`gridsweep_TP.csv`	`23:flashDrum.V_over_F`	0.303984
`csv`	`gridsweep_TP.csv`	`38:flashDrum.V_over_F`	0.641142
`csv`	`gridsweep_TP.csv`	`46:flashDrum.V_over_F`	0.385974
`csv`	`gridsweep_TP.csv`	`0:flashDrum.V_over_F`	1
`csv`	`gridsweep_TP.csv`	`1:flashDrum.V_over_F`	0

flash08_co2_water_package

tutorials/steady/flash/flash08_co2_water_package

What it does: Carbonated water flash whose thermo is an INLINE property-package MANIFEST: constant/thermoPhysPropDict IS the full record (components, methods per phase, solvent+solutes, the CO2-water Henry pair declared+verified). CO2 dissolves by the unsymmetric Henry convention; water stays Raoult. Read the run log: the assembly announces itself.

The story: the property-package convention, minimal.

Soda water at 298 K: 1 bar flash of a CO2-rich feed. The THERMO is not a pile of flat keys – the case's constant/thermoPhysPropDict IS the full INLINE manifest (name flash08_co2Water) and the ThermoPackageBuilder assembles everything from it: liquid = solution.henryDilute (water on the Raoult rung, CO2 on the infinite-dilution Henry rung – ONE Gibbs surface, two conventions), vapour = ideal gas, the CO2-water Henry pair declared and VERIFIED at assembly (delete the pair path from the package and the run REFUSES, naming the file). Expect K_CO2 = H(298)/P ~ 1.6e3 (Sander): a huge K sets the vapour's PURITY, not the split – the vapour is nearly pure CO2 (y_CO2 ~ 0.97) but its AMOUNT is tiny (V/F ~ 4e-4), so only ~40% of the fed CO2 leaves in it; the liquid keeps x_CO2 ~ y_CO2/K ~ 6e-4.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	flatWater	F	0.000277662593734
stream	gasOff	F	1.15184043824e-07
stream	sodaWater	F	0.000277777777778
kpi	node	F_alpha	0.000277662593734
kpi	node	F_beta	1.15184043824e-07
kpi	node	F_in	0.000277777777778
kpi	node	P	101325
kpi	node	Q	0
kpi	node	Q_kW	0
kpi	node	T	298.15
kpi	node	V_over_F	0.000414662557767
kpi	node	phaseSet	0
stream	flatWater	T	298.15
stream	gasOff	T	298.15

. . . and 3 more reference values in the case's `expected`.

flash09_n2ch4_stryjek

tutorials/steady/flash/flash09_n2ch4_stryjek

What it does: phi-phi VLE VALIDATION vs Stryjek, Chappelear & Kobayashi (1974), J. Chem. Eng. Data 19:334 (DOI 10.1021/je60063a023), Table III, -190 F = 149.82 K. The case carries its thermo as an INLINE manifest: liquid+vapour both eos.SRK (one cubic, two roots: K = phi_L/phi_V) with the N2-CH4 interaction constant kij = 0.0289 (a small measured correction to the mixing rule, from the Knapp/DECHEMA data compilation). Measured at 299 psia: x_N2=0.1503, y_N2=0.4579 – compare the Flash Result table in the log with these measured values (agreement ~2%).

The story: the phi-phi world pinned to a MEASURED isotherm

Nitrogen-methane at 149.82 K (-190.00 F), 20.616 bar (299 psia) – a point of Stryjek, Chappelear & Kobayashi (1974), J. Chem. Eng. Data 19:334, Table III (DOI 10.1021/je60063a023). MEASURED there: x_N2 = 0.1503 y_N2 = 0.4579 K_N2 = 3.047 K_CH4 = 0.638 The feed z_N2 = 0.30 sits inside the envelope, so the flash must split into exactly those phases if the thermodynamics is right. The K-values come from K = phi_L/phi_V – the SAME SRK cubic evaluated at its liquid and vapour roots, with kij(N2,CH4) = 0.0289 – a small MEASURED correction to the cubic mixing rule, from the Knapp/DECHEMA VLE data compilation (announced in this log). Neighbouring measured points bracket the model: K_N2 = 3.047 at 299 psia and 2.241 at 396.5 psia – the engine interpolates between them consistently (try P 22 bar in this dict: K_N2 ~ 2.9). This is the quantitative anchor of the phi-phi world, the same standard as Wiebe-1934 for the Henry world.

THE AGREEMENT IS EXECUTABLE (2026-08-10). Everything above was prose: the golden pinned F/P/T/Q – all SELF-RECORDED – so a model that drifted off Stryjek's isotherm would still have passed while this header went on claiming a measured anchor. 'expected' now carries two 'anchor' rows holding the PAPER's K_N2 = 3.047 and K_CH4 = 0.638; they are compared on every run and are never refreshed by '–record' (a published number is not the model's to rewrite). Measured agreement: K_N2 +2.25 %, K_CH4 -1.56 %, x_N2 -0.52 %, y_N2 +1.73 % – inside the declared 3 % band, with margin.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	liq	F	0.000145628387466
stream	vap	F	0.000132149390312
kpi	node	F_alpha	0.000145628387466
kpi	node	F_beta	0.000132149390312
kpi	node	F_in	0.000277777777778
kpi	node	P	2061600
kpi	node	Q	0
kpi	node	Q_kW	0
kpi	node	T	149.82
kpi	node	V_over_F	0.475737805121
kpi	node	phaseSet	0
kpi	node	K_N2	3.047
kpi	node	K_CH4	0.638

. . . and 3 more reference values in the case's `expected`.

flash09_nh3_water_reactive

tutorials/steady/flash/flash09_nh3_water_reactive

What it does: Reactive NH3/water flash: SIMULTANEOUS aqueous speciation (NH4+/H+/OH-, pH solved) + VLE at 368 K, 1 atm – ions never reach the vapour or the streams

The story: the volatile weak electrolyte, done honestly.

Ammonia in water IONISES (NH3 + H2O <→ NH4+ + OH-) and VOLATILISES (Henry) at the same time: the ionised fraction is unavailable to the vapour, so speciation and phase split must be solved TOGETHER – never flash-once/speciate-once. The flash unit implements NO chemistry: it delegates to the package's reactive equilibrium (unit-local speciation, property authority section 6b). Streams – including 0/ and converged/ – carry ONLY the apparent components NH3 and water; the aqueous species (NH4+, H+, OH-) exist in the internal state and the report.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquid	F	0.0224533469429
stream	vapor	F	0.00532443083488
kpi	flash01	F_alpha	0.0224533469429
kpi	flash01	F_beta	0.00532443083488
kpi	flash01	F_in	0.0277777777778
kpi	flash01	P	101325
kpi	flash01	Q	0
kpi	flash01	Q_kW	0
kpi	flash01	T	368.15
kpi	flash01	V_over_F	0.191679510056
kpi	flash01	pH	9.899823444
kpi	flash01	p_eq_sum_atm	1
kpi	flash01	phaseSet	0

. . . and 5 more reference values in the case's `expected`.

`flash10_acetic_water_reactive`

`tutorials/steady/flash/flash10_acetic_water_reactive`

What it does: Reactive acetic acid/water flash: SIMULTANEOUS aqueous speciation (Acetate-/H+/OH-, pH solved) + VLE – the weak ACID mirror of flash09; ions never reach the vapour or the streams

The story: the volatile weak ACID, done honestly.

Acetic acid in water IONISES (CH3COOH <→ CH3COO- + H+, pKa 4.756) and VOLATILISES at the same time – and its VAPOUR DIMERISES (2 HAc = (HAc)2, carboxylic hydrogen bonding), so even the saturation check must price the dimer's own partial pressure. Speciation and phase behaviour are solved TOGETHER: the flash unit implements no chemistry, it delegates to the package's reactive equilibrium (unit-local speciation, section 6b). Streams – 0/ and converged/ included – carry ONLY the apparent components aceticAcid and water; the aqueous species (CH3COO-, H+, OH-) exist in the internal state and the report.

THE OPERATING POINT IS DELIBERATELY SUBSATURATED (T fixed at 368.15 K, 1 atm; sum p_eq = 0.818 atm < P): the honest regime for a dilute, barely-volatile acid on the Davies rung. The unit is an ISOTHERMAL (T,P) flash – T is the specification, not a result. Temperature still ACTS, through BOTH van't Hoff legs (dissolution dH = -51.5 kJ/mol, dimerisation dH = -63.2 kJ/mol), and the flash publishes what it moves as KPIs: pH, p_eq_sum_atm, p_mono_atm, p_dimer_atm, dimer_share. flash11_acetic_T_sweep sweeps T across the subsaturated window so the student SEES the total volatility climb while the dimer share falls; flash13_acetic_ethanol_vacuum_flash crosses into the TWO-PHASE regime, where the vapour mole balance is re-weighted by the dimer explicitly (apparent moles = n_mono + 2 n_dim) and its association heat is priced into the duty – never a silent approximation.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`liquid`	`F`	`0.0277777777778`
`stream`	`vapor`	`F`	`0`
`kpi`	`flash01`	`F_alpha`	`0.0277777777778`
`kpi`	`flash01`	`F_beta`	`0`
`kpi`	`flash01`	`F_in`	`0.0277777777778`
`kpi`	`flash01`	`P`	`101325`
`kpi`	`flash01`	`Q`	`0`
`kpi`	`flash01`	`Q_kW`	`0`
`kpi`	`flash01`	`T`	`368.15`
`kpi`	`flash01`	`V_over_F`	`0`
`kpi`	`flash01`	`dimer_share`	`0.0948608286627`
`kpi`	`flash01`	`pH`	`2.15260602547`
`kpi`	`flash01`	`p_dimer_atm`	`0.00389347151965`
. . . and 6 more reference values in the case's `expected`*.*			

`flash10_ch4propane_pcsaft`

`tutorials/steady/flash/flash10_ch4propane_pcsaft`

What it does: PC-SAFT phi-phi binary flash, CH4/propane at 273.15 K / 20 bar, kij = 0 (announced): the PREDICTIVE baseline of the SAFT world – K = phi_L/phi_V from ONE a_res surface at its two density roots. Sanity anchor carried by the case itself: propane's curated Antoine gives Psat(273.15 K) = 4.7 bar, so the Raoult-magnitude K_propane ~ 0.24 – the PC-SAFT K_propane = 0.32 sits near it (the deviation IS the fugacity correction of a liquid holding ~11% dissolved CH4, visible); K_CH4 » 1 (supercritical light key). The published-kij refinement is a NAMED follow-up: it needs the cited pair record (data/standards/parameters/PCSAFT/), never an invented number.

The story: the PC-SAFT phi-phi world on a BINARY, kij = 0

Methane/propane, 273.15 K, 20 bar, z = 0.50/0.50. CH4 is supercritical (Tc = 190.6 K) − no Raoult route exists for it; the phi-phi K comes from the SAME PC-SAFT a_res at the liquid and vapour density roots. With NO binary correction (kij = 0, announced by the EoS), everything is PREDICTED from the two components' (m, sigma, epsilon/k) alone (Gross & Sadowski 2001, Table 1; CH4 and propane parameters independently cross-checked against the NIST teqp reference implementation's test values).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`liq`	`F`	`9.97838549688e-05`
`stream`	`vap`	`F`	`0.000177993922809`
`kpi`	`node`	`F_alpha`	`9.97838549688e-05`
`kpi`	`node`	`F_beta`	`0.000177993922809`
`kpi`	`node`	`F_in`	`0.000277777777778`
`kpi`	`node`	`P`	`2000000`
`kpi`	`node`	`Q`	`0`
`kpi`	`node`	`Q_kW`	`0`
`kpi`	`node`	`T`	`273.15`
`kpi`	`node`	`V_over_F`	`0.640778122112`
`kpi`	`node`	`phaseSet`	`0`
`stream`	`feed`	`T`	`273.15`
`stream`	`liq`	`T`	`273.15`
... and 3 more reference values in the case's `expected`*.*			

flash11_acetic_T_sweep

tutorials/steady/flash/flash11_acetic_T_sweep

What it does: Reactive acetic/water flash: T swept 328→368 K in the subsaturated window (p_eq climbs, dimer share falls, pH drifts)

The story: the same reactive flash as flash10, but the temperature specification is OWNED by system/outerDict: the sweep driver rewrites units[0].operation.T pass by pass (setScalarAtPath – the dict IS the schema) and records the flash's reactive KPIs at each point. Topology, chemistry, and feed are identical to flash10_acetic_water_reactive; see that case's header for the physics and for why the two-phase corner is refused.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`liquid`	`F`	`0.0277777777778`
`stream`	`vapor`	`F`	`0`
`kpi`	`flash01`	`F_alpha`	`0.0277777777778`
`kpi`	`flash01`	`F_beta`	`0`
`kpi`	`flash01`	`F_in`	`0.0277777777778`
`kpi`	`flash01`	`P`	`101325`
`kpi`	`flash01`	`Q`	`-43266.6666667`
`kpi`	`flash01`	`Q_kW`	`-43.2666666667`
`kpi`	`flash01`	`T`	`348.15`
`kpi`	`flash01`	`V_over_F`	`0`
`kpi`	`flash01`	`dimer_share`	`0.115426638388`
`kpi`	`flash01`	`pH`	`2.15093435013`
`kpi`	`flash01`	`p_dimer_atm`	`0.00184409797792`
. . . and 10 more reference values in the case's `expected`.			

flash12_nh3_acetic_ethanol_reactive

tutorials/steady/flash/flash12_nh3_acetic_ethanol_reactive

What it does: Reactive 4-component flash: NH3 + HAc neutralise into an ammonium-acetate buffer (pH ~ pKa_HAc); ethanol classified nonionising from its record fact, ideal VLE authorised; subsaturated at 358.15 K

The story: One isothermal flash, four components, two aqueous families + one authorised molecular co-volatile:

feed (2 HAc + 1 NH3 + 1 EtOH + 96 H2O, kmol/h) → [flash01, 358.15 K / 1 atm] → liquid (everything) + vapor (0)

The chemistry does the interesting work in the LIQUID: NH3 + HAc → NH4+ + Ac- consumes the base completely (K = Ka_HAc/Ka_NH4 ~ 10^4.5), leaving an equimolar Ac-/HAc buffer at pH ~ 4.6. The saturation check prices each volatile over THAT speciated liquid – ionised acid and ammonium are unavailable to the vapour, the acetic dimer is priced, and ethanol contributes x*psat (authorised ideal). KPIs: pH, p_eq_sum_atm, p_mono_atm / p_dimer_atm / dimer_share, p_ethanol_atm.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquid	F	0.0277777777778
stream	vapor	F	0
kpi	flash01	F_alpha	0.0277777777778
kpi	flash01	F_beta	0
kpi	flash01	F_in	0.0277777777778
kpi	flash01	P	101325
kpi	flash01	Q	0
kpi	flash01	Q_kW	0
kpi	flash01	T	358.15
kpi	flash01	V_over_F	0
kpi	flash01	dimer_share	0.0226239722903
kpi	flash01	pH	4.6182465052
kpi	flash01	p_dimer_atm	0.000106738278327

. . . and 7 more reference values in the case's `expected`.

flash13_acetic_ethanol_vacuum_flash

tutorials/steady/flash/flash13_acetic_ethanol_vacuum_flash

What it does: Reactive VACUUM flash on mixed-solvent v1 (NRTL backbone + Davies ions): V/F = 0.58 at 358.15 K / 0.65 atm, gamma_EtOH = 2.97, the buffer holds the ammonia, dimer heat priced exactly

The story: One isothermal VACUUM flash, four components, REAL vapour flow:

feed (2 HAc + 1 NH3 + 12 EtOH + 85 H2O, kmol/h) → [flash01, 358.15 K / 0.65 atm] → liquid (42.3 kmol/h, pH 4.98) + vapor (57.7 kmol/h)

On the NRTL backbone the feed's bubble pressure is ~1.1 atm (gamma_EtOH = 2.97 triples the ideal ethanol volatility): 0.65 atm strips well over half the feed without wringing the liquid dry. The vapour balance carries the acetic dimer explicitly (v_HAc = t_mono + 2 t_dim, K_dim(T) van't Hoff) and the buffer chemistry keeps the ammonia liquid-bound. KPIs: pH, p_mono_atm, p_dimer_atm, dimer_share, p_ethanol_atm, p_eq_sum_atm (= P, two-phase).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquid	F	0.011761964504
stream	vapor	F	0.0160158132738
kpi	flash01	F_alpha	0.011761964504
kpi	flash01	F_beta	0.0160158132738
kpi	flash01	F_in	0.0277777777778
kpi	flash01	P	65861.25
kpi	flash01	Q	665815.899842
kpi	flash01	Q_kW	665.815899842
kpi	flash01	T	358.15
kpi	flash01	V_over_F	0.576569277856
kpi	flash01	dimer_share	0.0276466273131
kpi	flash01	pH	4.98104745363
kpi	flash01	p_dimer_atm	0.000161043019564

. . . and 15 more reference values in the case's `expected`.

flash14_calcite_carbonate_basis

tutorials/steady/flash/flash14_calcite_carbonate_basis

What it does: Reactive calcite/carbonate flash: the FIRST component whose aqueous bridge spans TWO master families (CaCO3 → Ca+2 AND HCO3-), so the component-to-species map is a matrix, not a renaming

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0275033888889
stream	liquid	F	0.0269197611327
stream	vapor	F	0.000583627756178
kpi	flash01	F_alpha	0.0269197611327
kpi	flash01	F_beta	0.000583627756178
kpi	flash01	F_in	0.0275033888889
kpi	flash01	P	101325
kpi	flash01	T	313.15
kpi	flash01	V_over_F	0.0212202124813
kpi	flash01	pH	6.01561221041
kpi	flash01	p_eq_sum_atm	1
kpi	flash01	phaseSet	0
stream	feed	T	313.15
stream	liquid	T	313.15

. . . and 4 more reference values in the case's `expected`.

flash15_refused_salt_basis_rank

tutorials/steady/flash/flash15_refused_salt_basis_rank

What it does: DELIBERATE REFUSAL: K/Na x Cl/SO4 – four salt components onto four ion families with rank 3, so the component basis has exactly (c-1)(a-1) = 1 degree of freedom and the converged state cannot be read back as these components uniquely

`flash16_calcite_precipitation`

`tutorials/steady/flash/flash16_calcite_precipitation`

What it does: Reactive flash that PRECIPITATES: the calcite/carbonate basis of flash14 fed past saturation, with calcite admitted – the speciation drives SI to 0 and the freed proton drops the pH from 6.63 to 6.02 – ONE proton per mole of calcite, and exactly once: the crystal is neutral, so it moves no charge, and the acidification arrives through the master balances that electroneutrality closes

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0275083333333
stream	liquid	F	0.0269247487752
stream	vapor	F	0.000583584558179
kpi	flash01	F_alpha	0.0269247487752
kpi	flash01	F_beta	0.000583584558179
kpi	flash01	F_in	0.0275083333333
kpi	flash01	P	101325
kpi	flash01	T	313.15
kpi	flash01	V_over_F	0.0212148279253
kpi	flash01	pH	6.02023176857
kpi	flash01	p_eq_sum_atm	1
kpi	flash01	phaseSet	0
stream	feed	T	313.15
stream	liquid	T	313.15
. . . and 6 more reference values in the case's `expected`.			

`flash17_two_liquids_reactive`

`tutorials/steady/flash/flash17_two_liquids_reactive`

What it does: Reactive flash with TWO liquids: vapour + speciated aqueous + benzene-rich organic at 313.15 K / 0.35 atm – V/F = 0.324, organic 1.7 % of the liquid, pH 2.89

The story: One isothermal flash, four components, THREE phases:

feed (60 H2O + 20 benzene + 8 EtOH + 0.1 HAc, kmol/h) → [flash01, 313.15 K / 0.35 atm] → liquid (59.6 kmol/h: aqueous + organic) + vapor (28.5 kmol/h)

The two liquids leave as ONE apparent liquid stream. That is deliberate and it is the same contract the speciation already lives under: the split is INTERNAL state – reported in full, never persisted – exactly as the ions are. A stream file carries apparent components; it does not carry phases, and it does not carry species.

KPIs: V/F, pH, p_benzene_atm, p_ethanol_atm, p_eq_sum_atm (= P, two-phase), p_mono_atm / p_dimer_atm / dimer_share (the acetic vapour dimer).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0244722222222`
`stream`	`liquid`	`F`	`0.0165455900723`
`stream`	`vapor`	`F`	`0.00792663214995`
`kpi`	`flash01`	`F_alpha`	`0.0165455900723`
`kpi`	`flash01`	`F_beta`	`0.00792663214995`
`kpi`	`flash01`	`F_in`	`0.0244722222222`
`kpi`	`flash01`	`P`	`35463.75`
`kpi`	`flash01`	`Q`	`86.2488491825`
`kpi`	`flash01`	`Q_kW`	`0.0862488491825`
`kpi`	`flash01`	`T`	`313.15`
`kpi`	`flash01`	`V_over_F`	`0.323903243358`
`kpi`	`flash01`	`dimer_share`	`0.00665763422177`
`kpi`	`flash01`	`pH`	`2.89010173914`
`kpi`	`flash01`	`p_benzene_atm`	`0.232353913068`

. . . and 16 more reference values in the case's `expected`.

flash18_water_analysis_basis

tutorials/steady/flash/flash18_water_analysis_basis

What it does: A stream written in the AQUEOUS-SPECIES basis: a calcium-bicarbonate water analysis (Ca 0.0122, HCO3 0.0244 kmol/h) inverted onto the case's components by m = A n – the engine formulates the salts, the author does not

The story: Everything else in this corpus writes a stream in apparent components. This one writes it in the AQUEOUS SPECIES a laboratory actually measures, and lets the engine do the formulation:

speciesMolarFlows { network carbonate; basis analytical; Ca 0.0122; HCO3 0.0244; water 97.0; }

– m = A n, against the components' DECLARED bridges –

componentMolarFlows { CO2 0.0122; CaCO3 0.0122; water 97.0; }

WHY IT MATTERS. Converting an analysis into components by hand is a choice of LABELS, not a calculation: the same ions can be formulated as different component sets, and picking one silently is the arbitrariness the basis-rank test exists to refuse. Done here, the inversion is performed by the engine and REFUSED when it is not unique – the same refusal flash15_refused_salt_basis_rank raises, reached from the input side.

THREE REFUSALS GUARD IT, and check_stream_basis.py exercises all three: an analysis that does not close in CHARGE (a measurement to fix, never a residue for the solved pH to absorb); a 'network' naming a chemistry set this system does not declare; a missing 'basis' – because analytical and stoichiometric sets differ exactly in whether charge must close (H/OH are excluded mediators), so there is no safe default.

The answer is IDENTICAL to the same water written in components: the species basis is a change of coordinates, not a change of model. pH 7.757 – a natural calcium-bicarbonate water.

Contract: docs/design/aqueous-stream-basis-proposal.md.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0269512222222
stream	liquid	F	0.0269512222222
stream	vapor	F	0
kpi	flash01	F_alpha	0.0269512222222
kpi	flash01	F_beta	0
kpi	flash01	F_in	0.0269512222222
kpi	flash01	P	101325
kpi	flash01	T	313.15
kpi	flash01	V_over_F	0
kpi	flash01	pH	7.75731867771
kpi	flash01	p_eq_sum_atm	0.0881612782912
kpi	flash01	phaseSet	0
stream	feed	T	313.15
stream	liquid	T	313.15

. . . and 1 more reference values in the case's `expected`.

`flash19_organic_and_precipitate`

`tutorials/steady/flash/flash19_organic_and_precipitate`

What it does: FOUR phases at once: vapour + speciated aqueous + benzene-rich organic + solid calcite. flash16's feed plus benzene and ethanol, nothing else changed – the co-solvent moves the precipitation from 729 to 1096 mg/kg

One isothermal flash at 313.15 K / 1 atm, and four phases coexist:

The three condensed phases leave through one bottoms nozzle and are named inside that stream's phases {} block; the vapour is its own stream.

Until it was written, the reactive path had been exercised with an organic liquid (flash17) and with a precipitate (flash16) — but never both at once. The file format supported it and the solver was believed to; no run proved it. This is that run, and the joint residual closes to = 6.6e-14.

The feed is flash16's, unchanged, plus benzene 2.0 and ethanol 1.0 kmol/h. Same water, same CO_2, same $CaCO_3$, same T, same P. So the two cases differ by exactly those two components:

Adding a co-solvent pushed half again as much calcite out of solution.

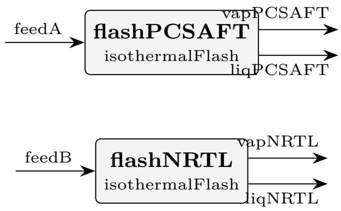
That number is not a mixed-solvent antisolvent calculation, and a student who reads it as one will be wrong about why.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0283416666667
stream	liquid	F	0.0275385746121
stream	vapor	F	0.000803092054611
kpi	flash01	F_alpha	0.0275385746121
kpi	flash01	F_beta	0.000803092054611
kpi	flash01	F_in	0.0283416666667
kpi	flash01	P	101325
kpi	flash01	T	313.15
kpi	flash01	V_over_F	0.0283360913124
kpi	flash01	pH	6.10723545836
kpi	flash01	p_benzene_atm	0.238716176589
kpi	flash01	p_eq_sum_atm	1
kpi	flash01	p_ethanol_atm	0.011504063078
kpi	flash01	phaseSet	0
... and 10 more reference values in the case's `expected`.			

flash20_ethanol_water_pcsaft

tutorials/steady/flash/flash20_ethanol_water_pcsaft



What it does: Ethanol/water flash: predictive PC-SAFT (assoc, kij=0) vs fitted NRTL

The story: flash20 – ethanol/water VLE, PREDICTIVE vs FITTED on the same state

The same 30/70 mol% ethanol/water feed flashed at 358.15 K / 1 atm through TWO thermodynamic worlds, side by side:

flashPCSAFT the GLOBAL world: phi-phi PC-SAFT with the ASSOCIATION term active on both components (2B water + 2B ethanol, Gross & Sadowski 2002 Table 1 – the paper lists water with TWO sites; the mixture witness of the association programme, validation item 4). The cross-association follows the Wolbach-Sandler combining rules and kij = 0 (announced) – so the MIXTURE behaviour is fully PREDICTED: nothing in this route was fitted to any ethanol/water datum. Only the PURE parameters are fits.

flashNRTL a per-unit thermo {} override: the classic gamma-phi route on the curated NRTL pair (fitted TO ethanol/water VLE data) + Antoine. This is the case's experimental anchor: the fitted route reproduces the measured azeotropic system, so the DIFFERENCE between the two units' K-values and splits is an honest measurement of what pure-component association physics alone can and cannot predict for a strongly hydrogen-bonded pair.

What the student compares, measured at the feed composition z (illustrative – what is goldened are the CONVERGED K's: PC-SAFT 11.47 / 0.581 vs NRTL 3.896 / 0.602 for ethanol / water):

predictive PC-SAFT fitted NRTL (gamma-phi) K_ethanol (at z) 2.185 2.238 K_water (at z) 0.769 0.695 V/F 0.649 0.512 x_ethanol / y_ethanol 0.038 / 0.441 0.121 / 0.471

The predictive route lands the K's within ~2 % (ethanol) and ~11 % (water) of the data-fitted route – from PURE-component parameters alone, a genuinely strong showing for a hydrogen-bonded pair – and the residual K_water error still moves V/F from 0.51 to 0.65: the pedagogical point, stated up front rather than hidden. Predictive means "no mixture data used", not "as accurate as a fit"; the fitted gamma-phi route stays the design tool, the predictive EoS is the screen for systems with NO data to fit.

This case is ALSO the witness that catches a curation error the pure witnesses cannot: with water miscurated as 4C (the same eps/kappa, the wrong scheme) the pure density read +0.6 % – fine – while THIS flash collapsed to K_water = 0.0044. The scheme is part of the fit.

A model boundary note: the two units run DIFFERENT thermo worlds on the same global component set (the general heterogeneous-solver contract); each stream leaves in its own unit's world.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feedA	F	0.0277777777778
stream	feedB	F	0.0277777777778
stream	liqNRTL	F	0.0135537277127
stream	liqPCSAFT	F	0.00975103940319
stream	vapNRTL	F	0.014224050065
stream	vapPCSAFT	F	0.0180267383746
kpi	flashNRTL	F_alpha	0.0135537277127
kpi	flashNRTL	F_beta	0.014224050065
kpi	flashNRTL	F_in	0.0277777777778
kpi	flashNRTL	K_ethanol	3.89559152547
kpi	flashNRTL	K_water	0.602023174315
kpi	flashNRTL	P	101325
kpi	flashNRTL	Q	0
kpi	flashNRTL	Q_kW	0
... and 23 more reference values in the case's `expected`.			

flash21_brine_co2_pitzer

tutorials/steady/flash/flash21_brine_co2_pitzer

What it does: NaCl brine + CO2 reactive flash under the pitzerHMW ionic rung – seam S5(b) witness: the declared model IS the served model.

The story: the S5(b) witness: a 2 mol/kg NaCl brine with dissolved CO2, flashed at 313.15 K / 1 atm, with the aqueous speciation priced by pitzerHMW (curated Na-Cl / Na-HCO3 / Na-CO3 / Na-OH / H-Cl pairs) instead of the charge-only Davies correlation that has no business at I = 2. The salt stays in the liquid by construction (nonvolatile, complete dissociation through its declared bridge); CO2 partitions against its ionised fraction exactly as in flash09's ammonia lesson – speciation and phase split solved TOGETHER, never in sequence.

WHAT THE MODEL CHOICE MOVES, measured at build time (2026-08-24) against an otherwise identical davies twin: the liquid pH (3.84 here vs 3.96 under davies – the charge-only bracket is out of its band at I = 2 and the Davies advisory says so on that run), while V/F barely moves (0.0211 vs 0.0210): the vapour is set by the NEUTRAL CO2aq, which Davies prices at exactly gamma = 1 and HMW at its real specific-interaction value.

A CAVEAT THIS CASE ANNOUNCES AND DOES NOT HIDE: the energy report tries to re-resolve every stream's own equilibrium, and the VAPOUR stream (93 % CO2, 7 % water, zero salt) read as an aqueous liquid has an implied molality near 770 mol/kg – outside any aqueous model's domain. pitzerHMW fails there loudly (singular Jacobian) and the report falls back to the carried state, announced in the caveat block. Davies returns a number at the same absurd state, which is not a virtue: a charge-only correlation cannot know it is out of its depth. The answer's own streams are unaffected.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquid	F	0.0271904240052
stream	vapor	F	0.000587353772572
kpi	flash01	F_alpha	0.0271904240052
kpi	flash01	F_beta	0.000587353772572
kpi	flash01	F_in	0.0277777777778
kpi	flash01	K_CO2	3299.5218183
kpi	flash01	K_NaCl	0
kpi	flash01	K_water	0.0696646141595
kpi	flash01	P	101325
kpi	flash01	T	313.15
kpi	flash01	V_over_F	0.0211447358126
kpi	flash01	pH	3.83985178539
kpi	flash01	p_eq_sum_atm	1

. . . and 4 more reference values in the case's `expected`*.*

flash21_freeze_concentration

tutorials/steady/flash/flash21_freeze_concentration

What it does: Freeze-concentration SLE flash: ice crystallises from a water/ethanol liquor at -15 C; the liquor concentrates until gamma_w ∗ x_w = exp(-dG_fus/RT)

What this witnesses (migration S4b, 2026-08-09): the flash's LIQUID+SOLID branch. A C3 uniform phases (?) declaration — one liquid, one crystallizing solid (component water;), no vapour — dispatches to the SLE solve, where the ice phase becomes a SolidCandidate on THE ONE solid-equilibrium mechanism (solidEq::equilibrate, ruling R1's active-set + simultaneous damped Newton). The candidate's lnSI is ln(x_w · fEff_liq / fEff_solid) and its remove pulls water from the liquid inventory — the one ledger.

The acceptance is a closed form, not a golden alone. At equilibrium the K = 1 condition collapses to the freezing-point-depression identity

gamma_w · x_w = exp(-dG_fus / (R T)), dG_fus = dHfus · (1 - T/Tfus)

with every quantity from the record's own data (water.dat Hfus 6008 J/mol, triple point 273.16 K). At 258.15 K the right side is 0.857434120855; the run publishes the liquor's activity as KPI a_water = 0.857434120931 — agreement to 9e-11. check_solid_service arm A10 recomputes the closed form independently in Python on every suite run.

The numbers: feed 90/10 water/ethanol (mol) at 25 °C, 1 atm; the freezer operates at -15 °C. Ice forms at solidFraction 0.476 mol per mol of feed — a function of the operating (T, P) alone, not of the feed temperature; the liquor concentrates to x_ethanol ≈ 0.19, exactly where the NRTL water activity meets the saturation value. The liquor and crystal leave through ONE outlet (liquor): its overall material equals the feed and the phases {} block decomposes it; vent is the structurally-required second outlet and carries F = 0 (no vapour phase is declared).

The duty (gap closed 2026-08-09 by docs/design/tp-stream-energy-coherence.md slice 1, R-E3/R-E4): h_formation gained its gas-natural → solid view (Hf - Hvap(298) - Hfus + ?cp_solid, from the record's own declared transition data), and the flash prices every crystallised mole on that solid formation rung — the SAME leg the balance report reads. The warm feed makes the duty visible as physics: Q ≈ -167.4 kW = sensible cooling of 100 kmol/h by 40 K (≈ -88 kW) + fusion of 47.6 kmol/h of ice (≈ -79 kW). Cooling is negative by the sign convention; gate arm A10 asserts the published Q is finite and negative.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquor	F	0.0277777777778
stream	vent	F	0
kpi	freezer01	F_alpha	0.0277777777778
kpi	freezer01	F_beta	0
kpi	freezer01	F_in	0.0277777777778
kpi	freezer01	P	101325
kpi	freezer01	Q	-167415.991569
kpi	freezer01	Q_kW	-167.415991569
kpi	freezer01	T	258.15
kpi	freezer01	V_over_F	0
kpi	freezer01	aEq_water	0.857434120855
kpi	freezer01	a_water	0.857434120931
kpi	freezer01	phaseSet	0

... and 5 more reference values in the case's `expected`.

flash_polymer_devolatilisation

tutorials/steady/flash/flash_polymer_devolatilisation

What it does: Devolatilise a polystyrene melt: flash off residual toluene; the polymer (fixed-MW pseudo-component, role nonvolatile) stays liquid

The story: polymer melt + residual solvent → flash (devolatiliser).

The flash boils off the residual toluene (vapour) and leaves the polymer in the liquid. polystyrene is ‘role nonvolatile;‘, so its K = 0: it stays 100 % in the liquid by construction (no Psat needed for a nonvolatile lump).

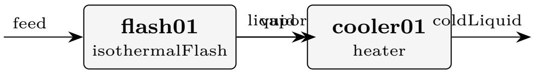
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	liquid	F	0.00246249927126
stream	meltFeed	F	0.00277777777778
stream	vapor	F	0.000315278506519
kpi	devol01	F_alpha	0.00246249927126
kpi	devol01	F_beta	0.000315278506519
kpi	devol01	F_in	0.00277777777778
kpi	devol01	K_polystyrene	0
kpi	devol01	K_toluene	24.2878386162
kpi	devol01	P	20000
kpi	devol01	Q	0
kpi	devol01	Q_kW	0
kpi	devol01	T	450
kpi	devol01	V_over_F	0.113500262347
kpi	devol01	phaseSet	0

. . . and 3 more reference values in the case’s `expected`.

`flash_pseudoComponent_petCut`

`tutorials/steady/flash/flash_pseudoComponent_petCut`



What it does: Flash of a petroleum pseudo-component (Riazi-Daubert lump) + toluene, with a heater closing the energy balance

The story: feed → isothermal flash → heater on the liquid.

The flash splits the feed into vapour + liquid (the pseudo-component participates in VLE through its critical{} + Ambrose-Walton Psat). The heater unit then REMOVES sensible heat from the liquid (Q -30 kW – a heater with negative duty is a cooler); energyBalance reports the implied duty, and it CLOSES because the lump carries gasIdeal + liquidPure Cp. No reaction → no formation datum needed (matching the lump's omitted standardThermochemistry).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`coldLiquid`	`F`	`0.00758956122005`
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`liquid`	`F`	`0.00758956122005`
`stream`	`vapor`	`F`	`0.0201882165577`
`kpi`	`cooler01`	`F`	`0.00758956122005`
`kpi`	`cooler01`	`P`	`100000`
`kpi`	`cooler01`	`Q`	`-30000`
`kpi`	`cooler01`	`Q_kW`	`-30`
`kpi`	`cooler01`	`Q_per_mol`	`-3952.79768226`
`kpi`	`cooler01`	`T_in`	`420`
`kpi`	`cooler01`	`T_out`	`405.894884539`
`kpi`	`cooler01`	`dT`	`-14.1051154607`
`kpi`	`cooler01`	`vf_out`	`0`
`kpi`	`flash01`	`F_alpha`	`0.00758956122005`

. . . and 16 more reference values in the case's `expected`.

leaf01_flash

tutorials/steady/flash/leaf01_flash

What it does: Leaf-node form: isothermal flash benzene/toluene (fractal base case)

The story: / flowsheetDict (LEAF NODE – fractal base case)

A leaf carries ‘type‘ + ‘operation‘ (the unit) + ‘boundary‘ (its interface) + ‘streams‘ (the default feeds, so it runs isolated). No ‘units (...)‘ list. Compare flash01_benzene_toluene, which writes the SAME flash as a one-element units list – identical physics, two roupas.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquid	F	0.019333789635
stream	vapor	F	0.00844398814279
kpi	flash	F_alpha	0.019333789635
kpi	flash	F_beta	0.00844398814279
kpi	flash	F_in	0.0277777777778
kpi	flash	P	100000
kpi	flash	T	370
kpi	flash	V_over_F	0.30398357314
kpi	flash	phaseSet	0
stream	feed	T	370
stream	liquid	T	370
stream	vapor	T	370
kpi	flash	K_benzene	1.65136809583

. . . and 3 more reference values in the case’s `expected`.

marcilla01_lls_tie_triangle

tutorials/steady/flash/marcilla01_lls_tie_triangle

What it does: LLS anchor contact vs Marcilla 1995 (Table 10A row 4 charge): brine + solid halite reproduced; the organic vertex is NOT – the corpus UNIFAC (VLE table) keeps water/1-butanol miscible, a named model finding, never tuned

The story: THE FLAGSHIP LLS WITNESS (solid-equilibrium migration S5).

One isothermal flash at the paper's own conditions: 25.0 C, 1 atm. The feed is Marcilla, Ruiz & Olaya's Table 10A initial mixture, row 4 – a published experimental charge, not an invented one – and the anchor is the tie-triangle that charge produced (Table 10B row 4): an aqueous brine, a butanol-rich organic liquid, and solid NaCl, all simultaneously at equilibrium. PRIMARY: A. Marcilla, F. Ruiz, M. M. Olaya, Fluid Phase Equilibria 105 (1995) 71-91, DOI 10.1016/0378-3812(94)02595-R.

What the engine must do at once here, through the ONE mechanism each: dissolve NaCl onto the Na/Cl master families (the typed dissociatesTo bridge), split the molecular backbone into two liquids by equality of activity, and hold the halite complementarity through the solid- equilibrium service while that split resolves. No unit below declares any of that chemistry – the case does (thermoPhysPropDict + chemistryDict), and the flash reads it.

WHAT THIS CASE CURRENTLY WITNESSES, honestly (2026-08-10): the brine + solid halite half of the anchor, and a MODEL FINDING on the other half. The corpus' UNIFAC is the VLE table (Hansen 1991), and under it the water/1-butanol pair is fully miscible at 25 C – g_mix stays convex (independently confirmed by a propertyScanBinary sweep), so the Gibbs decision correctly reports ONE liquid where the paper measures two. The organic vertex of the tie-triangle is therefore NOT reproduced, and no number here was tuned to change that. The named remedy is curation: a UNIFAC-LLE parameter set (Magnussen, Rasmussen & Fredenslund, Ind. Eng. Chem. Process Des. Dev. 20 (1981) 331 – the parameterisation created precisely because the VLE table misses binodals), or fitted NRTL pairs for water/1-butanol and ethanol/1-butanol. See README.md.

PRIMARY: A. Marcilla, F. Ru?z, M. M. Olaya, ∗Fluid Phase Equilibria∗ 105 (1995) 71–91, DOI 10.1016/0378-3812(94)02595-R. Water–ethanol–1-butanol–NaCl at 25.0 ± 0.1 °C. Anchor values staged in [docs/design/solid-migration-witness-data.md](../../../../docs/design/solid-migration-witness-data.md) ?2 (reviewStatus interim; this case cites the primary).

The feed is the paper's own Table 10A row 4 initial mixture — a published experimental charge, not an invented one (wt%):

That charge produced the paper's Table 10B row 4 tie-triangle — three phases simultaneously at equilibrium (wt%):

(The lever-rule check on the paper's own numbers closes to Σλ = 1.010.)

Davies ions on water-referenced molality + UNIFAC molecular backbone (the corpus table — the VLE parameterisation of Hansen et al. 1991) + halite complementarity through the solid-equilibrium service + a declared WET organic (water is a member; its cross-liquid equality carries the ionic a_w factor).

Reproduced: the saturated brine + solid halite coexistence. The run converges with solid NaCl present and a single fluid liquid; Davies is announced out of its validity band (I ≈ 3.6 mol/kg against a ~0.5 band), so the predicted saturation is ∗indicative∗ — the model holds ~3.6 mol/kg against Pinho & Macedo's measured 6.14, and that gap is Davies', reported.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000849885277778
stream	liquid	F	0.000849885277778
stream	vapor	F	0
kpi	flash01	F_alpha	0.000849885277778
kpi	flash01	F_beta	0
kpi	flash01	F_in	0.000849885277778
kpi	flash01	P	101325
kpi	flash01	T	298.15
kpi	flash01	V_over_F	0
kpi	flash01	pH	7.35672575928
kpi	flash01	p_eq_sum_atm	0.0415604200493
kpi	flash01	p_ethanol_atm	0.0111811211227
kpi	flash01	p_nButanol_atm	0.00389708966162
kpi	flash01	phaseSet	0
. . . and 3 more reference values in the case's `expected`.			

marcilla02_lls_ternary_invariant

tutorials/steady/flash/marcilla02_lls_ternary_invariant

What it does: The ternary LLS invariant vs Marcilla 1995 Table 4B: aqueous brine + WET butanol-rich organic + solid halite, all three at once – the first PRESENT wet-organic witness (UNIQUAC backbone, Winkelman pair)

The story: THE TERNARY LLS INVARIANT (solid-equilibrium migration S5b).

Water + 1-butanol + NaCl at 25.0 C, 1 atm: with solid NaCl present the two-liquid tie-line is INVARIANT (F = 0 at fixed T, P), so the paper reports ONE tie-triangle for the whole three-phase region – Table 4B: aqueous 73.14 / 0.72 / 26.14 (W / B / NaCl, wt%) organic 7.53 / 92.28 / 0.19 solid NaCl PRIMARY: A. Marcilla, F. Ruiz, M. M. Olaya, Fluid Phase Equilibria 105 (1995) 71-91, DOI 10.1016/0378-3812(94)02595-R.

This is the state marcilla01 could not reach: there the UNIFAC (VLE) backbone kept water/1-butanol miscible. Here the backbone is UNIQUAC on the CITED Winkelman 2009 water/1-butanol set – the same values vlle01 consumes inline, where they demonstrably open the gap – so the wet organic (water a declared member) is PRESENT, and all three phases coexist through their one mechanism each: the halite complementarity, the activity-equality split with the ionic a_w factor on water, and the Davies speciation of the brine.

PRIMARY: A. Marcilla, F. Ru?z, M. M. Olaya, ∗Fluid Phase Equilibria∗ 105 (1995) 71–91, DOI 10.1016/0378-3812(94)02595-R, Table 4B — water + 1-butanol + NaCl at 25.0 ± 0.1 °C. With solid NaCl present the two-liquid tie-line is invariant, so one tie-triangle describes the whole three-phase region. Anchors staged in docs/design/solid-migration-witness-data.md ?2a (interim).

This is the state marcilla01 could not reach (its UNIFAC-VLE backbone keeps water/1-butanol miscible). The backbone here is UNIQUAC on the cited Winkelman 2009 water/1-butanol set (case-local record — the same values vlle01_waterButanol consumes inline, where they demonstrably open the gap). The engine's first PRESENT wet organic: all three phases coexist, each through its one mechanism — halite complementarity, activity-equality split with the ionic a_w factor on water, Davies speciation of the brine.

Every deviation is a NAMED model approximation, announced by the run:

∗ Aqueous NaCl low (16.2 vs 26.1 wt%): Davies saturates halite at 3.65 mol/kg against the measured 6.14 — the same number marcilla01 measured, seven times outside Davies' band, announced. ∗ Aqueous butanol high (7.7 vs 0.72 wt%): Davies prices neutral solutes at $\gamma = 1$, so the brine's salting-out of butanol — an order of magnitude in the anchor — is invisible to the ionic leg. ∗ Organic too wet (13.6 vs 7.53 wt% water): Davies' $\varphi = 1$ water activity (a_w $\approx$ 0.88 vs the real $\approx$ 0.75 over saturated brine) under-prices the salt's pull on the water, so too much crosses.

The internal identities the gate pins: the cross-liquid equality residual including water's a_w factor is at machine precision (~ 1e-15 per member), and the dissolved salt over the AQUEOUS water alone reproduces the Davies saturation molality — the conversion-basis bug this case caught (a total-water basis overstated the crystal by exactly the organic water's share, while every mass balance still closed: a wrong basis conserves mass perfectly).

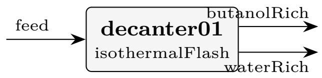
Energy balance: UNAVAILABLE (the apparent salt has no elements-datum route in this world — named diagnostic; mass and element balances close).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.000820413055556`
`stream`	`liquid`	`F`	`0.000820413055556`
`stream`	`vapor`	`F`	`0`
`kpi`	`flash01`	`F_alpha`	`0.000820413055556`
`kpi`	`flash01`	`F_beta`	`0`
`kpi`	`flash01`	`F_in`	`0.000820413055556`
`kpi`	`flash01`	`P`	`101325`
`kpi`	`flash01`	`T`	`298.15`
`kpi`	`flash01`	`V_over_F`	`0`
`kpi`	`flash01`	`pH`	`7.35672525831`
`kpi`	`flash01`	`p_eq_sum_atm`	`0.032687682758`
`kpi`	`flash01`	`p_nButanol_atm`	`0.00579877206855`
`kpi`	`flash01`	`phaseSet`	`0`
`stream`	`feed`	`T`	`298.15`
. . . and 2 more reference values in the case's ***`expected`****.*			

vlle01_waterButanol

tutorials/steady/flash/vlle01_waterButanol



What it does: Liquid-liquid flash of water + 1-butanol at 320 K, 1.013 bar – the real miscibility gap from the cited Winkelman et al. (2009) UNIQUAC LLE fit: water-rich ~98.3 mol% water, butanol-rich ~46 mol% butanol (their Figs 1-2). See README.

The story: liquid-liquid flash of water + 1-butanol with the cited Winkelman et al. (2009) UNIQUAC LLE parameters.

Water + 1-butanol is THE textbook partially-miscible binary. At 320 K and 1.013 bar an 80/20 water/butanol mixture is well below the bubble point (water boils at 373 K, 1-butanol near 390 K), so the equilibrium is GENUINELY two LIQUIDS – no vapour. A simple ‘phaseSet LL‘ flash (Michelsen TPD pre-check, then direct Gibbs-energy minimisation, multi-start Nelder-Mead) finds the split.

THE PARAMETERS (cited, NOT fabricated – see constant/thermoPhysPropDict and README.md): the UNIQUAC interaction parameters are the LLE-specific fit of

J.G.M. Winkelman, G.N. Kraai, H.J. Heeres, Fluid Phase Equilibria 284 (2009) 71-79, Table 2 (p.73); structural r,q from their Table 1 (p.72).

with the full temperature dependence A__ij(T) = a__ij + b__ij*T + c__ij*T^2 (their Eq. 10), tau__ij = exp(-A__ij/T) (their Eq. 7), valid 273-363 K. The numbers are inline here; the record is STAGED CASE-LOCALLY (reviewStatus interim) under marcilla02’s constant/parameters/UNIQUAC/, NOT curated into data/standards/ – promotion is the maintainer’s batch review.

a REAL liquid-liquid miscibility gap, reproduced from a cited primary source – the water-rich phase is ~98.3 mol% water (~1.7 mol% butanol) and the butanol-rich phase is ~54 mol% water (~46 mol% butanol), matching Winkelman’s Figs 1-2; the IsothermalFlash LL branch: a Michelsen TPD stability test that flags the feed as unstable, then a direct Gibbs-minimisation that resolves the two coexisting compositions; honest phase labelling – the two output streams are named by their actual composition (waterRich / butanolRich), with alphaRich/betaRich hints steering which liquid lands in which slot.

HISTORICAL NOTE: an earlier version of this case carried only a DECHEMA VLE-fit NRTL set, which (being a VLE fit) has NO miscibility gap, so the flash found a single liquid – it was kept as an honest "VLE-fit-misses-LLE" lesson pending a cited LLE set. Winkelman et al. (2009) IS that cited set, so the case now shows the genuine split. (The VLE-blind NRTL lesson lives on in the NRTL catalogue file’s provenance.)

*Water + 1-butanol (n-butanol) is *the* textbook partially-miscible binary. Below the bubble point it splits into two coexisting liquids: a water-rich phase carrying only a couple of mol % butanol, and a butanol-rich phase holding roughly half water. So a flash of a feed *inside the miscibility gap* should give two liquids — and here it does.*

At 320 K, 1.013 bar (well below the bubble point, so no vapour) a feed of 80 mol % water / 20 mol % butanol splits into:

*with split fraction $\beta \approx 0.41$ into the butanol-rich phase. The solver reports the Michelsen TPD instability (tm__min ≈ -0.45, *unstable = YES*) and then a direct Gibbs-energy minimisation that lands the two compositions.*

An independent re-implementation of the UNIQUAC γ (fresh code, isoactivity Newton) reproduces the engine’s split to 4 decimals across 298–350 K, confirming the parameters are entered correctly. The agreement with the published figures is essentially exact — the UNIQUAC fit is doing its job.

constant/thermoPhysPropDict carries the LLE-specific UNIQUAC for water/1-butanol, cited per value:

*- Primary source: J.G.M. Winkelman, G.N. Kraai, H.J. Heeres, *"Binary, ternary and quaternary liquid-liquid equilibria in 1-butanol, oleic acid, water and n-heptane mixtures"*, Fluid Phase Equilibria 284 (2009) 71–79, doi:10.1016/j.fluid.2009.06.013. - Binary interaction parameters — their Table 2 (p.73), binary 1-butanol(1)/water(3), valid 273–363 K, with the quadratic temperature correlation A__ij(T) = a__ij +*

b_ij·T + c_ij·T^2 (their Eq. 10) and τ_ij = exp(-A_ij/T) (their Eq. 7): - A_1,3 (butanol→water): a = 155.31, b = 1.0822, 10?·c = -43.711 - A_3,1 (water→butanol): a = -579.36, b = 2.7517, 10?·c = -6.7700 - Structural r, q — their Table 1 (p.72) (value Magnussen et al.): 1-butanol r = 3.9243, q = 3.668; water r = 0.9200, q = 1.400.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	butanolRich	F	0.0114431411907
stream	feed	F	0.0277777777778
stream	waterRich	F	0.016334636587
kpi	decanter01	F_alpha	0.016334636587
kpi	decanter01	F_beta	0.0114431411907
kpi	decanter01	F_in	0.0277777777778
kpi	decanter01	P	101300
kpi	decanter01	T	320
kpi	decanter01	betaFraction	0.411953082867
kpi	decanter01	phaseSet	1
stream	butanolRich	T	320
stream	feed	T	320
stream	waterRich	T	320

`vlle03_audit_artificial`

`tutorials/steady/flash/vlle03_audit_artificial`

What it does: Algorithm-audit synthetic VLLE: 3 components with identical Psat and exaggerated NRTL. Should converge to 3-phase by construction.

The story: algorithm-correctness audit for the VLLE Gibbs-min branch.

Three synthetic components A, B, C with identical Antoine (Psat ≈ 0.5 bar at 350 K). NRTL parameters chosen to GUARANTEE three-phase coexistence at T = 350 K, P = 1 bar:

A-B: τ_AB = τ_BA = 5 (very strong, near-immiscible) A-C: τ_AC = τ_CA = 5 (very strong, near-immiscible) B-C: τ_BC = τ_CB = 0 (ideal, fully miscible)

Expected behaviour by symmetry and construction: one liquid phase rich in A (the "isolated" component); another liquid phase rich in B+C (they pair together); a vapour phase carrying some of each (Psat is equal so all are equally volatile, ln(P/Psat) ≈ +0.301 at 1 bar).

Equimolar feed (1/3, 1/3, 1/3). This case is the litmus test for the VLLE solver: if it cannot find 3-phase here, the bug is in the algorithm, not in any real-system NRTL parameter set.

What to expect (golden-locked):

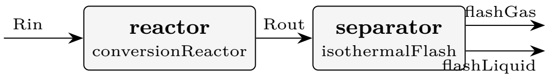
kind	unit/stream	key	expected
`stream`	`liquidA`	`F`	`0.011307571384`
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`liquidB`	`F`	`0.0113091924076`
`stream`	`vapor`	`F`	`0.00516101398624`
`kpi`	`decanter01`	`F_alpha`	`0.0164702063938`
`kpi`	`decanter01`	`F_beta`	`0.011307571384`
`kpi`	`decanter01`	`F_in`	`0.0277777777778`
`kpi`	`decanter01`	`F_vapor`	`0.00516101398624`
`kpi`	`decanter01`	`P`	`100000`
`kpi`	`decanter01`	`T`	`350`
`kpi`	`decanter01`	`V_over_F`	`0.407072569823`
`kpi`	`decanter01`	`beta_vapor`	`0.185796503505`
`kpi`	`decanter01`	`phaseSet`	`2`
`utility`	`decanter01`	`heating.steamLP.duty_kW`	`4.38784508636`
. . . and 12 more reference values in the case's `expected`*.*			

flowsheets

The theory you need. Recycle turns a flowsheet into a fixed-point problem: guess the tear stream, march the units, compare. Successive substitution converges linearly (and diverges past unit gain); Wegstein accelerates it; Newton-on-tears solves the tear residuals with a Jacobian. Choupo announces the tear choice, every iteration's residual, and any bound that binds – no silent crutch. Expect: purge streams controlling inert build-up (close the loop with no purge and watch the accumulation), and convergence behaviour that CHANGES with the tear location. *Full derivation: Theory Guide, the flowsheet-convergence chapter.*

acetone03_luyben_reaction_section

`tutorials/steady/flowsheets/acetone03_luyben_reaction_section`



What it does: Luyben's acetone REACTION SECTION: reactor → cooler → flash, against his Rout and the flash split

The story: Three units of Luyben's Figure 1 wired together – reactor, cooler, flash – and the first piece of the plant whose answer DOES depend on the mixture thermodynamics. acetone02 isolated the reaction; this adds the phase split that sends hydrogen and acetone one way and the aqueous liquid the other.

W. L. Luyben, Ind. Eng. Chem. Res. 50 (2011) 1206-1218, DOI 10.1021/ie901923a. Feed Rin as derived in acetone02 (57.82 kmol/h, IPA 0.668 / water 0.332, 389 K, 2.6 atm).

WHAT IS INDEPENDENT HERE, and it is the point. acetone02's outlet composition agreed with the paper because the feed had been DERIVED from that same outlet – a closed loop, and its golden says so. The flash split is not: nothing in 0/Rin was built from it. Luyben's Figure 1 gives

flash vapour 39.76 kmol/h H2 0.8742 A 0.0634 IPA 0.0002 W 0.0622 flash liquid 52.84 kmol/h (92.6 - 39.76; his F1 of 72.85 kmol/h is this stream AFTER the absorber bottoms of 20.05 kmol/h rejoin it)

and those are a real test of the thermodynamics this case declares.

THE THERMODYNAMICS IS DECLARED AND WEAK, said before the numbers rather than after. Luyben runs UNIQUAC; Choupo ships no UNIQUAC pair for any of these binaries, so this case runs PREDICTIVE UNIFAC on a lake-estimated isopropanol. The sibling props case measures what that costs on the same binary: the IPA boiling point is 4.5 K high and the IPA/water azeotrope lands at x 0.590 against a published 0.673. A separation built on that will not reproduce a separation built on fitted pairs, and the interesting question is HOW MUCH, not WHETHER.

Plus the one declared approximation, in constant/thermoPhysPropDict: H2 runs outside the activity model at gamma = 1, authorised by name, recorded in the answer.

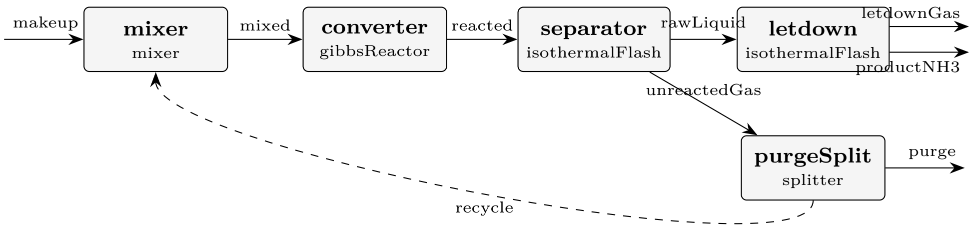
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	Rin	F	0.0160611111111
stream	Rout	F	0.0257186111111
stream	flashGas	F	0.0123053585397
stream	flashLiquid	F	0.0134132525714
kpi	reactor	F_in_kmol_h	57.82
kpi	reactor	F_out_kmol_h	92.587
kpi	reactor	Q_kW	840.263234652
kpi	reactor	T	623
kpi	reactor	conversion	0.9
kpi	reactor	dHrxn_kJ_per_mol	48.7173210118
kpi	reactor	extent_kmol_h	34.767
kpi	separator	F_alpha	0.0134132525714
kpi	separator	F_beta	0.0123053585397
kpi	separator	F_in	0.0257186111111

. . . and 20 more reference values in the case's `expected`*.*

ammonia01_synthesis_loop

tutorials/steady/flowsheets/ammonia01_synthesis_loop



What it does: Complete ammonia synthesis loop (Haber-Bosch) with recycle and purge: makeup 1:3 N2:H2 (+ Ar inert) → mix with recycle → SRK Gibbs converter at 700 K / 200 bar (real-gas fugacity) → chill-flash at 250 K to condense the NH3 product → split the unreacted gas into a small Ar-bleeding PURGE and the recycle. The recycle tear + purge are the whole point: high overall N2/H2 utilisation despite a modest per-pass conversion, with the inert Ar held at steady state by the purge.

The story: the classic Haber-Bosch loop, recycle + purge

makeup (N2:H2 = 1:3 + trace Ar) [mixer] recycle [converter] gibbsReactor (SRK) N2 + 3H2 <→ 2NH3, 700 K, 200 bar (fugacity-corrected equilibrium ~30% NH3) [separator] isothermalFlash 250 K, 200 bar (chill: NH3 condenses out) rawLiquid unreactedGas [purgeSplit] splitter [letdown] flash 20 bar purge recycle (bleeds Ar) (~95% back to the mixer) productNH3 letdownGas (97.8% NH3) (H2-rich)

THE LOOP. Per-pass conversion is equilibrium-limited (~30% NH3 at the reactor outlet), but the RECYCLE lifts overall N2/H2 utilisation high – the reason the loop exists. The reactor equilibrium AND the separator both use SRK FUGACITY (real gas at 200 bar): switch the package to idealGas and the conversion falls. Mass closes to 100% on N, H and Ar.

THE PURGE. Ar enters with the makeup and would accumulate without end; the purge bleeds it, holding Ar at steady state (Ar__in__makeup = Ar__out). Delete the purge and the recycle fills with Ar until the loop chokes – try it.

THE LET-DOWN. At 200 bar the SRK flash leaves a lot of N2/H2 DISSOLVED in the liquid (a real high-pressure effect, over-stated here because there is no N2/H2-NH3 binary interaction parameter – SRK over-predicts light-gas solubility). So the raw separator liquid is only ~80% NH3. A let-down drum (flash to 20 bar) boils the dissolved gas off → 97.8% NH3 product.

HONEST SIMPLIFICATIONS. (1) The loop is isobaric at 200 bar; the makeup compression and the feed/effluent heat integration are folded into the unit duties (reported), not drawn as separate machines. (2) The let-down gas (letdownGas, H2-rich) is RECOMPRESSED AND RECYCLED in a real plant; here it exits as a secondary stream to keep ONE clean recycle tear – so the H2 loss reads higher than industrial. Recovering it (a recompressor + a second tear) is the efficiency refinement. The topology keeps the loop's ESSENCE: reaction, separation, recycle, purge, product finishing.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	letdownGas	F	1.06533052494e-06
stream	makeup	F	0.000277777777778
stream	mixed	F	0.000562684679632
stream	productNH3	F	0.000131750445273
stream	purge	F	1.49951001055e-05
stream	rawLiquid	F	0.000132815775798
stream	reacted	F	0.000432717777908
stream	recycle	F	0.000284906902004
stream	unreactedGas	F	0.000299902002109
kpi	converter	F_in_kmol_h	2.02566484668
kpi	converter	F_out_kmol_h	1.55778400047
kpi	converter	N_in_mol_s	0.562684679632
kpi	converter	N_out_mol_s	0.432717777908
kpi	converter	P	20000000

. . . and 55 more reference values in the case's `expected`.

`ammonia02_full_plant`

`tutorials/steady/flowsheets/ammonia02_full_plant`

What it does: Full-scale ammonia plant (~59 t/h NH3, 1400 t/day): explicit syngas + recycle COMPRESSORS, feed-effluent HEAT EXCHANGER, ADIABATIC converter (the reaction heats itself – exothermic), water cooler + refrigerated separator, product let-down, and the recycle + purge. SRK fugacity throughout; dissolved H2/N2 by measured Henry constants (Wiebe 1934, Larson-Black 1925 via IUPAC SDS 5/6).

The story: the complete Haber-Bosch plant, ~50 t/h NH3

makeup 25 bar [makeupComp 25→200] recycle [recycleComp] [mixer] mixed (cool) cold side [FEHE] hot effluent (heat recovery) preheatedFeed (~600 K) [converter] gibbsReactor mode ADIABATIC N2+3H2<→2NH3 exothermic → self-heats to ~720 K hotEffluent hot side of FEHE → cooledEffluent [waterCooler] (cooling water) ~310 K [separator] flash 250 K/200 bar (refrig.) rawLiquid unreactedGas [letdown] [purgeSplit] productNH3 gas purge recycle (tear)

THE REACTOR HEAT. N2+3H2→2NH3 is EXOTHERMIC – there is NO external reactor heating. The converter runs ADIABATIC: the exotherm raises the gas T (feed ~705 K in, ~848 K out), and the feed-effluent exchanger (FEHE) recovers that heat to preheat the incoming feed to catalyst light-off. Self-sustaining once lit. The compressors set the 200-bar synthesis pressure; the water cooler (cooling-water utility) + refrigerated flash condense the NH3. Every machine is explicit and its duty/work is reported.

CAPACITY ~59 t/h NH3 (~1400 t/day) at 98.9% product purity – a world-scale single train.

DISSOLVED GASES BY HENRY'S LAW. H2 and N2 are supercritical at the separator (no physical Psat), so their dissolved amounts follow MEASURED Henry constants (pairs H2-NH3 from Wiebe 1934, N2-NH3 from Larson & Black 1925 as evaluated in IUPAC SDS Vol 5/6), with the SRK vapour fugacity at 200 bar: K = H(T)/(phi_V P). The raw liquid carries ~0.5% H2 + 0.2% N2 – physical, vs the ~13% a Psat extrapolated above Tc used to fake. KNOWN RESIDUAL: Ar has no measured Ar-NH3 Henry pair yet, so its dissolved ~1% is a Raoult-extrapolation artefact (~7x over by analogy with N2) – the product's main impurity is overstated.

THE SIMULATION STRATEGY (how you build a plant like this). 1. PICK THE THERMO BY THE PHYSICS, per phase ("the two worlds"): the 200-bar GAS side needs real-gas fugacity → a cubic EoS (SRK) declared once in the package; the reactor folds phi into its equilibrium automatically. The DISSOLVED gases are supercritical solutes in a condensable solvent → the unsymmetric convention (Henry / Krichevsky-Kasarnovsky), NOT Raoult: pair files H2-NH3 + N2-NH3 fitted to measured solubility, 'solvent NH3;' in the package, 'role solute;' overlays on H2/N2 in this case's constant/. 2. TEAR THE LOOP WHERE INFORMATION FLOWS BACKWARD: one MASS tear (recycle, guessed then Newton-converged) and one ENERGY tear (hotEffluent – the FEHE preheats the feed with the reactor's own product heat, so the converter's outlet is needed upstream of the converter). Both declared in solverDict tearStreams; the solver announces every iteration. 3. LET THE EXOTHERM DO THE HEATING: the converter is ADIABATIC (mode adiabatic; T is a guess, not a spec) – the reaction self-heats 705 → 848 K and the FEHE recycles that heat to light off the feed. No phantom reactor heater exists anywhere. 4. MAKE EVERY MACHINE EXPLICIT so it can be SIZED AND COSTED: compressors by shaft power, exchangers by area, the converter + drums by vessel design bases (space velocity / residence time) in system/postDict → Guthrie costs → the DCF economics in constant/economics. Every unit operation carries its own sizing – the doctrine. 5. VALIDATE AGAINST MEASUREMENT AT EVERY JOINT: the reactor equilibrium against the ideal-vs-SRK shift (gibbs10), the dissolved gas against Wiebe 1934 / Larson-Black 1925 (x_H2 at 400 atm: 0.0287 computed vs 0.0282 measured), the balances against 100% element closure.

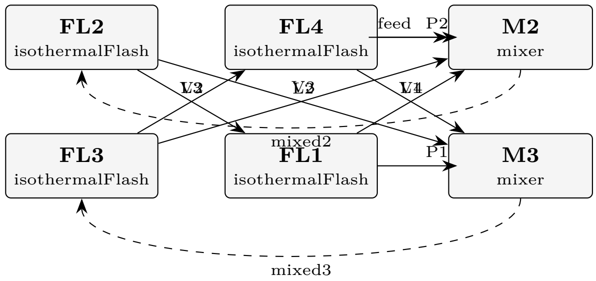
HONEST SIMPLIFICATIONS. (1) The converter is a SINGLE adiabatic bed, so it runs hotter (~575 C) than a real multi-bed converter with interbed quench cooling (which stays near the 400-500 C equilibrium sweet spot for more NH3 per pass). (2) The makeup compressor is one stage (real: multi-stage with intercooling), so its discharge is hot. (3) The let-down gas and the loop pressure drops are as in ammonia01. (4) 'water' is in the package ONLY as the cooling-water utility fluid (it is not a reaction species).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`chilledEffluent`	`F`	`10.5545178888`
`stream`	`compMakeup`	`F`	`2.22222222222`
`stream`	`compRecycle`	`F`	`9.29181627432`
`stream`	`cooledEffluent`	`F`	`10.5545178888`
`stream`	`cw`	`F`	`138.888888889`
`stream`	`cwOut`	`F`	`138.888888889`
`stream`	`hotEffluent`	`F`	`10.554505292`
`stream`	`letdownGas`	`F`	`0.00745588546829`
`stream`	`makeup`	`F`	`2.22222222222`
`stream`	`mixed`	`F`	`11.5140384965`
`stream`	`preheatedFeed`	`F`	`11.5140384965`
`stream`	`productNH3`	`F`	`0.967859425136`
`stream`	`purge`	`F`	`0.287376077346`
`stream`	`rawLiquid`	`F`	`0.975315310604`
... and 147 more reference values in the case's ***`expected`****.*			

cavett01_recycle_train

tutorials/steady/flowsheets/cavett01_recycle_train



What it does: Cavett (1963) four-flash recycle benchmark on the primary Rosen-Pauls spec: 2 tears, Newton 7 it (Wegstein defeated). NO published product comparison is carried – see the flowsheetDict header

The story: / flowsheetDict – the Cavett problem on its PRIMARY specification (Cavett 1963; Rosen & Pauls 1977; Rosen, CACHE News Fall 2005; Seader & Henley 1998, Exercise 10.35).

The flowsheet is a four-theoretical-stage NEAR-ISOTHERMAL distillation (pressure, not temperature, changes stage to stage – the Gunther arrangement, US 3,575,077): FL1 is the partial condenser, FL4 the partial reboiler, and the FEED enters at the second stage. Each stage's liquid cascades DOWN; each stage's vapour climbs back UP – those three up-going vapours are the recycles R1, R2, R3 of the literature.

P1 (vapour product) FL1 (100 F, 800 psia) V2 L1 (R1) feed M2 FL2 (120 F, 270 psia) V3 L2 (R2) M3 FL3 (96 F, 49 psia) V4 L3 (R3) FL4 (85 F, 13 psia) P2 (liquid product)

HISTORY OF THIS DICT: the first commit of this case carried a reconstructed wiring (feed into FL1's mixer; V4 into FL2's) built from paraphrases, with the header saying so. The primary source then became available and showed BOTH edges wrong – the feed enters FL2 and V4 recycles to FL3. The reconstruction is gone; what you read is the documented flowsheet. A topology built from paraphrase and one built from the source differed in 2 edges out of 11 – worth remembering when citing anything from memory.

THE MISSING ANCHOR, named rather than implied (2026-08-10). This case's controlDict used to advertise "SRK vs published APR/FLOWTRAN products". It carries NO such comparison: no published product table is staged anywhere in the case, and nothing checks a Choupo number against one. What IS anchored here is the SPECIFICATION – the feed, the four stage conditions and the topology, all from the primary – plus the solver behaviour (Wegstein defeated on the real conditions; Newton converges in 7 iterations). That is a reproduction of the PROBLEM, not agreement with a published ANSWER, and the description now says so.

To become a real external anchor this case needs Rosen & Pauls' (or FLOWTRAN's) product compositions transcribed from the primary and pinned as ‘anchor‘ rows in ‘expected‘ – published values, never refreshed by ‘–record‘. Until someone reads that paper, the gap stays visible: a claim of agreement nobody can check is worse than an admitted absence.

TWO tears still close all three loops: cut the two mixer outlets and every stream is computable in the declared order (validated by the sequential-plan contract, not asserted here).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	L1	F	23.6955550394
stream	L2	F	11.4542917866
stream	L3	F	5.71821815965
stream	P1	F	3.68430363786
stream	P2	F	3.91030744687
stream	V2	F	27.3798586772
stream	V3	F	7.54398434784
stream	V4	F	1.80791071278
stream	feed	F	7.59461111111
stream	mixed2	F	38.8341504983
stream	mixed3	F	13.2622024993
kpi	FL1	F_alpha	23.6955550394
kpi	FL1	F_beta	3.68430363786
kpi	FL1	F_in	27.3798586772
…and 134 more reference values in the case's `expected`.			

composite01_two_flashes

tutorials/steady/flowsheets/composite01_two_flashes

What it does: Composite node: two flashes in series (fractal step 2)

The story: / flowsheetDict (COMPOSITE node)

children : the child unit ops (folders flash1/, flash2/ – leaf nodes). connections : the wiring. ‘from‘/‘to‘ endpoints are either a bare name (this node’s boundary inlet/outlet) or ‘<child>/<port>‘. The engine flattens to namespaced streams (flash1.liquid → flash2.feed).

Two flashes in series, each a GENUINE vapour/liquid split. flash1 (370 K) splits the feed; its liquid (benzene-lean) feeds flash2 (373 K), which is still below that liquid’s dew point, so flash2 produces a real vapour AND a real liquid ‘bottoms‘ – not a total vaporiser.

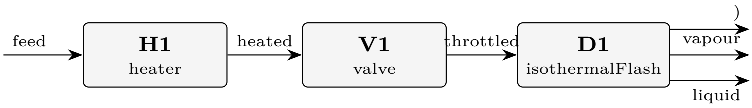
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.00370154472079
stream	feed	F	0.0277777777778
stream	vapor1	F	0.0212163063101
stream	vapor2	F	0.00285992674691
kpi	flash1	F_alpha	0.0065614714677
kpi	flash1	F_beta	0.0212163063101
kpi	flash1	F_in	0.0277777777778
kpi	flash1	P	100000
kpi	flash1	T	370
kpi	flash1	V_over_F	0.763787027163
kpi	flash1	phaseSet	0
kpi	flash2	F_alpha	0.00370154472079
kpi	flash2	F_beta	0.00285992674691
kpi	flash2	F_in	0.0065614714677

. . . and 26 more reference values in the case’s `expected`.

credo01_valve_heater_drum

tutorials/steady/flowsheets/credo01_valve_heater_drum



What it does: Info-follows-streams credo: heater (Q) + valve (P) set the state; the flash drum (no knobs) just separates

The story: "Information follows the streams" — every change of state is a VISIBLE box on the stream, and the separator has no operating knobs. Read the chain top-to-bottom, the way the plant is built:

feed (10 bar liquid) → [H1 heater : Q] the heat box. Its only knob is the duty; T_out | is a result. Stays liquid at 10 bar (well below | the bubble point). → [V1 valve : P] the pressure box. Its only knob is the | downstream pressure; the let-down to 1 bar is | isenthalpic, so the hot liquid flashes and T and | vf come out as RESULTS. → [D1 flash : {}] the separator. NO knobs — ‘operation {}‘ is | empty. It inherits T and P from the stream V1 | hands it and just splits the phases. → liquid + vapour

Compare with flash01_benzene_toluene, where the same separation is asked as ‘isothermalFlash; operation { T 370; P 1 bar; }‘. Those two numbers on the flash were, physically, a hidden heater (T) and a hidden valve (P). Here they are visible boxes, and the drum is honestly knob-less.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	H1	T_out	413.08978161
kpi	V1	T_out	369.30464285
kpi	V1	vf	0.20555173
kpi	V1	dP	900000
stream	liquid	F	0.0220680075
stream	vapour	F	0.0057097703
utility	H1	heating.steamMP.duty_kW	260
utility	H1	heating.steamMP.T	413.089781612
utility	H1	heating.steamMP.kg_s	0.1305876444
utility	H1	heating.steamMP.eur_h	14.04
stream	feed	F	0.0277777777778
stream	feed	T	350
stream	heated	F	0.0277777777778
stream	heated	T	413.089781612
. . . and 28 more reference values in the case’s `expected`.			

`plant01_two_sectors`

`tutorials/steady/flowsheets/plant01_two_sectors`

What it does: Recursive plant: factory of two composite sectors (fractal step 3)

The story: the RECURSIVE node (fractal step 3).

A FACTORY whose children are SECTORS that are themselves composite (each a flowsheet of one flash). Recursion: plant → sector → unit. Tests nested namespacing (secA.fa, secB.fb) and multi-level cabling (feed → secA → secA/liquid → secB). The whole plant runs from one command.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	0
`stream`	`feed`	`F`	0.0277777777778
`stream`	`vaporA`	`F`	0.0212163063101
`stream`	`vaporB`	`F`	0.0065614714677
`utility`	`secA.fa`	`heating.steamLP.duty_kW`	686.964038241
`utility`	`secA.fa`	`heating.steamLP.T`	370
`utility`	`secA.fa`	`heating.steamLP.kg_s`	0.314256193157
`utility`	`secA.fa`	`heating.steamLP.eur_h`	29.676846452
`utility`	`secB.fb`	`heating.steamLP.duty_kW`	226.733908578
`utility`	`secB.fb`	`heating.steamLP.T`	378
`utility`	`secB.fb`	`heating.steamLP.kg_s`	0.103720909688
`utility`	`secB.fb`	`heating.steamLP.eur_h`	9.79490485056
`stream`	`bottoms`	`T`	378
`stream`	`feed`	`T`	365
. . . and 2 more reference values in the case's `expected`.			

plant02_basename_twins

tutorials/steady/flowsheets/plant02_basename_twins

What it does: basename twins: a top-level ‘feed‘ and a DISTINCT sector-internal ‘A.feed‘ coexist

The story: the ADVERSARIAL alias case of design forum #87-P0.

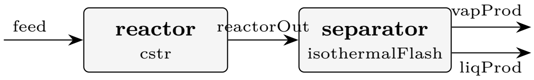
A top-level stream ‘feed‘ (the domain inlet) and a sector-INTERNAL edge also named ‘feed‘ (A.feed – a completely different stream between A’s two heaters) coexist. Identity is the PATH, never the basename: the manifest must carry BOTH (0/feed and 0/A/feed). The old basename filter silently dropped the top-level one from the manifest, so a legitimate case reported ORPHAN on its own inlet.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	0.00100000000002
`stream`	`hot`	`F`	0.00100000000002
`stream`	`out`	`F`	0.00100000000002
`kpi`	`h`	`F`	0.00100000000002
`kpi`	`h`	`P`	100000
`kpi`	`h`	`Q`	2000
`kpi`	`h`	`Q_kW`	2
`kpi`	`h`	`Q_per_mol`	1999.99999997
`kpi`	`h`	`T_in`	298.15
`kpi`	`h`	`T_out`	366.647115308
`kpi`	`h`	`dT`	68.497115308
`utility`	`A.h1`	`heating.steamMP.duty_kW`	1
`utility`	`A.h1`	`heating.steamMP.T`	400.742542906
`utility`	`A.h1`	`heating.steamMP.kg_s`	0.000502260170768
. . . and 13 more reference values in the case’s `expected`*.*			

`process01_reactor_flash`

`tutorials/steady/flowsheets/process01_reactor_flash`



What it does: Two-unit flowsheet: CSTR → isothermal flash

The story: Two-unit flowsheet: feed → CSTR → flash → (liquid, vapour)

feed reactorOut reactor separator (CSTR) (T 365, P 1 bar) liqProd vapProd

Reaction kinetics are in constant/reactions Solver defaults are in system/solverDict

A 1 kmol/h equimolar ethanol / acetic-acid feed enters a 5-litre CSTR at 350 K; the reactor outlet goes straight into a flash held at 365 K and 1 bar. Two units, three streams, and one idea: a flowsheet is a graph of streams, and information follows them.

The golden: conversion of ethanol X = 0.5259 at a residence time of 310.3 s (Damk?hler number 1.109, k = 3.575e-3 s?^{1} from the Arrhenius law), the reactor removing 1.88 kW; the separator flashes 45.2 % of what arrives (0.452 kmol/h vapour, 0.548 kmol/h liquid) with K_ethanol 1.696, K_ethylAcetate 1.608, K_aceticAcid 0.428, K_water 0.767, taking 4.98 kW to hold 365 K.

1. The topology is the whole of flowsheetDict. Two units, each naming its in and outputs; the stream reactorOut is produced by one and consumed by the other, and that is what makes it internal. No stream values live there — they live in 0/, one file per stream. 2. The reaction is a named record, constant/reactions (esterification_etac): stoichiometry, which reactant is limiting, and an Arrhenius A/Ea. Its own header says the kinetics are ILLUSTRATIVE pseudo-first-order — read that before quoting a rate. 3. Units run in declared order, reactor first. Reverse the two entries and the engine refuses, naming the stream that would be read before it was produced. 4. Every balance is drawn from the engine, by default. The Reports tab shows mass, per-element and energy closures for the whole sheet — the flash's 4.98 kW and the reactor's -1.88 kW are both in the ledger.

Double the reactor volume (operation { V_R 0.010; }): conversion rises, the flash feed changes, and the separator's duty follows — one edit, three consequences, all visible in Streams. Then open process03_recycle, where a stream loops back and the solver has to iterate the whole graph.

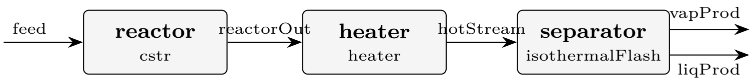
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`liqProd`	`F`	`0.000152263837157`
`stream`	`reactorOut`	`F`	`0.000277777777778`
`stream`	`vapProd`	`F`	`0.00012551394062`
`kpi`	`reactor`	`Da_kTau`	`1.10929132154`
`kpi`	`reactor`	`F_in_kmol_h`	`1`
`kpi`	`reactor`	`F_out_kmol_h`	`1`
`kpi`	`reactor`	`Q_kW`	`-1.87948262728`
`kpi`	`reactor`	`T`	`350`
`kpi`	`reactor`	`V_R`	`0.005`
`kpi`	`reactor`	`X_limiting`	`0.52590711876`
`kpi`	`reactor`	`k`	`0.00357499942014`
`kpi`	`reactor`	`minus_r_ethanol`	`14.6085310767`
`kpi`	`reactor`	`tau_s`	`310.291329081`

. . . and 22 more reference values in the case's `expected`.

`process02_reactor_hx_flash`

`tutorials/steady/flowsheets/process02_reactor_hx_flash`



What it does: Three-unit flowsheet: CSTR → heater → flash

The story: Three-unit flowsheet — physically realistic operation:

feed (350 K) hotStream (~359 K, 0.9 % vap) reactorOut CSTR 350 K, X=52% flash [reactor] [heater] [separator] vapProd liqProd

The flash inherits T from its input stream (no explicit operation.T) — propagation through the flowsheet.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	hotStream	F	0.000277777777778
stream	liqProd	F	0.000262168078511
stream	reactorOut	F	0.000277777777778
stream	vapProd	F	1.56096992667e-05
kpi	heater	F	0.000277777777778
kpi	heater	P	101325
kpi	heater	Q	400
kpi	heater	Q_kW	0.4
kpi	heater	Q_per_mol	1440
kpi	heater	T_in	350
kpi	heater	T_out	361.879907822
kpi	heater	dT	11.8799078219
kpi	heater	vf_out	0.00022200059932

. . . and 37 more reference values in the case's `expected`.

process02_with_design

tutorials/steady/flowsheets/process02_with_design

What it does: process02 with full reports: balances + sizing + Guthrie costing

The story: Three-unit flowsheet — physically realistic operation:

feed (350 K) hotStream (~359 K, 0.9 % vap) reactorOut CSTR 350 K, X=52% flash [reactor] [heater] [separator] vapProd liqProd

The flash inherits T from its input stream (no explicit operation.T) — propagation through the flowsheet.

process01 with a heater between the reactor and the flash, and the two reports a project needs most: equipment sizing and Guthrie costing. The chain runs from system/postDict; reports { design {} economics {} } in the controlDict ∗serialise∗ what it produced into reports/design/sizing.csv and reports/economics/costs.csv.

The golden (process side): heater 0.4 kW → 361.88 K; reactor X = 0.526; flash at 361.88 K with V/F 0.0562. The design side, from the run: reactor V_R = 0.005 m^3, wall 3.15 mm, 3.7 kg of SS316; heater A = 0.0667 m^2 of SS304; costing totals ?155,475 purchased / ?917,756 bare module / ?1,082,952 total module, CEPCI 820/397 = 2.0655, 0.92 EUR/USD.

1. The rules are declared, once, in postDict. sizing { units ({ unitName reactor; type stirredTank; material SS316; designRules { L_over_D 2.5; pressureDesign 3.0; ? } } ?) } and costing { method Guthrie; year 2026; cepci 820; cepci2001 397; usdToEur 0.92; }. Every number in the tables descends from those lines and the run's own results — nothing else. 2. The costing table prints its own arithmetic. Below the totals: the log-quadratic correlation, K1–K3 per item, B1/B2, F_M, the CEPCI ratio, the currency, the 1.18 contingency — and the source (Turton App. A). Recompute one line; check_cost_provenance does exactly that on every suite run and requires 0.1 %. 3. READ THE TWO WARNINGS — they are the real lesson of this case. [validity] WARNING: stirredTank 'reactor': size V_R = 0.005 is OUTSIDE the correlation range [0.3, 520] and the heater's 0.0667 m^2 against [10, 1000]. A 1 kmol/h process is laboratory scale; Turton's correlations were fitted to plant scale, and the engine ∗extrapolates and says so∗ rather than clamping. The ?1.06 M for a 0.4 kW exchanger is what an extrapolated correlation returns — a number to distrust because you were told to. 4. A report draws, it does not recompute. Until 2026-09-02 this case carried the rules inside the report blocks (an older grammar), the reports failed by name on every run, and the golden — which pins only the process KPIs — never noticed. The README you are reading is why it was found.

Scale the feed to 100 kmol/h in 0/feed: the reactor volume the design rule wants lands inside the correlation window and the warning goes away. Then open economics01_esterification_dcf, where the same chain continues into OPEX, NPV and payback.

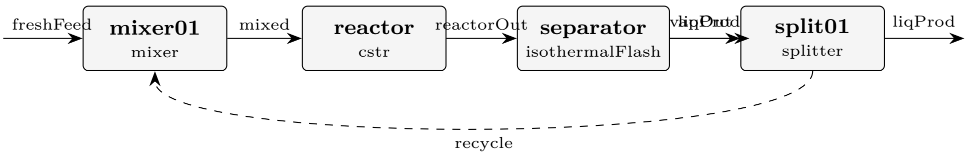
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`hotStream`	`F`	`0.000277777777778`
`stream`	`liqProd`	`F`	`0.000262168078511`
`stream`	`reactorOut`	`F`	`0.000277777777778`
`stream`	`vapProd`	`F`	`1.56096992667e-05`
`kpi`	`heater`	`F`	`0.000277777777778`
`kpi`	`heater`	`P`	`101325`
`kpi`	`heater`	`Q`	`400`
`kpi`	`heater`	`Q_kW`	`0.4`
`kpi`	`heater`	`Q_per_mol`	`1440`
`kpi`	`heater`	`T_in`	`350`
`kpi`	`heater`	`T_out`	`361.879907822`
`kpi`	`heater`	`dT`	`11.8799078219`
`kpi`	`heater`	`vf_out`	`0.00022200059932`

. . . and 37 more reference values in the case's `expected`.

`process03_recycle`

`tutorials/steady/flowsheets/process03_recycle`



What it does: Recycle loop: the tear stream is solved by Newton (2 iterations, |r|2 3.9e-6)

The story: the classic Reactor-Separator-Recycle architecture

freshFeed vapProd (product) [mixer] [reactor] [separator] (flash) liqProd [splitter] recycle purge

(the splitter is fed by liqProd)

Tear stream: ‘recycle‘ — declared explicitly; the Flowsheet runs NEWTON on the tear variables (F, z..., T) until the post-pass value matches the input guess to a tolerance. (‘recycleSolver Wegstein;‘ selects the fixed-point accelerator instead — try both and compare the iteration counts.)

The CSTR converts ~25% per pass (small volume); without recycle most ethanol would simply leave in liqProd. With ~80% recycle, the overall ethanol conversion approaches 100% at steady state.

process01 with a loop: reactor → flash → splitter, and 60 % of the flash liquid is sent ∗back∗ to a mixer ahead of the reactor. The reactor is four times larger (20 litres). The golden: the reactor now sees 1.497 kmol/h (1.0 fresh + 0.497 recycle), converts 80.9 % per pass at τ = 870 s, and the loop closes with the recycle stream at 0.4967 kmol/h, 364 K.

1. A cycle needs a cut, and the author declares it. system/solverDict says tearStreams (recycle); — the engine detects cycles, but ∗which∗ stream to tear is a choice, so it is written down. Reverse the units or forget the tear and the run refuses by name (INVALID ORDER, MISSING TEAR), never silently. 2. The tear starts from 0/recycle, a guess, and the log says so. Tear streams (initial guesses): recycle F = 0.7876 kmol/h ? — then a Newton over five variables (F, three compositions, T). The Log tab prints 2 per iteration: 1.34e-1 → 4.25e-3 → converged in 2 iterations at 3.9e-6. The seed was 60 % too high and it did not matter. 3. [plan] announces the cut before anything runs. material recycle: tear ’recycle’ cuts mixer01 → reactor → separator → split01 –recycle-→ mixer01 — read that line first; it is the whole topology in one sentence. 4. The recycle changes what the reactor is asked to do. Conversion per pass is 80.9 % because the reactor is larger and the feed is richer in what came back; overall, less ethanol leaves in liqProd. Compare the Streams tab against process01 line by line.

Change fractions (0.6 0.4) to (0.8 0.2): more recycle, a bigger loop flow, and watch whether the Newton still closes in two steps. recycle_autoinit_tear shows what happens when you give it ∗no∗ guess at all.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	freshFeed	F	0.000277777777778
stream	liqOut	F	0.000229958363658
stream	liqProd	F	9.19833454634e-05
stream	mixed	F	0.000415752593523
stream	reactorOut	F	0.000415752593523
stream	recycle	F	0.000137975018195
stream	vapProd	F	0.000185794229864
kpi	mixer01	F_out	0.000415752593523
kpi	mixer01	P_out	100000
kpi	mixer01	T_out	354.51289855
kpi	mixer01	n_in	2
kpi	mixer01	vf	0
kpi	reactor	Da_kTau	4.22484004006
kpi	reactor	F_in_kmol_h	1.49670933668

. . . and 33 more reference values in the case’s `expected`.

`process04_research_workflow`

`tutorials/steady/flowsheets/process04_research_workflow`

What it does: Case-local binary pair file (research workflow pattern)

The story: isothermal flash of ethanol/water at 355 K, 1 atm

Identical to flash02_ethanol_water, but this case demonstrates the RESEARCH WORKFLOW: parameters loaded from case-local parameters file rather than declared inline. See README.md for the full discussion.

The research workflow for managing thermodynamic parameters: instead of declaring NRTL pairs (...) inline in the thermoPhysPropDict, the case declares only the model and lets the simulator load the parameters from a file inside the case directory.

This is the pattern used when you have your own fitted parameters that you do not want to mix into the standards database. The case remains self-contained and reproducible: the parameters travel with the case.

When the simulator builds the NRTL model and discovers there are no inline pairs in thermoPhysPropDict, it falls back to file lookup:

1. Case-local – <case>/constant/parameters/NRTL/ethanol-water.dat ? used here 2. Standards – $CHOUPO_HOME/data/standards/parameters/NRTL/ethanol-water.dat

In this case the local file exists, so it wins. If you delete the local file, the run still succeeds because the standards file is the same DECHEMA set.

The result is numerically identical to flash02_ethanol_water (same NRTL parameters; just a different way of supplying them).

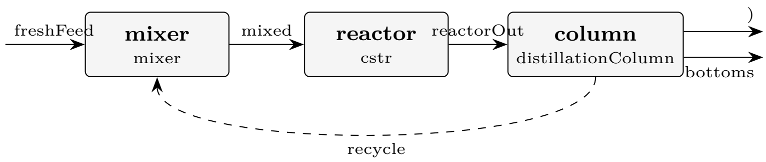
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	liquid	F	0.00562323865754
stream	vapor	F	0.0221545391202
kpi	flash01	F_alpha	0.00562323865754
kpi	flash01	F_beta	0.0221545391202
kpi	flash01	F_in	0.0277777777778
kpi	flash01	P	101325
kpi	flash01	T	355
kpi	flash01	V_over_F	0.797563408329
kpi	flash01	phaseSet	0
stream	feed	T	355
stream	liquid	T	355
stream	vapor	T	355
kpi	flash01	K_ethanol	2.22741301451

. . . and 3 more reference values in the case's `expected`.

process05_isomerization_recycle

tutorials/steady/flowsheets/process05_isomerization_recycle



What it does: Isomerization neoPentane→nPentane: mixer + CSTR + distillation with recycle

The story: Reactor + distillation with recycle of the unconverted reactant — the canonical flowsheet for exercising the reports{} layer (stream table, mass balance with closure, energy balance) on a non-trivial topology with a tear stream.

freshFeed (neoPentane) [mixer] recycle (neoPentane) [reactor] neoPentane → nPentane (CSTR) [column] (distillation, alpha ~ 2.5) distillate bottoms (neoPentane, (nPentane, recycled) (recycle) the PRODUCT

Chemistry: neoPentane → nPentane is an isomerization — 1:1 in moles AND mass (identical MW 72.15). So at steady state the fresh feed (1 kmol/h neoPentane) equals the product (1 kmol/h nPentane in the bottoms), and the overall MASS BALANCE CLOSES TO 100 %. The recycle carries per-pass unconverted neoPentane back to the reactor, so the OVERALL conversion is ~100 % (product is 99 % nPentane) even though the per-pass conversion is only ~80 %.

Tuning notes (honest): The loop runs at 5 bar so every neoPentane/nPentane stream is a genuine LIQUID at its temperature. neoPentane's Antoine here boils at ~275 K at 1 bar (Psat(320 K) ~4.3 bar), so a 1 bar loop would flash the feed + recycle to VAPOUR — the phase is a CONSEQUENCE of (T,P,z) (Duhem), not a silent liquid default. At 5 bar the bubble point clears ~326 K, so the warm reactor (~327 K) and the recycled distillate stay liquid, as the flowsheet intends. The reaction uses a LOW activation energy on purpose — a high-Ea rate would make the per-pass conversion collapse against the recycle temperature and destabilise the loop. Low Ea decouples conversion from the recycle temperature. 'distillateRate' is FIXED here (the column's spec). It cannot perfectly track the per-pass unconverted neoPentane, so the recycle carries some nPentane (product) along — exactly why real recycle loops need a purge. Closing the recycle PURITY would be a DesignSpec manipulating distillateRate; left out here to keep the case a single pass for reviewing the reports.

This is the case to open in the GUI and to read the reports/ CSVs from: streamTable.csv, massBalance.csv (+ _byUnit), energyBalance_byUnit.csv.

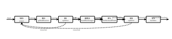
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.000277777777778`
`stream`	`freshFeed`	`F`	`0.000277777777778`
`stream`	`mixed`	`F`	`0.000363888888889`
`stream`	`reactorOut`	`F`	`0.000363888888889`
`stream`	`recycle`	`F`	`8.61111111111e-05`
`kpi`	`column`	`B`	`0.000277777777778`
`kpi`	`column`	`D`	`8.61111111111e-05`
`kpi`	`column`	`L_rect`	`0.000172222222222`
`kpi`	`column`	`L_strip`	`0.000536111111111`
`kpi`	`column`	`R`	`2`
`kpi`	`column`	`T_bottom`	`364.544713604`
`kpi`	`column`	`T_top`	`349.6579691`
`kpi`	`column`	`V_rect`	`0.000258333333333`
`kpi`	`column`	`V_strip`	`0.000258333333333`

. . . and 35 more reference values in the case's `expected`.

proxy01_gas_loop

tutorials/steady/flowsheets/proxy01_gas_loop



What it does: Proxy step 1: gas WGS loop with one recycle (mixer + Gibbs reactor + splitter)

The story: The GUI/credo bench in one case: TWO recycles + a shaft-work energy wire (W) + a utility STREAM (cooling water). Toggle the stream classes in the canvas: energy (the W wire), recycle (the two tears), utility (the cooling-water edge).

feed M1 mixer recycle1 (hot) recycle2 (cold, dry) R1 gibbsReactor (WGS) S1 splitter recycle1 (30 % hot gas back) toFlash CW (cooling water, UTILITY) HX1 warmCW (HX1: sensible cooling of the gas) F1 flash { } (no knobs: inherits T,P; condenses the water out) productWater (liquid) product dryGas (vapour) S2 splitter recycle2 (30 % dry gas back) productGas T1 turbine (power recovery) G1 electricLoad (W wire) expandedProduct

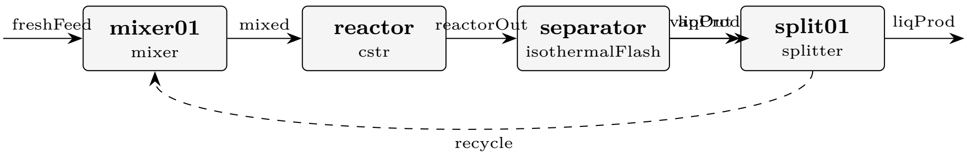
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	CW	F	0.0166666666667
stream	cooledGas	F	0.000375486726088
stream	dryGas	F	0.000325696494167
stream	expandedProduct	F	0.000227987545917
stream	feed	F	0.000277777777778
stream	mixed	F	0.000536409608696
stream	productGas	F	0.000227987545917
stream	productWater	F	4.97902319203e-05
stream	reacted	F	0.000536409608697
stream	recycle1	F	0.000160922882609
stream	recycle2	F	9.77089482502e-05
stream	toFlash	F	0.000375486726088
stream	warmCW	F	0.0166666666667
kpi	F1	F_alpha	4.97902319203e-05

. . . and 86 more reference values in the case's `expected`.

`recycle_autoinit_tear`

`tutorials/steady/flowsheets/recycle_autoinit_tear`



What it does: Recycle with NO authored tear guess – the solver auto-seeds the tear from the feed aggregate (announced), then converges

The story: same Reactor-Separator-Recycle as process03_recycle; this case's LESSON is the TEAR-SEED CONTRACT of the stream-state architecture:

the tear ‘recycle‘ has an AUTHORED state file, 0/recycle – the seed is a first-class case file, owned by the author; completeness is FATAL before the solve: delete 0/recycle and choupoSolve REFUSES (MISSING, zero iterations run); ‘bin/choupo-init0‘ refuses to INVENT a recycle seed (propagation is only honest on an acyclic graph) – it names the missing file and leaves the decision to you.

Compare the seed in 0/recycle with process03_recycle's: a recycle seed is a NUMERICAL aid the student owns, honest and visible ("no silent crutch", CLAUDE.md). (Historical note, marked legacy: on the retired ‘streams {}‘ reader path a missing tear guess was auto-seeded from the feed aggregate, announced on the log; the 0/ contract replaced that with the authored, refuse-if-missing seed above.)

freshFeed vapProd (product) [mixer] [reactor] [separator] (flash) liqProd [splitter] recycle purge

(the splitter is fed by liqProd)

Tear stream: ‘recycle‘ — declared explicitly; the Flowsheet runs Wegstein on (F, z..., T) until the post-pass value matches the input guess to a tolerance.

The CSTR converts ~25% per pass (small volume); without recycle most ethanol would simply leave in liqProd. With ~80% recycle, the overall ethanol conversion approaches 100% at steady state.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`freshFeed`	`F`	`0.000277777777778`
`stream`	`liqOut`	`F`	`0.000229958363961`
`stream`	`liqProd`	`F`	`9.19833455844e-05`
`stream`	`mixed`	`F`	`0.000415752593881`
`stream`	`reactorOut`	`F`	`0.000415752593881`
`stream`	`recycle`	`F`	`0.000137975018377`
`stream`	`vapProd`	`F`	`0.00018579422992`
`kpi`	`mixer01`	`F_out`	`0.000415752593881`
`kpi`	`mixer01`	`P_out`	`100000`
`kpi`	`mixer01`	`T_out`	`354.512898558`
`kpi`	`mixer01`	`n_in`	`2`
`kpi`	`mixer01`	`vf`	`0`
`kpi`	`reactor`	`Da_kTau`	`4.2248400393`
`kpi`	`reactor`	`F_in_kmol_h`	`1.49670933797`

. . . and 33 more reference values in the case's `expected`.

gas-solid

bagFilter01_dust

tutorials/steady/gas-solid/bagFilter01_dust

What it does: Bag filter (baghouse) on a dusty gas: >99% capture + cake pressure drop

The story: The high-efficiency complement to cyclone01 (same dusty gas: N2 carrying 50 kg/h of silica, PSD 1..50 um). Where the cyclone captured ~82 % by SIZE (fines escape), the fabric filter's dust cake captures ~99.9 % across all sizes — the price is a much larger pressure drop, dominated by the cake (dP = mu V (K1 + K2 W)).

Face velocity V = Q__gas / filterArea is the air-to-cloth ratio (~1.5 m/min here); with a cake load W = 0.5 kg/m^2 the pressure drop is ~2.7 kPa. The classic train is cyclone (cheap pre-cleaner) THEN bag filter (polish) — this case is the polish stage on the same dust.

Gas: 150 kmol/h N2, 300 K, 1 atm Dust: 50 kg/h silica, PSD 1..50 um Filter: 40 m^2 cloth

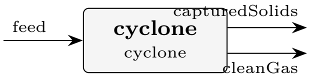
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`capturedSolids`	`F`	`0.000231019341285`
`stream`	`cleanGas`	`F`	`0.0416668205759`
`stream`	`feed`	`F`	`0.0418978399171`
`kpi`	`baghouse`	`airToCloth_m_min`	`1.53857549963`
`kpi`	`baghouse`	`arealDustLoad`	`0.5`
`kpi`	`baghouse`	`dP_filter`	`2745.33935569`
`kpi`	`baghouse`	`efficiency`	`0.999334225794`
`kpi`	`baghouse`	`faceVelocity`	`0.0256429249938`
`kpi`	`baghouse`	`filterArea`	`40`
`kpi`	`baghouse`	`mu_gas`	`1.78433840676e-05`
`kpi`	`baghouse`	`penetration`	`0.000665774205525`
`kpi`	`baghouse`	`rho_gas`	`1.13794383208`
`kpi`	`baghouse`	`solidsCaptured_mass`	`0.0138796420244`
`kpi`	`baghouse`	`solidsEscaped_mass`	`9.24686396528e-06`

. . . and 4 more reference values in the case's `expected`.

`cyclone01_dust_removal`

`tutorials/steady/gas-solid/cyclone01_dust_removal`



What it does: Cyclone dust removal: silica from N2 (Lapple, D_c = 0.5 m)

The story: A dusty gas (N2 carrying 50 kg/h of silica) passes through a cyclone. The cyclone separates BY SIZE: coarse particles spin to the wall and fall out (capturedSolids), fine particles follow the gas (cleanGas). This is the whole point of the PSD — a cyclone is a size classifier, not a simple splitter.

Lapple cut diameter d50 ~ 2.7 um for this geometry/flow. The grade efficiency η(d) = 1/(1+(d50/d)^2): ~12 % at 1 um, ~93 % at 10 um. Overall collection ~ 82 % of the dust mass; the escaping fines are the sub-d50 tail (why a cyclone is a pre-cleaner, followed by a bag filter).

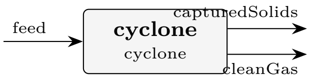
Gas: 150 kmol/h N2, 300 K, 1 atm Dust: 50 kg/h silica, ?_p = 2650 kg/m^3, PSD 1..50 um Cyclone: body diameter 0.5 m, 5 effective turns

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`capturedSolids`	`F`	`0.000189667114932`
`stream`	`cleanGas`	`F`	`0.0417081728022`
`stream`	`feed`	`F`	`0.0418978399171`
`kpi`	`cyclone`	`bodyDiameter`	`0.5`
`kpi`	`cyclone`	`d50`	`2.71094994999e-06`
`kpi`	`cyclone`	`d50_micron`	`2.71094994999`
`kpi`	`cyclone`	`efficiency`	`0.820454419117`
`kpi`	`cyclone`	`inletVelocity`	`32.8229439921`
`kpi`	`cyclone`	`mu_gas`	`1.78433840676e-05`
`kpi`	`cyclone`	`numberOfTurns`	`5`
`kpi`	`cyclone`	`rho_gas`	`1.13794383208`
`kpi`	`cyclone`	`rho_particle`	`2650`
`kpi`	`cyclone`	`solidsCaptured_mass`	`0.0113952002651`
`kpi`	`cyclone`	`solidsEscaped_mass`	`0.00249368862328`
. . . and 8 more reference values in the case's `expected`*.*			

`cyclone02_leith_licht`

`tutorials/steady/gas-solid/cyclone02_leith_licht`



What it does: Cyclone dust removal – Leith-Licht back-mixing model

The story: The same dusty gas as cyclone01, but the grade efficiency uses the Leith-Licht back-mixing model instead of Lapple. Accounting for turbulent re-entrainment in the vortex, Leith-Licht predicts a SMALLER cut diameter (finer collection) for the same geometry/flow.

Leith-Licht cut diameter d50 ~ 1.11 um here. The grade efficiency is ~48 % at 1 um, ~94 % at 10 um. Overall collection ~ 88 % of the dust mass — higher than Lapple's 82 % because more of the sub-micron tail is captured.

Gas: 150 kmol/h N2, 300 K, 1 atm Dust: 50 kg/h silica, ?_p = 2650 kg/m^3, PSD 1..50 um Cyclone: body diameter 0.5 m, 5 effective turns

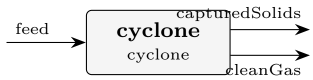
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`capturedSolids`	`F`	`0.000203443637145`
`stream`	`cleanGas`	`F`	`0.04169439628`
`stream`	`feed`	`F`	`0.0418978399171`
`kpi`	`cyclone`	`bodyDiameter`	`0.5`
`kpi`	`cyclone`	`d50`	`1.10984825815e-06`
`kpi`	`cyclone`	`d50_micron`	`1.10984825815`
`kpi`	`cyclone`	`efficiency`	`0.880048347849`
`kpi`	`cyclone`	`inletVelocity`	`32.8229439921`
`kpi`	`cyclone`	`mu_gas`	`1.78433840676e-05`
`kpi`	`cyclone`	`numberOfTurns`	`5`
`kpi`	`cyclone`	`rho_gas`	`1.13794383208`
`kpi`	`cyclone`	`rho_particle`	`2650`
`kpi`	`cyclone`	`solidsCaptured_mass`	`0.0122228937197`
`kpi`	`cyclone`	`solidsEscaped_mass`	`0.0016659951687`

. . . and 8 more reference values in the case's `expected`.

cyclone03_iozia_leith

tutorials/steady/gas-solid/cyclone03_iozia_leith



What it does: Cyclone dust removal – Iozia-Leith data-fitted logistic (beta)

The story: The same dusty gas as cyclone01, now through the REAL Iozia-Leith (1989) FLOW-PATTERN model: the cut diameter d50 is NOT the Lapple residence-time surrogate but falls out of the gas flow pattern – the critical particle sits at the core radius where the tangential velocity peaks, and d50 = sqrt(9 mu Q /(pi z_c rho_p Vt_max^2)) (their eq 4), with Vt_max, the core diameter and its length regressed from the cyclone's OWN geometry (eqs 5-7). The grade curve stays logistic, eta(d) = 1/(1+(d50/d)^beta), beta = 3.5. (See the forum note: the 1989 eq-5 prefactor 6.1 is a decimal typo for 0.61 – 6.1 gives a supersonic Vt_max, against the paper's own Figure 4; we use the Figure-4-consistent 0.61.)

Flow-pattern d50 ~ 1.0 um – FINER than Lapple's 2.71 um: a Stairmand-shaped cyclone spins harder than the residence-time picture assumes, so it collects smaller particles. Overall collection RISES accordingly. Geometry defaults to the Stairmand high-efficiency ratios (add an operation.geometry {} block to override a/b/De/S/h/H/B).

Gas: 150 kmol/h N2, 300 K, 1 atm Dust: 50 kg/h silica, ?_p = 2650 kg/m^3, PSD 1..50 um Cyclone: body diameter 0.5 m, 5 effective turns, beta = 3.5

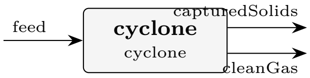
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	capturedSolids	F	0.000223552490081
stream	cleanGas	F	0.0416742874271
stream	feed	F	0.0418978399171
kpi	cyclone	Q_gas	1.02571699975
kpi	cyclone	bodyDiameter	0.5
kpi	cyclone	d50	9.86636018962e-07
kpi	cyclone	d50_micron	0.986636018962
kpi	cyclone	dP_cyclone	4903.83536027
kpi	cyclone	efficiency	0.96703441953
kpi	cyclone	eulerNumber	8
kpi	cyclone	inletVelocity	32.8229439921
kpi	cyclone	loading	0.0118992372584
kpi	cyclone	mu_gas	1.78433840676e-05
kpi	cyclone	numberOfTurns	5

. . . and 8 more reference values in the case's `expected`.

cyclone04_high_loading

tutorials/steady/gas-solid/cyclone04_high_loading



What it does: Cyclone high dust loading – Muschelknautz mass-loading effect

The story: The same cyclone geometry as cyclone01, but with a HEAVY dust load — 2000 kg/h of silica (inlet loading ~ 0.476 kg dust / kg gas, ~40x the ~0.012 kg/kg of cyclone01). At high loading the Muschelknautz model captures the mass-loading effect: above a critical loading the dust forms a wall strand that sweeps out particles independently of the single-particle cut, so the collection efficiency RISES sharply.

Muschelknautz overall collection ~ 98.4 % of the dust mass — far above the ~82 % a dilute Lapple cyclone gives, because the high solids loading itself enhances separation. Grade efficiency ~ 90 % even at 1 um.

Gas: 150 kmol/h N2, 300 K, 1 atm Dust: 2000 kg/h silica, ?_p = 2650 kg/m^3, PSD 1..50 um Cyclone: body diameter 0.5 m, 5 effective turns, cLimit = 0.05

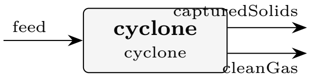
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	capturedSolids	F	0.00909835478473
stream	cleanGas	F	0.0418152419014
stream	feed	F	0.0509135966861
kpi	cyclone	bodyDiameter	0.5
kpi	cyclone	d50	1e-09
kpi	cyclone	d50_micron	0.001
kpi	cyclone	dP_cyclone	4903.83536027
kpi	cyclone	efficiency	0.983932479818
kpi	cyclone	eulerNumber	8
kpi	cyclone	inletVelocity	32.8229439921
kpi	cyclone	loading	0.475969490367
kpi	cyclone	mu_gas	1.78433840676e-05
kpi	cyclone	numberOfTurns	5
kpi	cyclone	rho_gas	1.13794383208

. . . and 8 more reference values in the case's `expected`.

cyclone05_dirgo_leith_validation

tutorials/steady/gas-solid/cyclone05_dirgo_leith_validation



What it does: VALIDATION vs Dirgo & Leith (1985): a Stairmand high-efficiency cyclone (D=0.305 m, 0.139 m3/s, ~15 m/s inlet, mineral-oil aerosol) run through the flow-pattern Iozia-Leith model. Swap the model slot to see the ~3x d50 spread across correlations on ONE real cyclone – why you validate against data.

The story: VALIDATION against Dirgo & Leith (1985), Aerosol Sci. Technol. 4:401 – the measured fractional-efficiency baseline for a Stairmand high-efficiency cyclone. Their pilot-plant cyclone: body diameter D = 0.305 m, design air flow 0.139 m^3/s (300 cfm) → inlet velocity ~15 m/s; test aerosol a nebulised mineral oil (rho_p = 860 kg/m^3), particles >= 1 um.

This case reproduces THOSE conditions and runs the flow-pattern Iozia-Leith (1989) model. Re-run it with 'model Lapple;' / 'LeithLicht;' / 'Barth;' / 'Muschelknautz;' and compare each model's d50 and grade curve against the paper's measured fractional-efficiency curves (their Figures 5-6). THE LESSON: on this ONE real cyclone the models disagree by ~3x – Iozia-Leith 2.2 um, Leith-Licht 2.6 um (the fine, high-efficiency end) Lapple / Muschelknautz 6.2 um, Barth 6.5 um (the coarse end) – which is EXACTLY why you validate against data rather than trust one correlation. Dirgo & Leith's own finding: the theories BRACKET the measured curve and no single one is universally best (they consistently underpredicted efficiency at the highest inlet velocity, 25 m/s). The geometry defaults to the Stairmand high-efficiency ratios (a/D 0.5, b/D 0.2, De/D 0.5, S/D 0.5, h/D 1.5, H/D 4, B/D 0.375) – exactly Dirgo & Leith's cyclone – so the Iozia-Leith flow pattern (Vt_max, core) is evaluated on the real test geometry; Vi = Q/(ab) ~ 15 m/s reproduces their design point.

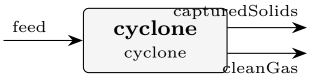
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	capturedSolids	F	6.95101380068e-05
stream	cleanGas	F	0.00566837296963
stream	feed	F	0.00573788310764
kpi	cyclone	Q_gas	0.138813700633
kpi	cyclone	bodyDiameter	0.305
kpi	cyclone	d50	2.24296780859e-06
kpi	cyclone	d50_micron	2.24296780859
kpi	cyclone	dP_cyclone	648.673413872
kpi	cyclone	efficiency	0.702163610002
kpi	cyclone	eulerNumber	8
kpi	cyclone	inletVelocity	11.9377544216
kpi	cyclone	loading	0.00879253984757
kpi	cyclone	mu_gas	1.78433840676e-05
kpi	cyclone	numberOfTurns	5

. . . and 8 more reference values in the case's `expected`.

cyclone06_barth

tutorials/steady/gas-solid/cyclone06_barth



What it does: WITNESS: the Barth (1956) equilibrium-orbit cyclone model on the Dirgo & Leith Stairmand geometry (same design point as cyclone05); self-recorded golden – exercises a registered model no case selected.

The story: WITNESS for the Barth (1956) cyclone model – the static-pressure / equilibrium-orbit correlation – on the SAME Stairmand high-efficiency geometry and design point as cyclone05_dirgo_leith_validation (D = 0.305 m, 0.139 m^3/s, Vi ~ 15 m/s, mineral-oil aerosol rho_p = 860 kg/m^3).

THIS IS A WITNESS, NOT A VALIDATION: its golden is self-recorded, so a PASS means Barth's answer has not moved – never that it is right. The validation discussion (the ~3x d50 spread across the five correlations, and Dirgo & Leith's measured curves) lives in cyclone05's header; this case exists because Barth was a registered model that NO case selected, which invariant I8 names as not-a-capability (2026-08-22 fresh-eyes audit). Until today its only exercise was an invitation in a comment.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	capturedSolids	F	2.14047701787e-05
stream	cleanGas	F	0.00571647833746
stream	feed	F	0.00573788310764
kpi	cyclone	Q_gas	0.138813700633
kpi	cyclone	bodyDiameter	0.305
kpi	cyclone	d50	6.49490874523e-06
kpi	cyclone	d50_micron	6.49490874523
kpi	cyclone	dP_cyclone	648.673413872
kpi	cyclone	efficiency	0.216222426411
kpi	cyclone	eulerNumber	8
kpi	cyclone	inletVelocity	11.9377544216
kpi	cyclone	loading	0.00879253984757
kpi	cyclone	mu_gas	1.78433840676e-05
kpi	cyclone	numberOfTurns	5

. . . and 8 more reference values in the case's `expected`.

gasSolidSplitter01_ideal

tutorials/steady/gas-solid/gasSolidSplitter01_ideal

What it does: Ideal gas-solid splitter: specified 90% solids recovery (no efficiency physics)

The story: The same dusty gas as cyclone01 / bagFilter01 (N_2 + 50 kg/h silica), but through an IDEAL, spec-driven separator: you SET the recovery (90 % of the solids to capturedSolids), no efficiency physics, no size classification.

Use it when a physics model is overkill ("assume 90 % capture"), as a placeholder, or to bound a design. Contrast: the cyclone COMPUTES ~82 % by size, the bag filter COMPUTES ~99.9 %; here the number is whatever you declare.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	capturedSolids	F	0.000208055925425
stream	cleanGas	F	0.0416897839917
stream	feed	F	0.0418978399171
kpi	separator	gasCarryover	0
kpi	separator	gasToCleanGas	0.0416666666667
kpi	separator	gasToSolids	0
kpi	separator	solidsCaptured_mass	0.0124999999995
kpi	separator	solidsEscaped_mass	0.00138888888884
kpi	separator	solidsIn_mass	0.0138888888884
kpi	separator	solidsRecovery	0.9
stream	capturedSolids	T	300
stream	cleanGas	T	300
stream	feed	T	300

gasSolidSplitter02_rosinRammler

tutorials/steady/gas-solid/gasSolidSplitter02_rosinRammler

What it does: gas-solid split with a FITTED Rosin-Rammler feed PSD (the model is declared, the engine discretises)

The story: The same dusty gas as cyclone01 / bagFilter01 (N2 + 50 kg/h silica), but through an IDEAL, spec-driven separator: you SET the recovery (90 % of the solids to capturedSolids), no efficiency physics, no size classification.

Use it when a physics model is overkill ("assume 90 % capture"), as a placeholder, or to bound a design. Contrast: the cyclone COMPUTES ~82 % by size, the bag filter COMPUTES ~99.9 %; here the number is whatever you declare.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	capturedSolids	F	0.000208055925425
stream	cleanGas	F	0.0416897839917
stream	feed	F	0.0418978399171
kpi	separator	gasCarryover	0
kpi	separator	gasToCleanGas	0.0416666666667
kpi	separator	gasToSolids	0
kpi	separator	solidsCaptured_mass	0.0124999999995
kpi	separator	solidsEscaped_mass	0.00138888888884
kpi	separator	solidsIn_mass	0.0138888888884
kpi	separator	solidsRecovery	0.9
stream	capturedSolids	T	300
stream	cleanGas	T	300
stream	feed	T	300

gasSolidSplitter03_tabular_psd

tutorials/steady/gas-solid/gasSolidSplitter03_tabular_psd

What it does: WITNESS: tabular (sieve-analysis) particle-size distribution on the gas-solid splitter; sibling of gasSolidSplitter02 (Rosin-Rammler).

The story: The same dusty gas as cyclone01 / bagFilter01 (N2 + 50 kg/h silica), but through an IDEAL, spec-driven separator: you SET the recovery (90 % of the solids to capturedSolids), no efficiency physics, no size classification.

Use it when a physics model is overkill ("assume 90 % capture"), as a placeholder, or to bound a design. Contrast: the cyclone COMPUTES ~82 % by size, the bag filter COMPUTES ~99.9 %; here the number is whatever you declare.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	capturedSolids	F	0.000208055925425
stream	cleanGas	F	0.0416897839917
stream	feed	F	0.0418978399171
kpi	separator	gasCarryover	0
kpi	separator	gasToCleanGas	0.0416666666667
kpi	separator	gasToSolids	0
kpi	separator	solidsCaptured_mass	0.0124999999995
kpi	separator	solidsEscaped_mass	0.00138888888884
kpi	separator	solidsIn_mass	0.0138888888884
kpi	separator	solidsRecovery	0.9
stream	capturedSolids	T	300
stream	cleanGas	T	300
stream	feed	T	300

gasSolidSplitter04_lognormal_psd

tutorials/steady/gas-solid/gasSolidSplitter04_lognormal_psd

What it does: WITNESS: log-normal particle-size distribution (dgm, sigmaG) on the gas-solid splitter; sibling of gasSolidSplitter02.

The story: The same dusty gas as cyclone01 / bagFilter01 (N2 + 50 kg/h silica), but through an IDEAL, spec-driven separator: you SET the recovery (90 % of the solids to capturedSolids), no efficiency physics, no size classification.

Use it when a physics model is overkill ("assume 90 % capture"), as a placeholder, or to bound a design. Contrast: the cyclone COMPUTES ~82 % by size, the bag filter COMPUTES ~99.9 %; here the number is whatever you declare.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	capturedSolids	F	0.000208055925425
stream	cleanGas	F	0.0416897839917
stream	feed	F	0.0418978399171
kpi	separator	gasCarryover	0
kpi	separator	gasToCleanGas	0.0416666666667
kpi	separator	gasToSolids	0
kpi	separator	solidsCaptured_mass	0.0124999999995
kpi	separator	solidsEscaped_mass	0.00138888888884
kpi	separator	solidsIn_mass	0.0138888888884
kpi	separator	solidsRecovery	0.9
stream	capturedSolids	T	300
stream	cleanGas	T	300
stream	feed	T	300

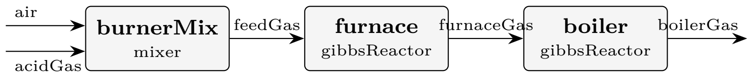
gibbs

The theory you need. Equilibrium without a mechanism: minimise total Gibbs energy $G = \sum_i n_i \mu_i$ over ALL species subject to elemental balances $\mathbf{A}\,\mathbf{n} = \mathbf{b}$ (Lagrange multipliers per element – printed as λ_E, the element potentials). The species list in the dict IS the model: what you include can form; what you omit cannot. With enough species this reproduces flame temperatures and radical pools with no kinetics at all: expect the adiabatic flame temperature to land within a few K of literature when radicals (H, O, OH…) are IN the pool, and to overshoot by ∼100–250 K when they are not (dissociation is endothermic). Above ∼1500 K the catalogue's polynomial c_p fits extrapolate WRONG – combustion cases carry case-local NASA-7 overrides, and the case headers say so.

Equilibrium maps. Sweeping the Gibbs solve over a $T \times P$ grid gives an equilibrium map – iso-lines of a product fraction over the operating plane – which shows why an industrial reactor fixes T and P in a narrow window (equilibrium wants one corner, the catalyst works in another; the gap is the compromise). The `temperatureApproach` ΔT evaluates the *reaction* equilibrium at $T + \Delta T$ while enthalpy and the energy balance stay at the physical T – an empirical, calibrated closeness-to-equilibrium knob (never predicted). *Gradeable exercise (ammonia):* on `nh3_equilibrium_map`, at $P = 200\,\text{bar}$ and $\Delta T = 0$, find the temperature at which the equilibrium NH_3 mole fraction crosses 30 %; report T to the grid resolution and verify against the case's anchor KPIs. Unique because the fraction is monotone in T at fixed P. *Full derivation: Theory Guide, the Gibbs-equilibrium chapter, §equilibrium maps.*

claus01_thermal_stage

tutorials/steady/gibbs/claus01_thermal_stage



What it does: Claus thermal stage: adiabatic Gibbs reactor over C-H-N-O-S

The story: Claus thermal stage: an adiabatic furnace and the boiler behind it, the same twelve species over the same five elements, 900 K apart.

Element order in every atoms( ) list is the order of ‘elements‘ below: (C H N O S). Nothing in this file declares a conversion, a temperature for the furnace, or which sulfur allotrope forms – all three are results. See system/controlDict for what comes out and what the model deliberately does not do.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	acidGas	F	0.0277777777778
stream	acidGas	T	320
stream	air	F	0.0595238094444
stream	air	T	298.15
stream	boilerGas	F	0.0786432844729
stream	boilerGas	T	600
stream	feedGas	F	0.0873015872222
stream	feedGas	T	305.967920996
stream	furnaceGas	F	0.0872586259716
stream	furnaceGas	T	1515.34061493
kpi	boiler	F_in_kmol_h	314.131053498
kpi	boiler	F_out_kmol_h	283.115824103
kpi	boiler	N_in_mol_s	87.2586259716
kpi	boiler	N_out_mol_s	78.6432844729

. . . and 55 more reference values in the case’s `expected`.

gibbs01_water_gas_shift

`tutorials/steady/gibbs/gibbs01_water_gas_shift`

What it does: Gibbs reactor: water-gas shift CO + H2O <→ CO2 + H2 at 800 K

The story: Water-gas shift reactor at 800 K, 1 bar. CO + water <→ CO2 + H2 Equimolar feed. The Gibbs-minimisation reactor gives CO conversion ~ 67 % (CO 0.50 → 0.164 mole fraction at equilibrium; CO2 = H2 = 0.336), consistent with the independent K(800 K) calculation in Smith, Van Ness, Abbott (Table 13.4).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	products	F	0.000277777777778
kpi	wgs	F_in_kmol_h	1
kpi	wgs	F_out_kmol_h	1
kpi	wgs	N_in_mol_s	0.277777777778
kpi	wgs	N_out_mol_s	0.277777777778
kpi	wgs	P	100000
kpi	wgs	T	800
kpi	wgs	converged	1
kpi	wgs	lambda_C	5549.05961227
kpi	wgs	lambda_H	-60095.3190791
kpi	wgs	lambda_O	-294758.913033
kpi	wgs	y_CO	0.16409231685
kpi	wgs	y_CO2	0.33590768315

... and 5 more reference values in the case's `expected`.

gibbs02_steam_reforming

tutorials/steady/gibbs/gibbs02_steam_reforming

What it does: Gibbs reactor: steam reforming of methane at 1000 K

The story: Industrial methane steam reforming feed: methane plus excess steam (steam-to-carbon ratio = 3, i.e. 1 mol CH4 + 3 mol H2O per pass). Expected at 1000 K, 1 bar: near-complete CH4 conversion, hydrogen yield ~ 2.7 mol H2 per mol CH4 (reforming + WGS combined).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.00111111111111
stream	syngas	F	0.0016586091548
kpi	reformer	F_in_kmol_h	4
kpi	reformer	F_out_kmol_h	5.97099295728
kpi	reformer	N_in_mol_s	1.11111111111
kpi	reformer	N_out_mol_s	1.6586091548
kpi	reformer	P	100000
kpi	reformer	T	1000
kpi	reformer	converged	1
kpi	reformer	lambda_C	-33527.5812363
kpi	reformer	lambda_H	-75161.3018601
kpi	reformer	lambda_O	-309185.611918
kpi	reformer	y_CH4	0.0024289965616
kpi	reformer	y_CO	0.0979819433622

. . . and 6 more reference values in the case's `expected`.

gibbs03_methane_combustion

tutorials/steady/gibbs/gibbs03_methane_combustion

What it does: Gibbs reactor: methane combustion in stoichiometric air at 1800 K

The story: Stoichiometric combustion of methane in air: CH4 + 2 (O2 + 3.76 N2) -→ CO2 + 2 H2O + 7.52 N2 (ideally) At 1800 K some CO and H2 appear from partial dissociation.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.00292222222222
stream	flueGas	F	0.00292404303636
kpi	burner	F_in_kmol_h	10.52
kpi	burner	F_out_kmol_h	10.5265549309
kpi	burner	N_in_mol_s	2.92222222222
kpi	burner	N_out_mol_s	2.92404303636
kpi	burner	P	100000
kpi	burner	T	1800
kpi	burner	converged	1
kpi	burner	lambda_C	-357484.903199
kpi	burner	lambda_H	-201140.892694
kpi	burner	lambda_N	-201371.845813
kpi	burner	lambda_O	-268835.220885
kpi	burner	y_CH4	7.51542148893e-20

. . . and 9 more reference values in the case's `expected`*.*

`gibbs04_wgs_temperature_sweep`

`tutorials/steady/gibbs/gibbs04_wgs_temperature_sweep`

What it does: Water-gas shift equilibrium swept over T = 600..1200 K

The story: / flowsheetDict Water-gas shift equilibrium swept over T = 600..1200 K (outerDict sweep on the Gibbs reactor): CO conversion falls as T rises – exothermic equilibrium – reproducing the textbook K(T) curve point by point.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`products`	`F`	`0.000277777777859`
`kpi`	`wgs`	`F_in_kmol_h`	`1`
`kpi`	`wgs`	`F_out_kmol_h`	`1.00000000029`
`kpi`	`wgs`	`N_in_mol_s`	`0.277777777778`
`kpi`	`wgs`	`N_out_mol_s`	`0.277777777859`
`kpi`	`wgs`	`P`	`100000`
`kpi`	`wgs`	`Q_kW`	`-2.0000513466`
`kpi`	`wgs`	`T`	`900`
`kpi`	`wgs`	`converged`	`1`
`kpi`	`wgs`	`lambda_C`	`-12610.3166837`
`kpi`	`wgs`	`lambda_H`	`-69027.9700281`
`kpi`	`wgs`	`lambda_O`	`-299577.53726`
`kpi`	`wgs`	`y_CO`	`0.199139146827`

. . . and 5 more reference values in the case's `expected`.

gibbs05_adiabatic_flame

tutorials/steady/gibbs/gibbs05_adiabatic_flame

What it does: Adiabatic flame of lean CH4 in air ($\varphi = 0.5$)

The story: Lean methane combustion at $\varphi = 0.5$ (100 % excess air): 1 mol CH4 + 4 mol O2 + 15.04 mol N2 Feed at ambient 298.15 K, 1 bar. Reactor finds its own adiabatic flame temperature.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.00556666666667
stream	flueGas	F	0.00556667001791
kpi	burner	F_in_kmol_h	20.04
kpi	burner	F_out_kmol_h	20.0400120645
kpi	burner	N_in_mol_s	5.56666666667
kpi	burner	N_out_mol_s	5.56667001791
kpi	burner	P	100000
kpi	burner	T	1491.58454904
kpi	burner	converged	1
kpi	burner	lambda_C	-432004.832024
kpi	burner	lambda_H	-204860.137156
kpi	burner	lambda_N	-162961.775795
kpi	burner	lambda_O	-186456.945423
kpi	burner	outerIterations	3

... and 9 more reference values in the case's `expected`*.*

gibbs06_h2_flame_radicals

`tutorials/steady/gibbs/gibbs06_h2_flame_radicals`

What it does: Stoichiometric H2/air adiabatic flame – the radical pool at equilibrium

The story: THE RADICAL DATABASE AT WORK: stoichiometric H2/air adiabatic flame, equilibrium by Gibbs minimisation over the FULL radical pool (H, O, OH, HO2, H2O2 – GRI-Mech 3.0 / Burcat NASA-7 thermo) plus equilibrium thermal NO.

1 H2 + 0.5 O2 + 1.88 N2 at 298.15 K, 1 atm, adiabatic.

THE ANSWER (golden-locked): T__ad = 2385.8 K – the literature gives 2380-2390 K (Glassman, Combustion, 4th ed.; Law, Combustion Physics). Equilibrium pool: OH 0.55 %, H 0.18 %, O 0.06 %, NO 0.27 % – all inside the published equilibrium ranges. NO here is the Zeldovich CEILING (what kinetics approaches with residence time, never exceeds).

TWO LESSONS THIS CASE TEACHES: 1. THE RADICALS' WORK: endothermic dissociation (H2O $\rightarrow$ OH + H, O2 $\rightarrow$ 2 O) soaks up enthalpy near 2400 K. Without the radical species the same reactor answers ~2640 K – the first-year textbook overshoot. 2. THE EXTRAPOLATION TRAP (why constant/components/ overrides exist here): the catalogue's poly3 Cp fits are 250-1500 K PROCESS fits; extrapolated to 2641 K, N2's cubic turns NEGATIVE (-33 J/mol/K) and water's reads 45 instead of ~56. The case-local overrides switch the four stable species to the NASA-7 two-range form (GRI-Mech 3.0, valid 200-3500 K) – the same source as the radical entries. Run it once with the overrides removed to SEE the 250 K error a silent extrapolation buys.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`burnt`	`F`	`0.000808562123724`
`stream`	`feed`	`F`	`0.000938888888889`
`kpi`	`flame`	`F_in_kmol_h`	`3.38`
`kpi`	`flame`	`F_out_kmol_h`	`2.91082364541`
`kpi`	`flame`	`N_in_mol_s`	`0.938888888889`
`kpi`	`flame`	`N_out_mol_s`	`0.808562123724`
`kpi`	`flame`	`P`	`101325`
`kpi`	`flame`	`T`	`2385.83679448`
`kpi`	`flame`	`converged`	`1`
`kpi`	`flame`	`lambda_H`	`-240335.177161`
`kpi`	`flame`	`lambda_N`	`-277509.500244`
`kpi`	`flame`	`lambda_O`	`-344071.838893`
`kpi`	`flame`	`outerIterations`	`3`
`kpi`	`flame`	`y_H`	`0.00181958387904`

. . . and 11 more reference values in the case's `expected`*.*

gibbs06_thermal_nox

tutorials/steady/gibbs/gibbs06_thermal_nox

What it does: Methane combustion at 1800 K with thermal NOx formation

The story: / flowsheetDict Methane combustion at 1800 K with thermal NOx: the Gibbs reactor finds the full equilibrium including NO/NO2 – the EQUILIBRIUM ceiling on NOx, to contrast with the KINETIC path (nox01_zeldovich_postflame, where the rate, not equilibrium, controls).

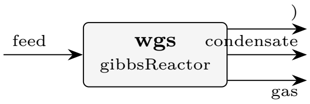
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.00292222222222`
`stream`	`flueGas`	`F`	`0.00292416183331`
`kpi`	`burner`	`F_in_kmol_h`	`10.52`
`kpi`	`burner`	`F_out_kmol_h`	`10.5269825999`
`kpi`	`burner`	`N_in_mol_s`	`2.92222222222`
`kpi`	`burner`	`N_out_mol_s`	`2.92416183331`
`kpi`	`burner`	`P`	`100000`
`kpi`	`burner`	`T`	`1800`
`kpi`	`burner`	`converged`	`1`
`kpi`	`burner`	`lambda_C`	`-355589.713611`
`kpi`	`burner`	`lambda_H`	`-200666.346513`
`kpi`	`burner`	`lambda_N`	`-201373.229482`
`kpi`	`burner`	`lambda_O`	`-269787.239957`
`kpi`	`burner`	`y_CH4`	`9.68349314225e-20`

. . . and 10 more reference values in the case's `expected`.

`gibbs07_wgs_cooled`

`tutorials/steady/gibbs/gibbs07_wgs_cooled`



What it does: Cooled WGS Gibbs reactor (350 K): water condenses, shifting the equilibrium (multi-phase)

The story: multi-phase (gas + liquid) Gibbs reactor.

CO + H2O <→ CO2 + H2, excess steam (1 CO + 3 H2O), 350 K, 1 bar.

The ‘elementPotential‘ method (default) does a gas-only Gibbs solve, sees that the leftover water is supersaturated (y_H2O.P > Psat(350 K) ~ 0.42 bar), and switches to the two-phase V+L solve. The condensing water is pulled out of the gas, which drives the WGS further forward (Le Chatelier) than a gas-only reactor would — the coupling a single-phase Gibbs misses.

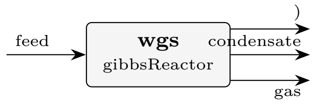
Outlets: ‘gas‘ (the dry-ish product gas) + ‘condensate‘ (liquid water).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`condensate`	`F`	`3.76244802832e-05`
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`gas`	`F`	`0.000240153299998`
`kpi`	`wgs`	`F_condensate_kmol_h`	`0.135448129019`
`kpi`	`wgs`	`F_in_kmol_h`	`1`
`kpi`	`wgs`	`F_out_kmol_h`	`0.864551879994`
`kpi`	`wgs`	`N_in_mol_s`	`0.277777777778`
`kpi`	`wgs`	`N_liq_mol_s`	`0.0376244802832`
`kpi`	`wgs`	`N_out_mol_s`	`0.277777780282`
`kpi`	`wgs`	`P`	`100000`
`kpi`	`wgs`	`T`	`350`
`kpi`	`wgs`	`converged`	`1`
`kpi`	`wgs`	`lambda_C`	`50085.9480936`
`kpi`	`wgs`	`lambda_H`	`-24736.1727481`
. . . and 12 more reference values in the case’s `expected`.			

gibbs08_wgs_cooled_reactiveflash

tutorials/steady/gibbs/gibbs08_wgs_cooled_reactiveflash



What it does: Cooled WGS via reactiveFlash (B1) – cross-checks the elementPotential V+L result

The story: the gibbs07_wgs_cooled problem solved by ‘model reactiveFlash;‘ (route B1: an equilibrium-reaction + isothermal-flash inner loop) instead of the default elementPotential (A). Cross-checks the two-phase V+L answer by an independent method.

The physics is gibbs07’s: multi-phase (gas + liquid) Gibbs reactor.

CO + H2O <-> CO2 + H2, excess steam (1 CO + 3 H2O), 350 K, 1 bar.

The ‘elementPotential‘ method (default) does a gas-only Gibbs solve, sees that the leftover water is supersaturated (y_H2O.P > Psat(350 K) ~ 0.42 bar), and switches to the two-phase V+L solve. The condensing water is pulled out of the gas, which drives the WGS further forward (Le Chatelier) than a gas-only reactor would — the coupling a single-phase Gibbs misses.

Outlets: ‘gas‘ (the dry-ish product gas) + ‘condensate‘ (liquid water).

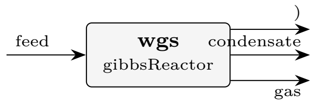
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	condensate	F	3.76244802827e-05
stream	feed	F	0.000277777777778
stream	gas	F	0.000240153299999
kpi	wgs	F_condensate_kmol_h	0.135448129018
kpi	wgs	F_in_kmol_h	1
kpi	wgs	F_out_kmol_h	0.864551879995
kpi	wgs	N_in_mol_s	0.277777777778
kpi	wgs	N_liq_mol_s	0.0376244802827
kpi	wgs	N_out_mol_s	0.277777780281
kpi	wgs	P	100000
kpi	wgs	T	350
kpi	wgs	converged	1
kpi	wgs	lambda_C	50085.9480935
kpi	wgs	lambda_H	-24736.1727482

... and 12 more reference values in the case’s `expected`.

`gibbs09_wgs_cooled_directmin`

`tutorials/steady/gibbs/gibbs09_wgs_cooled_directmin`



What it does: Cooled WGS via directMin (B2, Nelder-Mead direct G-min) – cross-checks A/B1 to ~0.3%

The story: the gibbs07_wgs_cooled problem solved by ‘model directMin;‘ (route B2: direct Nelder-Mead minimisation of the total Gibbs energy G) – the third independent route, cross-checking A (elementPotential, gibbs07) and B1 (reactiveFlash, gibbs08) to ~0.3%.

The physics is gibbs07’s: multi-phase (gas + liquid) Gibbs reactor.

CO + H2O <→ CO2 + H2, excess steam (1 CO + 3 H2O), 350 K, 1 bar.

The ‘elementPotential‘ method (default) does a gas-only Gibbs solve, sees that the leftover water is supersaturated (y_H2O.P > Psat(350 K) ~ 0.42 bar), and switches to the two-phase V+L solve. The condensing water is pulled out of the gas, which drives the WGS further forward (Le Chatelier) than a gas-only reactor would — the coupling a single-phase Gibbs misses.

Outlets: ‘gas‘ (the dry-ish product gas) + ‘condensate‘ (liquid water).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`condensate`	`F`	`3.77165683291e-05`
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`gas`	`F`	`0.000240061209449`
`kpi`	`wgs`	`F_condensate_kmol_h`	`0.135779645985`
`kpi`	`wgs`	`F_in_kmol_h`	`1`
`kpi`	`wgs`	`F_out_kmol_h`	`0.864220354015`
`kpi`	`wgs`	`N_in_mol_s`	`0.277777777778`
`kpi`	`wgs`	`N_liq_mol_s`	`0.0377165683291`
`kpi`	`wgs`	`N_out_mol_s`	`0.277777777778`
`kpi`	`wgs`	`P`	`100000`
`kpi`	`wgs`	`T`	`350`
`kpi`	`wgs`	`converged`	`1`
`kpi`	`wgs`	`liquidFraction`	`0.135779645985`
`kpi`	`wgs`	`twoPhase`	`1`

. . . and 12 more reference values in the case’s `expected`.

gibbs10_ammonia_fugacity

`tutorials/steady/gibbs/gibbs10_ammonia_fugacity`

What it does: Ammonia synthesis equilibrium at 200 bar with the REAL-gas fugacity correction: N2 + 3 H2 <→ 2 NH3 at 700 K, 200 bar over an SRK equation of state. At this pressure the ideal-gas equilibrium (30.0 % NH3) is wrong – the Gibbs reactor now folds the SRK fugacity coefficients into the equilibrium (Kphi ~ 0.75, favouring the fewer-moles NH3 side), giving 32.4 % NH3, toward the measured ~35-40 %. Switch the equationOfState to idealGas and watch NH3 drop back to 30.0 % – the fugacity correction is the whole point of a high-pressure reactor.

The story: the high-pressure equilibrium done RIGHT

N2 + 3 H2 <→ 2 NH3 at 700 K and 200 bar, a stoichiometric feed. The Gibbs reactor minimises the REAL-gas Gibbs energy: it pulls the fugacity coefficients from the package EoS (SRK here) and adds ln(phi_i) to each species' chemical potential. At 200 bar this MATTERS – the fugacity ratio Kphi = phi_NH3^2/(phi_N2 phi_H2^3) ~ 0.75 shifts the equilibrium toward NH3 (the fewer-moles side): 30.0 % (ideal) → 32.4 % (SRK), toward the measured ~35-40 %. Set the package EoS to idealGas and NH3 falls back to 30.0 %. (A cornerstone of the classic ammonia synthesis loop: the reactor is only quantitative at pressure once fugacity is in.)

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	products	F	0.000209823048593
kpi	ammonia	F_in_kmol_h	1
kpi	ammonia	F_out_kmol_h	0.755362974933
kpi	ammonia	N_in_mol_s	0.277777777778
kpi	ammonia	N_out_mol_s	0.209823048593
kpi	ammonia	P	20000000
kpi	ammonia	Q_kW	-3.68175302075
kpi	ammonia	T	700
kpi	ammonia	converged	1
kpi	ammonia	lambda_H	-34945.3954982
kpi	ammonia	lambda_N	-59416.3950887
kpi	ammonia	y_H2	0.507099864715
kpi	ammonia	y_N2	0.169033288238

. . . and 3 more reference values in the case's `expected`.

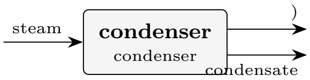
heat

The theory you need. Heat exchange couples streams without mixing them: $Q = U\,A\,\Delta T_{\mathrm{lm}}$, effectiveness-NTU when outlets are unknown. Utilities (steam levels, cooling water) are allocated by temperature level – a reboiler cannot be heated by a colder utility, and the allocator says which level it picked. Heat-links (forward and feedback) couple distant units into an energy tear the solver converges like any recycle. Pinch analysis finds the minimum utilities the composite curves allow.

Exchanger DESIGN vs rating. A shell-and-tube poses two inverse questions: RATE it (geometry given → find U, area, ΔP, duty – `model geometry;`) or DESIGN it (duty given → size the geometry – `model design;`). U is BUILT from the two film coefficients (Gnielinski tube, Kern shell) and the wall; the pressure drop (Kern both sides) is the duty-vs-pumping half a real design lives by. The standard workflow – establish the duty, THEN design, THEN rate, never invent the geometry up front – is walked end-to-end in the two-part sequence `hxWorkflow1_design_from_duty` → `hxWorkflow2_rate_designed`, and the design ends in a printable datasheet (GUI: the unit panel). *Full derivation: Theory Guide, the heat-integration chapter, §heat-exchanger design.*

`condenser01_film_nusselt`

`tutorials/steady/heat/condenser01_film_nusselt`



What it does: Horizontal-tube steam condenser: Nusselt film HTC → duty (geometry mode)

The story: A horizontal-tube steam condenser whose duty is NOT specified – it EMERGES from the physics of the condensing film, exactly as a real condenser's does.

Saturated steam at 1 bar enters a bundle of horizontal tubes cooled by cooling water. The 'phaseChanger' runs in 'model geometry;' (the seventh resolution): a Nusselt (1916) LAMINAR FILM coefficient h_cond is computed on the tube, the wall conducts, and the cooling-water film carries the heat away. Because the condensing-film coefficient itself depends on the driving dT_film = Tsat - T_wall, the unit solves a 1-D energy balance at the wall (q_cond == q_cool) – the wall temperature is the unknown. Then the duty is Q = h_cond ∗ A ∗ dT_film and the outlet is the dome-flash at that duty.

steam (saturated vapour, 1 bar, vf = 1) → condenser (model geometry: NusseltFilm + cooling-water film) → condensate (two-phase, vf ~ 0.55) + Q (RESULT)

PARTIAL CONDENSATION: with this tube area the duty Q ~ -507 kW removes only part of the latent heat, so the outlet is NOT fully condensed — it leaves at the dome as a wet two-phase stream, vf ~ 0.55 (about 45 % of the vapour condensed) at Tsat. Fully condensing it would need more area; here the area is fixed and the outlet quality is the result.

Method (glass-box): h_cond : Nusselt film, horizontal tube, coefficient 0.729 h = 0.729 [k_l^3 rho_l (rho_l-rho_v) g h_fg' /(mu_l D dT_film)]^1/4 h_fg' = h_fg + 0.68 cp_l dT_film (Rohsenow subcooling corr.) (Incropera & DeWitt, Fund. of Heat & Mass Transfer, Ex. 10.4) wall : carbon-steel cylindrical resistance r_o ln(r_o/r_i)/k coolant: a cooling-water film coefficient h (Dittus-Boelter order) the three resistances + which CONTROLS are printed; the duty is cross-checked against U∗A∗dT.

All film/saturation properties are IAPWS-IF97 water. The case carries its own constant/components/water.dat (case-local copy). ASSUMPTION stated plainly: the coolant is treated as ISOTHERMAL at its inlet T (its ~10 K heat-up is neglected, so dT = Tsat - T_in slightly OVERSTATES the true LMTD – the computed area errs a few % small). The utility allocation still reports the real cooling-water flow and duty.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`condensate`	`F`	`0.0277777777778`
`stream`	`steam`	`F`	`0.0277777777778`
`kpi`	`condenser`	`F`	`0.0277777777778`
`kpi`	`condenser`	`P`	`100000`
`kpi`	`condenser`	`Q`	`-506700.51677`
`kpi`	`condenser`	`Q_UAdT_kW`	`506.617311982`
`kpi`	`condenser`	`Q_kW`	`-506.70051677`
`kpi`	`condenser`	`R_coolant`	`0.00014880952381`
`kpi`	`condenser`	`R_film`	`0.000349003587291`
`kpi`	`condenser`	`R_wall`	`4.35883467862e-05`
`kpi`	`condenser`	`Re_film`	`15.1166609728`
`kpi`	`condenser`	`T_in`	`372.76`
`kpi`	`condenser`	`T_out`	`372.755918611`
`kpi`	`condenser`	`T_wall`	`325.855275434`

. . . and 14 more reference values in the case's `expected`*.*

coolingTower01_merkel

`tutorials/steady/heat/coolingTower01_merkel`

What it does: Water cooling tower, Merkel enthalpy driving force – rating: the packing is given, the cold-water T is the result

The story: A counter-current evaporative cooling tower in RATING mode: the packing characteristic is declared (Me = KaV/L = 1.5) and the cold-water temperature is the RESULT. Hot water at 45 C meets ambient air at 25 C (Y = 0.01286 kg/kg dry, about 62 % RH on the N2 carrier), L/G ~ 0.99.

The whole construction lives on the Merkel diagram the unit publishes as its profile: air enthalpy h against water temperature T, the saturation curve h∗(T) above, the operating line h1 + (L cpL / G)(T - T_out) below, and Me the integral of cpL dT over the gap between them.

Method hypotheses (each announced at run time): Lewis = 1; the water loss is neglected INSIDE the integral but reported and honoured at the boundary (the coolWater outlet carries L_in minus the evaporation); the exit air is assumed saturated; cp at mean film temperatures.

What to look at in the output: T_water_out against the wet-bulb floor (T_wb ~ 293 K): the approach is the tower's real currency – a tower can never cool BELOW T_wb. merkelNumber_chebyshev4 beside the converged integral: the CTI four-point hand method against Simpson on 200 intervals. The gap is the price of the hand method, visible, not hidden. evaporation_kg_s = makeup_kg_s: the water the cooling COSTS. A tower cools ~1 % of L per ~7 K of range by evaporating it.

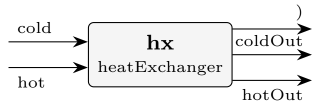
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`air`	`F`	`0.0184166666667`
`stream`	`coolWater`	`F`	`0.027110090426`
`stream`	`hotWater`	`F`	`0.0277777777778`
`stream`	`humidAir`	`F`	`0.0190843540184`
`kpi`	`tower01`	`L_over_G`	`0.98937581178`
`kpi`	`tower01`	`Q_kW`	`36.4315830585`
`kpi`	`tower01`	`T_air_out`	`307.613817692`
`kpi`	`tower01`	`T_water_out`	`300.778649138`
`kpi`	`tower01`	`T_wb_in`	`293.021697982`
`kpi`	`tower01`	`Y_in`	`0.0128618855531`
`kpi`	`tower01`	`Y_out`	`0.0366432593178`
`kpi`	`tower01`	`approach_K`	`7.75695115581`
`kpi`	`tower01`	`evaporation_kg_s`	`0.0120283876424`
`kpi`	`tower01`	`evaporation_pct_of_L`	`2.40367446642`

. . . and 8 more reference values in the case's `expected`.

heatExchanger01_water_water

tutorials/steady/heat/heatExchanger01_water_water



What it does: Two-stream heat exchanger (eps-NTU): hot/cold water recovery

The story: Two-stream heat exchanger by the effectiveness-NTU method (the built-in ‘heatExchanger‘). A hot water stream is cooled by a cold water stream in a counter-current unit — the canonical heat-recovery problem.

Hardware (the credo): area + U. The outlet temperatures and the duty are RESULTS. "I need T_hot_out = X" would be a DesignSpec on $area.

Hot : 100 kmol/h water at 360 K Cold : 120 kmol/h water at 300 K Hardware: A = 10 m^2, U = 500 W/(m^2.K), counter-current

Expected (eps-NTU, Cp_water ~ 75 J/mol/K): C_hot = 100/3.6 * 75 ~ 2083 W/K (= C_min) C_cold = 120/3.6 * 75 ~ 2500 W/K NTU = U.A/C_min ~ 2.4, C_r ~ 0.83, effectiveness ~ 0.75 Q ~ 93 kW T_hot_out ~ 315 K (cooled ~45 K) T_cold_out ~ 337 K (heated ~37 K) LMTD ~ 22 K, and U.A.LMTD ~ Q (cross-check) energy balance: hot stream loses what the cold stream gains.

Hot water (100 kmol/h, 360 K) against cold water (120 kmol/h, 300 K) in a counter-current exchanger of 10 m^2 at $U = 500$ W/m^2K. Nothing about the outlet temperatures is declared; the ε-NTU method finds them. The golden: Q = 93.79 kW, hot out 315.28 K, cold out 337.27 K, NTU 2.384, C_r 0.833, $\varepsilon = 0.745$, LMTD 18.76 K.

*1. Rating, not design. The hardware (area, U, flow arrangement) is given in operation {}; the question is what it *does* with these two streams. hxWorkflow1_design_from_duty asks the reverse. 2. ε-NTU in four printed numbers. The Log tab prints NTU, C_r, ε and Q on one line, and then checks itself: U·A·LMTD = 93.79 kW beside the ε-NTU result. Two routes, one answer — that agreement is the method being right, and you can redo either on paper. 3. The smaller heat-capacity rate sets the limit. C_hot = 2097 W/K is the minimum (fewer moles), so ε is measured on the hot stream: (360 - 315.28)/(360 - 300) = 0.745. Read C_hot and C_cold in the KPIs and confirm C_r. 4. No utility is consumed. Both sides are process streams; the 93.79 kW is *recovered*, which is the whole reason to install an exchanger rather than a heater and a cooler.*

Switch flow counter; to flow co; and run again: same area, same U, smaller ε, less heat — the classic comparison, in one word of the dict.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	cold	F	0.0333333333333
stream	coldOut	F	0.0333333333333
stream	hot	F	0.0277777777778
stream	hotOut	F	0.0277777777778
kpi	hx	C_cold	2516.66666667
kpi	hx	C_hot	2097.22222222
kpi	hx	C_r	0.833333333333
kpi	hx	LMTD	18.7584806629
kpi	hx	NTU	2.38410596026
kpi	hx	Q	93792.4033146
kpi	hx	Q_kW	93.7924033146
kpi	hx	T_cold_in	300
kpi	hx	T_cold_out	337.268504628
kpi	hx	T_hot_in	360

. . . and 10 more reference values in the case’s `expected`.

`heatExchanger02_geometry_U`

`tutorials/steady/heat/heatExchanger02_geometry_U`

What it does: Shell-and-tube heat exchanger sized from GEOMETRY: the overall U and area are COMPUTED from the tube bundle + per-side convective correlations (Gnielinski tube-side, Kern shell-side), not specified. Water-water clean service. The computed U lands in the textbook clean water-water band (~2000-3000 W/(m2.K), Kern / Sinnott). Tube-side h_i via Gnielinski (Incropera & DeWitt 6th ed.); shell-side h_o via the Kern method (Process Heat Transfer, 1950). U and area are RESULTS that then feed the unchanged eps-NTU duty.

The story: A shell-and-tube heat exchanger whose overall coefficient U and area are COMPUTED from the bundle geometry plus per-side convective correlations (the ‘model geometry;‘ path), instead of being specified. This is the glass-box answer to "where does U come from?".

Hot water (shell side) is cooled by cold water (tube side), clean service.

Method: tube-side h_i : Gnielinski Nu = (f/8)(Re-1000)Pr/[1+12.7 sqrt(f/8)(Pr^2/3-1)] (Incropera & DeWitt, Fund. of Heat and Mass Transfer, 6th ed.) shell-side h_o: Kern method Nu = 0.36 Re_s^0.55 Pr^1/3 (Kern, Process Heat Transfer, 1950) on the bundle equivalent diameter D_e and crossflow mass flux G_s. U on the OUTSIDE area: 1/U = 1/h_o + r_o ln(r_o/r_i)/k_wall + r_o/(r_i h_i) area = pi d_o L n_tubes PRESSURE DROP (Kern, both sides): tube f = 0.079 Re^-0.25 (Blasius), dP = (4 f L n_p/d_i + 4 n_p)(rho u^2/2) – friction + turnaround losses; shell f = exp(0.576 - 0.19 ln Re_s), dP = f G_s^2 D_s (N_b+1) /(2 rho D_e) over the N_b baffles. (Kern, Process Heat Transfer, 1950.)

Properties are IF97 water (rho, mu, lambda) at each stream temperature.

Expected order of magnitude (clean water-water): U ~ 2000-3000 W/(m2.K), the SHELL side controlling, both sides turbulent (Re ~ 2-4 e4); tube-side dP ~ 9 kPa, shell-side dP ~ 35 kPa (shell > tube: baffled crossflow is more dissipative – the duty-vs-pumping trade-off you design around, NOT just the area). The exact numbers are pinned by –record – they are RESULTS.

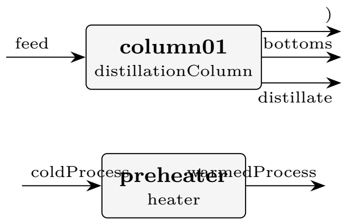
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`coldOut`	`F`	`2.22027777778`
`stream`	`coldTube`	`F`	`2.22027777778`
`stream`	`hotOut`	`F`	`1.66527777778`
`stream`	`hotShell`	`F`	`1.66527777778`
`kpi`	`hx`	`C_cold`	`167630.972222`
`kpi`	`hx`	`C_hot`	`125728.472222`
`kpi`	`hx`	`C_r`	`0.750031277368`
`kpi`	`hx`	`LMTD`	`15.5339177262`
`kpi`	`hx`	`NTU`	`1.26058061182`
`kpi`	`hx`	`Nu_shell`	`171.621364606`
`kpi`	`hx`	`Nu_tube`	`287.732215939`
`kpi`	`hx`	`Pr_shell`	`3.16405821902`
`kpi`	`hx`	`Pr_tube`	`5.87094963164`
`kpi`	`hx`	`Q`	`2065604.42087`

. . . and 27 more reference values in the case’s `expected`.

heatlink01_condenser_to_heater

tutorials/steady/heat/heatlink01_condenser_to_heater



What it does: Forward heat-link: column condenser preheats a downstream stream

The story: feed column01 distillate bottoms (condenser heat) preheater warmedProcess coldProcess

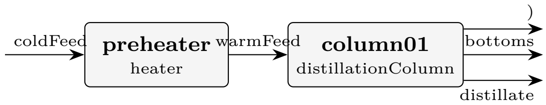
The preheater is heated by the column's rejected condenser duty (~1.28 MW), warming a 1500 kmol/h benzene stream from 320 K by ~23 K. column01 is listed FIRST so it solves before the preheater reads its condenser KPI (forward heat-link).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0138888888889
stream	coldProcess	F	0.416666666667
stream	distillate	F	0.0138888888889
stream	feed	F	0.0277777777778
stream	warmedProcess	F	0.416666666667
kpi	column01	B	0.0138888888889
kpi	column01	D	0.0138888888889
kpi	column01	L_rect	0.0277777777778
kpi	column01	L_strip	0.0555555555556
kpi	column01	Q_condenser_kW	-1281.04348002
kpi	column01	Q_reboiler_kW	1279.30480005
kpi	column01	R	2
kpi	column01	T_bottom	382.893530951
kpi	column01	T_top	354.21089412
... and 26 more reference values in the case's `expected`.			

heatlink02_condenser_feed_preheat

tutorials/steady/heat/heatlink02_condenser_feed_preheat



What it does: Feedback heat-link (energy tear): column condenser preheats its OWN feed

The story: FEEDBACK heat-link == an ENERGY TEAR.

coldFeed preheater column01 distillate bottoms (condenser heat)

The preheater (UPSTREAM of the column) is heated by the column's CONDENSER duty (DOWNSTREAM) → circular. The Flowsheet detects the feedback, turns the recycle outer loop on, and converges the duty by successive substitution (Wegstein). Sized so the ~1 MW condenser preheats the large 1000 kmol/h feed by a modest ~25 K (stays liquid).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.263888888889
stream	coldFeed	F	0.277777777778
stream	distillate	F	0.0138888888889
stream	warmFeed	F	0.277777777778
kpi	column01	B	0.263888888889
kpi	column01	D	0.0138888888889
kpi	column01	L_rect	0.0208333333333
kpi	column01	L_strip	0.298611111111
kpi	column01	Q_condenser_kW	-1067.88365467
kpi	column01	Q_reboiler_kW	1846.42629755
kpi	column01	R	1.5
kpi	column01	T_bottom	365.99791143
kpi	column01	T_top	354.558892551
kpi	column01	V_rect	0.0347222222222

. . . and 24 more reference values in the case's `expected`.

hx01_heater

tutorials/steady/heat/hx01_heater

What it does: Heater: 50/50 ethanol/water from 300 K to ~338 K (sensible)

The story: Stand-alone heater — credo-pure (v0.34+).

Hardware spec: Q = 100 kW (the absolute thermal power delivered by the heater – think of it as the electrical rating).

Streams IN → OP (Q only) → Streams OUT

Feed is 100 kmol/h of 50/50 ethanol/water at 300 K. The simulator solves the enthalpy balance H_out = H_in + Q/F to compute T_out as a RESULT. With Q = 100 kW the outlet temperature lands near 338 K (Cp_mix ~ 94 J/(mol*K) at this composition); the exact value falls out of the Newton-1D in T.

NOTE (duty reduced 160 → 100 kW). At 160 kW the outlet reached ~360 K, ABOVE the 50/50 ethanol/water bubble point at 1 atm – the old single-phase heater silently reported a physically-impossible "superheated liquid". The new saturation guard refuses that (a heater is sensible-only); a duty that boils belongs to a ‘phaseChanger‘ / ‘boiler‘ (see steady/heat/phasechange01_partial_condenser and steady/power/rankine02_water). 100 kW keeps this the clean sensible-heat lesson it was meant to be.

If you instead need a specific T_out (say "I want exactly 338 K"), wrap the heater in a DesignSpec outerDict that manipulates $Q with a target on ‘heater01.T_out‘. The unit op itself never specifies its outlet state – that is the credo we adopted in v0.33.

50/50 ethanol/water (100 kmol/h) at 300 K enters a heater that delivers 100 kW. The dict says how much heat; the energy balance says how hot: T_out = 338.34 K, a 38.3 K rise, and the stream stays liquid (vf_out = 0).

*1. Hardware is declared, results are solved. operation { Q 100.0 kW; } is the only number the unit takes. T_out is *found* — the Log tab shows a Newton on the outlet temperature closing the enthalpy balance H_out = H_in + Q. 2. 3600 J/mol is the whole story in one number. Q per mole of feed (Q_per_mol) divided by the mixture's liquid Cp gives the rise; read the Cp's the run used in the Props tab and check 38 K by hand. 3. Somebody has to supply those 100 kW. The run's result records which catalogue utility was picked to serve the duty at the outlet temperature: low-pressure steam, 0.0457 kg/s, at the catalogue's price 4.32 ?/h. That choice is the same rule every steady report uses (pickForDuty); the price is a catalogue number, not a market one, and data/standards/utilities/ is where it lives. 4. Where the stream came from matters. The feed is a subcooled liquid; heat it far enough and it starts to boil — the heater will say so with a vapour fraction, not by refusing.*

Raise Q until vf_out leaves zero: the outlet temperature stops climbing as fast, because the heat is now going into vaporisation. phasechange01_partial_condenser is the same balance run in reverse.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	0.0277777777778
`stream`	`hotStream`	`F`	0.0277777777778
`kpi`	`heater01`	`F`	0.0277777777778
`kpi`	`heater01`	`P`	101325
`kpi`	`heater01`	`Q`	100000
`kpi`	`heater01`	`Q_kW`	100
`kpi`	`heater01`	`Q_per_mol`	3600
`kpi`	`heater01`	`T_in`	300
`kpi`	`heater01`	`T_out`	338.338658147
`kpi`	`heater01`	`dT`	38.338658147
`utility`	`heater01`	`heating.steamLP.duty_kW`	100
`utility`	`heater01`	`heating.steamLP.T`	338.338658146
`utility`	`heater01`	`heating.steamLP.kg_s`	0.0457456541629
`utility`	`heater01`	`heating.steamLP.eur_h`	4.32

... and 3 more reference values in the case's ***`expected`****.*

hxWorkflow1_design_from_duty

tutorials/steady/heat/hxWorkflow1_design_from_duty

What it does: PART 1 of 2 – DESIGN: the DUTY TARGET is given (cool the hot shell 330→315 K) and the exchanger GEOMETRY is SOLVED for (model design;). Glass-box Kern sizing: fix the tube choices, then find the tube count (and the shell diameter it implies, Kern/Sinnott bundle) so U∗A = Q_req/LMTD; U/dP then rated with Gnielinski (tube) + Kern (shell). The detailed-design (sizing) step, glass-box. Part 2 (hxWorkflow2_rate_designed) rates the bundle this case produces.

The story: PART 1 of 2: DESIGN from the duty

THE HEAT-EXCHANGER DESIGN WORKFLOW (the standard design sequence), part 1. DESIGN mode (‘model design;‘) – detailed sizing, glass-box. Instead of GIVING the tube count and shell diameter (that is ‘model geometry;‘, the RATING problem: geometry → U), here we give the DUTY TARGET (cool the hot shell stream from 330 K to 315 K) and the engine SOLVES for the geometry: fix the tube choices (OD/ID/length/pitch/passes/material), then find the number of tubes – and the shell diameter it implies (Kern/Sinnott bundle correlation, the chosen pattern + passes) so the exchanger DELIVERS THE DUTY – the design loop sizes N by the eps-NTU duty (incl. the 1-2 penalty for a 2-pass U-tube), not merely U∗A = Q/LMTD, so the outlet actually reaches the target. It then rates that designed bundle exactly like a given one (same Gnielinski tube + Kern shell correlations) and reports U, area, and the pressure drop.

DO I HAVE TO SPECIFY EVERYTHING? No – and this is the whole point. A HX poses two OPPOSITE questions, and this is the DESIGN one:

RATING (‘model geometry;‘): you GIVE the full hardware (tube count, shell diameter, ...) and ask "what outlet T do I get?". The temperature is a RESULT. DESIGN (‘model design;‘, THIS case): you GIVE the outlet T you NEED and ask "what hardware delivers it?". The GEOMETRY is the result.

So HOW does the outlet reach the temperature you need? You do NOT tune the geometry by hand until it works. You DECLARE the target and the engine sizes the hardware to hit it. Look at the two blocks below:

‘design { spec { stream hotShell; T 315 K; } }‘ <– WHAT YOU WANT: the required outlet temperature. THIS is where the final temperature is set. ‘geometry { tubeID ...; passes 2; tubePattern ...; wallMaterial ...; }‘ <– the FIXED CHOICES a designer commits to (a standard tube, the number of passes, the layout, the metal). Notice what is MISSING: there is NO ‘nTubes‘ and NO ‘shellID‘. Those are exactly what the engine SOLVES.

So the student specifies the REQUIREMENT (the outlet T) + the design CHOICES (tube size, passes, pattern, material) – NOT the answer. The engine returns the tube count, the shell diameter, U, the pressure drop, and confirms the outlet lands on the target. You said "cool this to 315 K with 1/2-inch steel tubes in a 2-pass U-tube"; it answers "then you need 105 tubes in a 0.31 m shell – here is U, the dP, and yes it leaves at 315 K".

This is the workflow that matches industrial practice: establish the DUTY, THEN design the exchanger from it – you do NOT invent the geometry up front. (Standard practice splits this into a shortcut that gets the duty + a detailed-design step that gets the geometry; Choupo's design mode folds both into one honest step – it computes the duty from your target and ANNOUNCES it, then sizes.) PART 2 (hxWorkflow2_rate_designed) then RATES the bundle this case produced – a detailed-rating step – to verify it and probe off-design (turn the coolant down and watch the outlet drift off the target; the geometry is now FIXED, only the performance moves).

Water-water, clean service, 2-pass U-tube (the general arrangement). Expect: ~100 tubes, a ~0.3 m shell; rated U ~ 4000 W/(m2.K) (2 tube passes → high tube velocity), and the delivered hot outlet 315 K (the target, reproduced by the 1-2 eps-NTU on the designed geometry). Numbers are RESULTS pinned by –record. Single-phase; the 1-shell/2-tube-pass eps-NTU carries the built-in LMTD F-correction (F<1), so a 2-pass needs more area than pure counter-current.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`coldOut`	`F`	`2.22027777778`
`stream`	`coldTube`	`F`	`2.22027777778`
`stream`	`hotOut`	`F`	`1.66527777778`
`stream`	`hotShell`	`F`	`1.66527777778`
`kpi`	`hx`	`C_cold`	`167630.972222`
`kpi`	`hx`	`C_hot`	`125728.472222`
`kpi`	`hx`	`C_r`	`0.750031277368`
`kpi`	`hx`	`LMTD`	`16.7868881533`
`kpi`	`hx`	`NTU`	`1.0053658663`
`kpi`	`hx`	`Nu_shell`	`195.331265319`
`kpi`	`hx`	`Nu_tube`	`416.603846461`
`kpi`	`hx`	`Pr_shell`	`3.16405821902`
`kpi`	`hx`	`Pr_tube`	`5.87094963164`
`kpi`	`hx`	`Q`	`1888506.69951`

… and 27 more reference values in the case's `expected`.

hxWorkflow2_rate_designed

tutorials/steady/heat/hxWorkflow2_rate_designed

What it does: Shell-and-tube heat exchanger sized from GEOMETRY: the overall U and area are COMPUTED from the tube bundle + per-side convective correlations (Gnielinski tube-side, Kern shell-side), not specified. Water-water clean service. The computed U lands in the textbook clean water-water band (~2000-3000 W/(m2.K), Kern / Sinnott). Tube-side h_i via Gnielinski (Incropera & DeWitt 6th ed.); shell-side h_o via the Kern method (Process Heat Transfer, 1950). U and area are RESULTS that then feed the unchanged eps-NTU duty.

The story: PART 2 of 2: RATE the designed exchanger

THE HEAT-EXCHANGER DESIGN WORKFLOW (the standard design sequence), part 2.

Part 1 (hxWorkflow1_design_from_duty) gave the engine a DUTY TARGET (cool the hot shell 330 → 315 K) and it SIZED the bundle: 105 tubes, shell ID 0.306 m (Kern/Sinnott, 2-pass U-tube). That is the detailed-design step.

Part 2 – THIS case – is the detailed-rating / checking step: take the DESIGNED geometry as a FIXED given (‘model geometry;‘) and RATE it – confirm it delivers the target outlet, and read the honest pressure drop. This is the verification a real project always does after sizing: the design loop used a bundle correlation for the shell diameter and rounded the tube count UP, so the rated exchanger is slightly OVER-surfaced – the outlet comes out a touch below 315 K (good: margin), and that is what you check here.

OFF-DESIGN (try it): drop the cold-tube flow F to ~6000 kmol/h and re-run – less coolant → the hot stream leaves WARMER than 315 K, and the shell dP falls. That is the rating question ("will my exchanger cope when the plant turns down?") the design step cannot answer. The geometry is now FIXED; only the performance moves.

Numbers are RESULTS pinned by –record. Same clean water-water service, same Gnielinski (tube) + Kern (shell) correlations, single-phase, 2-pass U-tube (1-2 eps-NTU).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	coldOut	F	2.22027777778
stream	coldTube	F	2.22027777778
stream	hotOut	F	1.66527777778
stream	hotShell	F	1.66527777778
kpi	hx	C_cold	167630.972222
kpi	hx	C_hot	125728.472222
kpi	hx	C_r	0.750031277368
kpi	hx	LMTD	16.7876807682
kpi	hx	NTU	1.00522927337
kpi	hx	Nu_shell	195.283521062
kpi	hx	Nu_tube	416.603846461
kpi	hx	Pr_shell	3.16405821902
kpi	hx	Pr_tube	5.87094963164
kpi	hx	Q	1888394.47754

. . . and 27 more reference values in the case’s `expected`*.*

`mheatx01_two_hot_one_cold`

`tutorials/steady/heat/mheatx01_two_hot_one_cold`

What it does: Multi-stream heat exchanger: 2 hot + 1 cold, balance closes

The story: Multi-stream heat exchanger (the commercial multi-stream detailed-rating analogue): TWO hot water streams are cooled by ONE cold water stream inside a single adiabatic shell — the canonical multi-stream heat-recovery problem.

THE SPEC (glass-box, "information follows the streams"): every inlet carries its OWN outlet target temperature. The unit does NOT rate hardware; it VERIFIES the first law:

Q_k = F_k (h_k,out - h_k,in) per stream Q_hot = Σ Q_k over the streams that cool (< 0, released) Q_cold = Σ Q_k over the streams that heat (> 0, absorbed) imbalance = Σ Q_k over ALL streams (external duty)

For a truly adiabatic shell the imbalance is ~0; a non-zero value is the external duty the topology would have to supply, REPORTED honestly (never a silently-invented duty). The unit also builds the hot/cold composite curves and reports the INTERNAL minimum-approach temperature (the in-unit pinch).

Streams (water, liquid): hot1 : 100 kmol/h, 380 → 330 K (gives up ~105 kW) hot2 : 60 kmol/h, 360 → 330 K (gives up ~38 kW) cold : 340 kmol/h, 300 → 320 K (takes up ~143 kW)

The cold flow is sized so the bundle is adiabatic: cold absorbs what the two hot streams release. Both hot outlets (330 K) stay above the cold outlet (320 K), so the composite curves never cross — dT_min > 0.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`cold`	`F`	`0.0944444444444`
`stream`	`coldOut`	`F`	`0.0944444444444`
`stream`	`hot1`	`F`	`0.0277777777778`
`stream`	`hot1Out`	`F`	`0.0277777777778`
`stream`	`hot2`	`F`	`0.0166666666667`
`stream`	`hot2Out`	`F`	`0.0166666666667`
`kpi`	`mhx`	`Q_cold`	`142611.111111`
`kpi`	`mhx`	`Q_cold_kW`	`142.611111111`
`kpi`	`mhx`	`Q_hot`	`-142611.111111`
`kpi`	`mhx`	`Q_hot1_kW`	`-104.861111111`
`kpi`	`mhx`	`Q_hot2_kW`	`-37.75`
`kpi`	`mhx`	`Q_hot_kW`	`-142.611111111`
`kpi`	`mhx`	`balanceCloses`	`1`
`kpi`	`mhx`	`dT_min`	`30`

. . . and 10 more reference values in the case's `expected`.

phasechange01_partial_condenser

`tutorials/steady/heat/phasechange01_partial_condenser`

What it does: Partial condenser: benzene/toluene vapour cooled to quality 0.4

The story: A superheated benzene/toluene vapour enters a condenser that is asked to land its outlet at a vapour QUALITY of 0.4 (40 % of the moles stay vapour, 60 % condense). The duty Q is the RESULT – the unit solves it as a built-in self-target (no outerDict, no variables block) and ANNOUNCES that it did so. Because the mixture is multicomponent, the condensed liquid is toluene-RICH and the remaining vapour benzene-RICH – the partial- condensation lesson: a partial condenser is also a SEPARATION.

feedVapour (superheated, 1 bar, 390 K, vf = 1) → condenser (outletQuality 0.4) → wetOut (two-phase, 0 < vf < 1)

Same outlet vocabulary as rankine02_water's IF97 boiler/condenser, on the GENERIC (Antoine + Raoult) backend.

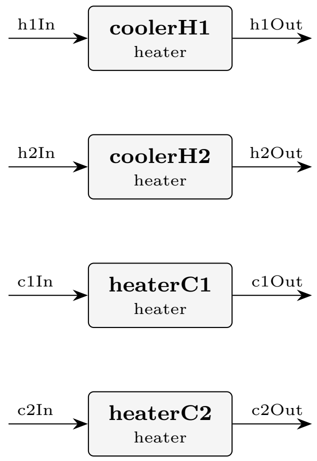
What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`condenser`	`vf_out`	`0.399999998746`
`kpi`	`condenser`	`T_out`	`367.494289759`
`kpi`	`condenser`	`Q_kW`	`-630.508868406`
`kpi`	`condenser`	`Tsat`	`352.826063711`
`stream`	`wetOut`	`F`	`0.0277777777778`
`stream`	`wetOut`	`vf`	`0.399999998746`
`utility`	`condenser`	`cooling.coolingWater.duty_kW`	`-630.508868406`
`utility`	`condenser`	`cooling.coolingWater.T`	`367.494289759`
`utility`	`condenser`	`cooling.coolingWater.kg_s`	`15.0839442202`
`utility`	`condenser`	`cooling.coolingWater.eur_h`	`0.907932770504`
`stream`	`feedVapour`	`F`	`0.0277777777778`
`stream`	`feedVapour`	`T`	`390`
`stream`	`wetOut`	`T`	`367.494289759`
`kpi`	`condenser`	`F`	`0.0277777777778`

...and 6 more reference values in the case's `expected`.

pinch01_four_stream_classic

tutorials/steady/heat/pinch01_four_stream_classic



What it does: Pinch P1 targets: classic 4-stream problem table (dTmin 20 K)

The story: Four independent duty units realise the classic four-stream pinch population. The working fluid (hxFluid, case-local SYNTHETIC teaching fluid) has a CONSTANT liquid cp = 100 J/(mol K), so each stream's CP = F∗cp is exact and the targets are hand-checkable:

stream F [mol/s] CP [kW/K] T_in [C] T_out [C] Q [kW] H1 hot 20 2.0 150 60 -180.0 H2 hot 80 8.0 90 60 -240.0 C1 cold 25 2.5 20 125 +262.5 C2 cold 30 3.0 25 100 +225.0

Problem table at dTmin = 20 K (shift hot -10 K, cold +10 K; shifted boundaries in C, descending: 140 135 110 80 50 35 30; a positive dH is a heat SURPLUS cascading downward):

interval T∗ CP_hot CP_cold dH [kW] cascade [kW] 140 → 135 2 0 +10.0 +10.0 135 → 110 2 2.5 -12.5 -2.5 110 → 80 2 5.5 -105.0 -107.5 <- minimum 80 → 50 10 5.5 +135.0 +27.5 50 → 35 0 5.5 -82.5 -55.0 35 → 30 0 2.5 -12.5 -67.5

Targets: Q_H,min = -min(cascade) = 107.5 kW, Q_C,min = final + Q_H,min = -67.5 + 107.5 = 40 kW,

pinch at T∗ = 80 C → hot 90 C / cold 70 C.

Each unit is a 'heater' with a SIGNED hardware duty Q; the outlet T is the enthalpy-balance RESULT (with cp constant it lands exactly on the table's target temperatures). The four units are thermally independent here – the pinchPass only TARGETS and ANALYSES; the network is never rewritten.

P2 candidate-match table (hand-worked; pinch hot 90 C / cold 70 C, segments clipped at the pinch; Q_max = min(Q_hot, Q_cold, min(CP_h,CP_c)∗(Th_hi - Tc_lo - dTmin))):

region hot cold CP rule at the pinch Q_max [kW] limited by above H1 C1 2.0 <= 2.5 holds 120 approach (and H1's 120 kW) above H1 C2 2.0 <= 3.0 holds 90 C2's above-duty below H1 C1 2.0 >= 2.5 VIOLATED 60 H1's below-duty (away from pinch only) below H1 C2 2.0 >= 3.0 VIOLATED 60 H1's below-duty (away from pinch only) below H2 C1 8.0 >= 2.5 holds 125 approach == C1's below-duty below H2 C2 8.0 >= 3.0 holds 135 approach == C2's below-duty

Violation diagnostics (each unit is utility-served here, so every duty on the wrong side of the pinch is a coursework violation): cooler above the pinch: H1's 120 kW above 90 C; heaters below the pinch: C1's 125 kW + C2's 135 kW below 70 C. Identity check: 120 + 260 = 380 kW = Q_heat_current - Q_H_min = 487.5 - 107.5 – the excess over target IS the sum of the violations.

Four streams, the textbook problem: two hot (H1 150 → 60 °C at CP 2 kW/K, H2 90 → 60 °C at CP 8 kW/K) and two cold (C1 20 → 125 °C at CP 2.5 kW/K, C2 25 → 100 °C at CP 3 kW/K), each modelled as a heater or cooler with a declared duty, on a synthetic constant-Cp fluid (hxFluid, 100 J/mol K, case-local) so that CP = F·cp exactly. With ΔT_min = 20 K the pinch pass finds the targets the whole subject is named after: Q_H,min = 107.5 kW, Q_C,min = 40 kW, pinch at shifted T∗ = 353.15 K (hot 363.15 K / cold 343.15 K). Today's unintegrated flowsheet buys 487.5 kW of heating and rejects 420 kW — the gap is what integration could recover.

1. The problem table is printed, interval by interval. Six shifted intervals, ΣCP hot and cold, ΔH per interval and the cascade — [pinch] lines in the Log tab. Redo the cascade by hand (start at 0, carry each ΔH down, then add the most negative value back); the golden is what you get. 2. The targets are a consequence, not a design. Nothing was matched yet. reports/pinch/candidateMatches.csv lists every admissible hot–cold pair per region (6 candidates, 6 admissible) with its counter-current end-approach bound. The word ∗optimal∗ never appears — the pass recommends, it never rewrites the flowsheet. 3. The three violations name themselves. coolerH1 rejects 120 kW ∗above∗ the pinch; heaterC1 and heaterC2 draw 125 + 135 kW ∗below∗ it. Their sum is exactly the current-minus-target excess. Those are the coursework rules, read off your own flowsheet. 4. Composite curves are a file, not a claim. reports/pinch/compositeCurves.csv — plot it; the two curves touch at ΔT_min and the overhangs are Q_H,min and Q_C,min.

Change dTmin in system/postDict to 10 K and rerun: both targets fall and the pinch moves. Then look at energyBalance for this case — it says UNAVAILABLE, because hxFluid carries no formation enthalpy on purpose: a pinch analysis needs only CP, and the case declares nothing it does not need.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`c1In`	`F`	`0.025`
`stream`	`c1Out`	`F`	`0.025`
`stream`	`c2In`	`F`	`0.03`
`stream`	`c2Out`	`F`	`0.03`
`stream`	`h1In`	`F`	`0.02`
`stream`	`h1Out`	`F`	`0.02`
`stream`	`h2In`	`F`	`0.08`
`stream`	`h2Out`	`F`	`0.08`
`kpi`	`coolerH1`	`F`	`0.02`
`kpi`	`coolerH1`	`P`	`101325`
`kpi`	`coolerH1`	`Q`	`-180000`
`kpi`	`coolerH1`	`Q_kW`	`-180`
`kpi`	`coolerH1`	`Q_per_mol`	`-9000`
`kpi`	`coolerH1`	`T_in`	`423.15`
. . . and 66 more reference values in the case's `expected`.			

reboiler_water_copper

tutorials/steady/heat/reboiler_water_copper

What it does: Steam-heated copper-surface pool reboiler: Rohsenow nucleate flux → duty, Zuber CHF ceiling (geometry mode)

The story: A steam-heated copper-surface POOL REBOILER whose duty is NOT specified – it EMERGES from the physics of NUCLEATE BOILING, exactly as a real reboiler's does.

Saturated-liquid water at 1 bar sits in a pool boiling on a bundle of horizontal copper tubes heated INSIDE by condensing steam. The 'phaseChanger' runs in 'model geometry;' with a 'boiling { ... }' block (the boiling resolution): a Rohsenow (1952) NUCLEATE-boiling flux q" is computed on the tube surface, the copper wall conducts, and the heating-medium film delivers the heat. Because the nucleate flux itself depends on the wall superheat dT_excess = T_surface - Tsat (and on its CUBE), the unit solves a 1-D energy balance at the surface (q_boil == q_heat) – the surface temperature is the unknown. Then the duty is Q = q" ∗ A and the outlet is the dome-flash at that duty.

water (saturated liquid, 1 bar, vf = 0) → reboiler (model geometry: Rohsenow nucleate flux + steam film) → wet vapour (partially boiled) + Q (RESULT)

HONESTY-AS-STRUCTURE (the binding constraint): the nucleate flux is +/-100% uncertain – C_sf is surface-FINISH lab data (polish, oxidation), NOT a fluid property. There is NO default C_sf and NO catalogue in v1: it is STUDENT-SUPPLIED here (water-copper 0.013, polished) WITH a mandatory citation. the result carries the [q/2, 2q] scatter band (C_sf +/-25%, cubed). the RELIABLE figure is the Zuber (1959) CHF ceiling, printed FIRST. A design above CHF is HARD-REFUSED (burnout) – a pool boiler cannot operate above the critical heat flux.

Method (glass-box): q_boil : Rohsenow nucleate flux, water-copper, C_sf 0.013, s 1.0 q" = mu_l h_fg sqrt(g(rho_l-rho_v)/sigma) [cp_l dT_excess / (C_sf h_fg Pr_l^s)]^3 (Rohsenow 1952; Incropera & DeWitt, Ch.10 Table 10.1) q_CHF : Zuber 1959, q"_max = 0.149 h_fg rho_v^0.5 [sigma g (rho_l-rho_v)]^0.25 (~1.26 MW/m^2 for water at 1 atm) wall : copper cylindrical resistance r_o ln(r_o/r_i)/k medium : a condensing-steam film coefficient h (high) the CHF margin SAFE/BURNOUT + the cube sensitivity are printed.

All film/saturation properties are IAPWS-IF97 water (incl. sigma via R1-76). The case carries its own constant/components/water.dat (case-local copy).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0277777777778
stream	wetVapour	F	0.0277777777778
kpi	reboiler	F	0.0277777777778
kpi	reboiler	P	100000
kpi	reboiler	Q	111804.644945
kpi	reboiler	Q_kW	111.804644945
kpi	reboiler	T_in	372
kpi	reboiler	T_out	372.755918611
kpi	reboiler	T_surface	381.390508143
kpi	reboiler	Tsat	372.755918611
kpi	reboiler	area	1.25663706144
kpi	reboiler	chf_margin	0.0709724846287
kpi	reboiler	dT_excess	8.6345895313
kpi	reboiler	q_CHF	1253960.82217

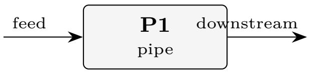
...and 10 more reference values in the case's `expected`.

hydraulics

The theory you need. Pressure drop and flow distribution: Darcy–Weisbach with friction factors, fittings as equivalent lengths, pump curves against system curves. Expect the operating point at the curves' intersection.

`pipe01_water_line`

`tutorials/steady/hydraulics/pipe01_water_line`



What it does: Pipe pressure drop: 2000 kmol/h water in a 0.1 m x 50 m commercial-steel line (Churchill friction)

The story: A single PIPE element. Water enters at 5 bar and flows through 50 m of 0.1 m commercial-steel pipe with two elbows, a globe valve, and a 2 m rise. The pressure DROP is a COMPUTED RESULT of the geometry and the flow — the pipe has NO pressure knob (information follows the streams: a pipe only dissipates, a pump would raise P, a valve would set a let-down P).

feed (5 bar liquid, 2000 kmol/h) → [P1 pipe] D, L, roughness, dz, fittings → ΔP (result) → downstream (P reduced by the computed ΔP, T held)

Friction factor: Churchill (default) — one explicit correlation across laminar / transition / turbulent. Re ~ 1.5e5 here, so the flow is turbulent and all three ΔP terms (friction, fittings, elevation) matter.

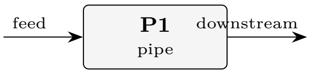
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`downstream`	`F`	`0.555555555556`
`stream`	`feed`	`F`	`0.555555555556`
`kpi`	`P1`	`F`	`0.555555555556`
`kpi`	`P1`	`P_in`	`500000`
`kpi`	`P1`	`P_out`	`462986.302076`
`kpi`	`P1`	`dP_elevation`	`19520.1545706`
`kpi`	`P1`	`dP_fittings`	`9626.38419279`
`kpi`	`P1`	`dP_friction`	`7867.15916089`
`kpi`	`P1`	`deltaP`	`37013.6979243`
`kpi`	`P1`	`density`	`995.250904772`
`kpi`	`P1`	`frictionFactor`	`0.0192870918591`
`kpi`	`P1`	`head_loss_m`	`1.79235705234`
`kpi`	`P1`	`regime`	`2`
`kpi`	`P1`	`reynolds`	`149291.31749`

... and 4 more reference values in the case's `expected`.

`pipe02_airwater_twophase`

`tutorials/steady/hydraulics/pipe02_airwater_twophase`



What it does: Gas-liquid two-phase pressure drop: air (N2) + water in a horizontal 0.05 m x 10 m line, Lockhart-Martinelli. The two-phase path activates automatically because the feed flashes two-phase (0 < V/F < 1).

The story: GAS-LIQUID two-phase pressure drop

feed (air N2 + water, two-phase at 2 bar / 25 C) → [P1 pipe] D, L, roughness, horizontal → ΔP (result) → downstream (P reduced by the two-phase ΔP)

The two-phase path activates AUTOMATICALLY: the feed flashes two-phase at (T, P) (N2 is supercritical → all vapour; water → mostly liquid), so 0 < V/F < 1 and the pipe routes to the gas-liquid models instead of the single-phase liquid path. Model: Lockhart-Martinelli (Chisholm multiplier + Butterworth holdup). Try ‘model homogeneous;‘ in twoPhase to compare.

The Lockhart-Martinelli correlation was developed and VALIDATED on isothermal air-water (and similar gas-liquid) pipe data – Lockhart & Martinelli, Chem. Eng. Prog. 45 (1949) 39 – so air-water is exactly the case it is trusted on.

Basis: N2 3 kmol/h (the fast light gas) + water 200 kmol/h (the slow heavy liquid) → a liquid-dominated two-phase flow (j_G ~ a few m/s, j_L ~ 0.5 m/s).

Air (N_2) + water flowing through a horizontal 0.05 m × 10 m commercial-steel line. The pipe's two-phase path activates automatically: the feed flashes two-phase at 2 bar / 25 °C (N_2 is supercritical → all vapour; water → mostly liquid), so 0 < V/F < 1 and the pipe routes to the gas-liquid models instead of the single-phase liquid path.

The two teaching points: 1. The multiplier φ_L^2 = 8.6: two-phase friction is far above either phase alone — the classic Lockhart-Martinelli amplification, visible in the printout via X and the Chisholm C. 2. Slip → holdup: the homogeneous (no-slip) model would put the liquid fraction at j_L/(j_L+j_G) ≈ 0.12; L-M gives 0.36 because the liquid moves slower than the gas and accumulates. Switch twoPhase { model homogeneous; } to see the difference for yourself.

The Lockhart-Martinelli correlation (Chem. Eng. Prog. 45 (1949) 39) was developed and validated on isothermal air-water (and similar gas-liquid) pipe data — so air-water is exactly the system it is trusted on. The model structure here (X, φ_L^2, the tt/vt/tv/vv C-selection, the Butterworth 1975 void fraction) reproduces that framework.

Switch twoPhase { model ...; } and compare — the spread IS a lesson (two-phase ΔP correlations routinely disagree by ~50 %):

Layer 2 (Friedel, BeggsBrill) needs surface tension — supplied here by transport { surfaceTension { model BrockBird; } } (from water's Tc/Pc/Tb). Beggs-Brill also classifies the flow pattern (segregated / intermittent / distributed) and corrects holdup for pipe inclination (here horizontal, so $\psi = 1$).

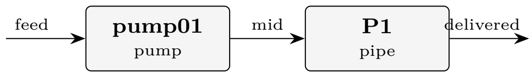
- Adiabatic: the inlet vapour fraction is held along the pipe (no flashing / condensation in the line). Flashing flow is a future increment. - Liquid density: ?_L = 877 kg/m^3 comes from the Rackett branch, ~12 % low for water (a known anomaly, also in pipe01_water_line). It biases the ∗absolute∗ ΔP; the correlation ∗structure∗ is unaffected. - Liquid viscosity via operation { viscosity ...; } (water's value): the flash leaves a trace of dissolved N_2 in the liquid and the Vogel model has no N_2 entry.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	downstream	F	0.0563888888889
stream	feed	F	0.0563888888889
kpi	P1	F	0.0563888888889
kpi	P1	P_in	200000
kpi	P1	P_out	193989.686272
kpi	P1	Re_Gs	26035.4493901
kpi	P1	Re_Ls	28789.8441141
kpi	P1	X_martinelli	2.63535511116
kpi	P1	dP_elevation	0
kpi	P1	dP_friction	6010.31372773
kpi	P1	deltaP	6010.31372773
kpi	P1	holdup_liquid	0.357791422881
kpi	P1	jG	4.07152675186
kpi	P1	jL	0.514023769077
. . . and 8 more reference values in the case's `expected`.			

pumpSystem01_operating_point

`tutorials/steady/hydraulics/pumpSystem01_operating_point`



What it does: Pump vs system curve: sweep the feed flow, the operating point is the intersection

The story: A pump and its piping system IN SERIES, swept over the feed flow (see system/outerDict). The classical operating-point construction:

PUMP characteristic – the shaft delivers a FIXED 2 kW at eta 0.65, so the pressure rise FALLS with flow: dP_pump = eta W / Q_vol. This is the constant-shaft-power characteristic of the pump MODEL, not a manufacturer's tested H(Q) curve – the engine says what it models, never pretends to a datasheet it was not given.

SYSTEM curve – the same flow through 50 m of 0.1 m commercial-steel pipe with two elbows, a globe valve and a 2 m rise: dP_system = friction(Q^2) + fittings(Q^2) + rho g dz.

Where the two curves CROSS, the pump's rise exactly pays the system's demand – the operating point. Hand estimate (water, v ~ 1.807e-5 m3/mol, A = 7.854e-3 m2, f ~ 0.018, sum K = 11.8):

pump: dP ~ 1300 / (F 1.807e-5) ~ 7.19e4 / F Pa system: dP ~ 5.45e4 F^2 + 1.956e4 Pa (F in kmol/s) cross: F* ~ 0.99 kmol/s (~3.6e3 kmol/h), dP* ~ 0.73 bar

The sweep straddles that crossing (0.4..1.6 kmol/s); each point is a complete converged pass, and the curves are read off the pump's and the pipe's own published KPIs – nothing is fitted or interpolated here.

What to expect (golden-locked):

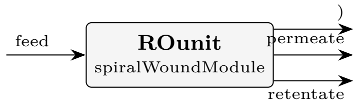
kind	unit/stream	key	expected
`stream`	`delivered`	`F`	`1`
`stream`	`feed`	`F`	`1`
`stream`	`mid`	`F`	`1`
`kpi`	`P1`	`F`	`1`
`kpi`	`P1`	`P_in`	`371942.446043`
`kpi`	`P1`	`P_out`	`297097.627131`
`kpi`	`P1`	`dP_elevation`	`19553.4351728`
`kpi`	`P1`	`dP_fittings`	`31136.3992358`
`kpi`	`P1`	`dP_friction`	`24154.9845033`
`kpi`	`P1`	`deltaP`	`74844.818912`
`kpi`	`P1`	`density`	`996.947743257`
`kpi`	`P1`	`frictionFactor`	`0.0183083994383`
`kpi`	`P1`	`head_loss_m`	`5.65541381864`
`kpi`	`P1`	`regime`	`2`
... and 23 more reference values in the case's `expected`*.*			

membranes

The theory you need. Solution-diffusion (Wijmans & Baker 1995): inside a dense membrane the pressure is uniform, so permeation is concentration-driven. Linearised: $J_w = A_w(\Delta P - \Delta\pi)$ and $J_s = B_s(c_m - c_p)$ – water driven by NET pressure, solute blind to pressure; (A_w, B_s) are MATERIAL constants from `data/standards/assets/`. Concentration polarisation (film model) makes the wall concentration $c_m = c_b\, e^{J_w/k}$ exceed the bulk – the flux–polarisation–osmotic feedback is THE nonlinearity of RO/NF, solved per module segment. Expect: rejection improving with pressure ($c_p = B_s c_m/(J_w + B_s)$), polarisation eating driving force at high flux, and charged-membrane (Donnan) effects on multivalent ions that plain solution-diffusion cannot see – that is what the DSPM-DE sibling model is for. *Full derivation: Theory Guide, the membrane chapter.*

`membrane01_RO_NaCl_seawater`

`tutorials/steady/membranes/membrane01_RO_NaCl_seawater`



What it does: Spiral-wound RO of 35 g/L NaCl feed at 55 bar (canonical seawater desalination)

The story: the canonical first-membrane case.

Feed: 35 g/L NaCl (seawater concentration), water makes up the rest. With c_NaCl = 35/58.44 = 0.599 kmol/m^3 and c_water in seawater ≈ 1025 kg/m^3 ? 18 kg/kmol = 56.94 kmol/m^3, the mole fractions are z_NaCl = 0.599 / (0.599 + 56.94) = 0.01040 and z_water = 0.9896. Feed molar flow F = 575 kmol/h (≈ 10 m^3/h of seawater).

Module: a single SW30HR-style spiral-wound element, 35 m^2 active area, 1 m channel length. Feed pressure 55 bar (typical for seawater RO — enough head to overcome ~30 bar of osmotic pressure at 35 g/L plus channel ΔP plus a net driving force of ~22 bar). Permeate pressure 1 bar.

Mass transfer: k_film = 5e-5 m/s (typical spacer-enhanced value; a Schock-Miquel correlation at this flow gives ~5 × 10?? m/s; we hardcode the value here and explore the effect of changing it in tutorial membrane03).

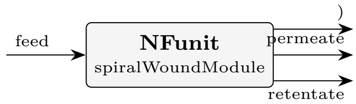
Expected (with the SW30HR parameters): water recovery ≈ 18 % (single pass) observed NaCl rej. ≈ 99.5 % (typical for SW30HR) average water flux ≈ 20–25 LMH feed ΔP = 2 bar (set linearly via dP_feed_total)

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`ROunit`	`R_obs_NaCl`	`0.997859093278`
`kpi`	`ROunit`	`water_recovery`	`0.157762204919`
`kpi`	`ROunit`	`J_w_avg`	`1.32725173967e-05`
`stream`	`feed`	`F`	`0.159722222222`
`stream`	`feed`	`T`	`298.15`
`stream`	`permeate`	`F`	`0.0257849252597`
`stream`	`permeate`	`T`	`298.15`
`stream`	`retentate`	`F`	`0.133938394975`
`stream`	`retentate`	`T`	`298.15`
`kpi`	`ROunit`	`dP_feed`	`200000`
`kpi`	`ROunit`	`mass_closure_rel`	`1.50817535928e-16`

membrane02_NF_sugar

tutorials/steady/membranes/membrane02_NF_sugar



What it does: NF270 partial rejection of glucose at 10 bar (canonical NF: 10 g/L sugar feed)

The story: loose-NF separation of glucose from water, the canonical NF test that pedagogically distinguishes NF from RO:

RO tight membrane, high pressure, near-100 % rejection NF (here) loose membrane, low pressure, PARTIAL rejection (~50 %)

Feed: 10 g/L glucose (a typical pharma / food-processing feed). c_glucose = 10 / 180.16 = 0.0555 kmol/m^3 With pure-water c_water = 1000/18 = 55.56 kmol/m^3, z_glucose = 0.0555 / (0.0555 + 55.56) = 9.97e-4, z_water = 0.999003. Feed F = 200 kmol/h (≈ 3.6 m^3/h of dilute solution).

Module: NF270-style spiral-wound element, 35 m^2 area, 1 m length. Feed pressure 10 bar (an order of magnitude less than the SW30HR seawater case — the price of loose rejection is that membrane permeability A_w is 10× higher, so the same operating flux is achieved with 10× less driving pressure). Permeate at atmospheric. Same k_film = 5e-5 m/s.

Expected (with the NF270 parameters): water recovery ≈ 20-25 % observed glucose rej.≈ 50-65 % (? the SW30HR's 99 %+ !) average water flux ≈ 30-40 LMH

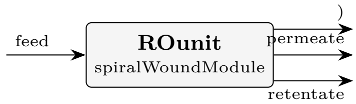
The partial rejection is the defining NF feature: the membrane is not tight enough to fully reject glucose (MW 180, NF270 nominal MWCO 200 Da), so a fraction of the sugar leaks through with the water — and that leakage is what makes NF useful for fractionation rather than just total rejection.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.0555555555556
stream	permeate	F	0.0149607433177
stream	retentate	F	0.0405951326871
kpi	NFunit	J_w_avg	5.40656892958e-05
kpi	NFunit	R_obs_glucose	0.662821422531
kpi	NFunit	dP_feed	100000
kpi	NFunit	water_recovery	0.267701129953
stream	feed	T	298.15
stream	permeate	T	298.15
stream	retentate	T	298.15
kpi	NFunit	mass_closure_rel	2.19886558047e-16

membrane03_concentration_polarization

tutorials/steady/membranes/membrane03_concentration_polarization



What it does: Concentration polarization study: sweep k_film over a SW30HR seawater RO case

The story: Same SW30HR seawater RO case as membrane01_RO_NaCl_seawater, but here we use the SweepDriver (see outerDict) to vary the mass-transfer coefficient k_film over a decade and plot how observed rejection and average flux change.

The physics: c_m / c_b = exp (J_w / k_film) (film model)

When k_film is large (good cross-flow mixing, e.g. high-velocity channel + good spacer), the boundary layer is thin and the concentration at the membrane wall is close to the bulk: c_m $\approx$ c_b. $\Delta\pi$ then reflects the bulk concentration, fluxes are high, and observed rejection is close to the intrinsic value.

When k_film is small (poor mixing, low velocity, dead zones), the solute concentrates at the wall: c_m ? c_b. $\Delta\pi$ rises sharply, J_w collapses, and a larger fraction of the wall solute leaks through, dropping observed rejection.

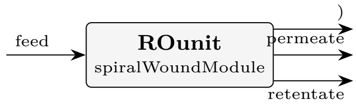
This sweep illustrates why membrane-module designers obsess over spacer geometry and feed velocity — they are not adjusting the membrane itself, they are adjusting k_film.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.159722222222
stream	permeate	F	0.0312284897591
stream	retentate	F	0.128495408945
kpi	ROunit	J_w_avg	1.60743521195e-05
kpi	ROunit	R_obs_NaCl	0.998356903981
kpi	ROunit	dP_feed	200000
kpi	ROunit	mass_closure_rel	3.01635071855e-16
kpi	ROunit	water_recovery	0.191065881266
stream	feed	T	298.15
stream	permeate	T	298.15
stream	retentate	T	298.15

`membrane04_spacer_schock_miquel`

`tutorials/steady/membranes/membrane04_spacer_schock_miquel`



What it does: Seawater RO with k_film computed from spacer-channel hydraulics (Schock-Miquel Sherwood); k_film falls ~6.6 to ~5.6e-5 m/s along the channel

The story: the same seawater RO as membrane01, but k_film is COMPUTED from the spacer-channel hydraulics (Schock-Miquel Sherwood correlation) instead of being a fixed number. The ‘massTransfer‘ block makes k_film a selectable model: Sh = 0.065 Re^0.875 Sc^0.25 with Re from the local crossflow velocity u = Q_b(z)/A_channel — so k_film FALLS along the channel as the retentate flow drops (the profile column ‘k_film‘ shows this). At the inlet u ~ 0.26 m/s gives k_film ~ 7e-5 m/s, a touch above membrane01’s fixed 5e-5 (better mixing → less polarisation → slightly higher recovery/rejection). Declare a bare ‘k_film <value>;‘ (no massTransfer block) to recover membrane01’s fixed-coefficient behaviour – the ONE way to pin it (the redundant ‘model constant;‘ route was removed 2026-08-23).

membrane01_RO_NaCl_seawater – the canonical first-membrane case.

Feed: 35 g/L NaCl (seawater concentration), water makes up the rest. With c_NaCl = 35/58.44 = 0.599 kmol/m^3 and c_water in seawater ≈ 1025 kg/m^3 ? 18 kg/kmol = 56.94 kmol/m^3, the mole fractions are z_NaCl = 0.599 / (0.599 + 56.94) = 0.01040 and z_water = 0.9896. Feed molar flow F = 575 kmol/h (≈ 10 m^3/h of seawater).

Module: a single SW30HR-style spiral-wound element, 35 m^2 active area, 1 m channel length. Feed pressure 55 bar (typical for seawater RO — enough head to overcome ~30 bar of osmotic pressure at 35 g/L plus channel ΔP plus a net driving force of ~22 bar). Permeate pressure 1 bar.

Mass transfer: k_film = 5e-5 m/s (typical spacer-enhanced value; a Schock-Miquel correlation at this flow gives ~5 × 10?? m/s; we hardcode the value here and explore the effect of changing it in tutorial membrane03).

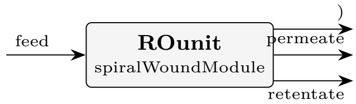
Expected (with the SW30HR parameters): water recovery ≈ 18 % (single pass) observed NaCl rej. ≈ 99.5 % (typical for SW30HR) average water flux ≈ 20–25 LMH feed ΔP = 2 bar (set linearly via dP_feed_total)

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.159722222222`
`stream`	`permeate`	`F`	`0.0286373914761`
`stream`	`retentate`	`F`	`0.131086212764`
`kpi`	`ROunit`	`J_w_avg`	`1.47407124897e-05`
`kpi`	`ROunit`	`R_obs_NaCl`	`0.99810213206`
`kpi`	`ROunit`	`dP_feed`	`26065.9064084`
`kpi`	`ROunit`	`water_recovery`	`0.175213731874`
`stream`	`feed`	`T`	`298.15`
`stream`	`permeate`	`T`	`298.15`
`stream`	`retentate`	`T`	`298.15`
`kpi`	`ROunit`	`mass_closure_rel`	`7.54087679638e-16`

membrane05_train

tutorials/steady/membranes/membrane05_train



What it does: Multi-element membrane train: 3 SW30HR elements in series (seawater RO)

The story: a MULTI-ELEMENT TRAIN: 3 SW30HR spiral-wound elements in series, seawater RO (the same feed + per-element geometry as membrane04, which is the single element). 'elements 3;' chains them: the retentate of element 1 is the feed of element 2, and so on; the permeate is collected over the whole train. Both the spacer k_film and the spacer pressure drop (Schock-Miquel) are computed from the local hydraulics, so they fall element by element as the flow drops.

The pedagogical point is the per-element DEGRADATION reported in the KPIs (recovery_elemK, c_ret_elemK): element 1 sees fresh feed (high P, low c) and recovers the most; the last element sees concentrated, lower-pressure brine (P dropped by friction + the inter-element connector loss) and recovers far less. The train recovery is the cumulative sum — higher than a single element, but with diminishing returns down the train.

Feed: 35 g/L NaCl seawater, z_NaCl = 0.01040, F = 575 kmol/h (~10 m^3/h), 55 bar, 298.15 K — the same feed as membrane01/04. Each of the 3 elements is 35 m^2 / 1 m, SW30HR, permeate at 1 bar, with a 0.2 bar inter-element connector loss.

What this run produces (golden master): train water recovery 35.3 % (3 elements, cumulative) observed NaCl rejection 99.68 % train-average water flux 34.85 LMH (9.68e-6 m/s) total feed-side ΔP ~0.99 bar (Schock-Miquel friction + 2 connector losses)

The per-element DEGRADATION (the teaching point) reads off the KPIs: recovery_elem1 17.2 % c_ret_elem1 0.697 (fresh feed) recovery_elem2 28.4 % c_ret_elem2 0.805 (cumulative) recovery_elem3 35.3 % c_ret_elem3 0.891 (cumulative) The retentate concentration climbs element to element while each successive element adds less recovery: a single SW30HR element at this feed recovers ~17 %, the second adds ~11 %, the third only ~7 % — the diminishing return of stacking elements in series at fixed feed.

NOTE (known issue): this train case currently has an open mass-balance discrepancy under investigation; it is run-only in the regression and the KPIs above are the present golden values, not a validated material balance.

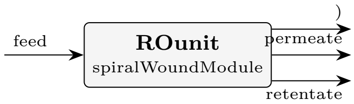
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.159722222222
stream	permeate	F	0.0590874153104
stream	retentate	F	0.100637449271
kpi	ROunit	J_w_avg	1.01384186683e-05
kpi	ROunit	R_obs_NaCl	0.99694467035
kpi	ROunit	c_ret_elem1	0.68349184216
kpi	ROunit	c_ret_elem2	0.793343136295
kpi	ROunit	c_ret_elem3	0.881961644632
kpi	ROunit	dP_feed	101143.55107
kpi	ROunit	elements	3
kpi	ROunit	recovery_elem1	0.175213731874
kpi	ROunit	recovery_elem2	0.289843097471
kpi	ROunit	recovery_elem3	0.361527335552
kpi	ROunit	water_recovery	0.361527335552

. . . and 4 more reference values in the case's `expected`*.*

membrane06_pitzer

tutorials/steady/membranes/membrane06_pitzer



What it does: Seawater RO with the Pitzer osmotic model (vs membrane01 van't Hoff)

The story: the canonical seawater RO (same as membrane01) but with the PITZER osmotic-pressure model instead of van't Hoff. van't Hoff (phi = 1) over-estimates the osmotic pressure of a real brine: for NaCl the osmotic coefficient is phi ~ 0.92 at seawater concentration (and dips further before rising at high molality). The Pitzer ion-interaction model captures phi(I), so the osmotic back-pressure is ~7-8 % LOWER → a slightly HIGHER water flux and recovery than membrane01 (15.55 %). The 'osmotic' block makes it a selectable model with the NaCl coefficients (calibratable); remove the block to recover van't Hoff.

membrane01_RO_NaCl_seawater – the canonical first-membrane case.

Feed: 35 g/L NaCl (seawater concentration), water makes up the rest. With c_NaCl = 35/58.44 = 0.599 kmol/m^3 and c_water in seawater ≈ 1025 kg/m^3 ? 18 kg/kmol = 56.94 kmol/m^3, the mole fractions are z_NaCl = 0.599 / (0.599 + 56.94) = 0.01040 and z_water = 0.9896. Feed molar flow F = 575 kmol/h (≈ 10 m^3/h of seawater).

Module: a single SW30HR-style spiral-wound element, 35 m^2 active area, 1 m channel length. Feed pressure 55 bar (typical for seawater RO — enough head to overcome ~30 bar of osmotic pressure at 35 g/L plus channel ΔP plus a net driving force of ~22 bar). Permeate pressure 1 bar.

Mass transfer: k_film = 5e-5 m/s (typical spacer-enhanced value; a Schock-Miquel correlation at this flow gives ~5 × 10?? m/s; we hardcode the value here and explore the effect of changing it in tutorial membrane03).

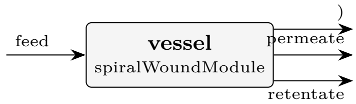
Expected (with the SW30HR parameters): water recovery ≈ 18 % (single pass) observed NaCl rej. ≈ 99.5 % (typical for SW30HR) average water flux ≈ 20–25 LMH feed ΔP = 2 bar (set linearly via dP_feed_total)

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.159722222222`
`stream`	`permeate`	`F`	`0.0284588599363`
`stream`	`retentate`	`F`	`0.13126472432`
`kpi`	`ROunit`	`J_w_avg`	`1.46488544181e-05`
`kpi`	`ROunit`	`R_obs_NaCl`	`0.997986472373`
`kpi`	`ROunit`	`dP_feed`	`200000`
`kpi`	`ROunit`	`water_recovery`	`0.174121871794`
`stream`	`feed`	`T`	`298.15`
`stream`	`permeate`	`T`	`298.15`
`stream`	`retentate`	`T`	`298.15`
`kpi`	`ROunit`	`mass_closure_rel`	`6.03270143711e-16`

membrane07_scaling_si

tutorials/steady/membranes/membrane07_scaling_si



What it does: Brackish RO, 6 x 8-inch modules: per-module scaling audit (SI at wall vs bulk)

The story: WHERE along the vessel does scaling start?

Brackish RO on the SAME representative groundwater analysis as tutorials/props/electrolyte/scaling_ro_brackish (~1500 mg/L TDS):

ion mg/L mol/kg w ion mg/L mol/kg w Ca+2 84 0.0021 Cl- 510 0.0144 Mg+2 27 0.0011 SO4-2 250 0.0026 Na+ 363 0.0158 HCO3- 183 0.0030 K+ 12 0.0003

Here the analysis is a PROCESS STREAM: the ions are flowsheet components (case-local constant/components/, MW from ions.dat) and the mg/L totals become mole fractions against water (x_i = m_i / 55.55; m_i = (mg/L)/MW at rho ~ 1 kg/L):

Ca 3.7731e-5 Mg 1.9998e-5 Na 2.8425e-4 K 5.5253e-6 Cl 2.5897e-4 SO4 4.6852e-5 HCO3 5.3989e-5 water = balance

HARDWARE BY NOMINAL DIAMETER (no ‘area‘): one pressure vessel with 6 x 8-inch standard 40-inch elements – ‘moduleDiameter 8 in; nModules 6;‘. The per-module area (37.2 m2, 400 ft2 class) comes from the engine’s nominal table and the derivation is announced: "6 x 37.2 m2 (8in/40in standard element) = 223.2 m2".

THE SCALING AUDIT (‘scaling {}‘): at each module exit the local BULK concentrate AND the membrane-WALL composition (c_m from the module’s own film-model polarisation) are speciated (Davies) and the calcite / gypsum saturation indices recorded. The WALL crosses SI=0 modules before the bulk does – concentration polarisation, normally an invisible flux penalty, is seen here as THE scaling driver.

pH 7.0 GIVEN, not solved: the raw well water is already calcite- supersaturated (scaling_ro_brackish solves the raw feed to pH 7.72, SI_calcite = +0.24 – it would scale in the PIPE, let alone the vessel). Real brackish plants dose acid for exactly that reason; here the feed is dosed and CONTROLLED at pH 7.0, and the audit asks the honest follow-up: with the dosing in place, where along the vessel does the concentrate STILL reach calcite saturation? (‘pH solve;‘ is also accepted – it re-solves electroneutrality per module, the undosed scenario.)

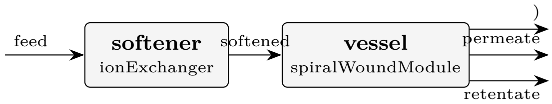
Expected story (KPIs locked by the golden master; recovery 55 %): calcite WALL crosses SI=0 at module 3 of 6 (r_cum = 0.31); calcite BULK is still undersaturated there (-0.27) and only crosses in module 6 (+0.03) – the wall leads the bulk by three full modules: polarisation IS the scaling driver; gypsum stays undersaturated everywhere (max wall SI = -0.73).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.194444444431
stream	permeate	F	0.107212342592
stream	retentate	F	0.0872324772734
kpi	vessel	J_w_avg	8.65338211369e-06
kpi	vessel	R_obs_Ca	0.999474633761
kpi	vessel	R_obs_Cl	0.994770268574
kpi	vessel	R_obs_HCO3	0.996072731843
kpi	vessel	R_obs_K	0.993471097928
kpi	vessel	R_obs_Mg	0.999605925236
kpi	vessel	R_obs_Na	0.994770268574
kpi	vessel	R_obs_SO4	0.999737250092
kpi	vessel	SI0_module_calcite	3
kpi	vessel	SI0_module_gypsum	-1
kpi	vessel	SI0_recovery_calcite	0.30517794604
. . . and 27 more reference values in the case's `expected`.			

membrane08_softened_scaling

tutorials/steady/membranes/membrane08_softened_scaling



What it does: Softener + brackish RO: ion-exchange removes the scaling driver upstream of the vessel

The story: SOFTEN the feed, and the scaling is GONE.

THE CONTRAST (read membrane07_scaling_si first – the BEFORE):

membrane07 the SAME brackish groundwater (~1500 mg/L TDS, Ca 84 / Mg 27 mg/L) goes straight into a 6 x 8-inch brackish-RO vessel. As the concentrate builds along the train the membrane-WALL composition crosses calcite SI = 0 at MODULE 3 of 6 (r_cum = 0.31) – concentration polarisation is THE scaling driver; the vessel would foul.

membrane08 here a cation-exchange SOFTENER (‘ionExchanger‘, SAC resin in the Na form, Gaines-Thomas) sits UPSTREAM of the SAME vessel. Ca and Mg drop ~99% onto the resin (Na rises eq-for-eq – the salt penalty), so the water entering the vessel is essentially hardness-free. With Ca gone the calcite IAP collapses and the per-module WALL SI_calcite stays NEGATIVE through ALL 6 modules – the scaling driver is removed at source.

ZERO change to the membrane. The spiralWoundModule is byte-for-byte the membrane07 vessel (moduleDiameter 8 in; nModules 6; the same scaling{} audit); it simply receives a BETTER feed. Removing the hardness, not chasing the supersaturation in the vessel, is the cure – the engineering lesson.

HONESTY (printed, non-suppressible): the softener returns the LIMITING (fully-equilibrated) effluent – the best a fresh / fully-loaded contactor does per pass, NOT a bed in service (breakthrough / regeneration are TRANSIENT and not modelled). See the banner on every run.

Data: curated USGS PHREEQC records (speciation, minerals, ions, exchange; public domain) resolved from data/standards/ at runtime + case-local ion .dats (constant/components/), the SAC_Na resin nameplate and the SW30HR membrane (constant/membranes/) – the same records as membrane07 / softener_brackish; self-DESCRIBING (inline manifest), seal with bin/choupo-import for a runtime-self-contained copy.

Expected story (golden master): softener removes ~99.99% hardness (Ca/Mg leakage ~ 1e-2 mg/L, Na added ~ 6.7 meq/L at the given pH 7.0 closure – the eq-for-eq salt penalty for the full hardness load); the vessel’s maxSI_calcite_wall is NEGATIVE (-3.68: no SI = 0 crossing in any module, SI0_module_calcite = -1), vs +0.27 / crossing at module 3 in membrane07.

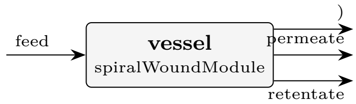
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.194444444431
stream	permeate	F	0.105450929615
stream	retentate	F	0.0890050816188
stream	softened	F	0.19445566796
kpi	softener	CEC_utilised_pct	100
kpi	softener	Ca_leakage_mgL	0.00999051830577
kpi	softener	I	0.024397591267
kpi	softener	Mg_leakage_mgL	0.00549118647587
kpi	softener	Na_added_meqL	6.71875413026
kpi	softener	Na_added_mgL	154.464157455
kpi	softener	bedVolume_m3	1.5
kpi	softener	dH_exchange_Jkgw	21.9905023247
kpi	softener	hardness_in_mgL_CaCO3	320.962820378
kpi	softener	hardness_out_mgL_CaCO3	0.0475632817167

... and 44 more reference values in the case’s `expected`*.*

membrane09_index_vs_rigorous

tutorials/steady/membranes/membrane09_index_vs_rigorous



What it does: Seawater RO at high recovery: industry calcite indices (LSI / Stiff-Davis / Ryznar) vs the rigorous activity SI at the membrane WALL

The story: the bag-of-indices industry shortcut is NOT safe at the membrane wall in a high-recovery concentrate.

THE LESSON (activity vs concentration – the whole point of the case):

The industry calcite-scaling indices read every plant data-sheet ships – LSI Langelier saturation index (Langelier 1936) Stiff-Davis brine-corrected LSI (Stiff & Davis 1952) RSI Ryznar stability index (Ryznar 1944) are built from ION CONCENTRATIONS plus an EMPIRICAL ionic-strength correction (a published pK-shift / chart-fit K). Choupo's rigorous SI_calcite instead forms real ION ACTIVITIES a_i = gamma_i m_i from the Davies model:

LSI = pH - logK_cc(T) - p[Ca2+] - p[HCO3-] (CONCENTRATIONS) SI = pH - logK_cc(T) - p{a_Ca2+}- p{a_HCO3-} (ACTIVITIES)

They share the SAME thermodynamic anchor logK_cc(T) (the HCO3-basis calcite log K, = pK2 - pKsp) – so the ONLY difference is concentration vs activity. At LOW ionic strength gamma → 1 and LSI == SI (validation); as the activity coefficients fall away from 1 the two DIVERGE.

WHERE IT BITES – THE WALL. Concentration polarisation piles solute against the membrane wall: the LOCAL ionic strength at the wall runs far above the bulk. There the empirical index over-predicts calcite saturation (it cannot see the gamma reduction), while the rigorous activity SI does not. Reading the index off the bulk analysis – as the industry shortcut does – therefore MISLEADS at exactly the place scaling actually starts.

THE CASE: standard surface seawater (S = 35, the major-ion subset), one pressure vessel of 7 x 8-inch SW elements, pushed to high recovery. Acid is dosed and CONTROLLED at pH 7.0 (real SWRO practice). At each module exit the BULK concentrate AND the membrane-WALL composition are speciated (Davies) and BOTH the rigorous SI_calcite AND the empirical LSI / Stiff-Davis / Ryznar are recorded, bulk + wall.

THE STORY (locked by the golden master): - This is already a SEAWATER concentrate (bulk I ~ 0.68 mol/kg at module 1, far above the I → 0 regime where LSI == SI), so the empirical index OVER-PREDICTS calcite saturation at EVERY module, including the first: at module 1 LSI(wall) = +0.98 vs rigorous SI_calcite(wall) = +0.20 (gap ~ 0.78); LSI(bulk) = +0.56 vs SI(bulk) = -0.28 (gap ~ 0.84). At this ionic strength there is no point where index and activity-SI agree to hundredths – to SEE that limit you must drop to a brackish / low-I feed (the index family is only validated there). - the gap widens further into the vessel: at the deepest module LSI(wall) over-predicts the rigorous SI_calcite(wall) by ~ 0.78 – the gap (LSI - SI) printed in the audit footer is the headline number. Stiff-Davis pulls PART of the way back toward the rigorous SI (its empirical brine K), but not all the way – it is still a chart fit, not the real gammas. - Ryznar (RSI = 2 pHs - pH) reads the same saturation pHs as LSI on the practical 0..14 stability scale.

Engineering take-away (Choupo's transparency differentiator): the concentration-based index family is convenient but UNSAFE for high-recovery membrane scaling – the rigorous activity SI, visible term-by-term, is the honest call.

Sibling cases: membrane07_scaling_si (bulk vs wall SI, brackish); tutorials/props/electrolyte/pitzer_vs_davies_ro_concent (Davies vs Pitzer activity fork). Index kernel + primary sources: src/thermo/electrolyte/ScalingIndices.{H,cpp}.

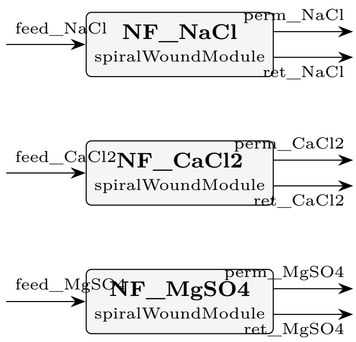
PRIMARY SOURCES: Langelier, W.F. (1936) J. AWWA 28(10) 1500-1521. Stiff, H.A. & Davis, L.E. (1952) Trans. AIME 195, 213-216. Ryznar, J.W. (1944) J. AWWA 36(4) 472-486.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.250000000004
stream	permeate	F	0.067554824405
stream	retentate	F	0.182446484542
kpi	vessel	J_w_avg	4.67400706828e-06
kpi	vessel	LSI_calcite_bulk_end	0.765353874066
kpi	vessel	LSI_calcite_wall_end	0.985330125213
kpi	vessel	RSI_calcite_wall_end	5.02933974957
kpi	vessel	R_obs_Ca	0.999255215369
kpi	vessel	R_obs_Cl	0.992602201838
kpi	vessel	R_obs_HCO3	0.994441275819
kpi	vessel	R_obs_K	0.990769979906
kpi	vessel	R_obs_Mg	0.999441306528
kpi	vessel	R_obs_Na	0.992602201838
kpi	vessel	R_obs_SO4	0.99962746766
. . . and 29 more reference values in the case's `expected`.			

membrane10_dspmde_divalent

tutorials/steady/membranes/membrane10_dspmde_divalent



What it does: DSPM-DE NF: divalent (MgSO4) > mixed (CaCl2) > monovalent (NaCl) rejection at 10 bar

The story: the charge-aware NF signature, glass-box.

THREE parallel NF270 elements, one per salt, all on the SAME membrane (NF270_dspmde, with the DSPM-DE charged-pore tier) at the SAME 10 bar / 5 mM feed. The point is the REJECTION ORDERING that the Donnan-Steric Pore Model with Dielectric Exclusion produces from first principles:

R_obs(MgSO4) > R_obs(CaCl2) > R_obs(NaCl) (2:2 divalent) (2:1 mixed) (1:1 monovalent)

WHY (Bowen-Welfoot 2002): the membrane carries a NEGATIVE fixed charge (X_d = -40 mol/m3). The CO-ION (the anion, here Cl- / SO4–) is excluded by Donnan; a DIVALENT co-ion (SO4–) is excluded far more strongly than a monovalent one (z^2 in the Donnan + Born partition). So MgSO4 – whose co-ion is divalent – is rejected most; NaCl – whose co-ion is the monovalent Cl- – least. This is the classic NF fingerprint that plain solution-diffusion (one B_s per salt) cannot predict mechanistically.

Data: the salts (case-local constant/components/) carry an electrolyte{} ion decomposition and the membrane (constant/membranes/) carries the poreModel{} block; the ions' radius/D0 transport tier resolves from data/standards/ at runtime – self-DESCRIBING (inline manifest), seal with bin/choupo-import for a runtime-self-contained copy.

'transport DSPM-DE;' selects the charged-pore law; drop it (or set 'transport solutionDiffusion;') to compare against the baseline.

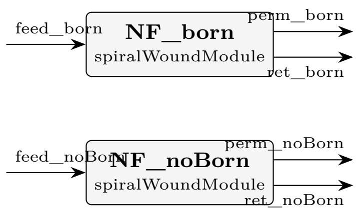
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed_CaCl2	F	0.0555555555556
stream	feed_MgSO4	F	0.0555555555556
stream	feed_NaCl	F	0.0555555555556
stream	perm_CaCl2	F	0.0191775929422
stream	perm_MgSO4	F	0.020040469745
stream	perm_NaCl	F	0.0208297354228
stream	ret_CaCl2	F	0.0363780274704
stream	ret_MgSO4	F	0.0355151987788
stream	ret_NaCl	F	0.034725847724
kpi	NF_CaCl2	J_w_avg	6.91063443929e-05
kpi	NF_CaCl2	R_obs_CaCl2	0.704588151782
kpi	NF_CaCl2	R_obs_MgSO4	1
kpi	NF_CaCl2	R_obs_NaCl	1
kpi	NF_CaCl2	dP_feed	100000

... and 25 more reference values in the case's `expected`*.*

`membrane11_dspmde_born_toggle`

`tutorials/steady/membranes/membrane11_dspmde_born_toggle`



What it does: DSPM-DE NF: Born dielectric-exclusion toggle (same NaCl feed, poreDielectric on vs off)

The story: SEE the dielectric-exclusion (Born) term.

The SAME NaCl feed (50 mM, 10 bar) through TWO NF270 elements that differ in ONE thing only:

NF_born : membrane NF270_dspmde – poreDielectric 35 (Born ON) NF_noBorn : membrane NF270_dspmde_noBorn – poreDielectric absent (Born OFF)

Everything else – pore radius, porosity/thickness, fixed charge X_d, feed, pressure, area, k_film – is IDENTICAL. The ONLY physics that changes is the Born dielectric-exclusion term

dW_i = (z_i^2 e^2 / 8 pi eps0 r_i)(1/eps_p - 1/eps_b)

(Bowen & Welfoot 2002, Eq. 16). With eps_p < eps_b an ion pays an extra solvation-energy penalty to enter the low-dielectric pore, so it is excluded MORE strongly. The student reads the Born contribution straight off the two R_obs(NaCl) KPIs: R_obs(NF_born) > R_obs(NF_noBorn).

This is the pedagogical core of DSPM-"DE": the DE (dielectric exclusion) is what distinguishes the 2002 Bowen-Welfoot model from the 1997 steric-Donnan DSPM. Toggling poreDielectric is a single, visible switch.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed_born`	`F`	`0.0555555555556`
`stream`	`feed_noBorn`	`F`	`0.0555555555556`
`stream`	`perm_born`	`F`	`0.0103299082671`
`stream`	`perm_noBorn`	`F`	`0.0112254914132`
`stream`	`ret_born`	`F`	`0.0452257284146`
`stream`	`ret_noBorn`	`F`	`0.0443301389741`
`kpi`	`NF_born`	`J_w_avg`	`3.7229087272e-05`
`kpi`	`NF_born`	`R_obs_NaCl`	`0.861027895843`
`kpi`	`NF_born`	`dP_feed`	`100000`
`kpi`	`NF_born`	`water_recovery`	`0.185615581216`
`kpi`	`NF_noBorn`	`J_w_avg`	`4.0473218085e-05`
`kpi`	`NF_noBorn`	`R_obs_NaCl`	`0.65954044001`
`kpi`	`NF_noBorn`	`dP_feed`	`100000`
`kpi`	`NF_noBorn`	`water_recovery`	`0.201790063872`

. . . and 8 more reference values in the case's `expected`.

optimisation

The theory you need. The outer driver wraps the whole simulation in $f(\mathbf{x})$: sweeps map it, Nelder–Mead descends it, parameter estimation fits it to data (Levenberg–Marquardt, with identifiability diagnostics – χ^2_{red}, correlation, conditioning). Expect every function evaluation to be a full, honest flowsheet pass.

designSpec01_triple_equal_areas

tutorials/steady/optimisation/designSpec01_triple_equal_areas

What it does: DesignSpec: equal-area triple-effect sugar evaporator (manipulate $A so L3 = 4500 kg/h)

The story: SOURCE. The exercise number below is the numbering of the course notes this case was built from; the TEXTBOOK those notes drew on is not recorded anywhere in this case. It used to be labelled with a course acronym, and that acronym named the WRONG course (a CFD one), so it was removed on 2026-08-12 rather than replaced with a guess. The gap is left visible: a missing citation a reader can see beats a plausible one nobody can check. (‘sucrose.dat‘ does carry a primary for the Cp correlation – Coulson & Richardson 6th ed. Example 14.4.)

Same physics as evaporator02_triple_effect_sugar (textbook Example 10.4 — three cocurrent effects on a 22 500 kg/h, 5 wt% sucrose feed). The twist: instead of HARD-CODING each effect’s area at 100 m^2, we declare A as a CASE-LEVEL VARIABLE and use the outer DesignSpec driver to find the equal-areas value that delivers L_3 ~ 4500 kg/h (25 wt% sucrose), the textbook design target.

This tutorial showcases two new pieces of v0.30 machinery:

1. $references in the dict (‘variables { ... }‘ block + ‘$A‘ syntax inside any scalar). The same A is shared by the three effects without copy-paste — which makes it trivially a single degree of freedom for the outer solver.

2. The ‘designSpec‘ outer driver: a Newton-ND wrap on Flowsheet::solve, manipulating the case’s $variables to satisfy named targets. Here it is a 1-D Newton in $A on the residual L3.F - 0.001249 kmol/s. Convergence in 4-5 evaluations.

F=22 500 kg/h L1 L2 L3 (target ~ 4500 kg/h) x=0.05 →[1] ——→ [2] ——→ [3] —-→ ^ ^ ^ | V1 V2 V3 → condenser Steam (drives 2) (drives 3) 200 kPa P1 = 80 P2 = 45 P3 = 15 kPa

Constraints we leave fixed: - P_i pressures (textbook cascade) - U_i overall HTCs per effect (textbook values) - Feed and steam conditions - Equal areas: enforced trivially by the single $A variable

Read the run log to see the Newton converge in <10 evaluations. At the optimum, the case ends with the converged final state on stdout, exactly like a hand-tuned evaporator02 run — but the area was DERIVED, not specified.

Initial guess A_0 = 100 m^2 (the educated guess from evaporator02); the solver should home in on ~ 110-120 m^2 to hit the target.

Target tolerance: ±5 kg/h on L_3, encoded via tol = 1.4e-6 kmol/s (= 5 kg/h ? 3600 s/h ? 18 kg/kmol).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	L1	F	0.242853616254
stream	L2	F	0.15012886479
stream	L3	F	0.0529527028804
stream	V1	F	0.0876457894447
stream	V2	F	0.0927247514636
stream	V3	F	0.0971761619099
stream	cond1	F	0.13885829138
stream	cond2	F	0.0876457894447
stream	cond3	F	0.0927247514636
stream	feed	F	0.330499405699
stream	utilitySteam_200kPa	F	0.13885829138
kpi	effect1	A	108.00857667
kpi	effect1	A_m2	108.00857667
kpi	effect1	BPE	0.107441424817
... and 92 more reference values in the case's `expected`*.*			

fitNRTL01_ethanol_water

tutorials/steady/optimisation/fitNRTL01_ethanol_water

fit_NRTL_ethanol_water
fitParameters

What it does: Fit NRTL ethanol/water pair against 1 atm bubble-T data (Levenberg-Marquardt)

The story: fitNRTL01 – Levenberg-Marquardt regression of the ethanol-water NRTL pair against 1 atm bubble-T data, on the CANONICAL fit engine (fitParameters, choupoProps). This case previously ran the condemned FitBinaryPair outer driver in choupoSolve, which abused a dummy flowsheet and steered the feed composition through the legacy streams{} block per data row – the design forum (#53) ordered it moved here, WITH a golden: its chi^2 once degraded 100x with nothing going red, because a fit without numerical verification does not belong in a green corpus.

Data: transcribed from constant/experiments/ethanol-water-101kPa (the raw- data home; DECHEMA / Gmehling-Onken Vol. I Part 1, pedagogical subset). Starting point: the catalogue NRTL values in constant/thermoPhysPropDict.

The parameter-estimation workflow: regress the four NRTL binary interaction parameters of the ethanol-water pair (a__ij, b__ij, a__ji, b__ji; α held fixed) against experimental bubble-temperature data at 1.01325 bar, using hand-rolled Levenberg-Marquardt inside the fitParameters engine (successor of the retired fitBinaryPair driver).

∗ See an outer Marquardt loop (3-up/5-down trust region on λ) wrapped around an inner Newton-1D solving Σ K x = 1 for each (x_i, T) pair. ∗ Watch λ oscillate as the algorithm switches between Gauss-Newton (λ small) and gradient-descent (λ large) regimes. ∗ Watch some LM steps get rejected — chi^2 gets worse, the step is reversed, λ is multiplied by 5, retry. The whole "trust region" idea visible in one column of the log.

Note how minimal the forward case is: one stream, one bubbleT unit. The outer driver is what does the work — iterating over the dataset internally and looping LM over the four parameters.

Four keywords (plus optional maxIter, tolerance, fdStep, lambda0). The driver:

1. locates the pair in thermoPhysPropDict.activityModel.pairs by name, 2. reads the dataset (a two-column flat list of x_i T_i), 3. finds the bubbleT unit's input stream from the flowsheetDict, 4. for each LM step: - 8 forward evals to build a central-difference Jacobian over 4 params × N data points, - solve the 4×4 normal equations with Marquardt damping, - try the step; accept if chi^2 decreased, otherwise raise λ and retry.

After the run header you get (the iteration trace below is illustrative – it shows the SHAPE of an LM run, accept/reject and λ adaptation; the actual run converges to chi^2 = 1.0426, RMS = 0.3079 K with fitted a__ij = 9.328, b__ij = -3609.18):

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	fit_NRTL_ethanol_water	chi2	1.04279
diag	fit_NRTL_ethanol_water	chi2_reduced	0.14897
diag	fit_NRTL_ethanol_water	cond_JtJ	3.55369e+11
diag	fit_NRTL_ethanol_water	converged	1
diag	fit_NRTL_ethanol_water	corr_computed	1
diag	fit_NRTL_ethanol_water	dof	7
diag	fit_NRTL_ethanol_water	identifiable	0
diag	fit_NRTL_ethanol_water	invertible	1
diag	fit_NRTL_ethanol_water	iter	24
diag	fit_NRTL_ethanol_water	max_abs_corr	0.999967
diag	fit_NRTL_ethanol_water	max_abs_resid	0.625729
diag	fit_NRTL_ethanol_water	n_data	11
diag	fit_NRTL_ethanol_water	n_params	4
diag	fit_NRTL_ethanol_water	params_at_bound	0
. . . and 6 more reference values in the case's expected.			

fitNRTL02_thermoml_isobars

tutorials/steady/optimisation/fitNRTL02_thermoml_isobars

fit__NRTL__across_	score__catalogue__pair_	score__catalogue__pair__at__vacuum
fitParameters	fitParameters	fitParameters

What it does: Fit the ethanol/water NRTL pair across three measured isobars, then predict a fourth held out from a different laboratory

The story: fitNRTL02 – A GOOD FIT THAT PREDICTS WORSE. The ethanol/water NRTL pair regressed across three measured vacuum isobars, then asked about a fourth isobar, at atmospheric pressure, from a different laboratory.

THE RESULT FIRST, BECAUSE IT IS NOT THE ONE THE CASE WAS BUILT EXPECTING. The regression converges in 19 iterations to a reduced chi-square of 0.023 over 45 points – an in-sample rms of 0.144 K, which is at the scatter of the measurements. Every number on the fit's own report says it went well. Then the fitted pair meets 21 points it never saw, at 101.3 kPa, and its average absolute deviation is 0.386 K. The pair the case STARTED from – the catalogue's DECHEMA set, adjusted by nothing – manages 0.082 K on the same 21 points. The fit made the model 4.7 times worse where it was not looking.

The two control operations below make that symmetric rather than a one-sided accusation. All four numbers, each measured by this run:

vacuum data atmospheric data (Voutsas, 13-33 kPa) (Kamihama, 101.3 kPa) catalogue pair rms 0.1619 K aad 0.0821 K pair fitted here rms 0.1438 K (in-sample) aad 0.3857 K

Each pair is at its best where its evidence was taken. The regression bought an 11 % improvement where it looked, and lost a factor of 4.7 where it did not. A student who reads only 'converged 1', 'chi2_reduced 0.023' and 'rms 0.144 K' would ship the new pair; the held-out column is the only thing in the run that says not to.

WHY IT HAPPENS, AND THE SECOND LESSON. NRTL's interaction is tau = a + b/T, so identifying 'a' and 'b' separately needs data that span 1/T. Three isobars from 13 to 33 kPa span about 28 K around 320 K, which moves 1/T by roughly 9 % – better than the 12 K of a single isobar (fitNRTL01), and it shows: the condition number of J'J falls from 3.6e11 to 5.6e9, two orders of magnitude. But 'max|correlation|' is still 1.000 and the run still reports 'identifiable 0'. Two orders of magnitude of conditioning did NOT buy identifiability, and the confidence intervals say why: b_ij = -539 +/- 456, b_ji = 682 +/- 768. The parameters are a correlated pair that happens to describe the vacuum data; nothing pins either one, so nothing constrains what they do at 101 kPa. The catalogue record's own declared validity window is 298-373 K – it was fitted where it is being asked, and this fit was not.

THE EVIDENCE, AND WHERE IT CAME FROM. Both datasets were read out of the NIST/TRC ThermoML Archive by 'bin/choupo-thermoml extract-vle' and carry the citation of the article they were measured in, as a field the engine reads rather than a comment it discards:

FIT Voutsas, Pamouktsis, Argyris & Pappa, Fluid Phase Equilib. (2011), doi:10.1016/j.fluid.2011.06.009 – 45 points on three isobars, 13.15 / 19.71 / 32.86 kPa.

HELD OUT Kamihama, Matsuda, Kurihara, Tochigi & Oba, J. Chem. Eng. Data (2012), doi:10.1021/je2008704 – 21 points at 101.3 kPa.

The held-out set is not a slice withheld from the same measurement: it is a DIFFERENT LABORATORY, a different journal, a different year, at a pressure a full decade above anything the fitter was shown. That is what makes its number mean something the in-sample one cannot. The identity collision the engine CAN settle (the same DOI in both roles) is settled by construction here, because the DOIs differ; whether two studies are scientifically independent stays a human judgement, and the run announces what it knows rather than adjudicating it.

ITS NEIGHBOUR, AND WHAT IS DIFFERENT. 'tutorials/props/curation/ curate02_vle_heldout_ethanol_water' already holds out three of eleven measured points on this same system, and already reports the same identifiability finding – it was the first place in the corpus where a fitted binary pair met real evidence it had not been handed, and this case does not claim that. What is new here is the SHAPE of the test: curate02 withholds points from ONE series at ONE pressure, so it asks whether the model interpolates within a study. This case withholds an ENTIRE INDEPENDENT STUDY at a pressure a decade away, which asks whether the model transfers at all – and the two questions get different answers. Read curate02 first; it is the cleaner statement of the contract.

NO ACCEPTANCE BAND IS DECLARED, deliberately. This run REPORTS the held-out residual and claims no pass or fail on it, because a limit chosen after seeing the residual is not a limit. Every number quoted above is pinned in 'expected', so none of them can move unnoticed – including the ones that make the fit look bad. Nothing here was tuned to produce this outcome: the fit is the catalogue pair as its starting point, the bounds are fitNRTL01's, and the two datasets were the only ethanol+water bubble-temperature studies in the mirror that suited the roles.

WHAT THE RUN'S TWO CAVEATS ARE ABOUT, said plainly. The end-of-run block reports ethanol's Antoine correlation evaluated at 371.8 K and 372.8 K, outside its declared 273-369 K window. Both are the bubble-point Newton's STARTING temperature – the mole-fraction-weighted normal boiling point of the first datum in each set – and neither is a state any published answer contains. The mechanism that would say so ('AdvisoryFrame') is opened around this operation's parameter search but NOT yet inside the bubble-point Newton itself, so those two lines are reported as though they qualified the answer. That gap is named in docs/design/advisory-attribution.md rather than papered over here.

Regress the ethanol/water NRTL pair against 45 bubble points measured on three vacuum isobars, then ask the fitted pair about 21 points at atmospheric pressure, measured by a different laboratory in a different year. Both datasets come out of the NIST/TRC ThermoML Archive and carry the citation of the article they were published in.

The regression converges in 19 iterations to a reduced chi-square of 0.023 over 45 points — an in-sample rms of 0.144 K, which is at the scatter of the measurements. Everything on the fit's own report says it went well.

The regression bought an 11 % improvement where it looked, and lost a factor of 4.7 where it did not. A reader who saw only converged 1, chi2_reduced 0.023 and rms 0.144 K would ship the new pair.

*Each pair is at its best where its evidence was taken. The two evaluate operations in system/propsDict are what make that symmetric rather than an accusation: they score the *catalogue* pair on both datasets, so all four numbers come from this one run.*

NRTL's interaction is tau = a + b/T, so separating a from b needs data that span 1/T. Three isobars from 13 to 33 kPa span about 28 K around 320 K — better than the 12 K of the single-isobar twin (fitNRTL01), and it shows: the condition number of J'J falls from 3.6e11 to 5.6e9. But max is still 1.000 and the run still reports identifiable 0. Two orders of magnitude of conditioning did not buy identifiability, and the confidence intervals say why — b_ij = -539 ± 456, b_ji = 682 ± 768. The parameters are a correlated pair that happens to describe the vacuum data; nothing pins either one, so nothing constrains what they do at 101 kPa.

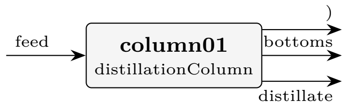
The catalogue record's own declared validity window is 298–373 K. It was fitted where it is being asked; this fit was not.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	fit_NRTL_across_isobars	aad_heldout_K	0.3857
diag	fit_NRTL_across_isobars	aad_heldout_pct	0.1088
diag	fit_NRTL_across_isobars	chi2	0.9303
diag	fit_NRTL_across_isobars	chi2_in_sample	0.9303
diag	fit_NRTL_across_isobars	chi2_reduced	0.0227
diag	fit_NRTL_across_isobars	cond_JtJ	5604973889.6716
diag	fit_NRTL_across_isobars	converged	1.0000
diag	fit_NRTL_across_isobars	corr_computed	1.0000
diag	fit_NRTL_across_isobars	dof	41.0000
diag	fit_NRTL_across_isobars	identifiable	0.0000
diag	fit_NRTL_across_isobars	invertible	1.0000
diag	fit_NRTL_across_isobars	iter	19.0000
diag	fit_NRTL_across_isobars	max_abs_corr	0.9995
diag	fit_NRTL_across_isobars	max_abs_resid	0.3678
... and 32 more reference values in the case's `expected`.			

optim01_column_reflux

tutorials/steady/optimisation/optim01_column_reflux



What it does: Optimise reflux ratio + feedStage of benzene/toluene column to minimise reboiler vapour V_strip

The story: Benzene/toluene distillation, used by the optimisation driver. Same column as sensitivity01_column_reflux; we add looser tolerances on the outer loop so the simplex can probe odd feedStages without blowing up.

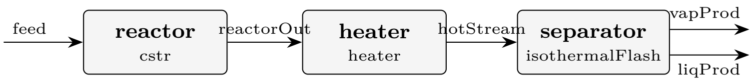
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0138888888889
stream	distillate	F	0.0138888888889
stream	feed	F	0.0277777777778
kpi	column01	B	0.0138888888889
kpi	column01	D	0.0138888888889
kpi	column01	L_rect	0.0416666666667
kpi	column01	L_strip	0.0694444444444
kpi	column01	R	3
kpi	column01	T_bottom	383.299688974
kpi	column01	T_top	353.770171691
kpi	column01	V_rect	0.0555555555556
kpi	column01	V_strip	0.0555555555556
kpi	column01	feedStage	9
kpi	column01	nStages	15

... and 15 more reference values in the case's `expected`.

optim02_process_cost

tutorials/steady/optimisation/optim02_process_cost



What it does: Minimise total module Guthrie cost of reactor + heater by varying reactor V_R and heater Q

The story: Same reactor → heater → separator flowsheet as process02_with_design. The optimisation driver varies two design variables:

reactor V_R in [0.002, 0.010] m^3 heater Q in [0, 500] W (credo-pure: hardware, not outlet T)

In v0.34 the heater is credo-pure: it specifies Q (the absolute thermal power, hardware), not Tout (which would be a DesignSpec disguised as a unit-op parameter). The Nelder-Mead simplex therefore varies the heater's Q directly; the heater's T_out becomes a KPI the student can plot, not a control variable.

Initial guesses are placeholders – overwritten by Nelder-Mead.

A small reactor → heater → separator process. The OptimizationDriver (Nelder-Mead) tunes two design variables to minimise the total Guthrie module cost of the two costed units (reactor + heater):

The heater is credo-pure: it specifies Q (the thermal power, hardware), not an outlet T. T_out is a reported KPI, not a control variable.

The optimum is not at the lower corner. Nelder-Mead converges (8 moves, simplexInit 0.15) to:

Increasing Q monotonically lowers the objective: the simplex walks the cost from ~2.0e6 EUR at Q≈170 W down to ~9.35e5 EUR at Q=500 W.

The dominant cost is the heat exchanger (~916 k of the ~935 k total). Its area follows A = Q / (U·LMTD) = Q / 6000 m^2, so Q=500 W gives only A ≈ 0.083 m^2. The Turton shell-and-tube correlation used by the Guthrie model is valid for A ? [10, 1000] m^2; our exchanger is two orders of magnitude below Smin. In that extrapolated tail the correlation

*turns *upward* as A shrinks (the quadratic term dominates for log10 A < 0), so a larger area is cheaper. The optimiser correctly exploits this: pushing Q to its upper bound grows A toward the correlation's valid range and slides down the cost curve. The reactor volume settles at an interior 0.00807 m^3 — large enough for adequate conversion (X ≈ 0.64), small enough that its vessel cost (~19 k) stays a rounding error next to the exchanger.*

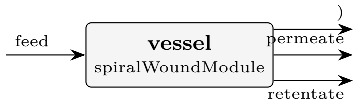
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	hotStream	F	0.000277777777778
stream	liqProd	F	0.000257419353963
stream	reactorOut	F	0.000277777777778
stream	vapProd	F	2.03584238147e-05
kpi	heater	F	0.000277777777778
kpi	heater	P	101325
kpi	heater	Q	500
kpi	heater	Q_kW	0.5
kpi	heater	Q_per_mol	1800
kpi	heater	T_in	350
kpi	heater	T_out	361.533466984
kpi	heater	dT	11.5334669843
kpi	heater	vf_out	0.0112269153032

. . . and 37 more reference values in the case's `expected`.

optim04_membrane_recovery_scaling

tutorials/steady/optimisation/optim04_membrane_recovery_scaling



What it does: Brackish RO: maximise water recovery up to the scaling limit (SQP, scaling constraints active at the optimum)

The story: HOW HARD can we push recovery before the membrane scales?

The flagship CONSTRAINED-OPTIMISATION case. Same brackish-RO vessel and scaling physics as tutorials/steady/membranes/membrane07_scaling_si (6 x 8-inch elements, the ~1500 mg/L groundwater analysis, the bulk-vs-wall saturation-index audit), but now we ASK the engine the design question the audit only diagnosed:

maximise water_recovery varying feed pressure and the (continuous) module count subject to NO mineral scaling at the membrane wall: maxSI__gypsum__wall <= 0 maxSI__calcite__wall <= 0 (rigorous Davies saturation index – smoother than the Stiff-Davis/Langelier chart fit)

Recovery and wall saturation pull AGAINST each other: every extra % recovery concentrates the retentate and drives the WALL composition (bulk x polarisation) further into supersaturation. The optimum therefore sits ON the scaling limit – at least one of the two constraints is ACTIVE, with a strictly positive Lagrange multiplier (the shadow price: how much more recovery one unit of scaling head-room would buy). That is exactly what the SQP trace prints.

GOLDEN-CHECKED regression: the SQP now converges to a CERTIFIED KKT optimum (||grad L||inf ~ 3e-8, all feasibility 0, complementarity ~1e-8, calcite-wall constraint ACTIVE with lambda ~ 1.14 > 0). Because this is a certified stationary point – not a guessed answer – its KPIs are pinned in 'expected', which ALSO guards against a regression to the singular-QP failure a readiness review caught earlier (a "QP subproblem failed: KKT system singular – LICQ failure" at iteration 1 that left the design infeasible). The KPIs still inherit the polarisation + Davies model assumptions; the golden locks the answer THIS model gives at the certified optimum, not an independent ground truth. (If the engine ever fails to converge again, the optimum block prints an explicit "NO KKT certificate ... NOT a certified optimum" block – never the misleading all-zeros certificate it used to forge on the failure path.)

Two design variables, written through $references so the SQP can drive them and the trace shows their canonical-SI values.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.194444444431
stream	permeate	F	0.0842121209871
stream	retentate	F	0.110232483957
kpi	vessel	J_w_avg	5.94313791976e-06
kpi	vessel	LSI_calcite_bulk_end	0.408647968153
kpi	vessel	LSI_calcite_wall_end	0.592350216873
kpi	vessel	RSI_calcite_wall_end	5.81529956625
kpi	vessel	R_obs_Ca	0.999395465329
kpi	vessel	R_obs_Cl	0.993987079839
kpi	vessel	R_obs_HCO3	0.995483580386
kpi	vessel	R_obs_K	0.992495032466
kpi	vessel	R_obs_Mg	0.999546531069
kpi	vessel	R_obs_Na	0.993987079839
kpi	vessel	R_obs_SO4	0.999697642081

. . . and 29 more reference values in the case's `expected`*.*

optim05_reactor_npv

tutorials/steady/optimisation/optim05_reactor_npv

What it does: Esterification CSTR sized & operated for MAX NPV by hand-rolled SQP, subject to a reactor-size cap and a capital-budget cap (active-constraint shadow prices)

The story: CONSTRAINED ECONOMIC optimisation: size & operate an esterification CSTR plant for MAXIMUM NPV.

Same esterification plant as the economics01 flagship –

feed -→ reactor (CSTR) -→ heater -→ separator (flash) |→ vapProd (ethyl-acetate-rich | overhead, the prime | product @ 1.55 EUR/kg) |→ liqProd (water/acid-rich bottoms, low-value byproduct @ 0.30 EUR/kg)

– but instead of a FIXED design we ask the engine the design question:

maximise economics.NPV varying V_R (reactor volume, m^3) T_feed (feed/reactor T, K) subject to reactor.V_R <= V_max (a physical SIZE cap) economics.FCI <= FCI_max (a CAPITAL-BUDGET cap)

THE ECONOMIC TRADE-OFF (why this is a genuine optimum, not a corner)

The heater runs at a FIXED partial duty, so the downstream flash SPLITS the reactor effluent into a light, ethyl-acetate-rich VAPOUR overhead (sold high) and a heavy, water/acid-rich LIQUID bottoms (sold cheap). Esterification conserves total mass; what conversion buys you is the RESHUFFLING of that mass toward the valuable overhead. Both knobs raise conversion – and the CSTR is ISOTHERMAL at the feed temperature (CSTR.cpp: T = feed.T), so:

V_R (reactor volume) – bigger V_R raises the residence time tau = V_R/Q, so conversion X = k.tau/(1 + k.tau) climbs, more acetic acid + ethanol turn into ethyl acetate, and the priced overhead grows and enriches (REVENUE up). But a bigger vessel costs more steel: the StirredTank sizing → Guthrie costing drives FCI (CAPEX) UP. Revenue rises with DIMINISHING returns as X saturates toward 1, while CAPEX keeps climbing – so an NPV maximum exists.

T_feed (feed/reactor temperature) – higher T raises the Arrhenius rate constant k = A.exp(-Ea/RT), so X climbs too. T_feed is the cheap conversion ACCELERATOR; its bounds are kept inside the window where the flash splits cleanly (below the temperature that boils the bottoms dry).

We deliberately set the size cap TIGHTER than where the NPV-greedy design would otherwise stop, so at the optimum the reactor-size constraint BINDS.

HOW TO READ THE SHADOW PRICES (the whole point of the tutorial)

The SQP prints, at the certified optimum, a Lagrange multiplier (lambda) for each inequality. A constraint with lambda > 0 is ACTIVE (binding) and its multiplier is the SHADOW PRICE – the marginal NPV you would gain by relaxing that cap by one unit:

reactor.V_R cap ACTIVE → lambda = d(NPV)/d(V_max) [EUR of NPV per extra m^3 of reactor head-room]. economics.FCI cap → lambda = d(NPV)/d(FCI_max) [EUR of NPV per extra EUR of capital you may spend].

A constraint with lambda ~ 0 is SLACK (not binding) – relaxing it buys nothing. Here the FCI cap is deliberately generous: it stays SLACK so the student sees BOTH an active and an inactive constraint side by side. The trace marks the active set and the KKT certificate proves the point is a true constrained optimum (||grad L|| ~ 0, complementarity ~ 0).

The two design variables are case-level $references so the SQP can drive them through ‘variables.<name>‘ and the trace shows their canonical-SI values.

THE ECONOMICS INVERTED ON 2026-08-09, AND THE OPTIMISER DID NOT

Everything above about the SQP is unchanged and still true: the run converges, the reactor-size cap is ACTIVE with a positive multiplier, the FCI cap stays SLACK, and the KKT certificate closes. What changed is the SIGN of the money.

The ‘heater‘ used to invert its fixed 200 kW on a SENSIBLE enthalpy relation, which walked the stream 11.8 K to 364.27 K without paying any latent heat. The flash then found 98.8 % vapour, and the ethyl-acetate-rich overhead – the whole revenue of this case – existed because the heater had crossed the bubble point for free. The heater now solves for the outlet STATE whose enthalpy is H_in + Q: 200 kW reaches 356.55 K and 7.7 % vapour, the flash splits V/F 0.077, revenue falls 321.7 → 88.2 M EUR against an essentially unchanged 221 M EUR cost of manufacture, and the optimum NPV is -646 M EUR (IRR and payback are UNDEFINED and published as null – the project never breaks even).

So this case still demonstrates constrained SQP, shadow prices and a KKT certificate, and it now demonstrates them on a project that loses money. Restoring the intended economics is a DESIGN decision, not a code fix: the duty (or the pressure, or the product spec) has to be chosen so the overhead is actually made. It is deliberately NOT chosen here – picking a number to make the old answer come back would be fitting the case to a result nobody re-derived. Record: docs/design/tp-stream-energy-coherence.md.

Take the economics01 ethyl-acetate plant and stop ∗fixing∗ the design — let the engine choose it. The hand-rolled SQP sizes and operates the reactor to maximise the net present value, subject to inequality caps, and prints the shadow prices (Lagrange multipliers) of the binding caps.

The objective economics.NPV is a KPI published by the EconomicsPass in the system/postDict chain (sizing → costing → economics). The optimiser runs that whole post chain on every evaluation, so the economics scalars (economics.NPV, economics.FCI, ?) are reachable as ordinary objective/constraint paths — exactly like a sweep reads them.

The heater runs at a fixed partial duty, so the flash ∗splits∗ the reactor effluent into a light ethyl-acetate-rich vapour overhead (the prime product at 1.55 ?/kg) and a heavy water/acid-rich liquid bottoms (a low-value byproduct at 0.30 ?/kg). Esterification conserves total mass; what conversion buys you is the reshuffling of that mass toward the valuable overhead.

∗ V_R — a bigger reactor raises the residence time τ = V_R/Q, so the isothermal-CSTR conversion X = k·τ/(1+k·τ) climbs and the priced overhead grows. But a bigger vessel costs more steel — the StirredTank sizing → Guthrie costing drives FCI (CAPEX) up. Revenue rises with diminishing returns (X saturates toward 1) while CAPEX keeps climbing → an NPV maximum exists. ∗ T_feed — a higher feed temperature raises the Arrhenius rate k = A·exp(-E?/RT), the cheap conversion ∗accelerator∗. Its bounds stay inside the window where the flash splits cleanly (below the ~353 K edge that boils the bottoms dry).

We deliberately set the size cap tighter than where the NPV-greedy design would otherwise stop, so the reactor-size constraint binds at the optimum.

The optimiser drives the money-losing initial design (V_R = 4 m^3, T_feed = 350 K → NPV ≈ -164 M?) to a profitable, certified optimum:

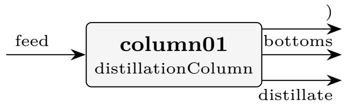
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.138888888889
stream	hotStream	F	0.138888888889
stream	liqProd	F	0.126262849909
stream	reactorOut	F	0.138888888889
stream	vapProd	F	0.01262603898
kpi	economics	COM_d	221248623.305
kpi	economics	FCI	522154.982187
kpi	economics	NPV	-632834040.588
kpi	economics	TCI	600478.229515
kpi	economics	WC	78323.2473281
kpi	economics	revenue	90983516.5487
kpi	heater	F	0.138888888889
kpi	heater	P	101325
kpi	heater	Q	200000

. . . and 43 more reference values in the case’s `expected`.

optim06_integer_stages

tutorials/steady/optimisation/optim06_integer_stages



What it does: The classic N-vs-R trade solved in ONE run: nStages declared integer → exhaustive enumeration, one full SQP per stage count, ALL values on the table

The story: sensitivity01_column_reflux – the column01 benzene/toluene column (classical Raoult textbook problem, no azeotrope), reused UNCHANGED as the inner problem of a sensitivity sweep: system/outerDict varies units[0].operation.refluxRatio over R = 1.5 .. 5.0 (10 points) and reads T_top, T_bottom, x_D_LK, x_B_HK and V_strip (the reboiler-duty proxy) – the classic purity-vs-energy trade-off, one CSV row per R.

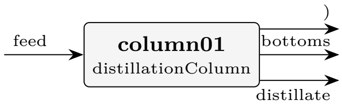
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0138888888889
stream	distillate	F	0.0138888888889
stream	feed	F	0.0277777777778
kpi	column01	B	0.0138888888889
kpi	column01	D	0.0138888888889
kpi	column01	L_rect	0.0313732166828
kpi	column01	L_strip	0.0591509944605
kpi	column01	Q_condenser_kW	-1391.66557074
kpi	column01	Q_reboiler_kW	1389.90800504
kpi	column01	R	2.25887160116
kpi	column01	T_bottom	382.835800828
kpi	column01	T_top	354.273383646
kpi	column01	V_rect	0.0452621055716
kpi	column01	V_strip	0.0452621055716

... and 18 more reference values in the case's `expected`.

pareto01_purity_energy

tutorials/steady/optimisation/pareto01_purity_energy



What it does: The purity-energy PARETO FRONT by epsilon-constraint: minimise V_strip subject to x_D_LK >= eps, eps swept 0.96..0.994 – each front point one auditable SQP run

The story: sensitivity01_column_reflux – the column01 benzene/toluene column (classical Raoult textbook problem, no azeotrope), reused UNCHANGED as the inner problem of a sensitivity sweep: system/outerDict varies units[0].operation.refluxRatio over R = 1.5 .. 5.0 (10 points) and reads T_top, T_bottom, x_D_LK, x_B_HK and V_strip (the reboiler-duty proxy) – the classic purity-vs-energy trade-off, one CSV row per R.

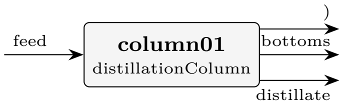
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0138888888889
stream	distillate	F	0.0138888888889
stream	feed	F	0.0277777777778
kpi	column01	B	0.0138888888889
kpi	column01	D	0.0138888888889
kpi	column01	L_rect	0.052352377807
kpi	column01	L_strip	0.0801301555848
kpi	column01	Q_condenser_kW	-2035.41756863
kpi	column01	Q_reboiler_kW	2033.86732819
kpi	column01	R	3.7693712021
kpi	column01	T_bottom	383.489767144
kpi	column01	T_top	353.562992445
kpi	column01	V_rect	0.0662412666959
kpi	column01	V_strip	0.0662412666959

. . . and 18 more reference values in the case's `expected`.

sensitivity01_column_reflux

tutorials/steady/optimisation/sensitivity01_column_reflux



What it does: Sweep reflux ratio R = 1.5..5.0 (10 pts) across the column01 case; responses: x_D purity + V_strip (reboiler-duty proxy)

The story: the column01 benzene/toluene column (classical Raoult textbook problem, no azeotrope), reused UNCHANGED as the inner problem of a sensitivity sweep: system/outerDict varies units[0].operation.refluxRatio over R = 1.5 .. 5.0 (10 points) and reads T_top, T_bottom, x_D_LK, x_B_HK and V_strip (the reboiler-duty proxy) – the classic purity-vs-energy trade-off, one CSV row per R.

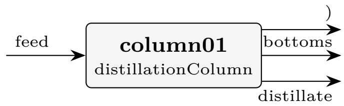
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0138888888889
stream	distillate	F	0.0138888888889
stream	feed	F	0.0277777777778
kpi	column01	B	0.0138888888889
kpi	column01	D	0.0138888888889
kpi	column01	L_rect	0.0424382716049
kpi	column01	L_strip	0.0702160493827
kpi	column01	Q_condenser_kW	-1730.95769425
kpi	column01	Q_reboiler_kW	1729.37389301
kpi	column01	R	3.05555555556
kpi	column01	T_bottom	383.383107764
kpi	column01	T_top	353.679332066
kpi	column01	V_rect	0.0563271604938
kpi	column01	V_strip	0.0563271604938

. . . and 15 more reference values in the case's `expected`*.*

sensitivity02_feasibility_map

tutorials/steady/optimisation/sensitivity02_feasibility_map



What it does: Feasibility MAP: the reflux sweep with declared constraints – every row marked (value, bound, residual, satisfied), nothing filtered; the feasible window between the purity spec and the boilup budget read off the CSV

The story: sensitivity01_column_reflux – the column01 benzene/toluene column (classical Raoult textbook problem, no azeotrope), reused UNCHANGED as the inner problem of a sensitivity sweep: system/outerDict varies units[0].operation.refluxRatio over R = 1.5 .. 5.0 (10 points) and reads T_top, T_bottom, x_D_LK, x_B_HK and V_strip (the reboiler-duty proxy) – the classic purity-vs-energy trade-off, one CSV row per R.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	bottoms	F	0.0138888888889
stream	distillate	F	0.0138888888889
stream	feed	F	0.0277777777778
kpi	column01	B	0.0138888888889
kpi	column01	D	0.0138888888889
kpi	column01	L_rect	0.0424382716049
kpi	column01	L_strip	0.0702160493827
kpi	column01	Q_condenser_kW	-1730.95769425
kpi	column01	Q_reboiler_kW	1729.37389301
kpi	column01	R	3.05555555556
kpi	column01	T_bottom	383.383107764
kpi	column01	T_top	353.679332066
kpi	column01	V_rect	0.0563271604938
kpi	column01	V_strip	0.0563271604938

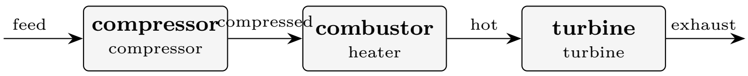
. . . and 18 more reference values in the case's `expected`.

power

The theory you need. Gas (Brayton) and steam (Rankine) cycles, alone and combined. Compressor and turbine hardware is specified by SHAFT WORK and isentropic efficiency – outlet pressure is a RESULT, not an input (the anti-DesignSpec credo). Expect: pressure ratio setting Brayton efficiency, the combined cycle's gain coming from bottoming the gas turbine's exhaust exergy, and every energy flow visible in the ledger. *Full derivation: Theory Guide, the power-cycle chapter.*

`brayton01_gas_turbine`

`tutorials/steady/power/brayton01_gas_turbine`



What it does: Closed Brayton (gas-turbine) cycle: compress air → combust (gas-aware heater) → expand to feed P

The story: a closed-loop Brayton (gas-turbine) cycle, the showcase for the gas-aware heater (v0.65).

feed (air, 1 bar, 298 K) → compressor (+10 kW, eta 0.80) : 1 → ~9.9 bar, 298 → 633 K → combustor (heater, +20 kW) : add heat at constant P; the air is gas (vf=1) so the heater integrates the ideal-gas H_ig (N2/O2 carry no liquid Cp) → 633 → ~1246 K → turbine (-W_turb, eta 0.85) : expand back to 1.013 bar

The turbine shaft power is NOT guessed: a DesignSpec (system/outerDict) manipulates $W_turb until the turbine discharges at the feed pressure, closing the cycle. Net shaft work and thermal efficiency are computed $variables (v0.64), evaluated after the solve from the unit KPIs.

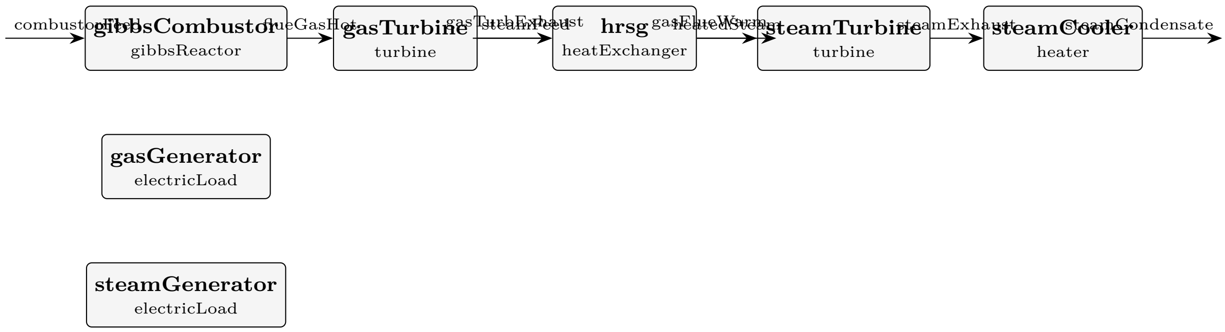
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`compressed`	`F`	`0.001`
`stream`	`exhaust`	`F`	`0.001`
`stream`	`feed`	`F`	`0.001`
`stream`	`hot`	`F`	`0.001`
`kpi`	`combustor`	`F`	`0.001`
`kpi`	`combustor`	`P`	`991200.306811`
`kpi`	`combustor`	`Q`	`20000`
`kpi`	`combustor`	`Q_kW`	`20`
`kpi`	`combustor`	`Q_per_mol`	`20000`
`kpi`	`combustor`	`T_in`	`633.374172563`
`kpi`	`combustor`	`T_out`	`1246.40394079`
`kpi`	`combustor`	`dT`	`613.029768231`
`kpi`	`compressor`	`F`	`0.001`
`kpi`	`compressor`	`P_in`	`100000`

... and 38 more reference values in the case's `expected`.

`combined01_brayton_rankine`

`tutorials/steady/power/combined01_brayton_rankine`



What it does: Combined-cycle: Brayton (CH_4 + air, Gibbs combustor) + HRSG + Rankine + 2 generators

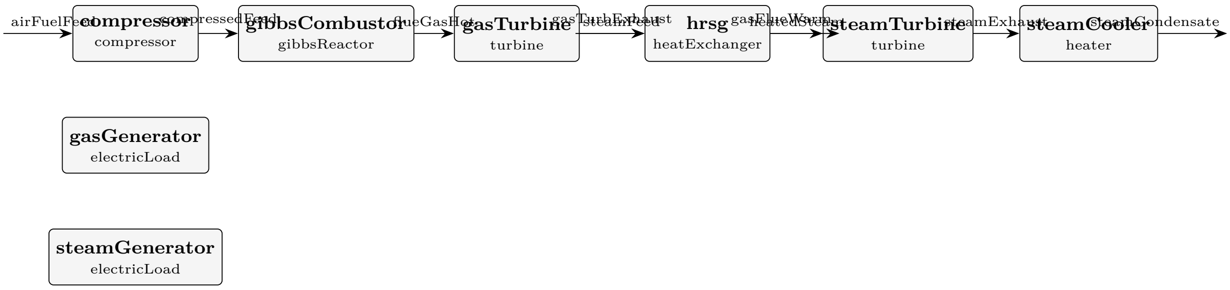
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`combustorFeed`	`F`	`0.1025`
`stream`	`flueGasHot`	`F`	`0.1025`
`stream`	`gasFlueWarm`	`F`	`0.1025`
`stream`	`gasTurbExhaust`	`F`	`0.1025`
`stream`	`heatedSteam`	`F`	`0.01`
`stream`	`steamCondensate`	`F`	`0.01`
`stream`	`steamExhaust`	`F`	`0.01`
`stream`	`steamFeed`	`F`	`0.01`
`kpi`	`gasGenerator`	`W_electric`	`582000`
`kpi`	`gasGenerator`	`W_electric_kW`	`582`
`kpi`	`gasGenerator`	`W_losses`	`18000`
`kpi`	`gasGenerator`	`W_shaft`	`600000`
`kpi`	`gasGenerator`	`W_shaft_kW`	`600`
`kpi`	`gasGenerator`	`eta_generator`	`0.97`

. . . and 98 more reference values in the case's `expected`.

combined02_brayton_rankine_shaft

tutorials/steady/power/combined02_brayton_rankine_shaft



What it does: Combined cycle with SHAFT-SPLIT: gas turbine drives the compressor + net to the generator

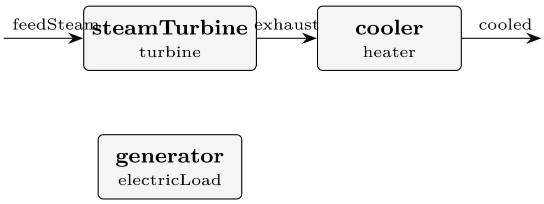
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	airFuelFeed	F	0.1025
stream	compressedFeed	F	0.1025
stream	flueGasHot	F	0.1025
stream	gasFlueWarm	F	0.1025
stream	gasTurbExhaust	F	0.1025
stream	heatedSteam	F	0.01
stream	steamCondensate	F	0.01
stream	steamExhaust	F	0.01
stream	steamFeed	F	0.01
kpi	compressor	F	0.1025
kpi	compressor	P_in	100000
kpi	compressor	P_out	828817.033759
kpi	compressor	T_in	300
kpi	compressor	T_out	592.090873473

... and 115 more reference values in the case's `expected`.

rankine01_water

tutorials/steady/power/rankine01_water



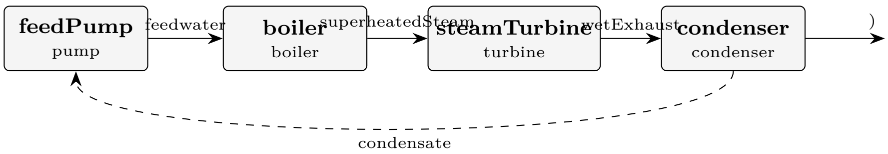
What it does: Simplified gas-phase Rankine: superheated steam → turbine (-10 kW) → cooler (gas-only, no phase change)

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	cooled	F	0.001
stream	exhaust	F	0.001
stream	feedSteam	F	0.001
kpi	cooler	F	0.001
kpi	cooler	P	52827.3976295
kpi	cooler	Q	-5000
kpi	cooler	Q_kW	-5
kpi	cooler	Q_per_mol	-5000
kpi	cooler	T_in	530.878445241
kpi	cooler	T_out	388.087132345
kpi	cooler	dT	-142.791312896
kpi	generator	W_electric	9700
kpi	generator	W_electric_kW	9.7
kpi	generator	W_losses	300
. . . and 30 more reference values in the case's `expected`.			

`rankine02_water`

`tutorials/steady/power/rankine02_water`



What it does: Closed IF97 Rankine cycle: pump → boiler → turbine → condenser → recycle

The story: a CLOSED Rankine steam cycle on IAPWS-IF97

recycle (condensate) [feedPump] [boiler] [steamTurbine] [condenser] raise P cross the extract work cross the dome to 80 bar dome up, (W__shaft, eta), DOWN, land on (liquid) superheat exhaust ~0.1 bar saturated liquid

A real Rankine: the working fluid crosses the saturation dome TWICE – up in the boiler (subcooled liquid → superheated vapour) and down in the condenser (wet vapour → saturated liquid). Pure water on IF97 supplies the steam-table h/s/v surface, so:

the boiler is a dome-crossing phaseChanger driven to superheatedVapour (its duty Q__boiler is the result), the turbine expands on the IF97 entropy surface (H__real/S__real routed), the condenser is a dome-crossing phaseChanger driven to saturatedLiquid (its duty Q__condenser is the result), the pump returns the condensate to boiler pressure (closed-form v∗dP).

The loop is CLOSED: the condensate is the tear stream, recycled to the pump. Wegstein/Newton drives the post-pass condensate to match the guess.

First-law check (KPIs): Q__boiler + Q__condenser + W__pump + W__turbine ~ 0.

Contrast: rankine01_water runs the SAME architecture on the ideal gas, in the gas phase only (no liquid leg, no phase change) – the pedagogical baseline. rankine02 is the steam-table truth.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`boiler`	`T_out`	`773.15`
`kpi`	`boiler`	`vf_out`	`1.0`
`kpi`	`boiler`	`Tsat`	`568.159121229`
`kpi`	`boiler`	`Q_kW`	`47.8559413747`
`kpi`	`condenser`	`vf_out`	`0.0`
`kpi`	`condenser`	`Tsat`	`446.515894355`
`kpi`	`condenser`	`Q_kW`	`-39.017165456`
`kpi`	`steamTurbine`	`P_out`	`858590.397228`
`kpi`	`steamTurbine`	`W_gen_kW`	`9.0`
`stream`	`superheatedSteam`	`vf`	`1.0`
`stream`	`condensate`	`vf`	`0.0`
`utility`	`boiler`	`heating.-.unserved`	`1`
`utility`	`condenser`	`cooling.coolingWater.duty_kW`	`-39.0172479639`
`utility`	`condenser`	`cooling.coolingWater.T`	`446.515894355`
. . . and 55 more reference values in the case's `expected`*.*			

reactors

The theory you need. Batch reactors integrate $\mathrm{d}n_i/\mathrm{d}t = \nu_i\, r\, V$ from a charged initial state – no in/out flows; time replaces residence time. Adiabatic runs couple the energy balance $\sum_i n_i \bar{c}_{p,i}\, \mathrm{d}T/\mathrm{d}t = -\Delta H_{\mathrm{rxn}} r V$, so composition and temperature evolve together. Stiffness appears the moment fast and slow reactions coexist (radical chains: μs radical lifetimes vs ms induction) – the explicit RK4 integrator then needs Δt at the FASTEST scale or blows up, while Rosenbrock23 (L-stable, linearly implicit) steps at the scale of the SOLUTION. The stiffness ratio is printed; watch it. *Full derivation: Theory Guide, the batch/ODE chapter.*

acetone02_luyben_reactor

tutorials/steady/reactors/acetone02_luyben_reactor

What it does: Luyben’s acetone reactor: IPA dehydrogenation at 90 % conversion, 623 K – outlet composition and duty against the published Rout

The story: the FIRST PROCESS slice of the acetone plant

W. L. Luyben, "Design and Control of the Acetone Process via Dehydrogenation of 2-Propanol", Ind. Eng. Chem. Res. 50 (2011) 1206-1218, DOI 10.1021/ie901923a – the Turton base case of his Figure 1.

ONE UNIT, chosen because it is the piece of the flowsheet whose answer does NOT depend on the mixture thermodynamics. At 623 K and 2.6 atm everything is vapour; no activity model is consulted. What is exercised is stoichiometry, the elements-datum heat of reaction and the energy balance – and those can be held against published numbers without first solving the harder problem the separations pose (acetone01_ipa_water_azeotrope measures how hard).

THE FEED IS DERIVED FROM THE PAPER’S OWN STREAM TABLE, and the derivation is a check in itself. Luyben gives Rin = 57.83 kmol/h at 389 K, 2.6 atm and Rout = 92.6 kmol/h with H2 0.3755, acetone 0.3755, IPA 0.0417, water 0.2073. Water is inert and hydrogen is produced one-for-one with acetone, so:

H2 produced = 92.6 x 0.3755 = 34.77 kmol/h = IPA converted IPA out = 92.6 x 0.0417 = 3.86 kmol/h IPA in = 34.77 + 3.86 = 38.63 kmol/h water in = 92.6 x 0.2073 = 19.19 kmol/h (inert) total in = 38.63 + 19.19 = 57.82 kmol/h vs the stated 57.83 conversion = 34.77 / 38.63 = 90.0 % vs the stated 90 %

Two independent closures on a table transcribed by hand. The feed in 0/ is those numbers; the conversion spec is the paper’s 90 %.

WHAT IS BEING COMPARED (see ‘expected‘): the outlet composition, four mole fractions from Luyben’s Rout; the reactor DUTY against his 0.960 MW.

THE DUTY IS THE INTERESTING ONE. Choupo derives the heat of reaction from each species’ own formation enthalpy – it is never read from the reactions dict – and isopropanol’s dHf here is a JOBACK ESTIMATE, because Choupo has no curated isopropanol. So the published duty tests a chain that runs from a group-contribution estimate of one molecule all the way to a megawatt of furnace heat. That is the whole point of building this rather than asserting it: a case says where in the chain the error entered.

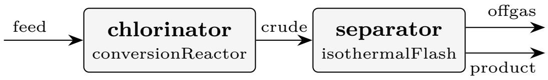
NOT MODELLED, and named rather than implied: the reactor’s SIZE. Luyben specifies 450 tubes with a 624 K molten-salt jacket, and the paper does not give the tube dimensions this record was built from, so a ‘pfr‘ on his kinetics cannot be dimensioned from what is in hand. A ‘conversionReactor‘ at his stated 90 % asks the thermodynamic question without pretending to the kinetic one. The kinetics (Luyben Table 1) are transcribed in the design record and remain unused here, on purpose.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	Rin	F	0.0160611111111
stream	Rout	F	0.0257186111111
kpi	reactor	F_in_kmol_h	57.82
kpi	reactor	F_out_kmol_h	92.587
kpi	reactor	Q_kW	840.263234652
kpi	reactor	T	623
kpi	reactor	conversion	0.9
kpi	reactor	dHrxn_kJ_per_mol	48.7173210118
kpi	reactor	extent_kmol_h	34.767
stream	Rout	isopropanol	0.0417229200644
stream	Rout	acetone	0.375506280579
stream	Rout	H2	0.375506280579
stream	Rout	water	0.207264518777
kpi	reactor	Q_reaction_kW	470.487527672
... and 3 more reference values in the case's `expected`.			

chlorophenol01_chlorination

tutorials/steady/reactors/chlorophenol01_chlorination



What it does: Chlorophenol: classic phenol chlorination (PhOH + Cl2 → 4-CP + HCl) + HCl flash

The story: the CLASSIC chlorophenol process.

Direct chlorination of phenol with chlorine gas – the industrial route to the chlorophenols (here the mono, 4-chlorophenol):

chlorinator : C6H5OH + Cl2 → C6H4ClOH + HCl (exothermic, ~-130 kJ/mol) separator : a flash at 80 C, 1 atm splits the volatile HCl (+ excess Cl2) off the liquid phenol / chlorophenol.

Feed: 1.1 kmol/h phenol + 1.0 kmol/h Cl2 (slight phenol excess → Cl2 limiting). phenol + 4-chlorophenol are JOBACK ESTIMATES (glass-box, unvalidated).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	crude	F	0.000583333333333
stream	feed	F	0.000583333333333
stream	offgas	F	0.000275331266235
stream	product	F	0.000308002067098
kpi	chlorinator	F_in_kmol_h	2.1
kpi	chlorinator	F_out_kmol_h	2.1
kpi	chlorinator	T	353
kpi	chlorinator	conversion	0.9
kpi	chlorinator	extent_kmol_h	0.9
kpi	separator	F_alpha	0.000308002067098
kpi	separator	F_beta	0.000275331266235
kpi	separator	F_in	0.000583333333333
kpi	separator	K_Cl2	26.0852015979
kpi	separator	K_HCl	122.317563764

... and 12 more reference values in the case's `expected`.

conversion01_hda

tutorials/steady/reactors/conversion01_hda

What it does: Conversion reactor: HDA toluene+H_2 $\rightarrow$ benzene+CH_4 at 75%/pass (gas-basis, no kinetics, no Vliq)

The story: A single gas-phase conversion reactor: toluene + H_2 $\rightarrow$ benzene + CH_4, 75 % of the limiting reactant (toluene) per pass. H_2 is fed in excess (3:1), as in the real HDA.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.111111111111
stream	product	F	0.111111111111
kpi	reactor	F_in_kmol_h	400
kpi	reactor	F_out_kmol_h	400
kpi	reactor	Q_kW	-1007.67392146
kpi	reactor	T	900
kpi	reactor	conversion	0.75
kpi	reactor	dHrxn_kJ_per_mol	-48.3683482303
kpi	reactor	extent_kmol_h	75
kpi	reactor	Q_reaction_kW	-1007.67392146
kpi	reactor	Q_sensible_kW	0
stream	feed	T	900
stream	product	T	900

conversion02_parallel_selectivity

tutorials/steady/reactors/conversion02_parallel_selectivity

What it does: Conversion reactor, PARALLEL network A→B / A→C: selectivity as a SPEC (75 %)

The story: a stoichiometric (conversion) reactor with a PARALLEL network, A → B / A → C.

Multi-reaction grammar ‘reactions (a_to_b a_to_c);‘. Each reaction’s extent is a SPEC, not a result: ‘conversions ({ reaction ...; conversion ...; } ...)‘ gives the fraction of the FEED A that each leg consumes. 60 % of A goes the wanted way and 20 % the side way, so:

total A conversion = 80 % selectivity B/(B+C) = 0.36 / 0.48 = 75 %

This is the honest reactor when you have neither rate data (cstr/pfr) nor an equilibrium (equilibriumReactor) – just a catalyst whose split you measured. A single-reaction conversionReactor cannot express a selectivity at all.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777777778
stream	product	F	0.000188888888888889
kpi	reactor	F_in_kmol_h	1
kpi	reactor	F_out_kmol_h	0.68
kpi	reactor	T	350
kpi	reactor	extent_a_to_b_kmol_h	0.12
kpi	reactor	extent_a_to_c_kmol_h	0.04
kpi	reactor	nReactions	2
stream	feed	T	350
stream	product	T	350

cstr01_first_order

tutorials/steady/reactors/cstr01_first_order

What it does: CSTR with pseudo-first-order Arrhenius esterification (AcOH + EtOH <→ EtOAc + H2O), ~53% conversion of ethanol

The story: CSTR single-reaction tutorial. The reaction kinetics live in constant/reactions; here the unit references it by name.

A 1 kmol/h equimolar ethanol / acetic-acid feed at 350 K enters a 5-litre CSTR. One reaction (Fischer esterification), pseudo-first-order in ethanol, Arrhenius kinetics from constant/reactions. The golden: X = 52.59 % conversion of ethanol, extent ? = 0.0730 mol/s, residence time τ = 310.3 s, rate constant k = 3.575e-3 s?^{1} at 350 K, and the reactor removing 1.88 kW to stay isothermal.

1. One equation, one unknown, and the unknown is the extent. The Log tab prints Newton in extent ? (?_max = 1.389e-01 mol/s) and a two-row table: from a guess of ?_max/2 the residual goes -7.6e-3 → -2.4e-13 in one step — a first-order rate law in a CSTR is linear in ?, so Newton lands exactly. 2. Damk?hler is the whole design in one number. Da = kτ = 1.109, and for first order X = Da/(1 + Da) = 0.526. Do that division by hand; it is the golden to three figures. 3. k comes from A and Ea, at this T. The reaction record says the pair is ILLUSTRATIVE. Read the Arrhenius line in the log (k_fwd: 3.5750e-03) and reproduce it from A exp(-Ea/RT). 4. Isothermal means somebody removes the heat. The reaction is exothermic; Q_kW = -1.88 is what the jacket must take away. cstr04_adiabatic removes nothing and lets T answer.

Double V_R to 0.010: τ doubles, Da doubles, and X moves to Da/(1+Da) — check it before you run. Then open pfr01_first_order: the same volume, the same kinetics, and a different answer.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	reactor	X_limiting	0.52590711876
kpi	reactor	T	350
kpi	reactor	k	0.00357499942014
stream	feed	F	0.000277777777778
stream	feed	T	350
stream	reactorOut	F	0.000277777777778
stream	reactorOut	T	350
kpi	reactor	Da_kTau	1.10929132154
kpi	reactor	F_in_kmol_h	1
kpi	reactor	F_out_kmol_h	1
kpi	reactor	Q_kW	-1.87948262728
kpi	reactor	V_R	0.005
kpi	reactor	minus_r_ethanol	14.6085310767
kpi	reactor	tau_s	310.291329081

. . . and 1 more reference values in the case's `expected`.

cstr02_reversible_wgs

tutorials/steady/reactors/cstr02_reversible_wgs

What it does: Reversible water-gas shift (CO + H2O <→ CO2 + H2) in a CSTR

The story: water-gas-shift reaction in a CSTR with reverse rate from thermodynamic K_eq.

Feed: equimolar CO + H2O at 600 K, 10 bar. The forward Arrhenius kinetics drives a high conversion in the early residence times; the reverse term (auto-derived from g_pure_ig of the four species at T = 600 K) caps the conversion at the Le Chatelier equilibrium.

At T = 600 K: dG_rxn = g_CO2 + g_H2 - g_CO - g_H2O ≈ -16.3 kJ/mol K_eq = exp(16300 / (R · 600)) ≈ 26.5 For equimolar CO + H2O, K_eq = X^2 / (1-X)^2 -→ X_eq ≈ 0.84

V_R is picked large enough (1e-3 m^3 for a 1 kmol/h feed) that the kinetics are fast relative to residence; we see the conversion approach (but not exceed) X_eq ≈ 0.84 — the thermodynamic ceiling.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	feed	vf	1
stream	products	F	0.000277777777778
stream	products	vf	1
kpi	wgs	Da_kTau	145.857299066
kpi	wgs	F_in_kmol_h	1
kpi	wgs	F_out_kmol_h	1
kpi	wgs	T	600
kpi	wgs	V_R	0.001
kpi	wgs	X_limiting	0.841431955275
kpi	wgs	k	1.08521882096
kpi	wgs	tau_s	134.403584096
kpi	wgs	xi_mol_per_s	0.116865549344
stream	feed	T	600

... and 3 more reference values in the case's `expected`.

cstr03_series_selectivity

tutorials/steady/reactors/cstr03_series_selectivity

What it does: CSTR with a SERIES network A→B→C: multi-reaction, the selectivity problem

The story: a CSTR with a SERIES reaction network, A → B → C, solved as ONE coupled system.

The multi-reaction grammar 'reactions (a_to_b b_to_c);' names both steps from constant/reactions; the reactor solves the R coupled design equations xi_j = r_j(xi) V_R together by the multivariate Newton. A single-reaction CSTR cannot express this at all: the intermediate B is BOTH a product and a reactant, so its yield peaks in residence time and SELECTIVITY becomes the design question.

Feed: pure A at 350 K. Try sweeping V_R (an outerDict sweep) and watch the B yield rise, peak, and fall as the over-reaction B → C takes over.

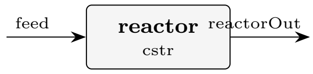
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	product	F	0.000199218293117
kpi	reactor	F_in_kmol_h	1
kpi	reactor	F_out_kmol_h	0.717185855222
kpi	reactor	T	350
kpi	reactor	V_R	0.005
kpi	reactor	k_a_to_b	0.00214522777035
kpi	reactor	k_b_to_c	0.000536306942589
kpi	reactor	nReactions	2
kpi	reactor	newtonResidual	6.93889390391e-18
kpi	reactor	tau_s	225
kpi	reactor	xi_a_to_b_mol_per_s	0.0392797423303
kpi	reactor	xi_b_to_c_mol_per_s	0.0126884461066
stream	feed	T	350

. . . and 2 more reference values in the case's `expected`.

cstr04_adiabatic

tutorials/steady/reactors/cstr04_adiabatic



What it does: ADIABATIC CSTR: T_out solved WITH the extents; Q=0 by construction (elements datum)

The story: an ADIABATIC continuous stirred-tank reactor.

The outlet temperature is an UNKNOWN, solved WITH the reaction extents:

xi_j = r_j(xi, T) V_R (R design equations) H_out(T, xi) = H_in (one enthalpy balance)

There is NO dH_rxn source term. With the enthalpy on the ELEMENTS datum, the heat of reaction already lives inside H: as reactants become products the formation enthalpies differ and T rises on its own. Q_kW = 0 by construction.

A non-isothermal CSTR can have up to THREE steady states (multiplicity). The Newton lands on ONE of them, selected by ‘T_guess‘ – sweep it to find the ignited and extinguished branches. That honesty is the point: the reactor reports which root it found, it does not pretend there is only one.

The cstr01 reactor with no heat removal. The exothermic esterification heats its own contents until the energy balance closes: the golden lands at T_out = 454.15 K, a 104.2 K rise from the 350 K feed, with the extent ? = 0.1384 mol/s — essentially complete conversion (?_max is 0.1389 mol/s) because the rate constant at 454 K is 0.889 s?^{1}, 250 times its value at 350 K.

1. Two unknowns now, solved together. The extent from the design equation and T from H_out(T, ?) = H_in: the dict says so in its own header, and Q_kW in the golden is 4.7e-12 — zero by construction, not by declaration. 2. An adiabatic CSTR can have more than one answer. Before solving, the engine ∗scans∗ the energy balance across T (the Log line [cstr] scanning the energy balance on T in [325.0, 479.5] K for steady states) and reports how many it found — 1 here. The scan is the van Heerden diagram; cstr05_multiplicity is the case with three. 3. T_guess is a declared aid, and the log says where it landed from. operation { T_guess 350; } picks the branch the Newton starts on; the result line prints landed from T_guess = 350.00 K. With one steady state it cannot matter; with three it decides the answer, which is why it is in the dict and not hidden. 4. Read the two caveats the run prints. Acetic acid’s Cp polynomial is declared to 390 K and this reactor asks it at 450 K: the enthalpy path is ∗extrapolated∗, announced, and still returned. The 454 K answer stands with that stated — as with adiabaticFlash01, a warned number is a defensible one. 5. "Newton 882 it" is the scan’s total, not the answer’s. The log now says so on the line. Every trial temperature of the scan re-enters the extent Newton; the final solve took a handful.

Set T_guess 460; — same single answer, from the other side. Then run cstr06_jacketed, where a coolant stream carries the heat away and the outlet T sits between the two extremes you have now seen.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	reactorOut	F	0.000277777777778
kpi	reactor	F_in_kmol_h	1
kpi	reactor	F_out_kmol_h	1
kpi	reactor	Q_kW	4.70379291073e-12
kpi	reactor	T	454.152948902
kpi	reactor	T_guess	350
kpi	reactor	T_out	454.152948902
kpi	reactor	V_R	0.005
kpi	reactor	dT_rise	104.152948902
kpi	reactor	k_esterification_etac	0.88934531923
kpi	reactor	nReactions	1
kpi	reactor	newtonResidual	3.5527136788e-15
kpi	reactor	tau_s	310.291329081

. . . and 4 more reference values in the case’s `expected`*.*

cstr05_multiplicity

tutorials/steady/reactors/cstr05_multiplicity

What it does: CSTR multiplicity: three steady states – the reactor prints them all, T_guess picks the branch

The story: an ADIABATIC CSTR with THREE steady states.

Eliminate the extents (at a fixed T the design equations are monotone) and what remains is ONE equation in the temperature:

phi(T) = H_out(T) - H_in - Q_ext(T) = 0

the textbook "heat generated equals heat removed". The generation curve G(T) is a sigmoid (the rate saturates once the reactant is gone); the removal line R(T) = F cp (T - T_in) is straight (adiabatic: the only sink is the sensible heat that leaves with the product). A straight line can cut a sigmoid THREE times: the extinguished state, an unstable middle state, and the ignited state.

The feed enters COLD (300 K), where the rate is negligible. Nothing forces this reactor to ignite – and nothing forces it to stay cold either.

Choupo scans, finds them all, prints them, and reports the one nearest ‘T_guess‘. Change T_guess and the reactor moves to another branch – the same equipment, the same feed, a different answer. That is not a numerical defect; it is the physics, and it is why real reactors need a start-up procedure.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	reactorOut	F	0.000277777777778
kpi	reactor	F_in_kmol_h	1
kpi	reactor	F_out_kmol_h	1
kpi	reactor	Q_kW	8.68283223099e-12
kpi	reactor	T	405.903598573
kpi	reactor	T_guess	380
kpi	reactor	T_out	405.903598573
kpi	reactor	V_R	0.005
kpi	reactor	dT_rise	105.903598573
kpi	reactor	k_esterification_etac	0.406108484365
kpi	reactor	nReactions	1
kpi	reactor	newtonResidual	8.32667268469e-16
kpi	reactor	steadyStates	3

. . . and 4 more reference values in the case’s `expected`.

cstr06_jacketed

`tutorials/steady/reactors/cstr06_jacketed`

What it does: JACKETED CSTR: the coolant carries the reaction heat away – Q = UA (T_c - T_out)

The story: the same esterification as cstr04, now with a COOLING JACKET instead of an adiabatic wall.

The energy balance gains one term, and only one:

H_out(T) = H_in + UA (T_coolant - T_out)

There is still NO dH_rxn source: on the elements datum the heat of reaction is already inside H. What the jacket does is remove some of it, so the outlet settles BELOW the adiabatic 454.2 K of cstr04.

Check it by hand from the printed KPIs: Q_kW should equal UA (T_c - T_out), and it is NEGATIVE – heat leaves the reactor. Cool harder (raise UA, lower T_coolant) and the outlet slides down the generation curve; cool too hard and the reactor extinguishes. Nothing here is switched on by a flag; it is the same one equation in T, with one more term.

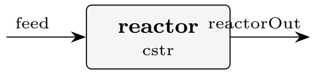
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`reactorOut`	`F`	`0.000277777777778`
`kpi`	`reactor`	`F_in_kmol_h`	`1`
`kpi`	`reactor`	`F_out_kmol_h`	`1`
`kpi`	`reactor`	`Q_kW`	`-1.85832125175`
`kpi`	`reactor`	`T`	`394.33285007`
`kpi`	`reactor`	`T_guess`	`400`
`kpi`	`reactor`	`T_out`	`394.33285007`
`kpi`	`reactor`	`V_R`	`0.005`
`kpi`	`reactor`	`dT_rise`	`44.3328500695`
`kpi`	`reactor`	`k_esterification_etac`	`0.0534254733127`
`kpi`	`reactor`	`nReactions`	`1`
`kpi`	`reactor`	`newtonResidual`	`1.38777878078e-16`
`kpi`	`reactor`	`steadyStates`	`1`

. . . and 4 more reference values in the case's `expected`*.*

cstr07_lhhw_methylAcetate

tutorials/steady/reactors/cstr07_lhhw_methylAcetate



What it does: LHHW on activities: methyl-acetate synthesis over Amberlyst 15 (Popken 2000, Eq 16)

The story: an isothermal CSTR packed with Amberlyst 15.

The rate constants are reported per GRAM of dry catalyst, so the reactor needs to know how much catalyst a cubic metre of it holds: ‘catalystLoading‘. It is a declared, visible number, not a fudge – change the packing and the reactor changes with it.

Run it and read the KPI ‘k_esterification_meoac‘ against the conversion: the forward rate constant says one thing, the adsorption denominator says another, and the outlet is where they settle.

This reactor is KINETICALLY limited, which is the point – if it sat at equilibrium the denominator would be invisible. Sweep V_R and watch it climb to the activity-based equilibrium and stop:

V_R = 0.005 m3 X = 49.02 % <- this case V_R = 0.020 m3 X = 63.74 % V_R = 0.100 m3 X = 72.92 % V_R = 1.000 m3 X = 76.18 % <- equilibrium

The plateau is not asserted anywhere in the dict. It is where k1 a’_HOAc a’_MeOH = k-1 a’_MeOAc a’_H2O, i.e. exactly the K_a = 25.50 that the authors’ own Eq 17 recovers from the same four rate constants and four adsorption constants at 333.15 K. Two independent equations of the paper agreeing is a cheap and honest check that the parameters were entered right.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	reactorOut	F	0.000277777777778
kpi	reactor	F_in_kmol_h	1
kpi	reactor	F_out_kmol_h	1
kpi	reactor	Q_kW	-2.09513270004
kpi	reactor	T	333.15
kpi	reactor	T_out	333.15
kpi	reactor	V_R	0.005
kpi	reactor	k_esterification_meoac	0.00280769603355
kpi	reactor	nReactions	1
kpi	reactor	newtonResidual	1.2490009027e-16
kpi	reactor	tau_s	366.524129505
kpi	reactor	xi_esterification_meoac_mol_per_s	0.0680930737737
stream	feed	T	333.15
... and 1 more reference values in the case’s `expected`*.*			

equil01_reforming

tutorials/steady/reactors/equil01_reforming

What it does: Equilibrium reactor (REquil): SMR + WGS to SIMULTANEOUS equilibrium at 1100 K

The story: the EQUILIBRIUM reactor (REquil) with MULTIPLE simultaneous reactions.

Steam-methane reforming: two independent reactions are driven to SIMULTANEOUS chemical equilibrium at 1100 K, 1 bar – no kinetics, no residence time. The two extents are a RESULT of each reaction's Kp(T) (from the standardThermochemistry data), solved together by the multivariate Newton:

smr : $CH_4 + H_2O <\rightarrow CO + 3\ H_2$ (strongly endothermic) wgs : $CO + H_2O <\rightarrow CO_2 + H_2$ (mildly exothermic)

The stoichiometry lives in constant/reactions and is referenced BY NAME – the same 'reactions (r1 r2);' grammar the batch / dynamic reactors take.

Feed: 1 kmol/h CH_4 + 3 kmol/h steam (steam-to-carbon = 3). At 1100 K, 1 bar the reforming is near-complete (Le Chatelier favours the mole-increasing SMR at low pressure); the shift then sets the CO/CO_2 split. This is the classic syngas problem – the reactor a 'conversionReactor' cannot do (it needs the extents GIVEN) and a 'cstr'/'pfr' cannot do without rate data.

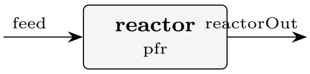
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.00111111111111
stream	syngas	F	0.00166593958576
kpi	reformer	F_in_kmol_h	4
kpi	reformer	F_out_kmol_h	5.99738250873
kpi	reformer	Kp_smr	312.740523794
kpi	reformer	Kp_wgs	0.978425617548
kpi	reformer	Q_kW	59.6424648629
kpi	reformer	T	1100
kpi	reformer	conversion_smr	0.998691254363
kpi	reformer	conversion_wgs	0.109768673641
kpi	reformer	extent_smr_kmol_h	0.998691254363
kpi	reformer	extent_wgs_kmol_h	0.329306020923
kpi	reformer	nReactions	2
kpi	reformer	newtonResidual	3.2596148003e-13

... and 3 more reference values in the case's `expected`.

pfr01_first_order

tutorials/steady/reactors/pfr01_first_order



What it does: PFR with the same kinetics as cstr01 (RK4 integration)

The story: PFR with the same kinetics and conditions as cstr01. For a first-order reaction the PFR gives X = 1 - exp(-kτ). With kτ = 1.10: X_PFR = 1 - exp(-1.10) = 0.667 (vs CSTR X = 0.524)

The reactor of cstr01 rebuilt as a plug-flow tube: the same 5 litres, the same 1 kmol/h feed at 350 K, the same Arrhenius pair. The golden: X = 67.02 % against the CSTR's 52.59 %, at the same τ = 310.3 s and the same Da = 1.109.

1. No back-mixing is the whole difference. In the CSTR every molecule reacts at the ∗outlet∗ concentration, the lowest in the vessel. In the tube the concentration falls along V, so most of the volume works at a higher rate. For first order: CSTR X = Da/(1+Da) = 0.526; PFR X = 1 - e^(-Da) = 0.670. Both from the same Da; both in the goldens. 2. The profile is printed, not just the endpoint. nSteps 100; writeInterval 10; in operation {} — the Log tab shows the axial table, eleven rows from V = 0 to 5e-3 m^3, each component's molar flow falling or rising along the tube. The integrator is named (RK4, 100 uniform steps, dV = 5.0e-05 m^3); halve nSteps and see what moves. 3. The heat of reaction is a KPI here. dHrxn_kJ_per_mol = -25.73 is computed from the species' formation enthalpies (the elements datum — the one base every reactor in Choupo prices on), and the tube's isothermal duty Q_kW = -2.40 follows from it and the extent. 4. Same Da, different X, is the pedagogical point of the pair. Put the two READMEs side by side; the numbers are the lecture.

Set nSteps 10 and rerun: X barely moves, because RK4 is fourth order and the profile is smooth. Then try pfr04_adiabatic, where the tube heats itself and T(V) becomes part of the profile.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	reactorOut	F	0.000277777777778
stream	feed	T	350
stream	reactorOut	T	350
kpi	reactor	Da_kTau	1.10929132154
kpi	reactor	F_out_kmol_h	1
kpi	reactor	Q_kW	-2.39518182773
kpi	reactor	T	350
kpi	reactor	V_R	0.005
kpi	reactor	X_limiting	0.670207404765
kpi	reactor	dHrxn_kJ_per_mol	-25.7313020373
kpi	reactor	tau_s	310.291329081

`pfr02_reversible_wgs`

`tutorials/steady/reactors/pfr02_reversible_wgs`

What it does: Reversible water-gas shift (CO + H2O <→ CO2 + H2) in a PFR; approaches the same X_eq as the CSTR

The story: the reversible water-gas shift in a PLUG-FLOW reactor, the PFR sibling of cstr02_reversible_wgs.

Same reaction, same feed (equimolar CO + H2O at 600 K, 10 bar), same reverse term auto-derived from g_pure_ig. The pedagogical point: a reversible reaction relaxes toward the THERMODYNAMIC equilibrium conversion regardless of reactor type — with enough volume the PFR approaches the very same ceiling the CSTR hit,

K_eq(600 K) ~ 26.5 → X_eq ~ 0.84 (equimolar feed),

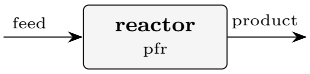
not the X = 1 a forward-only PFR would chase. V_R is taken large enough that the axial profile flattens onto X_eq well before the exit.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`feed`	`vf`	`1`
`stream`	`products`	`F`	`0.000277777777778`
`stream`	`products`	`vf`	`1`
`stream`	`feed`	`T`	`600`
`stream`	`products`	`T`	`600`
`kpi`	`wgs`	`Da_kTau`	`0.00175028758879`
`kpi`	`wgs`	`F_out_kmol_h`	`1`
`kpi`	`wgs`	`Q_kW`	`-4.55086758232`
`kpi`	`wgs`	`T`	`600`
`kpi`	`wgs`	`V_R`	`0.002`
`kpi`	`wgs`	`X_limiting`	`0.841431577908`
`kpi`	`wgs`	`dHrxn_kJ_per_mol`	`-38.9410707335`
`kpi`	`wgs`	`tau_s`	`268.807168191`

`pfr03_series_selectivity`

`tutorials/steady/reactors/pfr03_series_selectivity`



What it does: PFR with the SAME series network $A \to B \to C$: no back-mixing $\to$ MORE of the intermediate B

The story: the SAME series network $A \to B \to C$ as cstr03, now in a PFR. Compare them.

Multi-reaction grammar 'reactions (a__to__b b__to__c);'; the PFR marches dF__i/dV = SUM__j nu__ij r__j(F) by RK4 – an IVP, no Newton. At the SAME residence time (tau = 225 s):

species CSTR PFR compA (feed) 40.8 % 14.1 % compB (want) 52.8 % 79.2 % <– the point compC (over) 6.4 % 6.8 %

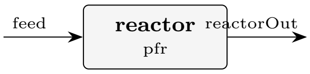
The PFR converts far more A AND keeps far more of the intermediate B: with no back-mixing, the freshly made B is not held in a vessel at low A concentration where the over-reaction $B \to C$ dominates. For positive-order kinetics the PFR beats the CSTR – the classic result a single-reaction reactor cannot show.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	0.000277777777778
`stream`	`product`	`F`	0.000176612457804
`kpi`	`reactor`	`F_in_kmol_h`	1
`kpi`	`reactor`	`F_out_kmol_h`	0.635804848095
`kpi`	`reactor`	`T`	350
`kpi`	`reactor`	`V_R`	0.005
`kpi`	`reactor`	`k_a_to_b`	0.00214522777035
`kpi`	`reactor`	`k_b_to_c`	0.000536306942589
`kpi`	`reactor`	`nReactions`	2
`kpi`	`reactor`	`nSteps`	200
`kpi`	`reactor`	`tau_s`	225
`stream`	`feed`	`T`	350
`stream`	`product`	`T`	350
`kpi`	`reactor`	`T_out`	350

pfr04_adiabatic

`tutorials/steady/reactors/pfr04_adiabatic`



What it does: ADIABATIC PFR: the exothermic esterification heats its own stream – T(V) is a RESULT

The story: an ADIABATIC plug-flow reactor. The temperature is a RESULT, marched with the species by the same RK4:

dF_i/dV = SUM_j nu_ij r_j(F,T) dT/dV = SUM_j (-dH_j(T)) r_j / SUM_i F_i cp_i(T)

The exothermic esterification heats its own stream, which accelerates the Arrhenius rate, which releases more heat – the positive feedback every reactor engineer must see. T(V) is in the axial profile: plot it.

'thermalMode' takes 'isothermal' (default – T imposed, duty a result), 'adiabatic' (here), or 'heatExchange' (a jacket: U, areaPerVolume, T_coolant). A non-isothermal reactor needs the heat of reaction, so every reacting species must carry 'standardThermochemistry' – a fictitious toy species cannot be used.

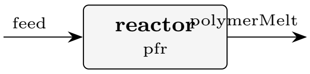
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.000277777777778`
`stream`	`reactorOut`	`F`	`0.000277777777778`
`kpi`	`reactor`	`F_in_kmol_h`	`1`
`kpi`	`reactor`	`F_out_kmol_h`	`1`
`kpi`	`reactor`	`Q_kW`	`-1.42108547152e-14`
`kpi`	`reactor`	`T`	`350`
`kpi`	`reactor`	`T_max`	`454.513818186`
`kpi`	`reactor`	`T_min`	`350`
`kpi`	`reactor`	`T_out`	`454.513818186`
`kpi`	`reactor`	`V_R`	`0.005`
`kpi`	`reactor`	`dT_rise`	`104.513818186`
`kpi`	`reactor`	`hotSpot_dT`	`104.513818186`
`kpi`	`reactor`	`k_esterification_etac`	`0.00357499942014`
`kpi`	`reactor`	`nReactions`	`1`

. . . and 4 more reference values in the case's `expected`*.*

pfr_polyesterification

tutorials/steady/reactors/pfr_polyesterification



What it does: PFR step-growth polyesterification — Carothers/Flory chain statistics

The story: Step-growth polyesterification in a PFR. A molten AB-type hydroxy-acid monomer self-condenses to polycaprolactone, releasing water. As the monomer conversion p rises along the reactor, the Carothers relation Xn = 1/(1-p) makes the average chain length climb steeply, and the polydispersity PDI = 1+p approaches 2 (the most-probable distribution).

Try a 'sweep' over operation.V_R and watch Xn, Mn and PDI climb and the Flory-Schulz distribution broaden – the whole story in one glass-box run.

A molten AB-type hydroxy-acid monomer (6-hydroxyhexanoic acid) self- condenses to polycaprolactone in a plug-flow reactor, releasing one water per ester bond:

As the monomer conversion p climbs along the reactor, the Carothers relation makes the average chain length rise steeply and the polydispersity approach 2 — the most-probable (Flory–Schulz) distribution.

and emits the Flory–Schulz distribution as a profile (chainLength vs n_x, w_x) so the GUI plots it: n(x)=(1-p)p^(x-1) decays monotonically, w(x)=x(1-p)^{2}p^(x-1) peaks near x≈Xn.

Run a sweep over operation.V_R (reactor volume → residence time → p) and watch PDI climb toward 2, Xn and Mn shoot up, and the weight distribution broaden. The whole Carothers/Flory story in one glass-box run.

Lives inside the reaction. Absent ? the reactor behaves as an ordinary PFR. p is taken from the reactor's own conversion of limitingReactant (single source of truth). The distribution + PDI = 1+p are valid because a PFR has a uniform residence time; on a CSTR the residence-time distribution broadens both, so the distribution is gated to PFR/batch only.

- The chain-statistics math is exact and standard: P.J. Flory, ∗Principles of Polymer Chemistry∗, Cornell Univ. Press (1953), Ch. VIII–IX (the most-probable distribution and PDI = 1+p); G. Odian, ∗Principles of Polymerization∗, 4th ed., Ch. 2; H.S. Fogler, ∗Elements of Chemical Reaction Engineering∗, 5th ed., ?9. - The monomer/repeat component properties are ILLUSTRATIVE (clearly labelled in each .dat) — plausible round numbers for a teaching case, not curated literature values. The Arrhenius rate is illustrative, tuned so Da = kτ ≈ 4 gives p ≈ 0.98. Curate these before any quantitative use.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	polymerMelt	F	0.000550395491925
kpi	reactor	Da_kTau	3.98587252785
kpi	reactor	F_out_kmol_h	1.98142377093
kpi	reactor	Mn_kg_kmol	6144.41174045
kpi	reactor	Mw_kg_kmol	12174.6834809
kpi	reactor	PDI	1.98142377093
kpi	reactor	T	450
kpi	reactor	V_R	0.2
kpi	reactor	X_limiting	0.981423770929
kpi	reactor	Xn	53.8322388335
kpi	reactor	Xw	106.664477667
kpi	reactor	p_conversion	0.981423770929
kpi	reactor	tau_s	5829.95951417

. . . and 4 more reference values in the case's `expected`.

rotating

The theory you need. Compressors, turbines, pumps: specified by shaft work W_s and isentropic efficiency η_s; the outlet state follows from $h_{\text{out}} = h_{\text{in}} + W_s/\dot{n}$ with the isentropic reference path. Outlet pressure is a RESULT (hardware, not a spec).

compressor01_air

tutorials/steady/rotating/compressor01_air

What it does: Adiabatic air compressor: 10 kW shaft, eta 0.75 — P_out and T_out are RESULTS

The story: The simplest credo-pure rotating-equipment case. An adiabatic compressor takes ambient air (1 bar, 25 degC) and a fixed shaft power of 10 kW with isentropic efficiency 0.75.

Hardware (the only things you specify): W_shaft = 10 kW motor rating delivered to the gas eta = 0.75 isentropic efficiency (machine characteristic)

RESULTS the simulator computes: P_out the discharge pressure the machine reaches T_out the discharge temperature (real, with losses) ratio P_out / P_in T_out_isen isentropic discharge T (diagnostic) dS_gen entropy generated by the irreversibility

Hand check (ideal gas, Cp ~ 29.2 J/(mol*K), gamma ~ 1.40): F = 3.6 kmol/h = 1 mol/s w_real = 10000 W / 1 mol/s = 10000 J/mol w_isen = 0.75 * 10000 = 7500 J/mol T_out_isen = 298.15 + 7500/29.2 ~ 555 K ratio = (T_isen/T_in)^(gamma/(gamma-1)) = (555/298)^3.5 ~ 8.8 T_out = 298.15 + 10000/29.2 ~ 640 K

"I need exactly 8 bar at discharge" is a DESIGN question — it goes in a DesignSpec that manipulates $W_shaft with target compressor.P_out.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.001
stream	out	F	0.001
kpi	compressor01	F	0.001
kpi	compressor01	P_in	100000
kpi	compressor01	P_out	890211.642995
kpi	compressor01	T_in	298.15
kpi	compressor01	T_out	633.374206884
kpi	compressor01	T_out_isen	551.37943898
kpi	compressor01	W_isen	7500
kpi	compressor01	W_isen_kW	7.5
kpi	compressor01	W_shaft	10000
kpi	compressor01	W_shaft_kW	10
kpi	compressor01	dS_gen	4.22615345389
kpi	compressor01	dT	335.224206884

... and 9 more reference values in the case's `expected`.

compressor02_designspec_pout

tutorials/steady/rotating/compressor02_designspec_pout

What it does: Air compressor sized to hit P_out = 8 bar exactly, via DesignSpec on $W_shaft

The story: Same air compressor as compressor01, but now answering the DESIGN question: "what shaft power do I need to discharge at exactly 8 bar?"

The credo forbids specifying P_out on the unit op (it's a result). Instead the outer DesignSpec manipulates the hardware variable $W_shaft until the computed compressor.P_out hits the 8-bar target. This is the rotating-equipment analogue of sizing an evaporator's area to hit a product concentration.

Converges to W_shaft ~ 9.36 kW (slightly below compressor01's 10 kW, which over-shot to 8.90 bar). At the solution T_out ~ 612 K (339 degC) and W_isen ~ 7.02 kW (eta ∗ W_shaft, eta = 0.75).

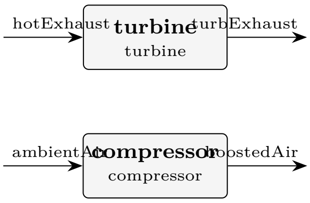
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.001
stream	out	F	0.001
kpi	compressor	F	0.001
kpi	compressor	P_in	100000
kpi	compressor	P_out	800004.793643
kpi	compressor	T_in	298.15
kpi	compressor	T_out	612.41049696
kpi	compressor	T_out_isen	535.384868081
kpi	compressor	W_isen	7017.30522226
kpi	compressor	W_isen_kW	7.01730522226
kpi	compressor	W_shaft	9356.40696301
kpi	compressor	W_shaft_kW	9.35640696301
kpi	compressor	dS_gen	4.08116534481
kpi	compressor	dT	314.26049696

. . . and 9 more reference values in the case's `expected`.

`compressorTurbine01_turbo_booster`

`tutorials/steady/rotating/compressorTurbine01_turbo_booster`



What it does: Turbo-booster: turbine shaft drives a compressor (energy wire)

The story: turbo-booster: turbine drives compressor through an ENERGY WIRE (v0.80)

Two physically independent gas streams, one shared shaft:

hotExhaust → turbine —-shaft—-+

ambientAir → compressor <———+

Hardware specs (the only knobs): turbine W_shaft = -5 kW (NEGATIVE: work extracted) eta = 0.80 compressor W_shaft is OMITTED (the wire provides it) eta = 0.75

The wire injects +W_turb into the compressor before its solve(), so the compressor's W_shaft is whatever the turbine produces — here, 5 kW (sign flipped: -W_shaft on the producer's expression → the consumer reads +W_shaft, the usual sign convention).

Hand check (ideal gas, Cp ~ 29 J/(mol*K), gamma ~ 1.40): Turbine: 1 mol/s, 800 K, 5 bar → extract 5 kW (= 5000 J/mol) w_isen = w_real / eta = 5000 / 0.80 = 6250 J/mol T_isen = 800 - 6250/29 ~ 585 K → ratio = (585/800)^3.5 ~ 0.35 P_out ~ 5 * 0.35 ~ 1.74 bar T_out (real) = 800 - 5000/29 ~ 627 K (warmer than T_isen: the losses stay as heat in the gas)

Compressor: 0.5 mol/s, 298 K, 1 bar, receiving 5 kW w_real = 5000 / 0.5 = 10000 J/mol w_isen = 0.75 * 10000 = 7500 J/mol T_isen = 298 + 7500/29 ~ 557 K → ratio = (557/298)^3.5 ~ 8.7 P_out ~ 8.7 bar T_out (real) = 298 + 10000/29 ~ 643 K

The wire makes the energy flow visible at the case level — a single edge in the flowsheet that the GUI can render dashed.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	ambientAir	F	0.0005
stream	boostedAir	F	0.0005
stream	hotExhaust	F	0.001
stream	turbExhaust	F	0.001
kpi	compressor	F	0.0005
kpi	compressor	P_in	100000
kpi	compressor	P_out	890211.642995
kpi	compressor	T_in	298.15
kpi	compressor	T_out	633.374206884
kpi	compressor	T_out_isen	551.37943898
kpi	compressor	W_isen	3750
kpi	compressor	W_isen_kW	3.75
kpi	compressor	W_shaft	5000
kpi	compressor	W_shaft_kW	5

. . . and 24 more reference values in the case's `expected`*.*

pump01_water

tutorials/steady/rotating/pump01_water

What it does: Water pump: 2 kW shaft, eta 0.65 — incompressible, P_out / T_out are RESULTS

The story: An incompressible-liquid pump. Liquid water enters at 1 bar, 298 K; the pump shaft delivers 2 kW with efficiency 0.65.

Hardware (the only OP parameters): W_shaft = 2 kW motor power delivered to the liquid eta = 0.65 pump efficiency

Because the liquid is incompressible (v ~ const), the discharge is closed-form — no EoS, no Newton: w_real = W_shaft / F_mol_s w_isen = eta ∗ w_real (the useful v∗dP part) dP = w_isen / v_molar P_out = P_in + dP (RESULT) dT = (1 - eta) ∗ w_real / Cp_liq (dissipation)

Hand check (water: v ~ 1.8e-5 m3/mol, Cp_liq ~ 75 J/(mol∗K)): F = 36 kmol/h = 10 mol/s w_real = 2000 W / 10 mol/s = 200 J/mol w_isen = 0.65 ∗ 200 = 130 J/mol dP = 130 / 1.8e-5 ~ 7.2e6 Pa = 72 bar... (so P_out ~ 73 bar) dT = 0.35 ∗ 200 / 75 ~ 0.9 K (tiny — pumps barely heat)

"Pump to exactly 50 bar" is a DesignSpec on $W_shaft targeting pump.P_out (converges in one step — the relation is linear).

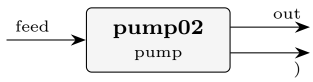
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.01
stream	out	F	0.01
kpi	pump01	F	0.01
kpi	pump01	P_in	100000
kpi	pump01	P_out	7294244.60432
kpi	pump01	T_in	298.15
kpi	pump01	T_out	299.077152318
kpi	pump01	W_hydraulic	1300
kpi	pump01	W_shaft	2000
kpi	pump01	W_shaft_kW	2
kpi	pump01	dP	7194244.60432
kpi	pump01	dT	0.927152317881
kpi	pump01	eta	0.65
kpi	pump01	head_m	735.848502288

. . . and 7 more reference values in the case's `expected`*.*

pump02_pressure_spec

tutorials/steady/rotating/pump02_pressure_spec



What it does: Water pump specified by DISCHARGE PRESSURE (P_out 30 bar) — the shaft power W_shaft is the RESULT (closed-form inverse of pump01)

The story: The SAME incompressible-liquid pump as pump01, specified the OTHER way: by the discharge pressure it must reach. In a real plant you rarely know a pump's shaft power up front — you know the HEADER it feeds (here 30 bar). So you state P_out and the shaft power is the RESULT.

This is NOT a DesignSpec. For an incompressible liquid the pump relation is invertible in CLOSED FORM,

ΔP = eta · W_shaft / Q_vol <→ W_shaft = Q_vol · ΔP / eta ,

with Q_vol = F · v set by the feed. Give P_out, get W_shaft — one shot, no Newton, no outer loop. (pump01 gives W_shaft and gets P_out; the two cases are exact inverses of one another.)

Specify EXACTLY ONE of { W_shaft | P_out | dP }. A pressure let-down (P_out < P_in) is not a pump — use a 'valve'.

Hand check (water: v ~ 1.8e-5 m3/mol): F = 36 kmol/h = 10 mol/s Q_vol = 10 ∗ 1.8e-5 = 1.8e-4 m3/s dP = 30 - 1 bar = 2.9e6 Pa W_shaft = Q_vol ∗ dP / eta = 1.8e-4 ∗ 2.9e6 / 0.65 ~ 803 W ~ 0.80 kW

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.01
stream	out	F	0.01
kpi	pump02	F	0.01
kpi	pump02	P_in	100000
kpi	pump02	P_out	3000000
kpi	pump02	T_in	298.15
kpi	pump02	T_out	298.523735099
kpi	pump02	W_hydraulic	524.03
kpi	pump02	W_shaft	806.2
kpi	pump02	W_shaft_kW	0.8062
kpi	pump02	dP	2900000
kpi	pump02	dT	0.373735099338
kpi	pump02	eta	0.65
kpi	pump02	head_m	296.620531272

. . . and 7 more reference values in the case's `expected`*.*

turbine01_air

`tutorials/steady/rotating/turbine01_air`

What it does: Adiabatic gas turbine: extract 10 kW from hot air, eta 0.85 — P_out / T_out are RESULTS

The story: The mirror of compressor01. Hot compressed air (10 bar, 900 K) drives an adiabatic turbine; the shaft extracts 10 kW with isentropic efficiency 0.85.

Hardware (the only things you specify): W__shaft = -10 kW shaft power extracted (NEGATIVE = out) eta = 0.85 isentropic efficiency

Sizing note: at 1 mol/s and 900 K inlet, the enthalpy available down to ambient bounds the extractable work to ~15 kW. The 10 kW asked for here expands the gas to ~1.32 bar (ratio ~7.6); asking for much more (e.g. 30 kW) would drive the isentropic discharge below 0 K — physically impossible, and the solver says so.

RESULTS the simulator computes (this run): P__out ~1.32 bar the discharge pressure the gas expands to T__out ~583 K the (real) exhaust temperature T__out__isen ~524 K the ideal (isentropic) discharge T ratio ~7.59 P__in / P__out (expansion ratio) W__generated +10 kW what the shaft delivers to the grid

Per the credo, P_out is a RESULT. If the turbine must exhaust at a fixed back-pressure (a condenser, the atmosphere), wrap the case in a DesignSpec that manipulates $W_shaft until turbine.P_out hits it.

What to expect (golden-locked):

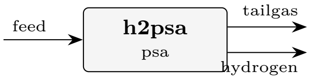
kind	unit/stream	key	expected
stream	exhaust	F	0.001
stream	hotAir	F	0.001
kpi	turbine01	F	0.001
kpi	turbine01	P_in	1000000
kpi	turbine01	P_out	131729.126973
kpi	turbine01	T_in	900
kpi	turbine01	T_out	582.935206477
kpi	turbine01	T_out_isen	524.463853734
kpi	turbine01	W_gen_kW	10
kpi	turbine01	W_generated	10000
kpi	turbine01	W_shaft	-10000
kpi	turbine01	dS_gen	3.19034018436
kpi	turbine01	dT	-317.064793523
kpi	turbine01	dT_isen	-375.536146266

. . . and 5 more reference values in the case's `expected`.

separation

`psa01_h2_psa`

`tutorials/steady/separation/psa01_h2_psa`



What it does: H2-PSA: 20-bar syngas (H2/CO2/CH4/CO/N2) over zeolite 5A → high-purity H2 raffinate + CO2-rich tail gas (equilibrium cyclic-steady-state shortcut)

The story: the canonical first PSA case: hydrogen recovery from steam-methane-reformer / shifted syngas.

Feed: a typical PSA feed gas (mol fractions) H2 0.70 (the LIGHT product → raffinate) CO2 0.18 (the strongest adsorbate → tail gas) CH4 0.05 CO 0.04 N2 0.03 F = 1000 kmol/h, 20 bar, 313 K.

Adsorbent: zeolite 5A (Ca-LTA), the classic H2-PSA sieve. CO2 » CH4 > CO ~ N2 » H2 on the competitive (extended) Langmuir isotherm, so the heavy impurities load onto the bed at P_high (20 bar) and are blown down with the tail at P_low (1.2 bar), while H2 barely adsorbs and sweeps through as the high-pressure raffinate product.

Operation: P_high 20 bar / P_low 1.2 bar swing; isothermal 313 K bed; purgeRatio 0.15 (a Skarstrom-style countercurrent light-product sweep that cleans the regeneration at the cost of product); eta 0.80 (the equilibrium-utilisation / LUB derating — the equilibrium ceiling is eta = 1); bedCapacity 2.0 kg adsorbent per mol of feed throughput (the lumped throughput knob; cycle time / bed volume are deliberately collapsed into it by the equilibrium shortcut).

Expected (this model, golden-checked): H2 recovery ~ 74 % (single train, with the 0.15 purge sweep) H2 purity ~ 98-99 % in the raffinate tail gas CO2-enriched ~ 18 % → ~ 38 %

The run header prints, out loud, what the equilibrium shortcut does NOT model (breakthrough / cycle time, thermal swing, blowdown gas loss, hydraulics, isotherm-validity window) — "all models are wrong, some useful".

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.277777777778`
`stream`	`hydrogen`	`F`	`0.145276064457`
`stream`	`tailgas`	`F`	`0.13250171332`
`kpi`	`h2psa`	`Dq_CH4`	`0.0260273101613`
`kpi`	`h2psa`	`Dq_CO`	`0.0367785976251`
`kpi`	`h2psa`	`Dq_CO2`	`1.74280135323`
`kpi`	`h2psa`	`Dq_H2`	`0.0186248522203`
`kpi`	`h2psa`	`Dq_N2`	`0.00987823175641`
`kpi`	`h2psa`	`eta_used`	`0.8`
`kpi`	`h2psa`	`pressureRatio`	`16.6666666667`
`kpi`	`h2psa`	`productivity`	`0.0715324198173`
`kpi`	`h2psa`	`purgeRatio`	`0.15`
`kpi`	`h2psa`	`purity_H2`	`0.984779152498`
`kpi`	`h2psa`	`recovery_H2`	`0.735762032407`

. . . and 8 more reference values in the case's `expected`.

tsa01_co2_twin_bed

tutorials/steady/separation/tsa01_co2_twin_bed

What it does: Twin-bed TSA shortcut: continuous CO2 removal from helium by alternating adsorption at 298 K and regeneration at 398 K

The story: cycle-averaged twin-bed thermal swing.

One column adsorbs while the other regenerates. tCycle is the switch interval and therefore the adsorption duration of one column. The unit refuses a bed that cannot hold every adsorbate mole arriving in that time.

This teaching case uses a CASE-LOCAL synthetic property anchor: cpSolid = 920 J/(kg.K), explicitly assumed for this numerical gate. It is not promoted to the global zeolite13X catalogue.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	2.77777777778e-4
stream	cleanHe	F	2.475e-4
stream	regenerationGas	F	3.02777777778e-5
kpi	co2tsa	recovery_He	0.9
kpi	co2tsa	purity_He	1.0
kpi	co2tsa	Dq_CO2	0.408156018947
kpi	co2tsa	capacity_utilisation_CO2	0.680567638068
kpi	co2tsa	Q_sensible_per_cycle_kJ	276.0
kpi	co2tsa	Q_desorption_per_cycle_kJ	37.5
kpi	co2tsa	Q_regeneration_kW	1.045
kpi	co2tsa	Q_adsorption_cooling_kW	-1.045
stream	cleanHe	T	298
stream	feed	T	298
stream	regenerationGas	T	298
. . . and 8 more reference values in the case's `expected`.			

tsa02_refused_capacity

`tutorials/steady/separation/tsa02_refused_capacity`

What it does: TSA refusal lesson: one undersized column cannot hold the CO2 loaded during the declared switch interval

solids

The theory you need. Solids carry mass and enthalpy without VLE: crystallisers, dryers, cyclones, gas–solid splitters. A dryer's air is a REAL stream (two inlets, adiabatic mixing, outlet humidity from the psychrometrics); a cyclone's cut size comes from the selected model (Lapple. . . Muschelknautz). Expect solid streams flagged separately (F_{solid}) and energy balances that close across phase change.

`pneumaticConveyor01_sand`

`tutorials/steady/solids/pneumaticConveyor01_sand`

What it does: Dilute-phase pneumatic conveying of sand in nitrogen: the glass-box dP breakdown + the saltation-margin safety check. Particle velocity from a mechanistic FORCE BALANCE (drag = wall friction + gravity); solids friction from Yang (1974, AIChE J 20:605); Rizk (1973) saltation. Loading ~2 kg/kg (in the Yang voidage range); solids friction DOMINATES the dP – conveying solids IS a pressure-drop problem. u_g/u_salt = 1.72 (safe margin above plugging).

The story: / flowsheetDict (single unit)

Operation block is HARDWARE only (the credo): pipe diameter, length, rise. Every velocity, the suspended density and the pressure drop are RESULTS.

A single pneumaticConveyor unit: silica sand (~200 μm) carried by nitrogen up a 100 mm line, 50 m long, with a 10 m vertical rise.

The total conveying-line pressure drop as a sum of physical contributions (glass-box) — Rhodes, ∗Introduction to Particle Technology∗, Ch. 8:

- superficial gas velocity u_g = 21.9 m/s, - particle velocity u_p = 21.1 m/s (Hinkle slip), - single-particle terminal velocity u_t = 1.03 m/s, - saltation velocity u_salt = 3.6 m/s (Rizk) — u_g is well above it, so transport is stable. The conveyor warns aloud (verbosity $\geq$ 1) if u_g ever falls below u_salt (risk of solids settling and blocking the line) — a first-class, announced design check, never a silent clip.

Solids loading = 0.20 kg solid / kg gas (dilute phase), suspended-solids density ?∗ = 0.47 kg/m^3.

- particle velocity: Hinkle u_p = u_g(1 - 0.68 d_p^0.92 ?_p^0.5 ?_g^-0.2 D^-0.54), with an honest u_g - u_t fallback if the correlation leaves the dilute regime; - gas friction: Fanning (Blasius turbulent / 16/Re laminar); - solids friction: Hinkle f_p = (3/8)(?_g/?_p) C_D (D/d_p)((u_g-u_p)/u_p)^2, C_D from Schiller–Naumann at the slip Reynolds number; - saltation: Rizk (1973).

Operation block is hardware only (the credo): pipe diameter D, length L, vertical rise dz. Every velocity, the suspended density and the pressure drop are results. Particle density comes from the solid component's solid { rho_p; } (silica = 2650 kg/m^3); the size from the inlet PSD (Sauter mean for the drag). The carrier-gas viscosity needs a transport { model Chung; } block in the thermoPhysPropDict (for the friction Reynolds numbers).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`Feed`	`F`	`0.0268345909167`
`stream`	`conveyed`	`F`	`0.0268345909167`
`kpi`	`node`	`P_out`	`234394.335965`
`kpi`	`node`	`accelerationLength`	`16.8207521299`
`kpi`	`node`	`deltaP`	`65605.6640351`
`kpi`	`node`	`deltaP_bends`	`0`
`kpi`	`node`	`deltaP_gasAccel`	`361.754574941`
`kpi`	`node`	`deltaP_gasFriction`	`1492.10736304`
`kpi`	`node`	`deltaP_solidsAccel`	`1383.86777284`
`kpi`	`node`	`deltaP_solidsFriction`	`61751.3873821`
`kpi`	`node`	`deltaP_static`	`616.546942187`
`kpi`	`node`	`nBends`	`0`
`kpi`	`node`	`particleWallFriction`	`0.4`
`kpi`	`node`	`rho_gas`	`3.39180174393`
. . . and 12 more reference values in the case's `expected`.			

`pneumaticConveyor02_bends`

`tutorials/steady/solids/pneumaticConveyor02_bends`

What it does: A SHORT (8 m) conveying line routed through 6 bends – the lesson: A FITTING ADDS PRESSURE DROP WITHOUT ADDING LENGTH. With long-radius sweeps the 6 bends cost ~10% of the total dP; swap them for blind tees (type blindTee;) and their share jumps to ~25% – the erosion-vs-pressure trade made visible. (On this short line the straight-pipe solids friction still carries most of the dP; the bends' share GROWS as lines get shorter and bendier.) Same gas/solids as pneumaticConveyor01 (which runs 30 m / 6 m rise, bend-free) – compare the PER-METRE dP, not the totals.

The story: a fitting adds pressure drop without adding length

The same sand-in-N2 service as pneumaticConveyor01 (loading ~2), on a SHORTER line: 100 mm, 8 m, 2 m rise, routed through 6 long-radius bends. (Case 01 runs 30 m / 6 m rise, bend-free – compare dP PER METRE, not totals.) Each bend decelerates the solids to ~0.75 u_p; the gas re-accelerates them downstream – the same momentum work as the line entry, once per bend: dP_bend = mDotS (u_p - u_p,bend) / A. Run both cases and compare the dP breakdown: the bends add ~10% (long-radius) to ~25% (blind tees) of the dP while adding NO length. Swap a bend 'type longRadius;' for 'type blindTee;' and watch its re-acceleration cost jump (ratio 0.75 → 0.25) – the blind tee trades pressure for near-zero erosion. Solids friction: Yang (1974); particle velocity: force balance; Rizk saltation.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`Feed`	`F`	`0.0268345909167`
`stream`	`conveyed`	`F`	`0.0268345909167`
`kpi`	`node`	`P_out`	`279210.165227`
`kpi`	`node`	`accelerationLength`	`15.5453372639`
`kpi`	`node`	`deltaP`	`20789.8347732`
`kpi`	`node`	`deltaP_bends`	`2070.63576095`
`kpi`	`node`	`deltaP_gasAccel`	`361.754574941`
`kpi`	`node`	`deltaP_gasFriction`	`397.89529681`
`kpi`	`node`	`deltaP_solidsAccel`	`1380.42384063`
`kpi`	`node`	`deltaP_solidsFriction`	`16373.2628921`
`kpi`	`node`	`deltaP_static`	`205.862407813`
`kpi`	`node`	`nBends`	`6`
`kpi`	`node`	`particleWallFriction`	`0.4`
`kpi`	`node`	`rho_gas`	`3.39180174393`
. . . and 12 more reference values in the case's `expected`.			

thermo

The theory you need. Consistency tests interrogate DATA, not models: the van Ness point test checks measured (x, y, T, P) sets against the Gibbs–Duhem constraint (residuals in $\ln(\gamma_1/\gamma_2)$); Herington's area test is its integral cousin. A dataset that fails is not "bad" – the test says WHERE and HOW MUCH, and the fit that follows should weight accordingly. Expect plain-language verdicts with severity, not a pass/fail stamp. *Full derivation: Theory Guide, the consistency chapter of the Props Guide.*

`air01_heater_expand`

`tutorials/steady/thermo/air01_heater_expand`

What it does: Predefined mixture: heat a stream fed with ‘components (air)‘, which expands to N2/O2/Ar at load time

The story: showcase for TRACK A predefined-mixture expansion.

The thermoPhysPropDict lists ‘components (air)‘. At load time the loader splices ‘air‘ into N2 + O2 + Ar (see constant/thermoPhysPropDict). A single heater then adds 5 kW to a 1 mol/s air stream at 1 bar, 298.15 K. The air is gas (vf = 1), so the heater integrates the ideal-gas mixture H_ig over the three real components → a correct MULTICOMPONENT result.

Z-SEEDING NOTE (glass-box honesty): The expansion captures the member mole fractions (N2 0.7808 / O2 0.2095 / Ar 0.0093) as a per-component seed on the ThermoPackage (‘mixtureSeedVector()‘). The feed below STATES the same fractions explicitly so the case is self-evident and the result is unambiguous – a stream that declares its own composition always wins over the seed. The seed exists so a future feed that OMITS its composition can be filled from the mixture defaults; that wiring lives in the stream-construction path (Flowsheet / Composition), not in this case.

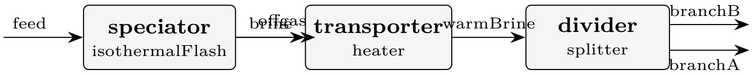
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.001`
`stream`	`hot`	`F`	`0.001`
`kpi`	`airHeater`	`F`	`0.001`
`kpi`	`airHeater`	`P`	`100000`
`kpi`	`airHeater`	`Q`	`5000`
`kpi`	`airHeater`	`Q_kW`	`5`
`kpi`	`airHeater`	`Q_per_mol`	`5000`
`kpi`	`airHeater`	`T_in`	`298.15`
`kpi`	`airHeater`	`T_out`	`468.478847995`
`kpi`	`airHeater`	`dT`	`170.328847995`
`utility`	`airHeater`	`heating.steamHP.duty_kW`	`5`
`utility`	`airHeater`	`heating.steamHP.T`	`468.478847995`
`utility`	`airHeater`	`heating.steamHP.kg_s`	`0.00291715285881`
`utility`	`airHeater`	`heating.steamHP.eur_h`	`0.324`

... and 3 more reference values in the case's `expected`.

basis01_two_unit_chain

tutorials/steady/thermo/basis01_two_unit_chain



What it does: Basis reconciliation, the two-unit chain: a speciating flash hands its brine to a unit running a DIFFERENT thermo world, which transports the species basis as matter instead of re-asking the chemistry

The story: THE TWO-UNIT CHAIN (basis-reconciliation spike S2, ratified 2026-08-03).

Topology only; state lives in 0/.

feed -→ [speciator: isothermalFlash] -→ offgas

brine (carries BOTH bases: the apparent components | that ARE the state, and the species the | network resolved in them) v [transporter: heater, LOCAL molecular thermo world]

WHAT THE CHAIN IS FOR. The ‘speciator‘ runs the case's electrolyte world and SOLVES the chemistry: its brine outlet carries a speciation block stamped ‘origin "solved:speciator"‘. The ‘transporter‘ runs a DIFFERENT world – a plain molecular gammaPhi declared in its own ‘thermo {}‘ block, which resolves no ions at all. It changes T (a duty) and nothing else: one material in, one material out, F and z untouched.

The question the chain asks: what happens to the species basis at that boundary?

Three answers were available and only one is honest. (a) Re-derive the block with the GLOBAL package. That is a chemistry the unit's own world does not do, attributed to the unit – silent respeciation across a model boundary. This is what the engine did before the spike, and it is what the transport contract now stops. (b) Drop the block. The outlet then reports one basis where its inlet reported two, and a student comparing them sees chemistry vanish across a heater. (c) CARRY it. The heater moved no atoms and no charge; the ions in the brine are the same ions, in the same amounts, at a higher T. The block crosses as MATTER, stamped ‘origin "transported:transporter"‘ so the reader can see it was carried and not re-solved.

(c) is the contract. ‘converged/warmBrine‘ is where it is visible.

AND THE PART THAT TOOK A SECOND OPINION TO GET RIGHT. A heater is a heater: the outlet is at 330.81 K while the speciation it carries was solved at 313.15 K. The first version of this case stamped ‘solvedAtT‘ and considered the matter closed – the file said where the block came from, so the reader could not be misled. That was wrong, and the counsel of 2026-08-03 named why:

AN EQUILIBRIUM IS NOT A COMPOSITION. It is a RELATION between a composition and a thermodynamic state. A composition solved at 313 K does not become the equilibrium composition at 331 K by being carried there with a note saying where it came from. Stamping the provenance makes the datum TRACEABLE; it does not make it VALID.

So the two claims are now separate, and only one of them travels:

the AMOUNTS are a material inventory – the transport conserved every species exactly, and they stand; the EQUILIBRIUM is a claim about this state, and it dies here.

‘converged/warmBrine‘ therefore carries ‘equilibriumValidHere false‘, both temperatures, and NO pH – the pH is WITHHELD rather than inherited, because inheriting it would publish a number nothing here computed. The same withholding applies to any activity, saturation index or conditional constant that would rest on the old equilibrium.

Staleness is DECIDED, not assumed: the block goes stale because T actually moved. A pump carrying the same brine at constant T and P across the same boundary would leave the equilibrium intact and the pH reported – the boundary alone does not invalidate anything; the change of state does. A reader who wants the equilibrium AT the outlet must put a unit there that can solve one; the molecular world cannot, which is the whole point of the boundary.

THE IDENTITY SECOND WITNESS. Delete the ‘thermo {}‘ block below: the transporter then runs the GLOBAL electrolyte world, which CAN re-solve the chemistry, and does – at 332.23 K, giving a different (and locally correct) block. What must be identical either way is the INTERMEDIATE stream ‘brine‘:

what unit 2 is cannot reach back and change what unit 1 solved. That is the invariant the gate probes; the differing outlet is not a failure, it is the boundary being visible.

AND SINCE 2026-08-09 IT IS VISIBLE IN THE TEMPERATURE TOO, which is worth reading slowly. A 'heater' now solves for the outlet STATE whose enthalpy is H_in + Q – in ITS OWN declared world (R-E1: an unpinned stream means the equilibrium of (T, P, z) in the CONSUMING unit's thermo world). The molecular world below finds this brine 1.3 % vapour at ~331 K, so 40 kW buys 17.66 K there; the global electrolyte world holds the same volatiles as ions, finds essentially no vapour, and the same 40 kW buys 19.08 K to 332.23 K. Same enthalpy, two temperatures – exactly the settled rule that H is the conserved truth and T is the model-dependent readout. The consequence a reader will meet: the energy-balance report prices EVERY stream in the GLOBAL package, so this unit's RAW closure reads 92.54 % rather than 100 %. That residual is the model boundary, not a first-law error – and SINCE THE SAME DAY THE REPORT SAYS SO. It was a named gap for exactly one afternoon (the report had no notion of a per-unit thermo world, recorded in docs/design/tp-stream-energy-coherence.md), and it was closed by the model-boundary LEDGER:

raw imbalance -2.9851 kW (dH 37.0149 vs 40.0000 kW of duty) boundary step -2.9851 kW (independently computed, see below) remaining 9.4e-9 kW → adjusted closure 100.00 %, no alarm

reports/balances/energyBalance_byUnit.csv carries all three, side by side and never collapsed, plus a ledger row naming BOTH worlds by their declared models. The step is not taken on trust: the report assembles this unit's declared world ITSELF, prices both streams in both packages at the same (T, P, z), and credits the step only if its own number reproduces the imbalance – an auditor that reuses the auditee's arithmetic checks nothing. See docs/design/model-boundary-energy-ledger.md.

Before the 2026-08-09 heater fix the sensible inversion happened to track the global datum and the gap read 100 %, which was luck, not agreement.

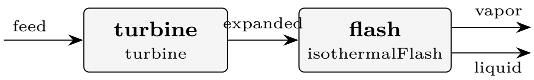
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	branchA	F	0.00689112315881
stream	branchB	F	0.0206733694764
stream	brine	F	0.0275644926352
stream	feed	F	0.0277222222222
stream	offgas	F	0.000157729586982
stream	warmBrine	F	0.0275644926352
kpi	speciator	F_alpha	0.0275644926352
kpi	speciator	F_beta	0.000157729586982
kpi	speciator	F_in	0.0277222222222
kpi	speciator	K_CO2	4.78835784059
kpi	speciator	K_N2	105036.780664
kpi	speciator	K_NH3	0.777804159627
kpi	speciator	K_ethanol	0.180718643037
kpi	speciator	K_water	0.0721940189402

... and 34 more reference values in the case's `expected`.

perUnitThermo01_srk_nrtl

tutorials/steady/thermo/perUnitThermo01_srk_nrtl



What it does: Per-unit thermo: turbine SRK (real gas) → flash NRTL (non-ideal liquid)

The story: Demonstrates PER-UNIT thermo models: the global package (Raoult) is the default, but each unit can override it with a ‘thermo { ... }‘ block that replaces the model it cares about. Components stay global.

turbine → equationOfState SRK (real-gas expansion, high P) flash → activityModel NRTL (non-ideal ethanol/water liquid)

Pedagogical caveat: the stream that crosses from the turbine (SRK) into the flash (NRTL + idealGas) is re-interpreted by a different model – a small thermodynamic inconsistency at the boundary. Real practice keeps one consistent method per section; mixing is an advanced, deliberate act.

What to expect (golden-locked):

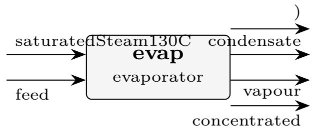
kind	unit/stream	key	expected
stream	expanded	F	0.0138888888889
stream	feed	F	0.0138888888889
stream	liquid	F	0.00854830793289
stream	vapor	F	0.005340580956
kpi	flash	F_alpha	0.00854830793289
kpi	flash	F_beta	0.005340580956
kpi	flash	F_in	0.0138888888889
kpi	flash	K_ethanol	1.4838682811
kpi	flash	K_water	0.647356121192
kpi	flash	P	100000
kpi	flash	Q	-413177.313739
kpi	flash	Q_kW	-413.177313739
kpi	flash	T	353
kpi	flash	V_over_F	0.384521828832

. . . and 30 more reference values in the case’s `expected`*.*

thermoTest

model1_lumped_evaporator

tutorials/steady/thermoTest/model1_lumped_evaporator



What it does: thermoTest model 1 – NaCl as a LUMPED molecular component: the boiling-point elevation is the IDEAL K_b∗m (van't Hoff), BPE ~ 2.11 K. Compare model2 (same evaporator, Pitzer a_w → BPE ~ 5.07 K): the REPRESENTATION axis, lumped vs dissociated, on one operation.

The story: thermoTest model1_lumped_evaporator / flowsheetDict NaCl treated as a LUMPED molecular component: the boiling-point elevation is the IDEAL van't Hoff K_b∗m (~2.11 K). Model 1 of the thermoTest ladder – the deliberately-too-simple baseline the pitzer (model 2) and eNRTL rungs correct. Same evaporator, climbing thermo.

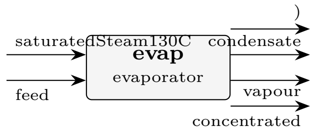
What to expect (golden-locked):

kind	unit/stream	key	expected
stream	concentrated	F	0.020084561515
stream	condensate	F	0.0125
stream	feed	F	0.0277777777778
stream	saturatedSteam130C	F	0.0125
stream	vapour	F	0.00769321626283
kpi	evap	A	20
kpi	evap	A_m2	20
kpi	evap	BPE	2.11524224539
kpi	evap	F_conc	0.020084561515
kpi	evap	F_conc_kmol_h	72.3044214538
kpi	evap	F_conc_mass	0.417969209025
kpi	evap	F_cond	0.2251875
kpi	evap	F_cond_kg_h	810.675
kpi	evap	F_in	0.0277777777778

. . . and 24 more reference values in the case's `expected`*.*

model2_pitzer_evaporator

tutorials/steady/thermoTest/model2_pitzer_evaporator



What it does: thermoTest model 2 – NaCl as a PITZER electrolyte (Na+ + Cl-, aqueous-inf-dilution reference): the boiling-point elevation comes from the real water activity a_w, BPE ~ 5.07 K vs the ideal 2.11 K of model1.

The story: BPE NOTE (2026-07-02, a story in three acts): (1) frozen 25 C surface → BPE 5.07 K; (2) LINEAR calorimetric T-slots (Parker fit) → 5.58 K – the 25 C derivative extrapolated straight to the boil; (3) the FULL Silvester & Pitzer 1977 T-functions (eqs 30-32, Table IV verbatim in Na-Cl.dat) + A_phi(T) curated against their Table II → BPE 5.06 K. beta0(T) has a MAXIMUM near 150 C (their Fig 4): the linear slope overshoots where the real curve has already bent over. The lesson: a first derivative is not a function – validity windows are data.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	concentrated	F	0.0203403273092
stream	condensate	F	0.0125
stream	feed	F	0.0277777777778
stream	saturatedSteam130C	F	0.0125
stream	vapour	F	0.00743745046862
kpi	evap	A	20
kpi	evap	A_m2	20
kpi	evap	BPE	5.0621680882
kpi	evap	F_conc	0.0203403273092
kpi	evap	F_conc_kmol_h	73.225178313
kpi	evap	F_conc_mass	0.422576829808
kpi	evap	F_cond	0.2251875
kpi	evap	F_cond_kg_h	810.675
kpi	evap	F_in	0.0277777777778

. . . and 26 more reference values in the case's `expected`.

model3_enrtl_antisolvent_crystalliser

tutorials/steady/thermoTest/model3_enrtl_antisolvent_crystalliser

What it does: Antisolvent (drowning-out) crystallisation of NaCl: a SATURATED aqueous brine is mixed with ethanol; the eNRTL mixed-solvent term lowers the dielectric, raises gamma_pm, and DROPS the saturation molality m_sat – so NaCl precipitates with NO cooling and NO evaporation. Saturation from the SAME ion-activity product Ksp anchored in pure water, solved with the mixed-solvent gamma_pm.

The story: thermoTest/model3_enrtl_antisolvent_crystalliser – step 3 of the thermoTest arc (same {NaCl, CaSO4, ethanol, water} system, five property models): here the package selects eNRTL MIXED-SOLVENT, the only model in the arc that can reach the ethanol-lowered dielectric. The flowsheet is crystalliser06_nacl_antisolvent's, unchanged:

ANTISOLVENT (drowning-out) crystallisation

[cryst] crystals (solid NaCl product) ethanol motherLiquor (saturated supernatant: water + ethanol + residual NaCl)

The crystalliser receives BOTH feeds directly – a saturated aqueous NaCl brine AND the ethanol antisolvent – combines them internally, and discharges its TWO natural products: the crystals and the saturated mother liquor. No separate mixer, no separate filter: an equilibrium crystalliser already KNOWS how much solid forms and what the saturated liquor is.

The physics: ethanol's low dielectric (24.3 vs water's 78.5) lowers the mixture permittivity, which RAISES the mean ionic activity gamma_pm (eNRTL mixed-solvent + Born), so the saturation molality m_sat COLLAPSES (6.14 → ~1.3 mol/kg) and NaCl precipitates with NO cooling and NO boil-off. The saturation is the SAME ion-activity product Ksp = (gamma_pm ∗ m_sat)^2 anchored in pure water, solved with the mixed-solvent gamma_pm at the feed's salt-free ethanol fraction – the physics the aqueous Pitzer model cannot reach (the reason eNRTL exists).

Basis: 100 kmol/h saturated brine (x_water 0.900, x_NaCl 0.100; ~6.14 mol/kg) + 60.0 kmol/h ethanol → salt-free x_ethanol ~ 0.40. Components live case-local in constant/components/; the eNRTL ion/tau records resolve from data/standards/ at runtime.

(Want imperfect separation – a wet cake with clinging liquor, fines loss, washing? Feed the single-output 'magma' form to a downstream 'filter'.)

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	brine	F	0.0277777777778
stream	crystals	F	0.00202035108762
stream	ethanol	F	0.0166666666667
stream	motherLiquor	F	0.0424240933568
kpi	cryst	Ksp_activity	34.8439846653
kpi	cryst	Q_kW	-6.55987377671
kpi	cryst	Q_removed	6559.87377671
kpi	cryst	T_op	298.15
kpi	cryst	antisolvent_xfrac	0.399920015997
kpi	cryst	c_sat	0.0517444790334
kpi	cryst	cakeLiquidFlow	0.000329661969914
kpi	cryst	cakeMoisture	0.1
kpi	cryst	cakeWetness_pct	9.09090909091
kpi	cryst	crystal_flow	0.0988038720389

... and 13 more reference values in the case's `expected`*.*

model5_nrtl_flash

tutorials/steady/thermoTest/model5_nrtl_flash

What it does: thermoTest model 5 – water + ethanol as MOLECULAR components (no ions): an isothermal VLE flash on the NRTL activity model. This is the antisolvent (ethanol) and the solvent (water) in their own role – the same substances that dissolve/precipitate NaCl in models 2-4, here separated by distillation.

The story: thermoTest model5_nrtl_flash / flowsheetDict Water + ethanol as MOLECULAR components (no ions): an isothermal VLE flash on the NRTL activity model, selected through the ethanolWater_NRTL property package. Model 5 of the thermoTest ladder – the molecular gamma-phi anchor beside the electrolyte rungs.

What to expect (golden-locked):

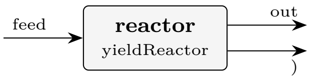
kind	unit/stream	key	expected
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`liquid`	`F`	`0.0170966158658`
`stream`	`vapor`	`F`	`0.010681161912`
`kpi`	`flash`	`F_alpha`	`0.0170966158658`
`kpi`	`flash`	`F_beta`	`0.010681161912`
`kpi`	`flash`	`F_in`	`0.0277777777778`
`kpi`	`flash`	`P`	`100000`
`kpi`	`flash`	`Q`	`0`
`kpi`	`flash`	`Q_kW`	`0`
`kpi`	`flash`	`T`	`353`
`kpi`	`flash`	`V_over_F`	`0.384521828832`
`kpi`	`flash`	`phaseSet`	`0`
`stream`	`feed`	`T`	`353`
`stream`	`liquid`	`T`	`353`
. . . and 3 more reference values in the case's `expected`.			

userops

The theory you need. User-defined operations: the extension seam. A custom property op or unit compiled into the binary via the explicit factory – the tutorial shows the registration pattern (`registerBuiltins`) end to end.

userOp01_yield_reactor

tutorials/steady/userops/userOp01_yield_reactor



What it does: User-defined yieldReactor (case-local code/): neoPentane → nPentane, X=0.8

The story: A single unit op — but a USER-DEFINED one (‘yieldReactor‘), written in this case’s code/ folder, not in the Choupo source tree. Build + run it with:

bin/buildCode tutorials/steady/userops/userOp01__yield__reactor

(NOT runCase: the type ‘yieldReactor‘ does not exist in the stock binary; buildCode compiles the case’s code/ against libchoupo.so.)

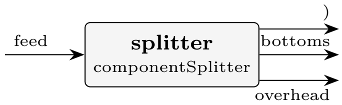
The op converts a SPECIFIED fraction of the reactant to the product (no kinetics) — a "yield reactor", which the built-in CSTR/PFR are not. Here: 80 % of neoPentane isomerised to nPentane. Because the two are isomers (same MW), the mass balance closes to 100 %.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	feed	F	0.000277777777778
stream	out	F	0.000277777777778
kpi	reactor	F_in_kmol_h	1
kpi	reactor	F_out_kmol_h	1
kpi	reactor	T	320
kpi	reactor	X	0.8
kpi	reactor	converted_kmol_h	0.8
stream	feed	T	320
stream	out	T	320

`userOp02_component_splitter`

`tutorials/steady/userops/userOp02_component_splitter`



What it does: User-defined componentSplitter (case-local code/): shortcut benzene/toluene split

The story: A USER-DEFINED unit op (‘componentSplitter‘), written in this case’s code/ folder. Build + run with:

runCase tutorials/steady/userops/userOp02__component__splitter

(runCase sees the code/ folder and compiles it first.)

The op is a SHORTCUT separator: a per-component recovery to the overhead. Here a benzene/toluene feed is split so 95 % of the benzene and only 8 % of the toluene leave overhead — an enriched light cut, the kind of stand-in you place before sizing a real column. Two outlet streams (overhead, bottoms); mass closes to 100 % (every component is conserved across the split).

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`bottoms`	`F`	`0.0134722222222`
`stream`	`feed`	`F`	`0.0277777777778`
`stream`	`overhead`	`F`	`0.0143055555556`
`kpi`	`splitter`	`F_bottoms_kmol_h`	`48.5`
`kpi`	`splitter`	`F_overhead_kmol_h`	`51.5`
`kpi`	`splitter`	`overhead_fraction`	`0.515`
`stream`	`bottoms`	`T`	`360`
`stream`	`feed`	`T`	`360`
`stream`	`overhead`	`T`	`360`

utilities

The theory you need. Utility streams (steam levels, cooling water, refrigerants, electricity) carry duty into and out of the plant at declared temperature levels and prices; allocation is by T-level feasibility, announced.

utility01_dowtherm_preheat

tutorials/steady/utilities/utility01_dowtherm_preheat

What it does: Dowtherm A loop preheats a hydrocarbon feed (eps-NTU)

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	hotOil	F	0.0416666666667
stream	oilReturn	F	0.0416666666667
stream	processFeed	F	0.0111111111111
stream	processOut	F	0.0111111111111
kpi	preheater	C_cold	2992.03981719
kpi	preheater	C_hot	17325.8215856
kpi	preheater	C_r	0.17269252153
kpi	preheater	LMTD	59.6763812217
kpi	preheater	NTU	5.01330226751
kpi	preheater	Q	895145.718325
kpi	preheater	Q_kW	895.145718325
kpi	preheater	T_cold_in	320
kpi	preheater	T_cold_out	619.175737295
kpi	preheater	T_hot_in	623.15

. . . and 10 more reference values in the case's `expected`.

`utility02_hitec_csp_heater`

`tutorials/steady/utilities/utility02_hitec_csp_heater`

What it does: HITEC molten salt loop sensibly heats pressurised (40 bar) boiler feedwater to ~244 C, just below T_sat, for a downstream steam generator (eps-NTU, no phase change), ~208 kW duty

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`boilerFeed`	`F`	`0.0222222222222`
`stream`	`boilerOut`	`F`	`0.0222222222222`
`stream`	`boilerOut`	`T`	`517.400926398`
`stream`	`saltReturn`	`F`	`0.0833333333333`
`stream`	`saltSupply`	`F`	`0.0833333333333`
`kpi`	`boilerHX`	`C_cold`	`1677.77777778`
`kpi`	`boilerHX`	`C_hot`	`12795.4797468`
`kpi`	`boilerHX`	`C_r`	`0.131122694184`
`kpi`	`boilerHX`	`LMTD`	`306.566828203`
`kpi`	`boilerHX`	`NTU`	`0.405298013245`
`kpi`	`boilerHX`	`Q`	`208465.443178`
`kpi`	`boilerHX`	`Q_kW`	`208.465443178`
`kpi`	`boilerHX`	`T_cold_in`	`393.15`
`kpi`	`boilerHX`	`T_cold_out`	`517.400926398`

... and 10 more reference values in the case's `expected`.

utility03_glycol_brine_chiller

tutorials/steady/utilities/utility03_glycol_brine_chiller

What it does: PG-30 brine sensibly cools an ethanol vapour stream from 330 K to ~273 K (eps-NTU, no phase change), ~20.8 kW duty

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	brineReturn	F	0.416666666667
stream	brineSupply	F	0.416666666667
stream	ethanolOut	F	0.00555555555556
stream	ethanolVapour	F	0.00555555555556
kpi	condChiller	C_cold	57599.6888818
kpi	condChiller	C_hot	623.888888889
kpi	condChiller	C_r	0.0108314628256
kpi	condChiller	LMTD	12.519257418
kpi	condChiller	NTU	4.48797862867
kpi	condChiller	Q	35053.9207703
kpi	condChiller	Q_kW	35.0539207703
kpi	condChiller	T_cold_in	273.15
kpi	condChiller	T_cold_out	273.758578301
kpi	condChiller	T_hot_in	330

. . . and 10 more reference values in the case's `expected`.

Category II: The property bench (choupoProps)

Questions asked of the thermodynamics BEFORE a simulation uses it – CREATE, SEE, TEST, FIT, PROMOTE.

adsorption

The theory you need. The isotherm bench: evaluating a curated adsorption-equilibrium record (Henry-limit, saturation, pin and unit-invariance gates run aloud) and REGRESSING one from data – the multi-temperature van't Hoff Langmuir fit with identifiability verdicts: single-T data refuses ΔH_{ads}, Henry-regime data yields only the product $q_{\max}b$, and a non-converged fit refuses its verdict by name. *Full derivation: Theory Guide, the adsorption chapter.*

isotherm01_eval_13x_co2

tutorials/props/adsorption/isotherm01_eval_13x_co2

eval_13x_co2
isothermEval

What it does: Adsorption isotherm bench: CO2/zeolite13X q(T,p) grid + Henry/saturation/pin/unit-invariance gates

The story: One isothermEval operation: the curated CO2/zeolite13X langmuir record on a 3-T x 4-p grid (K, Pa – grid values are RAW canonical SI), with the unit-invariance twin (gate d) declared case-locally in constant/. Temperatures match the Jaschik 2020 dataset the FIT tutorial (isotherm02_fit_jaschik) regresses – eval here, fit there.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	eval_13x_co2	T0	2.931e+02
diag	eval_13x_co2	T1	3.131e+02
diag	eval_13x_co2	T2	3.331e+02
diag	eval_13x_co2	b_T0	1.227e+01
diag	eval_13x_co2	b_T1	3.772e+00
diag	eval_13x_co2	b_T2	1.337e+00
diag	eval_13x_co2	henry_T0	6.133e-04
diag	eval_13x_co2	henry_T1	1.886e-04
diag	eval_13x_co2	henry_T2	6.683e-05
diag	eval_13x_co2	qsat	5.000e+00
diag	eval_13x_co2	tRef	2.981e+02
diag	eval_13x_co2	gate_henry	1.000e+00
diag	eval_13x_co2	gate_henry_maxdev	1.227e-07
diag	eval_13x_co2	gate_saturation	1.000e+00

... and 9 more reference values in the case's `expected`.

isotherm02_fit_jaschik

tutorials/props/adsorption/isotherm02_fit_jaschik

fit__jaschik
fitParameters

recovery__multiT
fitParameters

gate__singleT
fitParameters

gate__henry
fitParameters

What it does: Isotherm regression: CO2/13X Jaschik 2020 (CC BY) multi-T Langmuir fit + recovery/single-T/Henry identifiability gates

The story: Four fitParameters(kind isotherm) operations: the REAL Jaschik fit plus the three identifiability gates on zero-noise synthetics generated from a KNOWN surface (q__max 4.6 mol/kg, b__ref 3.0e-4 1/Pa @ 298.15 K, dH__ads -38000 J/mol) – so every gate has an exact truth to recover or to honestly refuse. Inline synthetic data is canonical SI (K, Pa, mol/kg); the vendored Jaschik dataset declares its own units (bar, mol/kg) and is converted at the data boundary.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	fit_jaschik	converged	1.00000e+00
diag	fit_jaschik	identifiable	1.00000e+00
diag	fit_jaschik	n_data	1.32000e+02
diag	fit_jaschik	n_params	3.00000e+00
diag	fit_jaschik	dof	1.29000e+02
diag	fit_jaschik	single_T	0.00000e+00
diag	fit_jaschik	henry_regime	0.00000e+00
diag	fit_jaschik	endothermic_suspicious	0.00000e+00
diag	fit_jaschik	fit.q_max.value	4.31448e+00
diag	fit_jaschik	fit.q_max.stderr	5.17246e-02
diag	fit_jaschik	fit.b_ref.value	4.06758e-04
diag	fit_jaschik	fit.b_ref.stderr	2.90989e-05
diag	fit_jaschik	fit.dH_ads.value	-3.73835e+04
diag	fit_jaschik	fit.dH_ads.stderr	1.90453e+03

. . . and 69 more reference values in the case's `expected`.

isotherm03_fit_refusals

tutorials/props/adsorption/isotherm03_fit_refusals

gate__henry__nonconverged	gate__nonconverge	gate__langfull__nonconverged
fitParameters	fitParameters	fitParameters

What it does: Fit refusal gates: non-converged fits refuse their verdicts by name

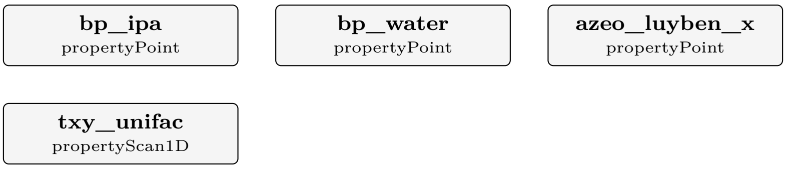
The story: Two ADVERSARIAL fitParameters(kind isotherm) operations – see the controlDict header. Synthetic data generated EXACTLY from the known surface (q__max 4.6 mol/kg, b__298 3.0e-4 1/Pa, dH__ads -38000 J/mol, tRef 298.15 K) – zero noise, so the ONLY failure mode under test is the strangled optimiser, never the data.

compare

The theory you need. Method comparison runs the SAME property through different models side-by-side – UNIFAC vs NRTL, Joback vs measured, ideal vs EOS. The gap between curves IS the modelling uncertainty; picking a method is a decision the case author owns, made with eyes open. *Full derivation: Theory Guide, the Props Guide.*

acetone01_ipa_water_azeotrope

tutorials/props/compare/acetone01_ipa_water_azeotrope



What it does: IPA/water T-x-y at 1 atm from PREDICTIVE UNIFAC on a lake-estimated isopropanol, held against Luyben's published azeotrope

The story: THE FIRST RUNNABLE SLICE of the acetone-from-2-propanol reference case (docs/design/acetone-ipa-reference-case.md). It answers ONE question, and the question is chosen because it can be answered honestly today:

how close does a component Choupo has never curated, priced by a model that saw no data for it, land on a published azeotrope?

WHAT IS BEING TESTED, and it is a stack of three things at once – said up front, because a single number that agrees can hide two errors that cancel:

1. ISOPROPANOL IS A LAKE ESTIMATE. It is NOT in data/standards/. The record in this case's constant/components/ came from data/groupEstimative/, where every NUMBER is a Joback / Lee-Kesler estimate from the molecular structure – including Tb, Tc, Pc, omega and hence the whole Ambrose-Walton vapour pressure. Its estimated Tb is 359.98 K; the measured normal boiling point Luyben quotes is 355.4 K. The pure component is already 4.6 K out before any mixture model is asked anything. 2. UNIFAC IS PREDICTIVE. No pair was fitted, because Choupo ships no UNIQUAC or NRTL pair for IPA-water and inventing one would convert ‘unsourced‘ into ‘falsely sourced‘. gamma comes from the groups. 3. WATER IS CURATED. It is the one leg of the three that rests on reviewed data, which is what makes the comparison legible at all.

THE COMPARISON POINTS: W. L. Luyben, "Design and Control of the Acetone Process via Dehydrogenation of 2-Propanol", Ind. Eng. Chem. Res. 50 (2011) 1206-1218, DOI 10.1021/ie901923a. He states the IPA/water azeotrope as 67.32 mol % IPA at 353.4 K and 1 atm, and normal boiling points IPA 355.4 K, water 373 K. Those numbers come out of his UNIQUAC model, fitted to VLE data neither this case nor its components have ever seen – so for the predictive route here they are an INDEPENDENT target.

A NOTE ON WHAT KIND OF ROW PINS THIS, because it is not an ‘anchor‘. Luyben's azeotrope is his UNIQUAC model's value, not a measurement, and the AAD computed here is derived from that number and this model TOGETHER – so nobody published it and no primary can be quoted for it. It is pinned as a self-recorded ‘aad‘ row: what the suite holds is that the agreement has not MOVED. Anchoring properly needs a bubble temperature published as an operation diagnostic, which is a separate gap – see the note at the end.

WHAT THIS CASE IS NOT. It is not a reproduction of Luyben's flowsheet, and it makes no claim about the acetone process. The full reproduction needs three curated UNIQUAC pairs that do not exist; the design record's section 4 lists them and section 5 says which case category that would be. This is the bottom rung: the thermodynamics, alone, against three published points.

A GAP FOUND BUILDING THIS, and left visible rather than worked around. The natural shape for this case is three ‘propertyPoint‘ ops asking for ‘T_bubble‘ at the three compositions, because a propertyPoint publishes into the operation diagnostics where a golden row can read it. It does not work: ‘PropertyPoint‘ never reads its own ‘properties (...)‘ list – it emits a fixed set and discards what the case asked for. Both the class header ("properties (all); // or pick a subset") and the published JSON schema (which names ‘T_bubble‘ among the accepted keywords) say otherwise. That is a parsed-and-dropped field, the shape this project has paid for three times, and it is why the comparison below rides the overlay instead.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	bp_ipa	Cp_ig	99.0857
diag	bp_ipa	H_R	0.0000
diag	bp_ipa	H_ig	-257924.6674
diag	bp_ipa	H_real	-257924.6674
diag	bp_ipa	P	101325.0000
diag	bp_ipa	S_R	0.0000
diag	bp_ipa	S_ig	329.0377
diag	bp_ipa	S_real	329.0377
diag	bp_ipa	T	350.0000
diag	bp_ipa	T_bubble	359.9075
diag	bp_ipa	Z	1.0000
diag	bp_ipa	gamma	1.0916
diag	bp_ipa	v_molar	0.0287
diag	bp_water	Cp_ig	34.0552
…and 30 more reference values in the case's `expected`.			

acetone04_acetone_water_vle

tutorials/props/compare/acetone04_acetone_water_vle

bp__acetone propertyPoint	**bp__water** propertyPoint	**dilute__acetone** propertyPoint
txy__top__fine propertyScan1D	**txy__unifac** propertyScan1D	

What it does: Acetone/water T-x-y at 1 atm on PREDICTIVE UNIFAC – both components curated, so this isolates the mixture model

The story: MEASURE THE THERMODYNAMICS BEFORE BUILDING THE UNIT THAT RESTS ON IT.

Luyben's absorber recovers acetone from the reactor's hydrogen-rich offgas by washing it with water, and it is the next unit of the flowsheet to build (docs/design/acetone-ipa-reference-case.md). What that unit costs, and whether it can work at all, is set by ONE thing: how volatile acetone is out of a dilute aqueous solution. So that is measured here first – the same order that made acetone01 useful before acetone03 was written.

WHAT MAKES THIS A CLEANER TEST THAN acetone01. Both components are CURATED: acetone and water are in data/standards/, neither comes from the estimate lake. acetone01 had to measure a mixture model and a lake-estimated component at once and separate them afterwards; here there is nothing to separate. Whatever deviation appears is UNIFAC, plus whatever the pure vapour pressures carry – and the two pure boiling points below say how much that second part is.

WHAT LUYBEN GIVES, and it is less than for the IPA binary: acetone's normal boiling point 329.4 K, and the qualitative statement that acetone/water has NO azeotrope but a pinch at the high-acetone end. A qualitative claim is still falsifiable – a model that invents an azeotrope here would be wrong in a way no AAD would show.

NOTE THE WATER LEG IS KNOWN TO BE 0.70 K LOW before it is measured again here: Choupo's curated water record puts the normal boiling point at 372.4536 K against the definition's 373.15 K, because its Antoine set is being used at the very top of its own declared window (see acetone01__ipa__water__azeotrope/CLAUDE.md). It is repeated in this case rather than assumed, because a number quoted from another case is a number with two homes.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	bp_acetone	Cp_ig	79.3862
diag	bp_acetone	H_R	0.0000
diag	bp_acetone	H_ig	-214666.2971
diag	bp_acetone	H_real	-214666.2971
diag	bp_acetone	P	101325.0000
diag	bp_acetone	S_R	0.0000
diag	bp_acetone	S_ig	302.9409
diag	bp_acetone	S_real	302.9409
diag	bp_acetone	T	330.0000
diag	bp_acetone	T_bubble	329.4179
diag	bp_acetone	Z	1.0000
diag	bp_acetone	gamma	1.1170
diag	bp_acetone	v_molar	0.0271
diag	bp_water	Cp_ig	34.2177

. . . and 33 more reference values in the case's `expected`*.*

compare_activity_etoh_water

tutorials/props/compare/compare_activity_etoh_water

What it does: Compare 3 activity models (ideal / NRTL / Wilson) for γ_ethanol in equimolar mixture with water vs T

The story: COMPARE 3 activity-coefficient models for γ_ethanol in an equimolar ethanol/water mixture, T = 280 K to 360 K, P = 1 atm.

ideal : $\gamma = 1$ always (Raoult's law assumption). NRTL : strong non-ideality, γ_ethanol > 1 (azeotrope behaviour). Wilson : similar to NRTL for this binary, slightly different curve.

Pedagogical point: a student studying ethanol/water VLE will see that "ideal" is a bad approximation here (γ_ethanol grows to ~2-3 at low T), and that NRTL/Wilson give similar non-ideal corrections. The choice between the two is then decided by which fits the project's experimental data better (workflow Mode B, see tutorials/props/README.md).

Each op overrides the global activityModel block via 'thermo {}'. The three CSVs (gamma_ideal.csv, gamma_nrtl.csv, gamma_wilson.csv) are auto-overlaid by the GUI.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	g_ideal	n_emptyCurves	0
diag	g_ideal	n_failures	0
diag	g_ideal	n_points	41
diag	g_nrtl	n_emptyCurves	0
diag	g_nrtl	n_failures	0
diag	g_nrtl	n_points	41
diag	g_wilson	n_emptyCurves	0
diag	g_wilson	n_failures	0
diag	g_wilson	n_points	41

`compare_eos_co2`

`tutorials/props/compare/compare_eos_co2`

z__ideal propertyScan1D | **z__srk** propertyScan1D | **z__pr** propertyScan1D

What it does: Compare 3 EoS (idealGas / SRK / PR) for Z of CO2 near critical (T=310 K, P from 1 to 100 bar)

The story: COMPARE 3 equations of state for the same property at the same conditions. This is what choupoProps is ∗for∗: helping a student or researcher CHOOSE the model that best describes the system at hand — not just look at pretty Z curves of one EoS at a time.

System: pure CO2 at T = 310 K (just above its critical temperature Tc = 304.13 K), pressure swept from 1 bar to 100 bar. Near the critical pressure (Pc = 7.38 MPa) the compressibility plunges as the fluid turns liquid-like; the three curves read (at P = 73.7 bar, the swept point nearest Pc):

idealGas: Z = 1 always (textbook reference; ignores attraction). SRK : Z = 0.538 at Pc. PR : Z = 0.509 at Pc — systematically lower than SRK (more attraction), and essentially on the experimental value. NIST : the ∗experimental∗ Z near the critical point is ~0.49.

The lesson: at and just above Tc, BOTH cubics track CO2 well — PR sits almost exactly on the ~0.49 datum and SRK is only ~0.05 high. This is a regime where cubic EoS are at their best, not their worst. Both Z curves keep falling past Pc to a shallow minimum (SRK ~0.30, PR ~0.275 near 98 bar) before rising again; the SRK-vs-PR gap is the diagnostic a student should read off the overlay.

Each operation overrides the global thermoPhysPropDict with a different EoS via the ‘thermo {}‘ per-op block. The three CSVs are independent, so the GUI overlays them on the same Plotly canvas (same X axis "P", same Y "Z"). The result is one figure with three curves, ready for the project report.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	z_ideal	n_emptyCurves	0
diag	z_ideal	n_failures	0
diag	z_ideal	n_points	50
diag	z_srk	n_emptyCurves	0
diag	z_srk	n_failures	0
diag	z_srk	n_points	50
diag	z_pr	n_emptyCurves	0
diag	z_pr	n_failures	0
diag	z_pr	n_points	50

compare_kinetics_order

tutorials/props/compare/compare_kinetics_order

rate_order1 kinetics1D

rate_order2 kinetics1D

What it does: Rate-law order from DATA: 1st- vs 2nd-order fits overlaid on measured c(t) — the data picks the order

The story: The "see, then decide" of CHEMICAL KINETICS: fit a 1st- and a 2nd-order rate law to the SAME measured c(t) and overlay both on the data. No ‘k‘ is given, so each op REGRESSES its own k from the points (provenance = fitted). The data here are second-order – the 1st-order curve misses the tail, the 2nd lands on the points. The student picks the order from the evidence.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	rate_order1	R2	0.973397
diag	rate_order1	chi2	0.0105136
diag	rate_order1	fitted	1
diag	rate_order1	k	0.00065871
diag	rate_order1	max_abs_resid	0.0544611
diag	rate_order1	n_data	8
diag	rate_order1	n_points	61
diag	rate_order1	order	1
diag	rate_order1	rms	0.0362518
diag	rate_order2	R2	0.999421
diag	rate_order2	chi2	0.000228631
diag	rate_order2	fitted	1
diag	rate_order2	k	0.000992909
diag	rate_order2	max_abs_resid	0.00829088

. . . and 8 more reference values in the case’s `expected`*.*

compare_psat_ambrose_walton

tutorials/props/compare/compare_psat_ambrose_walton

psat_aw_vs_antoine
propertyScan1D

What it does: Psat: Ambrose-Walton corresponding-states PREDICTION vs fitted Antoine DATA, for benzene (non-polar, good) and ethanol (associating, degrades) — see the error, then decide

The story: Ambrose-Walton corresponding states (Ambrose & Walton 1989; Poling et al. 5th ed., Eq. 7-2.7) predicts vapour pressure from Tc, Pc and omega ALONE – exactly the constants a Joback + Lee-Kesler estimate produces. So an estimated component becomes flashable with NO measured Psat data. But it is an ESTIMATE, and the glass-box rule is: SEE the error before you trust it.

This sweeps T = 270 → 360 K and tabulates four curves: Psat_benzene – fitted Antoine (catalogue DATA) } non-polar pair Psat_benzeneAW – Ambrose-Walton PREDICTION } Psat_ethanol – fitted Antoine (catalogue DATA) } associating pair Psat_ethanolAW – Ambrose-Walton PREDICTION }

Open 'psat_aw_vs_antoine.csv' (GUI plot, or gnuplot). What you SEE: Near the normal boiling point (~350-360 K) BOTH agree to ~1 % – corresponding states is anchored there and is excellent. Towards the cold end (270 K) the curves separate, and MORE for the associating ethanol than for non-polar benzene: benzene AW vs Antoine: +3.3 % average, +8.9 % at 270 K ethanol AW vs Antoine: +5.2 % average, -16 % at 270 K So corresponding states is a solid FREE first predictor (it makes an estimated component flashable with no data), best near Tb and degrading at low reduced temperature – the degradation worse for hydrogen-bonding species. The lesson: prediction gets you started; for an associating species, or far below Tb, fit Antoine to data (choupoProps vaporPressureFit) before any design number.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	psat_aw_vs_antoine	n_emptyCurves	0
diag	psat_aw_vs_antoine	n_failures	0
diag	psat_aw_vs_antoine	n_points	46

compare_transport_water

tutorials/props/compare/compare_transport_water

mu_iapws propertyScan1D | **mu_generic** propertyScan1D | **k_iapws** propertyScan1D

k_generic propertyScan1D

What it does: Compare water transport: IAPWS (IF97 R12-08/R15-11) vs generic correlations (Andrade / SatoRiedel), 280-370 K

The story: COMPARE water transport properties computed TWO WAYS on the SAME water, over the saturated-liquid window T = 280-370 K:

(a) IAPWS reference – water flagged ‘pureFluids { water { method IF97; } }‘ so the transport accessors route to the matching IAPWS releases: viscosity → R12-08 (industrial) thermal conductivity → R15-11 (background term) evaluated at the saturation state (the legacy accessors carry no P, so p = p_sat(T) – saturated liquid). These are THE international standard reference values (verified to 1e-8 by IAPWSTransport::verify(), which fires on every run).

(b) generic correlations – the same property KEY, but with a generic transport block and NO pureFluids flag, so the accessors fall to: viscosity → Andrade ln(mu) = A + B/T thermal conductivity → SatoRiedel fitted from the per-component .dat (Tc/Tb + the Andrade pair).

Because BOTH ops emit the same property key (‘viscosity_liquid‘, ‘thermal_conductivity_liquid‘) the two curves overlay directly – the student SEES where the cheap generic correlation deviates from the IAPWS reference.

THE LESSON (the actual computed gaps): VISCOSITY – Andrade tracks IAPWS within a few percent over the window (~7 % at 280 K, ~2-4 % mid-range, ~3 % at 370 K). The 2-parameter ln(mu)=A+B/T cannot follow the full curvature, but a few % is fine for a Reynolds number; it is NOT good enough for a precision steam duty. CONDUCTIVITY – SatoRiedel is a corresponding-states estimate and it FAILS BADLY for water: it underpredicts IAPWS by ~50-60 % across the window. Water is the textbook SatoRiedel failure (strong hydrogen bonding, anomalous k); this is exactly why "SEE before you commit" matters – a blind generic pick would have halved the heat-transfer coefficient. For water conductivity use IAPWS R15-11, full stop. Generic correlations are CHEAP and GENERAL (any compound with the handful of .dat parameters); IAPWS is EXACT but water-only. For a water-dominated process use the IF97 + IAPWS route; for a screening sweep over many fluids the generic correlations are the right tool. SEE the gap, then decide – do not pick blindly.

The GUI renders the CSVs via the existing compare machinery (overlay of the ∗_iapws / ∗_generic curves vs T) – no new Explorer kind.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	mu_iapws	n_failures	0
diag	mu_iapws	n_points	46
diag	mu_generic	n_failures	0
diag	mu_generic	n_points	46
diag	k_iapws	n_failures	0
diag	k_iapws	n_points	46
diag	k_generic	n_failures	0
diag	k_generic	n_points	46
diag	mu_iapws	n_emptyCurves	0
diag	mu_generic	n_emptyCurves	0
diag	k_iapws	n_emptyCurves	0
diag	k_generic	n_emptyCurves	0

`compare_visc_liquid_ammonia`

`tutorials/props/compare/compare_visc_liquid_ammonia`

mu_andrade
propertyScan1D

What it does: Liquid-viscosity Andrade for ammonia, 235-300 K (cross-check mu@239.85K ~ 0.255 mPa.s)

The story: Liquid-viscosity Andrade correlation for pure ammonia (NH3), 235-300 K. Coefficients live in data/standards/components/NH3.dat (PROVISIONAL — a 2-point Andrade fit; see that file's provenance block).

Validation anchor: mu_NH3(liquid) @ 239.85 K (normal b.p., -33.3 degC) is approx 0.255 mPa.s. This 2-point PROVISIONAL Andrade fit gives ~0.277 mPa.s at 240 K – about 9% ABOVE the anchor, i.e. it does NOT yet pass through the cross-check point. That gap is the honest state of a provisional fit (see the provenance block in NH3.dat); refitting against more low-T data would close it. Note this case scans a SINGLE method (Andrade) – it is a single-correlation curve, not a multi-method comparison. Ammonia is liquid only under pressure in this T window; viscosity_liquid is a pure-T correlation, so the state P is immaterial to the value.

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`mu_andrade`	`n_emptyCurves`	`0`
`diag`	`mu_andrade`	`n_failures`	`0`
`diag`	`mu_andrade`	`n_points`	`27`

`compare_visc_liquid_water`

`tutorials/props/compare/compare_visc_liquid_water`

mu_andrade
propertyScan1D

mu_vogel
propertyScan1D

What it does: Compare 2 liquid viscosity correlations (Andrade vs Vogel) for water, 280-360 K

The story: COMPARE 2 liquid-viscosity correlations for pure water, T = 280-360 K.

Andrade: $\ln(\mu) = A + B/T$ — 2-parameter, simplest fit, OK over a narrow T window. Vogel : $\ln(\mu) = A + B/(T + C)$ — 3-parameter, captures the curvature better, recommended for wider T ranges.

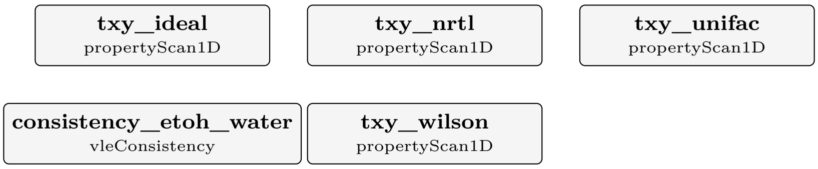
Pedagogical point: for a project using water transport in a non-trivial T window (e.g. 25-90 °C for a heated reactor), the student should know WHICH correlation he is committing to. Both correlations give viscosity FALLING with temperature (the physical trend): ~1.3-1.4 mPa·s near 280 K down to ~0.32 mPa·s at 360 K. This case shows the two models disagree MOST at the cold end (~6 % at 280 K), are within ~2 % at room T, and agree BEST at the hot end (~1 % at 360 K) — the 2-parameter Andrade fit drifts where it is extrapolated coolest. Validate against the textbook value: μ_water @ 298 K ≈ 8.9 × 10?? Pa·s — Vogel reads 8.93 × 10?? (essentially on it), Andrade 9.09 × 10?? (~2 % high).

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`mu_andrade`	`n_emptyCurves`	`0`
`diag`	`mu_andrade`	`n_failures`	`0`
`diag`	`mu_andrade`	`n_points`	`41`
`diag`	`mu_vogel`	`n_emptyCurves`	`0`
`diag`	`mu_vogel`	`n_failures`	`0`
`diag`	`mu_vogel`	`n_points`	`41`

compare_vle_etoh_water

tutorials/props/compare/compare_vle_etoh_water



What it does: Ethanol/water T-x-y at 1 atm: ideal vs NRTL model curves OVERLAID with measured VLE data — see the data, pick the model

The story: The canonical "SEE, then DECIDE" props case: ethanol/water T-x-y at 1 atm, with two MODEL curves and the MEASURED data on ONE plot.

Piece 1 – two scan operations, identical but for the activity model: txy_ideal Raoult (gamma = 1) → a smooth lens, NO azeotrope txy_nrtl NRTL → the real minimum-boiling azeotrope Each emits the (x, T_bubble, y_eq) trio → the T-x-y / x-y / T-x / T-y toggle (x-y is the McCabe-Thiele diagram).

Piece 2 – the ‘experimental {}‘ overlay: lists the two ops as ‘models‘, and points ‘dataset‘ at REAL measured points (constant/experiments/...). The GUI draws both model curves + the lab points together; the student SEES that the ideal model misses the azeotrope and NRTL captures it, and picks NRTL from evidence – not from a recommended-by badge.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	txy_ideal	n_emptyCurves	0
diag	txy_ideal	n_failures	0
diag	txy_ideal	n_points	51
diag	txy_nrtl	n_emptyCurves	0
diag	txy_nrtl	n_failures	0
diag	txy_nrtl	n_points	51
diag	txy_unifac	n_emptyCurves	0
diag	txy_unifac	n_failures	0
diag	txy_unifac	n_points	51
diag	consistency_etoh_water	gamma1_inf	4.68506
diag	consistency_etoh_water	gamma2_inf	2.27627
diag	consistency_etoh_water	gd_max_resid	0.0964023
diag	consistency_etoh_water	gd_mean_resid	0.05208
diag	consistency_etoh_water	herington_D	12.0482

. . . and 14 more reference values in the case’s `expected`.

curation

`curate01_water_evidence_partition`

`tutorials/props/curation/curate01_water_evidence_partition`

psat_water_validated vaporPressureFit	**psat_water_refused** vaporPressureFit

What it does: The FIT/HELD-OUT partition contract: the same property fitted twice, differing only in whether evidence was held out – one VALIDATED, one VALIDATION REFUSED

The story: THE FIT / HELD-OUT VALIDATION CONTRACT, and its refusal, side by side.

TWO OPERATIONS. SAME COMPONENT. SAME PROPERTY. SAME TWO DATASETS. The ONLY thing that differs is whether any evidence was held out – which is exactly the variable the contract governs, and isolating it is why this witness does not use two different properties.

psat_water_validated lowT ‘role fit‘ + highT ‘role validation‘ psat_water_refused lowT ‘role fit‘ + highT ‘role fit‘

The first can report a held-out residual because the fitter never received the 325-370 K points. The second cannot, and says so by name instead of presenting its in-sample R2 as though it were an external check:

VALIDATION REFUSED: No independent experimental evidence remains after fitting.

THE ANCHOR IS ENFORCED BY CONSTRUCTION. The partition is parsed and frozen before any fitting code runs, and the fitter is handed only the ‘role fit‘ points. It is not forbidden to move a held-out point into the fit – it is never given one. A rule can be broken by the next edit; a data-flow fact cannot. ‘role‘ has NO default, because a defaulted role would make every dataset a fitting dataset and bring the in-sample problem back invisibly.

WHAT THIS CASE DOES NOT SHOW. Both datasets are generated from the SAME curated Antoine correlation (see either dataset header), so the held-out residual is arithmetic rather than evidence about the physics. This is a STRUCTURAL witness – no number in it is validated against measurement.

Contract: src/propertyOps/EvidencePartition.H

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`psat_water_validated`	`A`	`5.40221`
`diag`	`psat_water_validated`	`B`	`1838.68`
`diag`	`psat_water_validated`	`C`	`-31.737`
`diag`	`psat_water_validated`	`R2`	`1`
`diag`	`psat_water_validated`	`aad_heldout_pct`	`5.95532e-08`
`diag`	`psat_water_validated`	`dHvap_kJ_per_mol`	`43.3787`
`diag`	`psat_water_validated`	`n_heldout`	`10`
`diag`	`psat_water_validated`	`n_points`	`10`
`diag`	`psat_water_validated`	`rms_heldout_log10P`	`2.76024e-10`
`diag`	`psat_water_refused`	`A`	`5.40221`
`diag`	`psat_water_refused`	`B`	`1838.68`
`diag`	`psat_water_refused`	`C`	`-31.737`
`diag`	`psat_water_refused`	`R2`	`1`
`diag`	`psat_water_refused`	`dHvap_kJ_per_mol`	`42.1162`

. . . and 2 more reference values in the case’s `expected`.

curate02_vle_heldout_ethanol_water

tutorials/props/curation/curate02_vle_heldout_ethanol_water

nrtl__etoh__water__heldout
fitParameters

What it does: A binary NRTL pair regressed on eight MEASURED bubble temperatures and scored on three it never saw – the first held-out validation in the corpus against real evidence

The story: curate02 – THE FIT / HELD-OUT CONTRACT ON A BINARY PHASE EQUILIBRIUM.

The NRTL ethanol-water pair is regressed against eight measured bubble temperatures at 101.325 kPa, and then asked to predict THREE it never saw.

WHY THIS CASE EXISTS BESIDE curate01. curate01 proves the machinery: its two halves come from the same Antoine equation, so its held-out residual is round-off and its acceptance band is a numerical criterion, which its own ‘acceptance.origin‘ says out loud. Here the evidence is MEASURED, so the held-out residual is a statement about the MODEL – the first place in the corpus where a fitted binary pair is scored on real evidence it was never handed.

THE SPLIT IS DECLARED AND ARITHMETIC. Every third point of the eleven-point series (x1 = 0.20, 0.50, 0.80) is held out, spread across the composition range rather than cut contiguously, because a model can interpolate well and extrapolate badly and a contiguous cut tests only one of those. The rule was fixed before anything was fitted.

THE ACCEPTANCE BAND IS DECLARED BEFORE THE FIT, WITH ITS ORIGIN, AND IS NOT ADJUSTED AFTERWARDS. Whatever verdict falls out of it is the result. A band chosen after seeing the residual would make the whole contract decorative.

Read the run’s four sections in order: FIT QUALITY is what the model does on what it saw; HELD-OUT EVIDENCE is what it did on what it did not.

TWO THINGS THIS CASE DOES NOT SHOW, BOTH VISIBLE IN ITS OWN GOLDEN.

(1) THE PARAMETERS ARE NOT IDENTIFIABLE. ‘identifiable 0‘, ‘cond_JtJ ~4e11‘ and pairwise correlations of 0.9999 between each a/b pair: eight points at ONE pressure cannot separate the entropic and enthalpic halves of an NRTL parameter, because they only ever appear together over a narrow T span. The model PREDICTS well and the four numbers are individually meaningless – a validated prediction is not a determined parameter set, and a student must be able to read both facts off the same run. This is why the verdict is scoped ‘binaryVLE.T_bubble‘ in the dossier and not to the pair itself.

(2) THE HELD-OUT MARGIN IS NOT LARGE. 0.0737 % against a 0.1 % band is a pass by a factor of 1.4, not by an order of magnitude. The band was declared before the fit and is not adjusted; the margin is reported rather than described as comfortable.

The run also announces an extrapolation: ethanol’s vapour pressure is evaluated at 372 K, outside its declared Trange (273-369). Announced, not judged – it rides the caveat surface like every other extrapolation.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	nrtl_etoh_water_heldout	aad_heldout_pct	0.0737
diag	nrtl_etoh_water_heldout	chi2	0.8097
diag	nrtl_etoh_water_heldout	chi2_in_sample	0.8097
diag	nrtl_etoh_water_heldout	chi2_reduced	0.2024
diag	nrtl_etoh_water_heldout	cond_JtJ	394331596991.7449
diag	nrtl_etoh_water_heldout	converged	1.0000
diag	nrtl_etoh_water_heldout	corr_computed	1.0000
diag	nrtl_etoh_water_heldout	dof	4.0000
diag	nrtl_etoh_water_heldout	identifiable	0.0000
diag	nrtl_etoh_water_heldout	invertible	1.0000
diag	nrtl_etoh_water_heldout	iter	29.0000
diag	nrtl_etoh_water_heldout	max_abs_corr	0.9999
diag	nrtl_etoh_water_heldout	max_abs_resid	0.5825
diag	nrtl_etoh_water_heldout	n_data	8.0000
... and 8 more reference values in the case's `expected`.			

curate03_thermoml_fixture_bubble

tutorials/props/curation/curate03_thermoml_fixture_bubble

thermoml_fixture_bubble
fitParameters

What it does: ThermoML → extract → partition → fit → held-out → dossier, on a SYNTHETIC structural fixture: the plumbing proven end to end, and no physical claim anywhere

The story: curate03 – THE THERMOML PATH, END TO END, ON A STRUCTURAL FIXTURE.

ThermoML .xml → extract-vle → EvidencePartition → FIT only → T_bubble regression → held-out → dossier

EVERY NUMBER HERE IS INVENTED. Both datasets were extracted by ‘bin/choupo-thermoml extract-vle‘ from synthetic fixtures under tutorials/props/curation/fixtures/thermoml/, and each carries ‘provenance { source synthetic; }‘ – so the run ANNOUNCES that its evidence was generated, and the dossier records it per dataset. A reader cannot arrive at the verdict without having been told what the numbers are.

WHAT THIS CASE PROVES: that a ThermoML file crosses into the evidence contract without losing its identity – archive file, DOI, system, pressure and checksum all survive into the dossier – and that the partition, the regression and the verdict work on evidence that arrived through the parser rather than being typed into a case.

WHAT IT DOES NOT PROVE: anything about ethanol, water, or NRTL. The acceptance band below is a PLUMBING tolerance and says so. The scientific witness on measured evidence is curate02, next door.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	thermoml_fixture_bubble	aad_heldout_pct	0.0529
diag	thermoml_fixture_bubble	chi2	0.6692
diag	thermoml_fixture_bubble	chi2_in_sample	0.6692
diag	thermoml_fixture_bubble	chi2_reduced	0.2231
diag	thermoml_fixture_bubble	cond_JtJ	272780104373.5624
diag	thermoml_fixture_bubble	converged	1.0000
diag	thermoml_fixture_bubble	corr_computed	1.0000
diag	thermoml_fixture_bubble	dof	3.0000
diag	thermoml_fixture_bubble	identifiable	0.0000
diag	thermoml_fixture_bubble	invertible	1.0000
diag	thermoml_fixture_bubble	iter	42.0000
diag	thermoml_fixture_bubble	max_abs_corr	1.0000
diag	thermoml_fixture_bubble	max_abs_resid	0.6562
diag	thermoml_fixture_bubble	n_data	7.0000

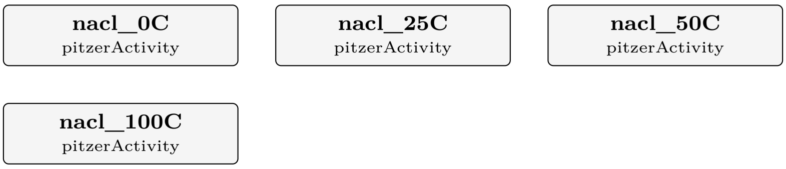
. . . and 8 more reference values in the case's `expected`.

electrolyte

The theory you need. Ions interact at long range (Debye–Hückel, $\sqrt{I}$ law) and short range (ion–ion, ion–solvent). Choupo's ladder: Davies (charge-only, $I \leq 0.5$), single-salt Pitzer ($\beta_0, \beta_1, C^{\varphi}$; to $I \sim 6$, and with the Silvester–Pitzer T-functions to 300°C), multi-ion Pitzer–HMW (per-ion, mixing terms θ, ψ – seawater-class brines), and eNRTL (local-composition; single-salt, mixed-solvent, and the 2009 symmetric multi-salt with both published and Gibbs–Duhem-consistent formulations). Speciation solves mass action + charge balance by Newton in log-molality space; saturation indices SI = $\log(\mathrm{IAP}/K_{\mathrm{sp}})$ flag scaling. Expect: models to DISAGREE above their validity (γ divergence is the lesson, e.g. Davies vs Pitzer at RO recoveries), and every activity model to announce itself. *Full derivation: Theory Guide, the electrolyte + speciation chapters.*

`archer01_nacl_cold_to_hot`

`tutorials/props/electrolyte/archer01_nacl_cold_to_hot`



What it does: NaCl Pitzer 0-100 C vs Archer 1992 – the cold edge the freezing curve stands on

The story: The catalogue's NaCl Pitzer surface (Appelo 25 C parameters + SP77 T-functions) against Archer, J. Phys. Chem. Ref. Data 21 (1992) 793, DOI 10.1063/1.555915, Tables 9 (phi) and 10 (gamma_pm) – HIS fitted equation's check values ("presented only as a means to test the validity of coding", the paper's own words), at 273, 298, 323 and 373 K.

WHY THE 273 K ROW IS THE POINT: the SP77 temperature extension was fitted ABOVE 25 C, and the freezing-point curve (fpd01_nacl_freezing) leans on that surface BELOW 0 C as an announced extrapolation. This case measures what the extrapolation costs at the last anchor before the ice field – the op announces the T-window violation at 273 K, and the AAD against Archer says whether the announced risk is a real one at the cold edge.

WHAT THIS COMPARISON IS, said precisely: Archer's tables are values of an independent CRITICAL FIT (his global equation, 250-600 K), not raw measurements, and his database overlaps the corpora SP77 drew on. Agreement here is CROSS-EVALUATION CONSISTENCY between two independent fits of overlapping data – stronger than fit-consistency (our parameters never saw Archer's equation), weaker than agreement with an independent measurement. The Hamer & Wu case (pitzer_gamma_hamer_wu) plays the same role at 25 C; this case extends it across T.

Temperatures are the table's printed values (273 K, not 273.15). If Archer meant T+0.15 K the shift in gamma_pm is below 2e-4 – inside the quoted AADs either way, and stated here so the reading is a recorded choice, not an accident.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	nacl_0C	aw_1m	0.9674
diag	nacl_0C	gamma_AAD_pct	1.0546
diag	nacl_0C	gamma_nMeas	5.0000
diag	nacl_0C	gamma_pm_1m	0.6418
diag	nacl_0C	gamma_pm_3m	0.6649
diag	nacl_0C	phi_1m	0.9204
diag	nacl_0C	phi_AAD_pct	0.3706
diag	nacl_0C	phi_nMeas	5.0000
diag	nacl_25C	aw_1m	0.9668
diag	nacl_25C	gamma_AAD_pct	0.1865
diag	nacl_25C	gamma_nMeas	5.0000
diag	nacl_25C	gamma_pm_1m	0.6571
diag	nacl_25C	gamma_pm_3m	0.7138
diag	nacl_25C	phi_1m	0.9363
...and 18 more reference values in the case's `expected`.			

bjerrum01_carbonate_pH

tutorials/props/electrolyte/bjerrum01_carbonate_pH

acid__5p0 speciate	**acid__3p0** speciate	**acid__2p0** speciate
acid__1p5 speciate	**acid__1p0** speciate	**acid__0p6** speciate
acid__0p35 speciate	**acid__0p2** speciate	**acid__0p1** speciate
acid__0p05 speciate	**base__0p0** speciate	**base__0p1** speciate

What it does: Carbonate speciation across the pH range – the Bjerrum plot, titrated

The story: THE SPECIATION OF A CARBONATE SOLUTION ACROSS THE WHOLE pH RANGE – the diagram every textbook draws, built the way Choupo has to build it.

THE ONE THING THAT MAKES THIS DIFFERENT FROM THE TEXTBOOK VERSION: here the pH is NOT an axis you set – and that is a CHOICE this case makes, in one word, which you can read three screens down. A textbook draws the fractions against pH by substituting pH into the mass-action expressions, which is arithmetic and always works. Choupo can do that too: ‘pH 7.8;‘ is a legal declaration, it fixes [H+], and it DROPS the electroneutrality row – the run then reports the net charge the composition carries (‘pH GIVEN – this is the net charge the analysis carries‘) instead of forcing it to zero. That mode exists because a laboratory sheet reports a MEASURED pH, which is a datum. It is not what this case wants. Every op below declares ‘pH solve;‘ instead, so [H+] joins the unknowns and the charge balance is the equation that closes the system. The price of that choice is the whole lesson: each point had to be a beaker somebody could weigh out, the 44 of them differ only in what is IN the beaker, and the x-axis is a RESULT, exactly as it is in a laboratory.

WHAT IS ACTUALLY IN EACH BEAKER. Total carbonate is held at 1 mmol/kg throughout. The 44 points differ only in how much strong acid or strong base sits beside it, and each one is a real neutral mixture somebody could weigh out:

Cl / C_T = 5 ... 0.05 hydrochloric acid added to dissolved CO2 Na / C_T = 0 dissolved CO2 alone (H2CO3) Na / C_T = 1 sodium bicarbonate (NaHCO3) Na / C_T = 2 sodium carbonate (Na2CO3) Na / C_T = 2 ... 8 sodium carbonate + caustic soda

That is what ‘totalsBasis stoichiometric;‘ declares below: these totals are neutral-molecule feeds, so the water-analysis charge-imbalance check – which is right for a LAB SHEET, where an imbalance means somebody mismeasured – does not apply. Here the imbalance IS the titrant, and the electroneutrality equation IS the weak-acid pH. Declaring it changes no number: it changes what the engine says about them.

WHAT TO LOOK FOR, in the order a student should look:

1. THE TWO CROSSOVERS LAND ON THE TWO pK’s, AND NOBODY PUT THEM THERE. CO2(aq) and HCO3- are equal at pH 6.341; the record’s log K1 is 6.352. HCO3- and CO3– are equal at pH 10.262; log K2 is 10.329. Both crossover pH’s are INTERPOLATED between the two beakers that bracket them – the axis is 44 sampled points, not a continuum, and no beaker lands exactly on a crossing. Neither pK was used as an input to any point – they are the mass- action constants inside the network, and the crossover is where the titration happens to arrive. The small shift DOWN in both is the activity coefficients: gamma_CO3 is 0.90 at the first crossover and 0.81 at the second, while a textbook plot takes every gamma as 1.

2. THE THREE FRACTIONS DO NOT SUM TO ONE. They dip to 0.99966 through the middle of the range, and the missing carbon is NOT round-off: it is ‘NaHCO3aq‘, the sodium-bicarbonate ION PAIR, which the curated chemistry carries and the three-curve drawing has no place for. The titrant is not a spectator. Look at the ‘m_NaHCO3aq‘ diagnostic – and note where it is ABSENT: the two beakers with no

sodium in them (the acid side, and Na/C_T = 0) have no such species to report, and the engine REFUSES to report a species that is not in the table rather than printing a zero. Absent and zero are different claims.

3. THE ACTIVITY COEFFICIENT OF THE DIVALENT ION MAKES A U. gamma_CO3 runs 0.74 at the acid end, up to 0.98 near Na/C_T = 0 where the ionic strength is lowest, and back down to 0.67 at the alkaline end – because BOTH ends of a titration are loaded with strong electrolyte and only the middle is dilute. A textbook diagram takes every gamma as 1 at every pH, which is the one assumption that is worst exactly where the diagram is usually read.

4. THE SATURATION INDICES ARE THERE ALL ALONG. Every point reports SI_nahcolite / SI_natron / SI_trona – how far the solution is from precipitating a sodium carbonate mineral. At 1 mmol/kg they stay far below zero, which is why this stays a clean acid-base diagram; make C_T a hundred times larger and the diagram acquires a ceiling.

THE CONSTANTS, and their lineage: log K1 = 6.352, log K2 = 10.329 and Kw = 1e-14 at 25 C, from the USGS PHREEQC database (public domain), which is where data/standards/chemistry/{CO2aq,CO3}-formation.dat and water-dissociation.dat come from. Each carries the van't Hoff dH and the full analytic log K(T), so this whole diagram can be redrawn at another temperature and both pK's will move – change 'temperature' below and watch which crossover moves further.

NOT VALIDATED AGAINST MEASUREMENT. This case is a STRUCTURAL witness: it shows that the engine's speciation network, its activity model and its charge balance compose into the diagram the chemistry predicts. No titration data is compared against; the two crossovers landing near the two pK's is an internal consistency check on the assembly, not agreement with an experiment.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	acid_5p0	I	0.00499268
diag	acid_5p0	aw	0.999802
diag	acid_5p0	chargeImposed	1
diag	acid_5p0	chargeResidual	-3.02883e-16
diag	acid_5p0	gamma_CO2aq	1
diag	acid_5p0	gamma_CO3	0.738626
diag	acid_5p0	gamma_HCO3	0.927056
diag	acid_5p0	m_CO2aq	0.000999896
diag	acid_5p0	m_CO3	1.31694e-15
diag	acid_5p0	m_HCO3	1.03591e-07
diag	acid_5p0	pH	2.33456
diag	acid_3p0	I	0.0029974
diag	acid_3p0	aw	0.999874
diag	acid_3p0	chargeImposed	1

... and 635 more reference values in the case's `expected`*.*

composition01_nacl

tutorials/props/electrolyte/composition01_nacl

naclWater
speciate

What it does: Apparent-salt input: the analysis is given as composition { NaCl 2.0 mol/kg }, expanded to Na + Cl ion totals through NaCl's dissociatesTo, with electroneutrality validated (Sum nu∗charge = 0). Byte-identical to writing totals { Na 2.0; Cl 2.0 } by hand.

The speciate op expands each salt to ion totals through its dissociatesTo (NaCl → Na 1, Cl 1), and validates electroneutrality (Sum nu∗charge = 0) so a formulated-salts input can never silently unbalance charge. This is byte-identical to writing the ions by hand (totals { Na 2.0; Cl 2.0 }): SI_halite -0.953, feed charge imbalance 0.00%.

composition{} is the apparent-salt input mode; analyticalTotals{} / totals{} stay as the mode for waters measured directly in ions/masters (roadmap Phase B).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	naclWater	I	2
diag	naclWater	SI_halite	-0.953442
diag	naclWater	aw	0.930475
diag	naclWater	gamma_Cl	1.01683
diag	naclWater	gamma_Na	1.01683
diag	naclWater	m_Cl	2
diag	naclWater	m_Na	2
diag	naclWater	pH	7.13303
diag	naclWater	chargeImposed	1
diag	naclWater	chargeResidual	-6.39009e-17

davies01_band_edge

tutorials/props/electrolyte/davies01_band_edge

davies_m0p05 speciate	**hmw_m0p05** speciate	**davies_m0p1** speciate
hmw_m0p1 speciate	**davies_m0p25** speciate	**hmw_m0p25** speciate
davies_m0p5 speciate	**hmw_m0p5** speciate	**davies_m1p0** speciate
hmw_m1p0 speciate	**davies_m2p0** speciate	**hmw_m2p0** speciate

What it does: The Davies band edge, MEASURED: gamma_pm and a_w for NaCl under Davies beside PitzerHMW on identical brines, m = 0.05..6 mol/kg – the cross-model curve behind every 'indicative beyond I ~ 0.5' advisory

The story: THE DAVIES BAND EDGE, measured instead of remembered.

Same NaCl brine, same totals, same solved pH – speciated twice per rung: once under Davies (charge-only, the teaching rung; phi = 1 water activity) and once under PitzerHMW (per-ion virials; rigorous phi). What this measures is CROSS-MODEL divergence between two of Choupo's own engines – weaker than agreement with an independent measurement, and said so; the Pitzer surface itself is anchored elsewhere (pitzer01_nacl, archer01_nacl_cold_to_hot, pitzer_gamma_hamer_wu).

WHY IT EXISTS: every electrolyte run above I ~ 0.5 mol/kg carries the advisory "Davies ... INDICATIVE"; the marcilla01/02 anchor contacts put numbers on what that costs at saturation (halite at 3.65 vs the measured 6.14 mol/kg). This case turns the advisory into a CURVE – per rung, gamma_Na, gamma_Cl and a_w under both closures, side by side. No threshold is invented: the divergence grows smoothly and there is no principled cut, so the table reports and the reader judges (the same posture as the neglected-dCp advisory in SolidPhase).

Same NaCl brine speciated twice per rung: Davies (teaching rung, charge-only, $\varphi = 1$) beside PitzerHMW (per-ion virials), 298.15 K. Cross-model divergence between two in-tree engines — weaker than an independent measurement and said so; the Pitzer surface itself is anchored elsewhere (pitzer01_nacl, archer01_nacl_cold_to_hot, pitzer_gamma_hamer_wu).

∗ The "I ~ 0.5 trust band" every advisory quotes is real: the $\gamma\pm$ error crosses ~2–8 % over m = 0.25–0.5 and explodes beyond it (Davies' +0.3·I term has no physics above the band — it is the fit's tail). ∗ At m = 6, Davies reports SI_halite = +1.10 where PitzerHMW reports -0.022 — the rigorous rung sits essentially AT saturation beside Archer's peritectic (6.096 mol/kg), and its a_w = 0.7592 beside the literature $\approx$ 0.753. This is the whole quantitative story of the marcilla01/02 anchor gaps (Davies saturating at 3.65 vs 6.14 mol/kg; the too-wet organic from a_w $\approx$ 0.88): the model, not the machinery.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	davies_m0p05	I	0.0500001
diag	davies_m0p05	SI_halite	-4.34316
diag	davies_m0p05	aw	0.9982
diag	davies_m0p05	gamma_Cl	0.821203
diag	davies_m0p05	gamma_Na	0.821203
diag	davies_m0p05	m_Cl	0.05
diag	davies_m0p05	m_Na	0.05
diag	davies_m0p05	pH	7.00292
diag	hmw_m0p05	I	0.0500001
diag	hmw_m0p05	SI_halite	-4.34351
diag	hmw_m0p05	aw	0.998302
diag	hmw_m0p05	gamma_Cl	0.820867
diag	hmw_m0p05	gamma_Na	0.820867
diag	hmw_m0p05	m_Cl	0.05
... and 146 more reference values in the case's `expected`*.*			

edwards01_sour_water_activity

tutorials/props/electrolyte/edwards01_sour_water_activity

sourWater__edwards speciate	sourWater__davies speciate

What it does: Edwards et al. (1978) activity model on a sour-water NH3/CO2 brine – the molecular solutes get a real gamma from the model their parameters were regressed in, where Davies can only return 1.

The story: the WITNESS for the ‘edwardsPitzer‘ activity model.

WHAT THIS CASE IS FOR. Choupo already carries two aqueous activity models, and each fails this fluid in its own way:

‘davies‘ is a charge-only correlation. Every singly-charged ion gets the same gamma and every NEUTRAL solute gets gamma = 1 – exactly, by construction, at every ionic strength. A sour-water stripper’s whole difficulty is the volatility of dissolved NH3 and CO2, i.e. of two NEUTRAL molecules, and Davies cannot say anything about either. ‘pitzerHMW‘ is the Harvie-Moller-Weare parameterisation: a DIFFERENT truncation, with a different parameter set (beta2, C-phi, theta, psi), fitted to strong electrolytes. Its numbers are not Edwards’ numbers, and Edwards’ numbers are not valid inside its equation.

‘edwardsPitzer‘ is the truncation Edwards, Maurer, Newman & Prausnitz (AIChE J. 24(6):966-976, 1978) REGRESSED HIS NH3/CO2/H2S/SO2 PARAMETERS IN: Pitzer’s expansion cut after the second virial coefficient, extended to molecular solutes. Using his parameters anywhere else would be quoting his numbers under someone else’s equation.

THE COMPOSITION is the sour-water region his Table 7 anchors: ammonia in large excess over carbon dioxide, the carbonate and carbamate networks both live, at an ionic strength around 1 mol/kg where a dilute correlation has no business being trusted.

WHAT TO READ IN THE OUTPUT. Run the two operations side by side and compare ‘gamma_NH3aq‘ and ‘gamma_CO2aq‘:

under ‘davies‘ they are 1.000000 and 1.000000. Not approximately – exactly, because the model’s gamma is a function of charge alone and the charge is zero. A student who has only ever seen Davies may not realise this is a STRUCTURAL silence rather than a small correction. under ‘edwardsPitzer‘ they are 1.0224 and 1.0101: both SALTED OUT, NH3 rather more than CO2. Both rise because both Table 4 self-terms are positive at 25 C – Eq 14 gives beta0(NH3,NH3) = 0.01522 and beta0(CO2,CO2) = 0.00822 kg/mol – and NH3 rises further because it is the more abundant of the two by three orders of magnitude, so its own self-term dominates its Eq 8 sum.

THE SPECIATION MOVES TOO, and by far more than the gammas: the answer is I = 2.78 against Davies’ 1.78, with m_NH3aq 0.41 against 1.01. That is not a molecular effect at all – it is the IONIC one. Davies at I ~ 2 returns gamma_NH4 = 0.956, because its bracket saturates and then turns back up; Edwards’ Eq 8 returns 0.301 at the same charge. The NH4+/NH3aq equilibrium reads those gammas, so the split follows. The lesson is the one the advisory prints on the Davies run: a dilute correlation used far outside its band does not merely lose accuracy, it changes the answer’s SHAPE.

Neither number here is validated against measurement – see below.

WHAT THIS CASE IS *NOT*. It is not the Table 7 anchor. Reproducing Table 7 needs the vapour side – Henry’s constants on the paper’s own convention, the two-phase equilibrium, the total pressure – and that slice is BUILT: ‘edwards02_table7_vle‘ (2026-08-23; record in docs/design/sour-water-stripper-scope.md section 6c). This case validates the ACTIVITY SURFACE alone, which is the piece the model owns.

WHAT IS AND IS NOT VALIDATED HERE. This case is a STRUCTURAL witness: it proves the model is wired, that its five parameter rules fire and are labelled, and that a neutral solute gets a gamma. It does NOT validate any number against data – there is no measurement in it. The kernel itself is separately checked against two closed forms the paper supplies (the Eq 8a low-dissociation limit and the Debye-Huckel limiting law), and the measured-vs-computed comparison is Table 7 – edwards02_table7_vle. A green run here means "the machine is right", never "the answer is right".

HONESTY ABOUT THE PARAMETERS. Edwards measured exactly one family: beta0 for LIKE molecule pairs (Eq 14, Table 4). Every other entry in the matrix Eq 8 sums over is ESTIMATED from those by his Eqs 21-24, and the paper calls both of its parameter tables "preliminary results subject to change as more and better data become available". The engine prints, at verbosity >= 3, WHICH RULE produced each number – one measured family and four estimation routes – so nothing here can be mistaken for a measured constant.

DATA SCOPE: SEALED (bin/choupo-import). The case carries its own species, reactions and EdwardsPitzer parameter records under constant/, each named and sha256-verified by constant/propertyManifest, and the runtime is FORBIDDEN the installation catalogue.

Sealing this case is where the importer's dependency closure was found not to know the EdwardsPitzer parameter home. The first seal kept 9 of the 28 pair parameters – the Bronsted like-sign zeros, which need no record at all – and the case still RAN, still converged, and answered with 19 terms missing: gamma_HCO3 and gamma_NH4 identical (the charge-only Debye-Huckel fallback) and both neutral gammas exactly 1, which is what a Davies run prints. Nothing refused. The importer now compares the staged sealed run against this case's own 'expected' golden, because running is not agreeing.

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`sourWater_edwards`	`I`	`2.77652`
`diag`	`sourWater_edwards`	`aw`	`0.954591`
`diag`	`sourWater_edwards`	`gamma_CO2aq`	`1.01011`
`diag`	`sourWater_edwards`	`gamma_CO3`	`0.0181147`
`diag`	`sourWater_edwards`	`gamma_HCO3`	`0.227994`
`diag`	`sourWater_edwards`	`gamma_NH3aq`	`1.02239`
`diag`	`sourWater_edwards`	`gamma_NH4`	`0.30091`
`diag`	`sourWater_edwards`	`m_CO2aq`	`0.000484785`
`diag`	`sourWater_edwards`	`m_CO3`	`0.537903`
`diag`	`sourWater_edwards`	`m_HCO3`	`0.911612`
`diag`	`sourWater_edwards`	`m_NH3aq`	`0.41019`
`diag`	`sourWater_edwards`	`m_NH4`	`2.48981`
`diag`	`sourWater_edwards`	`pH`	`9`
`diag`	`sourWater_davies`	`I`	`1.77822`
. . . and 16 more reference values in the case's `expected`.			

edwards02_table7_vle

tutorials/props/electrolyte/edwards02_table7_vle

table7__row1 speciate | **table7__row2** speciate | **table7__row3** speciate

row1__nocarbamate speciate

What it does: Edwards (1978) Table 7 anchor: NH3-CO2-H2O VLE at 100 degC – the paper's own predictions reproduced from its own equations, the experiment quoted as context.

The story: the LITERATURE ANCHOR for the edwardsPitzer model: Table 7 of Edwards, Maurer, Newman & Prausnitz, AIChE J. 24(6):966-976 (1978) – NH3-CO2-H2O at 100 degC, three points.

edwards01 is the STRUCTURAL witness: it proves the activity surface is wired, and its own header says none of its numbers is validated against anything. This case is the validation it points at. For each Table 7 point the paper states the total molalities of ammonia and carbon dioxide in the LIQUID and predicts the speciation (molecular NH3 and CO2, ionic strength) and the VAPOUR above it (y_NH3, y_CO2, total pressure). The whole model stack here is the paper's own:

liquid activities Eq 8/10 (edwardsPitzer, the case surface) equilibria K(T) Eq 7 / Table 1 + Eq 20 (case-local chemistry/) Henry's constants Eq 13 / Table 3 (parameters/EdwardsPitzer/henry-∗) pressure correction Eq 11 / Table 2 (vInfinity in the same records) vapour side Eq 5/6 (the speciate op's vapour{} block)

WHAT IS COMPARED, AND AGAINST WHAT. Table 7 carries TWO different columns and conflating them would be dishonest:

the paper's own PREDICTIONS (column I) – a code-to-code comparison. Those are the golden values here, with ONE declared gap: the paper computes vapour-phase fugacity coefficients from a model its transcription does not carry, so this case declares 'fugacity ideal;' (phi = 1) and the residual disagreement is expected to be of the size of that correction (a few percent at 2-3 atm), plus the numerical difference between two implementations of Eqs 7-24.

the EXPERIMENT (Otsuka et al. 1960) – quoted below as CONTEXT, never as a target. The paper's OWN agreement with it is rough (first row: y_NH3 0.066 measured vs 0.123 predicted – a factor of two) and the paper says so plainly. A student reading this case sees a correlation reproduced closely and its real-world agreement stated honestly, which is the opposite of a tool that prints one number.

Table 7, the paper's column I (the golden targets here): row mNH3tot mCO2tot m_NH3 m_CO2 I y_NH3 y_CO2 P[atm] 1 2.90 1.45 1.245 0.014 1.50 0.123 0.494 2.51 2 3.71 1.14 2.279 0.0056 1.26 0.293 0.230 1.98 3 4.30 0.907 3.087 0.0030 1.01 0.407 0.116 1.97 The experiment (Otsuka 1960, context only): 1 0.066 0.506 3.15 2 0.274 0.202 2.08 3 0.355 0.095 2.00

THE CARBAMATE ENTERS THE CATALOGUE HERE (case-local): NH2COO- with Eq 20's equilibrium. Without it, an ammonia-rich carbonated water reports molecular molalities the paper's own predictions contradict – it holds a substantial share of the dissolved CO2 at these conditions. Eq 20 is fitted 20-60 degC and used at 100 degC exactly as the paper uses it, with the extrapolation ANNOUNCED and the paper's own sensitivity argument in the record header (halving K20 moves P by under 2 %).

DECLARED APPROXIMATIONS, each announced by the run: 'fugacity ideal;' – phi = 1 on all vapour species (see above); the solvent Poynting factor of Eq 6 is omitted (< 0.2 % below 4 atm); Eq 20 extrapolated 60 → 100 degC (the paper's own move); water's Psat comes from the case catalogue's own curated Antoine, not from a value the paper does not print.

DATA SCOPE: sealed (bin/choupo-import); every record named and verified by constant/propertyManifest.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	table7_row1	I	1.547
diag	table7_row1	P_atm	2.51337
diag	table7_row1	aw	0.946304
diag	table7_row1	chargeImposed	1
diag	table7_row1	chargeResidual	2.5541e-13
diag	table7_row1	gamma_CO2aq	0.919147
diag	table7_row1	gamma_CO3	0.022685
diag	table7_row1	gamma_HCO3	0.268386
diag	table7_row1	gamma_NH2COO	0.528413
diag	table7_row1	gamma_NH3aq	0.989831
diag	table7_row1	gamma_NH4	0.318905
diag	table7_row1	m_CO2aq	0.0141073
diag	table7_row1	m_CO3	0.0555284
diag	table7_row1	m_HCO3	1.09498
… and 99 more reference values in the case's `expected`.			

edwards03_table8_vle

tutorials/props/electrolyte/edwards03_table8_vle

table8__row1	table8__row2	table8__row3
speciate	speciate	speciate

What it does: Edwards (1978) Table 8 anchor: NH3-H2S-H2O VLE at 80 degC – the paper's own predictions reproduced from its own equations, the experiment quoted as context.

The story: the SECOND literature anchor for the Edwards (1978) model stack: Table 8 of the paper, ammonia + hydrogen sulfide in water at 80 degC, against the paper's OWN prediction column I.

WHY A SECOND ANCHOR. edwards02 (Table 7) pins the NH3-CO2 system, where the difficulty is the carbonate/carbamate network. Table 8 exercises what Table 7 cannot: the SULFIDE family (the mi-H2Saq-∗ molecule-ion records curated 2026-08-24 from the paper's Table 6, including its two TABULATED ZEROS), a different temperature (80 degC, so every T-polynomial is read off its 25 degC calibration point), and ionic strengths up to 7.8 mol/kg – beyond the 6-molal Pitzer validity the paper itself quotes. Rows 2 and 3 are code-to-code targets and NOT physical claims; the paper runs them and says so ("if ionization is extensive, the upper limit is necessarily lower").

WHAT IS COMPARED, AND AGAINST WHAT. Exactly the edwards02 posture:

the paper's own PREDICTIONS (column I: all beta0 from Tables 4 and 6, the configuration these curated records carry) are the anchor targets; the EXPERIMENT (Miles & Wilson 1975) is CONTEXT, never a target. The paper's own worst multisolute disagreement lives here: rows 2 and 3 measure 16.50 and 13.20 atm against predictions of 12.91 and 12.64 atm (22-28 %), and its Table 9 sensitivity analysis shows the liquid-composition error needed to close such gaps.

THE CHEMISTRY IS THE PAPER'S OWN: case-local constant/chemistry/ carries Table 1 (H2O, NH3 and H2S rows) in the formation direction, each header recording the derivation. The H2S formation logK25 is 7.0033 against the independently curated PHREEQC 6.994 – two primaries 0.009 apart, a finding, not an error.

DECLARED GAPS, the same list as edwards02 plus none: 'fugacity ideal;' – the paper's own vapour-phi model is not transcribed; the residual is expected to carry that correction (largest on rows 2-3 at 13-16 atm); the solvent Poynting factor of Eq 6 is omitted; water's Psat comes from the case catalogue's own curated Antoine.

DATA SCOPE: sealed (bin/choupo-import); every record named and verified by constant/propertyManifest.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	table8_row1	I	1.13632
diag	table8_row1	P_atm	1.10768
diag	table8_row1	aw	0.896601
diag	table8_row1	chargeImposed	1
diag	table8_row1	chargeResidual	-4.35684e-17
diag	table8_row1	gamma_H2Saq	1.08209
diag	table8_row1	gamma_HS	0.516824
diag	table8_row1	gamma_NH3aq	1.06377
diag	table8_row1	gamma_NH4	0.531883
diag	table8_row1	m_H2Saq	0.0038303
diag	table8_row1	m_HS	1.13617
diag	table8_row1	m_NH3aq	3.97368
diag	table8_row1	m_NH4	1.13632
diag	table8_row1	pH	8.67302
. . . and 64 more reference values in the case's `expected`.			

`enrtl_mixed_nacl_ethanol_esteso`

`tutorials/props/electrolyte/enrtl_mixed_nacl_ethanol_esteso`

esteso
enrtlMixedSolvent

What it does: VALIDATION of the generalized segment-based eNRTL (Chen & Song 2004, AIChE J 50:1928, DOI 10.1002/aic.10151) against the PRIMARY measured gamma_pm of NaCl in ethanol-water at 25 C (Esteso et al., J. Solution Chem. 18 (1989) 277, emf cell, 0-90 wt% ethanol). ZERO refit: every tau/X:Z is the published Table 2/4/7 value; A_phi per composition is Esteso's measured Table II. Headline AAD ~4.3% over 47 points (paper quotes RMS 0.026 on gamma); ~1% in pure water, ~1.2% at 70 wt%. The +8% at 80/90 wt% is DECLARED physics the complete-dissociation model cannot carry: Esteso measures ion association K_A = 30-46 there (his Table V).

The story: can eNRTL predict a salt's activity in the antisolvent mixture?

The SAME physics the antisolvent crystalliser leans on – gamma_pm of NaCl as ethanol pushes the water out – validated against the PRIMARY dataset:

MODEL Chen & Song, "Generalized electrolyte-NRTL model for mixed-solvent electrolyte systems", AIChE J 50(8):1928-1941 (2004), DOI 10.1002/aic.10151. Ethanol = 1.811 (-C2H4-) + 0.609 (-OH) segments (Table 4); ion<→OH taus = the aqueous NaCl pair (Table 2); ion<→C2H4 taus from Table 7. ZERO refit here. DATA Esteso, Gonzalez-Diaz, Hernandez-Luis & Fernandez-Merida, "Activity coefficients for NaCl in ethanol-water mixtures at 25 C", J. Solution Chem. 18(3):277-288 (1989). Table I (emf, molal scale, per-solvent reference). A_phi per composition = Esteso Table II (measured dielectric + density). 0 wt% gate: Robinson & Stokes, Electrolyte Solutions, 2nd ed. (1959), NaCl appendix table; A_phi(water, 25 C) = 0.391.

WHAT THE NUMBERS SAY (locked by the golden): total AAD ~4.3% over 47 points; pure water ~1.0%; 70 wt% ethanol ~1.2%. 80/90 wt% sit +8% systematically – Esteso's own chemical-model fit finds ion-pair association K_A = 30-46 there (Table V), physics a complete- dissociation model cannot represent. Declared, not tuned away.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	esteso	AAD_total_pct	4.3
diag	esteso	AAD_wt0_pct	1
diag	esteso	AAD_wt20_pct	5.3
diag	esteso	AAD_wt40_pct	5.3
diag	esteso	AAD_wt60_pct	2.7
diag	esteso	AAD_wt70_pct	1.2
diag	esteso	AAD_wt80_pct	7.8
diag	esteso	AAD_wt90_pct	7.8
diag	esteso	n_points	47

`enrtl_multisalt_nacl_kcl`

`tutorials/props/electrolyte/enrtl_multisalt_nacl_kcl`

harned_nacl_kcl enrtlMultiSalt	**harned_nacl_kcl_refined** enrtlMultiSalt

What it does: Multi-salt symmetric eNRTL (Song & Chen 2009) – NaCl+KCl Harned sweep at 4 mol/kg

The story: THE MULTI-SALT eNRTL ARRIVES: the multicomponent symmetric electrolyte NRTL of Song & Chen 2009 (Ind. Eng. Chem. Res. 48, 7788, DOI 10.1021/ie9004578) on the classic Harned experiment – NaCl + KCl sharing water at a CONSTANT 4 mol/kg total molality, gamma_pm of BOTH salts as KCl replaces NaCl. This is the paper's own demonstration system (its Figures 1-2).

What the run shows (all golden-locked): gamma_pm(NaCl) slides from its pure-solution value to its TRACE value in 4 m KCl, and vice versa – both ln gamma curves are NEARLY LINEAR in the salt fraction: Harned's rule, EMERGING from the model rather than being assumed (R^2 > 0.99 with zero electrolyte-electrolyte parameters). The PURE endpoints reproduce the validated single-salt eNRTL kernel exactly (the reduction pin ~1e-15): the multicomponent formulation collapses correctly. The pair parameters are the EXACT Chen & Evans 1986 Table 1 values the paper uses (case-local constant/electrolyte/enrtl.dat, cited). The standards K-Cl.dat carries an INDEPENDENT refit to Hamer & Wu 1972 that lands within 0.2 % of the 1986 values – two roads, one answer. The 1986 paper itself (Table 2) validates tauEE = 0 for NaCl-KCl: sigma = 0.019 % on the Covington 1968 mixture osmotic data with the salt-salt energies set to zero – this case's default is the canonical choice. HONESTY, printed by the op: the published multi-electrolyte eNRTL activity-coefficient expressions are NOT exactly Gibbs-Duhem consistent (the composition-dependent mixing rules are differentiated as constants). The op's FD oracle quantifies that inconsistency at the mid-sweep state (gibbsDuhem_dev_mid) next to the pure-salt state where the same oracle sits at ~1e-9. THE REFINED FORMULATION SHIPS TOO (second op below): gamma as the EXACT derivative of the same Gex – the chain rule kept through the composition-dependent mixing rules (the Bollas, Chen & Barton 2008 idea on the symmetric form). Its oracle closes (~1e-9) at every composition; both reduce EXACTLY to the same single-salt kernel at the endpoints. Mid-sweep they differ by |dln gamma_pm| ~ 0.041 (~4 % in gamma), and the trace limits differ visibly (NaCl trace 0.617 published vs 0.678 consistent). WHICH is closer to nature is a question for MIXTURE data (Covington 1968 / Robinson 1961) – this case states the facts and locks both; it does not pick a winner without measurements.

tauEE (the same-anion electrolyte-electrolyte parameter tau_NaCl,KCl of the paper's Figures 1-2) defaults to 0; set it to +-1 to reproduce the paper's sensitivity fan.

THE EXPERIMENTAL VERDICT (Robinson 1961, J. Phys. Chem. 65, 662, DOI 10.1021/j100822a016 – isopiestic NaCl-KCl mixtures, Harned coefficients at M ~ 4.4 mol/kg: alpha_NaCl = +0.0243, alpha_KCl = -0.0091; the op now emits harned_alpha_1/2 on the same convention, golden-locked): alpha_NaCl: published +0.0285 (+17 %) consistent +0.0180 (-26 %) alpha_KCl: published -0.0256 (2.8x) consistent -0.0187 (2.1x) With tauEE = 0, PUBLISHED wins on the dominant NaCl coefficient; BOTH formulations overstate the (small) effect on KCl.

CROSS-CHECK vs THE REFINED-eNRTL PAPER (Bollas, Chen & Barton 2008, AIChE J 54, 1608, DOI 10.1002/aic.11485 – their Figure 1 is THIS system, 4 m, same 1986 parameters, Robinson's points overlaid): their Fig 1a (original eNRTL) matches our 'published' to the pixel (NaCl solids at x=1 for tau = 0/0.25/0.50: ours -0.216/-0.186/-0.157 vs read -0.215/-0.185/-0.15; KCl dashed at x=0: -0.144/-0.183 vs -0.145/-0.185); their Fig 1b (refined) shows the SIGNATURE our 'consistent' shows: the tau fan nearly COLLAPSES (exact derivatives are almost insensitive to tau_EE: ours moves only -0.176 → -0.184 for tau 0 → 0.5, where published moves -0.216 → -0.157), and the KCl trace lands at ~-0.16 (ours -0.163). Their refinement differentiates the 1986 UNSYMMETRIC Gex; ours differentiates the 2009 SYMMETRIC one – same idea, sibling surfaces, same qualitative physics, near-identical aqueous numbers. A single fitted tauEE cannot close both (best compromise tau ~ 0.29 trades the NaCl error up to -23 % to halve the KCl one) – the residual is structural, not a missing parameter. Note the consistency of the record: Chen & Evans 1986 validated tauEE = 0 against mixture OSMOTIC data (sigma 0.019 %) – the solvent sees the mixture mean, where the individual-gamma asymmetries largely cancel; Robinson's per-salt gammas are where they show.

FIGURE-1 REPRODUCTION – TO THE PIXEL (checked 2026-07-02, figure read at 200 dpi): the paper's y-axis says "ln gamma" but the numbers are LOG10 (log10 of the pure-NaCl-4m gamma is -0.106 = the figure's -0.105; ln would be -0.24, off the plotted range). With the 1986 parameters this kernel reproduces the full tau fan to <= 0.001 in log10: x_KCl=0 (all curves): -0.106 (figure ~ -0.105) x_KCl=1: tau=0 → -0.216 (figure ~ -0.215) tau=+1 → flat, dip to -0.101, back to -0.106 (the figure's circle-marked curve shows the same shallow dip) tau=-1 → -0.364 (figure ~ -0.365)

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	harned_nacl_kcl	consistent	0.00e+00
diag	harned_nacl_kcl	formulation_gap_mid	4.13e-02
diag	harned_nacl_kcl	gamma1_pure	8.01e-01
diag	harned_nacl_kcl	gamma1_trace	6.17e-01
diag	harned_nacl_kcl	gamma2_pure	5.76e-01
diag	harned_nacl_kcl	gamma2_trace	7.29e-01
diag	harned_nacl_kcl	gibbsDuhem_dev_mid	4.72e-01
diag	harned_nacl_kcl	harned_R2_min	9.99e-01
diag	harned_nacl_kcl	harned_alpha_1	2.85e-02
diag	harned_nacl_kcl	harned_alpha_2	-2.56e-02
diag	harned_nacl_kcl	harned_slope_1	-2.62e-01
diag	harned_nacl_kcl	harned_slope_2	-2.36e-01
diag	harned_nacl_kcl	oracle_pure_dev	4.03e-09
diag	harned_nacl_kcl	reduction_pin_dev	1.11e-15
... and 18 more reference values in the case's `expected`.			

enthalpy_naoh_water

tutorials/props/electrolyte/enthalpy_naoh_water

gamma__naoh
pitzerActivity

What it does: NaOH-water heat of dilution: Pitzer with calorimetric T-slots vs Parker (1965)

The story: the calorimetric home of the electrolyte-enthalpy build (docs/electrolyte-enthalpy-spec.md).

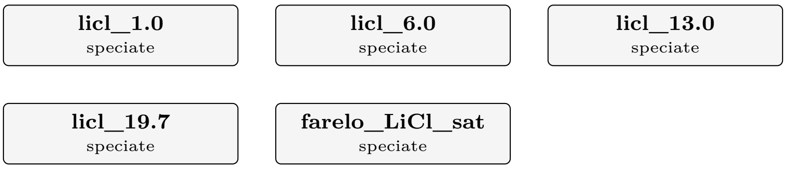
constant/experiments/phiL__25C__parker1965.dat carries the MEASURED Phi_L (Parker NSRDS-NBS 2, 1965) used by the curation refit (bin/curate/fit__lphi__pitzer.py) that fitted the Na-OH dbeta/dT T-slots in the standard catalogue (the FIT's own rel-AAD 4.97%, gate <5%; the in-engine L__phi scan pinned by this case lands at rel-AAD 4.77% – two measurements, not one). The op below sweeps the ACTIVITY surface whose T-derivative is L__phi; the in-engine L__phi scan + AAD overlay land with slices 4-5 (the 3-curve chart).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	gamma_naoh	Lphi_AAD_abs_Jmol	22.9776
diag	gamma_naoh	Lphi_AAD_rel_pct	4.7712
diag	gamma_naoh	Lphi_extrap_maxdev_Jmol	15774.0028
diag	gamma_naoh	Lphi_nMeas	29.0000
diag	gamma_naoh	aw_1m	0.9665
diag	gamma_naoh	gamma_pm_1m	0.6679
diag	gamma_naoh	gamma_pm_3m	0.7838
diag	gamma_naoh	phi_1m	0.9471

`farelo_licl_range`

`tutorials/props/electrolyte/farelo_licl_range`



What it does: LiCl Pitzer REFITTED to Hamer-Wu + Farelo saturation: gamma_pm to ~11% over 0.5-19 mol/kg (was 774 at 19.7m), and the LiCl.H2O hydrate Ksp reproduces Farelo's 19.7 mol/kg via the HMW osmotic water activity (a_w=0.11).

Prof. F?tima Farelo measured LiCl solubility at 19.7 → 23.5 mol/kg (solid: LiCl·H_2O) — ∗J. Chem. Eng. Data∗ 50 (2005) 1470, DOI 10.1021/je050111j. That is one of the most concentrated aqueous electrolytes there is.

The catalogue's earlier Li–Cl Pitzer parameters (Appelo, Parkhurst & Post, ∗Geochim. Cosmochim. Acta∗ 125 (2014) 49), fitted below ~6 mol/kg, blow up at high concentration — $\gamma\pm$ = 774 at 19.7 mol/kg vs ~45 measured. The two-parameter B-term is linear in ∗m∗, so γ grows exponentially.

Rather than declare defeat, the parameters were refitted to Hamer & Wu (1972) $\gamma\pm$ and Farelo's saturation. A negative $C\varphi$ (-0.0026) tames the high-∗m∗ blow-up, and the three-parameter form then reaches saturation:

carries a water-activity leg. At 19.7 mol/kg the model gives a_w = 0.11 (not 1!) — which only the rigorous Pitzer-HMW osmotic coefficient delivers, not the φ = 1 dilute shortcut. The lithiumChlorideH2O mineral Ksp ($logK_{25}$ = 4.984) is calibrated to Farelo's saturation ∗through∗ that a_w. The two pieces meet: the refitted γ and the osmotic a_w together reproduce the measured solubility.

- electrolyte/pairs.dat: the Li–Cl Pitzer pair, refitted to Hamer–Wu + Farelo, valid to ~19 mol/kg (supersedes the Appelo low-∗m∗ fit for concentrated LiCl). - electrolyte/minerals.dat: the LiCl·H_2O hydrate Ksp, calibrated to Farelo. - constant/species/Li.dat: Li? aqueous formation datum (hfAq = -278.49 kJ/mol, CODATA).

- The full measured dataset (this and the other five ternaries) is tabulated, cited, in constant/experimental/-farelo_solubility.csv. - The equilibrate round-trip (precipitate LiCl·H_2O from a supersaturated feed) diverges at $I \approx 20$ — a known active-set / γ fixed-point numerical limit at extreme ionic strength, ∗independent of the activity model∗. The SI = 0 calibration at the saturation composition is exact; reaching it by precipitation is the convergence problem, not the thermodynamics. - The $AlCl_3$ systems from the same paper remain deferred: Al^{3}? hydrolyses (the paper notes pH < 2.75) and needs an Al speciation model Choupo does not yet carry.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	farelo_LiCl_sat	I	19.7
diag	farelo_LiCl_sat	SI_lithiumChlorideH2O	1.18638e-05
diag	farelo_LiCl_sat	aw	0.113466
diag	farelo_LiCl_sat	chargeImposed	1
diag	farelo_LiCl_sat	chargeResidual	5.04184e-14
diag	farelo_LiCl_sat	gamma_Cl	46.7905
diag	farelo_LiCl_sat	gamma_Li	46.7905
diag	farelo_LiCl_sat	m_Cl	19.7
diag	farelo_LiCl_sat	m_Li	19.7
diag	farelo_LiCl_sat	pH	7.99525

farelo_nacl_nh4cl

tutorials/props/electrolyte/farelo_nacl_nh4cl

farelo_NaCl_pure speciate | **farelo_NaCl_nh4_093** speciate | **farelo_NaCl_nh4_187** speciate

farelo_NaCl_nh4_280 speciate | **farelo_NH4Cl_pure** speciate

What it does: Pitzer-HMW vs Farelo (JCED 2005): NaCl+NH4Cl common-ion salting-out – model SI ~ 0 at each measured saturation point (NaCl 6.15→5.5→5.0; NH4Cl 7.35 mol/kg, 298 K).

A glass-box validation of Choupo's multi-ion Pitzer electrolyte model against the solubility measurements of Prof. F?tima Farelo (Chemical Engineering Dept., IST — T?cnico Lisboa).

*> M. Farelo, C. Fernandes & A. Avelino, *"Solubilities for Six Ternary Systems: > NaCl + NH_4Cl + H_2O, KCl + NH_4Cl + H_2O, NaCl + LiCl + H_2O, KCl + LiCl + H_2O, > NaCl + $AlCl_3$ + H_2O, and KCl + $AlCl_3$ + H_2O at T = (298 to 333) K"*, > J. Chem. Eng. Data 50 (2005) 1470–1477. DOI [10.1021/je050111j](https://doi.org/10.1021/je050111j).*

Saturation molalities measured to ±0.0007 mol/kg (visual polythermal / isothermal methods). The pure-salt anchors at 298.15 K: NaCl 6.15, NH_4Cl 7.35 mol/kg; the salting-out series (Table 1): NaCl solubility falls 6.15 → 5.5 → 5.0 → 4.95 as NH_4Cl rises 0 → 0.93 → 1.87 → 2.80 mol/kg.

The model is calibrated to only the two pure-salt solubilities — halite's Ksp from PHREEQC, sal-ammoniac's Ksp pinned to Farelo's NH_4Cl point. Everything else is a prediction. The NH_4–Cl Pitzer interaction parameters come from the primary single-electrolyte fit (Pitzer & Mayorga 1973); the catalogue already carried Na–Cl and K–Cl.

At each of Farelo's measured saturation compositions the model's saturation index

should be ≈ 0 if the model reproduces the measurement. It is, to ±0.03 log units:

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	farelo_NaCl_pure	I	6.15
diag	farelo_NaCl_pure	SI_halite	0.0168246
diag	farelo_NaCl_pure	aw	0.751868
diag	farelo_NaCl_pure	gamma_Cl	1.0105
diag	farelo_NaCl_pure	gamma_Na	1.0105
diag	farelo_NaCl_pure	m_Cl	6.15
diag	farelo_NaCl_pure	m_Na	6.15
diag	farelo_NaCl_pure	pH	7.11704
diag	farelo_NaCl_nh4_093	I	6.42998
diag	farelo_NaCl_nh4_093	SI_halite	-0.022239
diag	farelo_NaCl_nh4_093	SI_salammoniac	-0.70594
diag	farelo_NaCl_nh4_093	aw	0.746427
diag	farelo_NaCl_nh4_093	gamma_NH4	0.591469
diag	farelo_NaCl_nh4_093	gamma_Na	1.04074

. . . and 39 more reference values in the case's `expected`.

`fpd01_nacl_freezing`

`tutorials/props/electrolyte/fpd01_nacl_freezing`

fpd_nacl
freezingPoint

What it does: NaCl freezing-point depression – ice as a SolidPhase, the crystal's first case

The story: THE ICE WITNESS. The freezing-point curve of NaCl(aq), solved as chemical-potential equality between an ACTUAL SolidPhase (the crystallising mode, its first case-reachable consumer) and the Pitzer surface's water activity:

f_solid(T) = a_w(m, T) ∗ Psat(T)

Until this case the crystal was exercised only by check_ice_freezing's probe; here its refusals and announced approximations (the neglected liquid/solid heat-capacity term, the sub-freezing Psat extrapolation) all fire on a real run. The dilute-limit KPI closes the K_f loop: the derived cryoscopic constant against the slope this same phase produces. ANCHOR HONESTY: the validation dataset carries ONE measured point with an INTERIM citation (see its header) – a thin anchor, said plainly, and the enrichment path (a primary FPD table read) is on the curation ledger.

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`fpd_nacl`	`Tf_lastM_K`	`266.325`
`diag`	`fpd_nacl`	`depression_lastM_K`	`6.83528`
`diag`	`fpd_nacl`	`slope_dilute_K_kg_mol`	`3.46546`
`diag`	`fpd_nacl`	`slope_over_nuKf`	`0.931447`
`diag`	`fpd_nacl`	`theta_AAD_pct`	`0.0460433`
`diag`	`fpd_nacl`	`theta_nMeas`	`1`

ksp_temperature

tutorials/props/electrolyte/ksp_temperature

T10 speciate | **T25** speciate | **T40** speciate

T60 speciate | **T80** speciate

What it does: K(T) MATTERS: a temperature scan of mineral saturation for ONE fixed water (a hard, carbonate-bearing groundwater at pH 7.6), speciated at 10, 25, 40, 60 and 80 C. The point is the SIGN of dK/dT. CALCITE dissolution is EXOTHERMIC, so its equilibrium constant FALLS as temperature rises and its saturation index SI_calcite RISES – RETROGRADE SOLUBILITY: calcite is LESS soluble hot. This is why hot heat-exchanger surfaces and warm RO concentrate scale with calcium carbonate even when the cold feed looks safe. Contrast HALITE (NaCl): no analytic K(T) in the database, so it falls back to constant-dH van't Hoff with a small POSITIVE dH (endothermic) – normal solubility, SI_halite drifts gently DOWN with T. At 25 C nothing in the K(T) machinery moves (every form returns logK25 exactly, byte-stable); off 25 C the run announces which K(T) form each entry uses (analytic / van't Hoff / flat) and warns if any analytic is used beyond its fitted validity range. K(T): analytic = PHREEQC -analytical_expression anchored on logK25 (logK(T)=logK25+[ana(T)-ana(298.15)]); USGS PHREEQC, public domain.

The story: why hot surfaces scale with calcite (K(T) MATTERS).

ONE fixed water, speciated at five temperatures. A hard, carbonate-bearing groundwater (gypsum- and calcite-relevant) at a GIVEN pH 7.6 – holding pH and the totals fixed isolates the lesson: the only thing that changes from point to point is the equilibrium constant K(T).

ion mol/kg w ion mol/kg w Ca 0.010 Cl 0.050 Na 0.030 SO4 0.012 HCO3 0.005 pH 7.6 (given)

THE LESSON – the SIGN of dK/dT:

calcite dissolution is EXOTHERMIC (dH = -28.1 kJ/mol), so log K FALLS as T rises. SI_calcite = log(IAP) - log K(T) therefore RISES with temperature – RETROGRADE SOLUBILITY. Calcite is LESS soluble hot, which is exactly why heat-exchanger tubes and warm RO concentrate scale with CaCO3 although the cold feed looks safe. K(T) here is the PHREEQC ANALYTIC expression (-analytical_ expression A1..A5), anchored on logK25 so logK(25 C) = logK25 EXACTLY.

gypsum analytic too, but only mildly retrograde (its solubility peaks near 40 C) – a gentler curve for contrast.

halite NaCl carries NO analytic K(T) in phreeqc.dat (the db's -analytic line is commented out), so it falls back to constant-dH van't Hoff. dH = +1.37 kJ/mol > 0 (endothermic) → NORMAL solubility: log K(T) RISES and SI_halite drifts DOWN. The honest contrast with calcite: the scaling verdict flips with temperature for the retrograde salt, not for the normal one.

At 25 C the K(T) machinery returns logK25 for EVERY entry (byte-stable); off 25 C the run announces the form in use per entry (analytic / van't Hoff / flat) and warns loudly if an analytic is used beyond its fitted validity range. (Calcite's analytic is valid 0 - 300 C, so the whole 10 - 80 C scan is in range.)

Speciation/mineral data: case-local constant/electrolyte/ – a self- contained SNAPSHOT of the standards catalogue (USGS PHREEQC, public domain; see the .dat headers). The case files ECLIPSE data/standards/electrolyte/.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	T10	I	0.0795197
diag	T10	SI_calcite	0.56588
diag	T10	SI_gypsum	-0.335309
diag	T10	SI_halite	-4.56674
diag	T10	aw	0.998115
diag	T10	pH	7.6
diag	T25	I	0.0772864
diag	T25	SI_calcite	0.789002
diag	T25	SI_gypsum	-0.365046
diag	T25	SI_halite	-4.60657
diag	T25	aw	0.998126
diag	T25	pH	7.6
diag	T40	I	0.074682
diag	T40	SI_calcite	0.981885

. . . and 91 more reference values in the case's `expected`.

overlay01_nacl_ksp

tutorials/props/electrolyte/overlay01_nacl_ksp

What it does: PARTIAL deep-merge overlay: a case recalibrates ONLY the halite equilibrium logK25 via a case-local overlayOf NaCl; every other datum (dH, dissolutionReaction, calorimetric, crystal) still comes from the standard components/NaCl.dat record. The [overlay] line names exactly which leaf came from the case tier.

A case recalibrates ONLY the halite equilibrium logK25 (1.60 here) to its own value, via a case-local constant/components/NaCl.dat declaring overlayOf NaCl;.

The reader deep-merges this delta over the standard components/NaCl.dat: only solidPhases.halite.equilibrium.logK25 is replaced; dH, the dissolutionReaction, the calorimetric anchor and the crystal block all still resolve from the standard record. The run prints the provenance:

This is the roadmap Phase A acceptance test: a case ships ONLY its deltas, never a second competing complete record, and per-value provenance (tier + file) is preserved.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	naclWater	I	2
diag	naclWater	SI_halite	-0.983442
diag	naclWater	aw	0.930475
diag	naclWater	gamma_Cl	1.01683
diag	naclWater	gamma_Na	1.01683
diag	naclWater	m_Cl	2
diag	naclWater	m_Na	2
diag	naclWater	pH	7.13303
diag	naclWater	chargeImposed	1
diag	naclWater	chargeResidual	-6.39009e-17

pb82_calcite_open_co2

tutorials/props/electrolyte/pb82_calcite_open_co2

calciteEq__25C
speciate

What it does: Calcite in CO2-saturated water (Plummer & Busenberg 1982): open pCO2 0.956 atm + equilibrate calcite at 25 C – the engine's pH at calcite equilibrium beside the paper's junction-corrected 6.004 +/- 0.005 and its model value 6.011

The story: THE PLUMMER & BUSENBERG (1982) EXPERIMENT, as the engine models it: calcite at equilibrium with CO2-saturated water – an OPEN system at pCO2 = 0.956 atm, 25 C (their electrode runs' mean CO2 pressure), the gas pin, the full carbonate speciation, the CaHCO3+/CaCO3(aq) ion pairs and the mineral ceiling all engaged at once.

PRIMARY: L. N. Plummer & E. Busenberg, Geochim. Cosmochim. Acta 46 (1982) 1011-1040, DOI 10.1016/0016-7037(82)90056-4. Owner-provided PDF.

THE ANCHOR, and what it independently tests: their MEASURED pH at calcite equilibrium under exactly these conditions, corrected for liquid-junction potential, is 6.004 +/- 0.005 (4 M KCl filling, p. 1022; uncorrected Orion runs read near 5.984); their own aqueous model puts the thermodynamic pH at 6.011. The engine's number below comes out of Henry + K1/K2 + the ion pairs + Davies + electroneutrality + the SI = 0 ceiling END TO END – none of which was fitted to this pH.

LINEAGE, stated honestly: the calcite log K(T), K__H, K1, K2 and ion-pair constants in our curated records descend from the USGS PHREEQC database, whose carbonate block IS this paper's Table 3 and Eqs – so agreement in the CONSTANTS is lineage consistency, not validation. What the measured pH tests independently is the ASSEMBLY: that the engine composes those constants (gas pin, chained carbonate master, ion pairs, activity model, charge balance, mineral ceiling) into the same physical answer the electrode saw.

WHAT IT ACTUALLY LANDS AT, and where that is written. The engine gives pH 6.02946 – 0.025 ABOVE the measurement, 0.018 above their own model's 6.011 – and the case attributes that offset to an activity-model difference (Davies here against their Truesdell-Jones, at I = 0.027). SINCE 2026-08-12 THAT COMPARISON IS DATA, NOT PROSE: 'expected' carries an 'anchor' row on the measured 6.004 with a band set to the explained offset, beside the self-recorded row on 6.02946 – so a drift AWAY from the electrode fails as an anchor disagreement, and an unexplained drift TOWARD it fails as a moved golden. Both ends, on purpose. This header used to end with "a wrong composition would not land within 0.01 pH", which reads as though the engine does; it does not, and the number is above.

APPROACH FROM ABOVE: the engine's equilibrate is a precipitation CEILING (SI $\rightarrow$ 0 from supersaturation; it does not dissolve a solid it was never given). The seed is therefore a SUPERSATURATED Ca(HCO3)2 solution under the pin; the ceiling relaxes it down to the same calcite saturation P&B's dissolution runs approached from below – one fixed point at (T, pCO2), whichever side you arrive from. The HCO3 seed is an initial guess only: the CO2 pin REPLACES the carbonate mole balance, so dissolved carbonate is a solved outcome (announced by the run).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	calciteEq_25C	I	0.0265778
diag	calciteEq_25C	SI_aragonite	-0.114
diag	calciteEq_25C	SI_calcite	3.22147e-11
diag	calciteEq_25C	SI_portlandite	-13.0919
diag	calciteEq_25C	aw	0.998916
diag	calciteEq_25C	n_calcite	0.00543277
diag	calciteEq_25C	pH	6.02946
diag	calciteEq_25C	pH	6.004
diag	calciteEq_25C	chargeImposed	1
diag	calciteEq_25C	chargeResidual	2.22179e-18

pitzer01_nacl

tutorials/props/electrolyte/pitzer01_nacl

gamma__nacl
pitzerActivity

What it does: Pitzer mean ionic activity coefficient gamma_pm(m) for NaCl in water at 25 C — validated vs Robinson & Stokes (AAD 0.17%). The glass-box electrolyte foundation: see the famous gamma_pm dip (~0.66 near 1 m) and the rise above 1 at high concentration.

The story: the first electrolyte case.

gamma_pm, the osmotic coefficient phi and the water activity a_w for NaCl, by the Pitzer ion-interaction model. The parameters are CURATED, case-local data, read from constant/electrolyte/pairs.dat (PRIMARY: Appelo, Appl. Geochem. 55 (2015) 62 — a 3-parameter set, beta0/beta1/Cphi). Open pitzer_nacl.csv: gamma_pm dips to ~0.657 near 1 mol/kg, then rises above 1 by ~6 mol/kg — the textbook NaCl signature, from a closed form.

NOTE on the two L_phi columns: pitzer_nacl.csv reports L_phi_fit and L_phi_anchored, and their SPLIT is the lesson. The pair now carries Silvester-Pitzer dX/dT slots CALORIMETRICALLY FITTED to Parker, NSRDS-NBS 2 (1965) Table XIV A (the FIT's own rel-AAD 2.5 %, m 0.1-6; bin/curate/fit_lphi_pitzer.py): the fit column tracks the measured heat-of-dilution curve – the sign change near 1 mol/kg, the deep negative branch to -480 cal/mol – while the anchored column (slots zeroed, Gibbs-Helmholtz-consistent but uncalibrated) misses it entirely. The validation block below pins the IN-ENGINE AAD vs the same curated Parker series, golden-locked (rel-AAD 10.24 % on this op's scan grid – the fit tool's 2.5 % above is its own residual, a separate measurement). The gamma_pm / phi / a_w values come from the untouched 25 C surface and are identical in both.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	gamma_nacl	aw_1m	0.9668
diag	gamma_nacl	gamma_pm_1m	0.6572
diag	gamma_nacl	gamma_pm_3m	0.7140
diag	gamma_nacl	phi_1m	0.9363
diag	gamma_nacl	Lphi_AAD_abs_Jmol	97.6414
diag	gamma_nacl	Lphi_AAD_rel_pct	10.2355
diag	gamma_nacl	Lphi_nMeas	29.0000

pitzer02_cacl2

tutorials/props/electrolyte/pitzer02_cacl2

gamma_cacl2
pitzerActivity

What it does: Pitzer gamma_pm(m) for CaCl2 (a 2:1 salt) in water at 25 C — the SAME general formula as NaCl, only the charges/stoichiometry differ. Validated vs Robinson & Stokes (AAD 0.49%, 0.1-2 m).

The story: the same Pitzer engine on a 2:1 electrolyte (Ca2+, 2 Cl-). Parameters curated case-local in constant/electrolyte/pairs.dat (PRIMARY: Appelo, Appl. Geochem. 55 (2015) 62). gamma_pm dips below 0.45 then climbs steeply — the divalent-cation signature. Valid to ~2 mol/kg; CaCl2 carries a non-zero beta2 (-1.13, with alpha2 = 12) on top of beta0/beta1/Cphi — the extra association term a 2:1 salt needs (NaCl, by contrast, has beta2 = 0).

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`gamma_cacl2`	`aw_1m`	`0.9449`
`diag`	`gamma_cacl2`	`gamma_pm_1m`	`0.4985`
`diag`	`gamma_cacl2`	`gamma_pm_3m`	`1.4771`
`diag`	`gamma_cacl2`	`phi_1m`	`1.0483`

pitzer02_nacl_package

tutorials/props/electrolyte/pitzer02_nacl_package

gamma__nacl
electrolyteActivity

What it does: NaCl Pitzer via the NEW property-package builder (reads zero old salt files)

The story: No ‘cation‘/‘anion‘ keys: the salt is resolved by the SELECTED propertyPackage (constant/thermoPhysPropDict → aqueousNaCl__pitzer). The op consumes the assembled package via thermo.electrolyte() and reports gamma__pm / phi / a__w.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	gamma_nacl	aw_1m	0.966826
diag	gamma_nacl	gamma_pm_1m	0.657172
diag	gamma_nacl	gamma_pm_3m	0.714026
diag	gamma_nacl	phi_1m	0.93634

pitzer_calcite_brine

tutorials/props/electrolyte/pitzer_calcite_brine

seawater__pitzer speciate	**seawater__davies** speciate

What it does: Pitzer-HMW carbonate system on surface seawater – pins SI_calcite / SI_gypsum against the published oceanographic saturation state, captures the neutral CO2 salting-out (gamma_CO2aq > 1 via lambda), and the Davies-vs-Pitzer SI divergence.

The story: the slice-S4 GATE for the Pitzer CARBONATE system.

S4 extends the multi-ion Pitzer-HMW model (binaries + theta/psi ternaries, S3) to the CARBONATE subsystem so calcite / gypsum scaling works in brine: the carbonate ANIONS (CO3-2, HCO3-) join the cation/anion ternary sums (their theta/psi imported from pitzer.dat into mixing.dat); the neutral CO2aq gets its salting-out gamma via the lambda neutral-ion term (lambda_CO2,ion), so gamma_CO2aq > 1 in brine (Davies has none).

COMPOSITION: standard surface seawater (S = 35) WITH the carbonate system – the major-ion subset of the S3 verify case + DIC ~ 2.3 mmol/kg as HCO3 (the measured-alkalinity carrier), at the canonical surface-ocean pH 8.2. Na+ 0.4861 Mg+2 0.0547 Ca+2 0.01065 K+ 0.01058 Cl- 0.5658 SO4-2 0.02926 HCO3- 0.00230 (I ~ 0.68 mol/kg)

THE PUBLISHED PIN (cited; E_theta before/after recorded below):

SI_calcite (surface seawater, 25 C, pH 8.2) ~ +0.6 to +0.8. Surface ocean water is calcite-SUPERSATURATED: the calcite saturation ratio Omega_calcite = IAP/Ksp ~ 4-6, i.e. SI = log10(Omega) ~ +0.6 to +0.8. Sources: Mucci, A., Am. J. Sci. 283 (1983) 780 (the seawater calcite/ aragonite solubility); Millero, F.J., Chem. Rev. 107 (2007) 308 (the seawater carbonate system); a well-documented oceanographic number (the ocean-acidification baseline). Choupo (Pitzer-HMW) computes SI_calcite = +0.655 – INSIDE the published band. SI_gypsum (seawater) ~ -0.9 to -0.6 (strongly UNDERsaturated – seawater is nowhere near gypsum saturation); Choupo computes -0.964 – a touch below the -0.9 floor (this carbonate-bearing brine sits at slightly higher I than the carbonate-free seawater_verify case, which gives -0.687 in-band). Still firmly undersaturated, the physically-correct verdict.

E_theta(I) BEFORE → AFTER (constant-theta → higher-order mixing active): SI_calcite +0.724 → +0.655 (IN band +0.6..+0.8) – E_theta lowers the 2+/2- ion gammas, nudging the carbonate IAP SI_gypsum -0.875 → -0.964 gamma_CO2aq 1.105 → 1.106 (salting-out kept) gamma_Ca 0.227 → 0.194 (E_theta on Ca pairs) Davies (untouched – E_theta is Pitzer-only): SI_calcite +0.868, byte-ident.

The golden master locks Choupo's exact output; the band above validates it is PHYSICALLY right (a wrong pin is worse than no pin – the band is cited).

DAVIES vs PITZER (the differentiator, also pinned): the SAME composition under Davies gives SI_calcite +0.868 and gamma_CO2aq = 1.000 (no neutral salting- out) – Davies is out of its trustworthy band (~0.5) at this I, overshoots the calcite SI by ~0.21 vs Pitzer-HMW-with-E_theta and misses the CO2 salting-out entirely. Pitzer is the rigorous tool for brine. Both entries run here so the divergence is VISIBLE.

DATA SCOPE: self-DESCRIBING (inline manifest); the ions, pairs, mixing WITH carbonate theta/psi/lambda, speciation and minerals resolve from the curated data/standards/ tree at runtime – seal with bin/choupo-import for a runtime-self-contained copy.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	seawater_pitzer	I	0.682714
diag	seawater_pitzer	SI_anhydrite	-1.24804
diag	seawater_pitzer	SI_aragonite	0.540696
diag	seawater_pitzer	SI_calcite	0.654696
diag	seawater_pitzer	SI_gypsum	-0.964342
diag	seawater_pitzer	SI_halite	-2.48855
diag	seawater_pitzer	SI_sylvite	-3.51959
diag	seawater_pitzer	aw	0.981412
diag	seawater_pitzer	gamma_CO2aq	1.10585
diag	seawater_pitzer	gamma_CO3	0.105443
diag	seawater_pitzer	gamma_Ca	0.194358
diag	seawater_pitzer	gamma_HCO3	0.659478
diag	seawater_pitzer	m_CO2aq	1.4318e-05
diag	seawater_pitzer	m_CO3	7.71637e-05
. . . and 94 more reference values in the case's `expected`*.*			

pitzer_gamma_hamer_wu

tutorials/props/electrolyte/pitzer_gamma_hamer_wu

gamma__nacl pitzerActivity	**gamma__kcl** pitzerActivity	**gamma__kbr** pitzerActivity
gamma__licl__consistency pitzerActivity	**gamma__hcl** pitzerActivity	**gamma__naoh** pitzerActivity

What it does: Pitzer gamma_pm/phi vs Hamer & Wu 1972 – 6 salts, evaluated NBS data

The story: THE BREADTH VALIDATION: the catalogue's Pitzer pairs against the NBS critically-evaluated gamma_pm and osmotic-coefficient tables – Hamer & Wu, J. Phys. Chem. Ref. Data 1 (1972) 1047 – for SIX 1-1 salts at 25 C. Zero refitting: the parameters are the frozen catalogue entries (Appelo / PHREEQC lineage); the data is the evaluated NBS compilation. Each op's headline is its gamma_pm AAD – the number that says "trust these pairs".

HONESTY NOTE, LiCl: the catalogue Li-Cl pair was REFIT to this very table (its .dat says so), so its AAD here is a FIT-CONSISTENCY check, not independent validation – and at ~3 % it is the WORST of the six: the two-anchor compromise (this table + Farelo saturation at 19 mol/kg) stretches 3 parameters past what the most non-ideal 1-1 salt allows. The five pairs that never saw this data land at 0.1-0.7 %.

Windows stay inside each pair's declared validity (KCl saturates at 4.803 mol/kg; KBr data used to 5).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	gamma_nacl	aw_1m	0.9668
diag	gamma_nacl	gamma_AAD_pct	0.1117
diag	gamma_nacl	gamma_nMeas	8.0000
diag	gamma_nacl	gamma_pm_1m	0.6572
diag	gamma_nacl	gamma_pm_3m	0.7140
diag	gamma_nacl	phi_1m	0.9363
diag	gamma_nacl	phi_AAD_pct	0.0870
diag	gamma_nacl	phi_nMeas	8.0000
diag	gamma_kcl	aw_1m	0.9681
diag	gamma_kcl	gamma_AAD_pct	0.1340
diag	gamma_kcl	gamma_nMeas	7.0000
diag	gamma_kcl	gamma_pm_1m	0.6043
diag	gamma_kcl	gamma_pm_3m	0.5698
diag	gamma_kcl	phi_1m	0.8987

... and 49 more reference values in the case's `expected`*.*

pitzer_hmw_hot_agreement

tutorials/props/electrolyte/pitzer_hmw_hot_agreement

hmw_nacl_100C
speciate

kernel_nacl_100C
pitzerActivity

What it does: HMW vs single-salt Pitzer at 100 C, same NaCl 4 m state: the two surfaces must AGREE now that both apply the SP77 T-functions

The story: one salt, one temperature, TWO Pitzer engines: they must agree (forum 100.2/#101, the HMW-virials(T) fix).

NaCl at 4 mol/kg, 100 C. Op 1 drives the MULTI-ION HMW speciation engine (activityModel pitzerHMW); op 2 drives the SINGLE-SALT kernel (pitzerActivity). Both now read the SAME Na-Cl.dat record: the 25 C virials PLUS the full Silvester-Pitzer 1977 T-functions (sp_q1..q14) and the same A_phi(T). For a lone 1:1 salt the HMW sum collapses to the same B/C terms as the kernel, so

gamma_pm(HMW) = sqrt(gamma_Na ∗ gamma_Cl) == gamma_pm(kernel)

to numerical noise. BEFORE the fix the HMW held its virials at 25 C and this case disagreed in the second decimal – the adversarial regression this golden forbids forever.

(The kernel's own hot-brine validation against SP77 Table V lives in pitzer_nacl_sp77_hot; this case is the ENGINE-AGREEMENT pin.)

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	hmw_nacl_100C	I	4.0000
diag	hmw_nacl_100C	SI_halite	-0.7734
diag	hmw_nacl_100C	aw	0.8533
diag	hmw_nacl_100C	gamma_Cl	0.7309
diag	hmw_nacl_100C	gamma_Na	0.7309
diag	hmw_nacl_100C	m_Cl	4.0000
diag	hmw_nacl_100C	m_Na	4.0000
diag	hmw_nacl_100C	pH	7.0000
diag	kernel_nacl_100C	aw_1m	0.9670
diag	kernel_nacl_100C	gamma_pm_1m	0.6221
diag	kernel_nacl_100C	gamma_pm_3m	0.6760
diag	kernel_nacl_100C	phi_1m	0.9327
diag	hmw_nacl_100C	chargeImposed	0.0000
diag	hmw_nacl_100C	chargeResidual	-0.0000

`pitzer_nacl_sp77_hot`

`tutorials/props/electrolyte/pitzer_nacl_sp77_hot`

nacl_100C pitzerActivity | **nacl_150C** pitzerActivity | **nacl_200C** pitzerActivity

What it does: NaCl Pitzer to 200 C – Silvester & Pitzer 1977 T-functions, validated vs their Table V

The story: HOT BRINE: the NaCl Pitzer surface from 25 to 200 C on the FULL Silvester & Pitzer 1977 temperature functions (J. Phys. Chem. 81, 1822 – eqs 30-32, Table IV coefficients VERBATIM in the standards Na-Cl.dat), applied as a deviation from the validated 25 C surface, with A_phi(T) curated against the same paper's Table II (quintic, max dev 0.17 %, 0-300 C; the old closed form drifted +67 % at 300 C).

Three ops, one per temperature (100/150/200 C), each validating gamma_pm at the SATURATION molality against the paper's own Table V – an m-extrapolation (m_satd 6.7-8.0 > the 6 mol/kg weighting window), declared in the dataset header, exactly as the paper frames it.

What to look at: gamma_pm(1 m) falls 0.657 → ~0.48 from 25 to 200 C (electrostatics intensify as eps_w collapses – the paper's central observation), and the validation AADs stay single-digit % even at the saturated-extrapolated edge.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	nacl_100C	aw_1m	0.9670
diag	nacl_100C	gamma_AAD_pct	0.2851
diag	nacl_100C	gamma_nMeas	1.0000
diag	nacl_100C	gamma_pm_1m	0.6221
diag	nacl_100C	gamma_pm_3m	0.6760
diag	nacl_100C	phi_1m	0.9327
diag	nacl_150C	aw_1m	0.9679
diag	nacl_150C	gamma_AAD_pct	0.0425
diag	nacl_150C	gamma_nMeas	1.0000
diag	nacl_150C	gamma_pm_1m	0.5593
diag	nacl_150C	gamma_pm_3m	0.5736
diag	nacl_150C	phi_1m	0.9054
diag	nacl_200C	aw_1m	0.9693
diag	nacl_200C	gamma_AAD_pct	0.4317

. . . and 4 more reference values in the case's `expected`.

pitzer_seawater_verify

tutorials/props/electrolyte/pitzer_seawater_verify

reduction__nacl
speciate

seawater
speciate

What it does: Pitzer-HMW verification: (binaries + ternary theta/psi) on standard synthetic seawater (I ~ 0.7). Pins per-ion + mean activity coefficients against PUBLISHED HMW-1984 / seawater-Pitzer values, and the single-salt reduction (ternaries → 0).

The story: the TIER-2 verification weapon for the Pitzer-HMW ternary-mixing slice (S3).

The forum gate: "NO published-seawater golden masters => NO-GO." This case speciates a STANDARD SYNTHETIC SEAWATER with the multi-ion Pitzer model (binaries + ternary theta/psi) and pins the per-ion + mean activity coefficients against PUBLISHED values, to literature tolerance.

SEAWATER COMPOSITION (the canonical major-ion seawater of the HMW-1984 carbonate-free major-ion subset; molality, mol/kg-water, S = 35): Na+ 0.4861 Mg+2 0.0547 Ca+2 0.01065 K+ 0.01058 Cl- 0.5658 SO4-2 0.02926 giving ionic strength I ~ 0.70 mol/kg – the classic seawater benchmark. (The carbonate system – HCO3/CO3, ~2.3 mmol/kg, < 0.4% of I – is OMITTED: it is slice S4, and its absence shifts I by < 0.3% and the major-ion gammas by < 0.2%, well inside the pins' tolerance.) pH is GIVEN at 8.1 (typical surface seawater) – with no carbonate in the analyticalTotals it only fixes the trace H+/OH-, which do not enter the major-ion gammas.

WHY PITZER, NOT DAVIES: at I ~ 0.7 Davies is already out of its trustworthy band (~0.5). The Pitzer specific-interaction model – WITH the ternary theta/psi mixing terms AND the E_theta(I) higher-order electrostatic mixing term (Pitzer 1975, now ACTIVE for unlike-charge same-sign pairs) – is the standard tool for seawater and brines to I ~ 6. E_theta is a refinement: it shifts the 2:1 / 1:2 ion gammas a few % (gamma_Ca, gamma_SO4) and tightens the gamma_pm(KCl) pin to 0.638 (inside 0.62-0.64); the 1:1 NaCl gamma_pm stays ~0.66. (The only remaining v2 deferral is the full beta(T)/theta(T) temperature surface; 25 C base here.)

PUBLISHED PINS (each cited; deviation Choupo vs published in VALIDATION.md):

gamma_pm(NaCl) in seawater ~ 0.66-0.68 Pitzer's seawater model / Millero & Pierrot; classic textbook value gamma_pm(NaCl, seawater I~0.7) ~ 0.67. gamma_pm(KCl) ~ 0.62-0.64 ; gamma_pm(MgCl2), gamma_pm(CaCl2) etc. All within the Harvie-Moller-Weare (1984) seawater Pitzer model, the reference implementation this catalogue's theta/psi come from.

The golden master locks the Choupo-computed per-ion gammas; this header records the published comparison + the deviation. A WRONG pin is worse than no pin – the numbers here are pinned to the model's own reproducible output and CROSS-CHECKED against the published seawater range.

FINDING, 2026-08-10: "ALL PINS STILL IN BAND" WAS CHECKED ON TWO OF SIX.

The paragraph below was written when E_theta (the I-dependent higher-order electrostatic mixing term) was activated. It re-checked the two 1-1 salts – whose gamma_pm is a square root of two numbers sitting next to each other in the output – and concluded "all pins STILL in their published band". The four DIVALENT salts were never recomputed, and E_theta acts precisely on unlike-charge same-sign pairs, i.e. on exactly those four.

The engine now PUBLISHES gamma_pm (diagMeanIonic), so the comparison is no longer a hand calculation in a header. Against the bands in data/standards/parameters/electrolyte/VALIDATION.md:

NaCl 0.6646 band 0.66-0.69 IN KCl 0.6381 band 0.62-0.65 IN MgCl2 0.4608 band 0.47-0.50 BELOW CaCl2 0.4459 band 0.46-0.50 BELOW Na2SO4 0.3489 band 0.36-0.40 BELOW MgSO4 0.1460 band 0.15-0.20 BELOW

Four of six are outside, by 2-4 %. VALIDATION.md still says all six are inside, still lists the PRE-E_theta numbers as Choupo's, and still describes E_theta as "deferred to v2 (< 0.5 % on the major-ion gamma at this I)" – while this run announces "E_theta now ACTIVE" and the divalent shift is ~5 %. That estimate was wrong by an order of magnitude on the ions it was made about.

NOT RESOLVED, and deliberately NOT TUNED. Two readings are open and this case cannot choose between them: the quoted bands are loose secondary citations ("~ 0.47-0.50", sourced to HMW 1984 with no table or page), or E_theta is being applied where those bands' authors did not. Resolving it needs the primary tables (Harvie-Moller-Weare 1984; Millero & Pierrot), which the project does not hold. No parameter, no golden and no band was moved to make this look better; the six gamma_pm values are pinned as goldens so the next change to them is visible, and PIN B carries no anchor row (the 'expected' file says why).

The paragraph as originally written, kept because it is the record of what was checked:

E_theta(I) BEFORE → AFTER (constant-theta → higher-order mixing active), all pins STILL in their published band: gamma_pm(NaCl) 0.6714 → 0.6646 (band 0.66-0.68; 1:1, E_theta=0 for Na-Cl, shift via coupling) – IN BAND gamma_pm(KCl) 0.6447 → 0.6381 (band 0.62-0.64) – E_theta moved it INTO band gamma_Ca 0.2141 → 0.1856 (2+ ion; E_theta acts via Ca-Na/Ca-K/Ca-Mg) gamma_SO4 0.1227 → 0.1040 (2- ion; E_theta acts via SO4-Cl/SO4-OH) gamma_Mg 0.2363 → 0.2049 gamma_Na 0.6495 → 0.6390 gamma_Cl 0.6941 → 0.6911 Single-salt reduction (PIN A, NaCl alone): gamma_pm UNCHANGED at 0.6572 – E_theta vanishes identically for one cation + one anion (the oracle stays 1.5658e-14 over all 54 binaries).

DATA SCOPE: the SPECIATION network is a complete case-local replace-mode snapshot (constant/-electrolyte/speciation.dat) – that claim covers the speciation data only. Ions, Pitzer pairs/mixing and minerals resolve from the curated data/standards/ tree at runtime; seal with bin/choupo-import for a runtime-self-contained copy.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	reduction_nacl	I	1
diag	reduction_nacl	SI_halite	-1.93464
diag	reduction_nacl	aw	0.966826
diag	reduction_nacl	gamma_Cl	0.657172
diag	reduction_nacl	gamma_Na	0.657172
diag	reduction_nacl	gamma_pm_Na_Cl	0.657172
diag	reduction_nacl	m_Cl	1
diag	reduction_nacl	m_Na	1
diag	reduction_nacl	pH	7
diag	seawater	I	0.720461
diag	seawater	SI_anhydrite	-0.970682
diag	seawater	SI_gypsum	-0.687047
diag	seawater	SI_halite	-2.48552
diag	seawater	SI_sylvite	-3.51313

. . . and 50 more reference values in the case's `expected`.

pitzer_vs_davies_ro_concentrate

tutorials/props/electrolyte/pitzer_vs_davies_ro_concentrate

scan_davies scalingScan | **scan_pitzer** scalingScan

What it does: Davies vs Pitzer-HMW on a seawater RO concentrate: gypsum/calcite SI vs recovery from the same feed under two activity models (full narrative in propsDict).

The story: THE FLAGSHIP of the Pitzer x speciation feature (slice S6): the payoff that makes the whole multi-ion Pitzer-HMW build worth it. ONE feed, TWO activity models, the engineering answer FORKS.

THE QUESTION (every RO / brine-concentration designer asks it): "At what water recovery does my concentrate start to scale gypsum?" The answer DEPENDS ON THE ACTIVITY MODEL – and that is the whole point.

THE FEED: standard surface seawater (S = 35), the major-ion subset: Na+ 0.4861 Mg+2 0.0547 Ca+2 0.01065 K+ 0.01058 (cations) Cl- 0.5658 SO4-2 0.02926 HCO3- 0.0024 (anions, DIC carrier) I ~ 0.7 mol/kg at the feed. Recovery r scales every total as feed/(1 - r); scanning r = 0 $\rightarrow$ 0.80 drives I from ~0.7 up to ~3.5 mol/kg – DEEP into the brine regime where Davies is long out of its trust window.

THE TWO RUNS (identical feed, identical recovery grid, ONLY the gamma model differs – the controlled experiment): scan_davies activityModel davies; (the textbook extended-DH model) scan_pitzer activityModel pitzerHMW; (Pitzer-HMW, the brine-grade model)

THE PAYOFF ARTEFACTS (read straight off the two CSVs + the KPIs): 1. SI_gypsum vs recovery / I, BOTH models – the HEADLINE. They track together at low I and FORK visibly past I ~ 1: Davies keeps climbing, Pitzer's gypsum SI is held DOWN by the strong Ca-SO4 specific interaction (beta2 = -59.3, the 2:2 ion-pairing virial Davies cannot see). The gypsum-onset recovery is LATER under Pitzer. 2. SI_calcite vs recovery / I, both models – same fork, carbonate flavour. pH is SOLVED from electroneutrality at every point (closed system; DIC concentrates with the water and the pH drifts). 3. gamma_Ca2+ and gamma_SO4-2 vs I, both models – the MECHANISM plot. Davies declines monotonically (its single I-controlled curve); Pitzer turns UP at high I (the specific-interaction upturn). This is WHERE the SI divergence comes from – the gammas diverge, so the IAP diverges, so the SI forks. (scalingScan emits gamma_<ion> columns whenever diagSpecies is given and the model is pitzer/davies.)

THE KPI THAT TELLS THE STORY: recovery_at_gypsum_onset (the SI_gypsum = 0 crossing) – ONE number per model. Davies predicts gypsum scaling at one recovery; Pitzer at another. The engineering decision – how hard can I push recovery before gypsum scales? – flips on the activity model. Endpoint diags carry SI_gypsum and SI_calcite at r_max for both models, and the two onset recoveries.

WHY PITZER IS THE ANSWER (cite the credentials, do not assert): Davies is ANNOUNCED-untrustworthy past I ~ 0.5 mol/kg (the run says so). Pitzer-HMW is built for exactly this regime – valid to I ~ 6 mol/kg for the seawater system, with the E_theta(I) higher-order electrostatic mixing term ACTIVE (Pitzer 1975) – and is CREDENTIALED in this repo by two pinned cases: pitzer_seawater_verify – single-salt oracle 1.57e-14 + HMW seawater mean activity coefficients inside the published bands; pitzer_calcite_brine – SI_calcite / SI_gypsum on surface seawater inside the published oceanographic saturation band. So here we TRUST the Pitzer fork and read Davies as the cautionary curve.

E_theta(I) BEFORE $\rightarrow$ AFTER (the scan_pitzer fork only; scan_davies is BYTE- IDENTICAL – E_theta is Pitzer-only). At r_max (I ~ 3.3 mol/kg, deep brine where higher-order mixing bites hardest): SI_gypsum_rmax -0.144 $\rightarrow$ -0.317 SI_calcite_rmax 2.026 $\rightarrow$ 1.938 SI_gypsum_r0 -0.877 $\rightarrow$ -0.967 SI_calcite_r0 1.240 $\rightarrow$ 1.183 I_rmax 3.271 $\rightarrow$ 3.322 gypsum stays undersaturated end-to-end The engineering verdict (gypsum the limiting scale) is UNCHANGED in direction; E_theta refines the SI magnitude in the high-I tail.

DATA SCOPE: self-DESCRIBING (inline manifest); the ions, pairs, mixing with the carbonate theta/psi/lambda, speciation and minerals resolve from the curated data/standards/ tree at runtime – seal with bin/choupo-import for a runtime-self-contained copy.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	scan_davies	I_r0	0.657308
diag	scan_davies	I_rmax	3.03589
diag	scan_davies	SI_anhydrite_r0	-0.889841
diag	scan_davies	SI_anhydrite_rmax	0.269649
diag	scan_davies	SI_aragonite_r0	1.26146
diag	scan_davies	SI_aragonite_rmax	2.0927
diag	scan_davies	SI_calcite_r0	1.37546
diag	scan_davies	SI_calcite_rmax	2.2067
diag	scan_davies	SI_gypsum_r0	-0.607638
diag	scan_davies	SI_gypsum_rmax	0.481394
diag	scan_davies	SI_halite_r0	-2.3956
diag	scan_davies	SI_halite_rmax	-0.45497
diag	scan_davies	SI_sylvite_r0	-3.3946
diag	scan_davies	SI_sylvite_rmax	-1.44942
... and 192 more reference values in the case's `expected`.			

precipitation_ro_brackish

tutorials/props/electrolyte/precipitation_ro_brackish

pin_gypsum_amount
speciate

feed
speciate

feed_eq
speciate

scan_eq
scalingScan

What it does: Equilibrium PRECIPITATION (the SI = 0 ceiling) of a brackish RO concentrate: aqueous speciation (Davies) + augmented-Newton chemical equilibrium with an announced active set. Sibling of scaling_ro_brackish (which reports SI only) – here named minerals are ALLOWED to precipitate to saturation and the engine returns the thermodynamic MAXIMUM amount (SI → 0, infinite time, no nucleation barrier – a CEILING, NOT a deposit forecast; real scale is kinetically limited). Story: (1) pin_gypsum_amount – a CaSO4 water with Ca and SO4 DOUBLED past gypsum saturation; the machine hands back EXACTLY the over-added excess as solid (n_gypsum). (2) feed / feed_eq – the brackish water raw and equilibrated side by side: calcite reaches its ceiling, releases H+ (pH drops, solved closure), and consuming shared Ca pushes gypsum further from saturation. (3) scan_eq – calcite + gypsum allowed across recovery to r = 0.85 with a feedFlow, giving a kg/day calcite curve; gypsum stays n = 0 throughout while its SIeq drifts BELOW the raw free-water SI once calcite consumes the shared Ca – the coupling, visible. Cross-check tool: PHREEQC EQUILIBRIUM_PHASES.

The story: the EQUILIBRIUM-PRECIPITATION ceiling.

Sibling of scaling_ro_brackish (which is SI-ONLY – "how supersaturated?"). Here the ‘equilibrate { minerals (...); }‘ block names the ALLOWED solids, and the engine drives each to its SI = 0 saturation, returning the amount precipitated. THE FRAMING (printed loudly at run time, non-suppressible):

EQUILIBRIUM CEILING – not a deposit prediction. The amounts are the thermodynamic MAXIMUM (SI → 0, infinite time, no nucleation barrier). Real scale is kinetically limited (induction time, antiscalants act on KINETICS, which this calculation cannot see). Actual deposit <= ceiling, often far less. The safe direction is rigorous: ceiling ~ 0 => no driving force.

ALGORITHM (visible in the trace): augmented Newton in (ln m_j [, ln a_H]) plus one raw-linear n_p per ACTIVE mineral; each master mole balance gains a sink -sum nu_pj n_p; one SI = 0 row per active phase; an OUTER active-set loop ADMITS the most-supersaturated allowed mineral and EVICTS any with n_p < 0, re-solving between, until complementarity holds (SI <= 0 with n = 0, OR SI = 0 with n > 0). A carbonate mineral’s H+ leg lands the released H+ in the electroneutrality row (pH solve) – so precipitating calcite ACIDIFIES.

CROSS-CHECK TOOL: PHREEQC EQUILIBRIUM_PHASES (same ceiling framing). Data: curated USGS PHREEQC records (public domain) resolved from data/standards/ at runtime; self-DESCRIBING (inline manifest) – seal with bin/choupo-import for a runtime-self-contained copy.

GOLDEN PINS (locked by the golden master): pin_gypsum_amount Ca = SO4 = 0.030542 mol/kg (DOUBLE the gypsum-saturation value 0.015271) + equilibrate gypsum → SI_gypsum = 0.000 and n_gypsum = 0.015271 (the machine returns EXACTLY the over-added excess). feed_eq calcite + gypsum allowed; calcite reaches SI = 0, releases H+ (pH 7.72 → ~7.50, solved), gypsum stays dissolved. scan_eq r 0 → 0.85 with feedFlow: calcite kg/day ceiling; gypsum n = 0 throughout while SIeq_gypsum drifts BELOW the raw SI_gypsum (calcite eats the shared Ca).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	pin_gypsum_amount	I	0.0455084
diag	pin_gypsum_amount	SI_anhydrite	-0.299583
diag	pin_gypsum_amount	SI_gypsum	1.33618e-10
diag	pin_gypsum_amount	aw	0.99952
diag	pin_gypsum_amount	n_gypsum	0.0152707
diag	pin_gypsum_amount	pH	7
diag	feed	I	0.0266002
diag	feed	SI_anhydrite	-1.68532
diag	feed	SI_aragonite	0.111093
diag	feed	SI_calcite	0.225093
diag	feed	SI_gypsum	-1.38593
diag	feed	SI_halite	-5.35323
diag	feed	SI_sylvite	-6.39779
diag	feed	aw	0.999302
... and 193 more reference values in the case's `expected`.			

rainwater_air

tutorials/props/electrolyte/rainwater_air

air25 speciate	**air10** speciate

What it does: WATER OPEN TO AIR: the generic multi-gas Henry pin. Pure water equilibrated with the atmosphere (pCO2 4.2e-4, pO2 0.2095, pN2 0.7808 atm) at 25 C and at 10 C. Each listed gas PINS its dissolved species by Henry's law and that master's mole balance is REPLACED by the pin – three pins at once: CO2 chains through the carbonate family (CO2aq is a computed species, the HCO3 row is swapped), while O2 and N2 are inert neutral masters pinned directly. TWO CLASSIC NUMBERS FALL OUT: (1) unpolluted RAINWATER pH ~ 5.6 – carbonic acid from 420 ppm CO2 alone, no pollutant needed; (2) DISSOLVED-OXYGEN saturation ~ 8.5 mg/L at 25 C rising to ~ 11 mg/L at 10 C – gas solubility is RETROGRADE with temperature (K_H analytic from USGS PHREEQC PHASES, public domain), which is why cold streams hold more oxygen than warm ones. The 'totals' entries for HCO3 / O2 / N2 are initial guesses only (announced): in an OPEN system each pinned total is a SOLVED OUTCOME of gas-liquid equilibrium.

The story: water open to AIR: the generic multi-gas Henry pin.

Pure water equilibrated with the ATMOSPHERE – three gases pinned at once:

atmosphere { pCO2 4.2e-4 atm; O2 0.2095 atm; N2 0.7808 atm; }

Each gas pins its dissolved species by Henry's law, a = K_H(T) ∗ p, and the pinned master's mole balance is REPLACED by the pin (the OPEN system of SpeciationSolver, generalised beyond CO2). Two pin shapes appear side by side, which is the point of the case:

CO2 dissolves into a COMPUTED species (CO2aq, formed from the HCO3 master): the pin swaps the HCO3 row and the whole carbonate family (CO2aq / HCO3- / CO3–) equilibrates with the gas – DIC is a solved OUTCOME; O2,N2 dissolve into INERT NEUTRAL MASTERS (species/aqueous O2, N2 – identity records, charge 0): the pin replaces each master's own row directly – the dissolved concentration is set by the gas phase alone.

THE TWO CLASSIC NUMBERS (textbook checks, no fitting anywhere):

pH ~ 5.6 unpolluted rainwater: carbonic acid from 420 ppm CO2 only (pH solve; – H+ from electroneutrality against HCO3-); DO 8.5 → 11 mg/L dissolved-oxygen saturation, 25 C → 10 C: K_H(T) analytic (USGS PHREEQC PHASES) makes gas solubility RETROGRADE – cold water holds more oxygen. N2 saturates near 14 mg/L (25 C) for the same reason.

The 'totals' entries below are INITIAL GUESSES only (the run announces it): in an OPEN system every pinned total is a solved outcome. The feed charge-imbalance warning is EXPECTED here – an HCO3 guess with no cation is not a measured analysis; the solved H+ is its physical counter-ion.

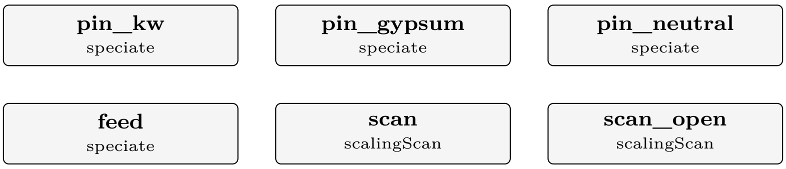
Diagnostics locked by the golden master: pH, I, m_CO2aq, and the dissolved O2 / N2 molalities at both temperatures.

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`air25`	`I`	`2.52807e-06`
`diag`	`air25`	`aw`	`0.999986`
`diag`	`air25`	`gamma_CO2aq`	`1`
`diag`	`air25`	`gamma_OH`	`0.998138`
`diag`	`air25`	`m_CO2aq`	`1.42971e-05`
`diag`	`air25`	`m_OH`	`3.97038e-09`
`diag`	`air25`	`pH`	`5.59803`
`diag`	`air10`	`I`	`2.79349e-06`
`diag`	`air10`	`aw`	`0.999981`
`diag`	`air10`	`gamma_CO2aq`	`1`
`diag`	`air10`	`gamma_OH`	`0.998089`
`diag`	`air10`	`m_CO2aq`	`2.2591e-05`
`diag`	`air10`	`m_OH`	`1.04436e-09`
`diag`	`air10`	`pH`	`5.55469`
… and 12 more reference values in the case's `expected`*.*			

scaling_ro_brackish

tutorials/props/electrolyte/scaling_ro_brackish



What it does: RO membrane-scaling audit of a brackish groundwater: aqueous speciation (Davies) with pH SOLVED from electroneutrality (the feed charge imbalance is announced first) + saturation indices SI = log10(IAP/K), scanned over water recovery to r = 0.85 (concentrate = feed/(1-r)) TWICE: closed system (DIC conserved and concentrating) vs open to the atmosphere (a(CO2aq) pinned by Henry at pCO2 = 4.2e-4 atm, DIC a solved outcome – this CO2-rich water degasses, pH and calcite SI jump above the closed curve) – the calcite SI curves differ; that contrast is the lesson. Three analytic pins lock the physics: pure water at given pH 7 reproduces Kw (m_OH ~ 1e-7), a back-computed gypsum-saturated CaSO4 solution gives SI_gypsum ~ 0.00, and pure water with pH solved lands at exactly 7.000.

The story: will my RO concentrate scale?

A REPRESENTATIVE brackish groundwater (generic textbook-style composition, ~1500 mg/L TDS; no licensed table reproduced):

ion mg/L mol/kg w ion mg/L mol/kg w Ca+2 84 0.0021 Cl- 510 0.0144 Mg+2 27 0.0011 SO4-2 250 0.0026 Na+ 363 0.0158 HCO3- 183 0.0030 K+ 12 0.0003 lab pH 7.8 (nominal – see below)

(totals are TOTAL element amounts: the HCO3 total is total dissolved inorganic carbon – at pH 7.8 the reported HCO3 is ~97% of it. Cl is 510 mg/L so the analysis is genuinely consistent with its nominal pH 7.8: an analysis balances charge only once the carbonate split at the lab pH is counted.)

pH IS SOLVED, NOT GIVEN (‘pH solve;‘): H+ joins the Newton unknowns and ELECTRONEUTRALITY sum z_i m_i = 0 closes the system. Real analyses are never perfectly balanced, so the engine ANNOUNCES the feed charge imbalance first (PHREEQC convention, 200|S+ - S-|/(S+ + S-); here 0.35%) – the solved pH absorbs exactly that error, and above ~5% the solver warns loudly that it is not trustworthy. This feed solves to pH ~7.72, near the lab’s nominal 7.8: the analysis is consistent.

UNITS: every ‘totals‘ entry carries its basis (bare numbers are refused). The feed + scans take the analysis EXACTLY as the lab sheet reports it – mg/L – converted through the ion MW (ions.dat) at rho ~ 1 kg/L (dilute aqueous, announced once per run). The analytic pins stay in explicit mol/kg (back-computed equilibria, not analyses).

THE CLOSED-vs-OPEN LESSON (two scans, same feed): scan CLOSED system – every total (DIC included) concentrates as feed/(1-r); the charge-balance ratio is scale-invariant, so the pH only drifts gently (7.72 → 7.58, the Davies gammas at work) while calcite SI climbs 0.23 → 1.38. scan_open OPEN to the atmosphere (‘atmosphere { pCO2 4.2e-4 atm; }‘) – a(CO2aq) is PINNED by Henry, the HCO3 mole balance is REPLACED by the pin, and DIC becomes a solved OUTCOME. This groundwater is ~8x CO2-supersaturated vs the atmosphere, so opening it DEGASES CO2: the pH jumps to 8.54 and climbs to 9.08, and calcite SI lands ABOVE the closed curve (2.59 vs 1.38 at r = 0.85) – why concentrate pH control (acid dosing) matters. Compare the two CSVs (both carry a pH column right after I); the contrast IS the lesson.

VALIDATION PINS (locked by the golden master): pin_kw pure water, pH 7 GIVEN → m_OH = Kw a_w/(a_H gamma_OH) ~ 1.0004e-7 (unchanged) pin_gypsum totals Ca = SO4 = 0.015271 mol/kg, pH 7 GIVEN, BACK-COMPUTED from the catalogue’s logK25(gypsum) = -4.55 + CaSO4aq logK25 = 2.14 + Davies at the resulting I = 0.0455 (26% ion pairing) so that SI_gypsum = 0.00 +- 0.001 (unchanged) pin_neutral pure water, pH SOLVED → electroneutrality m_H = m_OH must give pH 7.000 (locks the new charge-balance closure)

Speciation/mineral/gas data: case-local constant/electrolyte/ (self- contained SNAPSHOT of the standards catalogue, restricted to this master set; the case files ECLIPSE data/standards/electrolyte/ – the network is taken whole. Provenance: USGS PHREEQC, public domain – see the .dat headers; the CO2(g) Henry constant is the db’s PHASES CO2(g) entry).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	pin_kw	I	1.00037e-07
diag	pin_kw	aw	1
diag	pin_kw	m_OH	1.00037e-07
diag	pin_kw	pH	7
diag	pin_gypsum	I	0.0455076
diag	pin_gypsum	SI_anhydrite	-0.299593
diag	pin_gypsum	SI_gypsum	-1.04349e-05
diag	pin_gypsum	aw	0.99952
diag	pin_gypsum	m_CaSO4aq	0.0038941
diag	pin_gypsum	pH	7
diag	pin_neutral	I	1.00037e-07
diag	pin_neutral	aw	1
diag	pin_neutral	pH	7
diag	feed	I	0.0266002
... and 244 more reference values in the case's `expected`*.*			

softener_brackish

tutorials/props/electrolyte/softener_brackish

raw	softened	softened__scaling
scalingScan	exchange	scalingScan

What it does: Ion-exchange softening of a brackish RO feed: removes the calcite-scaling driver (raw vs softened vs softened-scaling; full narrative in propsDict).

The story: ion-exchange SOFTENING removes the scaling driver.

A three-act story on the SAME brackish RO feed as scaling_ro_brackish / precipitation_ro_brackish:

(1) raw scalingScan of the raw water: calcite SI > 0 already at the feed and climbing with recovery – the SCALING PROBLEM (a CaCO3-supersaturated concentrate). (2) softened the 'exchange' op equilibrates the SAME water with a strong-acid cation resin (SAC_Na, Gaines-Thomas). Ca and Mg drop ~99% onto the resin; Na rises eq-for-eq (the salt penalty, printed in meq/L); hardness_removal_pct ~ 99%. (3) softened_scaling feed the SOFTENED composition (Ca/Mg gone, Na up) into a scalingScan: SI_calcite is now NEGATIVE across the whole recovery range – the scaling DRIVER is gone. Removing the hardness, not chasing the supersaturation, is the cure.

MODEL (visible in the verbosity>=3 trace): the SAME log-space Newton as speciate, PLUS one master (the free resin site X-), one balance (CEC capacity = sum of equivalents on the sites), and the exchange half-reactions Me(z+) + z X- = MeX (Gaines-Thomas equivalent-fraction activity for the bound species, Davies for the aqueous ions). Divalent Ca2+ (logK 0.8) is equivalent-fraction-favoured and outcompetes Na+; watch the per-iteration selectivity ratio beta(CaX2)/beta(NaX)^2 climb.

CROSS-CHECK TOOL: PHREEQC EXCHANGE (same Gaines-Thomas half-reactions).

THE FRESH-RESIN CAVEAT (printed loudly, non-suppressible): the 'exchange' op returns the LIMITING (fully-equilibrated) effluent – the best a fresh / fully-loaded contactor does per pass. A real fixed bed leaks rising hardness as the exchange front migrates to breakthrough, then must be regenerated; cycle length and breakthrough are TRANSIENT and not modelled here.

Brackish water (I ~ 0.03-0.06): Davies-valid, no Pitzer needed; the honesty banner + the I>0.5 Davies advisory cover the boundary.

Data: curated USGS PHREEQC records (public domain) resolved from data/standards/ at runtime, + the SAC_Na resin nameplate; self-DESCRIBING (inline manifest) – seal with bin/choupo-import for a runtime-self-contained copy.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	raw	I_r0	0.0266002
diag	raw	I_rmax	0.165631
diag	raw	SI_anhydrite_r0	-1.68532
diag	raw	SI_anhydrite_rmax	-0.646016
diag	raw	SI_aragonite_r0	0.111093
diag	raw	SI_aragonite_rmax	1.26704
diag	raw	SI_calcite_r0	0.225093
diag	raw	SI_calcite_rmax	1.38104
diag	raw	SI_gypsum_r0	-1.38593
diag	raw	SI_gypsum_rmax	-0.349983
diag	raw	SI_halite_r0	-5.35323
diag	raw	SI_halite_rmax	-3.82475
diag	raw	SI_sylvite_r0	-6.39779
diag	raw	SI_sylvite_rmax	-4.87569

... and 193 more reference values in the case's `expected`.

tartaricAcid_acidulation

tutorials/props/electrolyte/tartaricAcid_acidulation

acidulation__pitzer speciate | **acidulation__davies** speciate | **gypsum__ceiling** speciate

pKa2__davies speciate | **pKa2__pitzer** speciate

What it does: Tartaric-acid acidulation liquor: rigorous Pitzer brine chemistry (Ca-SO4-K-Cl) + the gypsum precipitation ceiling, with tartrate honestly on the Debye-Hueckel limit (no published Pitzer params). Davies vs Pitzer shown side by side.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	acidulation_pitzer	I	1.6414
diag	acidulation_pitzer	SI_anhydrite	1.07386
diag	acidulation_pitzer	SI_arcanite	-2.80366
diag	acidulation_pitzer	SI_gypsum	1.36221
diag	acidulation_pitzer	SI_sylvite	-3.52403
diag	acidulation_pitzer	aw	0.986675
diag	acidulation_pitzer	gamma_Ca	0.0716778
diag	acidulation_pitzer	gamma_H2Tart	1
diag	acidulation_pitzer	gamma_HTart	0.409404
diag	acidulation_pitzer	gamma_SO4	0.0662673
diag	acidulation_pitzer	gamma_Tart	0.036756
diag	acidulation_pitzer	m_Ca	0.397714
diag	acidulation_pitzer	m_H2Tart	0.0656066
diag	acidulation_pitzer	m_HTart	0.0314016

. . . and 74 more reference values in the case's expected.

estimate

The theory you need. When a compound is missing, group contribution builds it: Joback sums first-order group increments into $T_b, T_c, P_c, V_c, \Delta H_f^\circ, \Delta H_{\mathrm{vap}}, c_p^{\mathrm{ig}}(T)$; Lee–Kesler closes ω from (T_b, T_c, P_c); Ambrose–Walton gives $P^{\mathrm{sat}}(T)$ from corresponding states; Rackett the liquid volume. The estimate is a PROPOSAL: a .dat you review against any reference data and promote by hand – the engine never estimates at runtime. Molecular structure is IDENTITY: components carry jobackGroups{} in their own record, and bin/estimate <name> is the one-command curation act. Expect few-percent errors on criticals (and the guide's worked example shows them), weaker T_b for hydrogen-bonded species, and polymers via van Krevelen / Yang on the repeat unit. *Full derivation: Theory Guide, the estimation chapter of the Props Guide.*

estimate_acetone

tutorials/props/estimate/estimate_acetone

estimate_acetone
estimateComponent

What it does: CREATE a component from its groups: Joback estimate of acetone (2 CH3 + 1 ketone), validated against NIST — see the estimate built up from groups

The story: The FIRST stage of the props lifecycle (CREATE → curate → wire → simulate): build a pure-component property set from its molecular GROUPS by Joback group contribution, and SEE each group's contribution add up.

Acetone CH3-CO-CH3 = 2 x CH3 + 1 x ketone (>C=O).

The reference block below validates FOUR numbers – Tb, Tc, Pc, omega – and no others: Joback lands 2.2 % low on Tb, 1.5 % low on Tc, 2.2 % high on Pc (read off the golden against the block, 2026-09-02), and you SEE the error, the whole point of "estimation, then trust". This header used to claim "~2 % on Tc/Pc/Vc and ~0.3 % on dHf"; the block carries no reference Vc and no reference dHf, so those two were claims nothing in the case checks. omega comes from (Tb,Tc,Pc) by Lee-Kesler, not a group sum.

Acetone is 2 × CH3 + 1 × ketone. This case runs under choupoProps and asks Joback's group-contribution method to create a property set from those three groups, then lays the estimate beside the reference values the case itself declares. The golden: Tb 322.11 K, Tc 500.56 K, Pc 48.02 bar, ? 0.30, Vc 209.5 cm^3/mol, ΔHf -217.83 kJ/mol.

1. Estimation is a resolution problem, not a runtime one. Choupo does not estimate properties while a flowsheet runs; it estimates them HERE, at curation time, so a student can look at the number before trusting it. output { proposal auto; } writes the estimate as a record you can read, edit, and only then wire into a case — the run prints where: constant/components/acetone.estimated.dat, with the GAPS the method left listed in its header. 2. The error is the point. Against the reference block in system/propsDict (Tb 329.2 K, Tc 508.1 K, Pc 47.0 bar, ? 0.307): Tb is 2.2 % low, Tc 1.5 % low, Pc 2.2 % high — Joback's usual band for a small ketone, and you can see it rather than be told. 3. ? is not a group sum. It comes from (Tb, Tc, Pc) through Lee-Kesler; an error in Tb propagates into it. That chain is printed in the run's log. 4. A reference is a citation, not an endorsement. The block names where its four numbers were read; the run compares against those four and nothing else.

Open estimate_ethanol_benzene: the same method on a hydrogen-bonding molecule, where Joback's Tb goes noticeably wrong — the honest limit of a group method, shown rather than warned about.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	estimate_acetone	Cp298	74.97
diag	estimate_acetone	Hvap_kJmol	27.53
diag	estimate_acetone	MW	58.08
diag	estimate_acetone	Pc_bar	48.02
diag	estimate_acetone	Psat_298_bar	0.39
diag	estimate_acetone	Psat_Tb_bar	0.98
diag	estimate_acetone	Tb_K	322.11
diag	estimate_acetone	Tc_K	500.56
diag	estimate_acetone	Vc_cm3mol	209.50
diag	estimate_acetone	Vliq298_cm3mol	82.11
diag	estimate_acetone	dGf_kJmol	-154.54
diag	estimate_acetone	dHf_kJmol	-217.83
diag	estimate_acetone	omega	0.30

estimate_ethanol_benzene

tutorials/props/estimate/estimate_ethanol_benzene

estimate__ethanol	estimate__benzene
estimateComponent	estimateComponent

What it does: Joback estimates: ethanol (H-bonding → Tb weak, the honest limit) and benzene (aromatic groups → good) — see where group contribution wins and loses

The story: Two Joback estimates, side by side, to SEE the method's spread:

ethanol (1 CH3 + 1 CH2 + 1 OH) – a strong H-bonder; Joback's Tb comes out ~14 K LOW (337 vs 351 K). The honest limit, in plain sight. benzene (6 arCH) – aromatic ring; Joback Tc/Tb within ~1.5%. Group contribution at its best.

The lesson is the CONTRAST: an estimate is a starting point whose error you must SEE, not a number to trust blindly.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	estimate_ethanol	Cp298	64.62
diag	estimate_ethanol	Hvap_kJmol	36.73
diag	estimate_ethanol	MW	46.07
diag	estimate_ethanol	Pc_bar	57.57
diag	estimate_ethanol	Psat_298_bar	0.16
diag	estimate_ethanol	Psat_Tb_bar	0.97
diag	estimate_ethanol	Tb_K	337.54
diag	estimate_ethanol	Tc_K	499.41
diag	estimate_ethanol	Vc_cm3mol	166.50
diag	estimate_ethanol	Vliq298_cm3mol	57.70
diag	estimate_ethanol	dGf_kJmol	-170.86
diag	estimate_ethanol	dHf_kJmol	-236.84
diag	estimate_ethanol	omega	0.57
diag	estimate_benzene	Cp298	81.82

. . . and 12 more reference values in the case's `expected`.

estimate_petroleum_cut

tutorials/props/estimate/estimate_petroleum_cut

estimate__petroleum__cut
estimateComponent

What it does: ESTIMATE a petroleum PSEUDO-COMPONENT by name from (Tb, SG) – Riazi-Daubert; see the SCREAMING ESTIMATE provenance and that the lump FLASHES

The story: CREATE a PETROLEUM PSEUDO-COMPONENT from the two bulk measurements every crude assay reports: the normal boiling point Tb and the specific gravity SG. The Riazi-Daubert (1987) correlations turn (Tb, SG) into the corresponding-states constants (MW, Tc, Pc, Vc); omega follows by Lee-Kesler, and the ideal-gas Cp by Kesler-Lee (1976).

This is the SCALAR-input sibling of the Joback group path: a petroleum cut is a LUMP of hundreds of species with NO single molecular structure, so it has NO group decomposition – it is anchored on bulk (Tb, SG), not groups. The estimator is a registered ConstantEstimator (‘model RiaziDaubert;‘), chosen exactly like Joback – the SAME explicit factory.

SEE in the console: the (Tb, SG) anchors and the Watson K factor; the constants built from them, validated against n-decane (a real species at this Tb/SG: Tc ~ exact, omega within ~2 %, Pc/MW within the correlation's stated ~5-7 %); the proposal .dat whose provenance SCREAMS status ESTIMATE / lumped true / "NOT a real species".

Promote with the printed ‘mv‘, then it FLASHES (see the sibling tutorial steady/flash/flash__pseudoComponent__petCut, which ships this very lump and closes an energy balance with it).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	estimate_petroleum_cut	Cp298	248.06
diag	estimate_petroleum_cut	MW	151.51
diag	estimate_petroleum_cut	Pc_bar	19.78
diag	estimate_petroleum_cut	Psat_Tb_bar	0.97
diag	estimate_petroleum_cut	SG	0.73
diag	estimate_petroleum_cut	Tb_K	447.30
diag	estimate_petroleum_cut	Tc_K	617.80
diag	estimate_petroleum_cut	Vc_cm3mol	627.46
diag	estimate_petroleum_cut	Vliq298_cm3mol	203.43
diag	estimate_petroleum_cut	omega	0.48
diag	estimate_petroleum_cut	Psat_298_bar	0.00

userEstimate01_linear_tb

tutorials/props/estimate/userEstimate01_linear_tb

tb__acetone
linearTbEstimate

What it does: User-code property estimation: a student's OWN linear group-contribution Tb estimator (linearTbEstimate), compiled from code/ and linked into choupoProps.

The story: A student's OWN property-estimation op (linearTbEstimate), defined in code/ and linked into choupoProps by bin/buildCode. Tb = base + sum(count∗incr). Here: 198.2 + 2∗23.58 + 1∗76.75 = 322.11 K (the Joback CH3,CH3,ketone sum). —

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`tb_acetone`	`Tb_K`	`322.11`

vanKrevelen01_polystyrene_density

tutorials/props/estimate/vanKrevelen01_polystyrene_density

vanKrevelen01__polystyrene__density
estimateComponent

What it does: ESTIMATE the density of polystyrene by Van Krevelen group contribution on the repeat unit – glass-box additive sum, Vw from Bondi 1964 (licence-clean via UNIFAC R)

The story: ESTIMATE a POLYMER's repeat-unit DENSITY by group contribution – the polymer sibling of the Joback small-molecule path. Polystyrene's repeat unit -[CH2-CH(C6H5)]- is decomposed into groups; each adds its molar mass and its Bondi van der Waals volume Vw; the sums give

M0 = sum n__i MW__i repeat-unit molar mass Vw = sum n__i Vw__i van der Waals volume (Bondi 1964) V = k ∗ Vw molar volume (k = packing factor) rho = M0 / V density

GLASS-BOX: the console prints every group's contribution – you can redo the whole estimate by hand. M0 is an exact mass-balance fact (104.15 g/mol); rho carries the packing-factor uncertainty (the teaching point below).

LICENCE (the honest part): the Vw group values are Bondi, J. Phys. Chem. 68 (1964) 441 – the PRIMARY per value – derived from the UNIFAC R__k volume parameters Choupo already ships (Vw = R__k x 15.17), NOT copied from the Van Krevelen book table (copyright). See data/standards/parameters/vanKrevelen.dat. Slice 1 ships DENSITY only; Tg / solubility-parameter / Tm are DEFERRED (their open parameters are a separate, correctly-attributed thing).

THE TEACHING POINT – the packing factor k: rho depends on k = V/Vw, the fraction of space the chains do NOT pack into. Default k = 1.60 (amorphous/glassy); crystalline packs denser (~1.43). k is YOURS: change 'polymer { packing k; }' and watch rho move. Atactic PS is amorphous (rho ~ 1.05) – the estimate lands there; a crystalline polymer would need the smaller k. That k is visible (never hidden) is the lesson.

M0 = 104.15 also feeds the step-growth reactor KPIs (Mn = M0/(1-p)) – the same repeat-unit mass, two places.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	vanKrevelen01_polystyrene_density	M0_g_per_mol	104
diag	vanKrevelen01_polystyrene_density	V_cm3_per_mol	101
diag	vanKrevelen01_polystyrene_density	Vw_cm3_per_mol	62.9
diag	vanKrevelen01_polystyrene_density	density_g_cm3	1.04
diag	vanKrevelen01_polystyrene_density	packing_k	1.6

yang2020_Tg_polystyrene

tutorials/props/estimate/yang2020_Tg_polystyrene

yang2020__Tg__polystyrene
estimateComponent

What it does: ESTIMATE the glass-transition Tg of polystyrene by the Yang 2020 modified group-contribution scheme (CC-BY) – the repeat unit -CH2-CH(phenyl)- gives Tg(inf)=444 K, an HONEST +19% over-estimate of the measured ~373 K (a teaching point about group-contribution limits)

The story: Yang-2020 Tg estimate + an HONEST limitation

ESTIMATE the GLASS-TRANSITION temperature Tg of polystyrene by the YANG 2020 modified group-contribution scheme. The repeat unit -CH2-CH(phenyl)- decomposes into two Yang main-chain groups:

CH2 : MW 14 Yg 4.026 CHphenyl: MW 90 Yg 42.153

M0 = 14 + 90 = 104 g/mol Yg(inf) = 4.026 + 42.153 = 46.179 (1e3 g.K/mol) Tg(inf) = 46.179*1e3 / 104 = 444 K

»> THE HONEST TEACHING POINT «< Measured polystyrene Tg ~ 373 K (atactic, high Mw). Yang 2020 gives 444 K – a +19% OVER-ESTIMATE. This is NOT a bug: it is the model telling the truth about itself. The CHphenyl contribution (Yg 42.153) was regressed across the whole training set, and the bulky monosubstituted phenyl pendant is one where an additive main-chain scheme over-predicts the limiting Tg. Box: "all models are wrong, some are useful". The student SEES the deviation (the console prints the per-cent error against the measured anchor) and learns where a group-contribution method earns trust (PVC: 351 vs 354 K, -0.8%) and where it does not (polystyrene: 444 vs 373 K, +19%).

Contrast the flagship ‘yang2020__Tg__pvc‘, where Tg(inf)=351 K reproduces the paper’s Fig 6 value EXACTLY – that case goldens Tg as validated physics. Here Tg is SHOWN (the machinery is identical) but the case exists to expose the over-estimate, so only M0 (an exact mass-balance fact) is goldened, not Tg.

This is Yang 2020 – NOT bare Van Krevelen. Nb (backbone-in-the-side-chain) = 0 for this vinyl polymer.

SOURCE + LICENCE: Yang, Zou, Ye, Zhu, Dong, Bi, ACS Omega 5 (2020) 19655, doi:10.1021/acsomega.0c04499 – CC-BY 4.0; values cited per group as "Yang 2020", never "Van Krevelen".

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	yang2020_Tg_polystyrene	M0_g_per_mol	104
diag	yang2020_Tg_polystyrene	Tg_K	444
diag	yang2020_Tg_polystyrene	YgSum_1e3gKmol	46.18

yang2020_Tg_pvc

tutorials/props/estimate/yang2020_Tg_pvc

yang2020__Tg__pvc
estimateComponent

What it does: ESTIMATE the glass-transition Tg of poly(vinyl chloride) (PVC) by the Yang 2020 modified group-contribution scheme (CC-BY) – the repeat unit -CH2-CHCl- gives Tg(inf)=351 K, EXACT vs the paper (Fig 6)

The story: FLAGSHIP Yang-2020 glass-transition estimate

ESTIMATE the GLASS-TRANSITION temperature Tg of poly(vinyl chloride) (PVC) by the YANG 2020 modified group-contribution scheme – the polymer sibling of the Joback small-molecule path, for Tg. The repeat unit -CH2-CHCl- is decomposed into two Yang main-chain groups; each adds its limiting Tg contribution Yg(inf):

M0 = sum n_i MW_i repeat-unit molar mass [g/mol] Yg(inf) = sum n_i Yg_i(inf) (+ Nb∗Ygb) limiting Tg function [1e3 g.K/mol] Tg(inf) = Yg(inf) ∗ 1e3 / M0 Tg at infinite Mw [K]

THE ADDITIVE SUM (shown on the console): CH2 : MW 14 Yg 4.026 CHCl: MW 48.5 Yg 17.911

M0 = 14 + 48.5 = 62.5 g/mol Yg(inf) = 4.026 + 17.911 = 21.937 (1e3 g.K/mol) Tg(inf) = 21.937∗1e3 / 62.5 = 351 K <– EXACT vs Yang 2020, Fig 6

This is Yang 2020 – NOT bare Van Krevelen. The scheme predicts the INFINITE- Mw limit Tg(inf); the molecular-weight dependence Tg(Mn) = Tg(inf) - K/Mn is a later slice. Nb (backbone-in-the-side-chain) = 0 for this vinyl polymer, so the bracket is just the sum of the two main-chain group contributions.

SOURCE + LICENCE (the honest part): Yang, Zou, Ye, Zhu, Dong, Bi, ACS Omega 5 (2020) 19655, doi:10.1021/acsomega.0c04499 – CC-BY 4.0, so the parameter values ARE redistributable. Cited per value as "Yang 2020", never "Van Krevelen".

THE GOLDEN: M0 (an exact mass-balance fact) AND Tg(inf)=351 K are checked – Tg here is VALIDATED PHYSICS (it reproduces the paper's Fig 6 value exactly), not just a mass balance. The measured anchor (PVC Tg ~ 354 K) confirms it.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	yang2020_Tg_pvc	M0_g_per_mol	62.5
diag	yang2020_Tg_pvc	Tg_K	351
diag	yang2020_Tg_pvc	YgSum_1e3gKmol	21.937

fit

cp01_polynomial_recovery

tutorials/props/fit/cp01_polynomial_recovery

cp__dce__liquid__fit
heatCapacityFit

What it does: Quadratic Cp fit on data generated from a quadratic – exact recovery, with the Kirchhoff Cp_ig - Cp_liq cross-check that ties Cp to dHvap

The story: A LINEAR FIT, AND THE PHYSICS THAT CHECKS IT.

'heatCapacityFit' regresses Cp(T) = A0 + A1 T + A2 T^2 by linear least squares – normal equations, no iteration. Fed data generated FROM a quadratic, it must return that quadratic, so the expected file pins the coefficients rather than a tolerance band.

The interesting number is not the fit. It is KIRCHHOFF:

d(dHvap)/dT = Cp_ig - Cp_liq

The op takes the FITTED liquid Cp and the component's CATALOGUE ideal-gas Cp and reports that difference at the mid temperature. It should be NEGATIVE – the enthalpy of vaporisation falls as T rises, to zero at the critical point – and its size is the slope of dHvap(T). That is the link tying Cp to dHvap to the Antoine fit next door (psat01__antoine__recovery, which reports dHvap from B): three ops, one consistency network, and a student can check one against the other by hand.

Why the case exists: 'heatCapacityFit' had NO case and no gate. Writing the first one for its sibling vaporPressureFit segfaulted choupoProps twice and found a thermoPhysPropDict being silently ignored.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	cp_dce_liquid_fit	A0	194
diag	cp_dce_liquid_fit	A1	-0.618063
diag	cp_dce_liquid_fit	A2	0.00170318
diag	cp_dce_liquid_fit	Cp_at_Tmid	166.076
diag	cp_dce_liquid_fit	Cp_ig_at_Tmid	84.2326
diag	cp_dce_liquid_fit	R2	1
diag	cp_dce_liquid_fit	dHvap_dT_kirchhoff	-81.8435
diag	cp_dce_liquid_fit	degree	2
diag	cp_dce_liquid_fit	n_points	21

psat01_antoine_recovery

`tutorials/props/fit/psat01_antoine_recovery`

psat__water__fit
vaporPressureFit

What it does: Antoine fit on data generated from Choupo's own water correlation: the coefficients must come back exactly, so any deviation is the op's arithmetic

The story: FITTING ANTOINE, AND KNOWING THE ANSWER BEFOREHAND.

'vaporPressureFit' regresses log10(Psat[bar]) = A - B/(T + C) on measured Psat(T). The fit is linear in (A, B) at fixed C, so the op golden-sections C and solves (A, B) by least squares at each one – derivative-free, and worth reading in the source.

This case feeds it data generated FROM THE SAME EQUATION, by evaluating Choupo's own curated water correlation at twenty temperatures. That is deliberate. Against laboratory data a fit can only be judged to the scatter of the measurements; against data the equation itself produced, the fit must RETURN THE COEFFICIENTS IT WAS GIVEN – so any deviation is the op's arithmetic and nothing else, and the expected file can pin it to the digit.

Why the case exists: 'vaporPressureFit' had NO case and no gate. It could have been broken for a year and the suite would have stayed green; the first to find out would be a student fitting their own data. The psychrometric chart was in exactly that position last week, and writing its first test found it reporting nothing.

What to read in the output: A, B, C – the recovered coefficients, against 5.40221 / 1838.675 / -31.737; R2 – 1 to floating point, because the data has no scatter; dHvap – the Clausius-Clapeyron reading of B WITH the (T/(T+C))^2 correction, at the highest T. Compare it to water's tabulated enthalpy of vaporisation: this is the physical cross-check on a fit that is otherwise just a straight line through transformed numbers.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	psat_water_fit	A	5.40221
diag	psat_water_fit	B	1838.68
diag	psat_water_fit	C	-31.737
diag	psat_water_fit	R2	1
diag	psat_water_fit	dHvap_kJ_per_mol	42.1162
diag	psat_water_fit	n_points	20

gibbs

The theory you need. Equilibrium without a mechanism: minimise total Gibbs energy $G = \sum_i n_i\,\mu_i$ over ALL species subject to elemental balances $\mathbf{A}\,\mathbf{n} = \mathbf{b}$ (Lagrange multipliers per element – printed as λ_E, the element potentials). The species list in the dict IS the model: what you include can form; what you omit cannot. With enough species this reproduces flame temperatures and radical pools with no kinetics at all: expect the adiabatic flame temperature to land within a few K of literature when radicals (H, O, OH…) are IN the pool, and to overshoot by ∼100–250 K when they are not (dissociation is endothermic). Above ∼1500 K the catalogue's polynomial c_p fits extrapolate WRONG – combustion cases carry case-local NASA-7 overrides, and the case headers say so.

Equilibrium maps. Sweeping the Gibbs solve over a $T \times P$ grid gives an equilibrium map – iso-lines of a product fraction over the operating plane – which shows why an industrial reactor fixes T and P in a narrow window (equilibrium wants one corner, the catalyst works in another; the gap is the compromise). The `temperatureApproach` ΔT evaluates the *reaction* equilibrium at $T + \Delta T$ while enthalpy and the energy balance stay at the physical T – an empirical, calibrated closeness-to-equilibrium knob (never predicted). *Gradeable exercise (ammonia):* on `nh3_equilibrium_map`, at $P = 200$ bar and $\Delta T = 0$, find the temperature at which the equilibrium NH_3 mole fraction crosses 30 %; report T to the grid resolution and verify against the case's anchor KPIs. Unique because the fraction is monotone in T at fixed P. *Full derivation: Theory Guide, the Gibbs-equilibrium chapter, §equilibrium maps.*

`nh3_equilibrium_map`

`tutorials/props/gibbs/nh3_equilibrium_map`

haber_map
gibbsMap

What it does: Ammonia equilibrium map (T x P): why Haber runs at 400-500 C and 150-300 bar

The story: THE HABER MAP – the forum-ratified Gibbs equilibrium map (docs/design/gibbs-map-forum-2026-07-02.md) on its canonical pilot: N2 + 3 H2 at stoichiometric feed, equilibrium NH3 mole fraction contoured over 300-700 C x 1-300 bar. The map shows the whole Haber story at a glance: equilibrium loves COLD and PRESSURE (exothermic, Delta_n < 0); the industrial window (400-500 C, 150-300 bar) sits where the catalyst can actually work – the tension IS the lesson.

VALIDATION ANCHORS (golden-locked; verified 2026-07-03): the classic measurements of Larson & Dodge, J. Am. Chem. Soc. 45 (1923) 2918 – 400 C / 10 atm: engine 4.09 % vs L&D 3.85 % (+6 %) 450 C / 10 atm: engine 2.22 % vs L&D 2.11 % (+5 %) The ~5 % offset is consistent with propagating the JANAF formation-data uncertainty of NH3 into Kp (exp(-dG/RT) amplifies ~0.2 kJ/mol into ~5 %) – honest agreement for an equilibrium constant spanning decades. IDEAL-GAS VALIDITY: the kernel is ideal-gas, so pins stay at <= 50 atm, 350-500 C (the forum's chair: at 300 bar a temperatureApproach would silently absorb the missing fugacity corrections – announced, and the high-P corner of this map is drawn as EQUILIBRIUM SHAPE, not quantitative prediction).

temperatureApproach: 0 here (true equilibrium). Re-run with e.g. 'temperatureApproach 25;' to see the contours slide – the empirical closeness-to-equilibrium parameter industry quotes in C (announced loud, the anchor KPI names then carry _dT25 so the golden breaks visibly).

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`haber_map`	`metric_T400C_P10atm`	`0.0409234`
`diag`	`haber_map`	`metric_T400C_P50atm`	`0.157785`
`diag`	`haber_map`	`metric_T450C_P10atm`	`0.0221983`
`diag`	`haber_map`	`metric_T500C_P100atm`	`0.104969`
`diag`	`haber_map`	`n_cells`	`625`
`diag`	`haber_map`	`n_unconverged`	`0`

heat

The theory you need. Heat exchange couples streams without mixing them: $Q = U\,A\,\Delta T_{\mathrm{lm}}$, effectiveness-NTU when outlets are unknown. Utilities (steam levels, cooling water) are allocated by temperature level – a reboiler cannot be heated by a colder utility, and the allocator says which level it picked. Heat-links (forward and feedback) couple distant units into an energy tear the solver converges like any recycle. Pinch analysis finds the minimum utilities the composite curves allow.

Exchanger DESIGN vs rating. A shell-and-tube poses two inverse questions: RATE it (geometry given → find U, area, ΔP, duty – `model geometry;`) or DESIGN it (duty given → size the geometry – `model design;`). U is BUILT from the two film coefficients (Gnielinski tube, Kern shell) and the wall; the pressure drop (Kern both sides) is the duty-vs-pumping half a real design lives by. The standard workflow – establish the duty, THEN design, THEN rate, never invent the geometry up front – is walked end-to-end in the two-part sequence `hxWorkflow1_design_from_duty` → `hxWorkflow2_rate_designed`, and the design ends in a printable datasheet (GUI: the unit panel). *Full derivation: Theory Guide, the heat-integration chapter, §heat-exchanger design.*

htbench01_correlations

tutorials/props/heat/htbench01_correlations

bench
heatTransferBench

What it does: Heat-transfer correlation bench – each convective correlation (Dittus-Boelter, Gnielinski, Kern) verified against its published Nusselt anchor. Dittus-Boelter and Gnielinski are pinned to Incropera & DeWitt 6th ed. Example 8.4; Kern's closed j_H form is self-consistency-checked (its original pin is a chart). AAD tolerance 5% honours textbook rounding + the correlations' own scatter.

The story: the heat-transfer correlation self-check bench.

Mirrors the membrane mass-transfer story (Sherwood) on the heat side (Nusselt): each registered convective correlation RUNS its verify() against a published Nu anchor and the deviation is printed + locked.

DittusBoelter Nu = 0.023 Re^0.8 Pr^0.4 (Incropera Ex 8.4, Nu=205) Gnielinski Nu = (f/8)(Re-1000)Pr/[..] (Incropera Ex 8.4 alt, Nu=224) Kern (shell) Nu = 0.36 Re_s^0.55 Pr^1/3 (Kern PHT; closed-form check)

The per-correlation deviation diags are the golden KPIs.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	bench	Nu_choupo_DittusBoelter	200.475
diag	bench	Nu_choupo_Gnielinski	223.734
diag	bench	Nu_choupo_Kern	97.5647
diag	bench	Nu_published_DittusBoelter	205
diag	bench	Nu_published_Gnielinski	224
diag	bench	Nu_published_Kern	97.5647
diag	bench	allPass	1
diag	bench	dev_DittusBoelter	0.0220718
diag	bench	dev_Gnielinski	0.00118956
diag	bench	dev_Kern	0
diag	bench	dev_NusseltFilm	0.0245797
diag	bench	dev_Rohsenow	0.000270621
diag	bench	dev_ZuberCHF	0.000684704
diag	bench	h_choupo_NusseltFilm	6680.26

... and 5 more reference values in the case's `expected`.

psychro01_n2_water

tutorials/props/heat/psychro01_n2_water

airWater
psychrometricChart

What it does: Psychrometric chart for N2/water at 1 atm, 0-60 C: saturation + four RH lines + three adiabatic-saturation lines, from the vapour-pressure model a flash would use

The story: THE PSYCHROMETRIC CHART, computed rather than looked up.

Nitrogen + water at 1 atm, 0 → 60 C: the saturation curve, four relative-humidity lines and three adiabatic-saturation (wet-bulb) lines, all from the SAME vapour-pressure model a flash would use. Nothing here is a chart digitised from a handbook – the curve is Y_sat(T) evaluated point by point, so the student can change the model and watch the chart move.

Why this case exists at all: ‘psychrometricChart‘ is reachable from the GUI's Property Explorer, and until now NOTHING exercised it. It could break and the whole suite would stay green; the first to find out would be a student opening the Explorer. An op a user can reach and a test cannot is a promise with no guard behind it.

What to read in the output (long-format CSV: T_C, Y, curve): ‘saturation‘ – Y_sat = 0.622 ∗ Psat/(P - Psat), the chart's ceiling; ‘RH=...‘ – the same expression at a fraction of Psat, so the lines fan out and steepen with T exactly as the printed chart does; ‘wetBulb=...‘ – the adiabatic-saturation lines, which are NOT parallel to the RH lines: they carry the latent heat of the water evaporated, so their slope is set by Hvap/cp of the moist air, not by geometry.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	airWater	Y_adiabatic_last	0.0150791
diag	airWater	Y_sat_first	0.00390022
diag	airWater	Y_sat_last	0.158606
diag	airWater	nRhCurves	4
diag	airWater	nSaturationPoints	61
diag	airWater	Lewis_last	0.816578
diag	airWater	nAdiabaticCurves	3
diag	airWater	nTrueWetBulbCurves	3

psychro02_air_water

tutorials/props/heat/psychro02_air_water

airWater
psychrometricChart

What it does: Psychrometric chart with dry AIR as the carrier (the textbook object): saturation, RH, adiabatic-saturation and true wet-bulb families at 101.325 kPa, Lewis ~ 0.851 – the case-local air component deliberately shadows the catalogue mixture alias

The story: THE TEXTBOOK CHART: dry AIR as the carrier, not a stand-in. Twin of psychro01_n2_water with the REAL carrier – the lumped pseudo-component air (MW 28.96, ChemSep v8.3), which this case declares CASE-LOCALLY in constant/components/air.dat. That declaration is load-bearing, not a convenience: the catalogue also holds mixtures/air.dat (an expansion table, air → N2 + O2 + Ar), and the settled precedence – the case always overrides – is what makes the token load as the COMPONENT here; the run announces the shadowed expansion. Until 2026-08-31 that tie went to the mixture unconditionally and this exact case refused with "carrier 'air' is not a component".

What the twin buys over psychro01: Y_sat(60 C) = 0.1534 against the ~0.152 a printed air chart reads (N2 gives 0.1586 – the 3.4 % MW error the registry note predicted), and Lewis = 0.851, the handbook air/water value the Le^(2/3) wet-bulb family exists to teach.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	airWater	Lewis_last	0.850543
diag	airWater	Y_adiabatic_last	0.0146521
diag	airWater	Y_sat_first	0.00377268
diag	airWater	Y_sat_last	0.15342
diag	airWater	nAdiabaticCurves	3
diag	airWater	nRhCurves	4
diag	airWater	nSaturationPoints	61
diag	airWater	nTrueWetBulbCurves	3

hydraulics

The theory you need. Pressure drop and flow distribution: Darcy–Weisbach with friction factors, fittings as equivalent lengths, pump curves against system curves. Expect the operating point at the curves' intersection.

moody01_friction_correlations

`tutorials/props/hydraulics/moody01_friction_correlations`

moody
frictionBench

What it does: The Moody chart's four correlations, each verified against its own published anchor and then all asked the same question – so the disagreement is visible instead of hidden behind a default

The story: moody01 – FOUR CORRELATIONS, ONE QUESTION, FOUR ANSWERS.

The Darcy friction factor is the most-used correlated quantity in chemical engineering, and every simulator computes it. What none of them shows is that the answer DEPENDS ON WHICH CORRELATION YOU PICKED.

Choupo has computed Blasius, Colebrook-White, Haaland and Churchill since the pipe unit was written – correctly, with their citations in comments, as free functions nobody could name or reach. This case is what happens when they become objects: each states the window its own authors gave it, each carries its primary citation, each reproduces a published anchor on every run, and all four can be asked the same question at once.

WHAT THE RUN DOES, IN TWO PARTS.

(1) VERIFY. Each correlation reproduces its own anchor, and the anchors are deliberately of DIFFERENT KINDS, because what can be checked differs:

Blasius against its own closed form. This checks the ARITHMETIC and nothing else, and the bench says so rather than letting a 0.000 % deviation look like a validation. Colebrook against the fully-rough von Karman limit, where the implicit equation has a closed form. This checks that the FIXED-POINT ITERATION reaches the answer it must. Haaland against COLEBROOK at the same point – because Haaland's own published claim is "I reproduce that equation within about 2 %". Anchoring it to anything else would be checking a claim nobody made. Churchill against the EXACT laminar law f = 64/Re, which its single expression contains as its own low-Re limit. This is the strongest of the four: a mis-typed exponent fails here while the turbulent branch still looks plausible.

(2) COMPARE. All four are evaluated at Re = 1e5, eps/D = 1e-3 – an ordinary turbulent water line in a commercial steel pipe – and the spread is published as a number.

WHAT TO READ IN THE OUTPUT. Blasius is the one to watch: it carries NO roughness term at all, so at eps/D = 1e-3 it answers as though the wall were smooth, and it is simply wrong. It says so on its own line. That is the whole lesson of a validity window – not that the correlation is bad, but that it was asked a question it was never fitted for, and only the window can tell you.

WHAT THIS CASE DOES NOT DO. It does not say which correlation is right. That depends on the pipe – its roughness, whether the flow is developed, whether the duct is even circular – and a bench that ranked them would hide the choice the engineer has to make, which is exactly what a default does today in every tool that has one.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	moody	allPass	1
diag	moody	dev_Blasius	0
diag	moody	dev_Churchill	1.73472e-16
diag	moody	dev_Colebrook	1.61329e-05
diag	moody	dev_Haaland	0.0134393
diag	moody	f_Blasius	0.01777
diag	moody	f_Churchill	0.0223432
diag	moody	f_Colebrook	0.0221745
diag	moody	f_Haaland	0.0219662
diag	moody	f_max	0.0223432
diag	moody	f_min	0.01777
diag	moody	n_outside_window	1
diag	moody	spread_inWindow_pct	1.71637
diag	moody	spread_pct	25.7358

kinetics

`rate01_arrhenius_recovery`

`tutorials/props/kinetics/rate01_arrhenius_recovery`

rate_arrhenius
kinetics1D

What it does: Second-order decay at three temperatures, generated from k0 = 4e6, Ea = 60 kJ/mol: the Arrhenius line must give both back

The story: THE ACTIVATION ENERGY COMES OUT OF A SLOPE.

'kinetics1D' is a PROPERTY evaluation, not a reactor: it uses the integrated rate law in closed form, so a student can overlay competing orders on the same lab data and SEE which one is straight. Given isotherms at more than one temperature it goes further – fits k on each one, then the line

ln k = ln k0 - (Ea/R) (1/T)

and Ea falls out of the SLOPE. That is the whole of Arrhenius, and it is worth watching happen on data whose answer is known.

So the data here is generated from k0 = 4.0e6 L/(mol s), Ea = 60 kJ/mol, second order, at 40/60/80 C. The fit must return those two numbers; the expected file pins them, and R2 = 1 because the points carry no scatter.

What to read: the per-isotherm k values (3.93e-4, 1.57e-3, 5.34e-3), which span a factor of 13.6 over 40 K – the reason a rate constant is never quoted without its temperature.

Why the case exists: 'kinetics1D' had NO case and no gate. Its sibling vaporPressureFit was in the same position, and writing the first case for THAT one segfaulted choupoProps twice.

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`rate_arrhenius`	`Ea`	`60000`
`diag`	`rate_arrhenius`	`Ea_kJ_per_mol`	`60`
`diag`	`rate_arrhenius`	`R2`	`1`
`diag`	`rate_arrhenius`	`arrhenius`	`1`
`diag`	`rate_arrhenius`	`fitted`	`1`
`diag`	`rate_arrhenius`	`k0`	`4e+06`
`diag`	`rate_arrhenius`	`n_temps`	`3`
`diag`	`rate_arrhenius`	`order`	`2`

molecular

The theory you need. Molecular activity models on display: NRTL/Wilson/UNIQUAC correlate binary γ's from pair parameters; UNIFAC PREDICTS them from group counts. Expect UNIFAC within a few percent of fitted models for families it was trained on – and to be honestly worse outside them. *Full derivation: Theory Guide, the activity-model chapter.*

cosmoSAC01_water_ethanol

tutorials/props/molecular/cosmoSAC01_water_ethanol

What it does: COSMO-SAC 2002 (NIST benchmark, VT-2005 profiles): pure→lngamma=0, binary/ternary multicomponent gammas, no NaN.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	pure_water	n_failures	0
diag	pure_water	n_points	2
diag	bin_wet	n_failures	0
diag	bin_wet	n_points	2
diag	ternary	n_failures	0
diag	ternary	n_points	2
diag	water_hexane	n_failures	0
diag	water_hexane	n_points	2
diag	pure_water	n_emptyCurves	0
diag	bin_wet	n_emptyCurves	0
diag	ternary	n_emptyCurves	0
diag	water_hexane	n_emptyCurves	0

entropy01_air_ledger

tutorials/props/molecular/entropy01_air_ledger

ledger propertyPoint | **explainS** explainProperty | **explainH** explainProperty

temperatureLine propertyScan1D | **pressureLine** propertyScan1D

What it does: The entropy ledger of 79/21 N2/O2 at 400 K, 2 bar on SRK: every line of s(T,P,y) published separately, so a reader can rebuild the total by hand

The story: THE ENTROPY LEDGER, published line by line – the witness behind the "What is entropy?" EduTool page.

The engine's mixture entropy is assembled in ThermoPackage::S_ig (src/thermo/ThermoPackage.cpp) as

S_ig(T,P,y) = sum_i y_i * s_ig_i(T) (pure-state line) - R * sum_i y_i * ln y_i (mixing line) - R * ln(P / 1 bar) (pressure line)

with each s_ig_i(T) = s_298_i + integral Cp_i/T dT off the component's own .dat (Component::s_formation), and S_real = S_ig + S_residual(SRK) (the model line, Sandler eq. 6.4-31). The 'ledger' op below publishes the per-component s_ig_i AND the assembled S_ig / S_R / S_real at one state, so a reader – or the EduTool page in the browser – can rebuild the total from the lines and check it against the engine's own number. The two scans draw the temperature line (integral Cp/T at fixed P) and the pressure line (-R ln P at fixed T, with the residual gap growing).

Nothing here is a validation against measurement: every number is the engine's own answer, decomposed. The value of the case is that the decomposition CANNOT drift from the simulator, because it is published by the same call that computes it.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	ledger	Cp_ig	29.4637
diag	ledger	H_R	-5.4858
diag	ledger	H_ig	2986.1689
diag	ledger	H_real	2980.6831
diag	ledger	P	200000.0000
diag	ledger	S_R	-0.0192
diag	ledger	S_ig	201.5778
diag	ledger	S_real	201.5586
diag	ledger	T	400.0000
diag	ledger	Z	1.0007
diag	ledger	gamma	1.3931
diag	ledger	s_ig_N2	200.1887
diag	ledger	s_ig_O2	213.8979
diag	ledger	v_molar	0.0166

… and 36 more reference values in the case's `expected`*.*

envelope01_natural_gas

tutorials/props/molecular/envelope01_natural_gas

envelope
phaseEnvelope

What it does: P-T phase envelope of a lean natural gas: the dew branch over its cricondentherm, and a bubble branch that never starts

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	envelope	bubble_points	0
diag	envelope	bubble_stopped_bar	1
diag	envelope	dew_cricondentherm_K	274.705
diag	envelope	dew_points	40

`envelope02_root_jump`

`tutorials/props/molecular/envelope02_root_jump`

envelope
phaseEnvelope

What it does: P-T envelope where the saturation solve jumps root: both branches stop AT the jump and report it as one, and no cricondentherm is claimed

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`envelope`	`bubble_points`	`17`
`diag`	`envelope`	`bubble_stopped_bar`	`18`
`diag`	`envelope`	`dew_points`	`25`
`diag`	`envelope`	`dew_stopped_bar`	`26`

`eval_nrtl_catalogue`

`tutorials/props/molecular/eval_nrtl_catalogue`

catalogue__as__assembled
fitParameters

What it does: EVALUATE mode: score the CATALOGUE NRTL pair against your own data – the scale before any fit

The story: 'mode evaluate': the SCALE, not the fit.

ONE operation, one question: how good is the ethanol-water NRTL pair against this 1-atm bubble-T dataset? Nothing is adjusted, nothing is written. The thermoPhysPropDict declares NO inline pairs, so what gets scored is a REAL provenance-bearing pair resolved through the search path (from this case's own vendored constant/parameters/ copy, identical parameters and full primary citation) – not numbers typed inline into the dict.

Read the verdict (rms/aad in kelvin) and THEN decide whether a refit is worth it: compare with fitNRTL01's fitted rms (0.31 K) on this same data. Failures are honest: a non-converged point becomes status=nonconverged in the parity CSV and never enters the statistics; no chi2_reduced is printed – without sigmas and degrees of freedom from THIS act, the name would lie.

The pinned-what-if grammar and the leakage sentinel live in the sibling case eval_nrtl_pinned (a pinned path must EXIST in the dict, so what-ifs run on an INLINE scaffold and say so).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	catalogue_as_assembled	aad	2.0400
diag	catalogue_as_assembled	chi2	60.1945
diag	catalogue_as_assembled	max_abs_resid	3.4379
diag	catalogue_as_assembled	max_resid_x1	0.5000
diag	catalogue_as_assembled	mode_evaluate	1.0000
diag	catalogue_as_assembled	n_data	11.0000
diag	catalogue_as_assembled	n_feasible	11.0000
diag	catalogue_as_assembled	partial	0.0000
diag	catalogue_as_assembled	penalised_points	0.0000
diag	catalogue_as_assembled	rms	2.3393
diag	catalogue_as_assembled	sse	60.1945
diag	catalogue_as_assembled	validity_declared	1.0000
diag	catalogue_as_assembled	n_outside_validity	0.0000

eval_nrtl_pinned

`tutorials/props/molecular/eval_nrtl_pinned`

scaffold__as__is fitParameters	rough__paper__values fitParameters	scaffold__again fitParameters

What it does: EVALUATE a pinned what-if: a pinned run scores the package but mutates NOTHING (the leakage sentinel)

The story: the pinned WHAT-IF, and the LEAKAGE SENTINEL.

TRANSITIONAL SCAFFOLD: pinning by dict path requires an inline pair; the permanent surface (ParameterOverrides on the RESOLVED parameter registry, design forum #67/#69) replaces this case when it lands.

Three evaluations of the same dataset: 1. scaffold__as__is – the inline pair, untouched (the baseline). 2. rough__paper__values – the SAME dict with four coefficients PINNED to deliberately rough values. The pin operates on a DEEP COPY (Dictionary::deepCopy, forum #63-7); nothing on disk or in memory survives the op. 3. scaffold__again – repeats 1 AFTER the pinned run. Its verdict must be IDENTICAL to 1's: the golden pins both, so the working-tree isolation is guarded by the corpus forever. Before deepCopy existed, the old alias bug would make THIS operation report the rough values' score.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	scaffold_as_is	aad	2.0400
diag	scaffold_as_is	chi2	60.1945
diag	scaffold_as_is	max_abs_resid	3.4379
diag	scaffold_as_is	max_resid_x1	0.5000
diag	scaffold_as_is	mode_evaluate	1.0000
diag	scaffold_as_is	n_data	11.0000
diag	scaffold_as_is	n_feasible	11.0000
diag	scaffold_as_is	partial	0.0000
diag	scaffold_as_is	penalised_points	0.0000
diag	scaffold_as_is	rms	2.3393
diag	scaffold_as_is	sse	60.1945
diag	rough_paper_values	aad	1.5942
diag	rough_paper_values	chi2	61.7621
diag	rough_paper_values	max_abs_resid	5.5866

... and 21 more reference values in the case's `expected`.

exergy01_air_dead_state

tutorials/props/molecular/exergy01_air_dead_state

b__state exergy | **b__dead** exergy

What it does: Physical exergy of 79/21 N2/O2 at 400 K, 2 bar against a DECLARED 298.15 K / 1 bar dead state – b = (h-h0) - T0(s-s0) from the engine's own H_real/S_real, with the structural zero at the dead state itself as the second row

The story: PHYSICAL EXERGY, the number the entropy ledger buys – the witness behind the exergy work that follows the "What is entropy?" page.

b = (h - h0) - T0∗(s - s0), every term the engine's own H__real/S__real at the SAME composition, so the enthalpy datum, the s__298 anchors, the mixing term and the reference pressure ALL cancel: the exergy is datum-independent. The dead state is DECLARED (an exergy is a statement about a state AND an environment; the engine refuses to assume one).

Two rows on purpose: the same state the entropy01 ledger decomposes, and the dead state itself – whose exergy is 0 identically, by construction, which the golden pins as the structural zero.

Nothing here is validated against measurement: every number is the engine's own answer, decomposed. The published dh and T0ds legs re-add to b on the reader's desk.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	b_state	P	200000
diag	b_state	P0	100000
diag	b_state	T	400
diag	b_state	T0	298.15
diag	b_state	T0ds	850.601
diag	b_state	b_physical	2136.71
diag	b_state	dh	2987.32
diag	b_state	h	2980.68
diag	b_state	h0	-6.63211
diag	b_state	s	201.559
diag	b_state	s0	198.706
diag	b_dead	P	100000
diag	b_dead	P0	100000
diag	b_dead	T	298.15

... and 8 more reference values in the case's `expected`.

flash01_operating_line

`tutorials/props/molecular/flash01_operating_line`

txy
propertyScan1D

What it does: The equilibrium curve a flash operating line is drawn on

The story: THE CURVE, AND ONLY THE CURVE.

A binary flash is fixed by two of {T, P, V/F} – Duhem – so at a frozen pressure the equilibrium curve $y*(x)$ is frozen too. Everything the flash tool draws on top of it (the operating line, its intersection, the lever rule) is GRAPHICAL construction on this one table, which is why turning a knob there redraws at once instead of re-solving.

So this case publishes exactly one thing: the curve. It does NOT flash anything. The flash the student watches is the intersection, performed on these rows – and the corpus already carries the ENGINE's own answer for the same pair (tutorials/steady/flash/flash01_benzene_toluene, V/F = 0.30398 at 370 K), which is the number to check the construction against.

51 points is not a rounding of taste: the construction interpolates linearly between rows (no spline – an overshoot near an azeotrope would invent equilibrium the engine never computed), so the sampling IS the resolution of every answer read off it.

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`txy`	`n_emptyCurves`	`0`
`diag`	`txy`	`n_failures`	`0`
`diag`	`txy`	`n_points`	`51`

mixrules01_natural_gas

tutorials/props/molecular/mixrules01_natural_gas

mix_30bar mixingRules | **mix_1bar** mixingRules

What it does: The van der Waals one-fluid mixing rule, decomposed: SRK a_mix/b_mix of a 90/5/5 CH4/N2/CO2 gas at 250 K, pair by pair – the one curated kij (N2-CH4, DECHEMA) beside the pairs that run the kij = 0 default, and the same decomposition at 1 bar showing a_mix is a (T, y) fact while only A, B, Z carry the pressure

The story: THE MIXING RULE, DECOMPOSED – the multicomponent lesson. A cubic prices a MIXTURE through two numbers, and the one-fluid rule says how n pure components and n(n-1)/2 binary corrections build them:

a_mix(T) = sum_i sum_j y_i y_j (1 - kij) sqrt(a_i a_j) b_mix = sum_i y_i b_i

Every term is on the table: the six pair terms of this ternary re-add to the a_mix the flash would run (the op REFUSES if they do not), the curated N2-CH4 kij sits beside the two CO2 pairs running the announced kij = 0 default, and the pair table is where a student sees that a ternary needs THREE binary decisions – there is no ternary parameter.

Two states on purpose, same composition and temperature: a_mix and b_mix are IDENTICAL at 30 bar and 1 bar (the rule sees only T and y), while A, B and Z carry the pressure – at 1 bar the same gas is nearly ideal (Z ~ 0.997) and at 30 bar it is not (Z ~ 0.93), priced by the SAME two numbers through the cubic.

Nothing here is validated against measurement: every number is the engine's own arithmetic, decomposed. The one cited constant is the N2-CH4 kij record (Knapp et al., DECHEMA VI, 1982).

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	mix_30bar	A	0.140523
diag	mix_30bar	B	0.0428434
diag	mix_30bar	P_Pa	3e+06
diag	mix_30bar	T_K	250
diag	mix_30bar	Z_vapour	0.901046
diag	mix_30bar	a_CH4	0.200506
diag	mix_30bar	a_CO2	0.429536
diag	mix_30bar	a_N2	0.0837883
diag	mix_30bar	a_mix	0.202383
diag	mix_30bar	b_CH4	2.98484e-05
diag	mix_30bar	b_CO2	2.96742e-05
diag	mix_30bar	b_N2	2.6754e-05
diag	mix_30bar	b_mix	2.9685e-05
diag	mix_30bar	dadT_mix	-0.000515994

. . . and 46 more reference values in the case's `expected`.

`nrtl01_ethanol_water_package`

`tutorials/props/molecular/nrtl01_ethanol_water_package`

g_nrtl
activityCoefficients

What it does: ethanol-water NRTL via the SAME property-package builder (molecular U4)

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`g_nrtl`	`gamma_ethanol`	`1.26245`
`diag`	`g_nrtl`	`gamma_water`	`1.4349`

pcsaft01_pure_nhexane

tutorials/props/molecular/pcsaft01_pure_nhexane

liquid__298K__1bar
propertyPoint

What it does: PC-SAFT pure n-hexane: Z + liquid molar volume at 298 K, 1 bar

The story: the NON-ASSOCIATING PC-SAFT core, validated on a pure n-alkane. n-hexane's segment parameters (m, sigma, epsilon/k) are the Gross & Sadowski 2001 Table 1 fit; the case evaluates the compressibility factor Z and the liquid molar volume at 298.15 K / 1 bar and locks them as a golden. The engine PROVES the model self-consistent on every run (PCSAFT::verify: a_res → 0 in the ideal limit, and the residual chemical potentials reproduce the bulk residual Gibbs energy to ~1e-9). Physical cross-check (see README): the PC-SAFT liquid density here is ~650 kg/m3, within ~0.8 % of the ~655 kg/m3 experimental value.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	liquid_298K_1bar	Cp_ig	1.43106e+02
diag	liquid_298K_1bar	H_R	-3.91460e+02
diag	liquid_298K_1bar	H_ig	-1.67200e+05
diag	liquid_298K_1bar	H_real	-1.67591e+05
diag	liquid_298K_1bar	P	1.00000e+05
diag	liquid_298K_1bar	S_R	-3.54309e-01
diag	liquid_298K_1bar	S_ig	3.88510e+02
diag	liquid_298K_1bar	S_real	3.88156e+02
diag	liquid_298K_1bar	T	2.98150e+02
diag	liquid_298K_1bar	Z	9.42530e-01
diag	liquid_298K_1bar	Z_liquid	5.35107e-03
diag	liquid_298K_1bar	gamma	1.06168e+00
diag	liquid_298K_1bar	v_molar	2.33649e-02
diag	liquid_298K_1bar	v_molar_liq	1.32651e-04

. . . and 1 more reference values in the case's `expected`.

pcsaft02_states_crosscheck

tutorials/props/molecular/pcsaft02_states_crosscheck

CH4__vapour__pcsaft propertyPoint	**CH4__vapour__srk** propertyPoint	**propane__liquid** propertyPoint
nButane__liquid propertyPoint		

What it does: PC-SAFT across states: CH4 Z vs SRK cross-check + propane/n-butane liquid density vs handbook

The story: PC-SAFT validated ACROSS STATES and AGAINST AN INDEPENDENT MODEL, before the model is trusted in a flowsheet.

Three probes, all goldened: (1) CH4 vapour Z at 300 K / 50 bar – PC-SAFT vs SRK at the SAME state (the per-op 'thermo {}' override): two independently-validated models must agree closely for a simple near-spherical vapour, and the run prints both so the student SEES the agreement (not told to assume it). (2) propane compressed liquid at 298.15 K / 15 bar – rho_liq vs the saturated-liquid handbook value (0.493 g/mL at 25 degC). (3) n-butane compressed liquid at 298.15 K / 5 bar – rho_liq vs 0.573 g/mL at 25 degC. Deviations sit at the 1-2 % liquid-density AAD Gross & Sadowski 2001 (Table 2) report for the n-alkane series – these parameters PREDICT the density; it was never refitted here.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	CH4_vapour_pcsaft	Cp_ig	3.56607e+01
diag	CH4_vapour_pcsaft	H_R	-8.28601e+02
diag	CH4_vapour_pcsaft	H_ig	-7.44541e+04
diag	CH4_vapour_pcsaft	H_real	-7.52827e+04
diag	CH4_vapour_pcsaft	P	5.00000e+06
diag	CH4_vapour_pcsaft	S_R	-1.24811e+00
diag	CH4_vapour_pcsaft	S_ig	1.53964e+02
diag	CH4_vapour_pcsaft	S_real	1.52716e+02
diag	CH4_vapour_pcsaft	T	3.00000e+02
diag	CH4_vapour_pcsaft	Z	9.12141e-01
diag	CH4_vapour_pcsaft	gamma	1.30404e+00
diag	CH4_vapour_pcsaft	v_molar	4.55038e-04
diag	CH4_vapour_srk	Cp_ig	3.56607e+01
diag	CH4_vapour_srk	H_R	-8.31158e+02

... and 42 more reference values in the case's `expected`.

pcsaft03_association_pure

tutorials/props/molecular/pcsaft03_association_pure

water_liquid_298K propertyPoint | **ethanol_liquid_298K** propertyPoint

What it does: PC-SAFT association (Wertheim TPT1): 2B water + 2B ethanol, pure liquids

The story: PC-SAFT WITH ASSOCIATION (Wertheim TPT1; Gross & Sadowski, Ind. Eng. Chem. Res. 41 (2002) 5510) – the hydrogen-bonded liquids the non-associating core cannot represent, as PURE liquids at 298.15 K, 1 bar.

The association parameters live on the component records (2B water, 2B ethanol – both G&S 2002 Table 1, fitted WITH the association term; the paper lists water with TWO sites, and the scheme is PART of the fit: this same set mis-curated as 4C passed the pure-density check by coincidence and collapsed the ethanol/water mixture flash); the case only declares 'model PCSAFT;', and the engine announces the roster it found on the records:

[pcsaft] associating: water (2B), ethanol (2B); cross: Wolbach-Sandler

Liquid densities PREDICTED here (goldened), vs the CRC Handbook 25 degC values – each deviation stated against what the SET is fitted for:

water v_molar_liq → rho_liq = 921.9 kg/m3 vs 997.05 (-7.5 % – the PUBLISHED trade-off of the 2-site water fit, which is Psat-accurate instead: -0.2 % at 358 K, +4 % at 298 K) ethanol v_molar_liq → rho_liq = 779.8 kg/m3 vs 789.3 (-1.2 %)

What the student sees at verbosity 3 (the widened validation, ratified 2026-08-02): the construction-time verify() now ALSO echoes, per associating component as a pure fluid, the converged site fractions X^D / X^A, the a_assoc share, and the CLOSED-FORM-vs-ITERATIVE oracle deviation (for nD = nA the mass-action closure collapses to a quadratic; the damped fixed point must land on its root – observed machine zero). X^A(water) ~ 0.039: 96 % of water's sites are hydrogen-bonded at ambient conditions – the physics the term adds.

The two ops publish the standard vapour-root family (Z, v_molar) plus the labelled liquid root (v_molar_liq); at 1 bar both fluids carry a metastable vapour root beside the stable liquid.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	water_liquid_298K	Cp_ig	3.36596e+01
diag	water_liquid_298K	H_R	-2.18780e+03
diag	water_liquid_298K	H_ig	-2.41826e+05
diag	water_liquid_298K	H_real	-2.44014e+05
diag	water_liquid_298K	P	1.00000e+05
diag	water_liquid_298K	S_R	-5.61640e+00
diag	water_liquid_298K	S_ig	1.88834e+02
diag	water_liquid_298K	S_real	1.83218e+02
diag	water_liquid_298K	T	2.98150e+02
diag	water_liquid_298K	Z	9.00945e-01
diag	water_liquid_298K	gamma	1.32805e+00
diag	water_liquid_298K	v_molar	2.23340e-02
diag	ethanol_liquid_298K	Cp_ig	6.54261e+01
diag	ethanol_liquid_298K	H_R	-4.01997e+03
...and 16 more reference values in the case's `expected`.			

solubility01_hildebrand_ladder

tutorials/props/molecular/solubility01_hildebrand_ladder

What it does: Hildebrand: one number for like-dissolves-like, and where it stops

Seven liquids on a ladder of cohesive energy density, from hexane to water.

δ is derived, never stored: sqrt((ΔHvap(T) - RT)/V_m), with the latent heat from the Watson correlation the enthalpy legs already use. Every input is already in the component record, so a stored δ would be a second home for a fact the record fixes — free to drift from it.

*The published column is an *anchor*, not an input. It is what ChemSep computed independently, and its only job is to give the derivation something to be wrong against. Worst deviation here is 0.85 %.*

That matters because the three ways this arithmetic fails are all silent:

∗ the RT term dropped — δ a few per cent high, and a solubility parameter has no intuitive magnitude to make that look wrong; ∗ V_m in the wrong unit; ∗ Pa^0.5 reported where the literature uses MPa^0.5 — a factor of 31.6, and the number still reads as a solubility parameter.

Same device as the Zc column in a property data bank: a redundant determination turns a plausible wrong number into a visible disagreement.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	`solubilityParameter_0`	`delta_acetone`	`19673.2`
diag	`solubilityParameter_0`	`delta_benzene`	`18648.2`
diag	`solubilityParameter_0`	`delta_cyclohexane`	`16733.6`
diag	`solubilityParameter_0`	`delta_ethanol`	`26301.9`
diag	`solubilityParameter_0`	`delta_nHexane`	`14859`
diag	`solubilityParameter_0`	`delta_toluene`	`18183.8`
diag	`solubilityParameter_0`	`delta_water`	`48266.4`
diag	`solubilityParameter_0`	`dev_acetone`	`0.00287938`
diag	`solubilityParameter_0`	`dev_benzene`	`0.00277184`
diag	`solubilityParameter_0`	`dev_cyclohexane`	`0.000380181`
diag	`solubilityParameter_0`	`dev_ethanol`	`0.00619352`
diag	`solubilityParameter_0`	`dev_nHexane`	`0.000736807`
diag	`solubilityParameter_0`	`dev_toluene`	`0.003629`
diag	`solubilityParameter_0`	`dev_water`	`0.00849075`
. . . and 6 more reference values in the case's `expected`.			

reactor

The theory you need. Three idealised steady reactors. Conversion reactor: stoichiometry only, $n_{\mathrm{out}} = n_{\mathrm{in}} + \nu\,\xi$, duty from the ONE enthalpy base $\Delta H_{\mathrm{rxn}}(T) = \sum_i \nu_i h_i(T)$ (formation datum – no separate "heat of reaction" input, and any dict override is cross-checked aloud). CSTR: outlet at reactor conditions with rate $r(c_{\mathrm{out}}, T)$, so $F\,(c_{\mathrm{in}} - c_{\mathrm{out}}) = r\,V$. PFR: the same rate integrated along the volume, $\mathrm{d}F_i/\mathrm{d}V = \nu_i\,r$. Arrhenius kinetics $k = A\,T^b e^{-E_a/RT}$, orders from the dict, reverse rates from the equilibrium constant (thermo-consistent by construction). Expect: CSTR conversion below PFR at equal volume (back-mixing), exponential T sensitivity, and equilibrium limits the kinetics can approach but never cross. *Full derivation: Theory Guide, the reaction-engineering chapter.*

thiele01_sphere_first_order

tutorials/props/reactor/thiele01_sphere_first_order

thiele_CO_alumina
thielePellet

What it does: Single catalyst pellet: intraparticle concentration field, effectiveness factor, and the eta(phi) family, each verified against the first-order isothermal closed form

The story: One thielePellet operation.

THE RATE CONSTANT IS DECLARED HERE, and not on the catalyst record, because kinetics are equipment- and reaction-specific: the asset carries the SOLID, never the chemistry run on it. Two reactors loaded with the same catalyst would otherwise carry two copies of one rate constant and drift apart.

k = 40 1/s is a TEACHING VALUE chosen so the generalised modulus lands near 1 − the transition region, where the field has a visible gradient and eta is neither ~1 nor ~1/phi. It is not measured and is not claimed to be.

Solves the intraparticle boundary-value problem for one porous catalyst pellet, publishes the concentration field, the effectiveness factor and the Thiele modulus, and checks every one of them against the closed form.

With xi the normalised coordinate from the pellet centre (0) to its external surface (1) and u = c/c_s:

Both moduli are published because both conventions are in circulation and they differ by a factor of three for a sphere. The engine's own announcement in src/unitOperations/reactor/CatalystPellet.H uses the generalised one.

The equation is discretised with second-order central differences and handed to the existing solver::newtonND (finite-difference Jacobian, LU with partial pivoting) — no new dependency, no new solver, every step readable in src/unitOperations/reactor/pellet/PelletDiffusion.cpp.

xi is one column, not two aliases: it is r/R for a sphere or a cylinder and x/L for a slab.

First-order isothermal is the rare case that has a closed form in all three geometries, so the numerical answer is not merely produced — it is checked:

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	thiele_CO_alumina	phi	1.0161e+00
diag	thiele_CO_alumina	phi_char	3.0483e+00
diag	thiele_CO_alumina	Lambda	5.0000e-04
diag	thiele_CO_alumina	dimension	1.5000e-03
diag	thiele_CO_alumina	epsilon_p	4.5000e-01
diag	thiele_CO_alumina	tau	3.0000e+00
diag	thiele_CO_alumina	rateConstant	4.0000e+01
diag	thiele_CO_alumina	T	5.7315e+02
diag	thiele_CO_alumina	P	1.01325e+05
diag	thiele_CO_alumina	D_molecular	6.45709e-05
diag	thiele_CO_alumina	D_eff	9.68564e-06
diag	thiele_CO_alumina	eta	6.65742e-01
diag	thiele_CO_alumina	eta_closedForm	6.65742e-01
diag	thiele_CO_alumina	eta_flux	6.65731e-01

. . . and 17 more reference values in the case's `expected`*.*

thiele02_prater_temperature

tutorials/props/reactor/thiele02_prater_temperature

thiele__CO__alumina__prater
thielePellet

What it does: Single catalyst pellet: intraparticle concentration field, effectiveness factor, and the eta(phi) family, each verified against the first-order isothermal closed form

The story: One thielePellet operation.

THE RATE CONSTANT IS DECLARED HERE, and not on the catalyst record, because kinetics are equipment- and reaction-specific: the asset carries the SOLID, never the chemistry run on it. Two reactors loaded with the same catalyst would otherwise carry two copies of one rate constant and drift apart.

k = 40 1/s is a TEACHING VALUE chosen so the generalised modulus lands near 1 – the transition region, where the field has a visible gradient and eta is neither ~1 nor ~1/phi. It is not measured and is not claimed to be.

The SAME pellet as thiele01__sphere__first__order, with one block added, so the difference between the two runs is the temperature.

Everything else — catalyst, species, T, P, rate constant, diffusion, grid, verification tolerances — is thiele01 unchanged. Run both and diff the output: nothing else moves.

The reaction is exothermic, so the pellet is hottest where the reactant is most depleted, which is the centre:

The field CSV gains two columns, T__over__Ts and T__K, one value per node — so the temperature is a FIELD, not a headline number, and the EduTool paints the same discs by it.

The species and energy balances inside a pellet carry the same reaction term. Eliminate it between them and, for constant D__eff and k__eff and any kinetics at all, what is left is a first integral — Prater's relation:

So the temperature field costs no second boundary-value problem. It is algebra over the concentration field the operation already solved.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	thiele_CO_alumina_prater	D_eff	9.68564e-06
diag	thiele_CO_alumina_prater	D_molecular	6.45709e-05
diag	thiele_CO_alumina_prater	Lambda	0.0005
diag	thiele_CO_alumina_prater	P	101325
diag	thiele_CO_alumina_prater	T	573.15
diag	thiele_CO_alumina_prater	T_centre_K	588.722
diag	thiele_CO_alumina_prater	c_centre	0.289874
diag	thiele_CO_alumina_prater	c_centre_closedForm	0.289873
diag	thiele_CO_alumina_prater	deltaT_centre_K	15.5718
diag	thiele_CO_alumina_prater	dimension	0.0015
diag	thiele_CO_alumina_prater	epsilon_p	0.45
diag	thiele_CO_alumina_prater	eta	0.665742
diag	thiele_CO_alumina_prater	eta_closedForm	0.665742
diag	thiele_CO_alumina_prater	eta_flux	0.665731

. . . and 23 more reference values in the case's `expected`*.*

scan

The theory you need. Property scans sweep a model across T, P or composition and plot what the thermodynamics DOES: $P^{\mathrm{sat}}(T)$ curves, $\gamma_{\pm}(m)$, VLE envelopes, ternary diagrams, phase boundaries. Nothing is fitted – these ops answer "how does the selected model behave?", the SEE step of the props loop (CREATE → SEE → TEST → FIT → PROMOTE). Expect model disagreements to be VISIBLE (overlay two methods and look), and every curve exportable as CSV next to the case. *Full derivation: Theory Guide, the Props Guide.*

binary01_lle_water_nbutanol

tutorials/props/scan/binary01_lle_water_nbutanol

lle
propertyScanBinary

What it does: Binary LLE g_mix + common-tangent (water/1-butanol) at 298.15 K via propertyScanBinary, cited Winkelman 2009 UNIQUAC

The story: Binary liquid-liquid teaching map – water / 1-butanol at 25 C.

The molar Gibbs energy of mixing g_mix(x1) swept across composition PLUS the two coexisting liquid compositions from ONE liquid-liquid IsothermalFlash (PhaseSet::LL). The two binodal points lie on the g_mix curve and share its common tangent – the student reads the split straight off the diagram.

gamma comes from the CITED Winkelman et al. (2009) UNIQUAC LLE set (Table 2; structural r,q from Table 1), so the g_mix curve goes NON-CONVEX and the flash emits the real water/1-butanol binodal: ~1.9 mol% butanol in the water-rich phase and ~49 mol% butanol in the butanol-rich phase (matching their Figs 1-2 at 298 K). See README.md.

The flash feed is set to z(water) = 0.80 (20 mol% butanol), INSIDE the asymmetric gap; an equimolar feed would lie outside it (both binodal branches sit below 50 mol% butanol). Run-only smoke (the CSV is a teaching curve, not pinned KPIs).

*Water + 1-butanol (n-butanol) is *the* classic partially-miscible binary. At 25 °C it splits into a water-rich liquid (~2 mol % butanol) and a butanol-rich liquid (~50 mol % butanol). A g_mix(x_water) scan should therefore show a clearly non-convex region: two binodal points sharing a common tangent, which a student reads the split straight off.*

and binaryLle.csv is a g_mix(x1) curve that goes non-convex, plus the two binodal points the liquid-liquid flash locates on it:

The agreement is essentially exact: the cited UNIQUAC fit reproduces the published mutual solubilities.

*- J.G.M. Winkelman, G.N. Kraai, H.J. Heeres, *"Binary, ternary and quaternary liquid-liquid equilibria in 1-butanol, oleic acid, water and n-heptane mixtures"*, Fluid Phase Equilibria 284 (2009) 71–79, doi:10.1016/j.fluid.2009.06.013.*

- Interaction parameters — their Table 2 (p.73), binary 1-butanol(1)/water(3), valid 273–363 K, with the quadratic temperature correlation A__ij(T) = a + b·T + c·T^2 (Eq. 10), τ__ij = exp(-A__ij/T) (Eq. 7): - A__1,3 (butanol→water): a = 155.31, b = 1.0822, 10?·c = -43.711 - A__3,1 (water→butanol): a = -579.36, b = 2.7517, 10?·c = -6.7700 - Structural r, q — their Table 1 (p.72): 1-butanol r = 3.9243, q = 3.668; water r = 0.9200, q = 1.400.

The numbers are inline here, fully provenanced above; the record is STAGED CASE-LOCALLY, not curated: it lives at tutorials/steady/flash/marcilla02_lls__ternary__invariant/constant/parameters/UNIQUAC/nButanol-water.dat with reviewStatus interim, and promotion into data/standards/ is the maintainer's batch review. This README used to claim the numbers already lived under data/standards/ – asserting a curation status that had been deliberately withheld, which is a worse error than saying nothing.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	lle	T	298.15
diag	lle	split	1

hxy01_ethanol_water_1atm

`tutorials/props/scan/hxy01_ethanol_water_1atm`

hxy
enthalpyConcentration

What it does: Enthalpy-concentration locus (h-x-y) of ethanol/water at 1 atm on the elements datum – the data a Ponchon-Savarit construction is drawn on

The story: The enthalpy-concentration locus of ethanol/water at 1 atm.

McCabe-Thiele carries equilibrium and material balance under CONSTANT MOLAR OVERFLOW – one mole condensing releases exactly what one mole vaporising absorbs, whatever the composition. This case publishes the quantity that assumption is about: the saturated-liquid enthalpy h(x) on its bubble line, the saturated-vapour enthalpy H(y) on its dew line, and the equilibrium tie between them, all on Choupo's ONE enthalpy datum (the elements at 298.15 K).

lambda(x) = H(y*) - h(x) is the molar heat of vaporisation ALONG the locus. CMO says it is a constant. Read the two lambda diagnostics and decide for yourself.

Run-only + KPI golden; see README.md for the provenance of every number and for what is NOT validated.

The saturated-liquid enthalpy on its bubble line, the saturated-vapour enthalpy on its dew line, and the equilibrium tie between them — the data an enthalpy-concentration (Ponchon–Savarit) construction is drawn on, and the first thing the engine publishes that carries enthalpy against composition.

enthalpyConcentration.csv — a regular table, one header, homogeneous rows:

Every row is one equilibrium. It carries ∗both∗ ends of a tie line — the liquid (x1, h_liquid) and the vapour (y1, H_vapour) that are in equilibrium with each other — at the one temperature T_K they share. So nothing downstream pairs rows by index and nothing interpolates one end of a tie. role says only which composition was the uniform grid node:

The saturated-liquid curve is (x1, h_liquid) over all rows; the saturated-vapour curve is (y1, H_vapour) over all rows. status is ok or names a refusal; a refused node keeps its grid coordinate and leaves the solved fields empty, so a gap is visible rather than silently bridged.

enthalpyConcentration.meta — a key,value sidecar, written on every run. It states the two components, the pressure, which enthalpy datum the two columns are on, which kernel computed each, the tie contract, whether the heat of mixing is included, and the reason for every refused node. The construction's lever arms are enthalpy ∗differences∗, so a constant offset cancels in the geometry — which is exactly why the file has to say what its axis means instead of letting the next reader assume.

Both enthalpy columns are absolute on Choupo's one enthalpy base, the elements at 298.15 K, from the same two kernels every energy balance and duty in the engine is priced on:

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	hxy	H_vapour_pure1_J_per_mol	-231233.2
diag	hxy	H_vapour_pure2_J_per_mol	-239304.0
diag	hxy	P_Pa	101325.0
diag	hxy	T_bubble_pure1_K	351.5
diag	hxy	T_bubble_pure2_K	372.5
diag	hxy	h_liquid_pure1_J_per_mol	-271912.8
diag	hxy	h_liquid_pure2_J_per_mol	-280791.7
diag	hxy	lambda_max_J_per_mol	45052.6
diag	hxy	lambda_min_J_per_mol	40679.6
diag	hxy	n_nodes	82.0
diag	hxy	n_refused	0.0

phase01_water_full

`tutorials/props/scan/phase01_water_full`

pd
purePhaseDiagram

What it does: Complete P-T phase diagram of water (purePhaseDiagram, solid+liquid+vapour)

The story: Complete pure-compound P-T phase diagram of water: liquid-vapour saturation (to the critical point) + solid-vapour sublimation + solid-liquid fusion (the negative-slope ice line) + triple and critical points. Clapeyron physics in the purePhaseDiagram engine op; cited solid data in the solid{} block (IAPWS/CRC, see solidPhaseData.ts). Run-only smoke.

What to expect (golden-locked):

kind	unit/stream	key	expected
`diag`	`pd`	`Pc_bar`	`220.64`
`diag`	`pd`	`Tc`	`647.14`
`diag`	`pd`	`has_solid`	`1`

scan2d01_co2_compressibility

tutorials/props/scan/scan2d01_co2_compressibility

Z__co2__TP
propertyScan2D

What it does: Z(T,P) for CO_2 on Peng-Robinson over a 15 x 20 grid that crosses the critical point: the compressibility surface, one row per node

The story: THE COMPRESSIBILITY SURFACE, NOT A CURVE.

‘propertyScan2D‘ sweeps TWO state variables over a rectangle and reports the property at every node – long-form CSV, one row per (T, P), ready to pivot into a heat map. Here it is Z(T, P) for CO_2 on Peng-Robinson, 280-420 K by 1-200 bar: a 15 x 20 grid that walks straight over the critical point (304.13 K, 73.77 bar).

What to read: along an isotherm below Tc the surface DROPS steeply and then turns up – the liquid-like branch; above Tc the same isotherm is smooth: the discontinuity is a coexistence line, and the grid crosses where it ends; Z $\rightarrow$ 1 in the whole low-P edge, which is the ideal-gas limit falling out of a cubic rather than being assumed.

The op reports ‘failures‘ – nodes where the property could not be evaluated – as a diagnostic, so a grid that quietly returned fewer points than it promised is visible in the payload rather than only in the CSV’s line count. The expected file pins it at zero: on this rectangle every node must solve.

Why the case exists: ‘propertyScan2D‘ had NO case and no gate, though ‘propertyScan1D‘, ‘propertyScanBinary‘ and ‘propertyScanTernary‘ all have one. Its sibling vaporPressureFit was in the same position, and writing the first case for THAT one segfaulted choupoProps twice.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	Z_co2_TP	failures	0
diag	Z_co2_TP	n_points	300
diag	Z_co2_TP	n_x	15
diag	Z_co2_TP	n_y	20

ternary01_audit_vlle

tutorials/props/scan/ternary01_audit_vlle

lle
propertyScanTernary

What it does: Ternary VLLE phase map (compA/compB/compC) at 350 K, 1 bar via propertyScanTernary

The story: ILLUSTRATIVE / SYNTHETIC DATA. compA / compB / compC are NOT real species: they are artificial components (case-local constant/components/ + inline NRTL pairs in thermoPhysPropDict) deliberately tuned to guarantee a ternary VLLE region, so this case exercises the propertyScanTernary classifier on a known-3-phase map. Architecture (see thermoPhysPropDict): A and B mutually immiscible (LL gap on the A-B axis), C a volatile bridge soluble in both liquids. At 350 K, 1 bar the equimolar feed gives vapour rich in C, L_alpha rich in A, L_beta rich in B.

Expected map (14 intervals/edge, 78 nodes): VLLE 36, LL 29, VL 0, 1-phase 13, 22 tie-lines, 0 failures. The point to SEE: a healthy spread of 3-phase (VLLE) and LL nodes with zero solver failures — the classifier resolves the engineered gap node-by-node.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	lle	failures	0
diag	lle	n_ll	29
diag	lle	n_nodes	78
diag	lle	n_onephase	13
diag	lle	n_tielines	22
diag	lle	n_vl	0
diag	lle	n_vlle	36

ternary02_bubbleT_benzene_toluene_water

tutorials/props/scan/ternary02_bubbleT_benzene_toluene_water

bt
propertyScanTernary

What it does: Ternary T__bubble surface (benzene/toluene/water) at 1 bar via propertyScanTernary mode bubbleT

The story: Ternary boiling-temperature surface (T__bubble over the composition triangle) via propertyScanTernary mode bubbleT — reuses BubblePoint per simplex node. Works with ideal binaries (only the 3 Psat are needed). Run-only smoke.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	bt	failures	0
diag	bt	n_nodes	150

ternary03_lle_water_ethanol_benzene

`tutorials/props/scan/ternary03_lle_water_ethanol_benzene`

lle
propertyScanTernary

What it does: Ternary LLE solubility (water/ethanol/benzene) at 298 K via UNIFAC + propertyScanTernary mode lle

The story: Ternary LIQUID-LIQUID solubility map (water / ethanol / benzene) — the classic extraction triangle. Activity from UNIFAC (group contribution): NO fitted binary pairs needed — gamma is predicted from molecular groups, so a real system shows a real miscibility gap with no curated pair data. mode lle uses the liquid-liquid flash (PhaseSet::LL): clean binodal + tie-lines, no spurious vapour. Run-only smoke (a phase map, not KPIs).

What to expect (golden-locked):

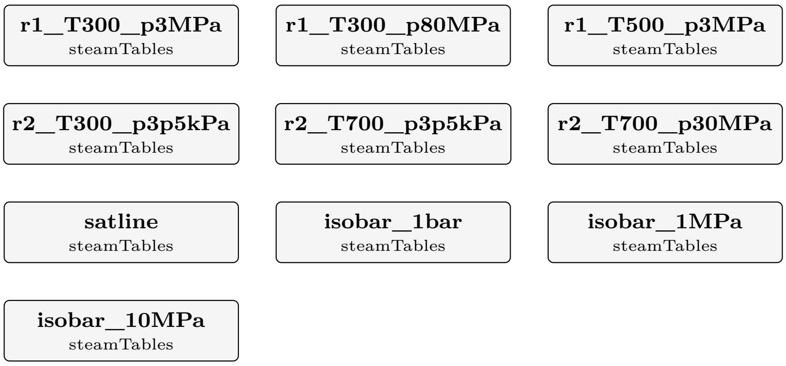
kind	unit/stream	key	expected
`diag`	`lle`	`failures`	`0`
`diag`	`lle`	`n_ll`	`69`
`diag`	`lle`	`n_nodes`	`105`
`diag`	`lle`	`n_onephase`	`36`
`diag`	`lle`	`n_tielines`	`18`
`diag`	`lle`	`n_vl`	`0`
`diag`	`lle`	`n_vlle`	`0`

steam

The theory you need. Water/steam properties from IAPWS-IF97: regions (liquid, vapour, supercritical), $T_{\mathrm{sat}}(P)$, and $h(T,P)/s(T,P)$ from the dimensionless Gibbs functions. Power cycles are enthalpy bookkeeping around them: pump work $\approx v\,\Delta P$, turbine work from isentropic expansion $\times$ efficiency, cycle efficiency $\eta = W_{\mathrm{net}}/Q_{\mathrm{in}}$ bounded by Carnot. *Full derivation: Theory Guide, the steam/power chapter.*

steam01_if97_verification

`tutorials/props/steam/steam01_if97_verification`



What it does: IAPWS-IF97 steam tables (R7-97(2012)) – the verification case. The six computer-program verification states of the release (Table 5 region 1, Table 15 region 2) evaluated as points, the saturation line 0.01-350 degC, and three isobars whose Tsat crossings pin the release's Table 36 backward values. The published numbers ARE the golden master (reltol 1e-8).

The story: the IAPWS-IF97 weapon case.

The release (IAPWS R7-97(2012)) publishes computer-program verification tables; this case evaluates EVERY one of them this slice covers and locks the published numbers as golden KPIs at reltol 1e-8 – a single wrong digit in a transcribed coefficient fails the regression:

Table 5 – region 1 at (300 K, 3 MPa), (300 K, 80 MPa), (500 K, 3 MPa) Table 15 – region 2 at (300 K, 3.5 kPa), (700 K, 3.5 kPa), (700 K, 30 MPa) Table 35 – psat at 300 / 500 K (the psat_MPa diag of the point ops) Table 36 – Tsat at 0.1 / 1 / 10 MPa (the Tsat_K diag of the isobars)

Plus the working artefacts: the saturated-steam table 0.01-350 degC (saturation.csv) and three isobars (h, s, v, cp vs T) crossing the saturation line – the crossing is ANNOUNCED in the log.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	r1_T300_p3MPa	region	1
diag	r1_T300_p3MPa	v	0.100215168e-2
diag	r1_T300_p3MPa	h	0.115331273e3
diag	r1_T300_p3MPa	u	0.112324818e3
diag	r1_T300_p3MPa	s	0.392294792
diag	r1_T300_p3MPa	cp	0.417301218e1
diag	r1_T300_p3MPa	w	0.150773921e4
diag	r1_T300_p3MPa	psat_MPa	0.353658941e-2
diag	r1_T300_p80MPa	region	1
diag	r1_T300_p80MPa	v	0.971180894e-3
diag	r1_T300_p80MPa	h	0.184142828e3
diag	r1_T300_p80MPa	u	0.106448356e3
diag	r1_T300_p80MPa	s	0.368563852
diag	r1_T300_p80MPa	cp	0.401008987e1

. . . and 47 more reference values in the case's `expected`.

steam02_water_transport

`tutorials/props/steam/steam02_water_transport`

pin__25C propertyPoint

pin__50C propertyPoint

pin__75C propertyPoint

pin__100C propertyPoint

transport__scan propertyScan1D

What it does: IAPWS water transport properties – viscosity (R12-08), thermal conductivity (R15-11) and surface tension (R1-76). Water flagged as a pure fluid (method IF97) routes the transport accessors to the matching IAPWS releases: the density comes from the IF97 thermodynamic kernel at the saturation state, fed to the (rho,T) transport kernels; surface tension is the R1-76 saturation-line form. A coarse 4-point scan (25/50/75/100 degC) pinned against the published IAPWS values, plus a fine sweep for the plot.

The story: the IAPWS water transport route.

Water flagged ‘pureFluids { water { method IF97; } }‘ routes the transport accessors to the matching IAPWS releases:

viscosity → R12-08 (industrial, critical enhancement off) thermal__conductivity → R15-11 (background, critical enhancement off) surface__tension → R1-76 (saturation line)

The (rho,T) viscosity / conductivity kernels get their density from the IF97 thermodynamic kernel at the SATURATION state (the legacy transport accessors carry no pressure, so p = p__sat(T) is the physical convention – saturated liquid / saturated vapour). Surface tension is a saturation-line property and takes only T.

‘pins‘ evaluates the three properties at 25/50/75/100 degC; the golden master locks them at reltol 1e-5 (the formulation is verified to 1e-8 by IAPWSTransport::verify(), which fires on every run). ‘scan‘ writes a fine sweep for the plot.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	pin_25C	surface_tension	0.0720
diag	pin_25C	thermal_conductivity	0.6065
diag	pin_25C	viscosity	0.0009
diag	pin_50C	surface_tension	0.0679
diag	pin_50C	thermal_conductivity	0.6406
diag	pin_75C	surface_tension	0.0636
diag	pin_75C	thermal_conductivity	0.6635
diag	pin_100C	surface_tension	0.0589
diag	pin_100C	thermal_conductivity	0.6772
diag	transport_scan	n_points	46
diag	transport_scan	n_failures	0
diag	pin_25C	Cp_ig	33.6596
diag	pin_25C	H_R	0.0000
diag	pin_25C	H_ig	-241826.0000

. . . and 49 more reference values in the case’s `expected`*.*

thermo

The theory you need. Consistency tests interrogate DATA, not models: the van Ness point test checks measured (x, y, T, P) sets against the Gibbs–Duhem constraint (residuals in $\ln(\gamma_1/\gamma_2)$); Herington’s area test is its integral cousin. A dataset that fails is not “bad” – the test says WHERE and HOW MUCH, and the fit that follows should weight accordingly. Expect plain-language verdicts with severity, not a pass/fail stamp. *Full derivation: Theory Guide, the consistency chapter of the Props Guide.*

temperature01_gas_thermometer

tutorials/props/thermo/temperature01_gas_thermometer

Z__of__P propertyScan1D | **nearly__ideal** propertyPoint | **laboratory__pressure** propertyPoint

What it does: The gas thermometer: why the ideal gas is a limit, not a substance

The story: A constant-volume gas thermometer reads a temperature off PV = nRT. A real gas does not obey that, so the reading depends on the PRESSURE it was taken at – and only in the limit P → 0 does it become the thermodynamic temperature. This case measures that departure, at the temperature the accompanying notes are about.

500.012 K is not a special temperature. It is the number the notes use to ask what the last digits of a temperature actually assert, and the scan is run there so the two are talking about the same state.

> ∗"Thermodynamics is a funny subject. The first time you go through it, you > don't understand it at all. The second time you go through it, you think you > understand it, except for one or two small points. The third time you go > through it, you know you don't understand it, but by that time you are so > used to it, it doesn't bother you any more."∗ > > — attributed to Arnold Sommerfeld. PROVENANCE UNVERIFIED: Sommerfeld > published no such sentence; it reaches us by report, and this repository does > not quote a source it has not read. It is here as an epigraph, marked, and > must be either sourced or dropped before it appears in any published guide.

What does the 012 assert? Not "roughly five hundred kelvin" — three digits past the decimal point is a specific claim about the world. Whose claim, and resting on what?

The kelvin is no longer defined by a substance. It is defined by fixing the Boltzmann constant:

Not measured. Fixed. So a temperature is a conversion factor between energy and degrees, and nothing else. The triple point of water — which ∗defined∗ the kelvin for sixty-five years — became, overnight, a quantity that is measured, with an uncertainty.

That is the first dogma, and it is worth sitting with: the definition tells you what a kelvin ∗means∗ and gives you no way whatsoever to ∗measure∗ one.

Nobody puts the Boltzmann constant inside a thermometer. To read a temperature you need a ∗realisation∗: a chain of reproducible fixed points and an instrument that interpolates between them. In practice that is ITS-90, with a platinum resistance thermometer doing the interpolating. At 500 K you are between the triple point of water and the freezing point of zinc.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	Z_of_P	n_emptyCurves	0.0000
diag	Z_of_P	n_failures	0.0000
diag	Z_of_P	n_points	25.0000
diag	nearly_ideal	Cp_ig	29.6051
diag	nearly_ideal	H_R	-0.0040
diag	nearly_ideal	H_ig	5915.3500
diag	nearly_ideal	H_real	5915.3460
diag	nearly_ideal	P	1000.0000
diag	nearly_ideal	S_R	-0.0000
diag	nearly_ideal	S_ig	245.0414
diag	nearly_ideal	S_real	245.0413
diag	nearly_ideal	T	500.0120
diag	nearly_ideal	Z	1.0000
diag	nearly_ideal	gamma	1.3905

. . . and 13 more reference values in the case's `expected`.

temperature02_engineers_ladder

tutorials/props/thermo/temperature02_engineers_ladder

rung__H2 propertyPoint	**rung__neon** propertyPoint	**rung__N2** propertyPoint
rung__Ar propertyPoint	**rung__O2** propertyPoint	**rung__CH4** propertyPoint
rung__C2H4 propertyPoint	**rung__benzene** propertyPoint	**rung__water** propertyPoint
rung__ethyleneGlycol propertyPoint	**rung__naphthalene** propertyPoint	**rung__biphenyl** propertyPoint

What it does: The range a chemical engineer actually works in

The story: THE LADDER A CHEMICAL ENGINEER ACTUALLY CLIMBS, and the engine CHECKS it rather than quoting it.

Each rung asks one question: at the temperature this substance's record calls its normal boiling point, what does its VAPOUR-PRESSURE CORRELATION say the pressure is? The answer should be one atmosphere – that is what a normal boiling point means – and how close it lands is a measure of the record, published rather than assumed.

THE OTHER WAY ROUND WOULD HAVE BEEN NICER AND DOES NOT WORK. Asking for 'T_bubble' at 1 atm solves for the temperature, which is the more direct question; 'BubblePoint::compute' carries its own initial guess and does not converge at 20 K, a gap this repository already has on record (see docs/design/held-out-pressure.md). Verifying the pressure at a declared temperature asks the same physics of the same correlation and is robust, so that is what runs here – said plainly rather than papered over.

WHY THE LADDER STOPS AT GLYCEROL, and it is not because the list got boring. Above roughly 600 K an ordinary organic does not have a normal boiling point to check: it CRACKS first, so there is no one-atmosphere equilibrium to ask about. The scale itself changes character there – ITS-90 stops interpolating on vapour pressure and gas thermometry altogether and realises its upper range on the FREEZING POINTS OF METALS, then on Planck radiation above the silver point. Those fixed points are declared values of the scale, not something this engine computes, and the page says so where it lists them.

DOWTHERM A IS ABSENT AND THAT IS A FINDING, not an oversight. The hot-oil fluid an engineer actually runs a 530 K loop on is curated here with 'role nonvolatile' and carries NO vapour-pressure correlation, so this case cannot ask it the question it asks the others. Named rather than quietly skipped: an omission a reader cannot see is the kind of gap this project refuses to leave.

Eight rungs, from liquid hydrogen to boiling water, checked by the engine rather than quoted at you.

At the temperature each substance's record calls its normal boiling point, what does its vapour-pressure correlation say the pressure is?

*It should be one atmosphere — that is what a normal boiling point *means*. How close it lands measures the record, and the answer is published rather than assumed.*

The substance every engineer trusts most is the worst rung on the ladder.

Water's Antoine fit declares Trange (273 373). Its normal boiling point is 373.15 K. The fit stops 0.15 K before the temperature everybody uses it at, and evaluated there it returns 1.026 atm instead of 1.000 — 2.6 % out, on the one substance whose boiling point every reader could recite.

Seven cryogens the reader has no intuition for land inside 1.4 %. The familiar one does not. Familiarity is not accuracy, and a declared validity range is worth more than a feeling.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	rung_H2	Cp_ig	27.3234
diag	rung_H2	H_R	0.0000
diag	rung_H2	H_ig	-7841.8140
diag	rung_H2	H_real	-7841.8140
diag	rung_H2	P	101325.0000
diag	rung_H2	Psat_H2	101850.7659
diag	rung_H2	S_R	0.0000
diag	rung_H2	S_ig	55.7336
diag	rung_H2	S_real	55.7336
diag	rung_H2	T	20.3900
diag	rung_H2	Z	1.0000
diag	rung_H2	gamma	1.4374
diag	rung_H2	v_molar	0.0017
diag	rung_neon	Cp_ig	20.7861
... and 155 more reference values in the case's `expected`.			

thermoTest

atoms01_formula_parser

tutorials/props/thermoTest/atoms01_formula_parser

atoms
elementalComposition

What it does: The shared formula parser, opened up: nested groups, both hydrate spellings, an ion charge that is not an element

The story: THE FORMULA PARSER, WITH ITS LID OFF.

The element balance is only as good as the thing that reads a formula, so that reader is ONE piece of code (src/thermo/ElementComposition) and ‘elementalComposition‘ is its glass-box surface: give it a formula, see the atoms it counted.

The formulas here are chosen to be the awkward ones, because those are where a hand-rolled parser goes wrong:

C2H6O plain, the control; Ca(OH)2 a nested group with a multiplier – 2 O and 2 H, not 1; CaSO4:2H2O a HYDRATE, the colon form: 6 O in total, 4 from the sulfate and 2 from the two waters; MgSO4:7H2O the same salt, colon form: Mg 1, S 1, O 11, H 14; MgSO4.7H2O THE SAME SALT AS A CHEMIST WRITES IT, and the one formula here that is REFUSED. The ASCII '.' is ambiguous, and Choupo resolved it in favour of the decimal subscript – see sepiolite below – so this spelling would silently count the seven waters into the oxygen: O 5.7 and H 2 where the salt has O 11 and H 14. A wrong element balance from a spelling nobody would flag by eye. It refuses and names both unambiguous forms; Mg2Si3O7.5OH:3H2O why the dot cannot simply BE the hydrate separator: this is curated sepiolite, and its 7.5 is a real half-integer; SO4-2 an ION: the charge is not an element and must not become one; (NH4)2SO4 a nested group of a nested group’s worth of hydrogen – 8 H.

A parser that silently mis-reads any of these produces an element balance that closes on the wrong atoms, which is worse than one that refuses.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	atoms	C2H6O_C	2
diag	atoms	C2H6O_H	6
diag	atoms	C2H6O_O	1
diag	atoms	CO2_C	1
diag	atoms	CO2_O	2
diag	atoms	ethanol_C	2
diag	atoms	ethanol_H	6
diag	atoms	ethanol_O	1
diag	atoms	formulas_available	10
diag	atoms	formulas_total	11
diag	atoms	water_H	2
diag	atoms	water_O	1

`h_surface_identity`

`tutorials/props/thermoTest/h_surface_identity`

h_identity
hConsistency

What it does: The state identities of the canonical enthalpy surface: dh/dT == Cp_phase, the 298 K vaporisation anchor, and Kirchhoff on Hvap_state – pinned to round-off

The story: thermodynamics as a REGRESSION TEST.

A surface that anchors each phase leg on the 298.15 K formation datum and integrates the TARGET phase's own Cp obeys, by construction:

(1) dh_liq/dT = Cp_liquid(T) (2) dh_gas/dT = Cp_idealGas(T) (3) h_g(298.15) - h_liq(298.15) = Hvap(298.15) (the anchor) (4) d[h_g - h_liq]/dT = Cp_g - Cp_liq (Kirchhoff)

This op checks all four by central finite difference for benzene, toluene, water and ethanol at 350 K and REFUSES above 1e-6 relative error – so no future "convenient" leg (a Watson-slope liquid, a mixed-datum gas) can re-enter the balances unnoticed. The golden pins every per-species error.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	h_identity	benzene_anchor298_rel	0.000e+00
diag	h_identity	benzene_dhgas_vs_cpig_rel	7.289e-11
diag	h_identity	benzene_dhliq_vs_cpliq_rel	4.729e-13
diag	h_identity	benzene_kirchhoff_rel	2.058e-10
diag	h_identity	ethanol_anchor298_rel	1.694e-16
diag	h_identity	ethanol_dhgas_vs_cpig_rel	3.826e-11
diag	h_identity	ethanol_dhliq_vs_cpliq_rel	1.907e-11
diag	h_identity	ethanol_kirchhoff_rel	1.287e-10
diag	h_identity	toluene_anchor298_rel	0.000e+00
diag	h_identity	toluene_dhgas_vs_cpig_rel	6.085e-11
diag	h_identity	toluene_dhliq_vs_cpliq_rel	1.854e-13
diag	h_identity	toluene_kirchhoff_rel	2.214e-10
diag	h_identity	water_anchor298_rel	6.529e-16
diag	h_identity	water_dhgas_vs_cpig_rel	3.627e-11

. . . and 3 more reference values in the case's `expected`.

model4_speciation_scaling

tutorials/props/thermoTest/model4_speciation_scaling

brine	concentrate
speciate	scalingScan

What it does: RO membrane-scaling audit of a brackish groundwater: aqueous speciation (Davies) with pH SOLVED from electroneutrality (the feed charge imbalance is announced first) + saturation indices SI = log10(IAP/K), scanned over water recovery to r = 0.85 (concentrate = feed/(1-r)) TWICE: closed system (DIC conserved and concentrating) vs open to the atmosphere (a(CO2aq) pinned by Henry at pCO2 = 4.2e-4 atm, DIC a solved outcome – this CO2-rich water degasses, pH and calcite SI jump above the closed curve) – the calcite SI curves differ; that contrast is the lesson. Three analytic pins lock the physics: pure water at given pH 7 reproduces Kw (m_OH ~ 1e-7), a back-computed gypsum-saturated CaSO4 solution gives SI_gypsum ~ 0.00, and pure water with pH solved lands at exactly 7.000.

The story: thermoTest / model4_speciation_scaling

The SAME system {NaCl, CaSO4, water} in its THIRD role: a MULTI-ION broth. Here we no longer think of "NaCl" or "CaSO4" – the aqueous speciation engine reasons in the shared ions Na+, Cl-, Ca2+, SO4-2 (plus the neutral ion pair CaSO4aq), and reports SATURATION INDICES SI = log10(IAP/Ksp) for every solid the ions can form: gypsum (CaSO4.2H2O) and halite (NaCl).

Na+ comes from NaCl, Cl- from NaCl, Ca2+/SO4-2 from CaSO4 – but once dissolved there are no salt boundaries, only ions. This is why a "salt" cannot carry its own activity model: the ion pool is shared (the species/ tier of the tree).

MODEL: Pitzer-HMW is MANDATORY here. The ionic strength is I ~ 3.3 mol/kg (NaCl-dominated); the Davies law is only trustworthy to I ~ 0.5 m, so the ‘activityModel‘ is declared explicitly (no silent default) – with Davies the saturation indices at this I are quantitatively meaningless.

The brine is NaCl-dominated (3 mol/kg, near seawater) with CaSO4 (0.10 mol/kg) right at gypsum saturation (SI ~ 0). Charge balances exactly (Na + 2 Ca = Cl + 2 SO4), so pH is pinned at 7. The scan concentrates the brine (evaporation; r = fraction of water removed): gypsum scales first, at r ~ 0.035 (it starts saturated), halite only near r ~ 0.50. Note m_CaSO4aq ~ 0.005: the neutral ion pair is a real fraction of the calcium – "dissociation" is partial.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	brine	I	3.26782
diag	brine	SI_anhydrite	-0.22134
diag	brine	SI_gypsum	-0.0221674
diag	brine	SI_halite	-0.910511
diag	brine	SI_mirabilite	-2.13799
diag	brine	SI_thenardite	-2.98985
diag	brine	aw	0.890402
diag	brine	gamma_CaSO4aq	1
diag	brine	m_CaSO4aq	0.00466294
diag	brine	pH	7
diag	concentrate	I_r0	3.26782
diag	concentrate	I_rmax	15.6285
diag	concentrate	SI_anhydrite_r0	-0.22134
diag	concentrate	SI_anhydrite_rmax	1.39807

. . . and 26 more reference values in the case’s `expected`.

Category III: Batch and time-dependent (choupoBatch)

$\mathrm{d}\mathbf{Y}/\mathrm{d}t = f(\mathbf{Y})$ integrated in real time, with recipes for discrete events; trajectories, not steady states.

adsorber

The theory you need. Adsorption in a vessel or a bed: the isotherm $q^*(T,p)$ (Langmuir $q = q_{\max}bp/(1+bp)$, van't Hoff $b(T)$) is the curated PAIR datum; the LDF rate $\mathrm{d}q/\mathrm{d}t = k\,(q^* - q)$ carries the EQUIPMENT datum k (this particle, this bed – it lives in the case, never the catalogue). Work the cases as a staircase. `batch09`: a closed vessel relaxes to the inventory equilibrium – one algebraic root, pinned independently of the run. `batch10`: two species compete for the same sites (extended Langmuir) – the selectivity is an equilibrium statement, and the model's Gibbs–Duhem inconsistency for unequal $q_{\max}$ is declared, not hidden. `batch11`: in the Henry limit the whole vessel is LINEAR and the integrator must match $q_\infty(1 - e^{-k_{\text{eff}}t})$ to 10^{-8} – the analytic gate. `batch12`: declaring the headspace pressure AND the inventory over-specifies a closed ideal-gas vessel – the contradiction FATALs; P is a result. `batch13`: the fixed bed – the stoichiometric time $t_{\text{st}} = (L/u)(\varepsilon + \rho_b q^*/c_{\text{in}})$ is announced BEFORE integration, breakthrough spreads around it, and the constant-u closure's fabricated carrier is printed, pinned and discounted: the DECLARED model residual is itself the lesson. `batch14`/`batch15` are the numerical hygiene: pure transport vs the analytic advection–dispersion solution, and the mesh study that earns $N = 100$. *Full derivation: Theory Guide, the adsorption chapter.*

batch09_adsorber_co2

`tutorials/batch/adsorber/batch09_adsorber_co2`

adsorber
batchAdsorber

What it does: Closed isothermal CO2 uptake on zeolite 13X (LDF, solid film): the vessel relaxes to the inventory equilibrium p_inf = 7229.13 Pa, q_inf = 1.9708 mol/kg

The story: closed, isothermal gas–solid uptake vessel.

2 mol of pure CO2 in a 0.01 m3 ideal-gas headspace over 1 kg of regenerated zeolite 13X at 298.15 K. Linear-driving-force kinetics on the solid film (dq/dt = k∗(q∗ - q), k = 0.05 1/s, equipment data, this case's sample) against the competitive-Langmuir equilibrium record of the pair catalogue (here a single pair, CO2/zeolite13X: q_max = 5.0 mol/kg, b_298 = 9.0 1/bar, dH_ads = -45 kJ/mol).

The final state is FIXED BY INVENTORY, not by kinetics: with n_gas = p V/(R T) and q = q_max b (p/1e5)/(1 + b p/1e5),

p V/(R T) + m_ads ∗ q_max ∗ b ∗ (p/1e5)/(1 + b∗(p/1e5)) = n0

whose root at 298.15 K is (a2_vectors.py, 10 digits)

p_inf = 7229.128209 Pa q_inf = 1.970838025 mol/kg n_gas = 0.02916197467 mol uptake = 1.970838025 mol (98.54 %) tau (linearised at the equilibrium) = 0.4768 s

The run must reproduce these to the golden tolerances – an independent analytic anchor, not a mirror of the code. The ODE state is ONLY q; n_gas is algebraic (n_tot - m_ads∗q), so the per-species closure column closure_worst_rel in trajectory.csv is exact by construction (<= 1e-13). Log: the unit announces the ideal-gas assumption, the initialLoading default, the RK4 stiffness bound and the isosteric-KPI label at start-up.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	adsorber	P_final	7229.128209
kpi	adsorber	Q_ads_isosteric_kJ	-88.68771114
kpi	adsorber	T_final	298.15
kpi	adsorber	closure_worst_rel	0
kpi	adsorber	n_CO2_final	2.916197467e-05
kpi	adsorber	q_CO2_final	1.970838025
kpi	adsorber	t_end	600
kpi	adsorber	totalMoles_final	2.916197467e-05
kpi	adsorber	uptake_CO2_mol	1.970838025
kpi	campaign	element_C_closure_rel	0
kpi	campaign	element_O_closure_rel	0
kpi	campaign	element_worst_closure_rel	0
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0

. . . and 12 more reference values in the case's `expected`.

batch10_adsorber_co2_n2

tutorials/batch/adsorber/batch10_adsorber_co2_n2

adsorber
batchAdsorber

What it does: Closed isothermal competitive CO_2/N_2 uptake on zeolite 13X: extended-Langmuir site competition drives the equilibrium selectivity S = 71.43

The story: COMPETITIVE binary uptake, closed vessel.

0.5 mol CO_2 + 1.5 mol N_2 in a 0.01 m3 ideal-gas headspace over 1 kg of regenerated zeolite 13X at 298.15 K. Both species carry an isotherm record (extended Langmuir: CO_2 q_max 5.0 / b_298 9.0 1/bar; N_2 3.5 / 0.18 1/bar) and both carry a declared LDF constant – so both integrate, and the strong adsorbate suppresses the weak one through the SHARED site denominator 1 + Sum_j b_j p_j.

The final state solves the 2-species inventory system n_gas_i + m_ads * q_i(p_CO2, p_N2) = n0_i, p_i = n_gas_i R T / V whose root (a2_vectors.py, Newton residual 0):

CO_2: p_inf = 0.01611568794 bar q = 0.4934990047 mol/kg N_2 : p_inf = 1.802652166 bar q = 0.7728183086 mol/kg P_total = 181876.7854 Pa selectivity S = (q_CO2/q_N2)/(p_CO2/p_N2) = 71.42857143 = (q_max*b)_CO2/(q_max*b)_N2 (extended-Langmuir identity)

The golden pins the COMPETITIVE path: get the shared denominator wrong and every one of these numbers moves. closure_worst_rel stays exact by construction (state = q only; n_gas algebraic).

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	adsorber	P_final	181876.7854
kpi	adsorber	Q_ads_isosteric_kJ	-36.1181845604
kpi	adsorber	T_final	298.15
kpi	adsorber	closure_worst_rel	0
kpi	adsorber	n_CO2_final	6.500995276e-06
kpi	adsorber	n_N2_final	7.271816914e-04
kpi	adsorber	q_CO2_final	0.4934990047
kpi	adsorber	q_N2_final	0.7728183086
kpi	adsorber	t_end	600
kpi	adsorber	totalMoles_final	0.000733682686703
kpi	adsorber	uptake_CO2_mol	0.4934990047
kpi	adsorber	uptake_N2_mol	0.7728183086
kpi	campaign	element_C_closure_rel	0
kpi	campaign	element_N_closure_rel	0

. . . and 16 more reference values in the case's `expected`*.*

`batch11_adsorber_henry`

`tutorials/batch/adsorber/batch11_adsorber_henry`

adsorber
batchAdsorber

What it does: Closed isothermal Henry-limit uptake (linear isotherm): the coupled gas+solid system is LINEAR and the run must match q(t) = q_inf∗(1 - exp(-k_eff t)) to 1e-8

The story: the EXACT-solution gate (G1a of the A2 spec).

1 mol of CO2 in a 0.01 m3 ideal-gas headspace over 1 kg of a SYNTHETIC Henry-limit sorbent (linear isotherm q∗ = K_H p, K_H = 0.4 mol/(kg bar) at 298.15 K – a teaching record, vendored case-local). With a linear isotherm and one species the coupled vessel is LINEAR and closes in analytic form:

alpha = K_H[mol/(kg Pa)] ∗ R∗T/V = 0.9915828118 1/kg k_eff = k ∗ (1 + alpha∗m_ads) = 0.09957914059 1/s q_inf = alpha∗n0 / (1 + alpha∗m) = 0.4978868094 mol/kg q(t) = q_inf ∗ (1 - exp(-k_eff∗t)) (q0 = 0)

q(5) = 0.1952670621 q(10) = 0.3139520079 q(30) = 0.4727835287 q(60) = 0.4966211107 mol/kg

The pedagogical point the closed vessel adds over the fixed-q∗ packet formula: k_eff > k – gas depletion ACCELERATES the approach to equilibrium (the packet's q∗ = const law is the V_gas → infinity limit of this same solution). The trajectory rows at t = 5/10/30/60 s must match the table above to 1e-8 mol/kg (RK4 dt = 0.01 s); the golden pins the t = 60 s state.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`adsorber`	`P_final`	`124785.4636`
`kpi`	`adsorber`	`Q_ads_isosteric_kJ`	`-9.93242221329`
`kpi`	`adsorber`	`T_final`	`298.15`
`kpi`	`adsorber`	`closure_worst_rel`	`0`
`kpi`	`adsorber`	`n_CO2_final`	`5.033788893e-04`
`kpi`	`adsorber`	`q_CO2_final`	`0.4966211107`
`kpi`	`adsorber`	`t_end`	`60`
`kpi`	`adsorber`	`totalMoles_final`	`5.033788893e-04`
`kpi`	`adsorber`	`uptake_CO2_mol`	`0.4966211107`
`kpi`	`campaign`	`element_C_closure_rel`	`0`
`kpi`	`campaign`	`element_O_closure_rel`	`0`
`kpi`	`campaign`	`element_worst_closure_rel`	`0`
`kpi`	`campaign`	`energy_balance_available`	`0`
`kpi`	`campaign`	`energy_records_invalid`	`0`

. . . and 12 more reference values in the case's `expected`.

batch12_adsorber_refused_P

tutorials/batch/adsorber/batch12_adsorber_refused_P

What it does: REFUSAL lesson: declared initial.P (10 bar) contradicts n∗R∗T/V (4.958 bar) – the honest cross-check FATALs; P of a closed ideal-gas headspace is a result, not an input

The story: the REFUSAL lesson (deliberate, must FAIL).

Identical to batch09_adsorber_co2 EXCEPT initial.P declares 10 bar while n∗R∗T/V = 4.9579 bar. P of a closed ideal-gas headspace is a RESULT (n, T, V fix it – Duhem); a declared P is honoured only as an honest cross-check, and a relative deviation > 1e-3 REFUSES, printing both numbers. Expected FATAL (exit != 0):

batchAdsorber 'adsorber': declared P inconsistent with n∗R∗T/V – initial.P = 1e+06 Pa (10 bar) but n∗R∗T/V = 495791 Pa (4.95791 bar), relative deviation 1.01698 > 1e-3. ...

Marked .expect-nonconvergence: counted EXPECTED-FAIL by runTests, never green, never recordable. Base case (the passing twin):

2 mol of pure CO2 in a 0.01 m3 ideal-gas headspace over 1 kg of regenerated zeolite 13X at 298.15 K. Linear-driving-force kinetics on the solid film (dq/dt = k∗(q∗ - q), k = 0.05 1/s, equipment data, this case's sample) against the competitive-Langmuir equilibrium record of the pair catalogue (here a single pair, CO2/zeolite13X: q_max = 5.0 mol/kg, b_298 = 9.0 1/bar, dH_ads = -45 kJ/mol).

The final state is FIXED BY INVENTORY, not by kinetics: with n_gas = p V/(R T) and q = q_max b (p/1e5)/(1 + b p/1e5),

p V/(R T) + m_ads ∗ q_max ∗ b ∗ (p/1e5)/(1 + b∗(p/1e5)) = n0

whose root at 298.15 K is (a2_vectors.py, 10 digits)

p_inf = 7229.128209 Pa q_inf = 1.970838025 mol/kg n_gas = 0.02916197467 mol uptake = 1.970838025 mol (98.54 %) tau (linearised at the equilibrium) = 0.4768 s

The run must reproduce these to the golden tolerances – an independent analytic anchor, not a mirror of the code. The ODE state is ONLY q; n_gas is algebraic (n_tot - m_ads∗q), so the per-species closure column closure_worst_rel in trajectory.csv is exact by construction (<= 1e-13). Log: the unit announces the ideal-gas assumption, the initialLoading default, the RK4 stiffness bound and the isosteric-KPI label at start-up.

batch13_breakthrough_co2

tutorials/batch/adsorber/batch13_breakthrough_co2

bed
fixedBedAdsorber

What it does: CO2 15%/He breakthrough on zeolite 13X – 1-D isothermal fixed bed (A3): t_st = 3040.5 s announced pre-run, t_b(5%) = 2890.7 s, ledger closure at machine level

The story: CO2/He breakthrough on a zeolite-13X bed.

THE A3 anchor case (spec 2026-07-12): a 0.5 m isothermal fixed bed of zeolite 13X, fed 15% CO2 in helium at u = 0.05 m/s superficial, 1 bar, 298 K. Helium carries NO isotherm record on 13X → announced inert CARRIER, transported but never adsorbed, closed by difference from the constant c_tot = P/(RT). At 15% CO2 this is NOT a dilute approximation: the constant-(u, P, T) closure fabricates He at the net uptake rate and the campaign material balance SHOWS that residual (KPIs carrier_fabricated_mol, physical_mass_closure_rel) – the named error bites, by design (forum #119); the genuinely clean transport anchor is batch14. The engine announces, BEFORE integrating, where the front must arrive:

q∗(c_in) = 5∗9∗0.15/(1 + 9∗0.15) = 2.872340426 mol/kg R_f = eps + rho_b q∗ / c_in = 304.0512509 t_st = (L/u) R_f = 3040.512509 s u_sh = u/R_f = 1.644459605e-4 m/s

Independent gates this run must reproduce (anchors_a3.py, FV N = 100):

t_b(5%) = 2890.669098 s t_50 = 3036.448293 s t_b(95%) = 3204.166058 s INT(1 - c_out/c_in)dt = t_st to 2.2e-10 (telescopic FV conservation) ledger mass closure <= 1e-10 (IN/OUT integrated as rows of the SAME ODE)

Numerics: conservative FV, nCells = 100 DECLARED (do the 25/50/100/200 mesh study – batch15 publishes it: first-order upwind, observed order ~1.03-1.05), Danckwerts inlet as the imposed face flux u∗c_in, outlet dc/dz = 0. u, P, T are DECLARED constants of this phase, and the price is said OUT LOUD: the real velocity drops ~15% across the uptake zone, so the constant-u closure FABRICATES carrier (He) at exactly the net uptake rate (9.19 mol here) – announced in the run header, pinned as the KPI carrier_fabricated_mol, discounted in the campaign inventory; Ergun and the energy balance land in A4 (the energy gap is NAMED).

Log: run header prints the hypotheses, the derived rho_p, Pe = 250, the Gershgorin step bound term by term and the stiffness verdict; profiles land in <t>/bed.profile every 100 s; trajectory.csv carries c_out, y_out, qbar and the per-species ledger closure.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	E_adsorption_seg0_kJ	-413.617021183
kpi	bed	E_adsorption_total_kJ	-413.617021183
kpi	bed	P_final	100000
kpi	bed	Pe	250
kpi	bed	T_final	298
kpi	bed	c_out_over_cin_final_CO2	0.999999989455
kpi	bed	carrier_fabricated_mol	9.19148935961
kpi	bed	integral_anchor_CO2	3040.51245606
kpi	bed	mass_closure_CO2	8.94231742202e-15
kpi	bed	n_CO2_final	1.21079552014e-05
kpi	bed	n_He_final	6.86117461711e-05
kpi	bed	physical_mass_closure_rel	0.108446875314
kpi	bed	qbar_CO2_final	2.87234042488
kpi	bed	retention_factor_CO2	304.051250948

. . . and 29 more reference values in the case's `expected`*.*

batch14_transport_only

tutorials/batch/adsorber/batch14_transport_only

What it does: Pure transport (k = 0) on the batch13 bed, N = 200: t_50 vs the Ogata-Banks analytic ADE solution (gate G1) under fixed RK4 with the Gershgorin bound printed term by term

The story: the k = 0 transport gate (A3 spec, gate G1).

Same bed, same feed, same mesh family as batch13 – but the LDF constant is DECLARED ZERO: the solid is frozen and CO_2 crosses the bed as a pure tracer. What is left is exactly the advection-dispersion equation with a Danckwerts inlet, and THAT has an analytic yardstick (Ogata-Banks, semi-infinite ADE; the finite-domain difference is O(1/Pe) = 0.4% at Pe = 250):

u_i = u/eps = 0.125 m/s D_i = Dax/eps = 2.5e-4 m2/s c/c0 (L, 3.5 s) = 0.07340274358 c/c0 (L, 4.0 s) = 0.5178057707 c/c0 (L, 4.5 s) = 0.913645218 t_50 analytic = 3.984074227 s

FV yardsticks pinned by this case (anchors_a3.py, N = 200):

t_50 numeric = 3.975611088 s (0.21% below analytic – first-order upwind; the deviation HALVES with each mesh doubling: 1.40% at N=25, 0.74% at 50, 0.40% at 100, 0.21% at 200, 0.11% at 400) INT(1 - c_out/c_in)dt = eps L / u = 4.0 s (telescopic conservation: exact identity once saturated) ledger closure ~ 1e-14 (k = 0)

A declared zero is not a silent default: the unit announces "k = 0 DECLARED: transport-only diagnostic, the solid is frozen (gate G1)". Note the He carrier fabrication of batch13 vanishes here (no uptake): carrier_fabricated_mol = 0.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	E_adsorption_seg0_kJ	0
kpi	bed	E_adsorption_total_kJ	0
kpi	bed	P_final	100000
kpi	bed	Pe	250
kpi	bed	T_final	298
kpi	bed	c_out_over_cin_final_CO2	0.999999999893
kpi	bed	carrier_fabricated_mol	0
kpi	bed	integral_anchor_CO2	3.99999999999
kpi	bed	mass_closure_CO2	4.09039795713e-14
kpi	bed	n_CO2_final	1.21079552058e-05
kpi	bed	n_He_final	6.86117461666e-05
kpi	bed	physical_mass_closure_rel	0
kpi	bed	qbar_CO2_final	0
kpi	bed	t_50_CO2	3.97561121931

. . . and 23 more reference values in the case's `expected`.

batch15_mesh_study

tutorials/batch/adsorber/batch15_mesh_study

What it does: Mesh-study member N = 25 of the batch13 breakthrough: t_b(5%) converges monotonically 2732 → 2839 → 2891 → 2916 s with observed order 1.03-1.05; full table in reports/meshStudy.md

The story: the declared-mesh lesson (A3 spec, gate G6).

The axial mesh is a DECLARED modelling choice (nCells; absent = FATAL), and this case is the receipt: the batch13 breakthrough run on the 25/50/100/200 family, everything else identical. First-order upwind advection dominates the error, so t_b(5%) must converge MONOTONICALLY with observed order p ~ 1:

N = 25 t_b5 = 2732.027658 s (this case) N = 50 t_b5 = 2838.542518 s N = 100 t_b5 = 2890.669098 s (batch13, the production mesh) N = 200 t_b5 = 2915.764796 s p(25/50/100) = 1.031 p(50/100/200) = 1.055 Richardson (p = 1): t_b5(inf) ~ 2940.86 s → N = 100 sits 1.7% low

The full table, the four overlaid breakthrough curves and the verdict are PUBLISHED in reports/meshStudy.md (generated from the four runs; the report header cites mesh, tolerances and integrator). The ledger mass closure stays <= ~1e-11 on EVERY mesh – conservation is telescopic in the FV form, it does not improve with refinement because it was never discretisation-limited (gate G3).

This case pins the N = 25 member; its numbers are DELIBERATELY the worst of the family (t_b5 5.8% below batch13) – coarse-mesh optimism made visible, the point of the lesson.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	E_adsorption_seg0_kJ	-413.616606535
kpi	bed	E_adsorption_total_kJ	-413.616606535
kpi	bed	P_final	100000
kpi	bed	Pe	250
kpi	bed	T_final	298
kpi	bed	c_out_over_cin_final_CO2	0.999973635802
kpi	bed	carrier_fabricated_mol	9.19148014522
kpi	bed	integral_anchor_CO2	3040.50945345
kpi	bed	mass_closure_CO2	7.40535661511e-15
kpi	bed	n_CO2_final	1.21079316034e-05
kpi	bed	n_He_final	6.86117697691e-05
kpi	bed	physical_mass_closure_rel	0.108446766597
kpi	bed	qbar_CO2_final	2.87233754538
kpi	bed	retention_factor_CO2	304.051250948

...and 29 more reference values in the case's `expected`.

batch16_ergun_profile

tutorials/batch/adsorber/batch16_ergun_profile

bed
fixedBedAdsorber

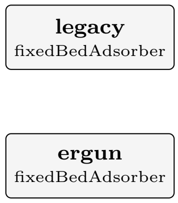
What it does: A4 F1: pure-He Ergun profile against the closed-form P-squared solution

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	P_first_cell_Pa	99999.4670134
kpi	bed	P_mid_cell_Pa	99946.1540012
kpi	bed	P_last_cell_Pa	99893.8796441
kpi	bed	P_out_boundary_Pa	99893.3459257
kpi	bed	carrier_fabricated_mol	0
kpi	bed	physical_mass_closure_rel	0
kpi	campaign	mass_closure_rel	0
kpi	bed	E_adsorption_seg0_kJ	0
kpi	bed	E_adsorption_total_kJ	0
kpi	bed	P_final	99946.6820773
kpi	bed	T_final	298.15
kpi	bed	deltaP_first_to_out_Pa	106.12034902
kpi	bed	gasInventory_mol	0.0806360746763
kpi	bed	n_He_final	8.06360746763e-05
. . . and 22 more reference values in the case's ***expected****.*			

batch17_dilute_ergun_limit

tutorials/batch/adsorber/batch17_dilute_ergun_limit



What it does: A4 F2: 400 ppm uptake, Ergun must approach the A3 constant-velocity limit

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	5.32078747864e-13
kpi	campaign	element_He_closure_rel	0.000139137956493
kpi	campaign	element_O_closure_rel	5.32078747864e-13
kpi	campaign	element_worst_closure_rel	0.000139137956493
kpi	campaign	element_worst_closure_vs_declared_rel	5.32078747864e-13
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	2
kpi	campaign	mass_closure_rel	0.000138528772162
kpi	campaign	mass_kg_external_out	0.161636709221
kpi	campaign	mass_kg_final	0.000558961907879
kpi	campaign	mass_kg_initial	0.162173202362
kpi	campaign	mass_residual_kg	2.24687671715e-05
kpi	campaign	moles_kmol_external_out	0.0403339312462
... and 53 more reference values in the case's `expected`.			

batch18_ergun_conservation

tutorials/batch/adsorber/batch18_ergun_conservation

bed
fixedBedAdsorber

What it does: A4 F3: 15 percent CO_2 with explicit carrier continuity and Ergun velocity

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	E_adsorption_seg0_kJ	-413.699637907
kpi	bed	E_adsorption_total_kJ	-413.699637907
kpi	bed	P_final	100047.159082
kpi	bed	P_first_cell_Pa	100092.418138
kpi	bed	P_last_cell_Pa	100001.886961
kpi	bed	P_mid_cell_Pa	100047.162776
kpi	bed	P_out_boundary_Pa	100000
kpi	bed	Pe	250
kpi	bed	T_final	298.15
kpi	bed	c_out_over_cin_final_CO2	1.00000763107
kpi	bed	carrier_fabricated_mol	-6.8212102633e-13
kpi	bed	deltaP_first_to_out_Pa	92.4181379529
kpi	bed	integral_anchor_CO2	3028.46636949
kpi	bed	mass_closure_CO2	1.84877618489e-14

... and 39 more reference values in the case's `expected`.

`batch19_feed_switch_purge`

`tutorials/batch/adsorber/batch19_feed_switch_purge`

bed
fixedBedAdsorber

What it does: A5 feed switch: CO2/He load to 3600 s (batch13 anchors), then pure-He purge to 9000 s – amended feed commitment ledgered, frozen breakthrough KPIs, desorption duty as the same state difference

The story: THE A5 WITNESS: load, then switch the feed.

batch13's bed (CO2 15% in He on zeolite 13X, the A3 anchor), run through a TWO-STEP cycle in one campaign:

0 .. 3600 s LOAD – identical to batch13's feed; the front arrives on the batch13 anchors (t_5 = 2890.7, t_50 = 3036.4, t_95 = 3204.2 s) 3600 s SWITCH – recipe setParameter bed.feed.CO2 = 0: the feed becomes pure He (concentration- swing regeneration) 3600 .. 9000 s PURGE – q∗(c → 0) = 0, the LDF drives the bed to desorb; the favourable (Langmuir) isotherm makes desorption the UNFAVOURABLE direction – watch the long tail on c_out

What the switch does, and where each consequence is VISIBLE:

The feed commitment is DECLARED matter (the campaign datum owns A∗u∗c_in∗(endTime - t) of future feed). Re-declaring the feed at t = 3600 s AMENDS that commitment: the un-fed CO2 leaves the datum and the extra He enters it, priced and recorded as 'feedAmendment' ledger records against the external boundary – the campaign mass/element balances read the records and still close (KPIs mass_kg_amend_in/out, element_worst_closure_vs_declared_rel ~ 1e-14).

The breakthrough sampler's reference basis (c_out/c_in) dies with the old feed: crossings and integral FREEZE at their pre-switch values (announced), so t_5/t_50/t_95 here must MATCH batch13's anchors – the frozen claim describes the first feed step. The desorption wave lives on in the trajectory's c_out_CO2 / qbar_CO2 columns.

The adsorption duty stays an EXACT state difference on the adsorbed inventory: E_adsorption = dH_ads ∗ (n_ads(end) - n_ads(0)). batch13 (load only) ledgers -413.6 kJ; here the purge takes part of the loading back OUT, so |E| is SMALLER – desorption is endothermic and the ledger shows the heat returning, with no new formula anywhere.

u, P, T stay DECLARED CONSTANTS: asking the bed for a thermal swing (feed.T), a flow transient (u) or a blowdown (P) is REFUSED by name (fired by bin/curate/check_feed_switch.py, batch12-style honesty).

The carrier-fabrication caveat of the constant-(u, P, T) closure applies unchanged (forum #119): the declared residual explains the He element closure, shown and never discounted.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	E_adsorption_seg0_kJ	-23.659607554
kpi	bed	E_adsorption_total_kJ	-23.659607554
kpi	bed	P_final	100000
kpi	bed	Pe	250
kpi	bed	T_final	298
kpi	bed	carrier_fabricated_mol	0.525769056755
kpi	bed	integral_anchor_CO2	3040.50572088
kpi	bed	mass_closure_CO2	1.23888355951e-14
kpi	bed	n_CO2_final	3.21638319252e-07
kpi	bed	n_He_final	8.03980630532e-05
kpi	bed	physical_mass_closure_rel	0.00289489594687
kpi	bed	qbar_CO2_final	0.164302830236
kpi	bed	t_50_CO2	3036.44839075
kpi	bed	t_breakthrough_5pct_CO2	2890.66919752

. . . and 31 more reference values in the case's `expected`*.*

batch20_thermal_breakthrough

tutorials/batch/adsorber/batch20_thermal_breakthrough

bed
fixedBedAdsorber

What it does: A5-T1 adiabatic CO2/He breakthrough (ergun, one T per cell): thermal wave + earlier front vs batch18, anchors u_th and DT_ad announced pre-run, campaign energy on vessel enthalpy alone

The story: THE A5-T1 WITNESS: the adiabatic bed.

batch18's ergun-mode bed (15% CO2/He, N = 25) with the energy balance DECLARED: 'energyBalance adiabatic;' – one temperature per cell, the isosteric heat staying IN the bed as a travelling thermal wave instead of being removed by an unnamed coolant. Design record: docs/design/fixed-bed-thermal-a5.md.

What the engine announces BEFORE integrating (equilibrium theory):

pure thermal-wave velocity u_th = u c_tot cpg / (eps c_tot cpg + rho_b cp_s) and the time L/u_th the heat needs to LEAVE the bed; the adiabatic LIMIT DT_ad = q∗(-dH_ads)/(cp_s + eps c_tot cpg/rho_b) – the all-heat-retained BOUND. Here u_th < u_sh: the heat LAGS the front, so the observed outlet-T rise sits well BELOW the bound (~ +38 K vs the 140 K limit) while the WARM bed breaks through far earlier than batch18's isothermal front (t_50 ~ 806 s vs 3042 s – q∗(T) falls via the van't Hoff b(T), the SAME datum the energy source term uses; one dH_ads, three readers).

The golden pins T_out/T_max, both anchors, the early crossings, and the campaign ENERGY closure – claimed, adiabatically, on vessel enthalpy against the boundary streams alone (per-step raffinate packages carry the outlet cell's own T).

cp_s enters as DECLARED case data (the kLDF pattern – scope + source say where the number came from; curating a primary-cited cp onto the adsorbent asset record is a separate curation act). The adsorbed-phase heat capacity is neglected, declared. Feed enters at the declared T; the raffinate package leaves at the OUTLET CELL's own T so the material ledger carries the thermal wave's enthalpy.

Adiabatic means NO energy record: nothing is exchanged with the environment; the campaign energy balance closes on vessel enthalpy (per-cell pricing: gas + adsorbed at T_j, solid sensible cp_s(T_j-T0)) against the boundary streams alone.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	P_final	100050.377191
kpi	bed	P_first_cell_Pa	100096.839501
kpi	bed	P_last_cell_Pa	100002.080676
kpi	bed	P_mid_cell_Pa	100050.98468
kpi	bed	P_out_boundary_Pa	100000
kpi	bed	Pe	250
kpi	bed	T_final	315.614986735
kpi	bed	T_max_final_K	334.272787919
kpi	bed	T_out_final_K	334.272787919
kpi	bed	c_out_over_cin_final_CO2	0.822859498342
kpi	bed	carrier_fabricated_mol	2.54374299402e-11
kpi	bed	dT_adiabatic_CO2_K	140.405462904
kpi	bed	deltaP_first_to_out_Pa	96.8395014859
kpi	bed	integral_anchor_CO2	2850.55000879

. . . and 37 more reference values in the case's `expected`*.*

batch21_tsa_hot_purge

tutorials/batch/adsorber/batch21_tsa_hot_purge

bed
fixedBedAdsorber

What it does: T1.5 TSA half-cycle: adiabatic load to 3600 s, then 400 K clean He purge – feed.T as the hot-purge control, the re-declared commitment ledgered OUT at 298 K / IN at 400 K, energy balance claimed across the swing

The story: THE T1.5 WITNESS: a real TSA half-cycle.

batch20's adiabatic bed run as LOAD → HOT CLEAN PURGE in one campaign:

0 .. 3600 s LOAD – 15% CO2/He at 298.15 K (batch20's feed; the warm front is through by ~806 s) 3600 s SWITCH – recipe, two setParameter events: feed.CO2 = 0 (clean He purge) feed.T = 400 K (the HOT purge) 3600 .. 12000 s PURGE – the hot clean carrier sweeps the bed: q∗(T) collapses via the van't Hoff b(T) and desorption runs FAR faster than batch19's cold concentration swing

This is the step the old refusal named: "thermal swing needs the non-isothermal bed energy balance". A5-T1 built that balance; feed.T is now the hot-purge CONTROL (in thermal mode ONLY – the isothermal bed still refuses it, by name).

The LEDGER story is the point. A feed at a new temperature re-declares the WHOLE remaining commitment – at the pinned feed pressure the total concentration drops to P/(R∗400) (composition preserved) and even unchanged matter changes enthalpy – so the switch retires the old commitment (one OUT feedAmendment package, all species, at 298.15 K) and declares the new one (one IN package at 400 K): the enthalpy repricing rides in the packages' own T, and the campaign energy balance stays CLAIMED across the swing, boundary streams alone.

Breakthrough crossings/integral freeze at the switch (they describe the load step); the desorption story lives in c_out/qbar/T_out. The frozen pre-run anchors (u_th, DT_ad bound) keep their initialise values – a later switch never rewrites an announced claim.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	P_final	100030.435428
kpi	bed	P_first_cell_Pa	100063.521237
kpi	bed	P_last_cell_Pa	100001.142356
kpi	bed	P_mid_cell_Pa	100028.717422
kpi	bed	P_out_boundary_Pa	100000
kpi	bed	Pe	250
kpi	bed	T_final	350.354335031
kpi	bed	T_max_final_K	399.999997842
kpi	bed	T_out_final_K	306.850590216
kpi	bed	carrier_fabricated_mol	-2.52953213931e-12
kpi	bed	dT_adiabatic_CO2_K	140.405462904
kpi	bed	deltaP_first_to_out_Pa	63.521236595
kpi	bed	integral_anchor_CO2	1312.9982971
kpi	bed	mass_closure_CO2	3.75114008529e-15

. . . and 39 more reference values in the case's `expected`.

batch22_wall_cooled

tutorials/batch/adsorber/batch22_wall_cooled

What it does: T2 wall-cooled CO2/He breakthrough (h=5 W/m2K, T_wall=298.15 K): the bed BETWEEN its limits – t_50 bracketed by the adiabatic 806 s and isothermal 3036 s, Q_wall as a state row of the same ODE, wallHeat record read from the state

The story: THE T2 WITNESS: the bed between its two limits.

batch20's adiabatic bed with 'energyBalance wallCooled;' – the energy row gains -h a_w (T_j - T_wall) and the bed sits BETWEEN the limits it must contain:

h → 0 batch20 (adiabatic: t_50 = 806 s, T_max ~ 336 K) h → infinity batch18 (isothermal: t_50 = 3036 s, T = T_wall)

so the golden pins the BRACKETING:

t_50(adiabatic) < t_50(wallCooled) < t_50(isothermal) T_wall < T_max(wallCooled) < T_max(adiabatic)

at h = 5 W/(m2 K), T_wall = 298.15 K, dBed 0.113 m DECLARED (never derived from the cross-section by a silent circle assumption); the engine announces a_w = 4/dBed and the wall NTU before integrating.

The LEDGER is the design point (docs/design/fixed-bed-thermal-a5.md section 6): the removed heat integrates as ONE MORE STATE ROW of the same ODE – the M_in/M_out ledger-row pattern – so the 'wallHeat' energy record READS a state (E = -Q_wall, T_service = T_wall) and the campaign closure dH_vessels = Q_ledger + H_external holds with every term a state function. Guards: 'adiabatic' + a wallHeatTransfer block refuses (contradiction), 'wallCooled' without one refuses, h <= 0 refuses (adiabatic is the way to SAY zero) – gate check_thermal_bed.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	E_wallHeat_seg0_kJ	-184.059495315
kpi	bed	E_wallHeat_total_kJ	-184.059495315
kpi	bed	P_final	100048.41634
kpi	bed	P_first_cell_Pa	100094.212734
kpi	bed	P_last_cell_Pa	100001.948175
kpi	bed	P_mid_cell_Pa	100048.658554
kpi	bed	P_out_boundary_Pa	100000
kpi	bed	Pe	250
kpi	bed	Q_wall_removed_kJ	184.059495315
kpi	bed	T_final	306.055276997
kpi	bed	T_max_final_K	312.299755078
kpi	bed	T_out_final_K	312.299755078
kpi	bed	c_out_over_cin_final_CO2	0.890450653484
kpi	bed	carrier_fabricated_mol	2.41868747253e-11

. . . and 44 more reference values in the case's `expected`.

`batch23_tsa_cycles`

`tutorials/batch/adsorber/batch23_tsa_cycles`

bed
fixedBedAdsorber

What it does: Toward A6: three hand-declared TSA cycles (cold load / 400 K purge) in one campaign – eleven ledgered feed amendments, balances closed, final KPIs at the cycle boundary; the CSS verdict machinery stays a named proposal

The story: toward A6: three FULL TSA cycles, hand-declared.

batch21's half-cycle repeated three times in ONE campaign, with the recipe grammar that already exists – nothing new in the engine:

cycle k (k = 0,1,2), each 7200 s: t = 7200k feed.T = 298.15 K (cold again) [k>0] feed.CO2 = 0.15 (feed returns) [k>0] t = 7200k + 3600 feed.CO2 = 0 (clean purge) feed.T = 400 K (hot purge)

ending EXACTLY at the third cycle boundary (21600 s), so the FINAL KPIs are end-of-cycle values. Every switch re-declares the feed commitment through the T1.5 ledger machinery (the T switches retire and redeclare the WHOLE commitment; the composition switches amend by delta) – FIFTEEN feedAmendment records (three per switching instant: the composition delta + the T retire/declare pair), and the campaign mass/element/energy balances still close on records, never jumps.

The A6 QUESTION this case makes visible (and does not yet verdict): does the bed state at successive cycle ends approach a fixed point – the cyclic steady state? MEASURED (this case's own run): qbar_CO2 at the three cycle ends is 0.3151 → 0.2415 → 0.1748 mol/kg – a slow geometric approach (ratio ~0.9; each purge leaves residual heat, the next load starts warmer and holds less), NOT yet converged in three cycles. That unfinished convergence IS the pedagogical point: eyes on a trajectory can see it, and only the A6 verdict machinery could CLAIM it. The golden pins the third-cycle end. A first-class CSS verdict (declared cycles, per- cycle state snapshots, an invariance tolerance the case declares) is an ARCHITECTURE feature – proposed in docs/design/fixed-bed-thermal-a5.md section 7, awaiting Vitor's decision; this case is deliberately its cheap, grammar-free witness.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`bed`	`P_final`	`100030.094422`
`kpi`	`bed`	`P_first_cell_Pa`	`100060.3912`
`kpi`	`bed`	`P_last_cell_Pa`	`100001.192563`
`kpi`	`bed`	`P_mid_cell_Pa`	`100030.206567`
`kpi`	`bed`	`P_out_boundary_Pa`	`100000`
`kpi`	`bed`	`Pe`	`250`
`kpi`	`bed`	`T_final`	`329.119724972`
`kpi`	`bed`	`T_max_final_K`	`399.878003769`
`kpi`	`bed`	`T_out_final_K`	`315.723301673`
`kpi`	`bed`	`carrier_fabricated_mol`	`9.60653778748e-12`
`kpi`	`bed`	`dT_adiabatic_CO2_K`	`140.405462904`
`kpi`	`bed`	`deltaP_first_to_out_Pa`	`60.3911997653`
`kpi`	`bed`	`integral_anchor_CO2`	`1312.99822496`
`kpi`	`bed`	`mass_closure_CO2`	`6.54546762708e-14`

. . . and 43 more reference values in the case's `expected`.

batch24_blowdown_inert

tutorials/batch/adsorber/batch24_blowdown_inert

What it does: P-swing witness A: inert isothermal blowdown, n/n0 = P/P0

The story: P-SWING WITNESS A: the inventory ratio.

The MATERIAL half of a blowdown, isolated from every thermal effect so it cannot pass by coincidence (design note fixed-bed-thermal-a5.md, section 8/9 amended battery, second-opinion review 2026-08-03):

ideal gas, INERT bed (kLDF = 0: the solid is frozen, pure transport), pure He; ISOTHERMAL ergun mode – T is a DECLARED constant of this teaching slice, so the energy question is out of frame BY DECLARATION (the thermal half is witness B, batch25); at t = 200 s the recipe closes the feed valve (P2, ‘setParameter u = 0‘ – the whole remaining feed commitment retires as a ledgered feedAmendment record) and drops the outlet boundary (P1, ‘setParameter P_out‘ 1.0 → 0.5 bar); the bed drains through the SAME Ergun face machinery every run already uses; the vented gas rides the telescopic M_out ledger rows.

THE ANCHORS (ideal-gas exactness, hand-computable):

n_end / n_0 = P_end / P_0 = 0.5 n(P) = eps A L P / (R T) = 0.0807 mol at 1 bar, 0.0403 at 0.5 bar vented = inventory drop (the mass-closure KPI is the proof)

The gate (check_pswing) recomputes both analytically and pins the ‘gasInventory_mol‘ KPI + the closure against them.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	E_adsorption_seg0_kJ	0
kpi	bed	E_adsorption_total_kJ	0
kpi	bed	P_final	49999.9998988
kpi	bed	P_first_cell_Pa	49999.9998996
kpi	bed	P_last_cell_Pa	49999.9998972
kpi	bed	P_mid_cell_Pa	49999.999899
kpi	bed	P_out_boundary_Pa	50000
kpi	bed	Pe	0
kpi	bed	T_final	298.15
kpi	bed	carrier_fabricated_mol	-8.881784197e-16
kpi	bed	deltaP_first_to_out_Pa	-0.000100418335933
kpi	bed	gasInventory_mol	0.0403395454642
kpi	bed	mass_closure_CO2	0
kpi	bed	n_CO2_final	0

. . . and 30 more reference values in the case’s `expected`.

batch24_tsa_until_css

tutorials/batch/adsorber/batch24_tsa_until_css

bed
fixedBedAdsorber

What it does: Toward A6: three hand-declared TSA cycles (cold load / 400 K purge) in one campaign – eleven ledgered feed amendments, balances closed, final KPIs at the cycle boundary; the CSS verdict machinery stays a named proposal

The story: A6 item 4: the cyclic steady state as the STOPPING CONDITION, not a count.

batch23 beside this one declares ‘repeat 3;‘ and reports honestly that three cycles did not settle – it makes the question visible and does not answer it. This case asks what an engineer actually wants to ask of a TSA bed: not "run three cycles" but "cycle until it repeats, and tell me how many that took".

cycle { period 7200; repeat untilCSS; cssTolerance 3e-2; maxCycles 6; ... }

The declared cap is SIX and the run uses TWO, announcing why it stopped.

TWO KEYS ARE MANDATORY under ‘untilCSS‘ and the engine refuses without either: the tolerance, because whether a bed has converged is a modelling judgement the engine must never invent; and the cap, because a bed that never settles has to stop somewhere and the run must be able to say it stopped for want of cycles rather than because it arrived. Reaching the cap is NOT an error – the trajectory is a physically valid campaign and every other KPI in it is real – but it means the case's own question went unanswered, so it is published (‘css_stopped_at_cap‘) and announced.

WHAT BUILDING IT FOUND, and it outlives the feature: the CSS norm and the loading are NOT the same quantity. See the measured numbers beside ‘cssTolerance‘ below. A tolerance chosen without looking can certify a bed whose loading is still visibly marching, because the norm over the whole packed state is dominated by its largest components. The verdict is only as meaningful as the number the case declares – which is the argument for the engine refusing to supply one.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	P_final	100029.553817
kpi	bed	P_first_cell_Pa	100059.686721
kpi	bed	P_last_cell_Pa	100001.177433
kpi	bed	P_mid_cell_Pa	100029.506183
kpi	bed	P_out_boundary_Pa	100000
kpi	bed	Pe	250
kpi	bed	T_final	324.614533765
kpi	bed	T_max_final_K	399.878004394
kpi	bed	T_min_final_K	297.926151955
kpi	bed	T_out_final_K	310.618231369
kpi	bed	carrier_fabricated_mol	1.35003119794e-11
kpi	bed	dT_adiabatic_CO2_K	140.405462904
kpi	bed	deltaP_first_to_out_Pa	59.6867210897
kpi	bed	gasInventory_mol	0.0747102437107

... and 45 more reference values in the case's `expected`.

batch25_blowdown_thermal

tutorials/batch/adsorber/batch25_blowdown_thermal

bed
fixedBedAdsorber

What it does: P-swing witness B: adiabatic blowdown COOLS (expansion work)

The story: P-SWING WITNESS B: the cooling.

The THERMAL half of a blowdown, isolated from adsorption so the only heat effect in frame is the expansion work itself (design note fixed-bed-thermal-a5.md, section 8/9 amended battery):

same inert pure-He ergun bed as witness A (batch24), but ‘energyBalance adiabatic;‘ – the section-9 general energy row (cv accumulation + the explicit eps R T dc_tot/dt source) is live; at t = 200 s the recipe closes the feed valve (P2) and halves the outlet boundary (P1) – the bed drains, dc_tot/dt < 0, and the expansion source pulls T DOWN; the solid heat capacity is a declared SYNTHETIC LOW value (9.2 J/(kg K), scope says so): the real ~920 of a zeolite pellet would thermally bury the gas-phase cooling under the solid’s mass (dT ~ -0.03 K); at the teaching value the drop is visible (~ -3 K class). Same honest-teaching-value discipline as the hxFluid of the pinch witness – the physics on display is the GAS’s, and the header says which knob made it visible.

THE ANCHOR: T_min_final_K sits BELOW the declared 298.15 K by a visible margin – a depressurising cell COOLS, which is exactly what the pre-widening cp-form could not show (the sabotage of the gate restores that form and this witness fails). The material anchors stay witness A’s business; this case pins the sign and visibility of the thermal effect, and the campaign energy claim stays closed.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bed	P_final	49999.9997347
kpi	bed	P_first_cell_Pa	49999.9996936
kpi	bed	P_last_cell_Pa	49999.999898
kpi	bed	P_mid_cell_Pa	49999.9997123
kpi	bed	P_out_boundary_Pa	50000
kpi	bed	Pe	0
kpi	bed	T_final	294.923818642
kpi	bed	T_max_final_K	295.066878523
kpi	bed	T_min_final_K	294.887442943
kpi	bed	T_out_final_K	295.066878523
kpi	bed	carrier_fabricated_mol	1.15463194561e-14
kpi	bed	deltaP_first_to_out_Pa	-0.00030635814619
kpi	bed	gasInventory_mol	0.0407808219207
kpi	bed	mass_closure_CO2	0

... and 32 more reference values in the case’s `expected`.

combustion

The theory you need. Radical chain chemistry: initiation creates the first radicals (slow – $H_2 + M$, $E_a \sim 105\,\text{kcal}$), branching multiplies them ($H + O_2 \to O + OH$: one in, two out), termination removes them. Ignition is the race between branching and termination; the induction time is set by the SLOW build-up, then heat release runs away. Rate constants $k = AT^b e^{-E_a/RT}$ come VERBATIM from cited mechanisms (GRI-Mech 3.0, Ó Conaire 2004, Glarborg 2018) in their native units – the engine converts and announces. Reverse rates from equilibrium: the kinetic end-state must land on the Gibbs equilibrium (the two-engine handshake the NOx tutorial pins at 98.9 %). Expect stiffness ratios of 10^6–10^8 (use Rosenbrock23), ignition delays in μs validated against shock-tube data, and thermal NO forming over SECONDS – the timescale gap behind quench-based NOx control. *Full derivation: Theory Guide, the combustion/kinetics chapter.*

ignition01_h2o2

tutorials/batch/combustion/ignition01_h2o2

bomb
batchReactor

What it does: Adiabatic constant-V H2/O2 ignition at 1100 K (GRI-Mech 3.0 H/O kinetics, curated thermo); Rosenbrock23 stiff solver; reports ignition delay

The story: closed rigid vessel, stoichiometric H2/O2 in N2 at 1100 K.

H2 : O2 : N2 = 2 : 1 : 4 (stoichiometric H2 + 1/2 O2 → H2O, 57% N2)

mode adiabatic + energy gasConstantV: the first law dU = 0 in a fixed volume, with the heat release coming from the species FORMATION enthalpies (curated thermo) – no per-reaction ΔH is supplied. Temperature, and the radicals H/O/OH/HO2, are integrated by the stiff solver until branching runs away.

solver { integrator Rosenbrock23; } – the L-stable stiff method. Swap to RK4 (see the ignition01_h2o2_rk4 sibling) and the same step blows up.

A closed rigid vessel holds a stoichiometric H_2/O_2 charge diluted in N_2 at 1100 K and ~5 bar. A tiny H-radical seed starts chain branching; after a short induction the radical pool explodes and the mixture ignites — a sharp temperature jump from 1100 K to ~2440 K.

and plot trajectory.csv (T and the radicals H/O/OH vs time). You will see:

The ignition delay τ_ign (the time of maximum dT/dt) is ≈ 0.9 ms here.

The radical lifetimes are ~10??–10?? s while the induction runs over ~10?^{3} s — a timescale separation of ~10?. That is stiffness. An explicit method (RK4) is stability-limited to a step of order the ∗fastest∗ scale, so it cannot take the ~10?? s steps this case uses. The solver here is Rosenbrock23, an L-stable linearly-implicit method that steps by ∗accuracy∗, not stability; at verbosity 3 it prints its stiffness verdict and step counts (the no-silent- crutch credo).

See it fail on purpose — the sibling case runs the same problem with explicit RK4 at the same step:

It blows up to NaN within a few steps. (That case ships a .expect-nonconvergence marker, so bin/runTests skips it — its whole point is to fail.) Switch the integrator back to Rosenbrock23 and the same step converges. ∗See, then decide∗, applied to numerics.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bomb	P_final	500000
kpi	bomb	T_final	2641.48171412
kpi	bomb	n_H2O2_final	2.59406092875e-11
kpi	bomb	n_H2_final	2.44308154218e-06
kpi	bomb	n_HO2_final	1.97492765295e-10
kpi	bomb	n_H_final	1.4513832323e-06
kpi	bomb	n_N2_final	2.857145e-05
kpi	bomb	n_O2_final	1.02931747994e-06
kpi	bomb	n_OH_final	1.08725431027e-06
kpi	bomb	n_O_final	5.68688863565e-07
kpi	bomb	n_water_final	1.05706749995e-05
kpi	bomb	t_end	0.0002
kpi	bomb	totalMoles_final	4.57220738612e-05
kpi	campaign	element_H_closure_rel	6.76051276934e-15

. . . and 17 more reference values in the case's `expected`*.*

ignition01_h2o2_eulersi

tutorials/batch/combustion/ignition01_h2o2_eulersi

bomb
batchReactor

What it does: WITNESS + EXPECTED FAILURE: semi-implicit Euler on the H2/O2 ignition – A-stable but first-order, goes unphysical at this step and the engine refuses by name (see .expect-nonconvergence).

The story: WITNESS for the EulerSI integrator (semi-implicit Euler) – an EXPECTED FAILURE: see .expect-nonconvergence for why first order at this step goes unphysical and how the engine now refuses instead of exiting 0. Registered model no case selected until 2026-08-22 (invariant I8).

A closed rigid vessel holds a stoichiometric H_2/O_2 charge diluted in N_2 at 1100 K and ~5 bar. A tiny H-radical seed starts chain branching; after a short induction the radical pool explodes and the mixture ignites — a sharp temperature jump from 1100 K to ~2440 K.

and plot trajectory.csv (T and the radicals H/O/OH vs time). You will see:

The ignition delay τ__ign (the time of maximum dT/dt) is ≈ 0.9 ms here.

The radical lifetimes are ~10??–10?? s while the induction runs over ~10?^{3} s — a timescale separation of ~10?. That is stiffness. An explicit method (RK4) is stability-limited to a step of order the ∗fastest∗ scale, so it cannot take the ~10?? s steps this case uses. The solver here is Rosenbrock23, an L-stable linearly-implicit method that steps by ∗accuracy∗, not stability; at verbosity 3 it prints its stiffness verdict and step counts (the no-silent- crutch credo).

See it fail on purpose — the sibling case runs the same problem with explicit RK4 at the same step:

It blows up to NaN within a few steps. (That case ships a .expect-nonconvergence marker, so bin/runTests skips it — its whole point is to fail.) Switch the integrator back to Rosenbrock23 and the same step converges. ∗See, then decide∗, applied to numerics.

ignition01_h2o2_rk4

tutorials/batch/combustion/ignition01_h2o2_rk4

bomb
batchReactor

What it does: Adiabatic constant-V H2/O2 ignition at 1100 K (GRI-Mech 3.0 H/O kinetics, curated thermo); EXPLICIT RK4 – fails on the stiff chemistry (the lesson)

The story: ignition01_h2o2 – closed rigid vessel, stoichiometric H2/O2 in N2 at 1100 K.

H2 : O2 : N2 = 2 : 1 : 4 (stoichiometric H2 + 1/2 O2 → H2O, 57% N2)

mode adiabatic + energy gasConstantV: the first law dU = 0 in a fixed volume, with the heat release coming from the species FORMATION enthalpies (curated thermo) – no per-reaction ΔH is supplied. Temperature, and the radicals H/O/OH/HO2, are integrated by the stiff solver until branching runs away.

solver { integrator Rosenbrock23; } – the L-stable stiff method. Swap to RK4 (see the ignition01_h2o2_rk4 sibling) and the same step blows up.

This is the failure half of the stiffness lesson. It runs the exact same adiabatic constant-volume H_2/O_2 ignition as the sibling case [ignition01_h2o2](../ignition01_h2o2/README.md), but with the explicit RK4 integrator instead of the stiff Rosenbrock23 — at the same fixed step (deltaT = 1e-5 s).

trajectory.csv is a column of nan from the very first output step (t = 1e-4 s) onward — there is no clean induction-then-jump, just immediate divergence:

*That immediate blow-up is the result. The radical lifetimes (~10??–10?? s) are six orders of magnitude shorter than the induction timescale (~10?^{3} s) — the chemistry is stiff. An explicit method like RK4 is stability-limited to a step of order the *fastest* scale, so the ~10?? s step this case uses is far too large: the integration is murdered by stability long before accuracy matters.*

*Compare side-by-side with the stiff sibling: switch to (or open) ignition01_h2o2, whose L-stable adaptive sub-stepping integrates the *same* chemistry to a clean burned state (~2440 K). *See, then decide*, applied to numerics.*

This case ships a .expect-nonconvergence marker, so bin/runTests skips it — its whole point is to fail, and it would otherwise trip the NaN guard.

Same as the sibling: the thermodynamics is real curated Choupo data, but the reaction rates are synthetic (hand-chosen Arrhenius constants reproducing the chain-branching structure, not a published mechanism). Do not quote any number from this case as quantitative.

`ignition02_h2air_slack`

`tutorials/batch/combustion/ignition02_h2air_slack`

bomb
batchReactor

What it does: Stoich H2/air ignition, 1100 K / 2 atm (Slack shock tube; O Conaire 2004 kinetics)

The story: / flowsheetDict Stoichiometric H2/air autoignition at 1100 K / 2 atm against the Slack shock-tube point (O Conaire 2004 kinetics): the radical pool (H, O, OH, HO2) builds exponentially through the induction time, then runaway. A stiff-integrator showcase – compare the ignition delay with the measured shock-tube value.

Stoichiometric H_2/air, 1100 K, 2 atm, constant volume — the reflected-shock condition of Slack (1977) as compiled in ? Conaire et al. 2004 (Fig 8). Kinetics: the ? Conaire H_2/O_2 mechanism, Table I verbatim (19 reactions; two Troe falloffs; two summed pairs; A in cm^3-mol-s, Ea in kcal — the engine converts and announces every one).

Result: τ_ign (max dT/dt) = 43 µs vs the measured ~40–60 µs window.

Two lessons: 1. No seed. This mechanism self-initiates (R5: H_2 + M = 2H, Ea = 105 kcal/mol — slow, but that slowness IS the induction chemistry). With the ignition01-style trace-H seed the delay collapses to 12 µs: initial conditions are physics, not knobs. 2. The engine now prints tau_ign in the epilogue (time of max dT/dt on the write grid) whenever a vessel ignites — no CSV spelunking.

Run ignition01_h2o2 (GRI kinetics, 5 bar) for the sibling case and the stiffness lesson (_rk4).

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	bomb	P_final	202650
kpi	bomb	T_final	3041.74576271
kpi	bomb	n_H2O2_final	3.94467753319e-11
kpi	bomb	n_H2_final	9.82828113933e-07
kpi	bomb	n_HO2_final	4.27531541611e-10
kpi	bomb	n_H_final	3.53516153966e-07
kpi	bomb	n_N2_final	1.391515e-05
kpi	bomb	n_O2_final	3.90750822458e-07
kpi	bomb	n_OH_final	4.44937008544e-07
kpi	bomb	n_O_final	1.5493529764e-07
kpi	bomb	n_water_final	6.00759209227e-06
kpi	bomb	t_end	0.0003
kpi	bomb	totalMoles_final	2.22501764671e-05
kpi	campaign	element_H_closure_rel	3.43861059252e-15

. . . and 17 more reference values in the case's `expected`*.*

`nox01_zeldovich_postflame`

`tutorials/batch/combustion/nox01_zeldovich_postflame`

postflame
batchReactor

What it does: Thermal NOx in post-flame H2/air gas, 2125 K / 1 atm (Glarborg 2018 N chemistry)

The story: / flowsheetDict Thermal NOx in post-flame H2/air gas at 2125 K / 1 atm: the Zeldovich N-chemistry (Glarborg 2018 rate set) integrated in a batch reactor – NO grows on the SECONDS scale while the O/H radical pool stays equilibrated, the classic separation of scales that makes thermal NOx a post-flame problem.

Hot burnt gas (lean H_2/air, 2125 K, 1 atm — Homer & Sutton Flame 1 as compiled in Glarborg 2018) held isothermal, starting from the equilibrated H/O radical pool with all nitrogen still as N_2. The Glarborg 2018 nitrogen chemistry (extended Zeldovich + NNH + N_2O routes, Tables 2/5 verbatim) then forms NO.

1. The cross-engine pin. The kinetic plateau lands on the Gibbs equilibrium computed by a ∗different engine∗ (steady Gibbs minimisation vs stiff batch integration). Kinetics choose the path and the clock; the destination belongs to the thermodynamics. Reverse rates come from the same equilibrium constants — so this agreement is a genuine consistency check of the mechanism transcription + thermo, not a tautology. 2. NOx forms in engineering time. Ignition takes microseconds (ignition02: 43 μs); thermal NO takes seconds at 2125 K. That timescale gap is the whole basis of NOx control by quench: cool the gas before the Zeldovich clock runs.

The burnt-gas initial pool was computed with this same thermo by a 14-species gibbsReactor at 2125 K/1 atm (25 % H_2 / 17.5 % O_2 / 57.5 % N_2 feed); the equilibrium NO share was returned to N_2/O_2 elementally.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	postflame	P_final	101325
kpi	postflame	T_final	2125
kpi	postflame	n_H2O2_final	1.05023679159e-12
kpi	postflame	n_H2_final	4.85355503797e-09
kpi	postflame	n_HO2_final	1.61492364784e-11
kpi	postflame	n_H_final	6.02372866884e-10
kpi	postflame	n_N2O_final	1.37258448136e-12
kpi	postflame	n_N2_final	3.72505182272e-06
kpi	postflame	n_NH_final	2.50521342942e-15
kpi	postflame	n_NNH_final	1.01561106781e-15
kpi	postflame	n_NO_final	2.94524618017e-08
kpi	postflame	n_N_final	2.26160617192e-14
kpi	postflame	n_O2_final	3.06851934926e-07
kpi	postflame	n_OH_final	2.0389175016e-08

. . . and 24 more reference values in the case's `expected`.

crystallisation

The theory you need. Crystallisation removes solute when the solution crosses saturation: $m > m_{\mathrm{sat}}(T)$, with $K_{\mathrm{sp}}(T)$ from ΔG_{diss} and its T-dependence from ΔH_{diss} (van 't Hoff). The heat of crystallisation is ION-DERIVED – $\Delta H_{\mathrm{cryst}} = -(\Delta H^{\infty}_{\mathrm{soln}} + \bar{L}_2(m_{\mathrm{sat}}))$ – never a stored constant (a solid's formation enthalpy is $\sum \nu_i h^{\mathrm{aq}}_{f,i} - \Delta H_{\mathrm{soln}}$, derived, single-sourced). Antisolvent cases add the mixed-solvent electrolyte model (eNRTL): ethanol drops the dielectric constant, $\gamma_{\pm}$ rises, salt crashes out. Expect supersaturation as the driving force and yields set by the solubility curve, not the kinetics (these cases are equilibrium-staged). *Full derivation: Theory Guide, the crystallisation + electrolyte-enthalpy chapters.*

batch13_nacl_pitzer

tutorials/batch/crystallisation/batch13_nacl_pitzer

What it does: Batch NaCl crystalliser: saturation from the Pitzer ion activity, not a solubility curve

The story: One closed vessel. The unit declares no saturation of its own: it asks the thermodynamics the case declares, and the case declares Pitzer.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_Cl_closure_rel	9.08953975799e-16
kpi	campaign	element_H_closure_rel	0
kpi	campaign	element_Na_closure_rel	9.08953975799e-16
kpi	campaign	element_O_closure_rel	0
kpi	campaign	element_worst_closure_rel	9.08953975799e-16
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	1
kpi	campaign	mass_closure_rel	3.49878229369e-16
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	1299.73034249
kpi	campaign	mass_kg_initial	1299.73034249
kpi	campaign	mass_residual_kg	4.54747350886e-13
kpi	campaign	moles_kmol_external_out	0

. . . and 17 more reference values in the case's `expected`.

batch14_nacl_pitzer_antisolvent

tutorials/batch/crystallisation/batch14_nacl_pitzer_antisolvent

cryst
batchCrystalliser

What it does: Batch NaCl crystalliser with ethanol present: a pairwise-aqueous model announces the mixed-solvent term it does not carry

The story: One closed vessel, identical to batch13 except for the ethanol in the charge. The unit declares no saturation of its own: it asks the thermodynamics the case declares, and the case declares Pitzer – which has nothing to say about a second solvent, and says so.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	0
kpi	campaign	element_Cl_closure_rel	6.05235197043e-16
kpi	campaign	element_H_closure_rel	0
kpi	campaign	element_Na_closure_rel	6.05235197043e-16
kpi	campaign	element_O_closure_rel	0
kpi	campaign	element_worst_closure_rel	6.05235197043e-16
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	1
kpi	campaign	mass_closure_rel	1.3612844745e-16
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	1670.28772973
kpi	campaign	mass_kg_initial	1670.28772973
kpi	campaign	mass_residual_kg	2.27373675443e-13

. . . and 19 more reference values in the case's `expected`.

drying

`dryer01_sucrose_tray`

`tutorials/batch/drying/dryer01_sucrose_tray`

dryer
batchDryer

What it does: Tray drying of wet sucrose into declared 60 C air: the classical constant-rate / falling-rate drying curve, with the wet bulb and the GAB equilibrium moisture as results

The story: the DRYING CURVE, both periods, in one batch.

A tray holding 2.0 kg of dry sucrose wet to X_0 = 0.30 kg water / kg dry solid, exposed over 0.5 m2 to air of DECLARED, CONSTANT condition: 60 C (333.15 K) and Y = 0.010 kg water / kg dry N2. Nothing about that air is integrated – it is an ENVIRONMENT, the FixedBedAdsorber's constant-carrier posture, and the water that leaves the solid leaves the campaign across that boundary (reported as the unit's DECLARED material residual, never as a silent leak).

THE HAND ESTIMATE, computed from this case's own curated records (water Antoine + Watson Hvap + the two ideal-gas Cp polynomials, the case-local GAB isotherm on sucrose) – the run must reproduce it:

air water activity p_v = P Y/(Y + Mv/Mc) = 1551 Pa a_w = p_v / Psat(333.15 K) = 0.0774 GAB equilibrium K a_w = 0.0743 X_eq = Xm C Ka /((1-Ka)(1-Ka+C Ka)) = 0.01580 kg/kg wet bulb (Lewis = 1) T_wb = 300.53 K (27.4 C, a 32.6 K depression) driving force Y_sat(T_wb) - Y = 0.014011 kg/kg constant-rate flux R_c = k_Y dY = 0.05 ∗ 0.014011 = 7.0057e-4 kg/(m2 s) R_c A = 3.5028e-4 kg/s latent load R_c A lambda(T_wb) = 0.8645 kW (the AIR's, not the campaign's – KPI only, never ledgered) break time t_c = m_s (X_0 - X_c)/(R_c A) = 1027.7 s falling-rate constant tau = m_s (X_c - X_eq)/(R_c A) = 595.0 s end of run X(3000 s) = X_eq + (X_c - X_eq) exp(-(3000-t_c)/tau) = 0.019584 kg/kg

What to look at in the output: trajectory.csv columns dryer.X_kg_kg and dryer.R_kg_m2_s: the drying curve and the rate curve, engine columns, not a post-processing. R is FLAT at 7.0057e-4 until X falls through X_c = 0.12, then decays linearly in (X - X_eq) – the textbook break, visible as a corner. the KPI t_critical is OBSERVED (the interpolated crossing between two accepted states), not the analytic 1027.7 s above; they agree because dX/dt is exactly constant on that leg. THE CAMPAIGN MASS BALANCE DOES NOT CLOSE, AND IT SHOULD NOT. Look at it, because it is the first thing an examiner will point at: it reports 2.600000 kg in against 2.039168 kg out, a 21.6 % shortfall. That shortfall is the DRIED WATER. It left the solid and went into the air – and the air here is a DECLARED ENVIRONMENT (a temperature and a humidity), not a stream this campaign owns, so there is no vessel on the books for the water to arrive in. The vessel's inventory genuinely shrinks.

What makes that a stated model boundary rather than a leak is that the unit PREDICTS the gap independently, from its own evaporation history, and the campaign checks the two against each other: the observed shortfall matches the declared prediction to 7e-17 (element_worst_closure_vs_declared_rel). The raw numbers are never discounted – 21.6 % is what the balance says and what the balance reports. If this dryer ever lost matter for a REAL reason, that cross-check is what would move, and it is pinned in 'expected'.

So the honest sentence to give at a viva is: "the balance is open at the air boundary by construction, the opening is the 0.5608 kg of water evaporated, and the model reproduces that number from the other side." Closing it for real means making the drying air a ledgered stream, which this unit deliberately does not do.

the campaign energy balance is UNAVAILABLE, and says why in one sentence: the drying heat comes from air this campaign never owned. The number is not hidden – it is the latentDuty_kW KPI.

Method hypotheses, each announced at run time: Lewis = 1 in the adiabatic-saturation equation; the humid heat evaluated at the air's own temperature; the surface held at T_wb while X > X_c; the LINEAR falling-rate law (a modelling CHOICE, not physics); the solid's own temperature NOT integrated.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	0
kpi	campaign	element_H_closure_rel	0.319043490708
kpi	campaign	element_O_closure_rel	0.319043490708
kpi	campaign	element_worst_closure_rel	0.319043490708
kpi	campaign	element_worst_closure_vs_declared_rel	7.11118003994e-17
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	0
kpi	campaign	mass_closure_rel	0.215704585481
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	2.03916808853
kpi	campaign	mass_kg_initial	2.60000001375
kpi	campaign	mass_residual_kg	0.560831925217
kpi	campaign	moles_kmol_external_out	0
. . . and 21 more reference values in the case's `expected`.			

membrane

diafilter01_nf_desalting

`tutorials/batch/membrane/diafilter01_nf_desalting`

retentate
batchDiafilter

What it does: Constant-volume NF diafiltration: NaCl washed out of an MgSO4 retentate, with the OBSERVED rejection published beside the constant-R washout law it violates

The story: constant-volume diafiltration on a loose NF membrane, and the exponential washout law failing in a way you can watch.

THE OPERATION. 10 litres of a brine carrying TWO salts sit behind 5 m2 of NF270 at 10 bar. Solvent is made up as fast as it permeates, so the volume is held and the run is measured in DIAVOLUMES – how many vessel volumes of wash water have passed through. NaCl is poorly rejected by this membrane and leaves; MgSO4 is well rejected and stays. That is the whole point of a nanofiltration desalting step, and it is the same arithmetic whether the retained species is a divalent salt, a sugar or a protein.

THE LESSON. Derive the washout by hand. Over a constant volume V, with the permeate carrying c__p = (1 - R) c,

V dc/dt = -Q__p (1 - R) c , N = Q__p t / V => c/c__0 = exp(-(1 - R) N)

which is a straight line on a log axis against N. The derivation needs R to be CONSTANT. It is not:

as NaCl leaves, the osmotic pressure of the retentate falls, so the driving force rises and so does the water flux; a higher flux sweeps more solute to the wall, so the polarisation factor rises and the WALL concentration is no longer the bulk one; the observed rejection is measured against the bulk, so it moves even when the membrane's own permeability has not changed.

The run therefore publishes R__obs for each solute at every instant, beside the diavolumes, and – because this is constant-volume mode – two KPIs per solute: 'washoutActual_<name>' (what the vessel did) and 'washoutIdeal_<name>' (what exp(-(1-R)N) predicts using THIS run's own initial R). The idealisation is computed from the run, not declared, so the gap between them cannot have been arranged.

Read them together. Where the two agree, the hand derivation was safe. Where they part, the R__obs column says which assumption failed and when.

WHAT IS PINNED AND WHAT IS NOT. The golden pins the answer this model gives; NOTHING here is validated against a measured diafiltration. The NF270 permeabilities are the case-fitted pedagogical set the record declares ('provenance fittedToCase', fitted against membrane02__NF__sugar), so the rejections are representative of a loose NF membrane and are not a claim about a particular element. Fouling is NOT modelled – the flux in a real rig would decline with time and this one does not.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_Cl_closure_rel	0
kpi	campaign	element_H_closure_rel	1.23912409123e-16
kpi	campaign	element_Mg_closure_rel	0
kpi	campaign	element_Na_closure_rel	0
kpi	campaign	element_O_closure_rel	1.23884758239e-16
kpi	campaign	element_S_closure_rel	0
kpi	campaign	element_worst_closure_rel	1.23912409123e-16
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	0
kpi	campaign	mass_closure_rel	0
kpi	campaign	mass_kg_amend_in	54.5637624826
kpi	campaign	mass_kg_amend_out	0
kpi	campaign	mass_kg_external_out	54.5637624826

. . . and 34 more reference values in the case's `expected`*.*

diafilter02_fouling_decline

tutorials/batch/membrane/diafilter02_fouling_decline

retentate
batchDiafilter

What it does: The same diafiltration with a DECLARED Hermia cake-law decline: what fouling costs a washout schedule

The story: the same diafiltration, with the membrane losing permeance as it works.

WHY THIS CASE EXISTS. diafilter01's flux RISES during the run (233 → 256 LMH): the salt leaves, the osmotic pressure falls, the driving force grows. That is the physics the model has. It is not what a rig does. On real equipment the flux falls, because the membrane fouls, and every schedule a student will ever write is sized on that decline.

THE MODEL. Resistance in series on the permeance the unit hands the transport law:

1 / A_eff = 1 / A_w + r_f(v), v = V_permeated / A

with r_f growing by one of HERMIA'S BLOCKING LAWS (Hermia, Trans. IChemE 60:183-187, 1982). Here ‘cake‘: r_f = k v, the deposited-layer form. Applied where the transport CONTEXT is assembled, so solution-diffusion and DSPM-DE are served alike and neither is modified.

THE CONSTANT IN THIS CASE IS HYPOTHETICAL AND IS NOT A FITTED VALUE FOR THIS SYSTEM. ∗∗ A brine of NaCl and MgSO4 is fully dissolved; there is no suspended matter to deposit, so ‘cake‘ is not a mechanism this feed can honestly claim. What this case demonstrates is the CONSEQUENCE of a declared decline on a diafiltration schedule – how many more minutes the same washout costs – on a system whose other numbers are already pinned by diafilter01, so the difference between the two runs is the fouling and nothing else. It is a STRUCTURAL witness, in the same sense as edwards01_sour_water_activity: no number in it is validated against a measured fouling experiment, and the tree carries no fouling data at all.

A fouling constant belongs to a RIG AND A MATERIAL, not to a membrane, so it is declared here in the case and no catalogue record is touched.

WHAT TO READ. ‘A_eff_over_A_w‘ in the trajectory is the permeance the membrane still has, as a fraction of clean. Put diafilter01 and diafilter02 side by side: same charge, same area, same pressure, and the diavolume count at t = 160 s is the whole lesson.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_Cl_closure_rel	0
kpi	campaign	element_H_closure_rel	4.17214965497e-16
kpi	campaign	element_Mg_closure_rel	0
kpi	campaign	element_Na_closure_rel	0
kpi	campaign	element_O_closure_rel	4.17110477583e-16
kpi	campaign	element_S_closure_rel	0
kpi	campaign	element_worst_closure_rel	4.17214965497e-16
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	0
kpi	campaign	mass_closure_rel	3.70318765373e-16
kpi	campaign	mass_kg_amend_in	47.5260962641
kpi	campaign	mass_kg_amend_out	0
kpi	campaign	mass_kg_external_out	47.5260962641

. . . and 35 more reference values in the case's `expected`.

reactor

The theory you need. Three idealised steady reactors. Conversion reactor: stoichiometry only, $n_{\mathrm{out}} = n_{\mathrm{in}} + \nu\,\xi$, duty from the ONE enthalpy base $\Delta H_{\mathrm{rxn}}(T) = \sum_i \nu_i h_i(T)$ (formation datum – no separate "heat of reaction" input, and any dict override is cross-checked aloud). CSTR: outlet at reactor conditions with rate $r(c_{\mathrm{out}}, T)$, so $F\,(c_{\mathrm{in}} - c_{\mathrm{out}}) = r\,V$. PFR: the same rate integrated along the volume, $\mathrm{d}F_i/\mathrm{d}V = \nu_i\,r$. Arrhenius kinetics $k = A\,T^b e^{-E_a/RT}$, orders from the dict, reverse rates from the equilibrium constant (thermo-consistent by construction). Expect: CSTR conversion below PFR at equal volume (back-mixing), exponential T sensitivity, and equilibrium limits the kinetics can approach but never cross. *Full derivation: Theory Guide, the reaction-engineering chapter.*

batch01_first_order

tutorials/batch/reactor/batch01_first_order

reactor
batchReactor

What it does: Isothermal batch Fischer esterification (AcOH + EtOH <→ EtOAc + H2O) at 350 K, pseudo-first-order in ethanol; analytical exp-decay comparison

The story: isothermal pseudo-first-order batch reactor.

Single vessel, pseudo-first-order esterification (AcOH + EtOH → EtOAc + H2O, rate = k.[ethanol]). Initial charge is an equimolar ethanol + acetic acid mix; kinetics from the reactions library (Arrhenius A=1e8 1/s, Ea=70 kJ/mol). At T = 350 K:

k(T) = 1e8 · exp(-70000 / (8.314 · 350)) ≈ 3.57e-3 s^-1 τ = 1/k ≈ 280 s

Analytical solution for first-order in a constant-volume batch:

n_A(t) = n_A(0) · exp(-k t)

The simulator integrates dn_A/dt = -k · (n_A / V) · V = -k n_A with RK4 at dt = 1 s; final t = 600 s gives X_A ≈ 1 - exp(-k·600) ≈ 1 - exp(-2.14) ≈ 0.882

18.5 mol of an equimolar ethanol / acetic-acid charge (about a litre) is held at 350 K in a closed vessel for 600 s and the Fischer esterification runs — the kinetics of cstr01, now integrated in time by RK4 at 1 s steps. The golden: ethanol falls from 9.25 mol to 1.083 mol (X = 88.3 %), 8.167 mol of ethyl acetate made, and the jacket has removed 210.15 kJ to hold the temperature — exactly -25.73 kJ/mol of reaction, the elements-datum heat every reactor in Choupo prices on.

1. choupoBatch, not choupoSolve. A batch vessel has no steady state; the case declares endTime 600; deltaT 1.0; writeInterval 30; in controlDict and the engine integrates dn/dt = ν r V. The Log tab prints a snapshot every 30 s: four inventories and T. 2. First order means an exponential, and you can check it. With k = 3.575e-3 s?^{1} at 350 K (the same Arrhenius pair as cstr01), n(600 s) = 9.25·e^(-2.145) = 1.083 mol. The table is that curve, sampled. 3. The campaign ledger closes at machine precision. [campaign] mass balance ? closure 4.5e-16, energy balance: dH_vessels = -210.15 kJ ? = Q_ledger -210.15 kJ, closure 0 — every joule the jacket took is a state difference on the charge, never a quadrature, which is why the closure is zero and not merely small. 4. A CSTR at the same τ does worse. 600 s here gives 88 %; the continuous reactor at τ = 310 s gave 53 % and a PFR 67 %. Three reactors, one rate law, three answers — the batch one is the upper envelope, because nothing leaves early.

Set T_setpoint 370 K;: k roughly triples, the exponential steepens, and the ledger's Q grows with the extent. Then batch02_adiabatic removes the jacket, and T becomes one more state on the trajectory.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	3.75075346157e-16
kpi	campaign	element_H_closure_rel	3.00060276926e-16
kpi	campaign	element_O_closure_rel	2.50050230771e-16
kpi	campaign	element_worst_closure_rel	3.75075346157e-16
kpi	campaign	energy_H_external_kJ	0
kpi	campaign	energy_Q_ledger_kJ	-210.150638715
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_closure_rel	0
kpi	campaign	energy_dH_vessels_kJ	-210.150638715
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	1
kpi	campaign	energy_residual_kJ	0
kpi	campaign	mass_closure_rel	4.5240474843e-16
kpi	campaign	mass_kg_external_out	0

... and 25 more reference values in the case's `expected`.

batch01b_first_order_eulersi

tutorials/batch/reactor/batch01b_first_order_eulersi

reactor
batchReactor

What it does: GREEN WITNESS: semi-implicit Euler (EulerSI) on batch01 mild isothermal esterification – first order suffices off-stiffness; twin of the ignition01__h2o2__eulersi expected failure.

The story: GREEN WITNESS for the EulerSI integrator: batch01's mild isothermal esterification, integrated by semi-implicit (linearly-implicit) Euler instead of the RK4 default. On a non-stiff problem the first-order method simply works, and the golden pins that its answer agrees with the physics batch01 already pins. Its EXPECTED-FAILURE twin is ignition01__h2o2__eulersi (stiff nonlinear ignition, where first order goes unphysical and the engine refuses by name). Registered model no case selected until 2026-08-22 (invariant I8).

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	3.75075346157e-16
kpi	campaign	element_H_closure_rel	4.50090415389e-16
kpi	campaign	element_O_closure_rel	2.50050230771e-16
kpi	campaign	element_worst_closure_rel	4.50090415389e-16
kpi	campaign	energy_H_external_kJ	0
kpi	campaign	energy_Q_ledger_kJ	-210.043853155
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_closure_rel	0
kpi	campaign	energy_dH_vessels_kJ	-210.043853155
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	1
kpi	campaign	energy_residual_kJ	0
kpi	campaign	mass_closure_rel	5.65505935537e-16
kpi	campaign	mass_kg_external_out	0

. . . and 25 more reference values in the case's `expected`.

batch02_adiabatic

tutorials/batch/reactor/batch02_adiabatic

reactor
batchReactor

What it does: Adiabatic batch esterification (AcOH + EtOH → EtOAc + H2O), exothermic, starting cold (320 K); thermal runaway to ~428 K then plateau as ethanol is consumed

The story: exothermic pseudo-first-order esterification (AcOH + EtOH → EtOAc + H2O) in an adiabatic batch reactor. Designed to show thermal runaway:

T_0 = 320 K (kinetics very slow: k_0 ~ 3.7e-4 s^-1, tau_0 ~ 2700 s). dH_rxn = -26.06 kJ/mol of ethanol, computed from the species' standardThermochemistry (elements datum); lifts T ~108 K at full conversion (320 → 428 K), enough to show the runaway. Equimolar ethanol + acetic acid charge; ethyl acetate + water are the products. Mass-conserving, so the energy balance is clean.

Expected qualitative trajectory:

T | __________ plateau (~428 K) | __/ | __/ | __/ | __/ | / T_0 |___/ +——————→ t induction runaway

The energy balance is just:

dT/dt = - r * V * dH_rxn / (n_A * Cp_A + n_B * Cp_B)

Constant-Cp here (polynomial degree 0 for both ethanol and water in the database), so the result extrapolates linearly above the nominal liquidHeatCapacity Trange (280-373 K) without numerical pathology — consistent with a pedagogical demonstration, not a quantitative prediction at 428 K.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	reactor	P_final	101300
kpi	reactor	T_final	428.2212252
kpi	reactor	n_aceticAcid_final	9.87885051063e-31
kpi	reactor	n_ethanol_final	9.87885051063e-31
kpi	reactor	n_ethylAcetate_final	0.00925
kpi	reactor	n_water_final	0.00925
kpi	reactor	t_end	600
kpi	reactor	totalMoles_final	0.0185
kpi	campaign	element_C_closure_rel	0
kpi	campaign	element_H_closure_rel	0
kpi	campaign	element_O_closure_rel	0
kpi	campaign	element_worst_closure_rel	0
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0

... and 12 more reference values in the case's `expected`.

batch03_consecutive

`tutorials/batch/reactor/batch03_consecutive`

reactor
batchReactor

What it does: Isothermal batch reactor: consecutive A → B → C with k1 > k2; intermediate B peaks then decays

The story: two first-order reactions in series:

with k1 > k2. This is the textbook series-reaction setup: B is produced from A faster than it is consumed to C, so n_B grows at first, reaches a maximum, then decays as A runs out.

Analytical solution (constant V, isothermal, first-order):

n_A(t) = n_A0 * exp(-k1 t) n_B(t) = n_A0 * k1 / (k2 - k1) * [exp(-k1 t) - exp(-k2 t)] n_C(t) = n_A0 - n_A(t) - n_B(t)

t_max = ln(k1 / k2) / (k1 - k2)

With k1 = 0.01 1/s and k2 = 0.002 1/s:

t_max = ln(5) / 0.008 ~ 201 s n_B_max / n_A0 = (k1/k2)^(k2/(k2-k1)) = 5^(-0.25) ~ 0.669

The case demonstrates: 1. multi-reaction support in BatchReactor (v0.21); 2. time-dependent selectivity — harvesting B at t ~ 200 s maximises yield, waiting longer wastes B to C.

No species supplies activation energy: both A_pre are quoted with Ea = 0 so that the rate constants are exactly the values above regardless of T (the case is isothermal anyway).

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	1
kpi	campaign	energy_records_valid	0
kpi	campaign	mass_closure_rel	5.55111512313e-16
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	0.5
kpi	campaign	mass_kg_initial	0.5
kpi	campaign	mass_residual_kg	2.77555756156e-16
kpi	campaign	transfers_H_invalid	0
kpi	campaign	transfers_H_valid	0
kpi	campaign	transfers_logged	0
kpi	reactor	P_final	101300
kpi	reactor	T_final	350
kpi	reactor	n_compA_final	3.05902320887e-09

... and 7 more reference values in the case's `expected`.

batch04_reversible_wgs

`tutorials/batch/reactor/batch04_reversible_wgs`

What it does: Batch reversible water-gas shift ($CO + H_2O <\rightarrow CO_2 + H_2$) at 600 K; conversion relaxes to X_eq

The story: the reversible water-gas shift in a closed BATCH reactor (choupoBatch), the time-domain sibling of cstr02 / pfr02.

Equimolar $CO + H_2O$ charged at 600 K. The forward Arrhenius rate drives conversion up; the reverse term (k_rev = k_fwd / K_eq, K_eq from the four species' g_pure_ig, re-evaluated each step) brakes it. The batch trajectory relaxes onto the SAME thermodynamic ceiling the CSTR and PFR reached — X → X_eq ~ 0.84 — then sits there (net rate → 0), instead of running to X = 1.

The forward A is the illustrative value tuned for stable RK4 (see constant/reactions); the equilibrium X_eq is A-independent.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	reactor	P_final	1000000
kpi	reactor	T_final	600
kpi	reactor	n_CO2_final	0.000420715762123
kpi	reactor	n_CO_final	7.92842378768e-05
kpi	reactor	n_H2_final	0.000420715762123
kpi	reactor	n_water_final	7.92842378768e-05
kpi	reactor	t_end	1200
kpi	reactor	totalMoles_final	0.001
kpi	campaign	element_C_closure_rel	1.95156391047e-15
kpi	campaign	element_H_closure_rel	1.95156391047e-15
kpi	campaign	element_O_closure_rel	1.95156391047e-15
kpi	campaign	element_worst_closure_rel	1.95156391047e-15
kpi	campaign	energy_H_external_kJ	0
kpi	campaign	energy_Q_ledger_kJ	-0.299204643882

. . . and 25 more reference values in the case's `expected`.

batch04_transient_dirs

`tutorials/batch/reactor/batch04_transient_dirs`

reactor
batchReactor

What it does: Adiabatic batch esterification with OpenFOAM-style REAL-TIME instant dirs: watch conversion climb 0 → 1 and T runaway 320 → ~428 K by scrubbing 0/ 30/ 60/ ... s

The story: the batch02_adiabatic runaway re-run with REAL-TIME instant directories: the integrator dumps 0/ 30/ 60/ ... s snapshots (OpenFOAM-style) so the trajectory – conversion 0 → 1, T 320 → ~428 K – can be scrubbed frame by frame in the GUI.

The physics below is inherited from batch02_adiabatic, unchanged: exothermic pseudo-first-order esterification (AcOH + EtOH → EtOAc + H2O) in an adiabatic batch reactor. Designed to show thermal runaway:

T_0 = 320 K (kinetics very slow: k_0 ~ 3.7e-4 s^-1, tau_0 ~ 2700 s). dH_rxn = -26.06 kJ/mol of ethanol, computed from the species' standardThermochemistry (elements datum); lifts T ~108 K at full conversion, enough to show the runaway. Equimolar ethanol + acetic acid charge; ethyl acetate + water are the products. Mass-conserving, so the energy balance is clean.

Expected qualitative trajectory:

T | __________ plateau (~428 K) | __/ | __/ | __/ | __/ | | / T_0 |___/ +——————————→ t induction runaway

The energy balance is just:

dT/dt = - r * V * dH_rxn / (n_A * Cp_A + n_B * Cp_B)

Constant-Cp here (polynomial degree 0 for both ethanol and water in the database), so the result extrapolates linearly above the nominal liquidHeatCapacity Trange (280-373 K) without numerical pathology — consistent with a pedagogical demonstration, not a quantitative prediction at ~430 K.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`reactor`	`P_final`	`101300`
`kpi`	`reactor`	`T_final`	`428.2212252`
`kpi`	`reactor`	`n_aceticAcid_final`	`9.87885051063e-31`
`kpi`	`reactor`	`n_ethanol_final`	`9.87885051063e-31`
`kpi`	`reactor`	`n_ethylAcetate_final`	`0.00925`
`kpi`	`reactor`	`n_water_final`	`0.00925`
`kpi`	`reactor`	`t_end`	`600`
`kpi`	`reactor`	`totalMoles_final`	`0.0185`
`kpi`	`campaign`	`element_C_closure_rel`	`0`
`kpi`	`campaign`	`element_H_closure_rel`	`0`
`kpi`	`campaign`	`element_O_closure_rel`	`0`
`kpi`	`campaign`	`element_worst_closure_rel`	`0`
`kpi`	`campaign`	`energy_balance_available`	`0`
`kpi`	`campaign`	`energy_records_invalid`	`0`

. . . and 12 more reference values in the case's `expected`*.*

batch05_crystalliser

tutorials/batch/reactor/batch05_crystalliser

What it does: Batch crystalliser: supersaturated sugar relaxes, PBE by method of moments

The story: Single closed vessel. Charge: ~500 kg water + ~1250 kg dissolved sucrose (c0 ~ 2.5 kg/kg, above c_sat(20 C) ~ 2.03 kg/kg, so S0 ~ 1.23). Unseeded: the moments start at zero and primary nucleation (j=0) bootstraps the population from the initial supersaturation. Kinetics from constant/crystallisation.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	6.81588841762e-15
kpi	campaign	element_H_closure_rel	4.18663266355e-15
kpi	campaign	element_O_closure_rel	4.18663266355e-15
kpi	campaign	element_worst_closure_rel	6.81588841762e-15
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	1
kpi	campaign	mass_closure_rel	5.07045231785e-15
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	1748.87224776
kpi	campaign	mass_kg_initial	1748.87224776
kpi	campaign	mass_residual_kg	8.86757334229e-12
kpi	campaign	transfers_H_invalid	0
kpi	campaign	transfers_H_valid	0

. . . and 21 more reference values in the case's `expected`.

batch06_adaptive_runaway

tutorials/batch/reactor/batch06_adaptive_runaway

reactor
batchReactor

What it does: Adiabatic esterification runaway (AcOH+EtOH); ADAPTIVE Rosenbrock23 time-stepping — the step grows after the fast transient (glass-box step history)

The story: the batch02_adiabatic runaway integrated with ADAPTIVE Rosenbrock23 time-stepping: the step shrinks through the fast runaway and GROWS again on the plateau, and the step history is printed glass-box so the student sees the controller at work.

The physics below is inherited from batch02_adiabatic, unchanged: exothermic pseudo-first-order esterification (AcOH + EtOH → EtOAc + H2O) in an adiabatic batch reactor. Designed to show thermal runaway:

T_0 = 320 K (kinetics very slow: k_0 ~ 3.7e-4 s^-1, tau_0 ~ 2700 s). dH_rxn = -26.06 kJ/mol of ethanol, computed from the species' standardThermochemistry (elements datum); lifts T ~108 K at full conversion, enough to show the runaway. Equimolar ethanol + acetic acid charge; ethyl acetate + water are the products. Mass-conserving, so the energy balance is clean.

Expected qualitative trajectory:

T | __________ plateau (~428 K) | __/ | __/ | __/ | __/ | / T_0 |___/ +——————→ t induction runaway

The energy balance is just:

dT/dt = - r * V * dH_rxn / (n_A * Cp_A + n_B * Cp_B)

Constant-Cp here (polynomial degree 0 for both ethanol and water in the database), so the result extrapolates linearly above the nominal liquidHeatCapacity Trange (280-373 K) without numerical pathology — consistent with a pedagogical demonstration, not a quantitative prediction at ~428 K.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	reactor	P_final	101300
kpi	reactor	T_final	428.221319512
kpi	reactor	n_aceticAcid_final	8.88140514762e-19
kpi	reactor	n_ethanol_final	1.10133235387e-34
kpi	reactor	n_ethylAcetate_final	0.00925
kpi	reactor	n_water_final	0.00925
kpi	reactor	t_end	600
kpi	reactor	totalMoles_final	0.0185
kpi	campaign	element_C_closure_rel	1.87537673079e-16
kpi	campaign	element_H_closure_rel	1.50030138463e-16
kpi	campaign	element_O_closure_rel	2.50050230771e-16
kpi	campaign	element_worst_closure_rel	2.50050230771e-16
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0

. . . and 12 more reference values in the case's `expected`*.*

batch07_lhhw_methylAcetate

tutorials/batch/reactor/batch07_lhhw_methylAcetate

reactor
batchReactor

What it does: LHHW in a BATCH reactor: methyl-acetate synthesis over Amberlyst 15 (Popken 2000, Eq 16)

The story: the SAME rate law as cstr07, in a BATCH vessel.

CH3OH + CH3COOH <→ CH3COOCH3 + H2O over Amberlyst 15

Popken, Gotze & Gmehling, Ind. Eng. Chem. Res. 39 (2000) 2601, Eq 16: the adsorption-based LHHW on UNIQUAC activities. Nothing about the rate law is restated here – it lives in constant/reactions, exactly as the steady CSTR reads it, because the SAME 'RateLaw' evaluates it in both. A batch reactor and a continuous one cannot disagree about what an adsorption denominator means.

What changes is only what a batch vessel does with a rate: it integrates it. The charge is equimolar methanol + acetic acid; the trajectory climbs to the ACTIVITY equilibrium, X = 76.6 %, and stops. That plateau is asserted nowhere: it is where k1 a'_HOAc a'_MeOH = k-1 a'_MeOAc a'_H2O, i.e. the K_a = 25.50 the authors' independent Eq 17 recovers from the same eight constants – and the same number the steady CSTR climbs to as its volume grows. Two reactors, two integration schemes, one equilibrium.

Water is the strongest adsorber (K/M = 0.29 against 0.052 for the acid), so as the reaction MAKES water it crowds the acid off the resin and the denominator grows. That self-inhibition is why reactive distillation, which pulls the water out as it forms, is worth its capital.

Do NOT try to see the inhibition by deleting the 'adsorption' block and keeping k1. It tells you nothing: the constants were REGRESSED for this rate law, on these activities, and a rate constant without its rate law is a number without a meaning. (Delete the denominator and the reaction gets FASTER, because the adsorbed activities a' are small and dividing by their square is a multiplication. That is an arithmetic accident, not physics.) To SEE the inhibition, spike the charge with water at the same acid and alcohol fractions:

methanol 0.4, aceticAcid 0.4, water 0.0 → initial rate 58.3 mmol/(L.s) methanol 0.4, aceticAcid 0.4, water 0.2 → initial rate 29.0 mmol/(L.s)

Half the rate, with every reactant fraction unchanged. The water is not a reactant here; it is sitting on the sites.

'catalystLoading' (kg of dry resin per m^3) is what turns the paper's per-gram rate constants into a volumetric rate. With concentrations in kmol/m^3 the conversion is exact and unit-free: mol/(g.s) x kg/m^3 = kmol/(m^3.s).

The vessel runs isothermal at 333.15 K, the top of the authors' data range. Do not extrapolate: their activation energies are APPARENT values, fitted over 303-333 K.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	reactor	P_final	101300
kpi	reactor	T_final	333.15
kpi	reactor	n_aceticAcid_final	0.00216454708498
kpi	reactor	n_methanol_final	0.00216454708498
kpi	reactor	n_methylAcetate_final	0.00708545291502
kpi	reactor	n_water_final	0.00708545291502
kpi	reactor	t_end	3600
kpi	reactor	totalMoles_final	0.0185
kpi	campaign	element_C_closure_rel	2.00040184617e-15
kpi	campaign	element_H_closure_rel	1.87537673079e-15
kpi	campaign	element_O_closure_rel	2.12542696156e-15
kpi	campaign	element_worst_closure_rel	2.12542696156e-15
kpi	campaign	energy_H_external_kJ	0
kpi	campaign	energy_Q_ledger_kJ	-218.009898425

. . . and 25 more reference values in the case's `expected`.

batch08_isothermal_duty

tutorials/batch/reactor/batch08_isothermal_duty

reactor
batchReactor

What it does: Isothermal jacket duty from a DECLARED dH_rxn: E = dH*dXi via extents (lumped species, no formation data)

The story: one isothermal batch reactor, one declared dH_rxn, one hand-checkable jacket duty. See system/controlDict for what this case tests and constant/reactions for the analytic anchor (E = -50 kJ/mol x 0.9975 mol = -49.88 kJ over 600 s).

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	energy_balance_available	0
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	1
kpi	campaign	mass_closure_rel	1.11022302463e-15
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	0.05
kpi	campaign	mass_kg_initial	0.05
kpi	campaign	mass_residual_kg	5.55111512313e-17
kpi	campaign	transfers_H_invalid	0
kpi	campaign	transfers_H_valid	0
kpi	campaign	transfers_logged	0
kpi	reactor	E_reaction_seg0_kJ	-49.8760623911
kpi	reactor	E_reaction_total_kJ	-49.8760623911
kpi	reactor	P_final	101300
. . . and 12 more reference values in the case's `expected`.			

recipes

The theory you need. A recipe is time-triggered discrete control over batch units: charge, heat/cool, hold, transfer, sample. Between events the DAE integrator runs; at each event the state jumps (a transfer moves holdup between vessels). Expect exact event times on the trajectory and holdup bookkeeping that conserves mass across transfers.

`recipe01_react_then_distill`

`tutorials/batch/recipes/recipe01_react_then_distill`

What it does: Two-vessel batch recipe: esterify (AcOH + EtOH → EtOAc + H2O) in a reactor, transfer the charge to a still, then distil

The story: the canonical 2-stage batch recipe.

Stage 1 (t = 0 → 400 s): A batch reactor at 350 K esterifies ethanol + acetic acid → ethyl acetate + water (the same first-order Arrhenius reaction as batch01). k ≈ 3.58e-3 s?^{1} at 350 K, tau ≈ 280 s. Over 400 s the ethanol conversion reaches ~76 %.

Stage 2 (t = 400 → 1200 s): The recipe TRANSFERs the reactor contents to an initially empty batch still. The still then runs a Rayleigh distillation at F_vap = 2.0e-5 kmol/s for 800 s, vaporising 0.016 kmol (~85 % of the charge) – the condensed distillate is routed, step by step, into a passive batchAccumulator (‘dischargeTo receiver;‘), so the campaign ends with the ester-rich distillate IN a vessel while the ethanol-depleted residue stays in the still’s own pot (the pot IS the bottoms tank; no third transfer needed). Mass is conserved across the three vessels at every accepted step. Starting from the converted mix (ethyl acetate + water + unreacted ethanol/acid), the still progressively strips the more volatile ethyl acetate + ethanol; pot composition follows the Raoult y-x curve. (The thermoPhysPropDict here uses ideal Raoult for pedagogical simplicity.)

This case demonstrates the recipe layer (v0.23): a list of time-triggered events acting on named units. Each event in the ‘recipe‘ block below has a ‘time‘, an ‘action‘, and action-specific keys. Actions implemented in v0.23:

transfer move all contents of ‘from‘ into ‘to‘ setParameter change a named operating parameter on a unit

Events fire at the next time-loop step boundary after t_trigger; the trajectory CSV thus shows a single-step discontinuity in the transferred quantities.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`campaign`	`element_C_closure_rel`	`1.87537673079e-16`
`kpi`	`campaign`	`element_H_closure_rel`	`3.00060276926e-16`
`kpi`	`campaign`	`element_O_closure_rel`	`5.00100461543e-16`
`kpi`	`campaign`	`element_worst_closure_rel`	`5.00100461543e-16`
`kpi`	`campaign`	`energy_H_external_kJ`	`0`
`kpi`	`campaign`	`energy_Q_ledger_kJ`	`-145.006283394`
`kpi`	`campaign`	`energy_balance_available`	`1`
`kpi`	`campaign`	`energy_closure_rel`	`7.81829067932e-11`
`kpi`	`campaign`	`energy_dH_vessels_kJ`	`-145.006282333`
`kpi`	`campaign`	`energy_records_invalid`	`0`
`kpi`	`campaign`	`energy_records_valid`	`4`
`kpi`	`campaign`	`energy_residual_kJ`	`1.06158961444e-06`
`kpi`	`campaign`	`mass_closure_rel`	`2.26202374215e-16`
`kpi`	`campaign`	`mass_kg_external_out`	`0`
... and 51 more reference values in the case’s `expected`*.*			

recipe02_setpoint_ramp

tutorials/batch/recipes/recipe02_setpoint_ramp

reactor
batchReactor

What it does: Single-reactor recipe with T_setpoint steps: 320 K → 350 K @ 200 s → 380 K @ 500 s

The story: isothermal batch reactor with a staged temperature programme controlled by the recipe layer.

The reactor starts at 320 K, where the first-order Arrhenius reaction is very slow (k_320 ≈ 3.7e-4 s?^{1}, $\tau \approx$ 2700 s). Two recipe events ramp the setpoint upward:

t = 200 s → 320 K → 350 K (k jumps ~10x to 3.58e-3 s?^{1}) t = 500 s → 350 K → 380 K (k jumps another ~7x to 0.025 s?^{1})

In the trajectory CSV the temperature column shows a clean step-change at each event, and the conversion curve shows the corresponding three regimes:

$0 \leq t < 200$ s barely-moving plateau (slow) $200 \leq t < 500$ s visible drop in n_ethanol (moderate) $500 \leq t$ rapid quench to full conversion (fast)

Demonstrates the recipe layer's 'setParameter' action without inter-unit transfers, so this is the cleanest possible test of the parameter-change pathway.

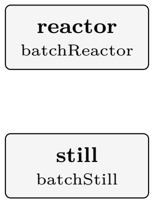
What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	3.75075346157e-16
kpi	campaign	element_H_closure_rel	3.00060276926e-16
kpi	campaign	element_O_closure_rel	2.50050230771e-16
kpi	campaign	element_worst_closure_rel	3.75075346157e-16
kpi	campaign	energy_H_external_kJ	0
kpi	campaign	energy_Q_ledger_kJ	-104.365240634
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_closure_rel	0
kpi	campaign	energy_dH_vessels_kJ	-104.365240634
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	5
kpi	campaign	energy_residual_kJ	0
kpi	campaign	mass_closure_rel	4.5240474843e-16
kpi	campaign	mass_kg_external_out	0

... and 33 more reference values in the case's `expected`.

recipe03_conditional_partial

tutorials/batch/recipes/recipe03_conditional_partial



What it does: Conditional + partial recipe: react until x_ethanol < 0.30, then transfer 50% to a still

The story: recipe extensions (v0.50).

Two recipe features beyond the time-triggered transfer/setParameter of v0.23:

1. CONDITION-TRIGGERED event (‘when { ... }‘ instead of ‘time‘): the event fires the first step its condition becomes true, evaluated on the live vessel state. Quantities: T, total, n_<comp>, x_<comp>; operators: gt, ge, lt, le.

2. PARTIAL transfer (‘fraction‘): a fraction (0..1) of the source moves; the source keeps the rest. Default 1.0 == empty the source.

Here the reactor runs the esterification (ethanol + acetic acid → ethyl acetate + water; the batch01 first-order Arrhenius test, k ~ 3.58e-3 s^-1 at 350 K, so n_ethanol = n_ethanol(0).exp(-k t)). The charge is equimolar (x_ethanol starts at 0.5); when the ethanol mole fraction drops below 0.30, the recipe transfers HALF the reactor contents to a still; the reactor keeps the other half and reacts on, while the still distils its charge.

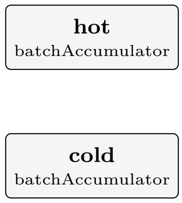
What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	reactor	P_final	101300
kpi	reactor	T_final	350
kpi	reactor	n_aceticAcid_final	6.33853236066e-05
kpi	reactor	n_ethanol_final	6.33853236066e-05
kpi	reactor	n_ethylAcetate_final	0.00456161467639
kpi	reactor	n_water_final	0.00456161467639
kpi	reactor	t_end	1200
kpi	reactor	totalMoles_final	0.00925
kpi	still	P_final	101300
kpi	still	T_final	390.833339675
kpi	still	n_aceticAcid_final	0
kpi	still	n_ethanol_final	0
kpi	still	n_ethylAcetate_final	0
kpi	still	n_water_final	0

. . . and 43 more reference values in the case’s `expected`*.*

`recipe04_charge_enthalpy`

`tutorials/batch/recipes/recipe04_charge_enthalpy`



What it does: H-equality recipe charge: T_mix from the first law, not the molar average; balance CLOSES across a transfer

The story: mixing is a FIRST-LAW act.

Two passive tanks. ‘hot‘ holds 0.5 mol of ethanol at 340 K; ‘cold‘ holds 1.0 mol of water at 300 K. At t = 100 s the recipe empties the hot tank into the cold one. The charge solves the mix temperature from enthalpy equality on the elements datum,

H(n_EtOH + n_W, T_mix) = H(n_EtOH, 340 K) + H(n_W, 300 K),

so heat capacity DIFFERENCES matter: ethanol carries ~112 J/(mol K) against water’s ~75, and the enthalpy-true T_mix (~317 K) sits ~4 K above the equal-Cp molar average (313.3 K) a naive mixing rule gives.

Because both species price on the elements datum, no jacket acts, and nothing leaves, the campaign ENERGY BALANCE is AVAILABLE and closes: dH_vessels = 0 to the mix-solve tolerance — the transfer moved enthalpy between tanks without creating or destroying any.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`campaign`	`element_C_closure_rel`	`0`
`kpi`	`campaign`	`element_H_closure_rel`	`0`
`kpi`	`campaign`	`element_O_closure_rel`	`0`
`kpi`	`campaign`	`element_worst_closure_rel`	`0`
`kpi`	`campaign`	`energy_H_external_kJ`	`0`
`kpi`	`campaign`	`energy_Q_ledger_kJ`	`0`
`kpi`	`campaign`	`energy_balance_available`	`1`
`kpi`	`campaign`	`energy_closure_rel`	`2.13028428746e-12`
`kpi`	`campaign`	`energy_dH_vessels_kJ`	`5.81991344006e-10`
`kpi`	`campaign`	`energy_records_invalid`	`0`
`kpi`	`campaign`	`energy_records_valid`	`0`
`kpi`	`campaign`	`energy_residual_kJ`	`5.81991344006e-10`
`kpi`	`campaign`	`mass_closure_rel`	`0`
`kpi`	`campaign`	`mass_kg_external_out`	`0`

...and 21 more reference values in the case’s `expected`.

still

The theory you need. A batch still is Rayleigh distillation: $\mathrm{d}(n\,x_i)/\mathrm{d}t = -\dot{V}y_i$ with y_i in equilibrium with the pot – the pot composition DRIFTS as the light component leaves (contrast the steady column, where stages hold profiles constant). Expect the distillate to start rich and fade, and the recipe layer (charge, heat, switch receivers) to drive the run. *Full derivation: Theory Guide, the batch-distillation section.*

still01_benzene_toluene

tutorials/batch/still/still01_benzene_toluene

What it does: Rayleigh batch distillation: benzene/toluene 50/50, 1 atm, ideal Raoult

The story: Rayleigh batch distillation, ideal Raoult.

Pot starts with 1 mol of equimolar benzene/toluene at 1 atm. A constant vapour offtake F_vap = 1.5e-6 kmol/s strips the pot over 600 s — by linear mass balance the pot should hold

i.e. L(600 s) = 1 mmol - 0.9 mmol = 0.1 mmol (90 % distilled).

Benzene is the more volatile component (alpha ~ 2.4 at 1 atm), so the still pot progressively concentrates in toluene as benzene flashes off preferentially. Classical Rayleigh integration with constant alpha gives the closed form

$$\ln(L / L_0) = (1 / (\alpha - 1)) \left[\ln(x / x_0) + \alpha * \ln((1 - x_0) / (1 - x)) \right]$$

where x = x_benzene in the still pot. Plotting (L/L_0) vs x_pot from trajectory.csv should track this analytical curve to RK4 precision.

One mole of 50/50 benzene/toluene in a pot at 1.013 bar, boiled off for 600 s with the vapour taken away as it forms — no reflux, no trays: the Rayleigh equation integrated in time. The pot starts at its bubble point, 365.00 K, and ends at 380.04 K with 0.1 mol left in it at 91.8 % toluene; 0.9 mol (0.0760 kg) has left as distillate. The reboiler supplied 30.33 kJ, the condenser took 29.39 kJ away.

1. The residue gets heavier, and hotter, because the light one leaves first. The Log tab prints the pot every 20 s: benzene falls faster than toluene, T climbs from 365 to 380 K as the pot approaches toluene's boiling point. That is the Rayleigh curve as numbers. 2. The whole campaign is a ledger. [campaign] mass balance: m0 = 0.0851 kg = mF 0.0091 + external 0.0760 kg, closure 0 — what left the domain is counted, not lost. The energy side closes at 4.6e-16 with two records, reboiler and condenser, each priced at its own service temperature (380.0 K and 365.3 K). 3. The thermodynamics is Raoult, and the caveats say where it is stretched. Benzene's Antoine window ends at 354 K and the pot starts at 365 K; toluene's ends at 380 K and the pot ends at 380.04. Both extrapolations are announced at the moment they happen, on the console, inside the table. 4. Utilities are scheduled in time. reports/utilities/ utilityDemand.csv carries the staircase of steam and cooling-water demand across the 600 s — a peak, not just a total, which is what a batch plant is actually sized on.

Halve endTime: the pot is still half full and barely enriched — most of the separation happens late, when there is little left. Then still04_rectifier_benzene_toluene puts three stages and a reflux over the same pot, and the distillate purity that a Rayleigh still cannot reach appears.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	2.66880534766e-16
kpi	campaign	element_H_closure_rel	1.23908819713e-16
kpi	campaign	element_worst_closure_rel	2.66880534766e-16
kpi	campaign	energy_H_external_kJ	38.7543769459
kpi	campaign	energy_Q_ledger_kJ	0.942780166065
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_closure_rel	4.58362894565e-16
kpi	campaign	energy_dH_vessels_kJ	-37.8115967798
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	2
kpi	campaign	energy_residual_kJ	1.7763568394e-14
kpi	campaign	mass_closure_rel	0
kpi	campaign	mass_kg_external_out	0.0760287317788
kpi	campaign	mass_kg_final	0.00909876822121

... and 29 more reference values in the case's `expected`.

still02_ethanol_water_azeo

tutorials/batch/still/still02_ethanol_water_azeo

What it does: Rayleigh batch distillation: ethanol/water 40/60 with NRTL, residue concentrates in water

The story: Rayleigh batch distillation of a non-ideal binary with an azeotrope.

System: ethanol / water with NRTL, pressure 1 atm. Minimum-boiling positive azeotrope at x_eth ~ 0.894 (mole), T ~ 351.3 K.

Pot starts at x_eth = 0.40 (a "wine"-like feed — well BELOW the azeotrope). Boil-off rate F_vap = 1.5e-6 kmol/s for 600 s consumes 90 % of the pot, leaving a 10 % residue.

Below the azeotrope, gamma_ethanol > 1 and gamma_water ~ 1, so the vapour is markedly richer in ethanol than the liquid (y_eth ~ 0.61 at x_eth = 0.40). Ethanol therefore vaporises preferentially and the still pot progressively concentrates in WATER — i.e. the residue at end-of-run is mostly water.

The pedagogical contrast with still01 (ideal Raoult, benzene/toluene) is the SHAPE of the enrichment curve:

Raoult: smooth, alpha-driven (alpha ~ 2.4, roughly constant). NRTL : curvature changes as x crosses concentration regimes in which gamma_ethanol shrinks toward 1; the still composition curve x_pot(t) is noticeably non-linear vs the linear pot-volume curve L(t) = L_0 - F_vap*t.

Crucially, this case demonstrates that a single Rayleigh still cannot cross the azeotrope. Starting below 0.894 in ethanol, the pot can only move DOWNWARD in ethanol fraction; no amount of boiling will lift the pot composition past 0.894. (To break the azeotrope one needs entrainer, pressure swing, or extractive distillation — topics for later tutorials.)

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	still	P_final	101300
kpi	still	T_final	372.446750369
kpi	still	n_ethanol_final	3.38836968904e-11
kpi	still	n_water_final	9.99999650927e-05
kpi	still	t_end	600
kpi	still	totalMoles_final	9.99999989764e-05
kpi	campaign	element_C_closure_rel	1.35525271561e-15
kpi	campaign	element_H_closure_rel	9.63735264432e-16
kpi	campaign	element_O_closure_rel	6.50521303491e-16
kpi	campaign	element_worst_closure_rel	1.35525271561e-15
kpi	campaign	energy_H_external_kJ	-249.391762836
kpi	campaign	energy_Q_ledger_kJ	0.490652916203
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_closure_rel	1.13740334008e-16

...and 30 more reference values in the case's `expected`.

still03_with_receiver

tutorials/batch/still/still03_with_receiver

What it does: Rayleigh still distilling INTO a receiver tank (dischargeTo accumulator)

The story: Rayleigh batch distillation INTO a receiver (accumulator vessel + continuous dischargeTo).

Same pot as still01 (1 mol equimolar benzene/toluene, F__vap = 1.5e-6 kmol/s, ideal Raoult), but the still's vapour is no longer lost: the ‘dischargeTo receiver;‘ line routes the condensed distillate, step by step, into a passive ‘batchAccumulator‘. So the trajectory shows the POT depleting AND the RECEIVER filling — the receiver collects the total distillate (amount + composition). Mass is conserved between the two vessels: pot + receiver = 1 mol at all times. Benzene is the more volatile component, so the receiver is benzene-rich early and the cumulative receiver composition drifts toward 0.5 as distillation nears completion (it ends holding everything the pot lost).

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	2.66880534766e-16
kpi	campaign	element_H_closure_rel	1.23908819713e-16
kpi	campaign	element_worst_closure_rel	2.66880534766e-16
kpi	campaign	energy_H_external_kJ	0
kpi	campaign	energy_Q_ledger_kJ	0.942780166065
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_closure_rel	3.00056993907e-11
kpi	campaign	energy_dH_vessels_kJ	0.942780163768
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	2
kpi	campaign	energy_residual_kJ	2.2974155911e-09
kpi	campaign	mass_closure_rel	0
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	0.0851275

. . . and 35 more reference values in the case's `expected`.

still04_rectifier_benzene_toluene

tutorials/batch/still/still04_rectifier_benzene_toluene

What it does: batch rectification: 3 stages + total condenser, R=3, with receiver tank (dischargeTo accumulator)

The story: the CLASSICAL batch rectification (model rectifier, design forum #85): the Rayleigh pot now feeds a column of 3 ideal stages topped by a total condenser at constant reflux R = 3.

F__vap is the internal BOILUP V = 1.5e-6 kmol/s; the distillate PRODUCT is D = V/(R+1) = 3.75e-7 kmol/s and the reflux L = R.D returns to the column. The pot loses PRODUCT only: dn_i/dt = -D.x_D_i, with x_D(t) solved at every instant from the quasi-steady cascade (stage balances + bubble-T equilibrium + total-condenser closure – a declared residual, printed on failure).

What to watch (trajectory.csv): still.xD__benzene starts ~0.949 (vs ~0.71 for the raw pot vapour – the column working) and DRIFTS DOWN as the pot strips: the constant-R policy trades purity for time. The receiver's ACCUMULATED benzene fraction sits between the instantaneous x_D and the feed – two different compositions, deliberately both visible. The pot loses only D.dt = 0.225 mmol over 600 s (vs 0.9 mmol in still01: same boilup, but 3/4 of it refluxes).

Physics anchors (verified at authoring): x_D(0) grows with nStages and R (N=1,R=1: 0.806; N=2,R=3: 0.909; N=3,R=3: 0.949) and the R → infinity limit (R=200: 0.9765) lands inside the Fenske band alpha^(N+1) for alpha = 2.35-2.6. With nStages 0 and refluxRatio 0 the model reproduces still01's Rayleigh trajectory to the last digit.

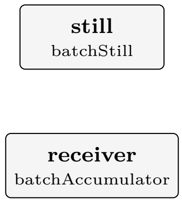
What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	0
kpi	campaign	element_H_closure_rel	0
kpi	campaign	element_worst_closure_rel	0
kpi	campaign	energy_H_external_kJ	0
kpi	campaign	energy_Q_ledger_kJ	0.534303338765
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_closure_rel	7.08707897844e-10
kpi	campaign	energy_dH_vessels_kJ	0.534303321211
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	2
kpi	campaign	energy_residual_kJ	1.75539529579e-08
kpi	campaign	mass_closure_rel	0
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	0.0851275

... and 40 more reference values in the case's `expected`.

still05_rectifier_constant_xD

tutorials/batch/still/still05_rectifier_constant_xD



What it does: batch rectification, constant-composition policy: R(t) holds x_D until refluxMax, then D=0

The story: the SECOND classical reflux policy (design forum #85-3/4): hold the distillate composition constant by RAISING the reflux as the pot strips, and STOP when the declared ceiling can no longer reach the target.

Same pot and column as still04 (1 mol equimolar benzene/toluene, V = 1.5e-6 kmol/s, 3 ideal stages + total condenser), but instead of a fixed R the policy solves R(t) each step – a BRACKETED solve over [0, refluxMax] on a feasibility map, never a bare Newton – so that x_D[benzene] = 0.93 exactly. As benzene depletes, R climbs (~2 at the start); when even refluxMax = 8 cannot reach the target the run announces ‘onRefluxLimit stopDistillation‘: D = 0, the pot stops losing mass, and the campaign idles VISIBLY to endTime with still.refluxLimitReached = 1 in the trajectory – never a magic termination.

What to watch (trajectory.csv): still.xD__benzene pinned at 0.9300 while R climbs – the mirror image of still04, which held R and let purity drift; still.R rising 2.0 → 8 (the ceiling), then the limit event; the receiver’s accumulated composition ~0.93 (the whole cut is on-spec – THE selling point of this policy); pot inventory constant after t_refluxLimit.

Declared refusals (each with its own diagnostic, try them): x_D__target 0.5 → BELOW the R=0 floor (even zero reflux over-purifies); x_D__target 0.995 with refluxMax 5 → UNREACHABLE from the charge; ethanol/water NRTL with x_D__target 0.95 → the 0.894 azeotrope caps the cascade at ANY reflux (the barrier is real VLE, not a message).

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	campaign	element_C_closure_rel	0
kpi	campaign	element_H_closure_rel	0
kpi	campaign	element_worst_closure_rel	0
kpi	campaign	energy_H_external_kJ	0
kpi	campaign	energy_Q_ledger_kJ	0.690222586439
kpi	campaign	energy_balance_available	1
kpi	campaign	energy_closure_rel	8.89386285064e-10
kpi	campaign	energy_dH_vessels_kJ	0.69022261392
kpi	campaign	energy_records_invalid	0
kpi	campaign	energy_records_valid	2
kpi	campaign	energy_residual_kJ	2.74809703882e-08
kpi	campaign	mass_closure_rel	0
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	0.0851275

. . . and 42 more reference values in the case’s `expected`*.*

still06_ledger_mixed_validity

tutorials/batch/still/still06_ledger_mixed_validity

What it does: Ledger H-validity is MONOTONIC: mixed valid/invalid packages never resurrect a poisoned record

The story: MONOTONIC enthalpy validity on the material ledger (the adversarial case of design forum #99/#101).

The campaign ledger integrates each transfer's enthalpy PACKAGE BY PACKAGE at each package's own temperature. A package containing a species with NO formation datum (here 'compA', a lumped teaching species that deliberately carries no standardThermochemistry block) cannot be priced — the record must mark H invalid and NAME the species. The contract under test is the temporal one:

once ONE package is unpriceable, the record's integrated H has lost a parcel FOREVER. Validity is MONOTONIC: a later priceable package must never resurrect it.

Two stills exercise the two orders on ONE accumulated record each:

potA (invalid → valid): starts with 10 % compA, so its distillate record potA→recA begins UNPRICEABLE. At t = 200 s the recipe dumps the whole pot to 'dump' and recharges clean benzene/toluene from 'fresh'; every later distillate package is priceable. The record must STAY invalid (H_missing = compA), and the discrete potA→dump record is invalid too (it carried the compA).

potB (valid → invalid): starts clean, so its distillate record potB→recB begins priceable and accumulates real enthalpy. At t = 200 s the recipe spikes it with pure compA from 'spike'; every later package is unpriceable. The record flips invalid and the partial integral is WITHHELD — an emitted H_kJ that silently lost its tail would be a lie.

Five ledger records result: potA→recA (continuous, poisoned early), potB→recB (continuous, poisoned late), potA→dump and spike→potB (discrete, carrying compA: unpriceable), fresh→potA (discrete, clean: the ONE priceable record). The golden pins the campaign census transfers_H_valid = 1 / transfers_H_invalid = 4; the results JSON carries H_missing = [compA] and NO H_kJ key on every poisoned record.

What to expect (golden-locked):

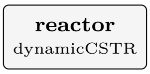
kind	unit/stream	key	expected
kpi	campaign	mass_closure_rel	2.1198000621e-16
kpi	campaign	mass_kg_external_out	0
kpi	campaign	mass_kg_final	0.26186975
kpi	campaign	mass_kg_initial	0.26186975
kpi	campaign	mass_residual_kg	5.55111512313e-17
kpi	campaign	transfers_H_invalid	4
kpi	campaign	transfers_H_valid	1
kpi	campaign	transfers_logged	5
kpi	dump	P_final	101300
kpi	dump	T_final	372.484229602
kpi	dump	n_benzene_final	0.000249727144577
kpi	dump	n_compA_final	9.64864338217e-05
kpi	dump	n_toluene_final	0.000353786421588
kpi	dump	t_end	600
. . . and 68 more reference values in the case's `expected`*.*			

Category IV: Dynamics and control (choupoCtrl)

Continuous processes in time with feedback: PID loops, setpoint schedules, disturbances and responses.

ctrl01_cstr_temp_control

tutorials/ctrl/ctrl01_cstr_temp_control



What it does: Dynamic CSTR (compA → compB) with PID jacket-temperature control; T climbs from 320 K into the +/-2% setpoint band within ~20 s (no overshoot) and tracks toward 350 K across the 150 s window

The story: the canonical first choupoCtrl test.

Process: Continuous CSTR with first-order reaction compA → compB – an ISOMERISATION: both stand-ins carry MW 30, so the pseudo-reaction conserves mass exactly and the run-end material ledger closes to machine precision. (The element balance stays honestly UNAVAILABLE: compA/compB are numerical stand-ins with no real formula, and the ledger says so by name instead of fabricating atoms.) Feed is pure compA at 320 K, 5×10?? kmol/s; reactor volume 1 L; mild exotherm (ΔH_rxn = -50 kJ/mol). Without control the reactor settles a few K above the inlet temperature (small reaction exotherm balanced by convective cooling), well below the desired setpoint of 350 K.

Controller: A PID acts on the jacket temperature (T_jacket) to drive the reactor temperature (T) to a setpoint of 350 K. Tuning is pedagogical, not optimal:

Kp = 4.0 (proportional band ≈ 7-8 K) Ki = 0.04 (integral time τ_I = Kp/Ki ≈ 100 s) Kd = 0.0 (no derivative; system is benign enough)

Jacket bounded to [280, 420] K — realistic limits of a utility loop. Anti-windup: integral does not advance when the unclamped output is outside the bounds.

Expected trajectory (150 s window): Open-loop: T relaxes a few K above the 320 K inlet (small exotherm balanced by convective cooling). Closed-loop: T rises fast (the jacket MV slams to its 420 K clamp, then backs off), entering the +/-2% setpoint band by ~ 20 s, well-damped with no overshoot. By 150 s T is ~ 348.5 K and still creeping up: the last ~ 1.5 K rides the slow chemical tail (the exotherm's conversion, tau ~ 120 s) toward 350 K beyond the window. The jacket (MV) settles around 348.5 K to keep heating in against the cool 320 K feed. (Widen endTime to watch the final approach to 350 K; the 150 s window is sized to SHOW the control transient, not the slow tail.)

To run the OPEN-LOOP comparison, comment out the entire 'controllers' block below — the case will then run the bare reactor with whatever T_jacket the 'operation' block initialised it to (here 320 K, i.e. no jacket heating).

A 1-litre continuous stirred reactor with a first-order A → B reaction (two stand-in components, both MW 30 — the pseudo-reaction says so by name rather than inventing atoms), fed at 320 K, with a jacket whose temperature a PID controller sets to hold the reactor at 350 K. 150 s of transient, 1 s steps. The golden: PV ends at 348.47 K, the jacket (MV) at 348.54 K, mass in = out = 0.225 kg over the run and the material balance closes at 1.8e-15.

1. The controller is a unit in the dict, with its tuning in the open. type PID; setpoint 350.0; gains { Kp 4.0; Ki 0.04; Kd 0.0; } output { min 280; max 420; bias 320; } — proportional band ≈ 7–8 K, integral time τ_I = Kp/Ki = 100 s, no derivative. Nothing is tuned for you. 2. Watch the manipulated variable saturate. At t = 2 s the jacket demand jumps to the 420 K clamp (the log's last column), then backs off as the reactor warms: 402, 385, 374 K ? The Log tab prints every 2 s; the clamp is a number in the dict and you can see it bind. 3. Into the band, then the slow part. The reactor crosses 343 K — the -2 % edge of the setpoint band — at t ≈ 14 s. At 150 s it sits 1.5 K under setpoint and still climbing: that is the integral action at τ_I = 100 s working off the offset. Widen endTime and watch it arrive. 4. The energy balance REFUSES here, and says why. [balance] energy: UNAVAILABLE — the dynamicCSTR energy equation is a Cp/convective model, not the exact derivative of a stored H. The stand-in components carry no formation enthalpy on purpose; a ledger that cannot be exact says so rather than printing a closure it did not earn. ctrl11 is the case where it can, and does (1.4e-7).

Set Kp 8.0;: faster into the band, more overshoot, the clamp held longer. Then Ki 0; — the offset never closes, which is what a proportional-only loop teaches in one run. ctrl02_disturbance_rejection steps the feed temperature and asks the same loop to hold.

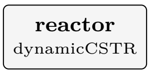
What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	TC1	MV_final	348.54491648
kpi	TC1	PV_final	348.468973192
kpi	TC1	SP_final	350
kpi	reactor	T_final	348.481893564
kpi	reactor	n_compA_final	0.0116161140368
kpi	reactor	n_compB_final	0.00038388596316
kpi	reactor	t_end	150
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	1.77327288655e-15
kpi	balance	mass_kg_final	0.36
kpi	balance	mass_kg_in_cum	0.225
kpi	balance	mass_kg_initial	0.36
kpi	balance	mass_kg_out_cum	0.225
kpi	balance	mass_residual_kg	6.38378239159e-16

. . . and 4 more reference values in the case's `expected`.

ctrl02_disturbance_rejection

tutorials/ctrl/ctrl02_disturbance_rejection



What it does: PID disturbance rejection: T_in steps down then up, PID maintains T_setpoint

The story: the textbook control demo.

Same DynamicCSTR as ctrl01 (compA → compB, mild exotherm, jacket heat transfer). Same PID temperature controller. Two added ingredients:

1. The simulation runs 2000 s (long enough to reach steady state after each disturbance). 2. A second "controller" of type Schedule injects step disturbances in the inlet temperature T_in:

0 | 320 (nominal) 700 | 305 (-15 K cold disturbance) 1300 | 335 (+15 K warm disturbance, back-and-up)

The PID controller has no foreknowledge of these disturbances — it only sees T_reactor drift away from setpoint and acts on the jacket to compensate. Expected behaviour:

0 < t < 700: normal startup, T → 350 K (≈ ctrl01). t = 700: T_in drops 15 K; the integral action absorbs it so tightly that T_reactor holds at 350 K with no visible sag (the jacket simply keeps heating). t = 1300: T_in rises 15 K; again T_reactor stays pinned at 350 K with no visible deviation.

The trajectory.csv columns now include both controllers' SP / PV / MV. The Schedule controller's SP and MV mirror the same step train (it has no PV); the PID's SP stays at 350 K and its PV (T_reactor) tracks closely.

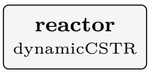
What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	FeedDist	MV_final	335
kpi	FeedDist	PV_final	0
kpi	FeedDist	SP_final	335
kpi	TC1	MV_final	347.976535585
kpi	TC1	PV_final	350.00207645
kpi	TC1	SP_final	350
kpi	reactor	T_final	350.002059115
kpi	reactor	n_compA_final	0.0110806465498
kpi	reactor	n_compB_final	0.000919353450202
kpi	reactor	t_end	2000
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	2.79221090693e-14
kpi	balance	mass_kg_final	0.36
kpi	balance	mass_kg_in_cum	3

. . . and 7 more reference values in the case's `expected`*.*

ctrl03_adaptive_disturbance

tutorials/ctrl/ctrl03_adaptive_disturbance



What it does: PID disturbance rejection with ADAPTIVE plant integration — controller samples on the fixed 1 s grid, Rosenbrock23 sub-steps between samples (MV held)

The story: the ctrl02_disturbance_rejection demo re-integrated with an ADAPTIVE plant integrator: the PID still samples on the fixed 1 s control grid (MV held between samples) while Rosenbrock23 sub-steps the plant adaptively in between. The flowsheet below is ctrl02's textbook disturbance-rejection setup, unchanged:

Same DynamicCSTR as ctrl01 (compA → compB, mild exotherm, jacket heat transfer). Same PID temperature controller. Two added ingredients:

1. The simulation runs 2000 s (long enough to reach steady state after each disturbance). 2. A second "controller" of type Schedule injects step disturbances in the inlet temperature T_in:

0 | 320 (nominal) 700 | 305 (-15 K cold disturbance) 1300 | 335 (+15 K warm disturbance, back-and-up)

The PID controller has no foreknowledge of these disturbances — it only sees T_reactor drift away from setpoint and acts on the jacket to compensate. Expected behaviour:

0 < t < 700: normal startup, T → 350 K (≈ ctrl01). t = 700: T_in drops 15 K; the integral action absorbs it so tightly that T_reactor holds at 350 K with no visible sag (the jacket simply keeps heating). t = 1300: T_in rises 15 K; again T_reactor stays pinned at 350 K with no visible deviation.

The trajectory.csv columns now include both controllers' SP / PV / MV. The Schedule controller's SP and MV mirror the same step train (it has no PV); the PID's SP stays at 350 K and its PV (T_reactor) tracks closely.

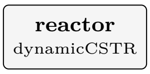
What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`FeedDist`	`MV_final`	335
`kpi`	`FeedDist`	`PV_final`	0
`kpi`	`FeedDist`	`SP_final`	335
`kpi`	`TC1`	`MV_final`	347.976535626
`kpi`	`TC1`	`PV_final`	350.002076468
`kpi`	`TC1`	`SP_final`	350
`kpi`	`reactor`	`T_final`	350.002059134
`kpi`	`reactor`	`n_compA_final`	0.0110806465416
`kpi`	`reactor`	`n_compB_final`	0.000919353458409
`kpi`	`reactor`	`t_end`	2000
`kpi`	`balance`	`energy_balance_available`	0
`kpi`	`balance`	`mass_closure_rel`	2.66083451568e-14
`kpi`	`balance`	`mass_kg_final`	0.36
`kpi`	`balance`	`mass_kg_in_cum`	3

. . . and 7 more reference values in the case's `expected`.

ctrl03_startup_transient

tutorials/ctrl/ctrl03_startup_transient



What it does: Open-loop CSTR START-UP transient with OpenFOAM-style real-time instant dirs: charged cold (320 K), no control, watch T(t) and the holdup approach the natural steady state by scrubbing 0/ 50/ 100/ ... s

The story: Process: A continuous CSTR (first-order compA → compB, mild exotherm) is CHARGED cold (320 K, pure compA) and then left to run with NO controller. The jacket sits at a fixed 320 K. From the cold start the reactor warms a little from the reaction exotherm, the feed/ outlet convection competes, and (T, composition) relax to the natural open-loop steady state over ~ several residence times.

What this tutorial shows: The OpenFOAM-style transient solution directories. With ‘solutionControl { write true; }‘ in controlDict, the case root fills with REAL-TIME directories ‘0/ 50/ 100/ ... 1000/‘. Each ‘<t>/internalState‘ is the reactor HOLDUP at that instant; each ‘<t>/streamFaces‘ is the instantaneous outlet face. Scrub the times and you watch T(t) and the inventory march to steady state — the same way you scrub a CFD transient in ParaView.

To compare CLOSED-LOOP control, see ctrl01_cstr_temp_control (a PID drives T to a 350 K setpoint) and ctrl05_disturbance_transient (a PID rejecting a feed-temperature disturbance), both also writing time dirs.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	reactor	T_final	320.759374011
kpi	reactor	n_compA_final	0.0117988900072
kpi	reactor	n_compB_final	0.000201109992805
kpi	reactor	t_end	1000
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	1.77265609598e-14
kpi	balance	mass_kg_final	0.36
kpi	balance	mass_kg_in_cum	1.5
kpi	balance	mass_kg_initial	0.36
kpi	balance	mass_kg_out_cum	1.5
kpi	balance	mass_residual_kg	-2.65898414398e-14
kpi	balance	material_available	1
kpi	reactor	F_in_final	5e-05
kpi	reactor	z_in_compA_final	1

. . . and 1 more reference values in the case's `expected`.

ctrl05_disturbance_transient

tutorials/ctrl/ctrl05_disturbance_transient

reactor
dynamicCSTR

What it does: PID disturbance rejection with OpenFOAM-style real-time instant dirs: T_in steps down (700 s) then up (1300 s); scrub 0/ 40/ 80/ ... s to watch the PID track the 350 K setpoint and the jacket MV move

The story: the textbook control demo, captured in OpenFOAM-style real-time instant directories (see controlDict's solutionControl block). Identical physics to ctrl02; the difference is that each written time drops a '<t>/internalState' + '<t>/streamFaces' you can scrub across the two disturbance steps to watch the PID reject them.

Same DynamicCSTR as ctrl01 (compA → compB, mild exotherm, jacket heat transfer). Same PID temperature controller. Two added ingredients:

1. The simulation runs 2000 s (long enough to reach steady state after each disturbance). 2. A second "controller" of type Schedule injects step disturbances in the inlet temperature T_in:

0 | 320 (nominal) 700 | 305 (-15 K cold disturbance) 1300 | 335 (+15 K warm disturbance, back-and-up)

The PID controller has no foreknowledge of these disturbances — it only sees T_reactor drift away from setpoint and acts on the jacket to compensate. Expected behaviour:

$0 < t < 700$: normal startup, T → 350 K (≈ ctrl01). t = 700: T_in drops 15 K; the integral action absorbs it so tightly that T_reactor holds at 350 K with no visible sag (the jacket simply keeps heating). t = 1300: T_in rises 15 K; again T_reactor stays pinned at 350 K with no visible deviation.

The trajectory.csv columns now include both controllers' SP / PV / MV. The Schedule controller's SP and MV mirror the same step train (it has no PV); the PID's SP stays at 350 K and its PV (T_reactor) tracks closely.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	FeedDist	MV_final	335
kpi	FeedDist	PV_final	0
kpi	FeedDist	SP_final	335
kpi	TC1	MV_final	347.976535585
kpi	TC1	PV_final	350.00207645
kpi	TC1	SP_final	350
kpi	reactor	T_final	350.002059115
kpi	reactor	n_compA_final	0.0110806465498
kpi	reactor	n_compB_final	0.000919353450202
kpi	reactor	t_end	2000
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	2.79221090693e-14
kpi	balance	mass_kg_final	0.36
kpi	balance	mass_kg_in_cum	3

...and 7 more reference values in the case's `expected`.

`ctrl06_sine_disturbance`

`tutorials/ctrl/ctrl06_sine_disturbance`

reactor
dynamicCSTR

What it does: Forced SINUSOIDAL inlet-T disturbance: T_in = 320 + 15 sin(2 pi t / 600). The PID chases a moving target → the reactor temperature settles into a forced sinusoidal response (attenuated + phase-lagged). Sweep the period to build a one-point Bode by trial-and-error (the frequency-response seed).

The story: the FREQUENCY-RESPONSE seed.

Same DynamicCSTR as ctrl01/ctrl02 (compA → compB, mild exotherm, jacket heat transfer) and the same PID temperature controller. The NEW ingredient is the disturbance: instead of the step train of ctrl02's 'Schedule' controller, a 'Signal' controller injects a SINUSOID into the inlet temperature T_in:

T_in(t) = 320 + 15 * sin(2*pi*t / 600) [K]

mean = 320 K amplitude = 15 K period = 600 s (f = 1/600 = 1.667e-3 Hz, omega = 0.01047 rad/s)

The PID controller has no foreknowledge of the forcing — it only sees T_reactor drift sinusoidally away from the 350 K setpoint and works the jacket to compensate. After the first ~period the reactor temperature settles into a FORCED SINUSOIDAL RESPONSE: same period as the input, but ATTENUATED (the closed loop rejects most of the swing) and PHASE-LAGGED.

THE LESSON (Seborg/Edgar/Mellichamp/Doyle, Ch. 13-14 frequency response): read the output/input amplitude ratio [dB] and the phase lag [deg] off the steady-cycling trajectory, and you have ONE POINT of a Bode plot. Sweep the 'period' (edit one number below), re-run, and you trace the closed loop's frequency response by trial-and-error — the differentiator.

Backward-compatible by construction: ctrl02's 'type Schedule; schedule(...)' staircase still works unchanged; this case swaps it for the more general 'type Signal; signal { type sinusoidal; ... }'.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	FeedDist	MV_final	319.842923238
kpi	FeedDist	PV_final	0
kpi	FeedDist	SP_final	319.842923238
kpi	TC1	MV_final	350.026347162
kpi	TC1	PV_final	350.170890591
kpi	TC1	SP_final	350
kpi	reactor	T_final	350.173436842
kpi	reactor	n_compA_final	0.0110844237702
kpi	reactor	n_compB_final	0.000915576229809
kpi	reactor	t_end	3000
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	5.72875080707e-14
kpi	balance	mass_kg_final	0.36
kpi	balance	mass_kg_in_cum	4.5

. . . and 7 more reference values in the case's `expected`.

`ctrl07_lhhw_inhibition`

`tutorials/ctrl/ctrl07_lhhw_inhibition`

reactor
dynamicCSTR

What it does: Product inhibition meets a PID: an LHHW reaction whose exotherm fades as its own product covers the catalyst

The story: ctrl01's temperature loop, with a reaction that poisons itself.

Same vessel, same PID, same setpoint. The one change is the rate law: the product compB now ADSORBS on the catalyst, so the denominator grows as the reaction proceeds,

r = k(T) . c_A / (1 + K_B c_B)

and the reaction throttles itself. A power law cannot express that at any order; only the adsorption term can. This is the same ‘RateLaw‘ that the steady ‘cstr‘ and the ‘batchReactor‘ evaluate – one definition, three reactors.

What the controller sees: an exotherm that FADES as conversion proceeds, not a constant one. The heat the loop is rejecting shrinks under it, and the jacket temperature must follow. A tuning that holds against a first-order exotherm is not the same tuning that holds against a decaying one.

The adsorption constant K0 = 2.0 m^3/kmol is DIDACTIC – chosen so the inhibition is visible over the control horizon, not measured on any catalyst. For a real, cited LHHW see ‘cstr07_lhhw_methylAcetate‘ and ‘batch07_lhhw_methylAcetate‘ (Popken 2000, Eq 16, on UNIQUAC activities).

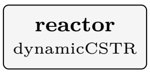
What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	TC1	MV_final	349.740705746
kpi	TC1	PV_final	348.328939265
kpi	TC1	SP_final	350
kpi	reactor	T_final	348.342145073
kpi	reactor	n_compA_final	0.0117152677233
kpi	reactor	n_compB_final	0.000284732276726
kpi	reactor	t_end	150
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	7.70988211545e-16
kpi	balance	mass_kg_final	0.36
kpi	balance	mass_kg_in_cum	0.225
kpi	balance	mass_kg_initial	0.36
kpi	balance	mass_kg_out_cum	0.225
kpi	balance	mass_residual_kg	2.77555756156e-16

. . . and 4 more reference values in the case's `expected`.

ctrl08_prbs_ident

`tutorials/ctrl/ctrl08_prbs_ident`



What it does: PRBS identification experiment: T_in = 320 +/- 10 K by a maximal-length LFSR (n=4, period 15 bits x 120 s) – deterministic, seed-declared, period-VERIFIED; the broadband excitation the sine of ctrl06 cannot give.

The story: PRBS identification experiment (forum #100.4/#101).

The classic system-identification input: a pseudo-random binary sequence on the inlet temperature, T_in = 320 +/- 10 K, driven by a MAXIMAL-LENGTH Fibonacci LFSR — n = 4 registers, canonical taps (4 3), DECLARED seed 1, period 2^4 - 1 = 15 bits of 120 s each (1800 s per full period).

Why a PRBS and not ctrl06's sine: one sinusoid excites ONE frequency; the PRBS is broadband (its autocorrelation approximates an impulse), so a single record carries the information for a full low-order model fit. And unlike real "random" noise it is DETERMINISTIC: seed + taps declared in this dict reproduce the experiment bit for bit, run after run.

THE HAND-DERIVED SEQUENCE (the exact-sequence contract of #101): with the frozen convention (output = s_n; feedback = XOR of taps; shift s_1 <- feedback; seed LSB → s_1), n=4 / taps (4 3) / seed 1 gives

0 0 0 1 0 0 1 1 0 1 0 1 1 1 1 (bits k = 0..14, then repeats)

– walk it yourself: state 0001 → 0010 → 0100 → 1001 → 0011 → 0110 → 1101 → 1010 → 0101 → 1011 → 0111 → 1111 → 1110 → 1100 → 1000 → 0001 (back at the seed after EXACTLY 15 steps: maximal). The run's announce line prints these same first bits; the initialiser REFUSES any tap set whose cycle is not exactly 2^n - 1.

The PID (same tuning as ctrl06) holds the reactor at 350 K against the binary disturbance; the trajectory's T_in column IS the PRBS, and the reactor response is the identification record.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`FeedPRBS`	`MV_final`	`330`
`kpi`	`FeedPRBS`	`PV_final`	`0`
`kpi`	`FeedPRBS`	`SP_final`	`330`
`kpi`	`TC1`	`MV_final`	`348.579621229`
`kpi`	`TC1`	`PV_final`	`350.006520504`
`kpi`	`TC1`	`SP_final`	`350`
`kpi`	`reactor`	`T_final`	`350.006465191`
`kpi`	`reactor`	`n_compA_final`	`0.0110804887195`
`kpi`	`reactor`	`n_compB_final`	`0.000919511280459`
`kpi`	`reactor`	`t_end`	`3600`
`kpi`	`balance`	`energy_balance_available`	`0`
`kpi`	`balance`	`mass_closure_rel`	`6.73638100034e-14`
`kpi`	`balance`	`mass_kg_final`	`0.36`
`kpi`	`balance`	`mass_kg_in_cum`	`5.4`

. . . and 7 more reference values in the case's `expected`.

ctrl09_stream_disturbance

tutorials/ctrl/ctrl09_stream_disturbance

reactor
dynamicCSTR

What it does: Inlet-field disturbance: componentMolarFlow.compA steps 3.5e-5 → 5.25e-5 kmol/s; F and z_in change by the DERIVED values (F = 6.75e-5, z_A = 7/9) – the disturbance never hides from the other components

The story: the inlet-field actuator, end to end.

The Signal controller targets the reactor's INLET FACE by field name ('inletField componentMolarFlow.compA') — the only legal disturbance surface (an outlet actuator is not even expressible). The AUTHORITATIVE inlet state is the per-component molar-flow vector; F_in and z_in are DERIVED from it on every write, so a composition disturbance can never hide inside a silent renormalisation (forum #101):

t < 600 s: nA = 3.5e-5, nB = 1.5e-5 kmol/s → F_in = 5.0e-5, z_A = 0.70, z_B = 0.30 t >= 600 s: nA stepped to 5.25e-5; nB UNTOUCHED at 1.5e-5 → F_in = 6.75e-5, z_A = 5.25/6.75 = 7/9 = 0.77778, z_B = 1.5/6.75 = 2/9 = 0.22222 (hand-derived)

The golden pins F_in and z_in_* finals to these fractions, and the Signal value is the ABSOLUTE molar flow (announced at start-up). The reactor's own response (more A in, higher conversion load, T transient against the PID) is the pedagogy; the CONTRACT is the derived pair.

The 'moleFraction.<c>' sibling field instead holds F_in constant and renormalises the OTHER components proportionally — two explicit semantics, each announced, never a guess.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	FeedComp	MV_final	5.25e-05
kpi	FeedComp	PV_final	0
kpi	FeedComp	SP_final	5.25e-05
kpi	TC1	MV_final	350.204904403
kpi	TC1	PV_final	350.000124372
kpi	TC1	SP_final	350
kpi	reactor	F_in_final	6.75e-05
kpi	reactor	T_final	350.000123673
kpi	reactor	n_compA_final	0.00879292389459
kpi	reactor	n_compB_final	0.00320707610541
kpi	reactor	t_end	1800
kpi	reactor	z_in_compA_final	0.777777777778
kpi	reactor	z_in_compB_final	0.222222222222
kpi	balance	energy_balance_available	0

... and 7 more reference values in the case's `expected`.

ctrl10_brine_concentration

tutorials/ctrl/ctrl10_brine_concentration

tank
dynamicCSTR

What it does: Dynamic brine tank on a Pitzer package: the electrolyte thermodynamics through a transient

The story: ONE stirred vessel, NO reaction. The dynamic CSTR tolerates an empty reaction list, and that is exactly the model wanted here: the holdup is a mixing lag, and the thermodynamics – Pitzer activity coefficients on the molality basis – is what travels through it.

The jacket loop is the same PID shape as ctrl01: it acts on T_jacket to hold the vessel temperature. A brine tank held on temperature is what sits upstream of every evaporator and crystalliser in the corpus.

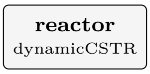
What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	TC1	MV_final	360
kpi	TC1	PV_final	310.319973652
kpi	TC1	SP_final	313.15
kpi	balance	element_Cl_closure_rel	7.17019034075e-16
kpi	balance	element_H_closure_rel	3.40011338976e-16
kpi	balance	element_Na_closure_rel	7.17019034075e-16
kpi	balance	element_O_closure_rel	3.40011338976e-16
kpi	balance	element_worst_closure_rel	7.17019034075e-16
kpi	balance	energy_balance_available	1
kpi	balance	energy_closure_rel	7.85289304221e-11
kpi	balance	energy_functional_H	1
kpi	balance	energy_kJ_final	-853632.741074
kpi	balance	energy_kJ_heat_cum	2658.78680029
kpi	balance	energy_kJ_in_cum	-87796.8114225

. . . and 17 more reference values in the case's `expected`.

ctrl11_esterification_jacket

tutorials/ctrl/ctrl11_esterification_jacket



What it does: Jacketed esterification CSTR under PID temperature control: the first ctrl case whose stored enthalpy exists, so the first-law ledger claims closure instead of refusing

The story: One jacketed vessel and one PID loop – the same shape as ctrl01, so the two can be read side by side. What differs is the chemistry: real species with a formation datum instead of numerical stand-ins, which is the whole reason this case can carry an energy claim and ctrl01 cannot.

The reaction declares NO dH_rxn. On the elements datum the heat of reaction is Sum(nu_i h_i(T)) – a consequence of the same enthalpy surface the vessel stores, not a second number to keep in step with it.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	TC1	MV_final	360.338985371
kpi	TC1	PV_final	364.93860506
kpi	TC1	SP_final	365
kpi	balance	element_C_closure_rel	1.32609972386e-14
kpi	balance	element_H_closure_rel	1.31993181817e-14
kpi	balance	element_O_closure_rel	1.32609972386e-14
kpi	balance	element_worst_closure_rel	1.32609972386e-14
kpi	balance	energy_balance_available	1
kpi	balance	energy_closure_rel	1.42432856618e-07
kpi	balance	energy_functional_H	1
kpi	balance	energy_kJ_final	-28146.0243548
kpi	balance	energy_kJ_heat_cum	-1199.4769622
kpi	balance	energy_kJ_in_cum	-65562.9346941
kpi	balance	energy_kJ_initial	-27888.8423982

. . . and 20 more reference values in the case's `expected`.

ctrl12_williams_otto

tutorials/ctrl/ctrl12_williams_otto

plant
williamsOttoPlant

What it does: Williams-Otto plant (arXiv:2004.07614 eqs. 3.6-3.11 verbatim) settling onto the published steady state $x* = (3.27, 7.47, 1.12, 9.81, 1.69, 0.22)$ klb

The story: One unit, because the benchmark IS one equation set: the Williams-Otto reactor + decanter + column + splitter fold into six ODEs (the recycle term (1-eta)mu, the removed G, the 0.1 F_tE column retention are all inside them – see the unit's own header). The five controls sit in 'operation', declared SI: with the MW 453.592 convention (1 kmol == 1 klb) every numeral below IS the paper's value in klb/h.

$u*$ of eq. (3.20): F_fA = 10, F_fB = 20, T = 580 degR, mu = 129.5, eta = 0.2. T rides in 0/internalState (322.2222 K = 580 degR exactly).

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	4.0063377475e-10
kpi	balance	mass_kg_final	10701.9131449
kpi	balance	mass_kg_in_cum	5443104
kpi	balance	mass_kg_initial	4989.512
kpi	balance	mass_kg_out_cum	5437391.60103
kpi	balance	mass_residual_kg	0.00218069130187
kpi	balance	material_available	1
kpi	plant	F_pP_klbh_final	3.90161524936
kpi	plant	F_wG_klbh_final	1.22338475064
kpi	plant	J_combined_klb_final	1071.35532885
kpi	plant	T_final	322.2222
kpi	plant	t_end	1440000
kpi	plant	waste_klb_final	481.451273663

. . . and 13 more reference values in the case's `expected`.

ctrl13_williams_otto_step

tutorials/ctrl/ctrl13_williams_otto_step

plant
williamsOttoPlant

What it does: Williams-Otto F_fB step 20→21 klb/h at t∗=100 h (paper Fig. 2): F_pP climbs ~3.90 → ~4.45 klb/h

The story: One unit, because the benchmark IS one equation set: the Williams-Otto reactor + decanter + column + splitter fold into six ODEs (the recycle term (1-eta)mu, the removed G, the 0.1 F_tE column retention are all inside them – see the unit's own header). The five controls sit in 'operation', declared SI: with the MW 453.592 convention (1 kmol == 1 klb) every numeral below IS the paper's value in klb/h.

u∗ of eq. (3.20) plus the paper's FIRST simulation experiment (section 3.2, Fig. 2): a Schedule steps F_fB from 20 to 21 klb/h at t∗ = 100 h. The MV crosses the boundary in SI (kmol/s), so the step values below are 20/3600 and 21/3600 – the paper's numbers over the one declared conversion.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	FeedStep	MV_final	0.0058333333
kpi	FeedStep	PV_final	0
kpi	FeedStep	SP_final	0.0058333333
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	1.45035580918e-09
kpi	balance	mass_kg_final	19794.220051
kpi	balance	mass_kg_in_cum	2766911.20181
kpi	balance	mass_kg_initial	4989.512
kpi	balance	mass_kg_out_cum	2752106.49778
kpi	balance	mass_residual_kg	0.00401300573503
kpi	balance	material_available	1
kpi	plant	F_pP_klbh_final	4.44568758172
kpi	plant	F_wG_klbh_final	1.90791543668
kpi	plant	J_combined_klb_final	529.364471374

. . . and 10 more reference values in the case's `expected`.

`ctrl14_williams_otto_pi`

`tutorials/ctrl/ctrl14_williams_otto_pi`

What it does: Williams-Otto PI channel (F_fB, F_pP), paper eq. 4.6 with the section-4.1 tunings: F_pP tracks 3.90 $\rightarrow$ 4.5 klb/h with the Fig. 4 overshoot

The story: One unit, because the benchmark IS one equation set: the Williams-Otto reactor + decanter + column + splitter fold into six ODEs (the recycle term (1-eta)mu, the removed G, the 0.1 F_tE column retention are all inside them – see the unit's own header). The five controls sit in 'operation', declared SI: with the MW 453.592 convention (1 kmol == 1 klb) every numeral below IS the paper's value in klb/h.

The paper's control channel (F_fB, F_pP), eq. (4.6), with the Skogestad-IMC gains of section 4.1: K3prop = 0.069 (dimensionless – klb/h per klb/h survives the SI conversion unchanged) and K3int = 1.282 per HOUR, declared per second (1.282/3600 = 3.5611111e-4). Bias = F∗_fB (bumpless), setpoint F_pP = 4.5 klb/h = 1.25e-3 kmol/s, output bounded to the paper's 0 <= F_fB <= 56 klb/h.

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`PC1`	`MV_final`	`0.00586183454679`
`kpi`	`PC1`	`PV_final`	`0.00124999896025`
`kpi`	`PC1`	`SP_final`	`0.00125`
`kpi`	`balance`	`energy_balance_available`	`0`
`kpi`	`balance`	`mass_closure_rel`	`2.05158151889e-09`
`kpi`	`balance`	`mass_kg_final`	`21425.7971042`
`kpi`	`balance`	`mass_kg_in_cum`	`1418738.01455`
`kpi`	`balance`	`mass_kg_initial`	`10701.9131449`
`kpi`	`balance`	`mass_kg_out_cum`	`1408014.1335`
`kpi`	`balance`	`mass_residual_kg`	`0.0029106566908`
`kpi`	`balance`	`material_available`	`1`
`kpi`	`plant`	`F_pP_klbh_final`	`4.49999626428`
`kpi`	`plant`	`F_wG_klbh_final`	`2.00328828826`
`kpi`	`plant`	`J_combined_klb_final`	`255.611908728`

... and 10 more reference values in the case's `expected`.

ctrl16_williams_otto_optimal

tutorials/ctrl/ctrl16_williams_otto_optimal

plant
williamsOttoPlant

What it does: Williams-Otto section-5.3 combined optimisation: Nelder-Mead over two K=5 Schedule profiles, J measured against the published collocation optimum 551.8

The story: One unit, because the benchmark IS one equation set: the Williams-Otto reactor + decanter + column + splitter fold into six ODEs (the recycle term (1-eta)mu, the removed G, the 0.1 F_tE column retention are all inside them – see the unit's own header). The five controls sit in 'operation', declared SI: with the MW 453.592 convention (1 kmol == 1 klb) every numeral below IS the paper's value in klb/h.

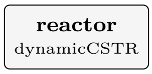
The section-5.3 combined optimisation: system/outerDict shapes the two profiles below to maximise J = yield - waste (alpha = beta = 1, eq. 5.7), read straight off the plant's J_combined_klb_final KPI.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	FfBprof	MV_final	0.00463988515428
kpi	FfBprof	PV_final	0
kpi	FfBprof	SP_final	0.00463988515428
kpi	Tprof	MV_final	300.373635533
kpi	Tprof	PV_final	0
kpi	Tprof	SP_final	300.373635533
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	1.14401865779e-07
kpi	balance	mass_kg_final	723583.798797
kpi	balance	mass_kg_in_cum	2128628.70559
kpi	balance	mass_kg_initial	10701.9131449
kpi	balance	mass_kg_out_cum	1415747.06346
kpi	balance	mass_residual_kg	0.243519095471
kpi	balance	material_available	1
. . . and 40 more reference values in the case's `expected`.			

ctrl17_signal_shapes

tutorials/ctrl/ctrl17_signal_shapes



What it does: WITNESS: ramp + pulse + staircase signals on three feed handles, with the temperature PID rejecting all three at once; the three registered shapes no case selected.

The story: WITNESS: the ramp, pulse and staircase SIGNALS, each on its own feed handle of the ctrl09 CSTR, with the ctrl09 temperature PID still in the loop rejecting all three at once – compA ramps +28.6 % over 1000 s, compB takes a 300 s rectangular pulse, the feed temperature walks a three-step staircase. These three registered signal shapes were selected by no case until 2026-08-23 (invariant I8). Self-recorded.

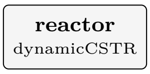
What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	FeedPulseB	MV_final	1.5e-05
kpi	FeedPulseB	PV_final	0
kpi	FeedPulseB	SP_final	1.5e-05
kpi	FeedRampA	MV_final	4.5e-05
kpi	FeedRampA	PV_final	0
kpi	FeedRampA	SP_final	4.5e-05
kpi	FeedStairT	MV_final	348
kpi	FeedStairT	PV_final	0
kpi	FeedStairT	SP_final	348
kpi	TC1	MV_final	347.378942902
kpi	TC1	PV_final	349.993146257
kpi	TC1	SP_final	350
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	2.98582911041e-14

...and 13 more reference values in the case's `expected`.

ctrl17_step_reaction_curve

tutorials/ctrl/ctrl17_step_reaction_curve



What it does: Open-loop process reaction curve: the jacket steps +5, -5, +20 and -20 K from one settled base, and the four responses are the identification record for the FOPDT fit in system/propsDict

The story: THE IDENTIFICATION, run over the trajectory the choupoCtrl half of this case wrote. Run the experiment first:

runCase tutorials/ctrl/ctrl17_step_reaction_curve choupoProps tutorials/ctrl/ctrl17_step_reaction_curve

TWO OPERATIONS OVER ONE EXPERIMENT, and the pair is the lesson. Both read the SAME four steps of the SAME jacket temperature; they differ only in which response they call the controlled variable.

[0] the COMPOSITION loop (jacket → product holdup). Two lags in series – the jacket heats the liquid, and the warmer liquid reacts faster – so the response is S-shaped and an FOPDT surrogate finds a real apparent dead time. All three tuning rules apply.

[1] the TEMPERATURE loop (jacket → reactor temperature). One dominant lag and no transport delay, so the identified theta comes out smaller than the trajectory's own sampling interval. The engine says so, and REFUSES every Ziegler-Nichols and Cohen-Coon setting by name: both rules put theta in a denominator, and a controller gain computed from an unresolved number is a number with nothing behind it. IMC survives, because its algebra never divides by theta – which is exactly why it is the rule of choice for a lag-dominant loop.

Nothing was tuned to make that happen. It is one plant, one experiment, and two honest answers, and the difference between them is the whole reason the reaction-curve method needs a dead time to work at all.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	JacketStep	MV_final	320
kpi	JacketStep	SP_final	320
kpi	reactor	T_final	320.759149686
kpi	reactor	n_compA_final	0.0117959393424
kpi	reactor	n_compB_final	0.000204060657576
kpi	reactor	F_in_final	5e-05
kpi	reactor	t_end	15000
kpi	JacketStep	PV_final	0
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	9.91527847481e-14
kpi	balance	mass_kg_final	0.36
kpi	balance	mass_kg_in_cum	22.5
kpi	balance	mass_kg_initial	0.36
kpi	balance	mass_kg_out_cum	22.5

... and 4 more reference values in the case's `expected`.

ctrl18_rtd_pulse_cstr

tutorials/ctrl/ctrl18_rtd_pulse_cstr

reactor
dynamicCSTR

What it does: RTD by pulse tracer on a pure mixing tank: tracer substitution at constant flow, tau = N/F = 240 s; engine-measured moments against Levenspiel closed-form anchors.

The story: RESIDENCE-TIME DISTRIBUTION by pulse tracer – the engine's first RTD tool (2026-08-23). A pure mixing tank (the ctrl09 vessel, reaction and controller stripped) holds N = 0.012 kmol and is fed F = 5e-5 kmol/s, so tau = N/F = 240 s exactly. For 30 s starting at t = 100 s the feed SWAPS 1e-5 kmol/s of carrier for tracer – compB pulses up by exactly what compA pulses down – so the total flow never moves and the vessel is rigorously linear and time-invariant in the tracer.

THE ANCHORS ARE CLOSED FORMS (Levenspiel, Chemical Reaction Engineering, ch. 13-14 – moment additivity of a linear vessel, exact for ANY inlet shape): tbar_out = tbar_in + tau = (100 + 15) + 240 = 355 s sigma2_out = sigma2_in + tau^2 = 30^2/12 + 240^2 = 57675 s^2 area = amplitude x width = 1e-5 ∗ 30 = 3.0e-4 kmol The engine measures the outlet response moments (rtd KPIs, trapezoid on the accepted-step grid, reports/rtd/E.csv); the arithmetic above is the case's, stated here. Deviations from the anchors are integration truncation (run = 2600 s ~ 10.4 tau past the pulse; e^-10.4 ~ 3e-5) plus the finite step – both stated, neither hidden.

What to expect (golden-locked):

kind	unit/stream	key	expected
kpi	CarrierDip	MV_final	5e-05
kpi	CarrierDip	PV_final	0
kpi	CarrierDip	SP_final	5e-05
kpi	TracerPulse	MV_final	0
kpi	TracerPulse	PV_final	0
kpi	TracerPulse	SP_final	0
kpi	balance	energy_balance_available	0
kpi	balance	mass_closure_rel	4.73695157173e-14
kpi	balance	mass_kg_final	0.36
kpi	balance	mass_kg_in_cum	3.9
kpi	balance	mass_kg_initial	0.36
kpi	balance	mass_kg_out_cum	3.9
kpi	balance	mass_residual_kg	1.84741111298e-13
kpi	balance	material_available	1

. . . and 15 more reference values in the case's `expected`.

`ctrl19_freq_response_cstr`

`tutorials/ctrl/ctrl19_freq_response_cstr`

reactor
dynamicCSTR

What it does: Frequency response of the LTI mixing tank at omega∗tau = 1: engine-fitted amplitude and phase against the exact first-order closed forms (AR = 1/sqrt(2), phase = -45 deg).

The story: FREQUENCY RESPONSE of the same LTI mixing tank (tau = N/F = 240 s): the tracer feed oscillates sinusoidally at omega∗tau = 1 – again as a SUBSTITUTION against the carrier, so total flow never moves and the vessel is rigorously first order in the tracer. The engine fits a + b sin + c cos over 4 cycles after discarding 5 (transient e^-t/tau through 5 periods = e^-31 – gone), and reports amplitude, phase and the unexplained-variance residual.

THE ANCHORS ARE THE FIRST-ORDER CLOSED FORMS (any process-control text; at omega∗tau = 1 exactly): AR = 1/sqrt(1 + (omega tau)^2) = 1/sqrt(2) → out amplitude = 2.0e-6 ∗ 0.7071068 = 1.4142136e-6 kmol/s phase = -atan(omega tau) = -45 deg = -0.7853982 rad (the drive is sin(w(t-tStart)) with tStart 0, so the fitted phase IS the lag). The measured amplitude lands within 1e-6 relative of the closed form and the fit residual is ~7e-15 (the vessel really is LTI); the phase carries a bias of EXACTLY omega∗dt/2 = 0.00208 rad = 0.12 deg – the half-sample lag of reading accepted states on a 1 s grid, the measurement system's own phase contribution, worth teaching in itself. The anchor band covers it and this header names it.

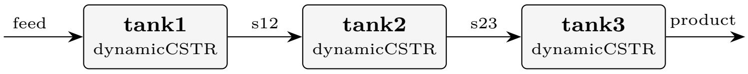
What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`CarrierMirror`	`MV_final`	`4.41011922784e-05`
`kpi`	`CarrierMirror`	`PV_final`	`0`
`kpi`	`CarrierMirror`	`SP_final`	`4.41011922784e-05`
`kpi`	`TracerSine`	`MV_final`	`5.89880772158e-06`
`kpi`	`TracerSine`	`PV_final`	`0`
`kpi`	`TracerSine`	`SP_final`	`5.89880772158e-06`
`kpi`	`balance`	`energy_balance_available`	`0`
`kpi`	`balance`	`mass_closure_rel`	`2.91033739366e-15`
`kpi`	`balance`	`mass_kg_final`	`0.36`
`kpi`	`balance`	`mass_kg_in_cum`	`20.8095`
`kpi`	`balance`	`mass_kg_initial`	`0.36`
`kpi`	`balance`	`mass_kg_out_cum`	`20.8095`
`kpi`	`balance`	`mass_residual_kg`	`-6.05626659933e-14`
`kpi`	`balance`	`material_available`	`1`

. . . and 14 more reference values in the case's `expected`*.*

`ctrl19_tanks_in_series`

`tutorials/ctrl/ctrl19_tanks_in_series`



What it does: Tanks-in-series RTD: three equal mixing tanks chained by the ctrl router; impulse in tank 1 → Erlang-3 response at tank 3 (tbar = 3 tau, sigma2/tbar2 = 1/3), Levenspiel closed forms as anchors.

The story: the SECOND HALF of the RTD lesson, unlocked by the ctrl router (ruled 2026-08-24): three EQUAL mixing tanks in series, each tau = n/F = 0.004/5e-5 = 80 s, chained by the same 'in'/'outputs' topology grammar every steady flowsheet already uses. The router applies each connection ONE-STEP-EXPLICITLY: at every accepted driver step the downstream tank integrates on the upstream outlet at the step's START – a transport delay of one driver step (1 s here, against tau = 80 s), announced by the [routing] lines at startup.

THE EXPERIMENT. The tracer impulse is an INITIAL-CONDITION substitution in tank 1 only (compB 0.0002 of the 0.004 kmol holdup – substitution, not addition, so every tank holds exactly tau = 80 s and the closed forms are exact). An initial holdup m in tank 1 is mathematically a perfect impulse into tank 1 at t = 0, so tank 3's outlet is the Erlang-3 density:

E(t) = t^2 e^(-t/tau) / (2 tau^3) tbar = 3 tau = 240 s sigma2 = 3 tau^2 = 19200 s^2 (sigma2/tbar2 = 1/3 exactly) peak at 2 tau = 160 s (visible in trajectory.csv)

Levenspiel, Chemical Reaction Engineering, ch. 14 (the tanks-in-series model): the variance ratio 1/N is THE diagnostic the lesson teaches – one tank gives 1, three give 1/3, a PFR gives 0. ctrl18 is the N = 1 control; this case is N = 3 on identical tanks.

WHAT THE ANCHORS TOLERATE, stated: the engine measures moments by trapezoid on the accepted 1 s grid and truncates at 1600 s (= 20 tau of one tank); both effects are far inside the anchor bands, and the router adds a one-step transport delay per connection (2 s total on tbar's 240 s – inside the 1 % band, and the honest price of an explicit scheme, stated rather than hidden).

What to expect (golden-locked):

kind	unit/stream	key	expected
`kpi`	`balance`	`energy_balance_available`	`0`
`kpi`	`balance`	`mass_closure_rel`	`2.15876699233e-16`
`kpi`	`balance`	`mass_kg_final`	`0.36`
`kpi`	`balance`	`mass_kg_in_cum`	`7.2`
`kpi`	`balance`	`mass_kg_initial`	`0.36`
`kpi`	`balance`	`mass_kg_out_cum`	`7.2`
`kpi`	`balance`	`mass_residual_kg`	`-1.55431223448e-15`
`kpi`	`balance`	`material_available`	`1`
`kpi`	`rtd`	`area_kmol`	`0.000198752512114`
`kpi`	`rtd`	`sigma2_s2`	`19198.7859557`
`kpi`	`rtd`	`tbar_s`	`241.502454857`
`kpi`	`rtd`	`truncation_final_over_peak`	`1.54894525802e-06`
`kpi`	`tank1`	`F_in_final`	`5e-05`
`kpi`	`tank1`	`T_final`	`330`
. . . and 23 more reference values in the case's `expected`.			

ctrl20_bode_cstr

`tutorials/ctrl/ctrl20_bode_cstr`

reactor
dynamicCSTR

What it does: Bode table of the LTI mixing tank: outer frequency sweep (one datum, lockstep mirror) over the engine frequency-response fit; anchors are the exact first-order closed forms at each point.

The story: THE BODE TABLE of the LTI mixing tank (tau = 240 s): an outer sweep drives the sinusoidal tracer substitution at four frequencies, omega∗tau = 0.5, 2.0, 3.5, 5.0 (linear grid in f), and the engine's frequencyResponse fit measures amplitude and phase at each – the sweep CSV IS the Bode table. The measurement block references the drive, so each pass's fit window adapts to that pass's own period, and the sweep varies ONE datum (the drive frequency, mirrored to the carrier by linkedTargets – the same value, lockstep, never a second independent number).

ANCHORS: the exact first-order closed forms at each grid point, AR = 1/sqrt(1+(wt)^2), phase = -atan(wt): wt=0.5: |out| = 2e-6∗0.894427 = 1.788854e-6, phi = -0.463648 rad wt=2.0: |out| = 2e-6∗0.447214 = 8.944272e-7, phi = -1.107149 rad wt=3.5: |out| = 2e-6∗0.274721 = 5.494423e-7, phi = -1.292497 rad wt=5.0: |out| = 2e-6∗0.196116 = 3.922323e-7, phi = -1.373401 rad Phase bands cover the omega∗dt/2 half-sample bias, which grows with omega (0.10 to 1.0 mrad here) – stated, not hidden.

What to expect (golden-locked):

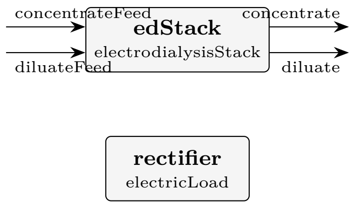
kind	unit/stream	key	expected
csv	sweep_results.csv	0:frequencyResponse.out_amplitude_kmol_s	1.788854e-6
csv	sweep_results.csv	1:frequencyResponse.out_amplitude_kmol_s	8.944272e-7
csv	sweep_results.csv	2:frequencyResponse.out_amplitude_kmol_s	5.494423e-7
csv	sweep_results.csv	3:frequencyResponse.out_amplitude_kmol_s	3.922323e-7
csv	sweep_results.csv	0:frequencyResponse.out_phase_rad	-0.463648
csv	sweep_results.csv	1:frequencyResponse.out_phase_rad	-1.107149
csv	sweep_results.csv	2:frequencyResponse.out_phase_rad	-1.292497
csv	sweep_results.csv	3:frequencyResponse.out_phase_rad	-1.373401
kpi	CarrierMirror	MV_final	4.79049069948e-05
kpi	CarrierMirror	PV_final	0
kpi	CarrierMirror	SP_final	4.79049069948e-05
kpi	TracerSine	MV_final	2.09509300524e-06
kpi	TracerSine	PV_final	0
kpi	TracerSine	SP_final	2.09509300524e-06

. . . and 20 more reference values in the case's `expected`*.*

Category V: Electrochemical systems

ed01_nacl_desalination

tutorials/electrochem/ed01_nacl_desalination



What it does: Electrodialysis stack: NaCl diluate desalination, 100 cell pairs

The story: ED01 NaCl desalination by electrodialysis

An electrodialysis stack of 100 cell pairs desalinates a dilute NaCl diluate stream (0.1 mol/kg) against a more-concentrated concentrate (0.5 mol/kg). A specified stack current (BELOW the limiting current) drives Na+ through the CMX (cation-exchange) membranes and Cl- through the AMX (anion-exchange) membranes, OUT of the diluate and INTO the concentrate.

diluateFeed → +——————+ → diluate (desalinated) | ED stack | concentrateFeed → | (CMX/AMX, N=100)| → concentrate (enriched) +——————+ | (power, energy wire) v rectifier (electricLoad)

The stack PRINTS (verbosity 3) the Faraday transfer, the Nernst membrane potential from REAL Davies activities, the Cowan-Brown limiting current i_lim (and i/i_lim), the ohmic drop and the stack voltage/power.

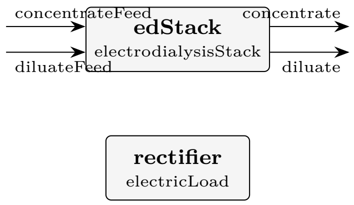
What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`concentrate`	`F`	`0.00501492454831`
`stream`	`concentrateFeed`	`F`	`0.005`
`stream`	`diluate`	`F`	`0.00498507545169`
`stream`	`diluateFeed`	`F`	`0.005`
`kpi`	`edStack`	`E_mem_pair`	`0.0794573845707`
`kpi`	`edStack`	`I`	`8`
`kpi`	`edStack`	`N_cellpairs`	`100`
`kpi`	`edStack`	`R_pair`	`0.00507753048751`
`kpi`	`edStack`	`U`	`13.5077628471`
`kpi`	`edStack`	`W_electric`	`108.062102777`
`kpi`	`edStack`	`W_electric_kW`	`0.108062102777`
`kpi`	`edStack`	`current_efficiency`	`0.9`
`kpi`	`edStack`	`demin_ratio`	`0.831450327631`
`kpi`	`edStack`	`i_density`	`40`

. . . and 17 more reference values in the case's `expected`.

`ed02_over_limiting_current`

`tutorials/electrochem/ed02_over_limiting_current`



What it does: Electrodialysis stack driven ABOVE its limiting current (loud warning)

The story: ED02 over-limiting current electrodialysis

Same stack as ED01, but run on a MORE DILUTE diluate (0.02 mol/kg) at a LOW cross-flow velocity and a HIGH current. The diluate-side limiting current density i__lim (Cowan-Brown) is small; the applied current density EXCEEDS it, so the stack runs in the OVER-LIMITING regime.

Choupo does NOT silently clamp the current (the "no silent crutch" credo): it computes the unclamped Faraday transfer and prints a LOUD warning that i > i__lim – the diluate boundary layer is ion-depleted, water splitting / pH swings set in, and the real current efficiency would collapse. The student SEES the regime and the i/i__lim ratio, then chooses to reduce the current or raise the flow / feed concentration.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`concentrate`	`F`	`0.00500205212539`
`stream`	`concentrateFeed`	`F`	`0.005`
`stream`	`diluate`	`F`	`0.0199979478746`
`stream`	`diluateFeed`	`F`	`0.02`
`kpi`	`edStack`	`E_mem_pair`	`0.156590406589`
`kpi`	`edStack`	`I`	`5.5`
`kpi`	`edStack`	`N_cellpairs`	`20`
`kpi`	`edStack`	`R_pair`	`0.0130038886532`
`kpi`	`edStack`	`U`	`6.06223588363`
`kpi`	`edStack`	`W_electric`	`33.34229736`
`kpi`	`edStack`	`W_electric_kW`	`0.03334229736`
`kpi`	`edStack`	`current_efficiency`	`0.9`
`kpi`	`edStack`	`demin_ratio`	`0.142508707778`
`kpi`	`edStack`	`i_density`	`27.5`

. . . and 17 more reference values in the case's `expected`.

Category VI: Multi-sector plant showcases

The fractal flowsheet at full size – read them like worked examples.

ChemicalPlantTutorial

The theory you need. The full multi-sector plant showcase: feed treatment, reaction, separation, utilities – the fractal flowsheet at its largest, run-only (no golden), for reading and dissecting rather than reproducing numbers.

CONCENTRATION

tutorials/plant/ChemicalPlantTutorial/CONCENTRATION

The story: / system/flowsheetDict (SECTOR – COMPOSITE)

Double-effect evaporator (vapour of effect 1 heats effect 2) feeding a crystalliser. Same shape as the plant flowsheetDict, one level down: children + boundary + default feeds + connections.

DRYING

tutorials/plant/ChemicalPlantTutorial/DRYING

The story: / system/flowsheetDict (SECTOR – COMPOSITE)

Spray dryer → additional solid drying → cyclone dust separator. The solid powder goes SD → BD → product; the spray-dryer exhaust air (carrying fines) goes SD → CY, which returns clean air + recovered dust.

FERMENTATION

`tutorials/plant/ChemicalPlantTutorial/FERMENTATION`

The story: / system/flowsheetDict (SECTOR – COMPOSITE with INTERNAL RECYCLE)

Continuous fermentation side-loop of the sugar plant: a slip-stream of fresh juice (Must) is hydrolysed + fermented into ethanol + CO2, the Flash splits the volatiles (EthanolVapour product) from the liquid bottoms, and a Splitter recycles 80 % of the bottoms back to the Mixer so unconverted sucrose gets another shot. Pedagogical purpose:

1. Reaction (Arrhenius first-order in sucrose). 2. Vapour-liquid flash for separation. 3. Tear-stream RECYCLE that the Wegstein/Newton outer loop must iterate to a fixed point.

Must [Mixer] MixedFeed [Fermentor] FermOut [Flash] EthanolVapour Liquid Recycle [Splitter]

Tear stream: Recycle (Splitter output → Mixer input).

`JuiceSplitter`

`tutorials/plant/ChemicalPlantTutorial/JuiceSplitter`

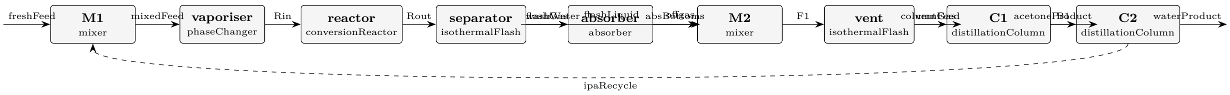
The story: / system/flowsheetDict (LEAF – plant-level juice split)

Diverts the fresh sugar-cane juice between the two production lines of the mill: 2/3 goes to the CONCENTRATION line (evaporators → crystals → dry sugar) and 1/3 to the FERMENTATION line (yeast → ethanol). This is what a real Brazilian sugar-ethanol plant does after the clarifier: the same juice header feeds both products, with the split ratio set by market price + crop quality.

Pedagogically the splitter is the single inter-sector cable that ties the two downstream branches together – without it, FERMENTATION would be a parallel island.

acetonePlant

tutorials/plant/acetonePlant



What it does: Acetone from 2-propanol: Luyben's Figure 1 closed, with the IPA recycle

The story: ACETONE FROM 2-PROPANOL, THE CLOSED FLOWSHEET

W. L. Luyben, "Design and Control of the Acetone Process via Dehydrogenation of 2-Propanol", Ind. Eng. Chem. Res. 50 (2011) 1206-1218, DOI 10.1021/ie901923a – his Figure 1 (the Turton base case), end to end:

fresh feed + IPA recycle → vaporiser → reactor → separator → absorber (offgas washed) → C1 → acetone product → C2 → water product → IPA RECYCLE (tear)

This is the case the six preceding ones were built to make possible. Each of them isolates one unit on Luyben's OWN published inlet, so that whatever it gets wrong is that unit and its thermodynamics. Here nothing is published except the fresh feed: every internal stream is the plant's own answer, and the errors compose.

WHAT THIS CASE CLAIMS, AND WHAT IT DOES NOT. It claims to be Luyben's TOPOLOGY and Luyben's EQUIPMENT SPECIFICATIONS solved on the thermodynamics Choupo actually has. It does NOT claim to reproduce his stream table – the siblings measured, in advance, exactly why it cannot, and the gap between his table and this one is the deliverable rather than a defect to be closed:

acetone01: UNIFAC puts the IPA/water azeotrope at x 0.590 against his 0.673, and the lake-estimated isopropanol boils 4.5 K high. acetone04: UNIFAC INVENTS an acetone/water azeotrope at x 0.978 where his UNIQUAC has none – 0.044 K deep, and decisive. acetone06: so C1 caps at 97.3 mol % acetone against his 99.9 % spec. acetone07: so C2's recycle caps at x_IPA 0.580 against his 0.650, and 11.6 % of the isopropanol entering it leaves in the water product.

Both column ceilings were WRITTEN DOWN BEFORE the columns existed and are pinned by check_azeotrope_ceiling. This case is where they stop being two separate findings and become one plant.

DEPARTURES FROM HIS FIGURE 1, each argued rather than absorbed

(a) NO HYDROGEN IN THE COLUMNS. His C1 has a partial condenser and a vent (0.0465 kmol/h, carrying 0.0130 kmol/h H2 and 0.0335 kmol/h acetone); Choupo's column has a total condenser and no vapour distillate. A permanent gas fed to a total condenser is asked to leave as a liquid, and the engine rightly refuses (hydrogen has no liquid heat capacity at 330 K). So the separator's LIQUID goes to C1 and the hydrogen leaves with the offgas, i.e. the columns are modelled downstream of his vent. Cost, stated: the 0.0335 kmol/h of acetone his vent loses is not lost here, which flatters this plant's acetone recovery by 0.10 %.

(b) THE SEPARATOR DOES THE COOLING. An isothermal flash at 318 K computes the duty to bring its feed there, so this one unit stands for his HX1 plus his separator and its Q is directly comparable to his 0.8993 MW. A separate 'heater' was tried in acetone03 and refused, correctly: Choupo's heater takes a DUTY and reports the temperature, because a heater with a specified outlet temperature is a design specification.

(c) THE SEPARATOR PRESSURE IS ARGUED, NOT FITTED. Luyben states 1.5 atm for the absorber gas, which is downstream; there is no compressor between reactor and separator and pressure only falls, so the separator sits at the reactor's 2.6 atm. acetone03 records the sweep that would have tempted a fitted value and why it was refused.

(d) THE ABSORBER'S SECOND OUTLET REJOINS C1's FEED, as in his Figure 1 (his F1 of 72.85 kmol/h is the flash liquid AFTER the absorber bottoms of 20.05 kmol/h rejoin it).

(e) NO VENT/PURGE ON THE RECYCLE LOOP. Luyben has none either – the loop closes through C2's overhead and nothing else – so this is his structure, not a simplification. It does mean any component that both enters and cannot leave would accumulate without bound; here water leaves in three places and isopropanol in two, so the loop is closed and bounded.

These are the dictionaries of the closed acetone flowsheet described in ../acetone-plant-closure-state.md. It does not converge yet, and the reason is one named engine gap (F3 in that record: no unit can remove a component completely).

tutorials/ is the verification and regression corpus. Everything in it is run by bin/runTests and is expected to answer. Putting a case there that cannot answer would have meant claiming one of two markers, and neither is true of this case:

∗ .expect-nonconvergence — every one of the five cases carrying it is a DELIBERATE TEACHING REFUSAL (flash15_refused_salt_basis_rank, tsa02_refused_capacity, ?): the refusal ∗is∗ the lesson, and the case is finished. This one is not refusing anything on purpose; it is unfinished. ∗ .known-broken — the corpus carries ZERO of these, and that zero is a claim the project makes about its own health. Spending it to park work in progress would be paying a real claim for my convenience.

So the case sits beside the record that explains it, makes no corpus claim, and moves to tutorials/plant/acetonePlant/ the day it converges — at which point it gets a golden, a seal and an entry in the case manifest like any other case.

0/ is NOT kept: it is generated by bin/choupo-init0 from the authored inlets plus the tear seed, and a stale copy of a generated state is a second home for numbers the tool derives. The tear seed itself — the one part of 0/ that is authored rather than propagated — is described in the closure record.

The absorber's second thermodynamic world is declared inline in system/flowsheetDict as a per-unit thermo {} block, not as a separate file, because that is where the model boundary belongs and where the run announces it.

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	B1	F	0.0118946232842
stream	F1	F	0.0195617446099
stream	Rin	F	0.0160666666667
stream	Rout	F	0.0255036799795
stream	absBottoms	F	0.00608647733675
stream	acetoneProduct	F	0.00762772388347
stream	columnFeed	F	0.0195223471677
stream	flashGas	F	0.0120284127064
stream	flashLiquid	F	0.0134752672731
stream	freshFeed	F	0.0144333333333
stream	ipaRecycle	F	0.00163333333333
stream	mixedFeed	F	0.0160666666667
stream	offgas	F	0.0114974909252
stream	ventGas	F	3.9397442245e-05

. . . and 155 more reference values in the case's `expected`.

`esterification2sector`

`tutorials/plant/esterification2sector`

What it does: Esterification plant – two sectors (REACTION + SEPARATION) sharing components; per-sector constant/ data (S1 kinetics, S2 NRTL pair); ethanol-water pair inherited from the standard library

A two-sector plant that exists to prove one thing: data lives with the sector that owns it, and a whole-plant run finds it there.

Components are shared across the plant. The kinetics belong to REACTION and are never copied to the root. The ethylAcetate-water NRTL pair belongs to SEPARATION — a ∗particular∗ datum, fitted for this separation — while ethanol-water walks up to the standard catalogue. Three levels of ownership, one run.

That perNode is the whole case. The sector's pair beat the standard library, and the log says so rather than leaving you to wonder which one it used.

Feed 100 kmol/h equimolar acetic acid + ethanol at 360 K. The CSTR (V_R = 0.5 m^3) reaches 34.2 % conversion — a PLACEHOLDER number: the kinetics in sectors/REACTION/constant/reactions are still the pseudo-first-order stand-in, not the 2nd-order fit this case's own property evidence supports (see CLAUDE.md, "Pending"). The flash at 355 K, 1.013 bar splits the product into a vapour rich in ethyl acetate (bp 350 K) and ethanol, and a liquid rich in water and the unconverted acid. Mass closes exactly: 34.18 + 65.82 = 100.00 kmol/h.

This case shipped for weeks marked .known-broken, and it earned the mark twice over. Its flash carried P 1.01325; with no unit, which is 1.01 pascal — a hard vacuum — so the K-values exploded and everything went to the vapour. And it was never wired: the sectors existed as thermo regions with the connections commented out, awaiting a "curation phase" that never ended.

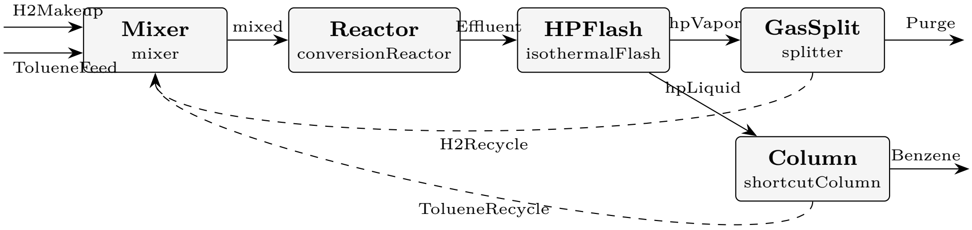
Fixing it surfaced two real engine bugs, both of the same shape — data owned by a ∗sector∗ was invisible from a whole-plant run:

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`Feed`	`F`	`0.0277777777778`
`stream`	`liquid`	`F`	`0.0114069310496`
`stream`	`reactorOut`	`F`	`0.0277777777778`
`stream`	`vapor`	`F`	`0.0163708467282`
`utility`	`SEPARATION.flash`	`cooling.coolingWater.duty_kW`	`-229.173881338`
`utility`	`SEPARATION.flash`	`cooling.coolingWater.T`	`355`
`utility`	`SEPARATION.flash`	`cooling.coolingWater.kg_s`	`5.48262874014`
`utility`	`SEPARATION.flash`	`cooling.coolingWater.eur_h`	`0.330010389127`
`stream`	`Feed`	`T`	`360`
`stream`	`liquid`	`T`	`355`
`stream`	`reactorOut`	`T`	`360`
`stream`	`vapor`	`T`	`355`

hda

tutorials/plant/hda



What it does: HDA plant – toluene + H2 → benzene + CH4. A single FLAT flowsheet with two plant-level recycles (an H2 gas loop and an unreacted-toluene liquid loop); cross-sector tears are unsupported, so it is not fractal. Furnace lumped into the isothermal reactor; light-gas K-values rest on Antoine extrapolation (see thermoPhysPropDict).

What to expect (golden-locked):

kind	unit/stream	key	expected
stream	Benzene	F	0.0308570333668
stream	Effluent	F	0.589422462034
stream	H2Makeup	F	0.0305555555556
stream	H2Recycle	F	0.522049697299
stream	Purge	F	0.0274762998579
stream	TolueneFeed	F	0.0277777777778
stream	TolueneRecycle	F	0.00903943150968
stream	hpLiquid	F	0.0398964648764
stream	hpVapor	F	0.549525997157
stream	mixed	F	0.589422462034
kpi	Column	B	0.00903943150968
kpi	Column	D	0.0308570333668
kpi	Column	N_min	8.3953506183
kpi	Column	N_theoretical	20.1717591626
. . . and 48 more reference values in the case's `expected`*.*			

`lithiumBrinePlant`

`tutorials/plant/lithiumBrinePlant`

What it does: lithiumBrinePlant – THE reference case: a FRACTAL lithium-carbonate plant from salar brine, WIRED into one solver. Five SECTORS, each a thermo WORLD of its own – BRINE (electrolyte/Pitzer), EXTRACTION (gamma-gamma NRTL LLE), CARBONATION (Li2CO3 retrograde solubility), HOTAIR (Gibbs combustion), FINISHING (drying + cyclone) – connected sector to sector, each sector declaring its own propertyPackage (its world) on the global component set, with optional per-unit thermo overrides where a single unit needs a different world. The proof that the architecture is fractal: sectors are composite boxes, streams flow between them, model boundaries audited.

The story: Five REAL sectors. Streams are named physical graph edges. Sector names are subdomains; unit operations live inside each sector.

What to expect (golden-locked):

kind	unit/stream	key	expected
`stream`	`carbLiquor`	`F`	`0.0277777777778`
`stream`	`cleanAir`	`F`	`0.0320991487231`
`stream`	`emulsion`	`F`	`0.0380203541089`
`stream`	`fines`	`F`	`0`
`stream`	`fuelAir`	`F`	`0.0108555555558`
`stream`	`halite`	`F`	`0.000868534779985`
`stream`	`hotAir`	`F`	`0.010855555556`
`stream`	`humidExhaust`	`F`	`0.0320991487231`
`stream`	`liquor`	`F`	`0.0269092429978`
`stream`	`loadedOrganic`	`F`	`0.0124069292995`
`stream`	`organic`	`F`	`0.0111111111111`
`stream`	`product`	`F`	`0.00653418461064`
`stream`	`raffinate`	`F`	`0.0256134248094`
`stream`	`salarBrine`	`F`	`0.0277777777778`

. . . and 16 more reference values in the case's `expected`.

polycaprolactonePlant

The theory you need. Ring-opening polymerisation plant: monomer to polycaprolactone with step-growth KPIs (Carothers, Flory) on the PFR, polymer properties from group contribution on the repeat unit.

properties

tutorials/plant/polycaprolactonePlant/properties

pclRepeat__Tg__yang2020
estimateComponent

What it does: Close the loop: ESTIMATE the PCL repeat-unit glass transition Tg by the Yang 2020 group-contribution scheme — the plant gives Mn/PDI, this gives the physical property from the SAME repeat unit -[O-(CH2)5-CO]-

The story: pclProperties – CLOSE THE LOOP: process → molar mass → physical property.

The plant (../) gave you the chain statistics from the KINETICS: Mn, Mw, PDI on the polycaprolactone it makes. Here you ESTIMATE the polymer's PHYSICAL PROPERTIES from the SAME repeat unit -[O-(CH2)5-CO]- by group contribution — the bridge from "how big are the chains" to "how does the material behave".

TWO estimators, two properties, on the repeat unit pclRepeat (M0 = 114.14):

(A) Tg, the GLASS-TRANSITION temperature — Yang 2020 (run below). Repeat unit -[O-(CH2)5-CO]- decomposes into Yang main-chain groups: O x1 : MW 16 Yg -14.718 CH2 x5 : MW 14 Yg 4.026 (each) CO x1 : MW 28 Yg 4.370

M0 = 16 + 5*14 + 28 = 114 g/mol (matches the reactor M0!) Yg(inf) = -14.718 + 5*4.026 + 4.370 = 9.782 (1e3 g.K/mol) Tg(inf) = 9.782e3 / 114 ~ 86 K (the additive estimate) (Measured PCL Tg ~ 213 K / -60 C; an additive main-chain scheme is weak for a flexible aliphatic polyester — the model tells the truth about itself, which IS the lesson. See the console's per-cent error vs the anchor.)

(B) DENSITY — Van Krevelen (Bondi Vw). NOT runnable on PCL with the shipped data: the Slice-1 Van Krevelen set ships only hydrocarbon + chloro groups (CH3 CH2 CH C ACH AC CHCl CH2Cl); it has NO ester -O- / -CO- group, so the repeat unit -[O-(CH2)5-CO]- cannot be decomposed. Running it would HARD- ERROR with "unknown polymer group 'O'" — the estimator refuses to invent a value (no silent zero). The honest path: density needs the ester groups curated into data/standards/-parameters/vanKrevelen.dat first. The worked Van Krevelen DENSITY example lives in tutorials/prop-s/estimate/vanKrevelen01__polystyrene__density (a pure- hydrocarbon repeat unit the Slice-1 set CAN do).

This is the same M0 = 114.14 the reactor's polymer{} block used for Mn = M0/(1-p) — one repeat-unit mass, two places (process AND property).

A choupoProps sub-case of [../](..) (the polycaprolactone plant). The plant gives you the chain statistics from the kinetics (Mn / Mw / PDI); here you estimate the polymer's physical property — the glass transition Tg — from the same repeat unit -[O-(CH2)5-CO]- (M0 = 114.14, the exact number the reactor used for Mn = M0/(1-p)).

∗ Tg via Yang 2020 (CC-BY) — runnable. Tg(inf) = 85.8 K; the console prints the -59.7 % deviation from the measured ~213 K (an honest limit of an additive scheme on a flexible aliphatic polyester). M0 = 114 is goldened (an exact mass-balance fact); Tg is shown but not goldened as validated physics. ∗ Density via Van Krevelen — ∗not∗ runnable on PCL with the shipped data (the Slice-1 group set has no ester -O-/-CO- group, so it would hard-error rather than invent a value). See the full explanation and the runnable hydrocarbon example linked from the parent ../README.md.

The full teaching narrative lives in the parent README. See system/propsDict for the exact group decomposition and the estimator invocation.

What to expect (golden-locked):

kind	unit/stream	key	expected
diag	pclRepeat_Tg_yang2020	M0_g_per_mol	114
diag	pclRepeat_Tg_yang2020	Tg_K	85.81
diag	pclRepeat_Tg_yang2020	YgSum_1e3gKmol	9.782

sugarPlantEconomicsSweep

tutorials/plant/sugarPlantEconomicsSweep

What it does: Sugar plant ECONOMICS SWEEP – IRR / payback / NPV vs RawJuice sucrose content (Phase-2 SweepDriver post-chain differentiator)

The story: / system/flowsheetDict (PLANT level – COMPOSITE)

The flagship sugar plant (the ChemicalPlantTutorial flowsheet, reused verbatim – sectors and units included), here driven by system/outerDict: a SweepDriver walks the RawJuice sucrose fraction and runs the FULL post chain (sizing → costing → economics, system/postDict) on every point, so IRR / NPV / payback vs feed quality come out as sweep responses.

A composite node: it has ‘children‘ (sub-flowsheets, here the sectors) and ‘connections‘ (the pipework between them). The same shape a sector uses to list its units – one level up. This is the fractal.

Naming convention (see docs/ai/case-layout.md): SECTORS in CAPS : CONCENTRATION, DRYING, FERMENTATION Units in PascalCase : Cryst, Evap1, Evap2 Streams in PascalCase : RawJuice, Magma, Powder, EvapVapour Components in lowercase : water, sucrose, N2

This is the flagship sugar plant (tutorials/plant/ChemicalPlantTutorial) driven by a SweepDriver instead of a single pass. It exists to show the one thing a single CAPEX number cannot: how the project economics move when a process variable changes.

The driver runs the FULL post-processing chain (sizing → costing → economics, in system/postDict) on every converged point, so economics.IRR, economics.paybackYears, economics.NPV, ? resolve as ordinary KPI look-ups (the Phase-2 SweepDriver post-chain wiring). The result lands in sweep_results.csv.

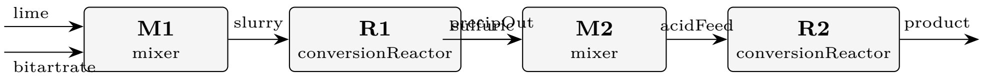
Sweeping the RawJuice sucrose content 0.09 → 0.16 mol fraction walks the plant from sub-economic, through break-even, to clearly profitable:

Richer juice → more crystalline sugar per unit feed → more revenue against an almost-unchanged plant cost → the IRR climbs and the payback appears and shortens. The NPV crosses zero between 0.104 and 0.118.

Both use the same constant/economics prices (EU Sugar Market Observatory white sugar, ECB EUR/USD, ∗Chemical Engineering∗ CEPCI, US statutory tax) and the same system/postDict cost chain — only the run mode differs.

`tartaricAcid`

`tutorials/plant/tartaricAcid`



What it does: Tartaric-acid plant (wine route): lime precipitation of calcium tartrate, then sulfuric acidulation to tartaric acid + gypsum. Molecular reactors; the solution chemistry sub-study is tutorials/props/electrolyte/-tartaricAcid_acidulation. HONEST LIMITS: H2SO4 declared nonvolatile by case overlay (Psat ~1e-5 Pa at process T); precipitates ride as nonvolatile solutes with NO rigorous solid-liquid separation; reactor duties honestly UNAVAILABLE (species without ideal-gas Cp – a named gap, never a fake zero).

What to expect (golden-locked):

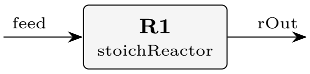
kind	unit/stream	key	expected
`stream`	`acidFeed`	`F`	`0.943266666667`
`stream`	`bitartrate`	`F`	`0.583333333333`
`stream`	`lime`	`F`	`0.166666666667`
`stream`	`precipOut`	`F`	`0.7766`
`stream`	`product`	`F`	`0.943266666667`
`stream`	`slurry`	`F`	`0.75`
`stream`	`sulfuric`	`F`	`0.166666666667`
`kpi`	`M1`	`F_out`	`0.75`
`kpi`	`M1`	`P_out`	`100000`
`kpi`	`M1`	`T_out`	`333`
`kpi`	`M1`	`n_in`	`2`
`kpi`	`M1`	`vf`	`0`
`kpi`	`M2`	`F_out`	`0.943266666667`
`kpi`	`M2`	`P_out`	`100000`
. . . and 20 more reference values in the case's `expected`.			

twoSectorDemo

The theory you need. The zippable two-sector demonstration plant – the case used to rehearse the online landing. Same physics as the esterification demo.

SECTOR_R

tutorials/plant/twoSectorDemo/SECTOR_R

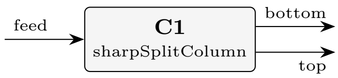


What it does: SECTOR_R – the reactor sector. Runs its OWN compiled unit op (stoichReactor) on the plant-level thermo (cascaded from ../constant).

The story: one custom reactor (compiled from code/). Build + run with: bin/buildCode tutorials/plant/twoSectorDemo/SECTOR_R runCase tutorials/plant/twoSectorDemo/SECTOR_R —

SECTOR_S

tutorials/plant/twoSectorDemo/SECTOR_S



What it does: SECTOR_S – the separation sector. Runs its OWN compiled unit op (sharpSplitColumn) on the plant-level thermo (cascaded from ../constant).

The story: one custom distillation (compiled from code/). Build + run: bin/buildCode tutorials/plant/twoSectorDemo/SECTOR_S runCase tutorials/plant/twoSectorDemo/SECTOR_S —